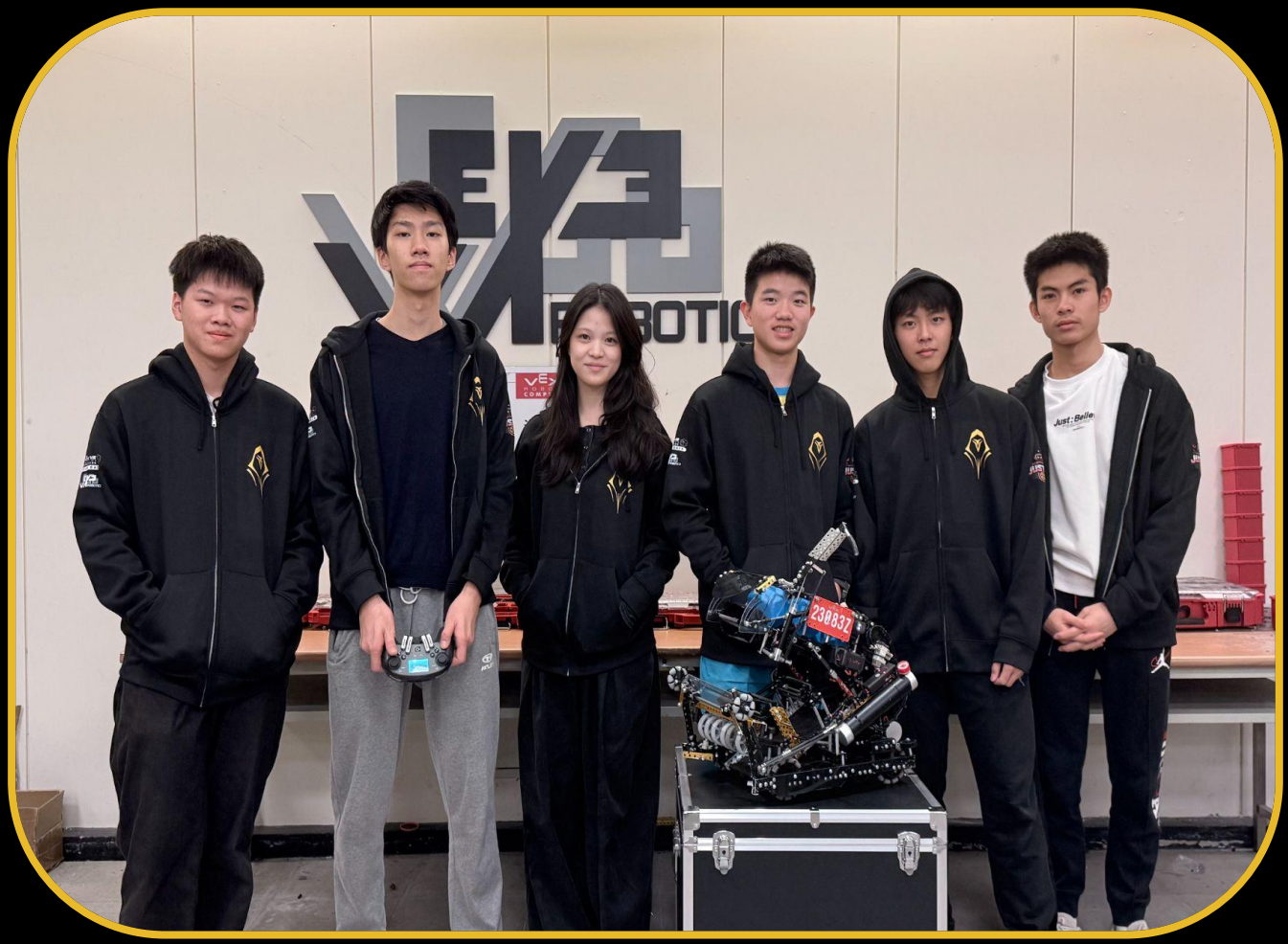
230832

REAPER

Taipei, Taiwan
2025-2026 Push Back



Start Date: **2025/5/29**

Notebook Introduction

Preface

Hello, we are Wego Robotics.

This season marks the third year of our VEX Robotics program since our club was founded. In addition to documenting our progress for reference and for judges' review, this year's engineering notebook also serves a second purpose — to act as a guide for future members of Wego Robotics.

We aim for this notebook to be more detailed and educational than before, explaining not only what we built and programmed, but also how our team operates, organizes, and learns. We hope that future members can use this record to understand our methods, improve upon them, and continue building a stronger club in the years to come.

How to Read This Notebook

This notebook is organized into nine major parts, following the same order as our real development process — from the people behind the robot, to the robot itself, and finally, to the lessons we want to pass on.

Each part within itself is organized chronologically, but can also stand alone as a complete record of that topic.

Table of Contents

1 TEAM 7

- Team members
- Team organization

2 GAME ANALYSIS 16

- Game rules and Scoring
- Scoring Observations
- Skills
- Updates

3 BRAINSTORMING 43

- Basic Structure Brainstorm
  - Chassis
  - Intake
  - Shooter
  - Aligner
  - Tongue
  - Descorer

4 ROBOT 1 91

- Research
- Robot Type
- Personalize
- Building
  - Chassis
  - Intake
  - Aligner
  - Tongue
  - Color Sorting
- Programming
  - Simulator
- TAI VEX Practice

5 ROBOT 2 227

- Reflection
- CAD
- Intake Design
- Building
  - Sprocket
  - Roof
  - Mat Support
  - Descorer
- Programming
- TAI VEX HS 26

6 ROBOT 2.1 293

- Reflection
- Shooter Design
- Intaker Design

Table of Contents

- Building
  - Tongue
  - Descorer
  - Aligner
  - Wing
- Programming
  - Simulator
- Skills Strategies Ver. 2
- PAS Signature

7 Robot 2.2 393

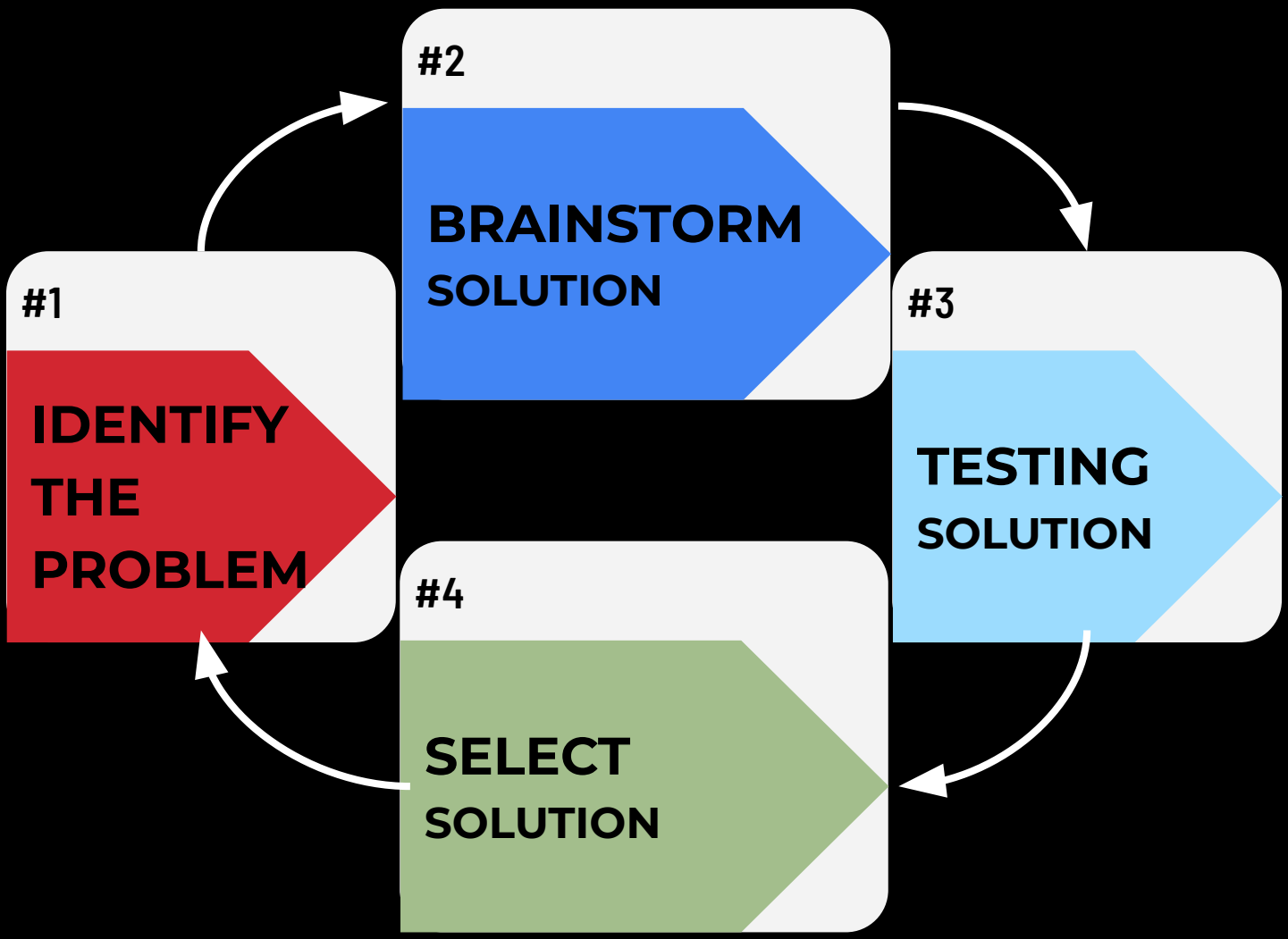
- Wing
- Game Strategies
- Skills Strategies Ver. 3

8 Team Management 415

- Club Website
- Club Budget
- Practice
- Team Impact
- Club Organization
- VEX Materials
- 3D Printing
- CNC
- Acrylic Bending Machine

Color Coding

TEAM



PROGRAMMING

DESIGN

COMPETITION ANALYSIS

TEAM & PROJECT MANAGEMENT

Engineering Design Process

#1
IDENTIFY THE PROBLEM

- Game Analysis/Rules
- Identify Problem througn Testing

#2
BRAINSTORM SOLUTION

- Brainstorm possible solution
- Research
- List Pros and Cons

#3
TESTING SOLUTION

- Experiment
- Graphs and Tables

#4
SELECT SOLUTION

- CAD, Prototype
- Analysis and Decisions

01
TEAM

Team Members

Pennlaine

- Grade 11
- Club President & Team Leader
- 3rd year in VEX
- *“Somewhere between calculation and chaos”*

Team Members

Adam

- Grade 11
- 2-year VEX experience
- Programmer/main driver
- Adaptive, fast learner

Team Members

Anthony

- Grade 11
- Complete newbie in Vex standards
- Builder/CAD/Art Lead
- *“There is no prize to perfection, only an end to pursuit.”*

Team Members

Benjamin

- Grade 11
- First year in VEX
- Builder/Planning/Programming
- "*Join in the middle of the season.....More*"

Team Members

Jefferson

- Grade 11
- First year in VEX
- Build, repair, improve

Team Members

Cecilia

- Grade 11
- First year in VEX
- Scout/PR/Notebook taker
- *“Mascot? Peacemaker? Pistachio!”*

Team Organization & Info

Rules and Goals for this year:

1. Always **record the process/take pictures** when finished one part, then put it in group chat and notebook.
2. Always ask questions **on the whole team's group chat** and discuss for the optimal solution
3. Always ask before changing what we planned to do by providing reasons and ask for the whole team's approval
   a. If teammates have different opinions, we vote.
   b. If it meets a tie, the team leader shall decide, but the whole team takes the responsibility; no one is to blame.
4. GET QUALIFIED INTO WORLDS!!!

02
GAME ANALYSIS

Meeting Logs

2025/06/09 — 45 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	Discuss - Teams - Game Manual

Teams:

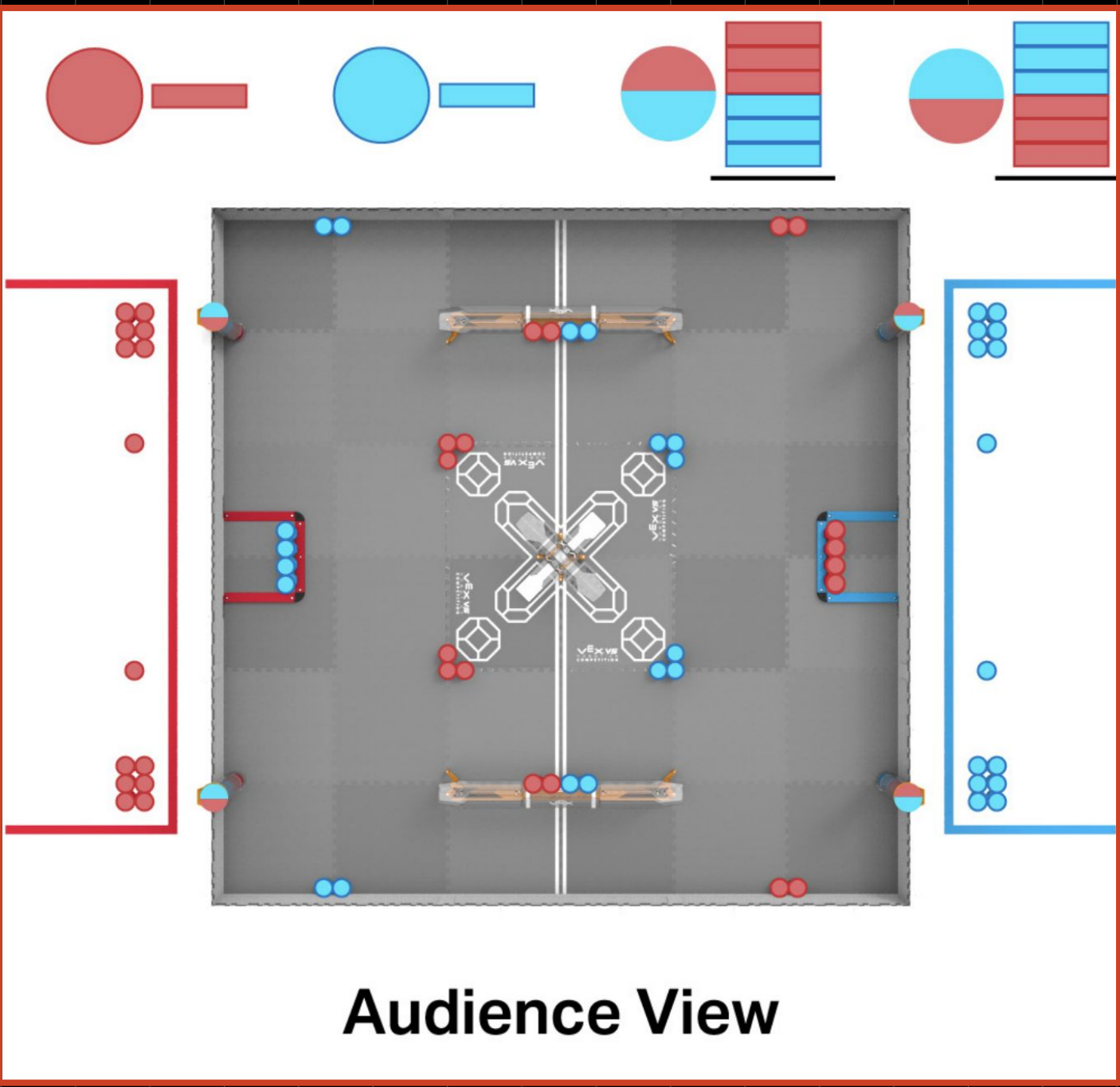
- 3 teams (1 junior high, 2 high school)
- Each team with 4-6 teammates
- We decided the both teams' leader by voting
- We also discussed and decided everyone's job and position in the team
- Stated general rules for the team (See Page 2)

Game Manual:

- We watched the animation of the 2025-26 VEX High Stakes together
- Everyone was told to finish reading the game manual by next Thursday

Game rules & Scoring

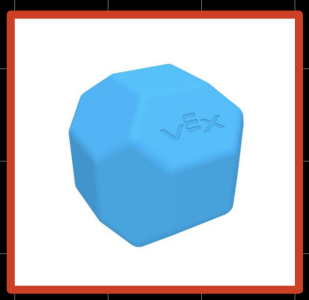
Field View

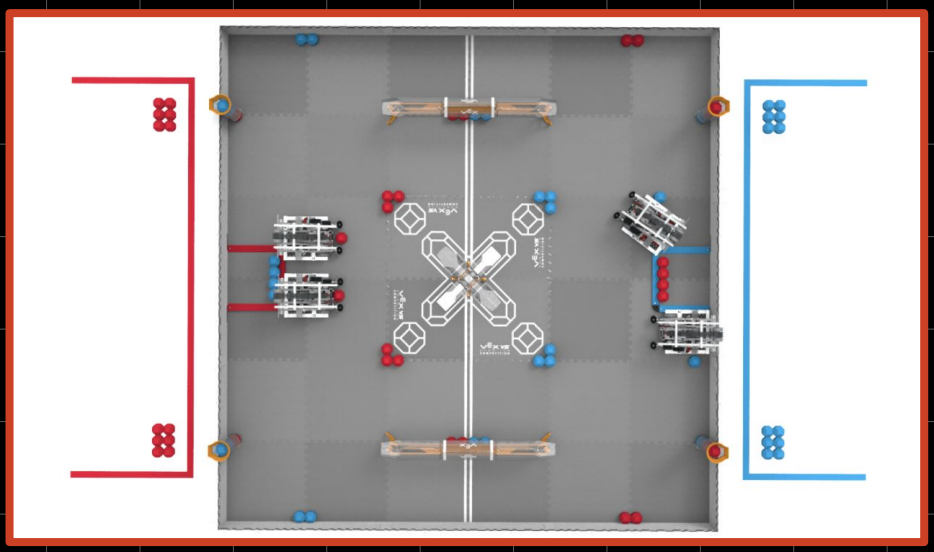


Audience View

Field Setup and Object

- 88 Blocks in total, 44 Blocks for each color, both containing:
  - 2 Preloads
  - 12 Match Loads
  - 18 that start the Match in predetermined locations on the Field
  - 12 that start the Match in Loaders
- 4 Loaders
- 4 Goals
  - 2 Long Goals (15 utmost each)
  - 2 Center Goals, one Upper and one Lower (7 utmost each)
- 2 Park Zones





Game Overview

Game Phases:

- Autonomous Period (15 seconds): Robots operate solely on pre-programmed instructions.
- Driver-Controlled Period (1 minute 45 seconds): Teams control their robots manually to score points.

Autonomous Bonus:

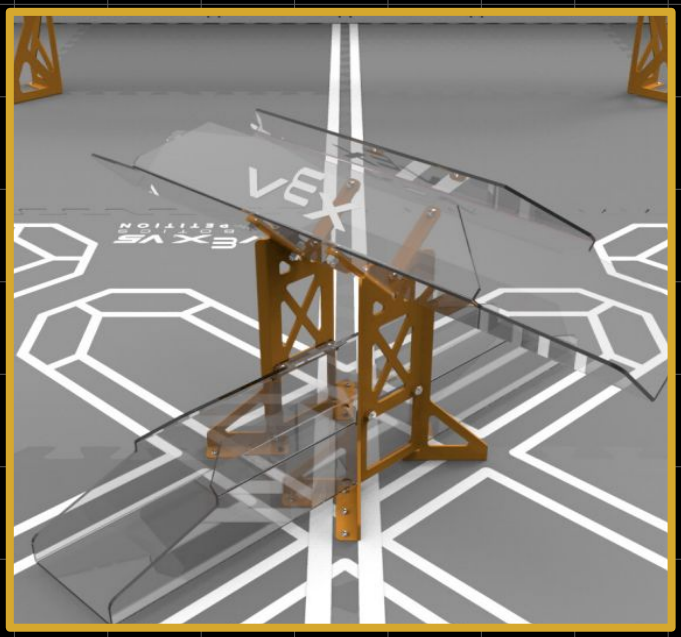
- Awarded to the alliance with the most points at the end of the Autonomous Period (+10 points).

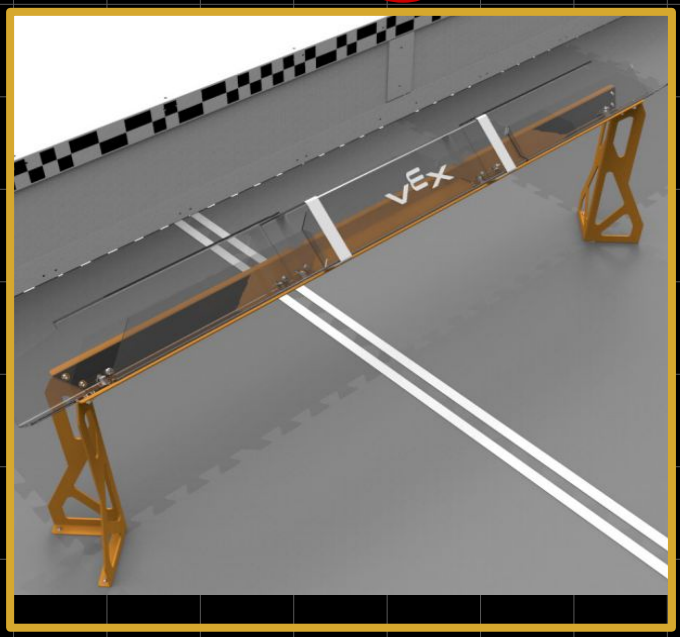
Autonomous Win Point (AWP):

- Score at least **7 blocks** of our color.
- Score blocks into at least **3 goals** out of 4 on the field.
- At least **3 blocks** of our color removed from **Loaders** adjacent to the Alliance's Alliance Station (upmost 6 blocks available)
- Neither Robot is contacting the Park Zone barrier.

→ THESE WILL BE MODIFIED FOR EVENTS QUALIFYING TO WORLDS AFTER 2025/9/4,. We will put it in UPDATES

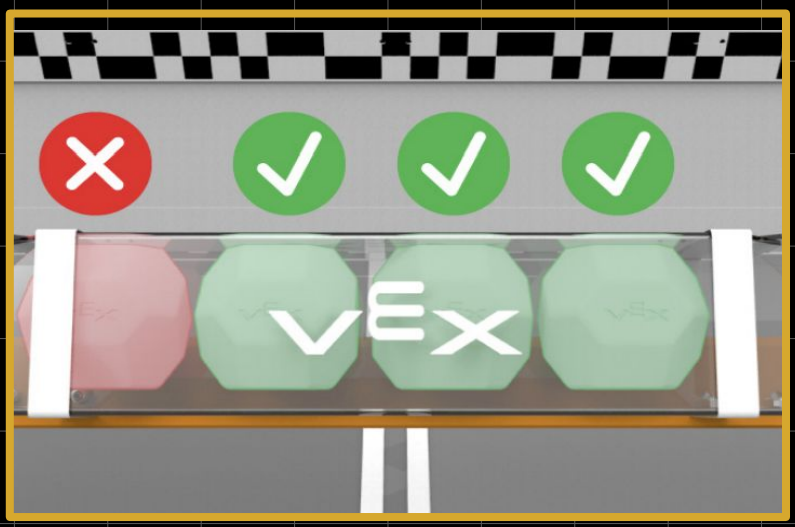
Game Scoring

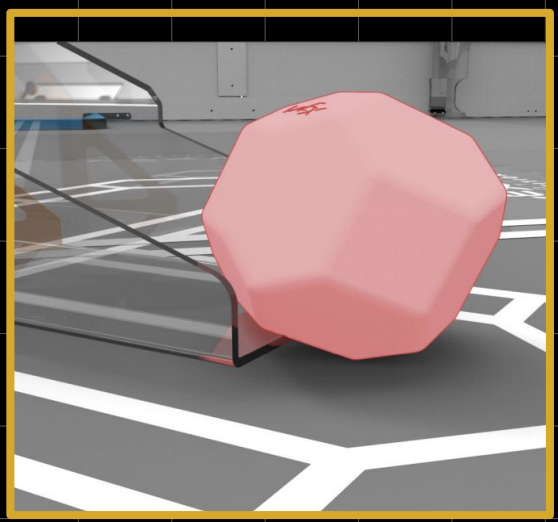




Scoring:

- Blocks Scored: Each block scored into a goal is worth 3 points.
- An Alliance Controls a Controlled Zone:
  - Scored in Long Goal: 10 points
  - Scored in Upper Center Goal: 8 points
  - Scored in Lower Center Goal: 6 points

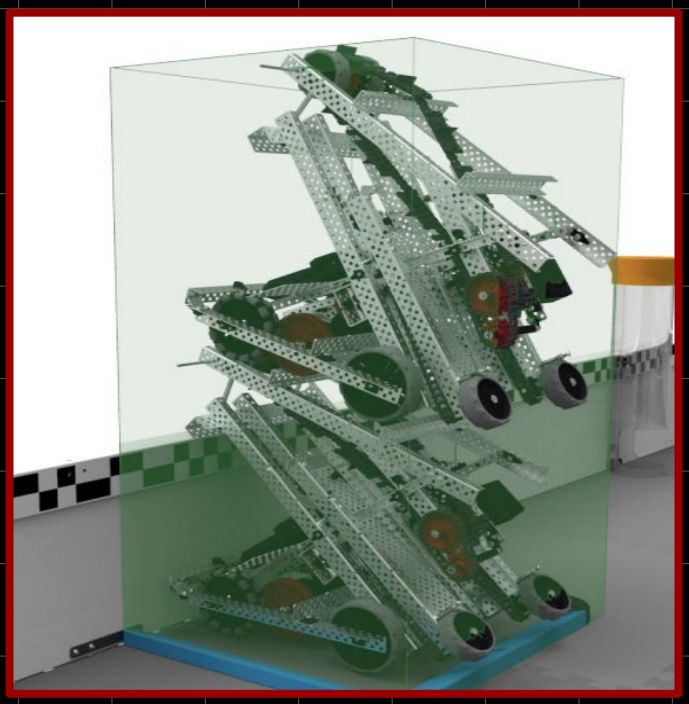




Game Scoring

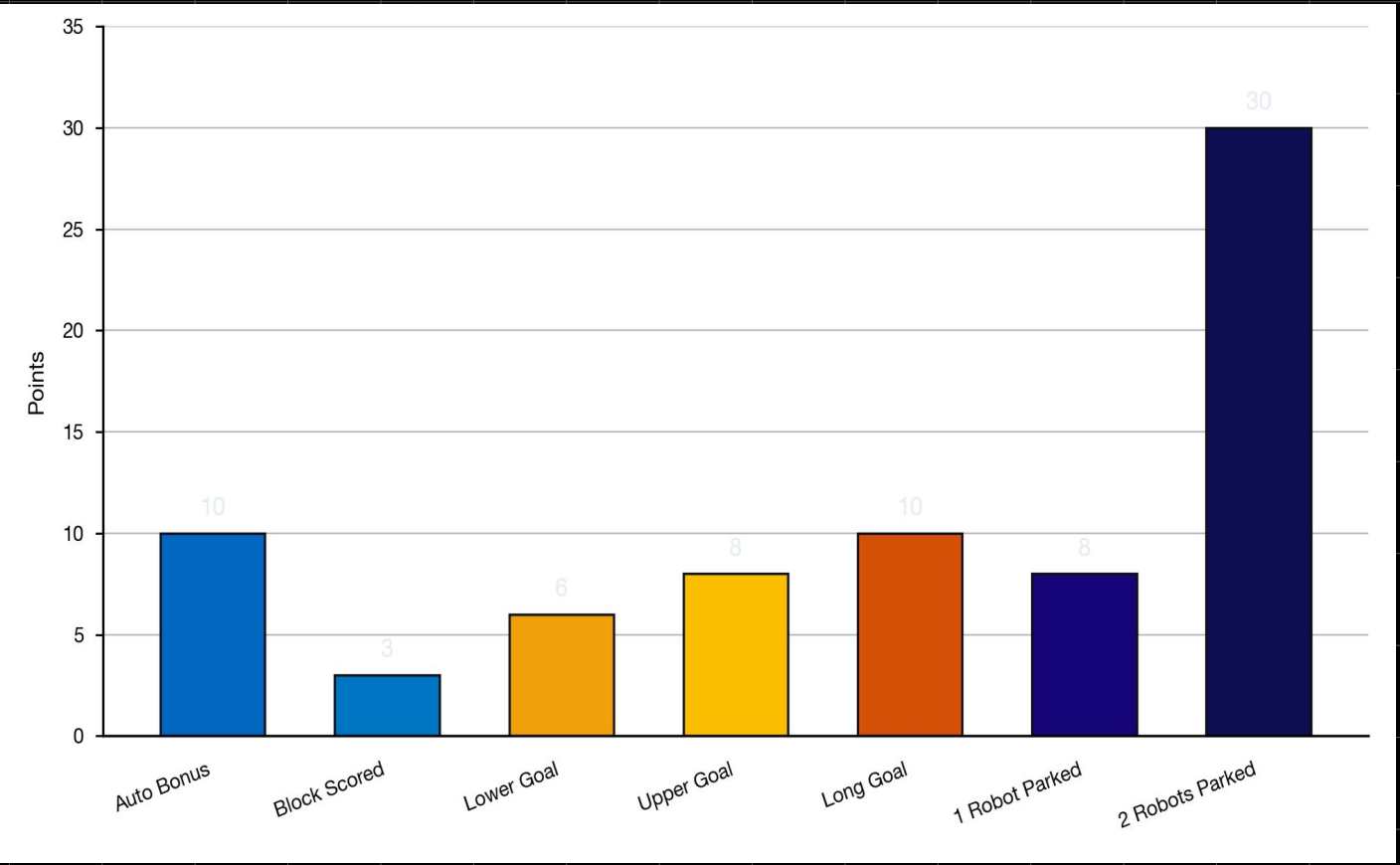
Scoring:

- Robots earn points by parking into the parking zone:
  - 1 robot: 8 points
  - Both robots: 30 points
  - A parked robot will only be touching the inside floor of the parking zone and/or the parking zone objects and/or another parked robot.



Game Scoring Analysis

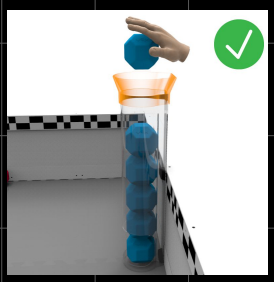
We would like to develop a strategy that we can implement to guarantee the most points received, as we all know, it is too idealistic to get all the scores.



1. Double parking is the highest single action
2. Gaining the control bonus is essential for higher scoring, as both the control bonus and scoring blocks give points.
3. The middle goals can be considered a game changer, as a robot only needs to score in one ball to control.

Game Specific Rules

1. Contacting the Alliance's **Park Zone** when the match starts
2. Size limit
   a. 18" x 18" x 18" **s**tarting size
   b. extension is restricted to 22" x 22" x 22"
3. 1 preload
4. Can't cross **Autonomous Line** during Autonomous period
5. Drive Team Members may introduce Match Load Blocks by placing them into Loaders adjacent to their Alliance Station.



6. Robots cannot directly affect blocks in enclosed sections of Goals (Long Goals)
7. Cannot contact the other Alliance's parking zone in the last 20 seconds
8. Limited to 12 individual pieces of plastic not larger than 4" x 8" x 0.070"
9. 3D printed parts for any purpose are prohibited

Game Overview

Each robot begins a match contacting their parking zone and can't cross the Autonomous line. Robots can at most score fully 44 blocks into four goals and have both the alliance's robots parked. This would result in the highest possible score if this team manages to win the autonomous bonus also. Of course, this is nearly impossible.

Problem Statement:

- There are 4 goals. The Drive Team Members shall collaborate seamlessly with the robot/the driver so it can intake the blocks as efficient as possible. Robots shall also defense or keep inputting blocks so that their scored blocks maintain at the center.

Game Strategies

1. Parking is important (+30 to different teams would make a 60 point difference)
2. Will need a lot of cooperation within Drive Team Members for both introducing Match Loads and keeping an eye on whether the blocks are removed or changed positions from the control zones by the opposing alliance
3. Collaboration with our alliance is crucial so that we can achieve a higher score

Design Statement:

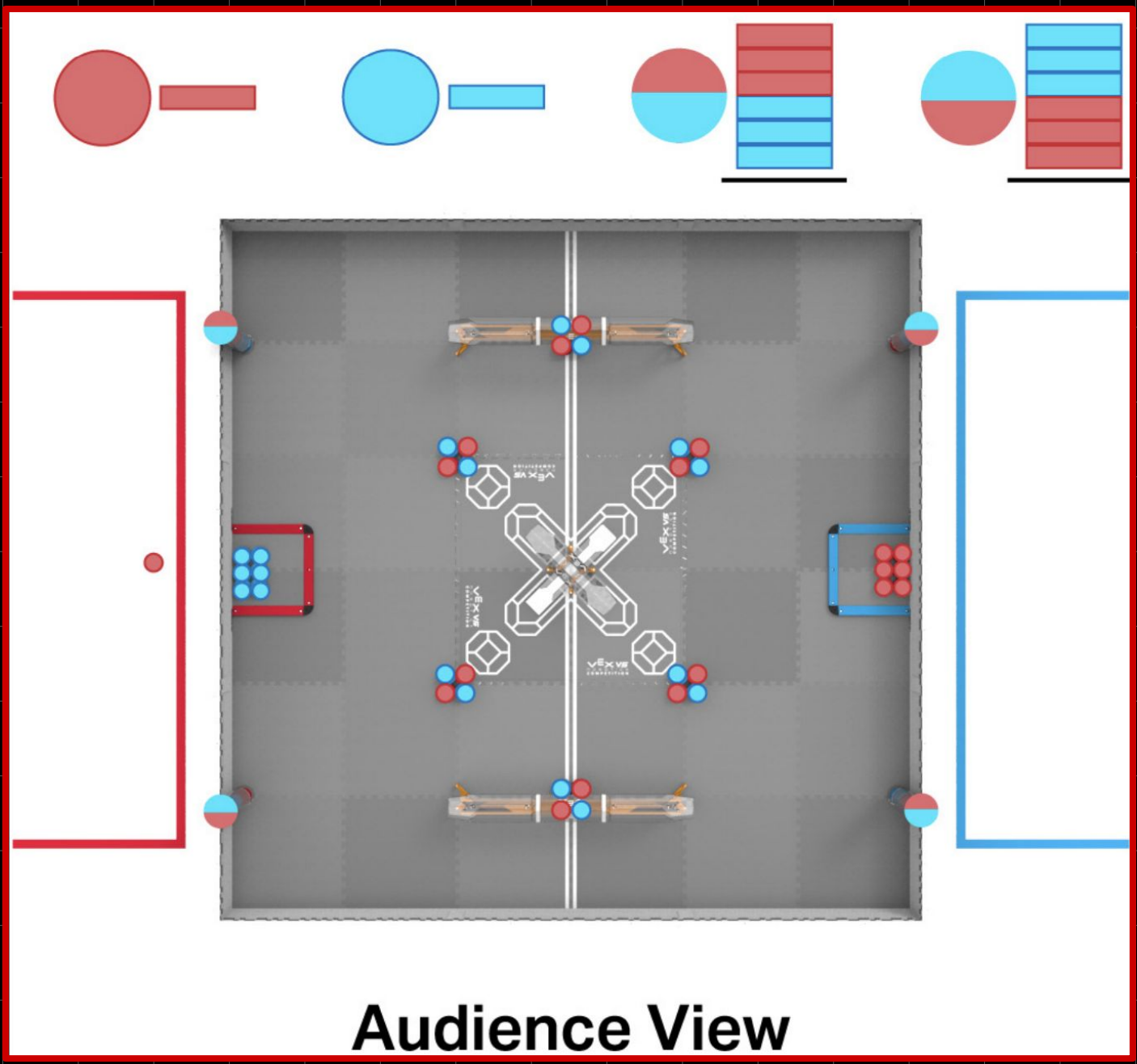
Design, build, and compete with a robot that can effectively score into different heights of goals, and park (make space for our teammate to park as well).

Game Contraints

1. Robot must be 18" x 18" x 18"
2. Expansion limited to 22" x 22" x 22"
3. VEX materials only
4. Custom plastic material limited to 12 individual pieces of plastic not larger than 4" x 8" x 0.070"
5. At most 88W motor power in total
6. First competition in late September

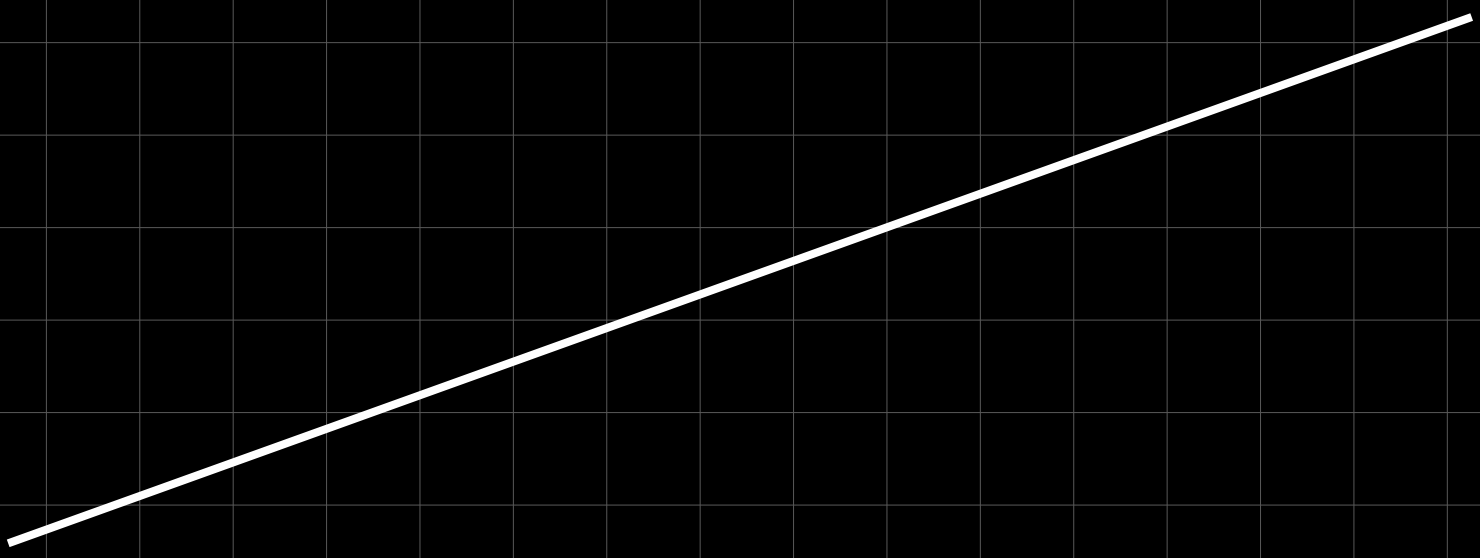
Skills Field

The Skills field and rules is different from regular matches. However, rules not mentioned explicitly follow



Skills Scoring

Points	Achievements	Total Points
1	Scored one block	44
5	Fill a Long Goal Control Zone	10
10	Fill a Central Goal Control Zone	20
5	Clear a Park Zone	10
5	Clear a Loader	20
15	Park robot	15
		119



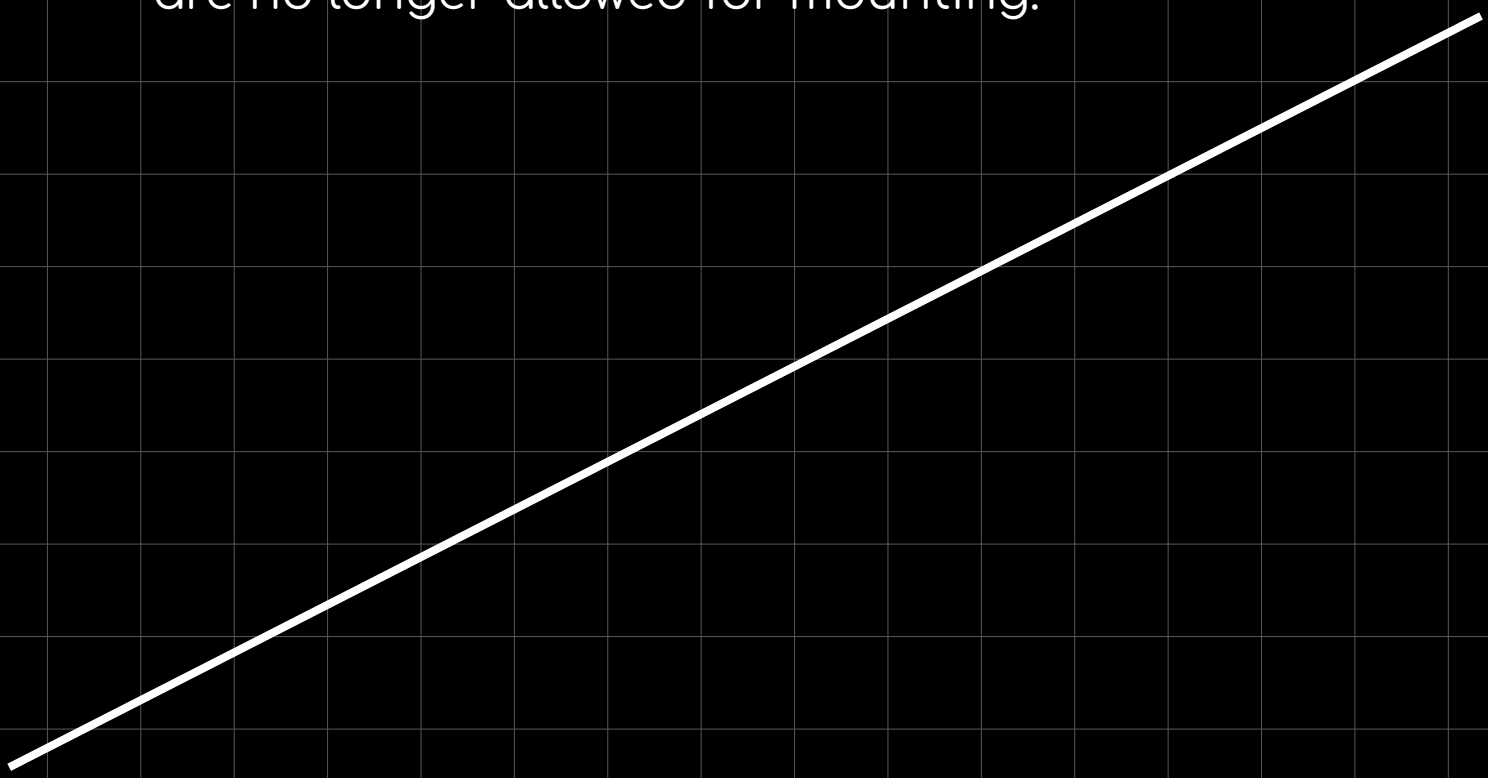
Game Updates

Version 0.2 (June 5, 2025)

- Definitions & Clarifications
  - Lifting/Trapping: Preventing a Robot already off the floor from returning may count as Lifting/Trapping.
  - Control Zone: Boundaries clarified; a Long Goal Control Zone holds up to 3 Blocks between (not including) the white tape lines.
  - Scored Blocks: Must be in contact with the inside surface of the plastic trough.
- Autonomous & Game Play
  - Teams committing any Violation during Autonomous make their alliance ineligible for Autonomous Win Point; bonus goes to opposing alliance.
  - Blocks that engage over the Autonomous Line (including objects/robots) are considered in contact with it.
  - Drive Team may add Match Load Blocks to Loaders during Autonomous and Driver Controlled Periods.

Game Updates

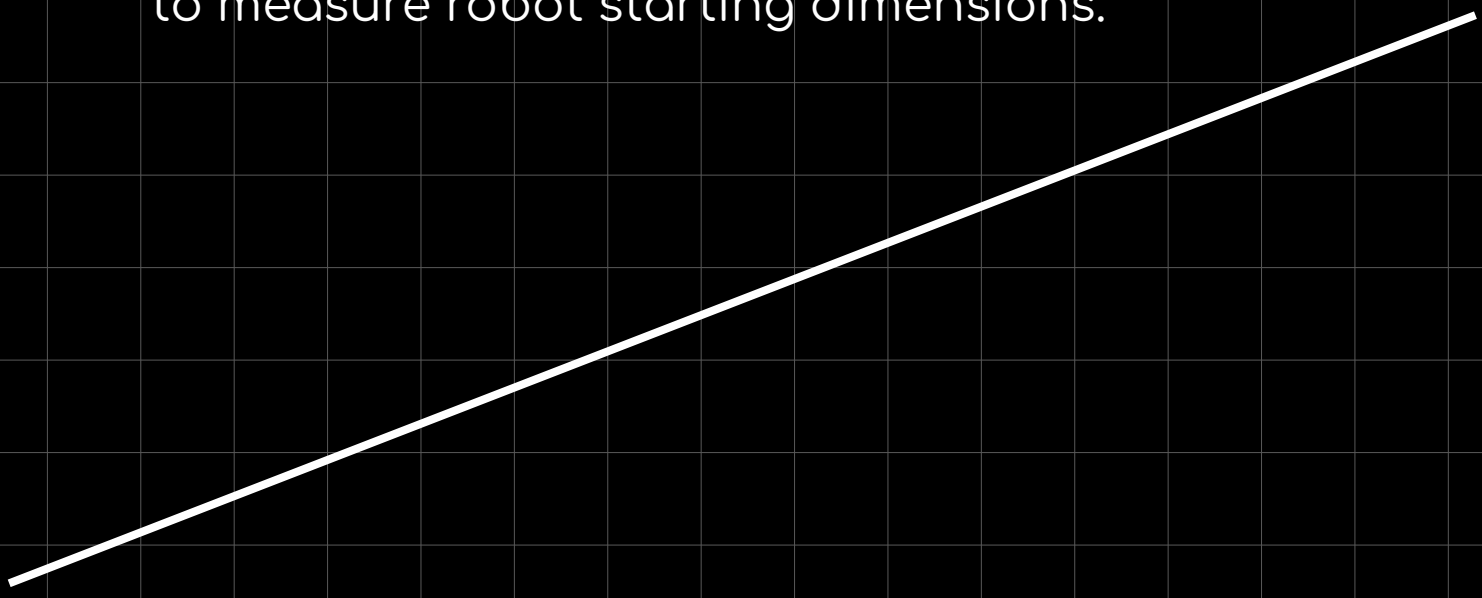
- Robot Size & Expansion
  - Robots must fit within a 22" × 22" × 22" box; vertical/horizontal expansion limits clarified.
- Interactions with Field & Blocks
  - Robots cannot contact Blocks fully inside enclosed Long Goal sections, but open sections are allowed.
  - Blocks bouncing out of Loaders during loading are not violations.
- Robot Rules & Hardware
  - License plates must remain stationary; VEX IQ pins are no longer allowed for mounting.



Game Updates

Version 1.0 (June 26, 2025)

- Autonomous Line & Blocks
  - <SG7eii> updated: all Blocks on the other side of the Autonomous Line are included.
  - <RSC2a> updated: Blocks contacting a Robot at the end of a Robot Skills Match do not count as Scored.
  - <RSC3> updated: Robot Skills Matches do not include Match Load Blocks, and blocks leaving the field are not returned.
- Robot Size
  - <R5> revised: Event Partners can use any sizing tool to measure robot starting dimensions.



Game Updates

Version 1.1 (August 7, 2025)

- Major Violations / Block Removal
  - <SG4> updated: Strategic or intentional removal of 3 or more Blocks from the Field in a single Match now results in a Major Violation.
- Autonomous & Incidental Violations
  - <SG7e> & <SG8b> updated: Incidental Violations of <SG7> will not be penalized and do not automatically give Autonomous Bonus to the opponent.
- Loader Rules & Drive Team Actions
  - <SG9> updated: Drive Team Members may use both hands when introducing Blocks into a Loader.
  - <SG9d> clarified: Blocks bouncing out of the bottom opening of a Loader after proper introduction are not violations.
  - <SG9> new bullet: Blocks can only be added to Loaders containing 5 or fewer Blocks.

Game Updates

Version 2.0 (September 4, 2025)

- Field & Zone Definitions
  - Loader: Definition clarified.
  - Park Zone: Includes floor tiles bordered by field perimeter and plastic extrusions; dimensions 18.87" × 16.86". Robots may be parked here at the end of a Match.
  - SC2a: Clarified inside surfaces for scoring. Figure SC2-2 updated.
- Autonomous Win Point (Worlds-qualifying events)
  - Alliance must complete all of the following:
  - ≥10 Blocks of alliance color scored.
  - ≥3 Goals with at least 2 scored Blocks of alliance color
  - ≥3 Blocks removed from Loaders adjacent to Alliance Station.
  - Neither Robot is contacting the Park Zone barrier.

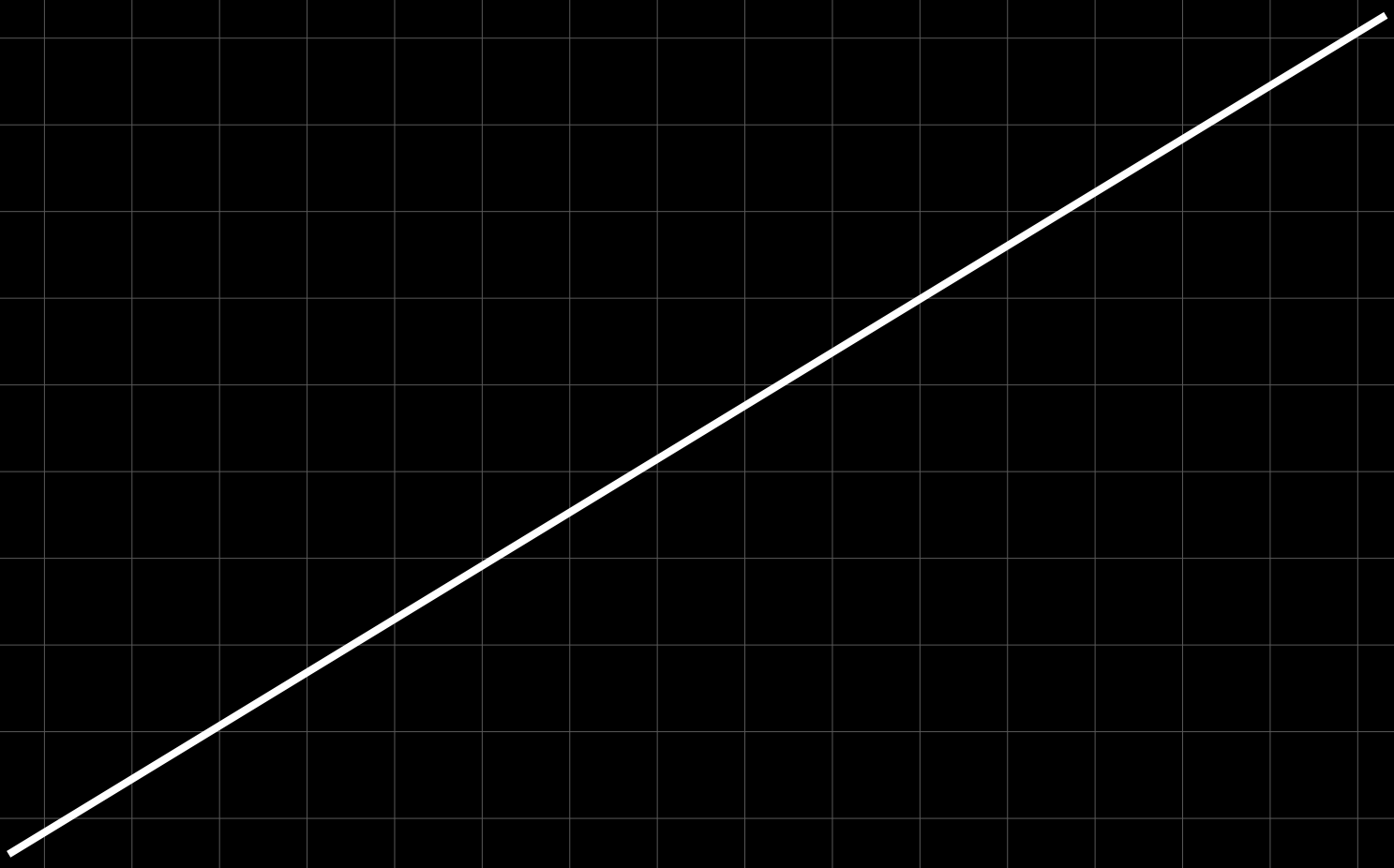
Game Updates

- SG / Loader Updates
  - <SG4>: “Intentionally/strategically” removed to simplify refereeing.
  - <SG9>: Loader rules clarified:
  - Only add Blocks if no blocks partially/fully in orange portion.
  - Illegal Match Loads considered 2 points for Match Affecting calculations.
  - Escalation to Major Violation for repeated Minor Violations (6+ illegal Match Loads in a single Qualification Match, or 3+ Qualification Matches with ≥1 illegal Match Load).
  - <SG11>: Protect Robots partially in their own Park Zone; no contact with opposing Park Zone or opponent Robots in last 20 seconds.

Game Updates

Version 2.1 (October 9, 2025)

- Scoring & Match Result Determination
  - <SC1> — Match scores must NOT be changed due to Violations during the Match.
- Conduct, Behavior, & Event Protocol
  - <GG17> — Holding
    - A Holding count begins immediately
    - Once the Head Referee observes a suspected Holding interaction



Game Updates

- Gameplay & Scoring Object Handling
  - <SG4a>
    - Added to clarify handling of blocks that should have exited the Field but were deflected back into play
  - <SG9> — Match Loads
    - Match Loads must be introduced one at a time
    - Added a Violation note reinforcing this requirement.
  - <SG10> — Reaching vs Anchoring
    - Added a new paragraph clarifying:
    - Robots may reach into open portions of a Goal only while actively moving Blocks
    - If a Robot:
      - Reaches into a Goal
      - Remains there without actively moving Blocks
    - The Head Referee should verbally warn the Team and issue a Violation if the Robot remains

Game Updates

- Added Offensive, Defensive, and Goalkeeping as new terms.
  - Offensive: A Robot is Offensive while it is engaged in actions that could directly increase its Alliance's score for the current Match.
  - Defensive: A Robot is Defensive while it is engaged in actions that cannot increase its Alliance's score for the current Match, and instead limits an opponent's ability to score or play the game.
  - A robot is Goalkeeping while it is reaching into any open portion of a Goal while not attempting to change the score of the Match through legal means.

03
BRAINSTORMING & DECISIONS

Defining the Robot

CHASSIS

1. Structural requirements/limits
2. Drivetrain Type
3. Gear Ratio
4. Wheels
5. Motors/Watts

SUPER STRUCTURE

1. Intake System
2. Shooter System
3. Aligner
4. Tongue
5. Descorer

Meeting Logs

2024/07/08 — 65 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- Brainstorm Planning

Before starting to build our chassis, we drafted a to do list so we can follow along

To Do List:

1. Evaluate different drivetrain types
2. Evaluate chassis speed
3. Compare different motors
4. Experiment different types of wheels
5. Choose the best fit above structures
6. Start off doing CAD for the chassis.

Building will start at mid-late July.

Brainstorm Timeline

Tasks	7/8-7/14	7/15-7/21
Chassis		
Intake		
Shooter		
Descorer		
Aligner		
Tongue		

Tasks per member:

- **Chassis**: Adam, Pennlaine
- **Intake**: Jefferson, Anthony
- **Shooter**: Pennlaine, Adam
- **Descorer**: Pennlaine, Adam
- **Aligner**: Anthony
- **Tongue**: Jefferson

Chassis Requirements

According to the game manual, the chassis for this season should achieve these requirements:

Parking Requirements

- Center of Gravity: at either front or back, for the car's one side to leave the ground.
- Wheels: Can't use a lot of wheels or else it might block the parking process
- Speed: can't be too fast or else torque would be too small.
- Lateral Stiffness: the bar structure across should be place at a higher position

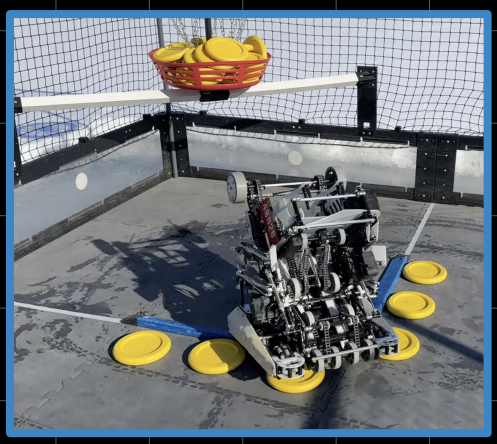
Chassis Type

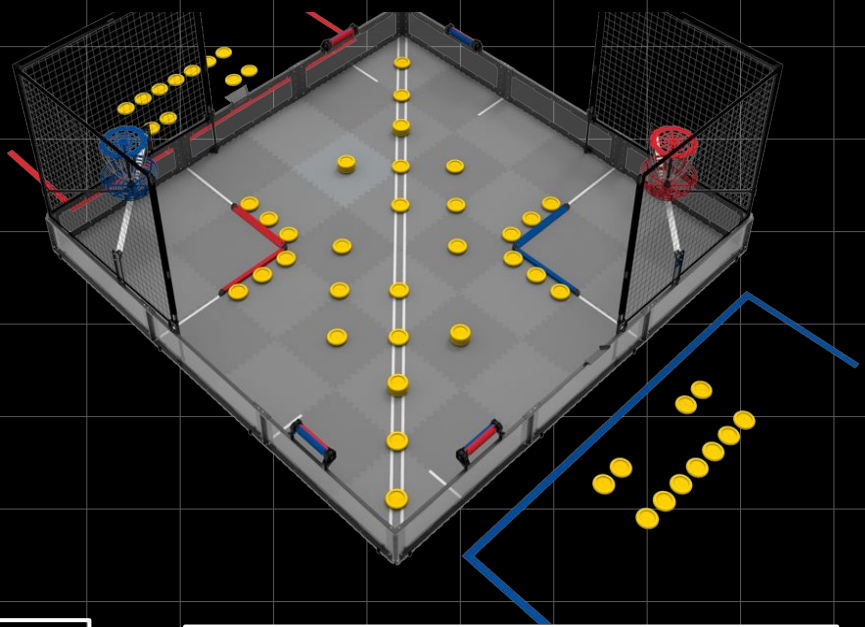
Parking Brainstorming

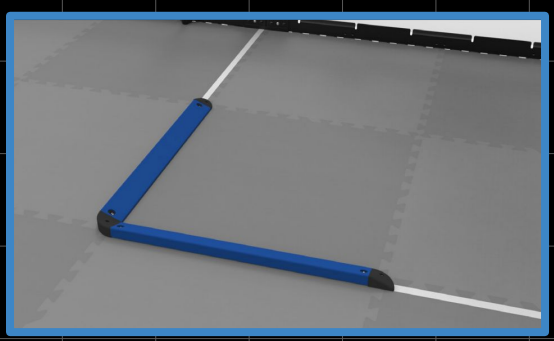
Parking Boards

This year's Parking Zone bars resembles 2022-23 Spin Up obstacle.

Teams attached customized plastic boards on the chassis to avoid robots' gears getting stuck with the bars. .



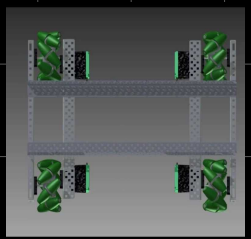




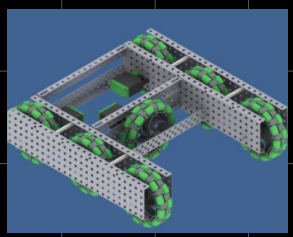
Chassis Type

Drivetrain Brainstorming Pros & Cons

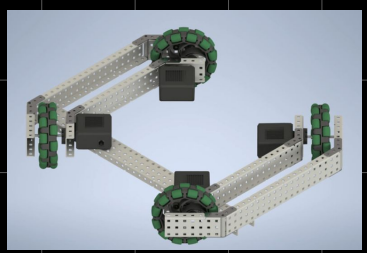
Drivetrain Type	Pros	Cons
Tank Drive	Simple, robust, easy to build and program, strong pushing power, good for straight-line movement.	Limited mobility; can only move forward and backward or turn; less maneuverable than other drives.
H Drive	Omnidirectional movement with minimal complexity, simple implementation of lateral movement, good for maintaining stability.	Less power and pushing force compared to tank drive, less precise than X drive, can be inefficient if not well implemented.
Mecanum Drive	Omnidirectional movement with high control and better traction than typical omnidirectional drives is good for smooth movement over obstacles.	Either too fast or too slow (600 rpm or 200 rpm) b/c one motor controls one wheel. Also, makes the chassis relatively big.
X Drive	Highly maneuverable, quick omnidirectional movement, good for agile robots that need precise, quick adjustments.	More complex mechanically, it can be harder to control and have less traction and power for pushing compared to tank drive.



Meccanum Drive



H Drive



X Drive

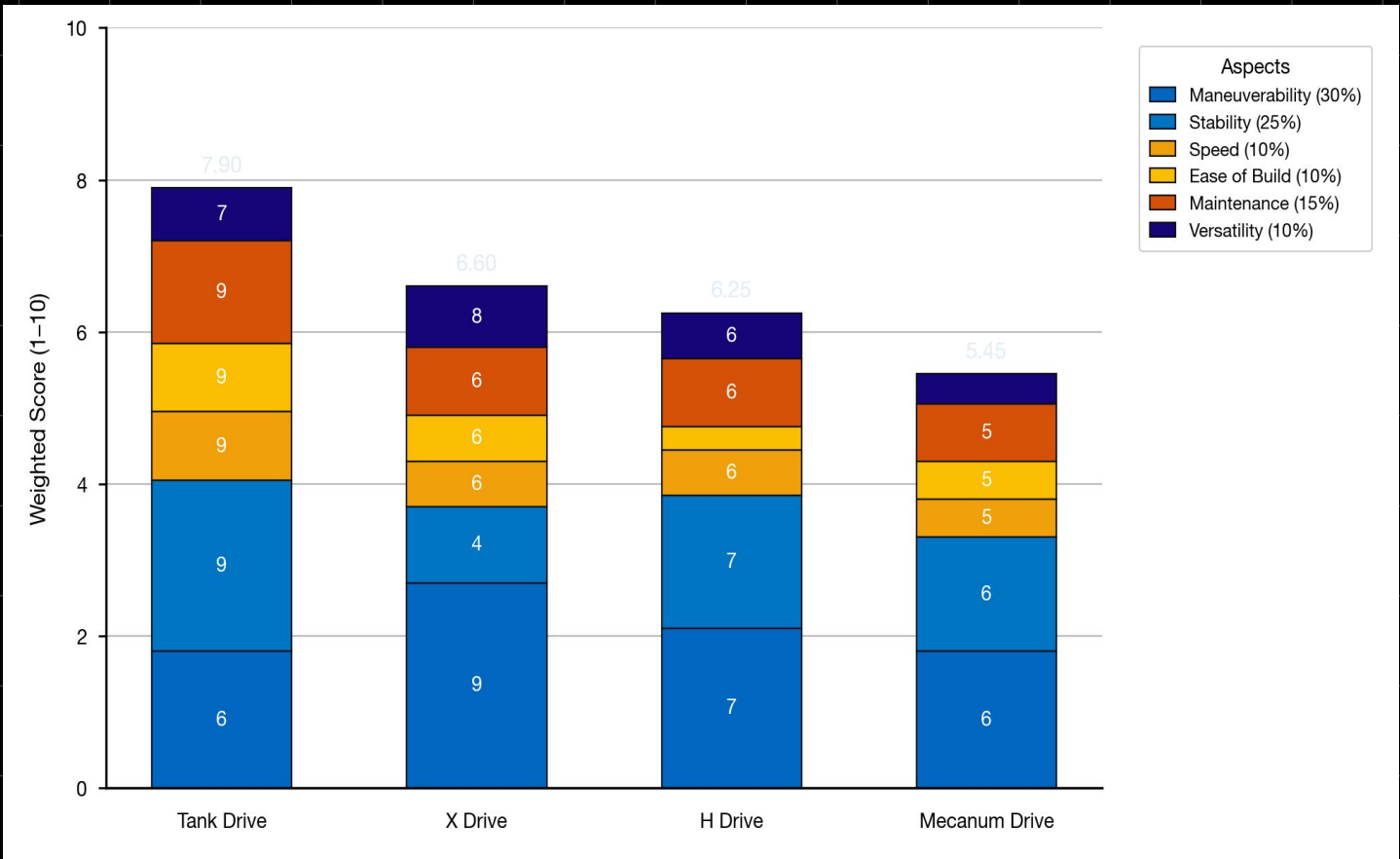
Chassis Type

We decided to evaluate chassis performances based on 6 metrics, with varied proportions.

1. **Maneuverability**: weights highest **(30%)** due to frequent alignment when scoring. This aspect reflects its agility, responsiveness, and omni-directional ability.
2. **Stability**: weights **25%** was prioritized to resist defense. This relates to how robust it is, its unlikeliness to break.
3. **Speed**: weights **10%**, it determines the robots ability to perform actions across the field fast enough.
4. **Ease of Build**: also weights **10%**, some chassis are inherently rather highly difficult to build, or taking up too much time, we might not be able to make maximize its performance.
5. **Maintenance**: rather important(**15%**), this reflects the difficulty of fixing or rebuilding the chassis, which would affect our schedules mainly during mid-season
6. **Versatility**: also weighting **10%**, this is affected by the easiness to work with different speed, torque, sizes of the drivetrain, so that we can change our chassis to different setups throughout the season.

Chassis Type

The resulting scoring for each chassis is shown:



From the scores we gave to each drive after evaluation, we decided to work on **tank drives** for our robot.

Chassis Type

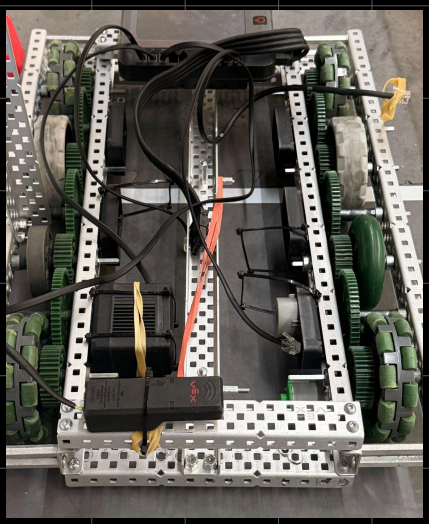
REASONS FOR REJECTION

We rejected the other 3 chassis because:

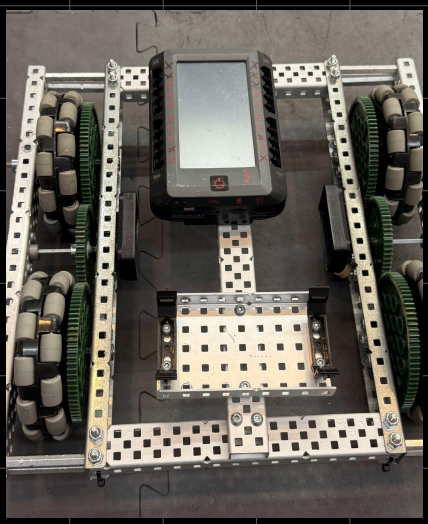
Type	Reason for Rejection
X Drive	Insufficient pushing power, high difficulty
Mecanum	Large size limited parking
H Drive	Low lateral stability & accuracy

Chassis Speed

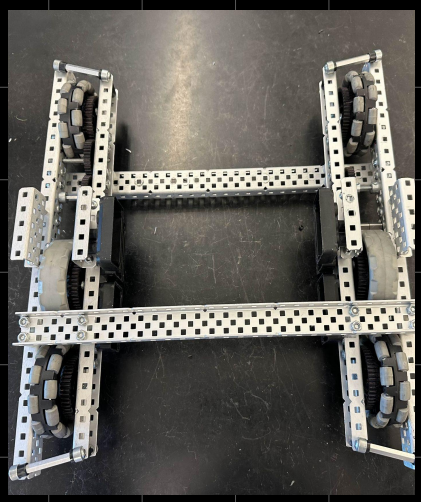
We made a 36T:60T (360rpm), 60T:84T (430 rpm), and 36T:48T (450 rpm) chassis during off-season:



360 RPM



430 RPM



450 RPM

Chassis Speed

We are analyzing which chassis speed gives the best performance overall, based on its speed, driver control, and torque.

Configuration	Speed (RPM)	Driver Control Speed	Torque	Overall Evaluation
36T : 60T	360	Too slow	High (0.58Nm)	Too slow for competitive play
60T : 84T	430	Good	Medium (0.48Nm)	Balance but restrictive
36T : 48T	450	Excellent	Medium (0.47Nm)	Best Overall

We decided a 450 rpm chassis would be optimal after evaluation for mainly (1)easier driver control, and (2)stronger torque, as demonstrated in the below graph.

Chassis Cartridge

To select the motor cartridges, which determines the speed and torque of the chassis, we made comparisons of each cartridges.

100 RPM Red Cartridges:

- Strong/Slow
- Usually used for mechanisms requiring extra **strength**

200 RPM Green Cartridges:

- Maintains the green cartridge preset
- Usually used for mechanisms requiring a balance of **strength and speed**

600 RPM Blue Cartridges:

- **Fast/ Low Friction**
- Usually used for when requiring extra **speed**

Source (VEX official):

VEX motors: https://kb.vex.com/hc/en-us/articles/360044325872-Understanding-V5-Smart-Motor-11W-Performance,

Chassis Gear Ratio

Since torque and speed are **inversely portional** ↓, we can choose any gear ratio to cater to the motors to achieve the desired chassis speed, 450 RPM.

$$\begin{aligned}\text{power} &= \frac{\text{force} \cdot \text{linear distance}}{\text{time}} \\ &= \frac{\left(\frac{\text{torque}}{r}\right) \cdot (r \cdot \text{angular speed} \cdot t)}{t} \\ &= \text{torque} \cdot \text{angular speed}.\end{aligned}$$

Cartridge	Raw Speed (RPM)	Gear Ratio	Possible Teeth Combo	Structural Impact	Overall Evaluation
Red	100	2:9	✖ (Not possible with VEX gears, nearest=300/600)	✖	✖
Green	200	4:9	36:72T is the nearest ratio → gives 400rpm	Large gears limit internal structure	Balanced but restrictive
Blue	600	4:3	48:36T	Compact, easy to integrate	Best Overall

Because 450 RPM is relatively close to 600 rpm, we decided to use blue cartridges:

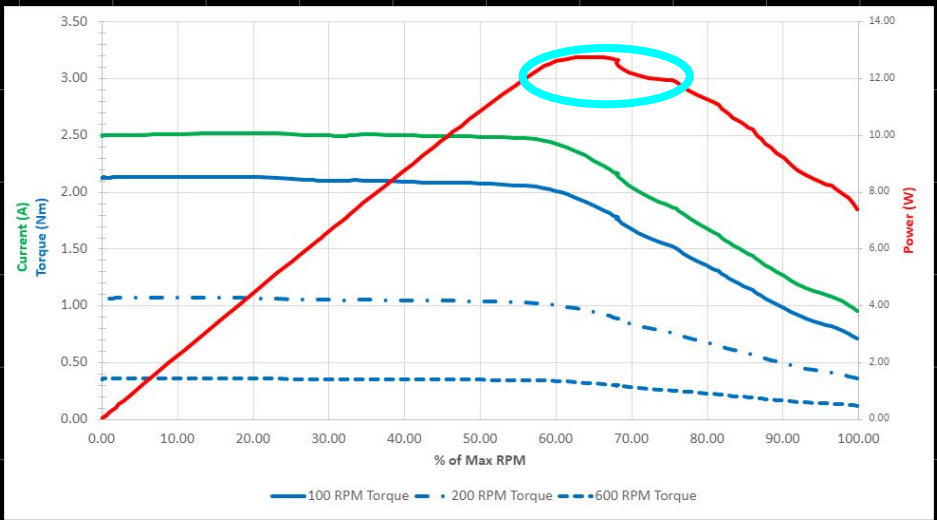
1. Smaller gear teeth differences (would be 4:3)
2. less volume interference to superstructure

Chassis Analysis

Power & Efficiency Analysis:

Choosing 600 rpm cartidriges for our chassis, we can analyze the efficiency of our chassis.

We geared the chassis to 450 rpm, which is 75% of the 600 rpm full speed, just in between the range which gives the largest power of the motor.



←

Based on the motor curve, increasing speed to about 55% - 75% of the motor's max RPM significantly increases the power to above 12 Watts.

Source (VEX official):

Torque-speed curve:

https://kb.vex.com/hc/en-us/articles/360044325872-Understanding-V5-Smart-Motor-11W-Performance,

Chassis Motors

Because we decided to use 600rpm blue cartridges, 6 11-Watt-motors would be optimal to contribute because we agree that a higher strength is required, when we have less torque.

With the available 22 Watts left, there could be 3 combinations (for using up to the **max Watts**):

5.5 Watts Motor	11 Watts Motor
0	2
2	1
4	0

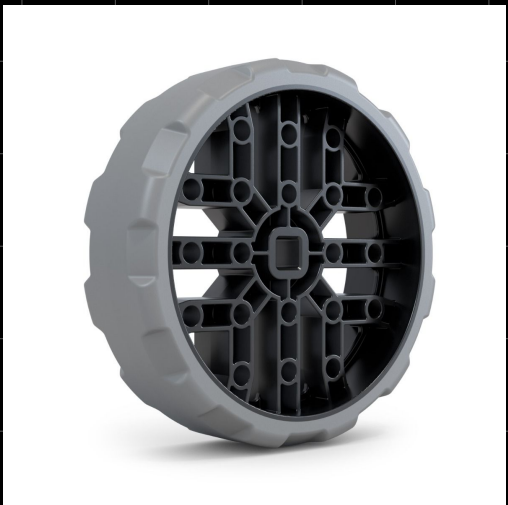
Source (VEX official):

VEX motors: https://kb.vex.com/hc/en-us/articles/360044325872-Understanding-V5-Smart-Motor-11W-Performance,

Chassis Wheel Type

Given the emphasis on above requirements, robots should be designed for quick, agile movement.

Wheel Comparison:



Omni-Wheels

- more lateral movement
- less traction

Traction-Wheels

- less likely to get pushed
- less lateral movement

Source (VEX official):

VEX wheels: https://kb.vex.com/hc/en-us/articles/38888566734612-Understanding-VEX-V5-Wheels

Chassis Wheel Type

TEST 1

To evaluate the **maneuverability & rotation efficiency** of different wheel types.

Test Procedure:

1. Run 10 rotations program
2. Start stopwatch as program starts
3. Stop stopwatch when prototype stops
4. Record results (in seconds)

Program Snippet:c

```
void test()
{
 // Run chassis for 10 rotations
 chassis.DriveL.resetPosition();
 chassis.DriveR.resetPosition();
 Controller1.Screen.clearScreen();
 Controller1.Screen.setCursor(1, 1);
 Controller1.Screen.print("Start!");
 wait(0.05, sec);
 chassis.DriveL.spinFor(forward, 10, turns, false);
 chassis.DriveR.spinFor(forward, 10, turns, true);
 Controller1.Screen.setCursor(2, 1);
 Controller1.Screen.print("Finish!");
}
```

Chassis Wheel Type

TEST 1

Results

Trial	4 Omni Wheels	4 Traction	Omni + Traction
1	1.24	1.36	1.30
2	1.29	1.41	1.34
3	1.22	1.38	1.28
4	1.31	1.44	1.36
5	1.27	1.35	1.33
6	1.26	1.47	1.32
7	1.33	1.42	1.37
8	1.21	1.39	1.29
9	1.28	1.45	1.35
10	1.25	1.40	1.31

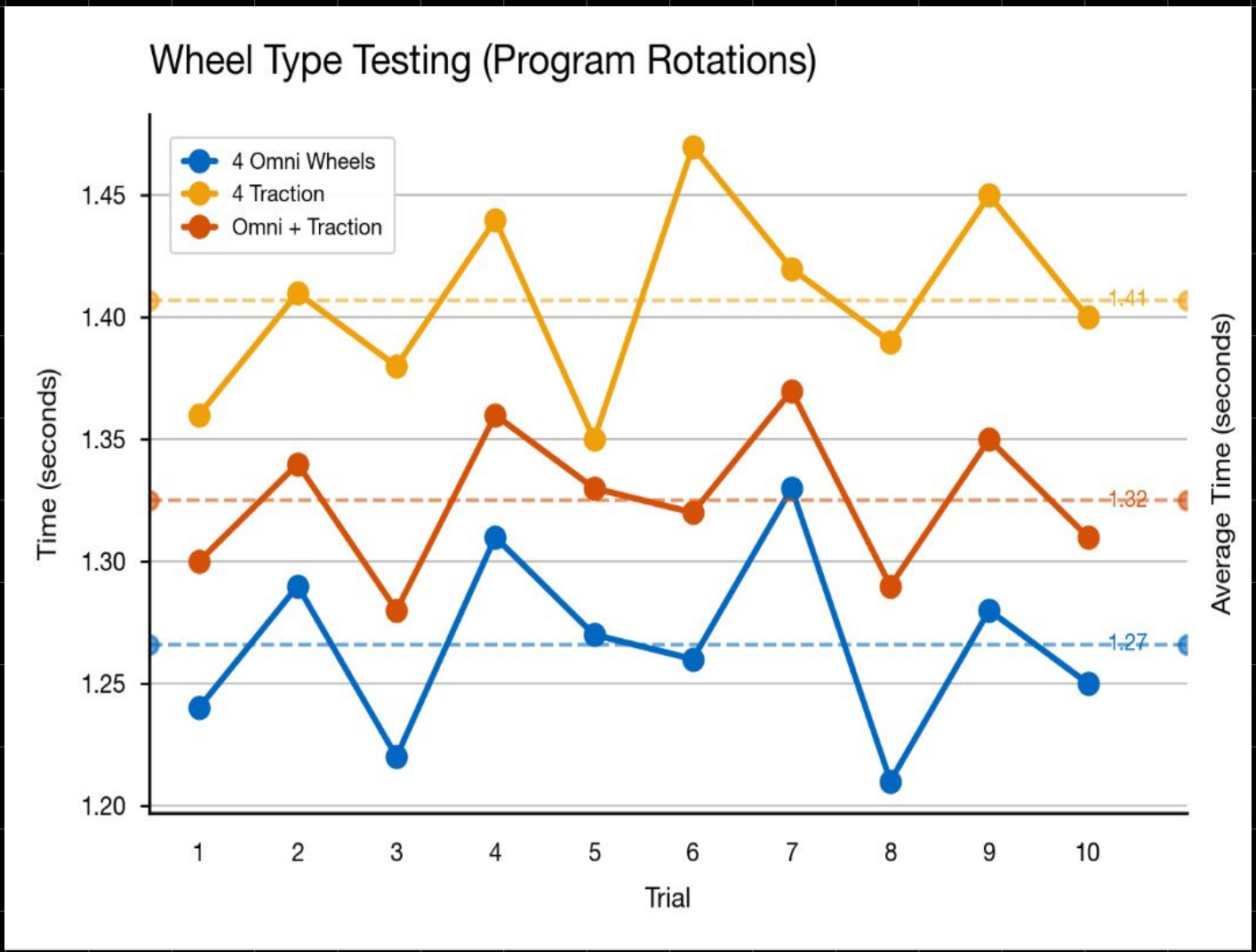
Average Rotation Time (seconds):

4 Omni	4 Traction	Omni + Traction
1.27	1.41	1.32

Chassis Wheel Type

TEST 1

Below plot shows TEST 1 results:



Chassis Wheel Type

TEST 1

Analysis

The **4 omni-wheel configuration** achieved the lowest average rotation time and the smallest variation, indicating turning efficiency.

Traction wheels required additional time due to increased friction, which limits rotational speed.

The mixed configuration improved **turning** compared to full traction wheels but remained slower than full omni wheels.

Decision

This year's game objects and goals, and actions are relatively **few**, therefore, fast escape and scoring would be essential. Therefore, the omni-wheel type was selected for the final drivetrain.

Chassis Wheel Type

TEST 2

To evaluate how efficiently the robot aligns with a target wall using turning and forward motion (test LATERAL)

Test Procedure:

1. Place robot 1.5 m from target at 30° offset
2. Start robot facing away from target
3. Use joystick to align and **drive** to target
4. Stop timer when robot is fully aligned

Record results

Trial	4 Omni Wheels	4 Traction Wheels	Omni + Traction
1	2.74	3.58	3.02
2	2.68	3.44	2.96
3	2.81	3.62	3.15
4	2.72	3.51	3.08
5	2.77	3.47	2.99
6	2.69	3.55	3.11
7	2.83	3.60	3.04
8	2.71	3.42	2.92

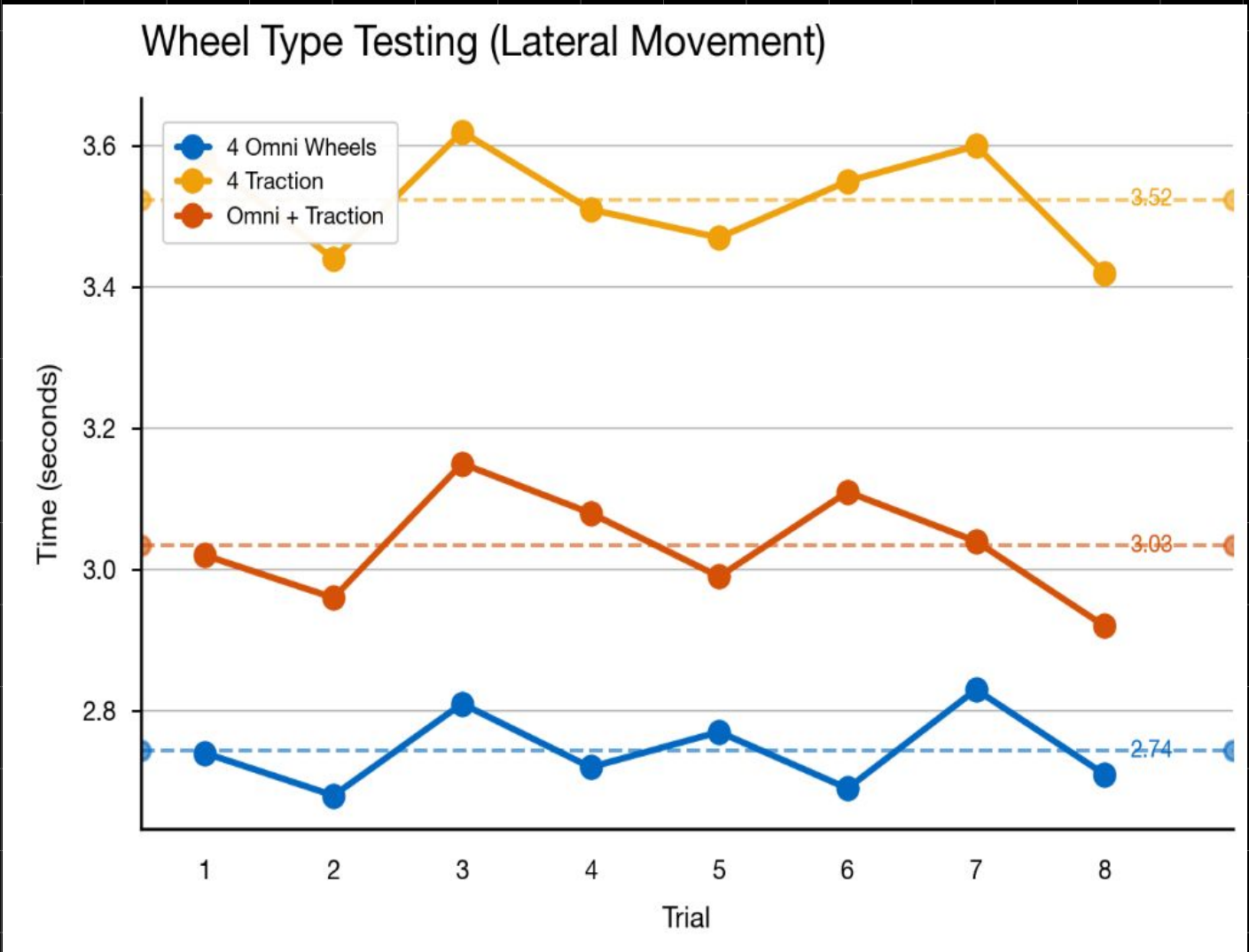
Chassis Wheel Type

TEST 2

Average Rotation Time (seconds):

4 Omni	4 Traction	Omni + Traction
2.74	3.52	3.03

Below plot shows TEST 2 results:



Chassis Wheel Type

TEST 2

Analysis

The omni-wheel configuration enables faster turning and smoother arc motion, reducing the time required for target alignment.

Traction wheels required additional corrective movements due to higher resistance, increasing alignment time.

Decision

Alignment is essential this year, since there are 4 goals and 8 open sections for scoring in total, each requiring relatively high precision when scoring, therefore, fast and accurate lateral adjustments are essential.

Chassis Wheel Size

To select the motor cartridges, which determines the speed and torque of the chassis, we made comparisons of each cartridges.

2.75” Omni Wheel:

- Great center of gravity
- Slow

3.25” Omni Wheel:

- Balance in speed and strength

4” Omni Wheel:

- High mobility/speed

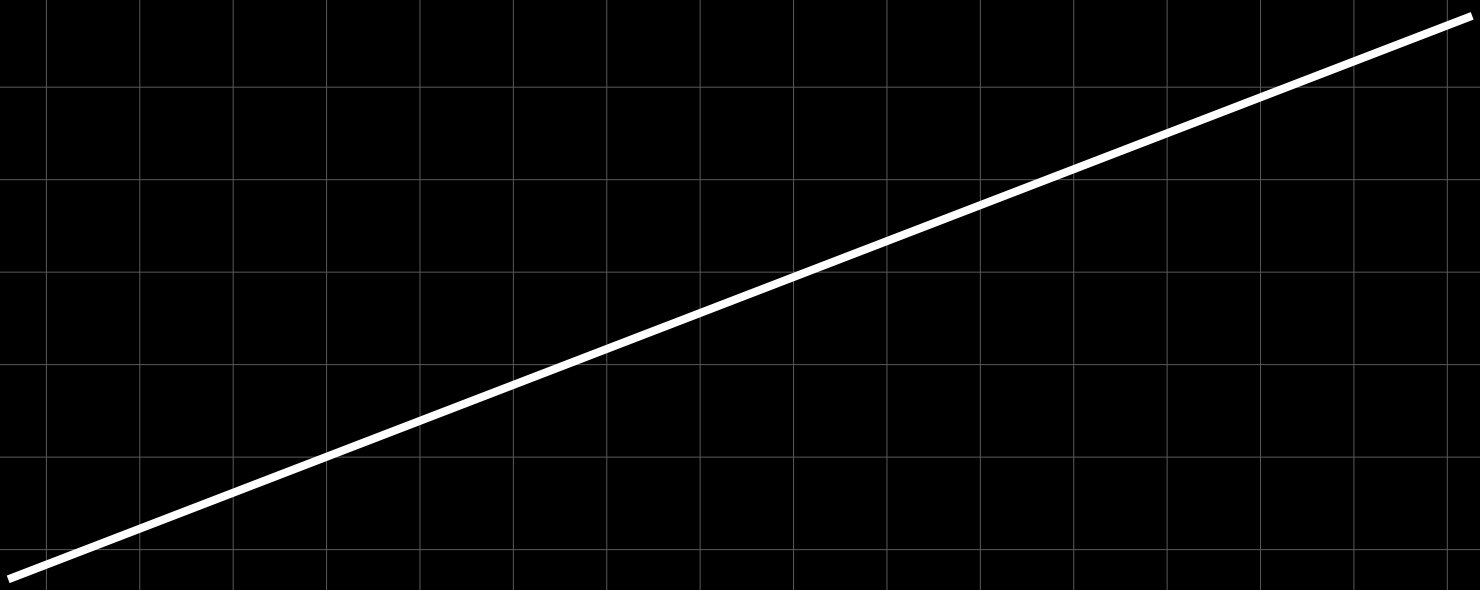
Source (VEX official):

VEX wheels: https://kb.vex.com/hc/en-us/articles/38888566734612-Understanding-VEX-V5-Wheels

Chassis Wheel Size

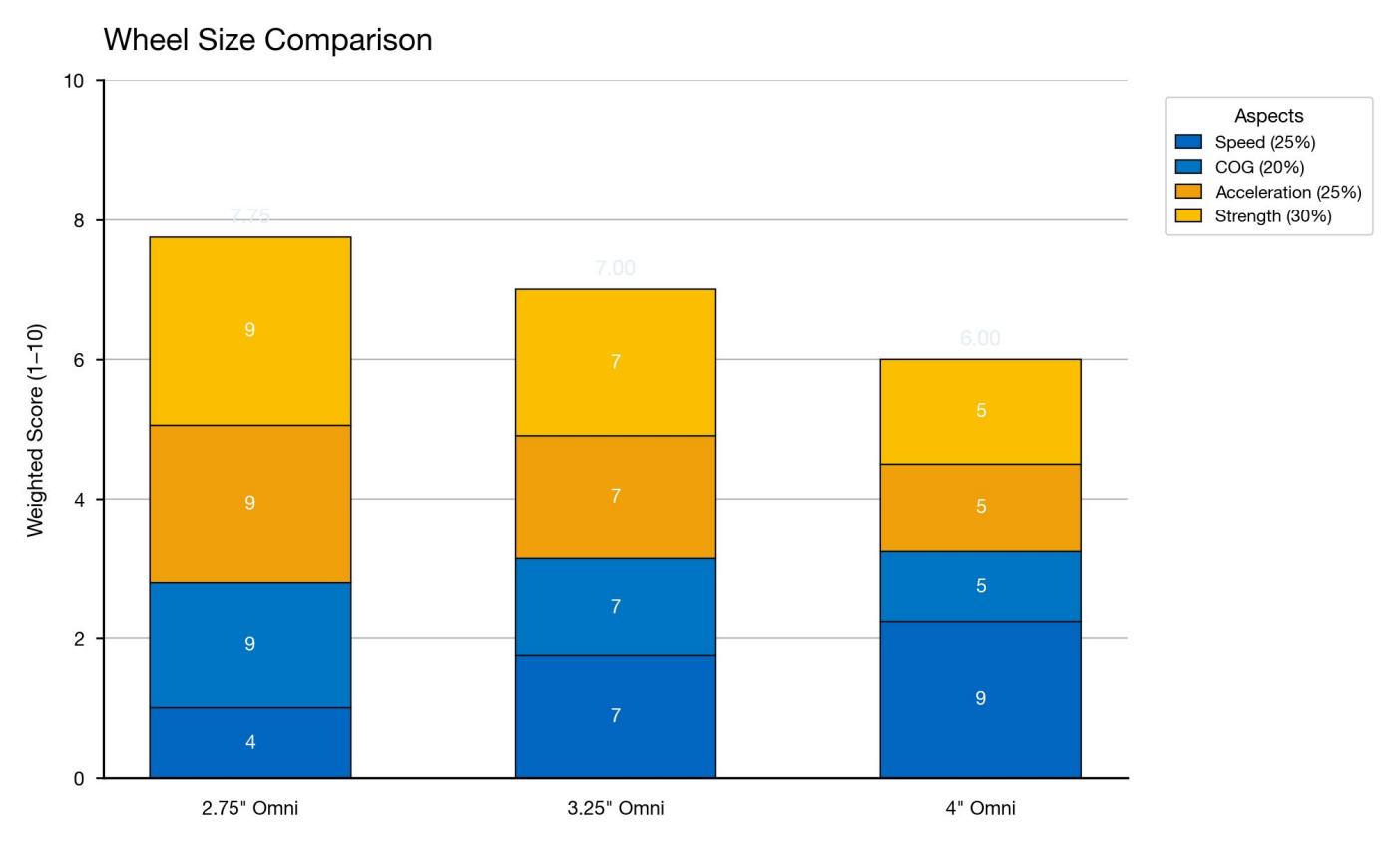
We evaluated 4 aspects that would affect our choice of wheel size:

Speed: 25%, it plays a relatively important aspect for the chassis, especially for this season.
Center of Gravity: 20%, it affects the defensibility and accuracy of the chassis.
Acceleration: 25%, it determines the efficiency of the chassis.
Strength: 30%, it is our prioritization since our chassis has a relatively high speed



Chassis Wheel Size

The resulting scoring for each chassis is plotted:



Final Decision:

The 3.25" omni-wheel achieved the best balance of comprehensively.

Chassis Summary

The ultimate drivetrain setup is summarized in the below table:

Component	Selection	Impact
Gear Ratio	36T:48T (450 RPM)	Compact gearbox
Wheels	3.25" Omni	Low COG, high agility
Motors	6×11W Blue	High torque
Type	Tank	High Compatibility & Maneuverability

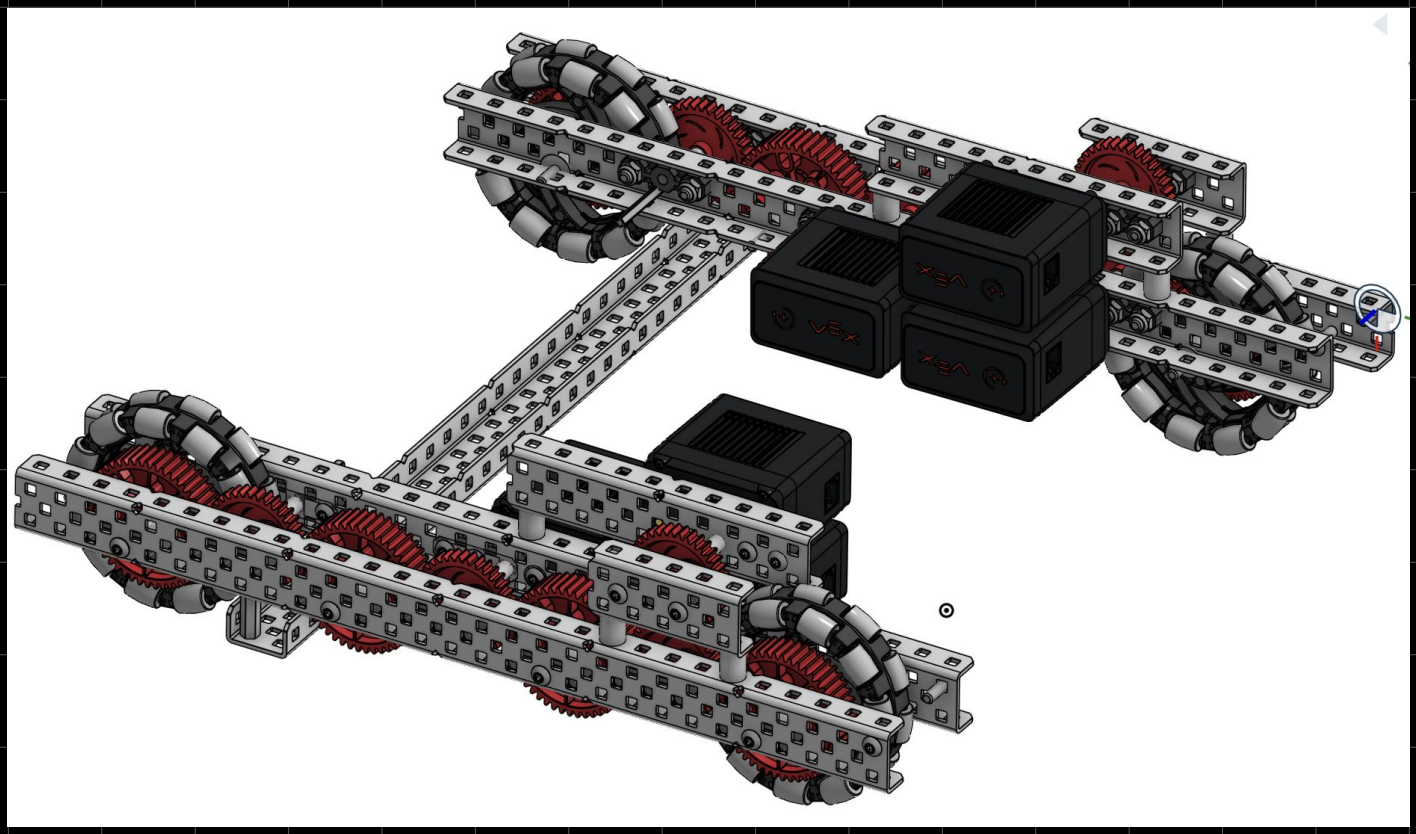
The trade-offs of our chassis setup is also summarized:

Trade-off	Benefit	Limitation
High speed	Agile movement	Less strength
Omni wheels	Maneuverability	Reduced traction

We have emphasized mobility for this season, therefore our chassis setup is tailored to this strategy.

Chassis CAD

The ultimate drivetrain setup is illustrated with the CAD provided.



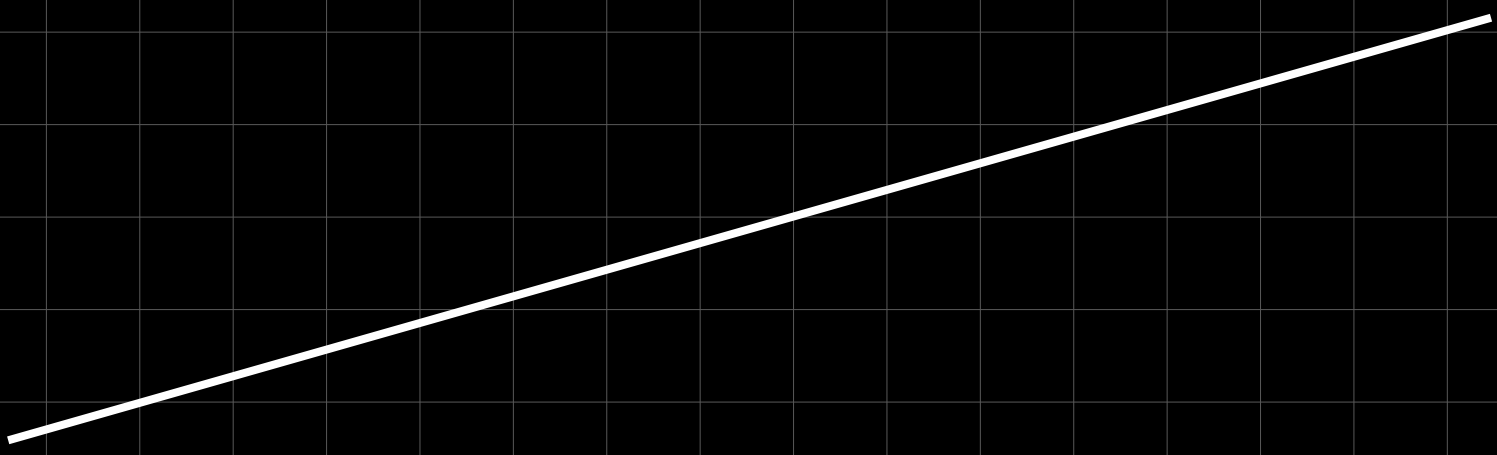
Meeting Logs

2024/07/14 — 30 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- Brainstorm Planning

After developing an outline of our chassis and its CAD, we established another to do list in today's meeting:

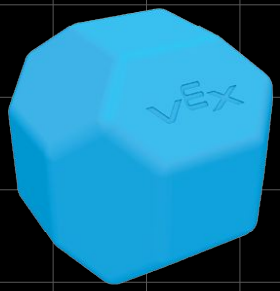
1. Evaluate and analyze different intake design
   - Research and compare
2. Brainstorm shooter design and structure
3. Analyze the function of the aligner, tongue, and descorer; brainstorm design



Intake

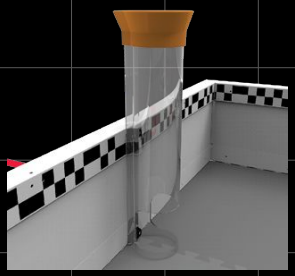
Game Object Analysis

This year's game object is a ball like block, but with 16 faces, making each rotating moment having different sizes.



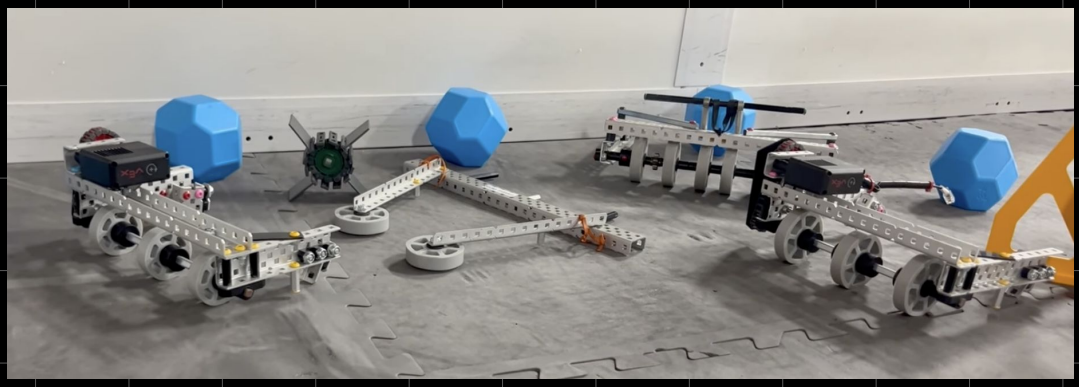
Field Object Analysis

The robot has to be able to load blocks from the loader, therefore a independent loader mech is required to help guide the blocks into the intake system.



Requirements

The intake system needs adequate amount of compression to smoothly tackle the inconsistent intaked size.

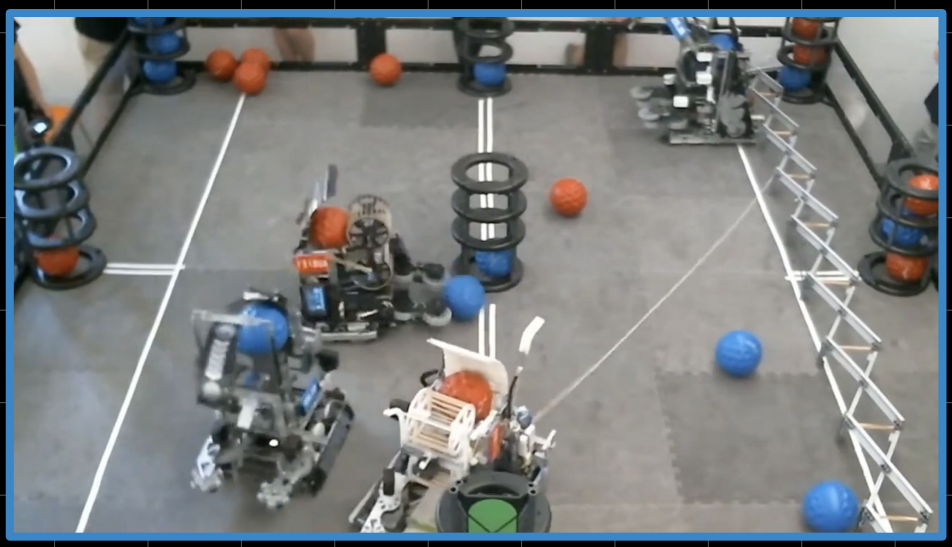


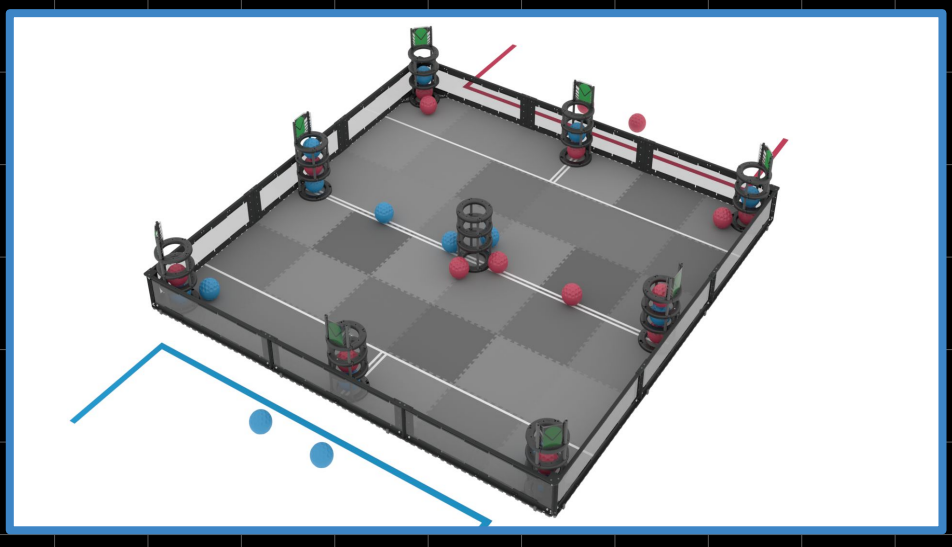
Intake

Horizontal Flex Wheels:

This year's game object is similar to 2020-21 Change Up game season.

Push Back's loader resembles Change Up's goal, and the meta that year is the horizontal flex wheels, which allows direct intaking from the vertical tubes.

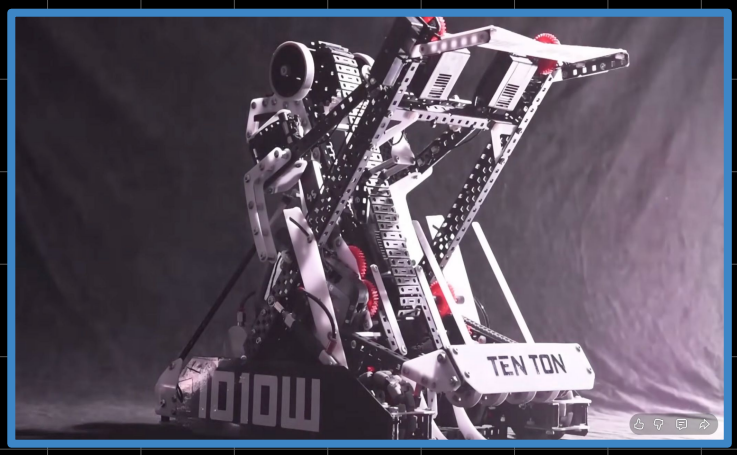




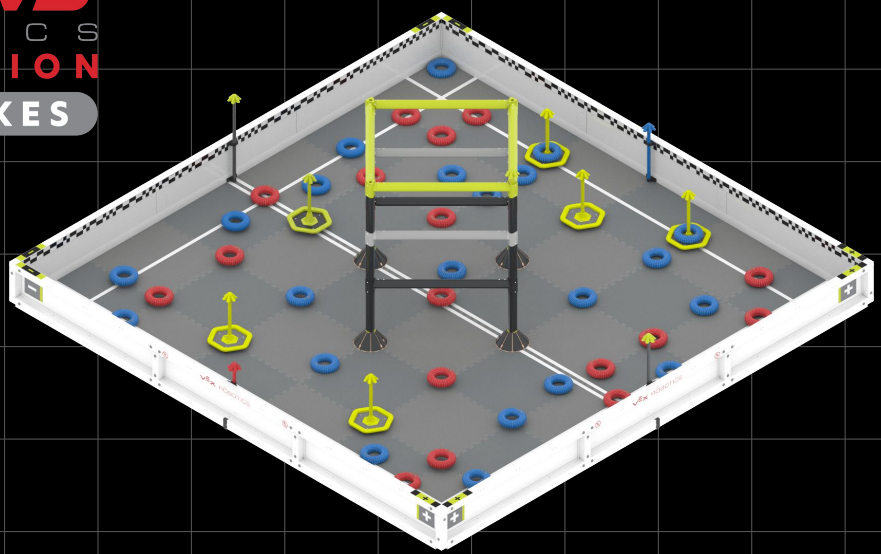
Intake

Flex Wheels in a row:

Last season (2024-25 High Stakes) had the intake meta of flex wheels, this mech is (ideally) applicable for this season's game objects as well.



VEX V5 ROBOTICS COMPETITION HIGH STAKES

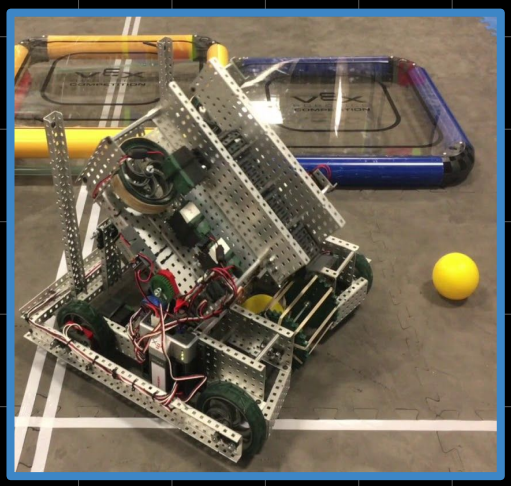


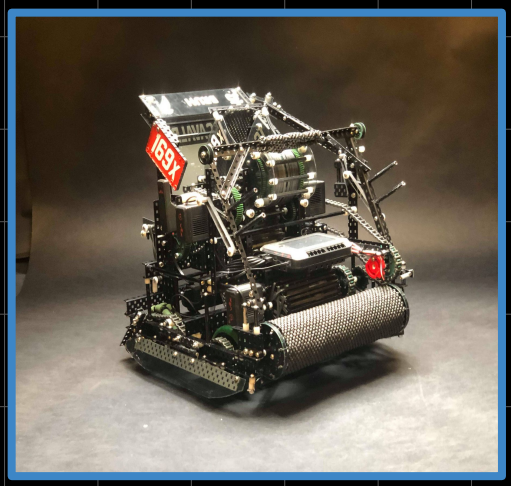
Intake

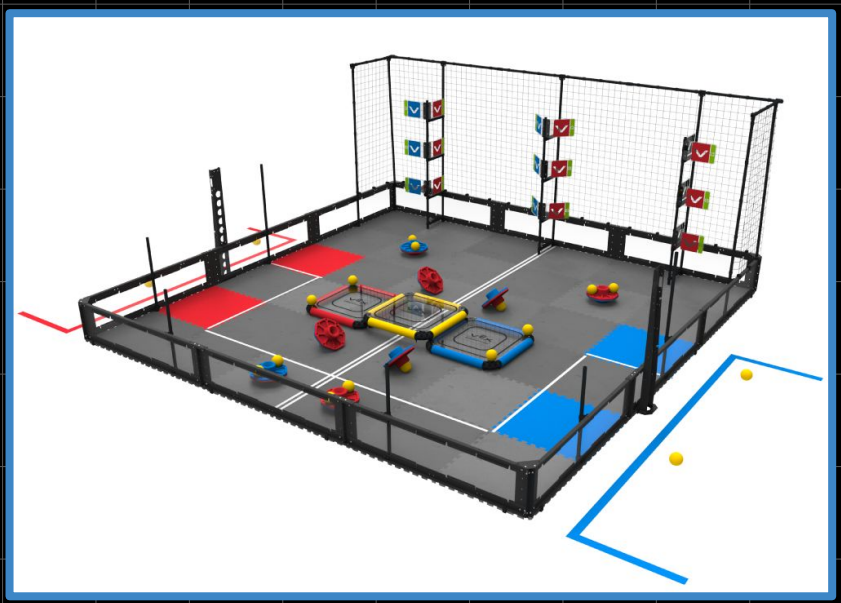
Horizontal Flex Wheels:

This year's game object is also similar to the 2018-19 Turning Point game season.

Both seasons need to intake ball-like objects from the ground, and the meta that year is the sprockets with rubberbands and/or with anti-static mats.

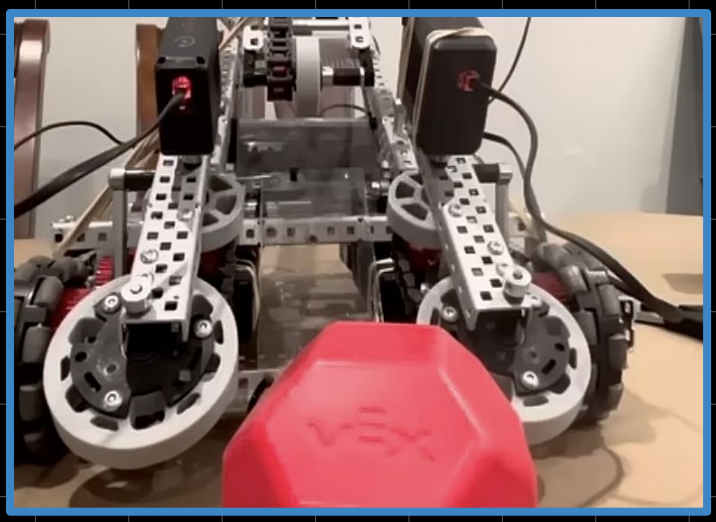


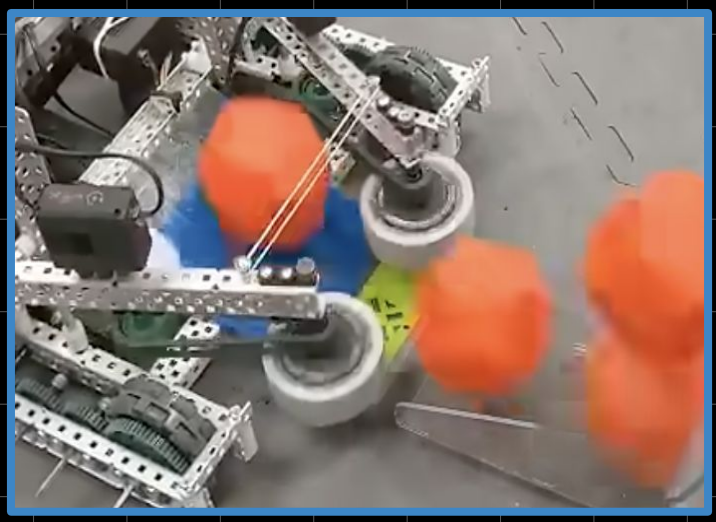




Intake

Intake Type	Pros	Cons
Flex wheels (horizontal)	Many teams use it as a claw structure which is flexible and can control when (possibly optical) to intake it	Means that it would require at least two motors, 1 for 1 side to intake. Also, the design would make the robot longer since the whole claw is near the ground.



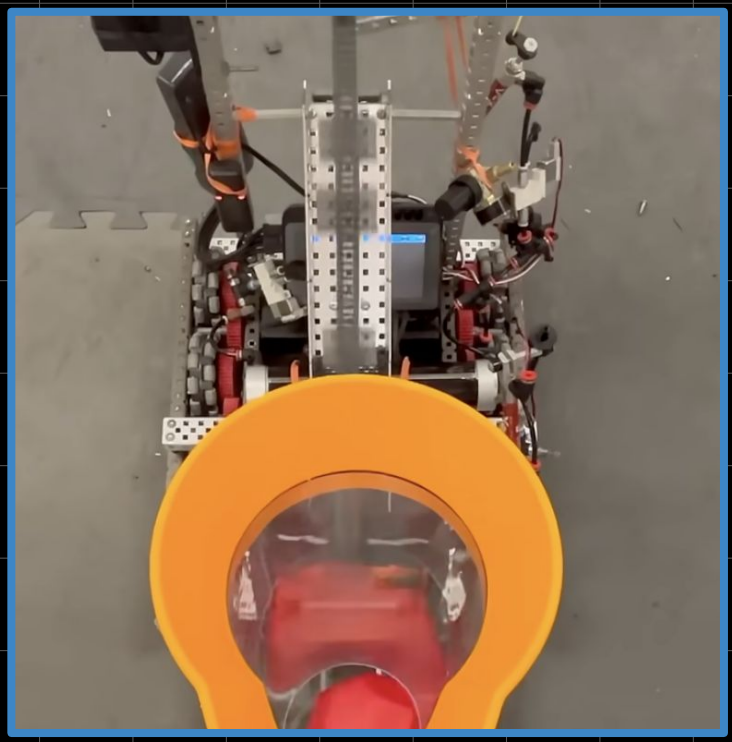


Early Season prototypes from Youtube

Intake

Intake Brainstorming PROS & CONS

Intake Type	Pros	Cons
Flex wheels (in a row)	Pretty simple and straightforward, especially that we experienced with this type of intake last year.	Might provide less elasticity for space, height and the game object.

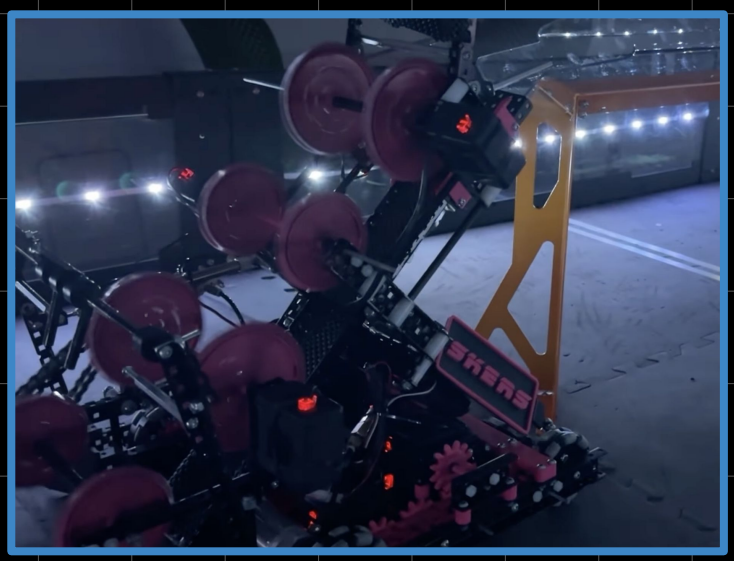


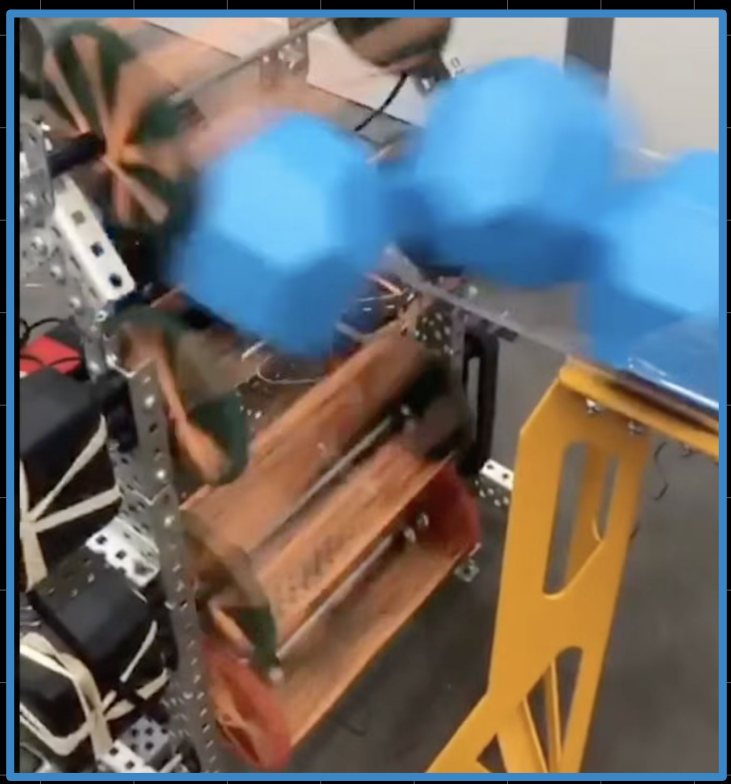
Early Season prototypes from Youtube

Intake

Intake Brainstorming PROS & CONS

Intake Type	Pros	Cons
Sprockets with Rubber Bands	Also quite simple and straightforward. We experienced with this structure in the over under season. It is also very flexible and can grab the balls pretty well with its high friction.	Rubber bands might snap during the game and would hurt the intake efficiency a lot.



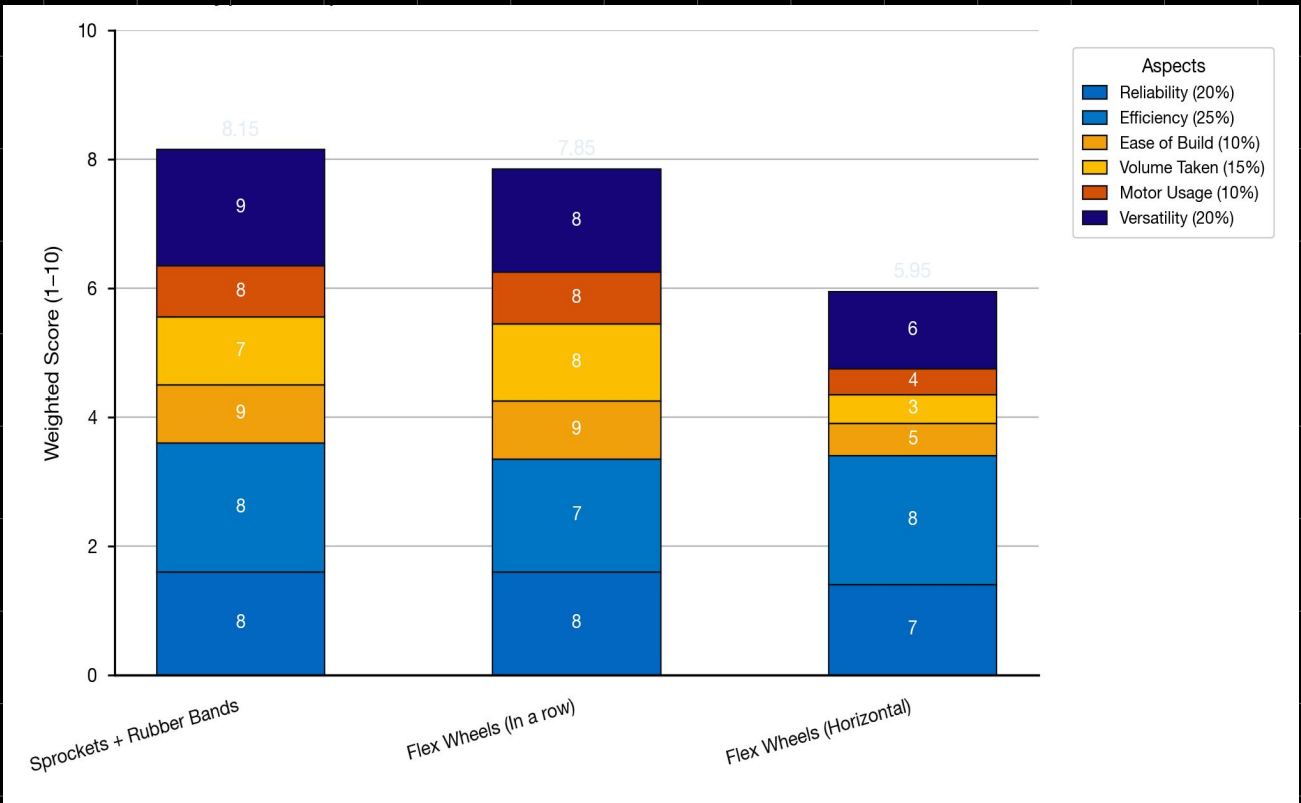


Early Season prototypes from Youtube

Intake

We decided that we evaluate the intake types together, considering that it helps to objectively evaluate and decide. A decent intake system should demonstrate high scores in:

1. **Reliability**: how stable and consistent it is
2. **Efficiency**: the speed and smoothness
3. **Ease of build**: how fast/difficult we can build it
4. **Volume taken**: how much it uptakes of the robot size
5. **Motor Usage**: motors/voltages does it require
6. **Versatility**: how easy it is to change/modify the design



Intake

After team discussion and brainstorming, we have decided our robot's intake setup:

- The control zone of the long goal would be an important part, so a robust intake system would dominate the field.
- The lower middle goal can use the intake system to score by reversing.
- We decided to go for an intake that is sprockets with rubber bands, because the compression amount is larger, allowing less accurate heights and structure, more compatibility.

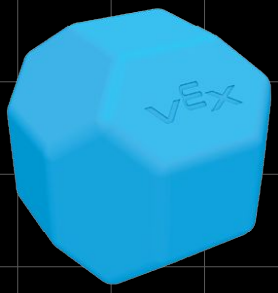
For Future Building:

1. Decide whether we want it to intake and score on the same side or opposite side
2. Decide if we want to do color sorting or store other teams' color's balls.

Shooter

Game Object Analysis

SImilar to the intake, the shooter system shall have an adequate amount of compression to score the 16 faces blocks into the goals.

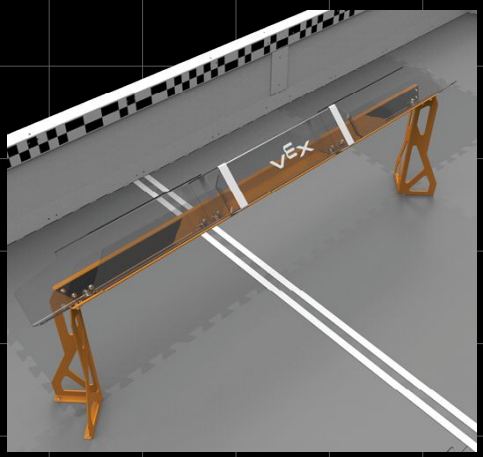


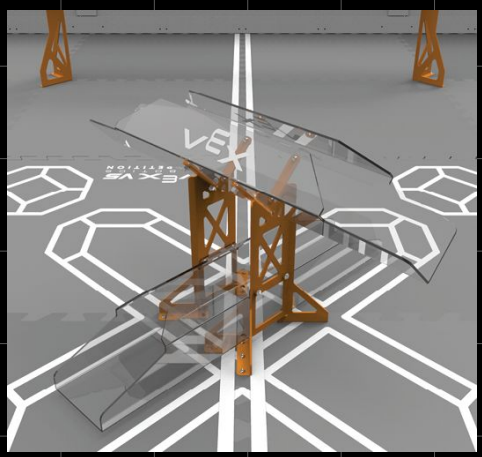
Field Object Analysis

The goals have 3 levels: lower middle, upper middle, and ong goal.

As mentioned before, the intake can score into the lower middle goals by reversing itself.

This leaves 2 levels of goals that are required to score into.





Requirements

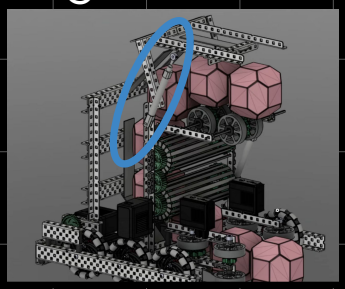
The shooter system shall contain 2 levels of outake scoring.

Shooter

The **shooter** would be connected to the **intake**, through **belts**, **sprockets**, or **flex wheels** that chain the blocks up towards the goal heights, these are possible solutions:

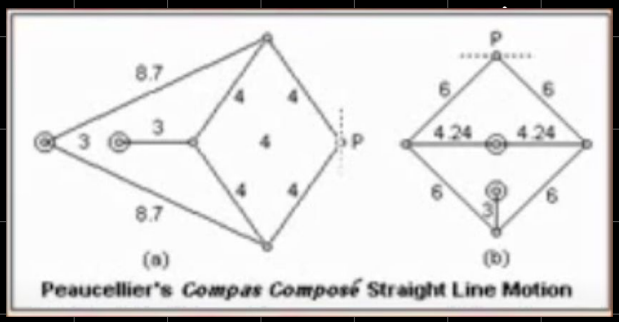
1. **Pistons**

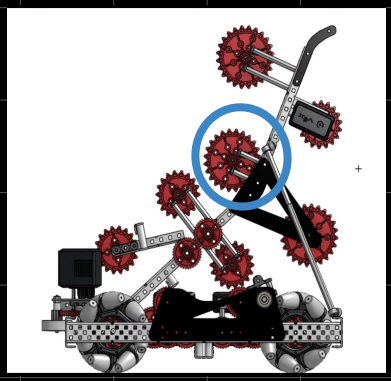
   Pistons can alter the height of the outake structure, providing exactly 2 levels of height.



2. **Parallel Structure**

   This is also a commonly seen mech in VEX, using motors to level up and down a structure





3. **Two pathways**

   Another feasible way is to create 2 pathways that can outake to two levels. The path can be controlled by rotating direction (by motors) or blocking mechs (pistons).

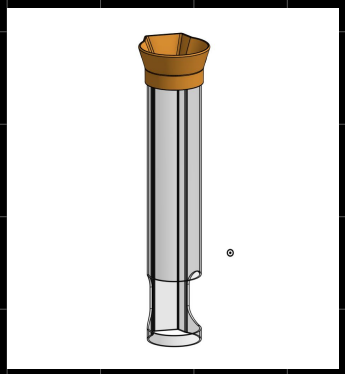
↑

Aligner

An aligner is an often overlooked design, but also essential because it allows faster and steadier scoring/loading.

1. **Loader**

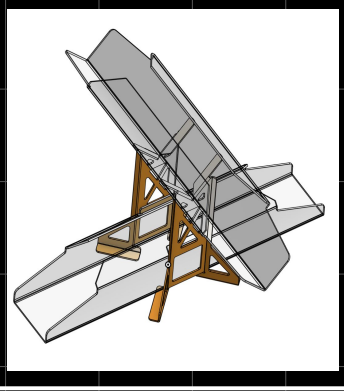
   The aligner for the loader could be tailored to fit the semi-circle shape of the loader tube.

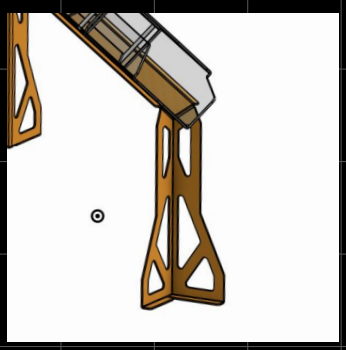


2. **Goals**

   The long goals have a triangular shape under the scoring section. The robot can design a triangular shape mech.

   However, the middle goals' triangle section are rather far from the scoring section, therefore an aligner that fits the the scoring section might work to center the robot.



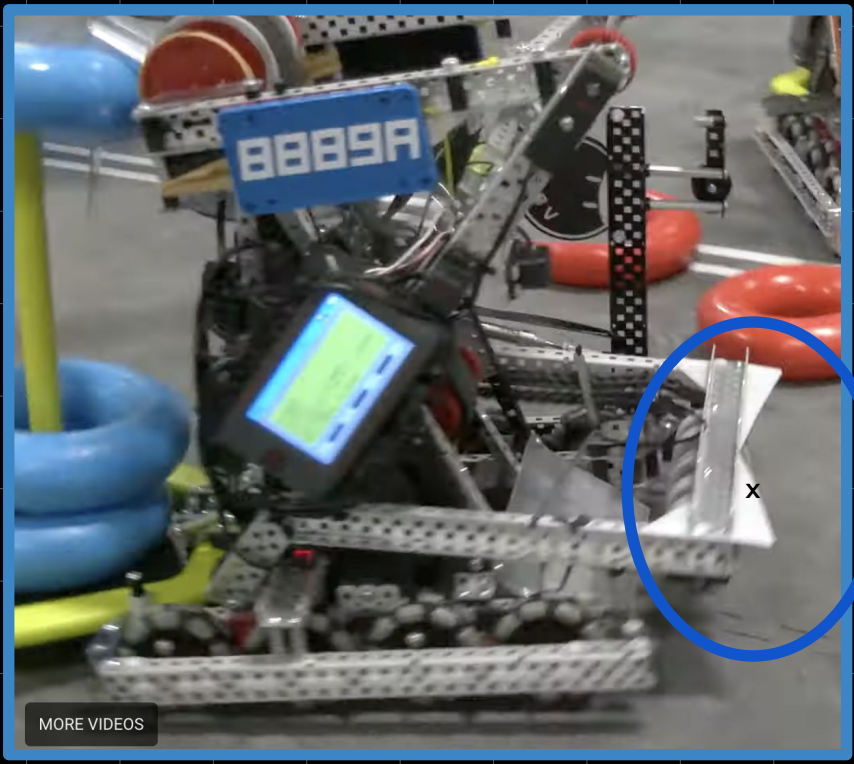


Aligner

Previous seasons also demonstrated that aligner mechs do not essentially need to be independent structures.

2024-25 High Stakes

Teams used plastic boards to design aligners that could center them to the **wall stakes** last year, incorporating their plastic boards with the intake design.



Tongue

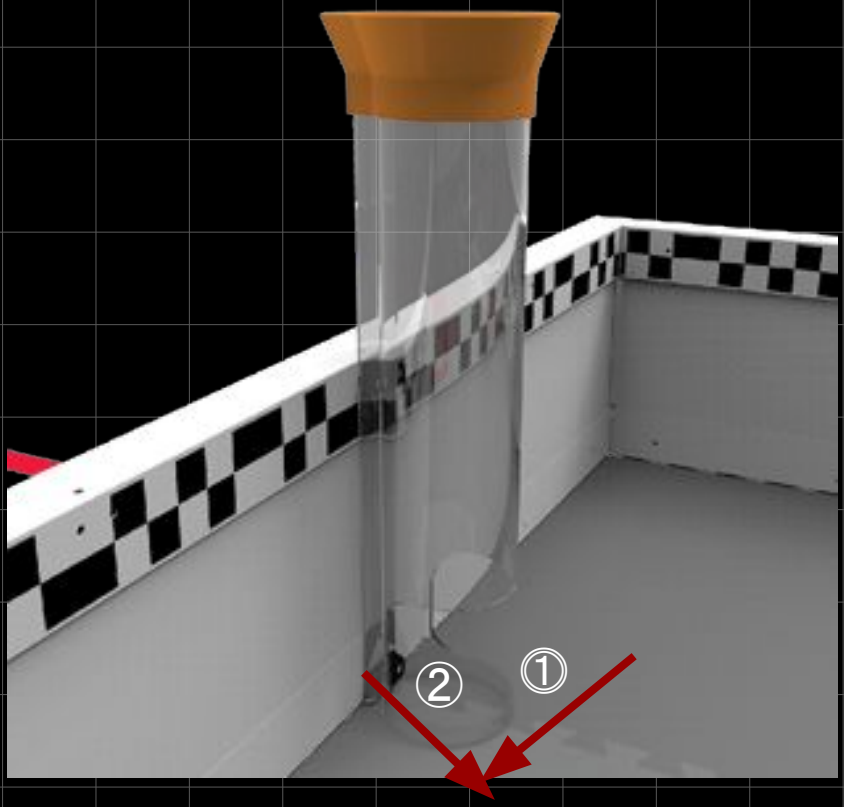
Robots have to load blocks from the loader throughout the game, both from the preset field and human player loading.

1. Enter Angle

   The tongue has to enter the goal with a downward angle.

2. Intake Angle

   The tongue will then guide the blocks out one by one with a upward guiding angle into the angle system

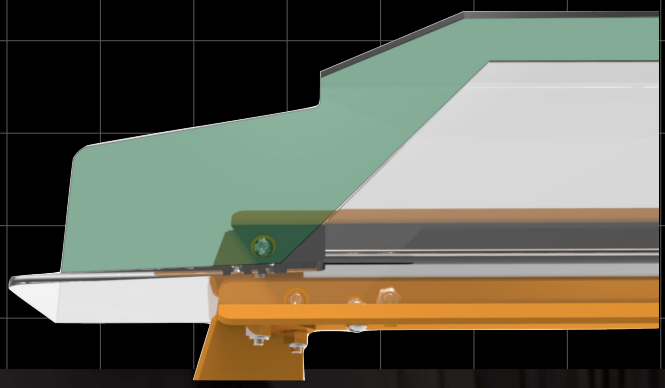


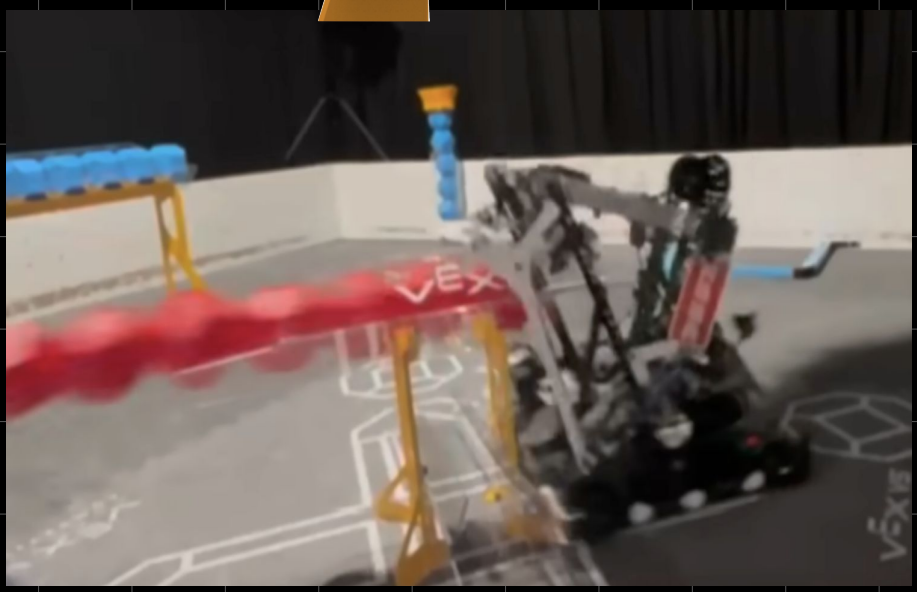
Descorer

As we have finished brainstorming the offensive superstructures, we shall think of defensive mechs, we would call it a descorer.

<SG10> Don’t reach inside enclosed sections of Goals.

We would like to design a descorer that would be a board that pushes scored blocks with great force, effectively displacing/relocating blocks.





Meeting Challenges

2025/07/17 — 30 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony	- Challenges Analysis - Strategy

In today's meeting, we discuss the challenges we might face in this season's matches and possible solutions we should import to our design and strategies:

Challenge 1:

Loaders might be the fastest way to intake, comparing to intaking from the field. Without another force coming from the other side stopping it, the intake system might only push it further away.

Solution:

Make the system higher, lighter, more flexible, perhaps use rubber bands.

Meeting Logs

Early Season Strategy Brainstorming

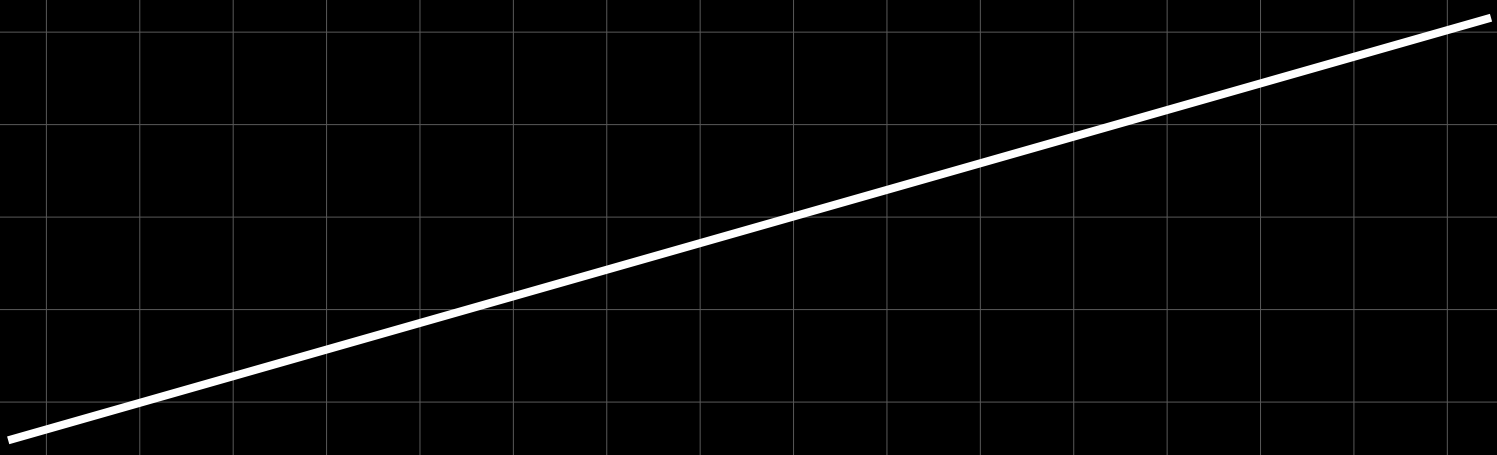
go for the loaders near the **alliance stations**

Intake alliance color blocks then score

intake then spit out the opponent blocks to the ground

Pros

It would be hard for opponent to intake their blocks



Meeting Logs

Challenge 2:

When the opponent alliance has already several blocks on **both sides** of the long goal, scoring more of our blocks might result in them controlling the zone (even 3 blocks*3 pts<10 pts!).

Solution:

Score in at least 4 more to make this a beneficial action.

- If we don't have a tally of 4 blocks on our car or in the immediate vicinity
  → go for another goal or play defense
- If they are already occupying the control zone
  → score to push them out and gain points.

04
ROBOT 1

Meeting BUILDING PLAN

2025/07/25 — 30 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Jefferson	- Establish a pre-building plan

1. We should CAD 70% of the robot before starting to build one, build the CADed part as CAD continues.
2. We consider being able to go under the long goal a neccessary characteristic, our team might stick to this idea the whole season after we watched some matches and anaylzed
3. Storage space is important because it seems like most defense happen at the goals and if we need to score frequently with only 3 or 4 balls each time we risk bigger.
4. We value agility a lot, so we hope we should design a chassis that would fulfill this goal

Pre-Building Timeline

Tasks	7/25-8/1	8/2-8/8	8/9-8/15	8/16-8/22	8/23-8/29
Research					
Chassis CAD					
Personalize					
Robot Types					
Prototype & CAD					

Tasks per member:

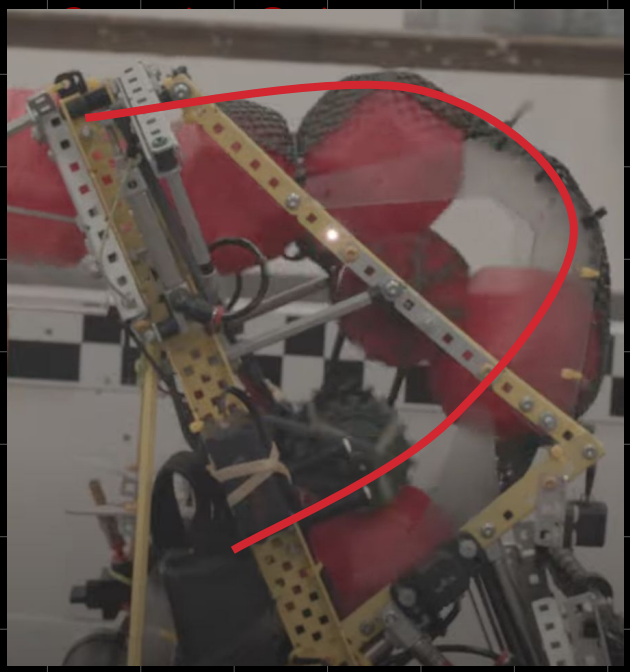
- Research: Pennlaine
- Chassis CAD: Pennlaine, Anthony
- Personalize: Jefferson
- Robot Types: Pennlaine, Adam, Anthony
- Prototype: Pennlaine, Anthony

Robot Type

1. To fulfill our building plan, we cannot build the robots with big trunks because we need to go under the long goal.
   Snail like or C-shaped designs with **pistons** for a higher level would be a more suitable choice

2. We decided to make the **chassis narrower** for more agile movements, namely
   a. easier parking
   b. better defending
   c. faster turning for scoring around the loaders and goals
3. We prioritize **sprockets with rubber bands** for intaking blocks because of its high compression ratio, when necessary, we might tie mats around it to make the friction higher and to decrease the compression value.

Research



Snacky Cakes

This is a newly revealed early season robot that inspired us. Previous robots for this season were of big trunks for storage, but that sacrificed the robot's mobility.

This robot has a

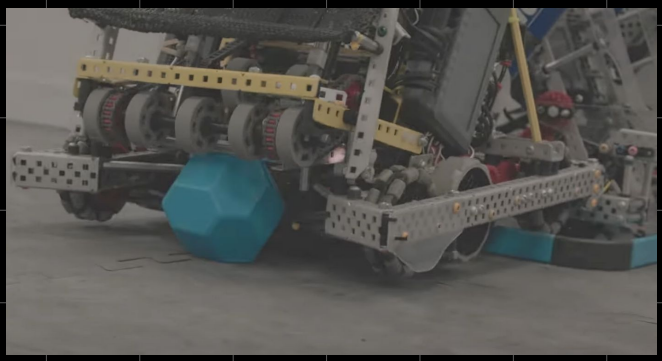
- S like design
- large storage

The robot uses

- Sprockets to divide the levels
- control its rotation direction to outake the blocks to different level goals.

However, it still doesn't meet our requirement of able to go under the long gaol .

Research



Snacky Cakes

Cool thing to note: their parking is dope! They use a block to lift their robot up so that they don't touch the floor out of the parking zone.

Takeaways:

We can adopt the S-like design and other structures to enlarge the storage. To allow our robot to go under the long goal, we can add pistons at a structure to allow it to lower down for the middle goal level.

Chassis Assembling

We are experiencing with different methods of assembling the drivetrain.

We want to choose the smoothest assembly method, including inserts and spacers.

We are first determining the shaft inserts that would be inserted into wheels. We are overwhelmed, not knowing which insert we should choose, since there are 3 types of inserts in total, including old versions that are discontined but we still have lots of stock of.

High Strength Shaft Insert
SKU: 276-3881
Green Discontinued!

High Strength Shaft Insert
SKU: 276-3881
New Version = Grey

High Strength Shaft Adapter
SKU: 276-8034

Wheels

Wheels Inserts Experiment:

TEST 1

To evaluate the rotational smoothness and friction characteristics of different shaft inserts to minimize drivetrain effeciency loss.

Test Procedure

1. Mount wheel with selected shaft adapter
2. Run motor at 50% power for 3 seconds
3. Cut power instantly
4. Start stopwatch when wheel fully stops

Results

Trial	Green Insert	Grey Insert	Gold Adapter
1	4.21	5.63	7.44
2	4.35	5.58	7.31
3	4.18	5.71	7.52
4	4.27	5.66	7.39
5	4.12	5.60	7.47
6	4.30	5.69	7.58
7	4.25	5.55	7.36
8	4.19	5.62	7.41

Wheels

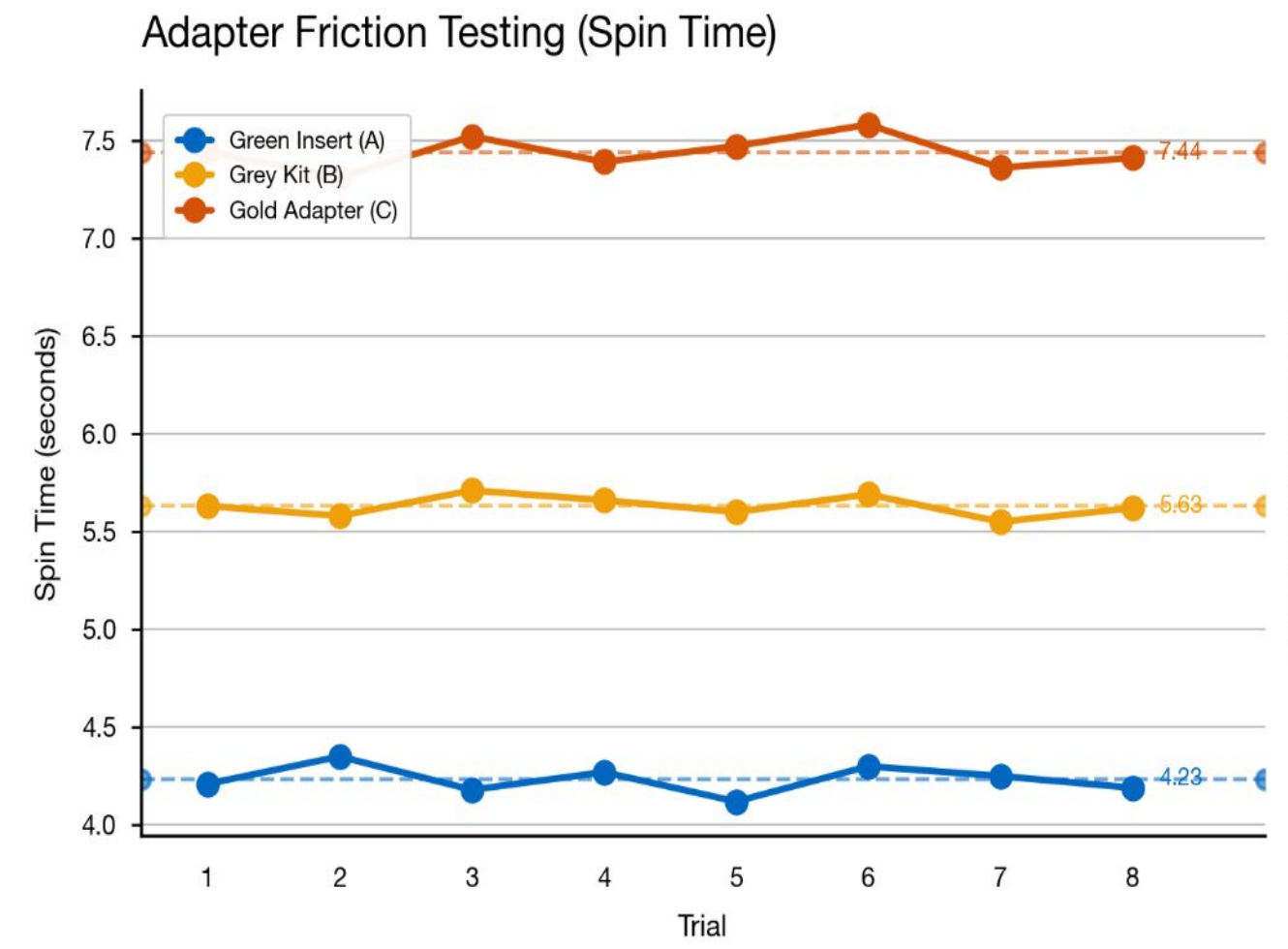
Wheels Inserts Experiment:

TEST 1

Average Spin Time

Adapter	Avg Time (s)	Relative Friction
Green	4.23	High
Grey	5.63	Medium
Gold	7.44	Low

Below plot shows TEST 1 results:



Wheels

Wheels Inserts Experiment:

TEST 1

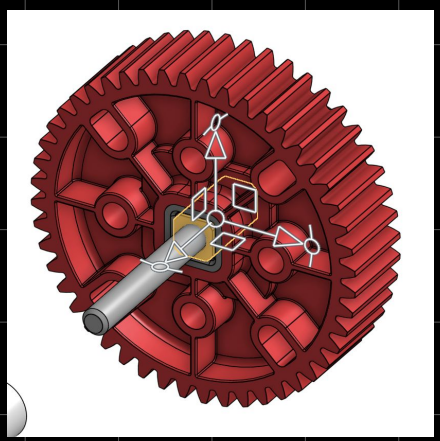
Analysis

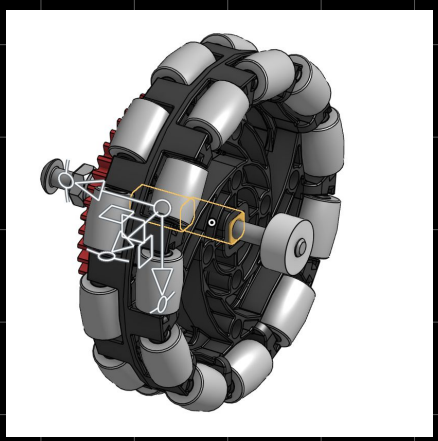
The gold shaft adapters consistently exhibited the longest free-spin time, indicating the lowest internal friction.

The grey insert kit showed moderate performance, while the green inserts produced the highest resistance, resulting in faster deceleration.

Decision

The gold shaft adapters improved efficiency and reliability, leading to their selection for both wheels and gears.

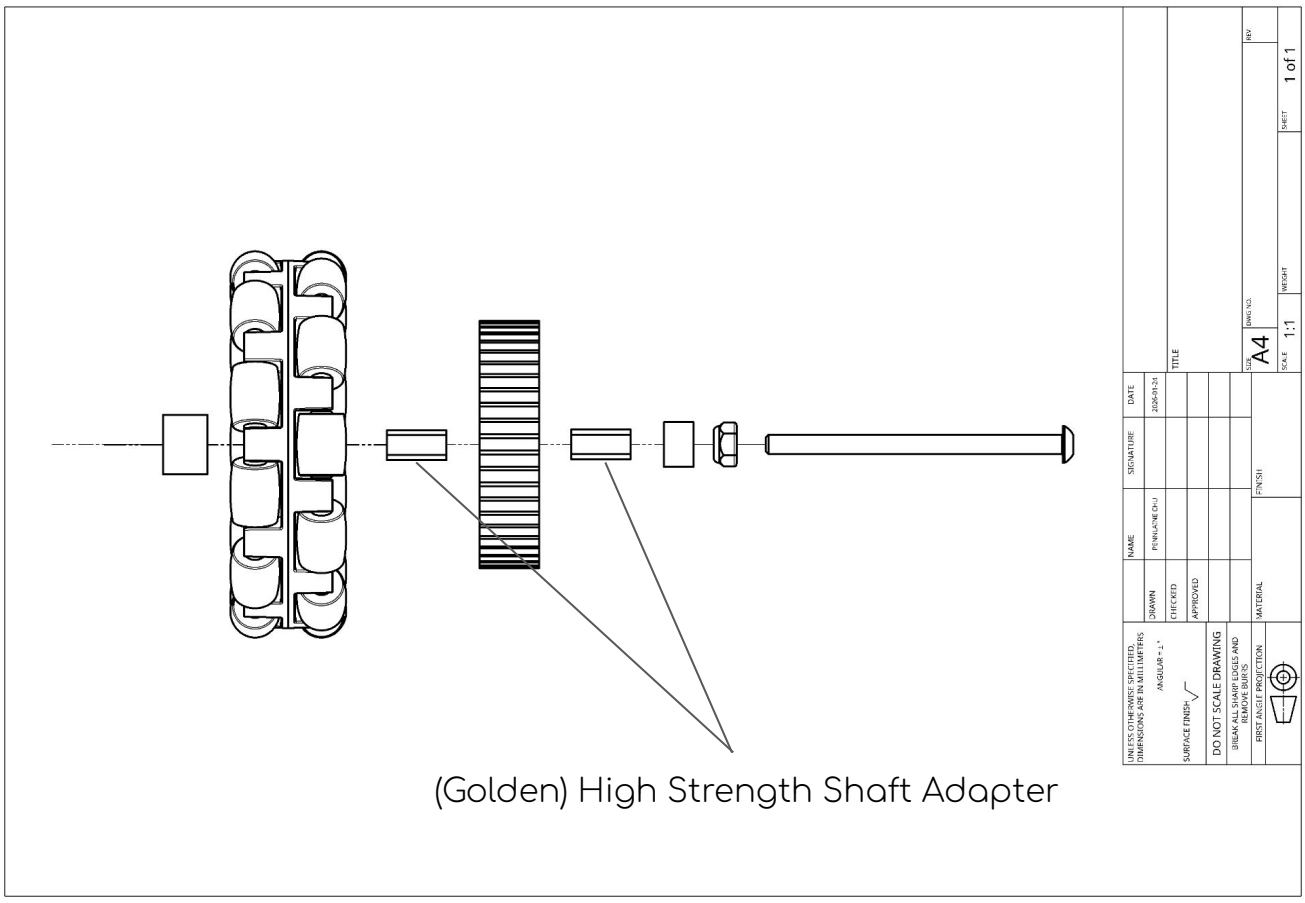




Wheels and Gears

Chassis Wheels and Gears Spacing

This is an exploded view of our wheel assembly set up, using golden shaft adapters.

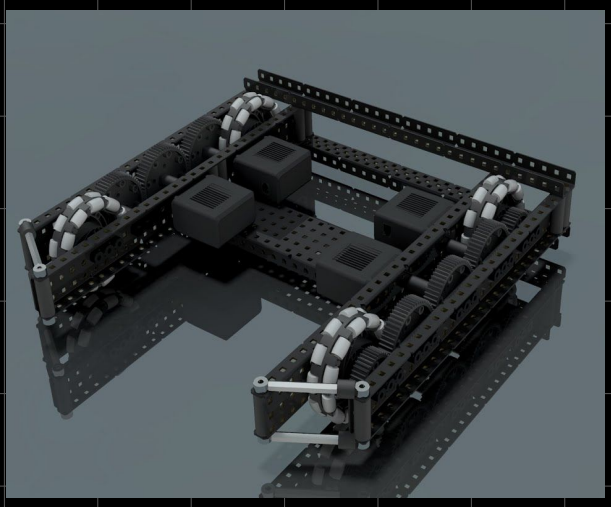


(Golden) High Strength Shaft Adapter

Gears Placement

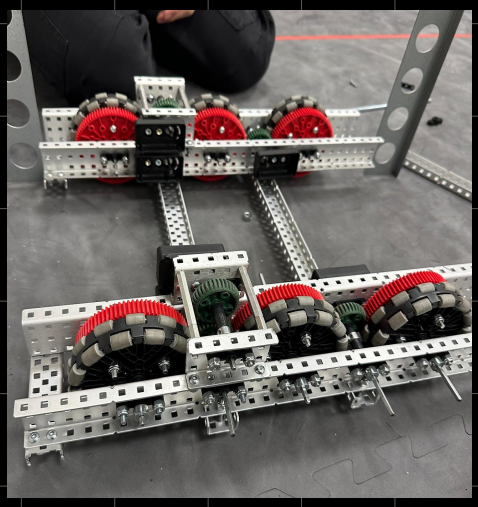
On **Discord** servers, we saw that a lot of teams had their gears placed on the outer side of the chassis, instead of what we previously experienced with.

If the gears were put on the outer side, the wheels would be on the inner side. The drivetrain would be more agile since for every same-degree turn, it results in a shorter wheel traveling distance, therefore faster.



From team 99999V on Discord
Gears on the outside

V.S

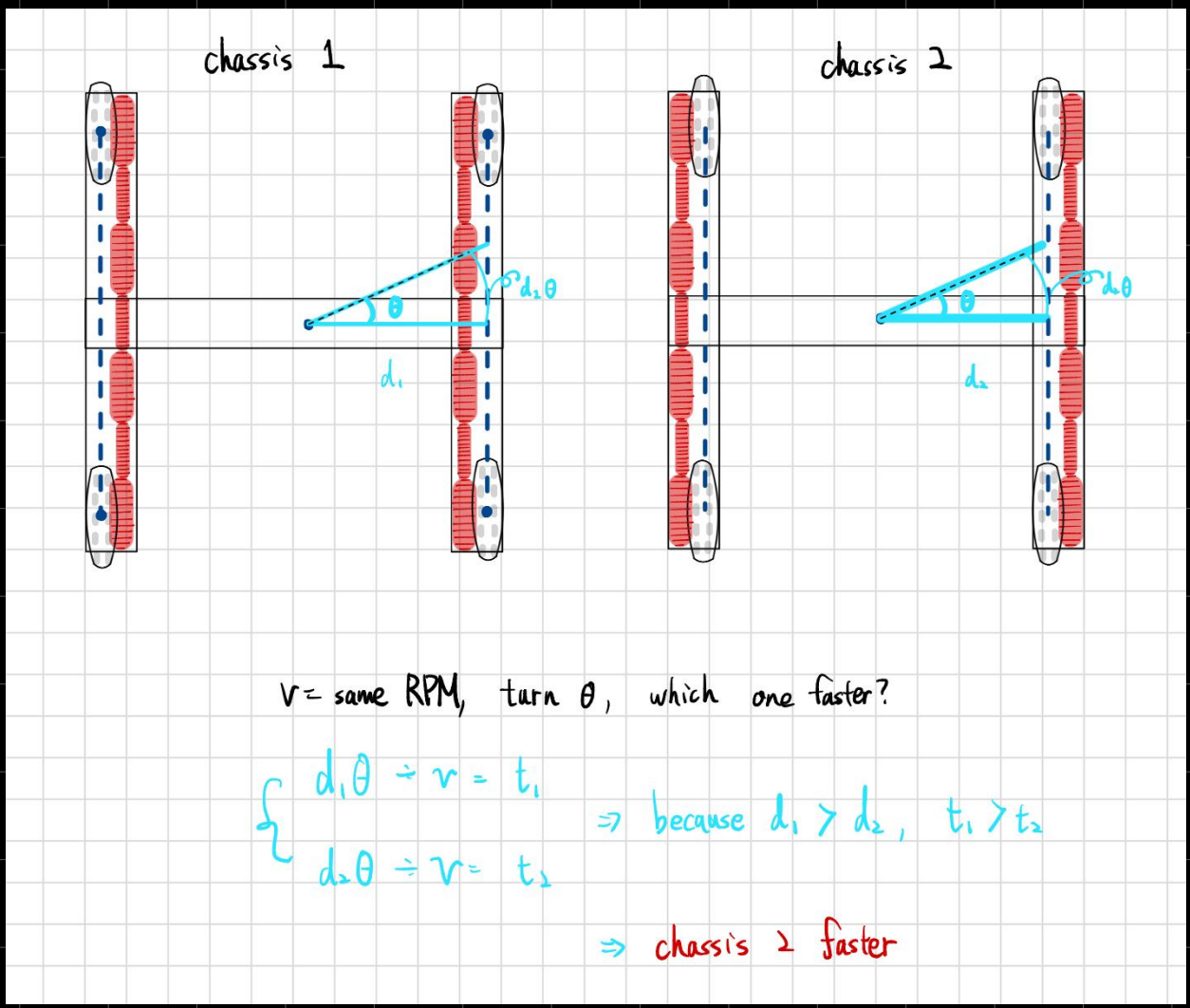


From our off-season training
Gears on the inside

Gears Placement

Gears Placement:

Deriving and comparing turning speed and efficiency of the two chassis.

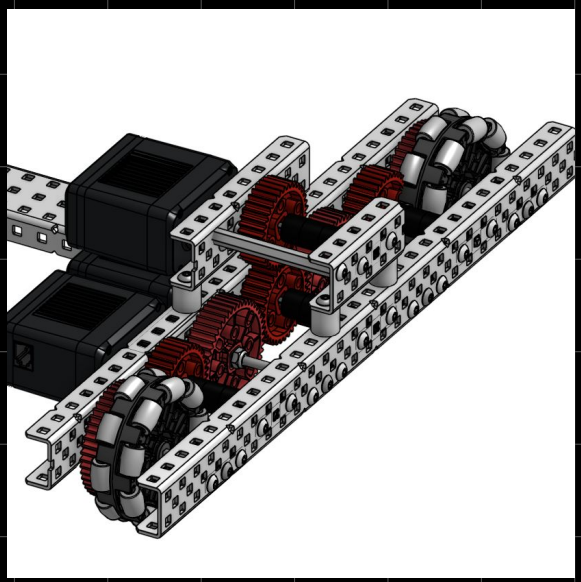


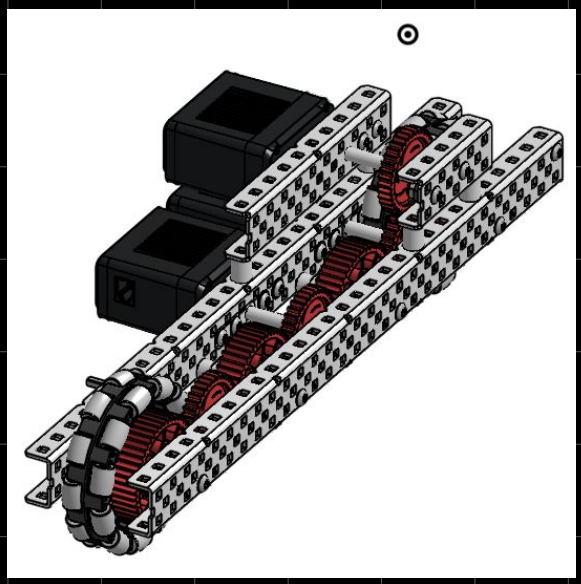
This shows that the ratio of the time the chassis takes to make a same angle turn is directly proportional to the ratio of the width between the center and the wheels. c

Gears Placement

Gears Placement:

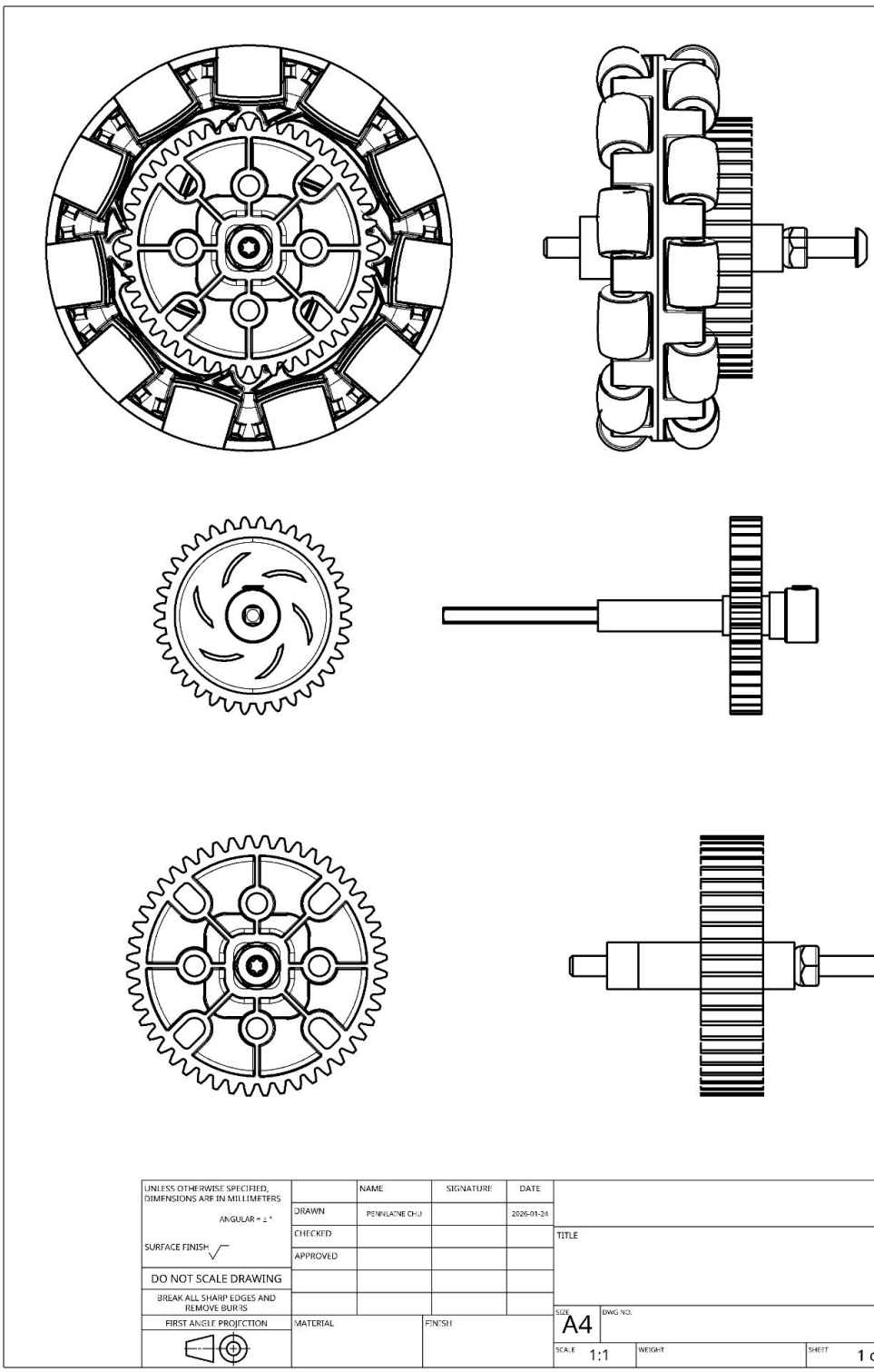
From the calculation, we decided to build a chassis with the wheels on the inner side and the gears on the outer side.





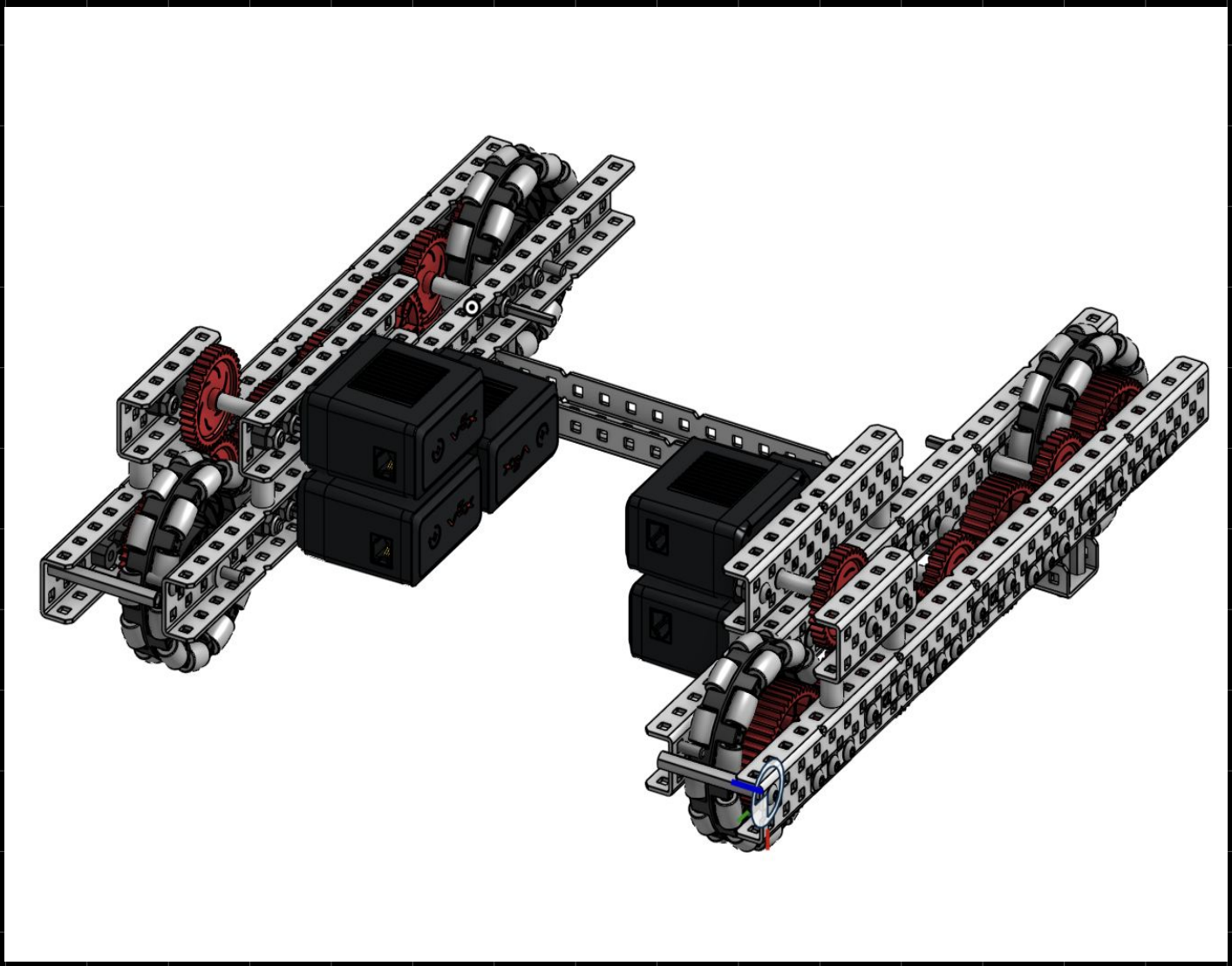
Gears Spacing

We are setting up our gears spacing.



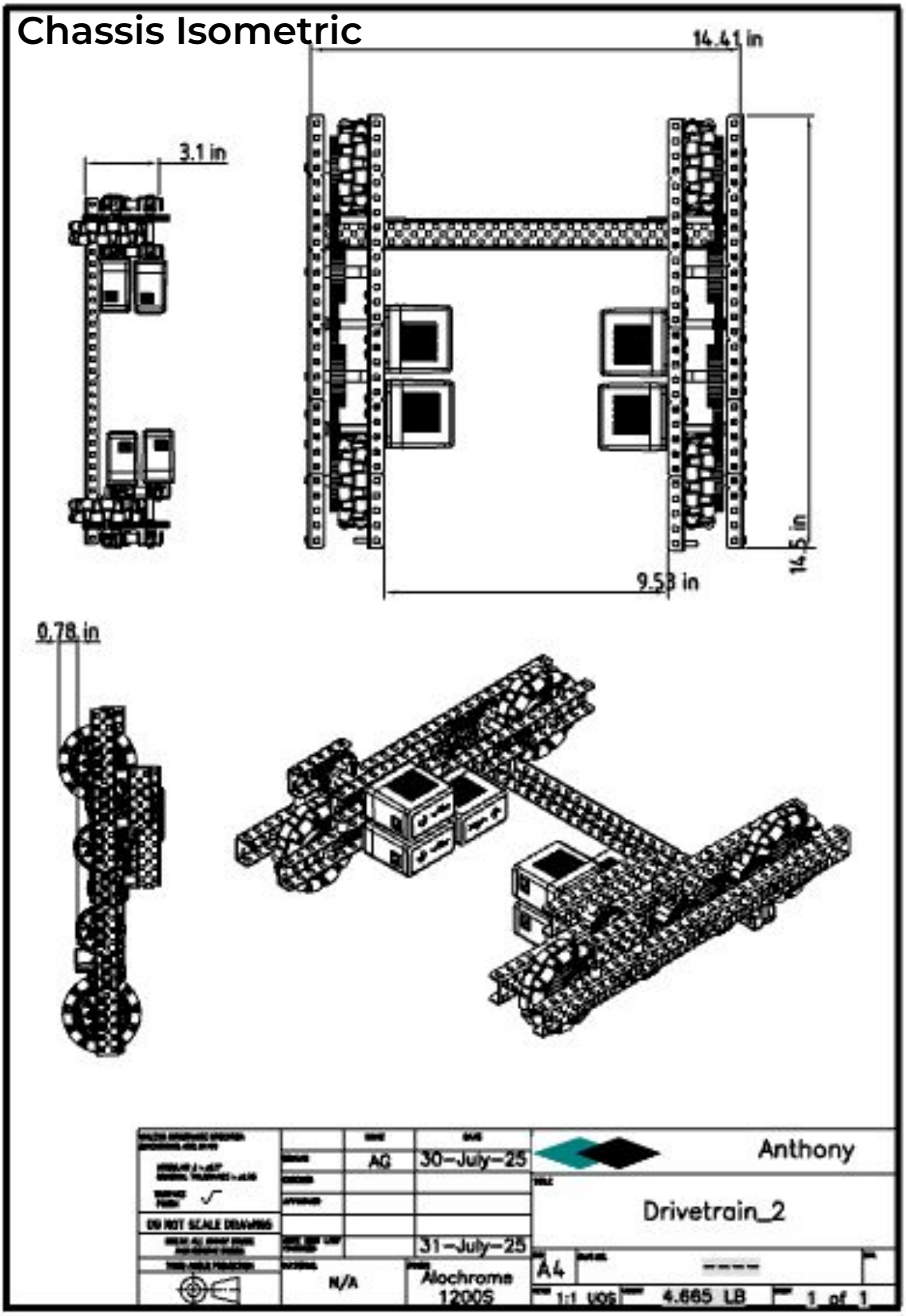
Chassis CAD

We finished the CAD of our chassis with new added details. And we are starting to build the super structure after having brainstormed it.

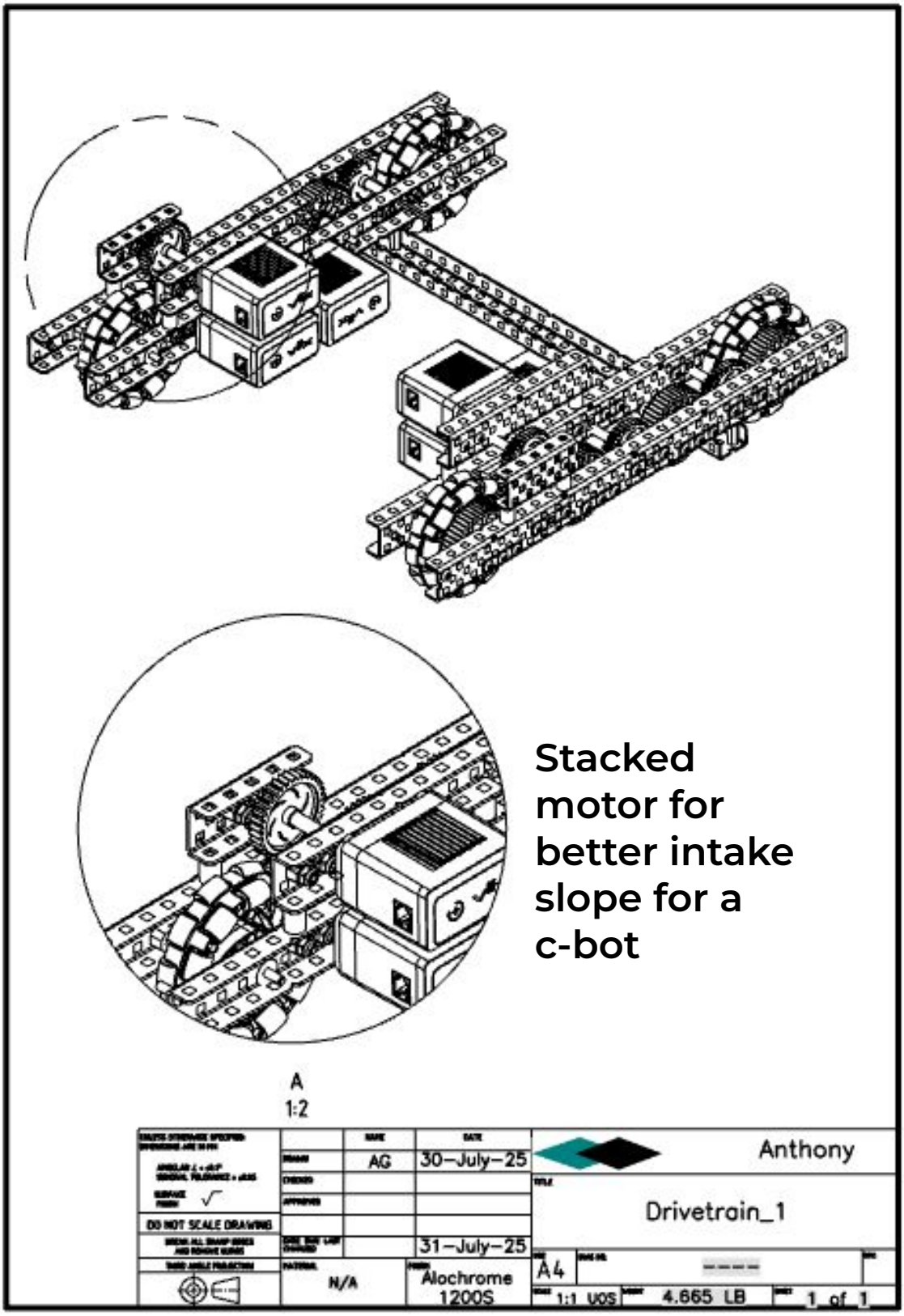


Chassis

Chassis Isometric



Chassis

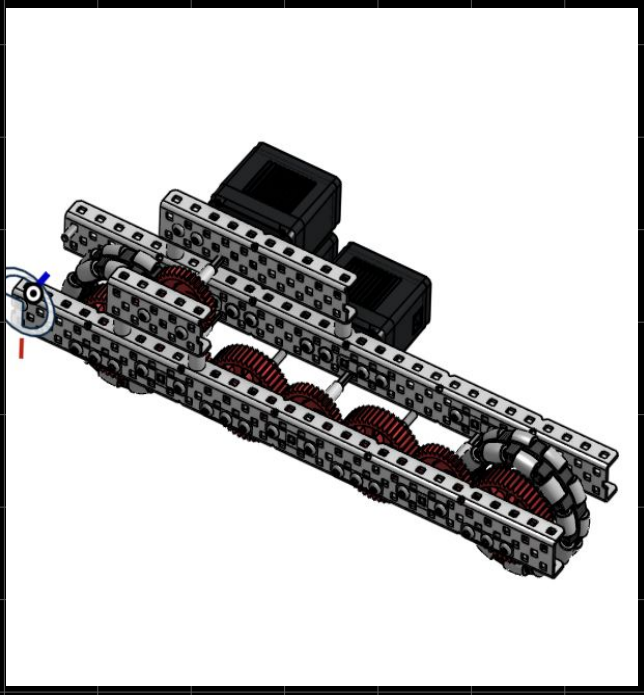


Stacked motor for better intake slope for a c-bot

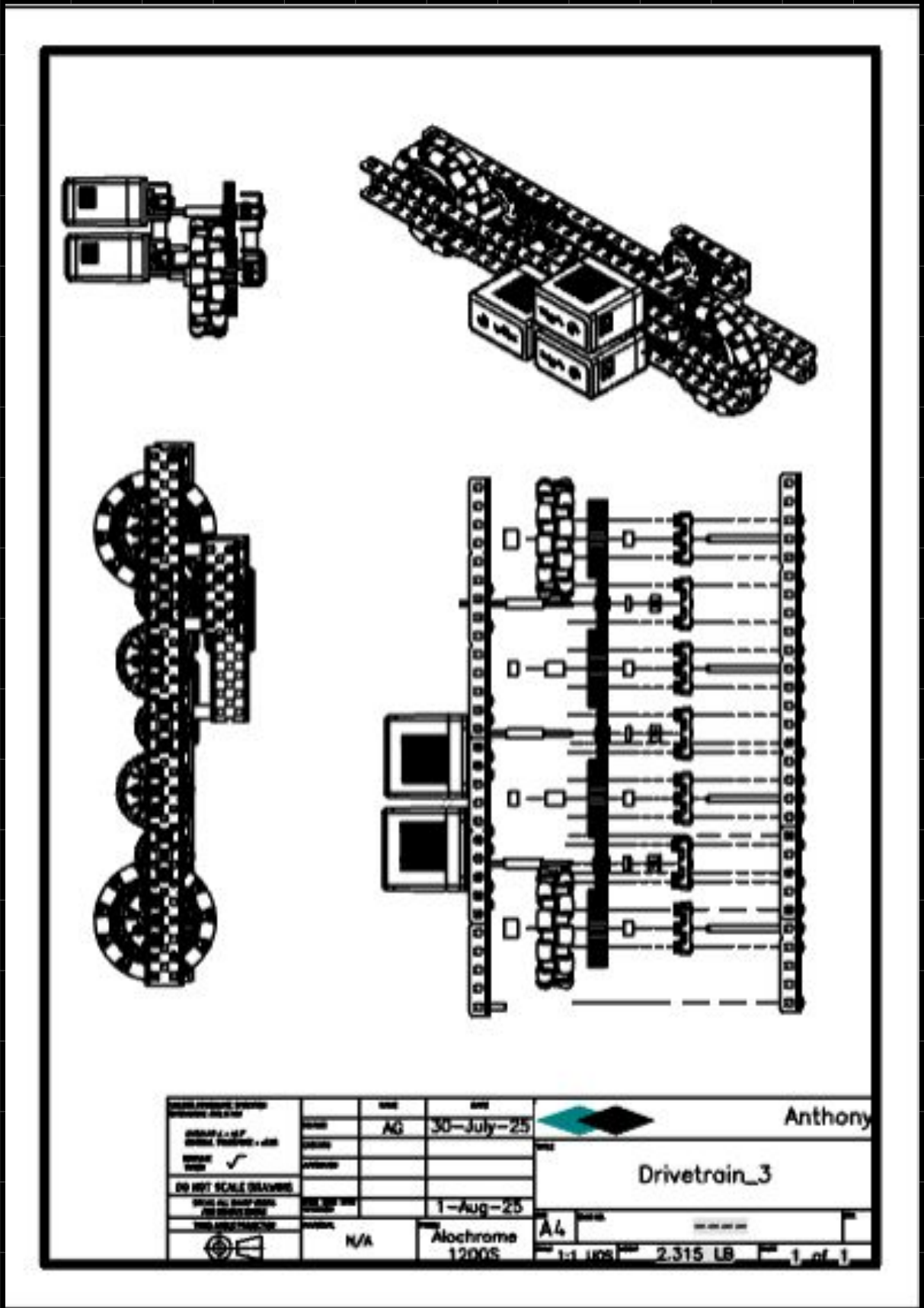
Chassis

After finishing CADing the chassis and ensured the spacing were correct. Our team started off building the drivetrain physically.

However, we decided to leave longer channels since we might need them later.



Chassis


AG
30-July-25
Anthony
Drivetrain_3
1-Aug-25
N/A
Alochrome 1200S
A4
2.315 LB
1 of 1

Personalize

Personalizing Our Robot

We learned from other teams that they dye their gears so that they look nicer. They also anodize their structure parts. So we thought that we would try this this year. We decided we want our robot **black and gold**, since it's our team color.

Therefore we sent 20 2*1*35 and 10 2*1*35 **aluminum channels**, 10 1*1*35 aluminum angles to a factory for anodizing. It costed us $3000NTD in total.

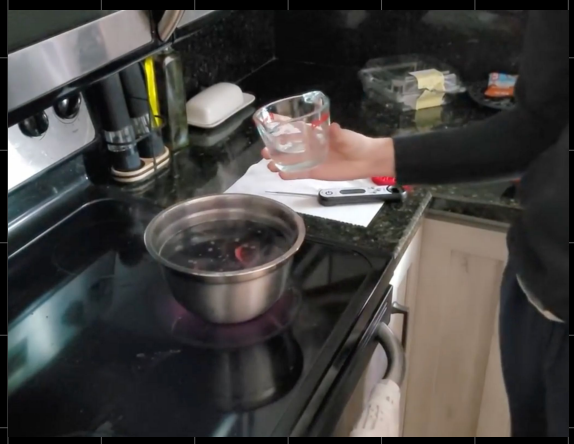
Next up we started to dye gears. We saw this video on Youtube from team 60470S teaching how to dye vex plastic parts. We bought the same type of dye – "Rit Dye" that was instructed. And we decided to make all the plastic parts black.

Dyeing VEX Parts Tutorial

Personalize

The video instructed to prepare:

1. Acetone
2. Rit Dyemore synthetic fiber dye
3. Stainless steel bowl
4. Thermometer
5. Electric stove



Then it instructs us how to dye:

1. Water in the bowl until it covers all the VEX parts
2. Dye + Acetone 1:1 in the bowl
   a. looks like they added about ⅔ of one bottle Rit Dye.
3. Heat it to 190 degrees Farenheits and keep it at that temperature
4. Put in the parts
5. Can add more dye after a few minutes if it doesn't show color

Personalize

Aside from looking at the Youtube tutorial, we also asked our Chemistry teacher. He suggested that the acetone is only used to clear the dust or wax on the plastic parts, so he actually advised us to just wipe the parts slightly with acetone and not boil them with it. He mentioned that boiling it for too much time might corrode the plastic and cause it to change shape. Our result was successful for the first try!

Personalize

This is our process:

1. Wipe the plastic parts with acetone with a mop rag (don't contact skin or it might damage it!!)
2. pour a bottle of Rit Dye (BLACK or NAVY) into the bowl and start boiling it.
3. Wait until it boils and softly add the plastic parts.
4. boil them until all the colors are shown perfectly, takes about 10 minutes.
5. Take a look if the dye is enough, dye can be added

We are very satisfied by the result so we wanted to also dye more, including flex wheels. That was a disaster. It became green. Lesson taught: do not dye flex wheels.

Apparently we did not do enough research...

Personalize

Anodized and Dyed Parts finished!
By the time we have gotten all our new parts for our now planning to build robot, we have to develop a solid idea of our robot before we use these parts.

We should first listed out the most bots now, and then listed out their Our decisions and planning is liste the meeting logs. Now we will start to build our new robot.

Meeting

2025/08/11 — 80 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony	- compare and decide robot type

Brainstorming:

We compared the most common robot types this season — basket bots, ramp bots, snail bots, and parallel linkage bots — and discussed their pros and cons.

(See our comparison on the next pages)

We decided that the C-like Design would best fit our goal, as it allows continuous intake and outtake, can stay flat enough to go under the long goal, and can store more blocks within the same height.

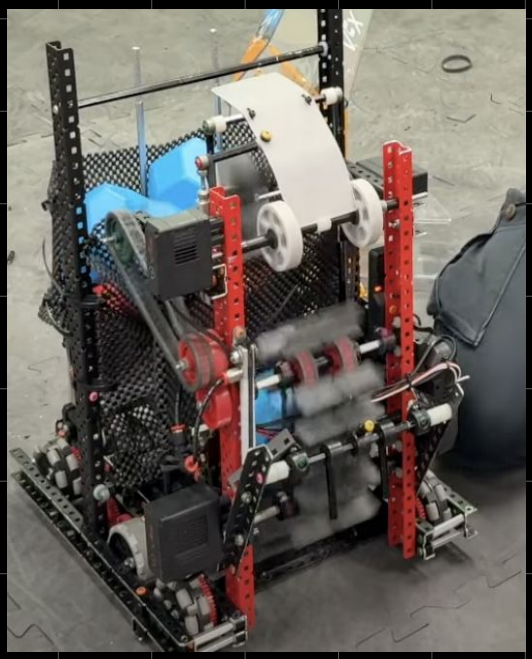
Robot Type

These are most of the types of robots now revealed for this season:

1. Basket Bots

PROs	CONs
- larger storage space	- hard to adjust its weight management

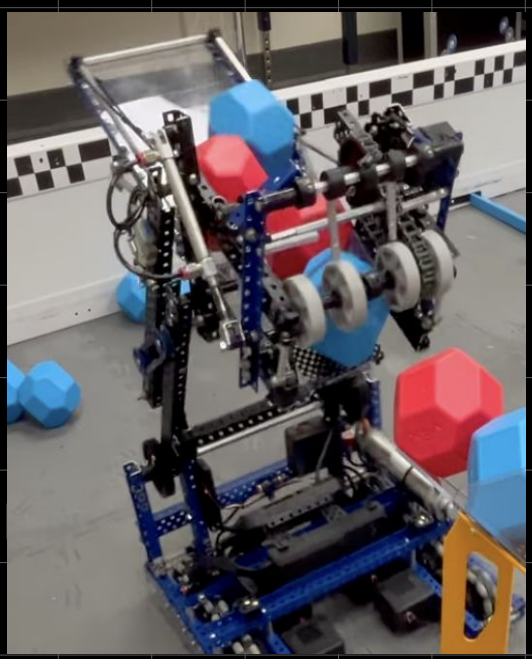
- center of gravity would change after every intaked block



2. Parallel Linkage Bots

PROs	CONs
- short - solid storage space	- slow - vulnerable to offenses

- it has to wait until the platform lifts in front of the long goal where center of gravity becomes and vulnerable to offenses.

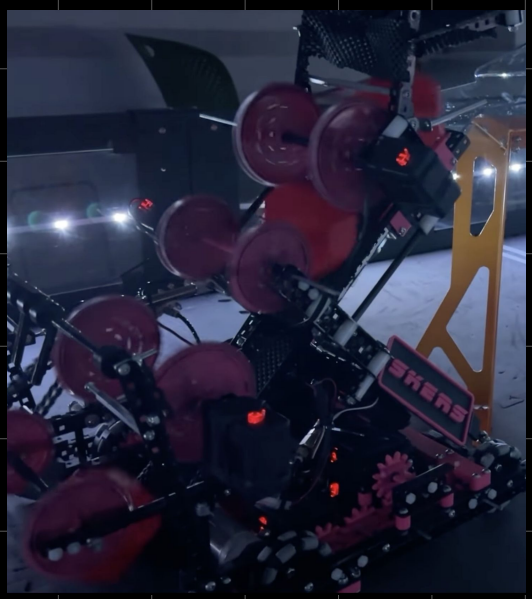


Robot Type

3. Ramp Bots

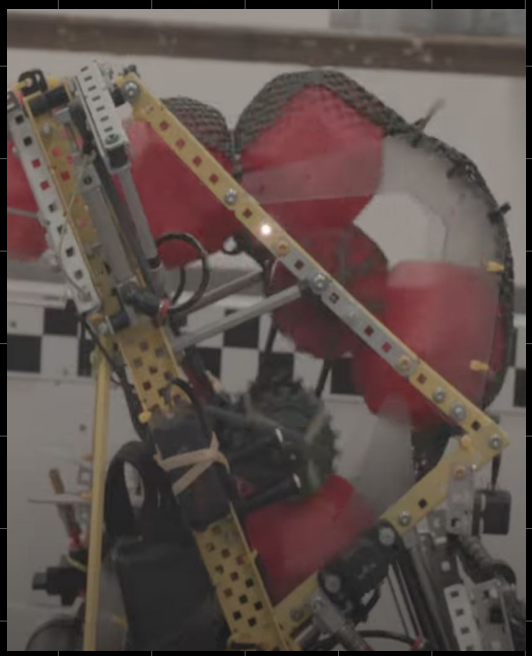
PROs	CONs
- allow two level scoring - fast - light	- small storage space - if no air can't score long goal

- single ramp makes its storage space relatively small.
- pistons are carrying the bot's main mech



4. S Bots

PROs	CONs
- allow two level scoring - large storage space	- heavy - relatively few bots like this were revealed



Robot Type

5. C Bots

PROs	CONs
- allow two level scoring - lighter than S bots - short	- lighter than S bots - defensive

Robot Type

We will evaluate these kinds of robots based on 6 aspects:

1. **Capacity**: How many blocks it can store
2. **Weight**: How heavy the bot is
3. **Agibility**: The maneuverability of the bot
4. **Speed**: How fast the robot is cable of
5. **Ease of Build**: How confident we are finishing succesfully
6. **Defense**: The bot's defense ability and performance

Criteria	Basket Bot	Parallel Linkage Bots	Ramp Bots	S Bots	C Bots
Capacity	10	3	3	10	6
Weight	3	6	10	5	9
Agibility	3	10	5	5	10
Speed	3	5	10	6	9
Ease of Build	9	9	10	6	6
Defense	5	3	6	9	10
TOTAL	33	36	44	41	50

Robot Type

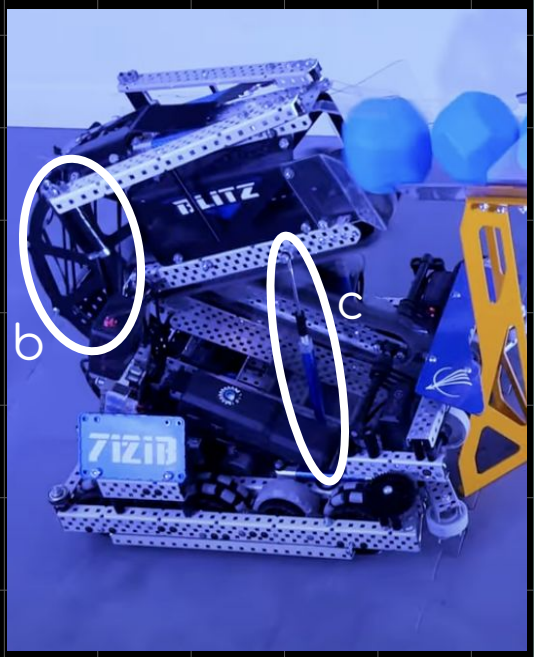
After initial researching and evaluation, we preferred C bots for several reasons:

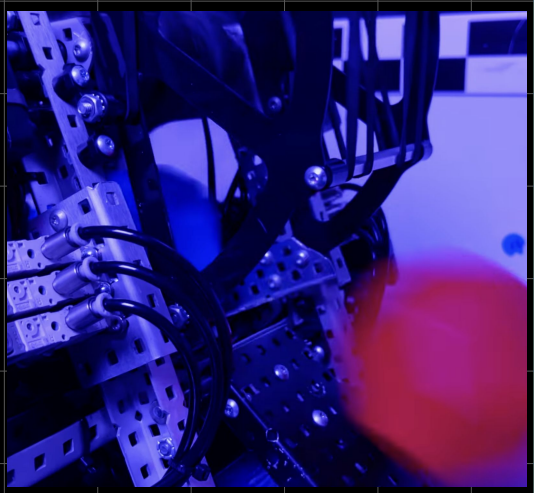
1. It rates the highest of the comprehensive scoring.
2. it is possible for it to accomplish our goals
3. relatively few bots are revealed that means we can design ourself and possibly get an award on our designs
4. Now we do’nt see any fatal defects for this kind of bots

Research

After this decision, we started to research on C bots.

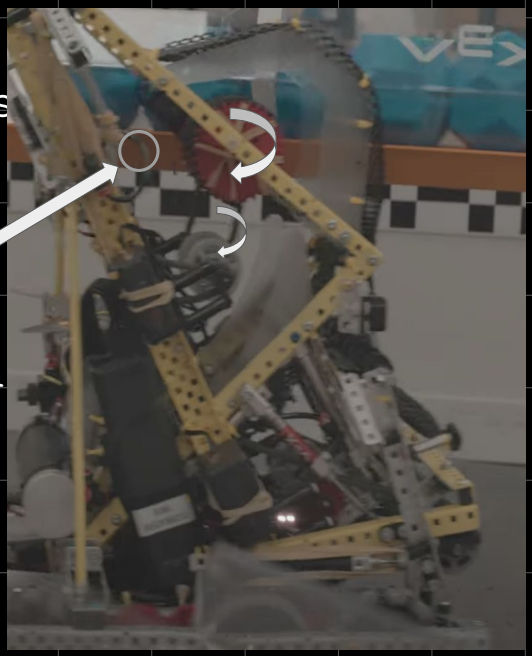
1. 7121B BLITZ
   a. Thier robot has two levels of motion.
      First level uses a belt, and the upper one uses flex wheels.
   b. They use plastic boards to connect the curve shape needed for block connection.
   c. They use pistons to adust which height of goals to score in.
   d. They have a well done color sort mech
      in the back of the car →

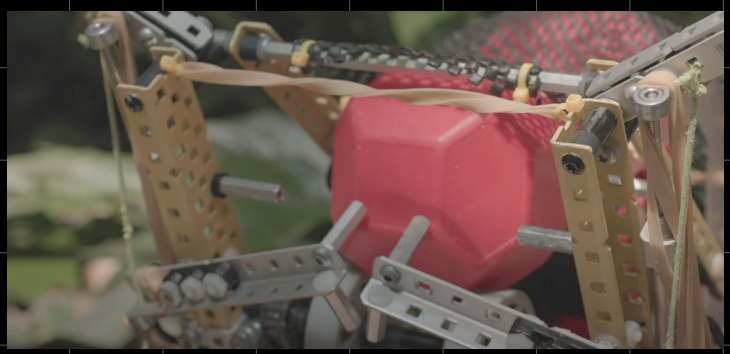


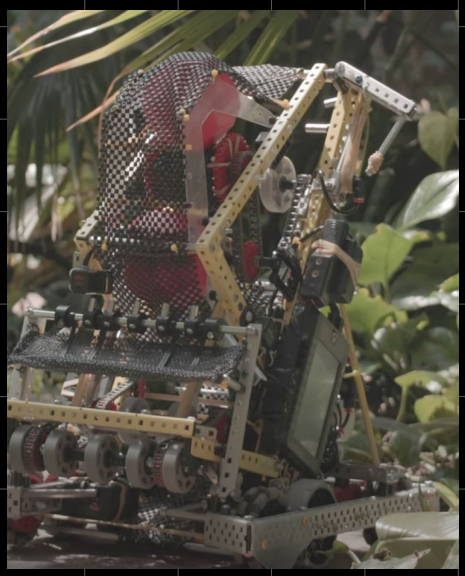


Research

2. 16610A Snacky Cakes
   a. Uses the rolling direction of flexwheels and sprockets to decide which level to outake at
   b. Flex wheel here outtakes to the center upper goal.
   c. A piston mech connected with a descore mech that prevents the blocks from falling out when robot is moving.
   d. Their descore mech looks like one we analyzed before in the first event of push back
   e. This bot has relatively less storage since it wastes some anterior to posterior space.
   f. Also, it cannot go under the long goal

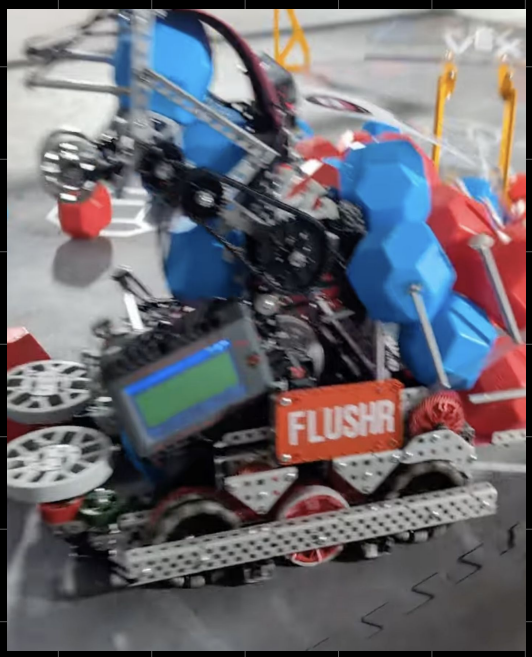


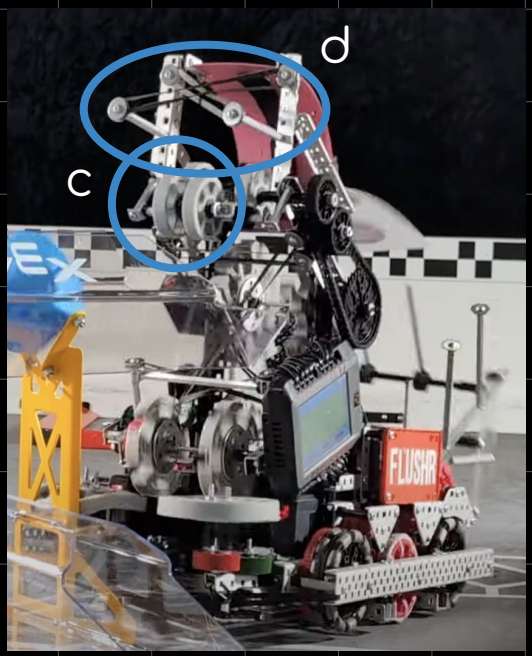




Research

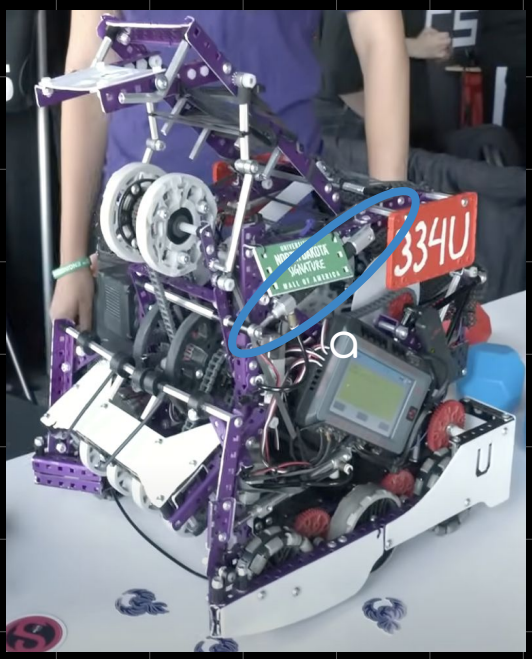
3. Robotics is EZ — FLUSHER
   a. It has a small truck that can store about 10 blocks
   b. Although they look similar to Snacky Cakes, they don't have a real center goal scoring level. They only use the long goal level to drop blocks down.
   c. However, they use that flex wheel to descore middle goals.
   d. They also have descore mech for long goal, which is also combined with the outake mech.

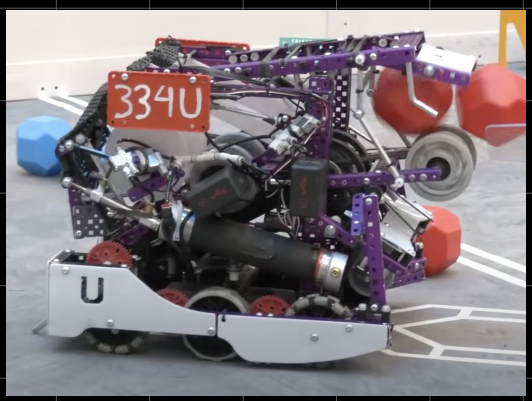


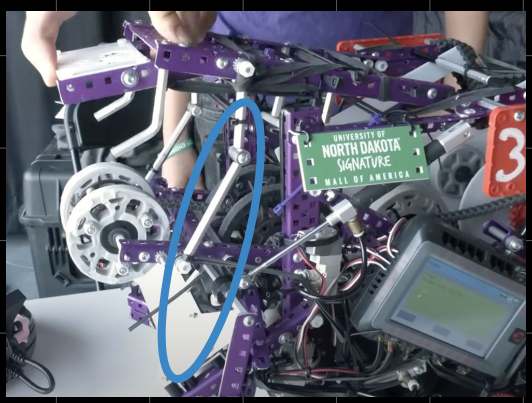


Research

4. 334U Supernova Redeem
   a. Uses pistons for the shooter part to adjust which level to outake at.
   b. Can go under the long goal.
   c. When the piston is retracted, the bot has a sequence of sprockets and flex wheels that helps with its scoring to center goal.
   d. When the piston is extended, the moved flex wheel still acts as the last wheel before outtakes.
   e. Well built descore mech that uses a passive design without electronic motion. →







Research

Research Conclusion

Combined with our goals, we now have our own idea of how to build our bot.

1. The upper level part should use pistons, like 334U.
2. To aim for more storage, we would extend our bot's anterior to posterior storage space, so that it is almost double as only the first level.
3. Descorer design might not be considered when prototyping since a lot of teams combine descorers with their structure design, like using their flex wheels at the outake part to descore.

Our Goal

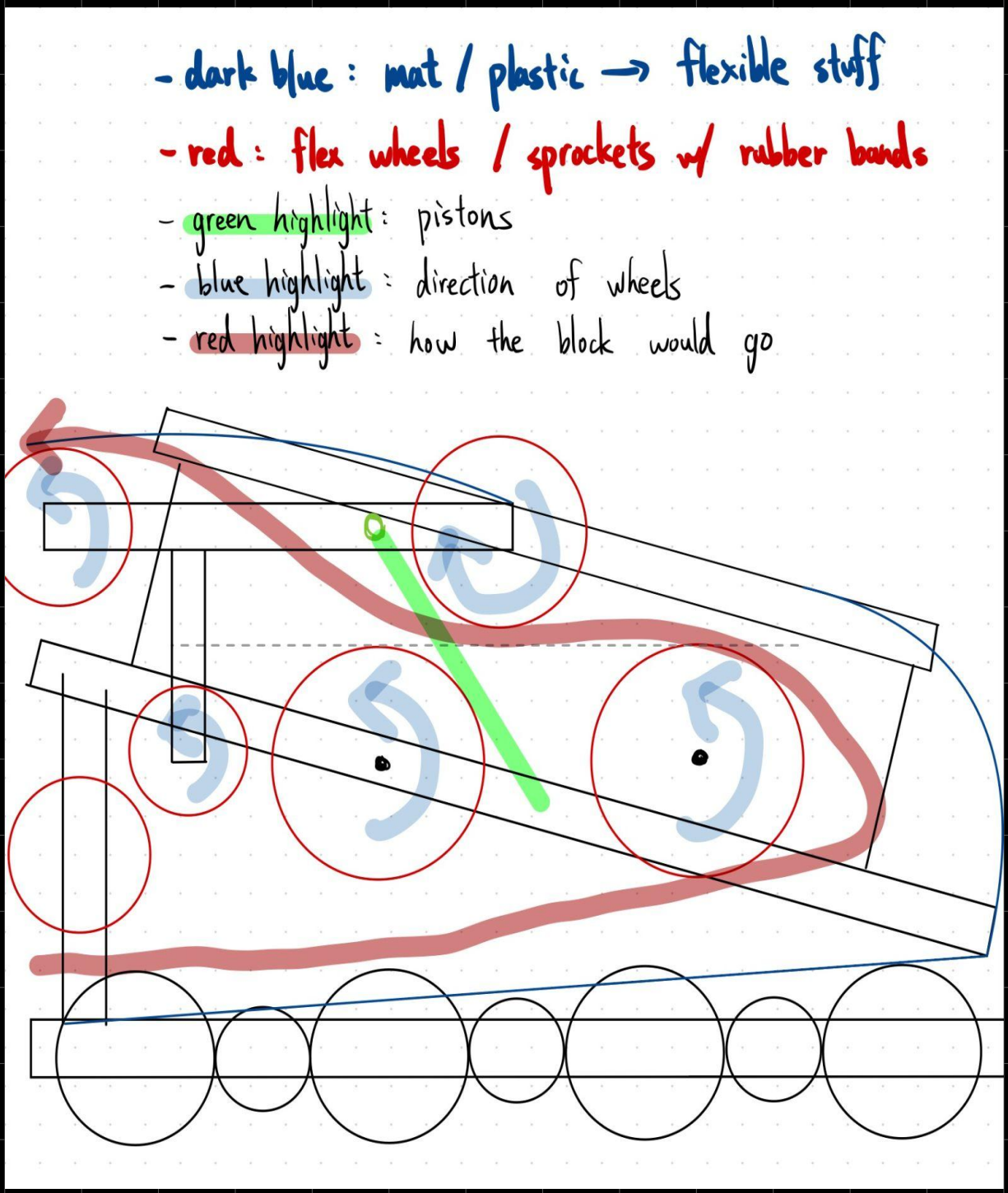
1. Prioritize **Agility**
2. **Storage** maximize

Main Ideas

1. Low enough to go under the long goal
2. Fast chassis
3. As long as possible to store more blocks
4. Use rubber band sprockets for more flexibility in space

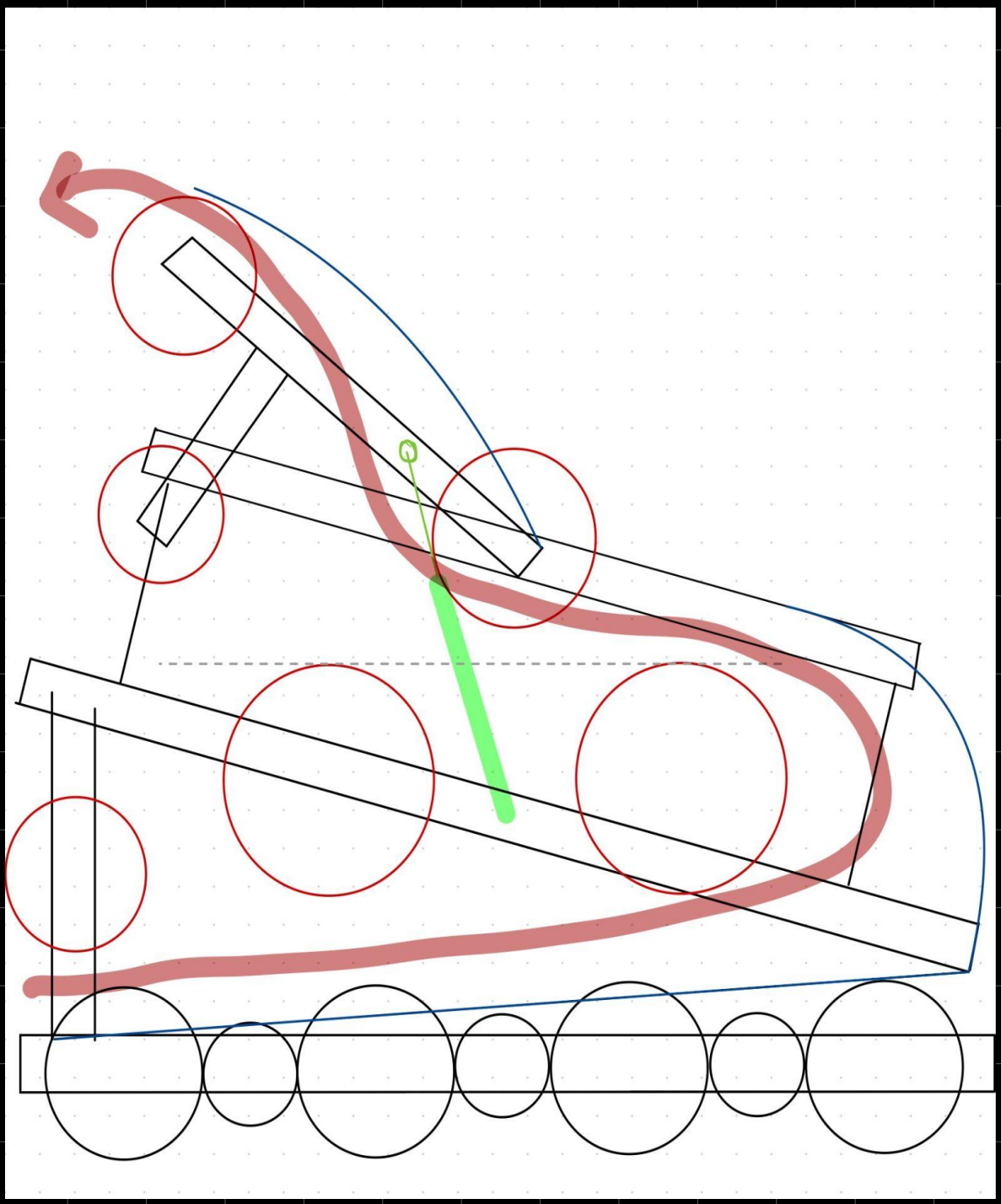
Prototype

Robot Sketch:



Prototype

This is when the piston is extended, for long goal scoring



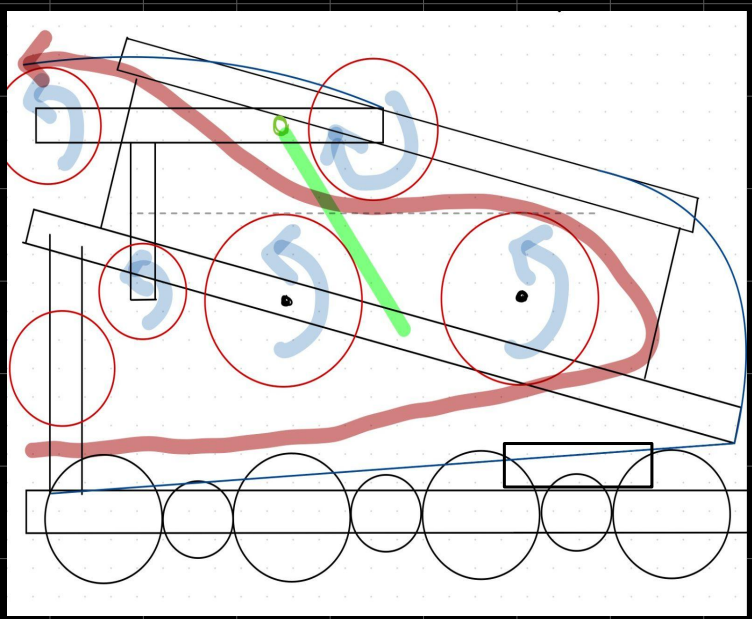
Prototype

Explanation:

If the whole bot is shared during two paths for outaking, we could save space and make our robot lower.

The first level is slightly ramped so that it could connect to the second level, also because we have motors back there, and the mat (blue) is on it.

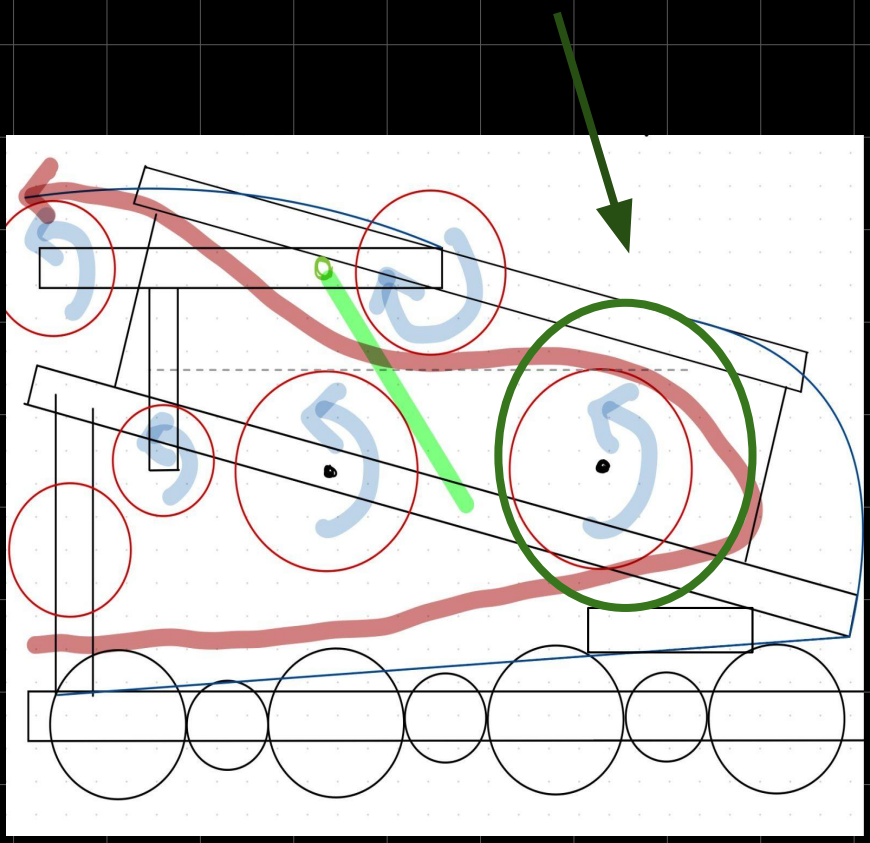
We will use mats as the ground for first level, as its shape is determined by the motors and we could easily restructure its shape using left-to-right direction standoffs.



Prototype

Explanation:

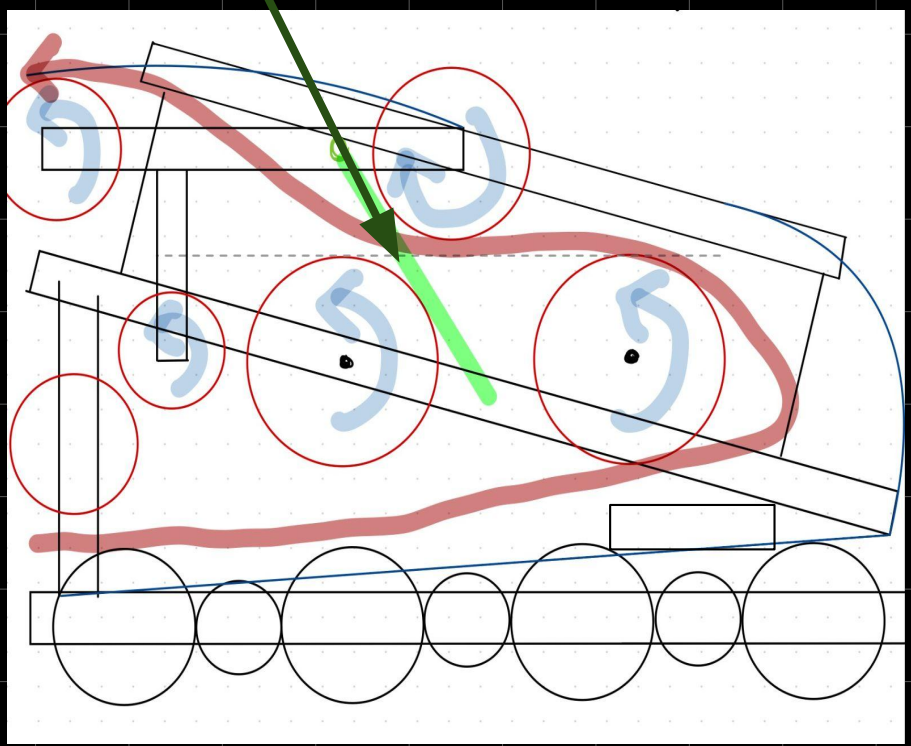
For the last sprocket at the first level, it connects to the second level with a plastic board/mat (needs testing, but same idea as 7121B BLITZ) to help the blocks go up. That is why we have to use large diameter sprockets with rubber bands. We have to make sure the rubber bands can touch the blocks for all 3 directions. If the diameter is small, the balls might easily get stuck at the sprocket's shaft. A larger sprocket guarantees a larger amount of compression.



Prototype

Explanation:

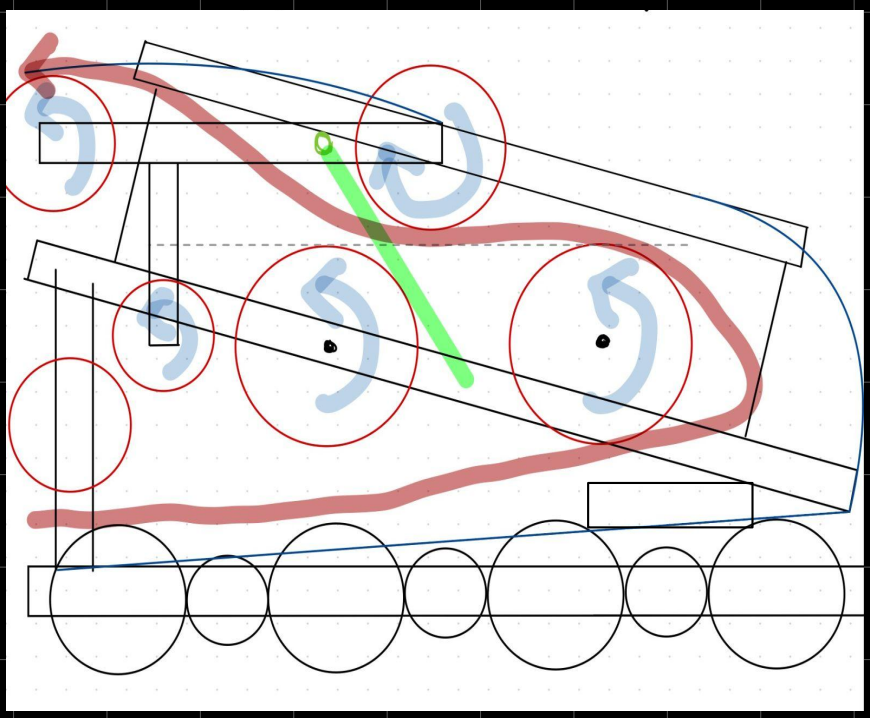
The ball then goes to the second level utilizing the first level's roof as the second level's ground. The second level's roof would be plastic boards or mats. We now think that the second level should be flex wheels, since if the ground is already very flexible when the ball is at this point: it's roof should not be so compressible or else it would outake with a weak force. After the ball is squeezed out of the two wheels at the point, it enters the shooter part which is controlled by a piston.



Prototype

Explanation:

For the shooter part, it rotates on an axis and it contains three flex wheels for the outake torque to be bigger. The flex wheel which is lower and extending from the main shooter structure acts as a blockage for the blocks so that it would drop down back to the first level. It rotates at the same direction as the farther most outake flex wheel and the back(near to center) flex wheel is the opposite direction. Therefore, the inner flex wheel would need a gear to change its rotation.



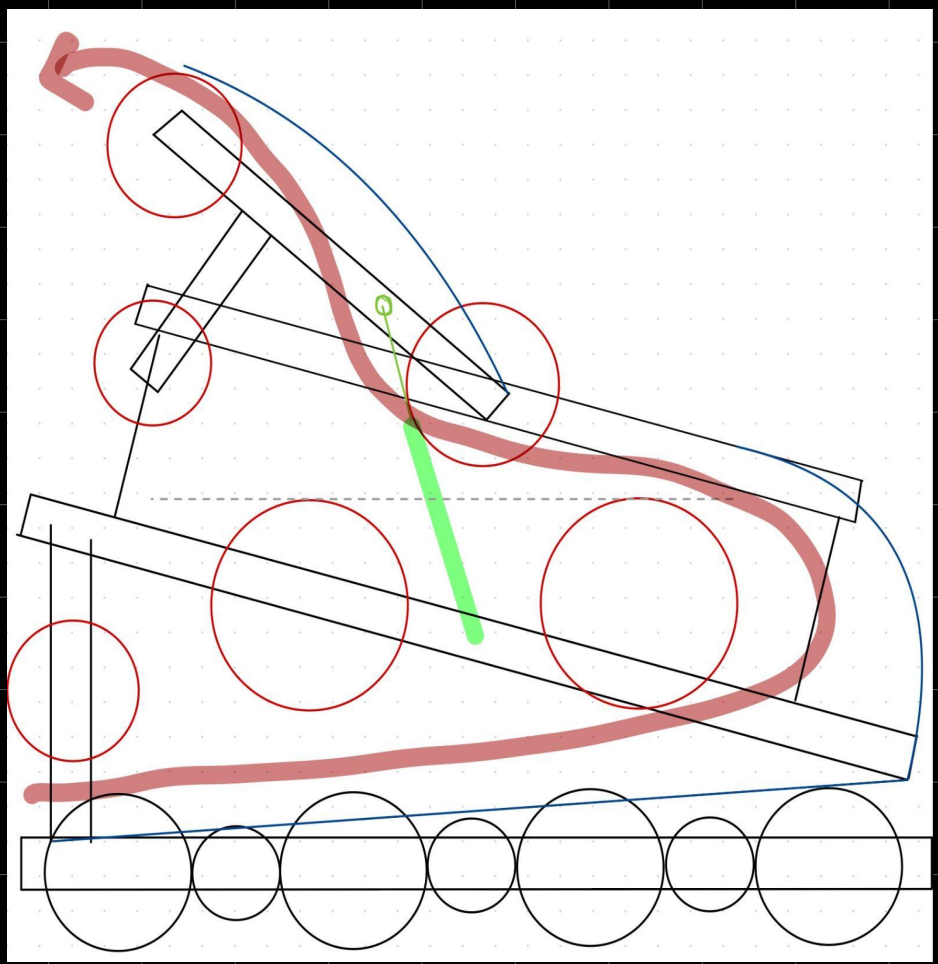
Prototype

Explanation:

When the piston is extended, the robot will score at the long goal.

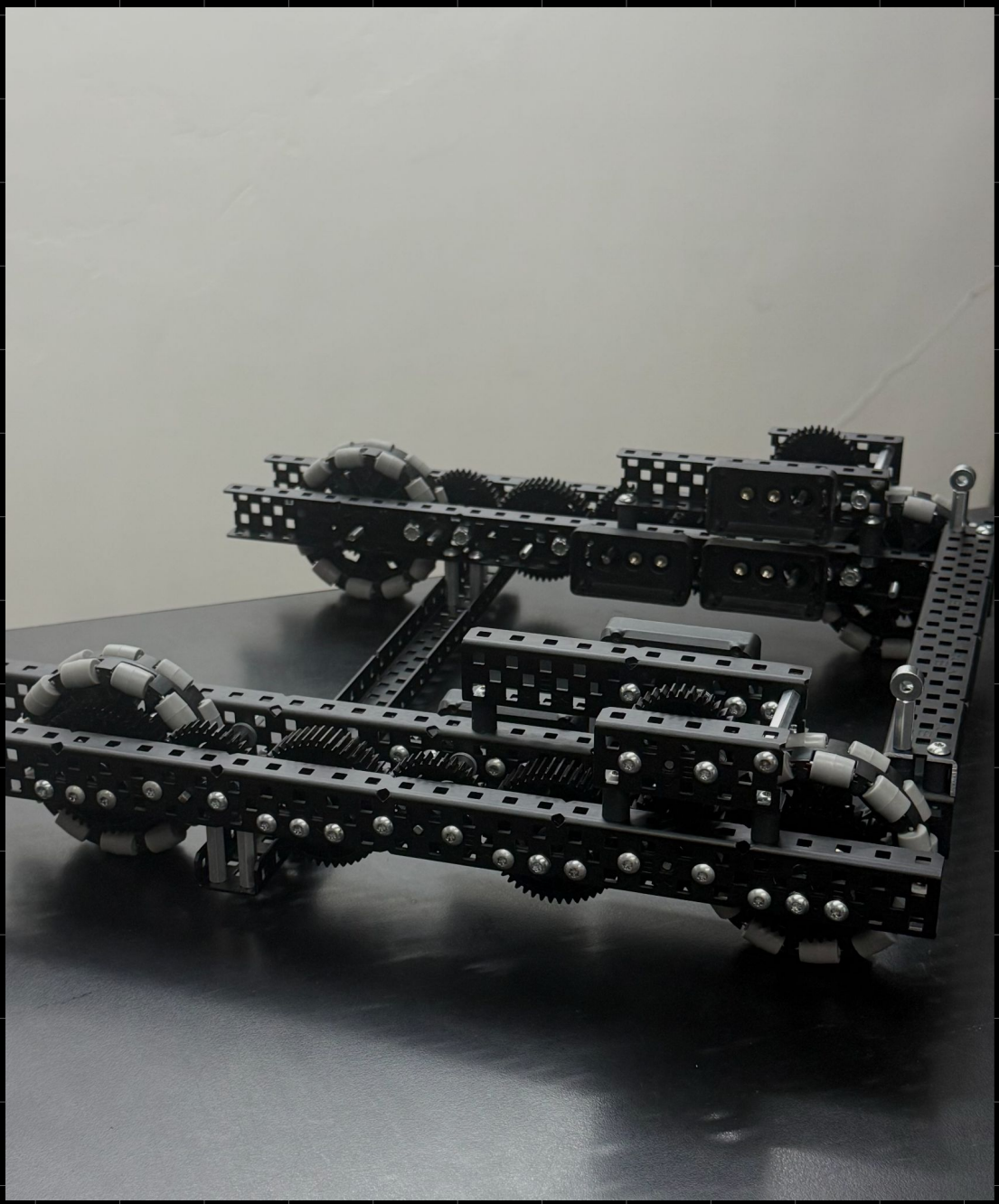
It would go the exact same route as when the piston is retracted. But now the lower flex wheel is more important as it guides the block up to the outake part.

And finally, the roof for the shooter part is the blue curve, which will be a plastic board, probably with anti-slip mats sticked to it.



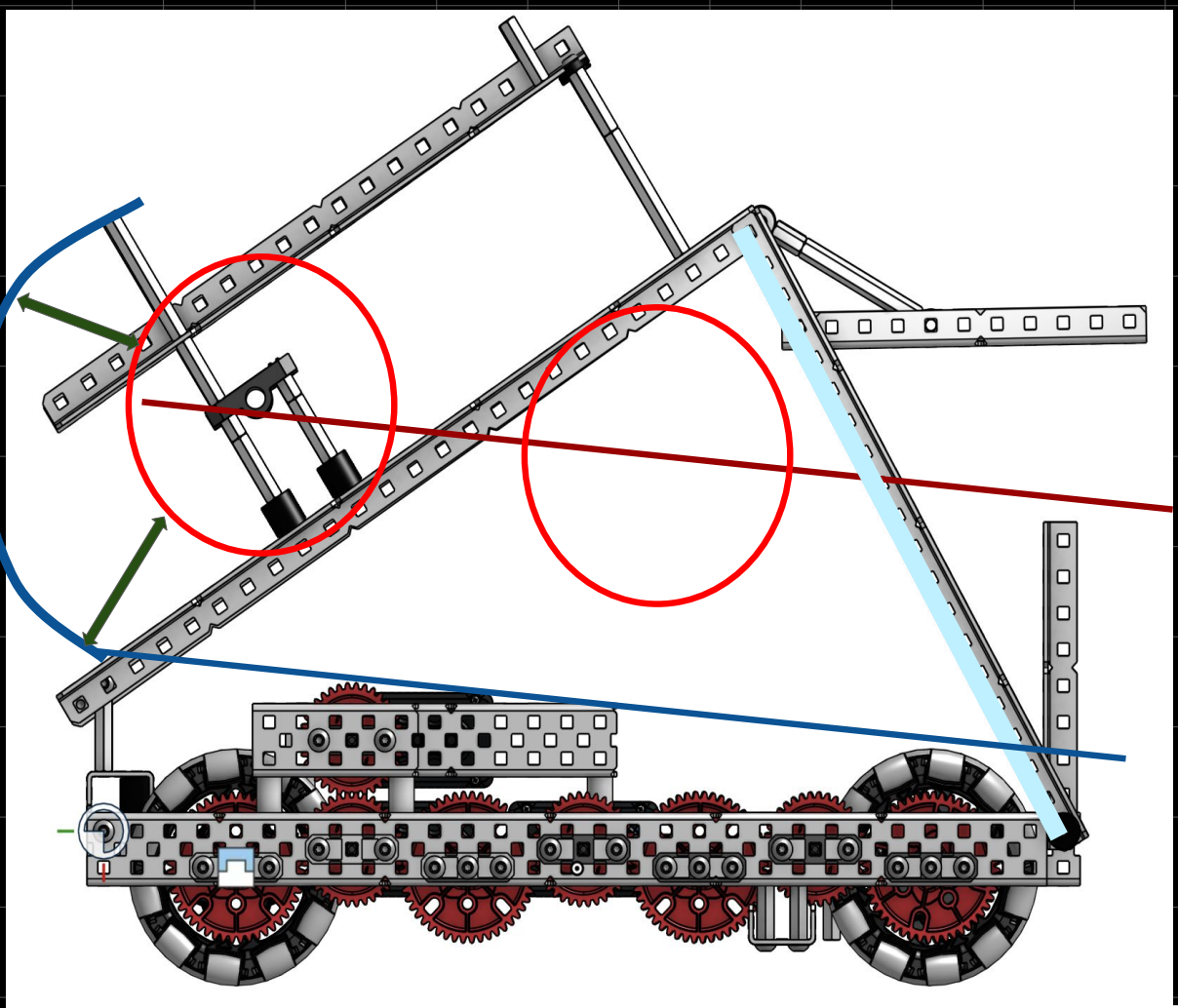
Building

As we have developed a solid plan of our robot, our chassis is also built with our new anoidized parts.



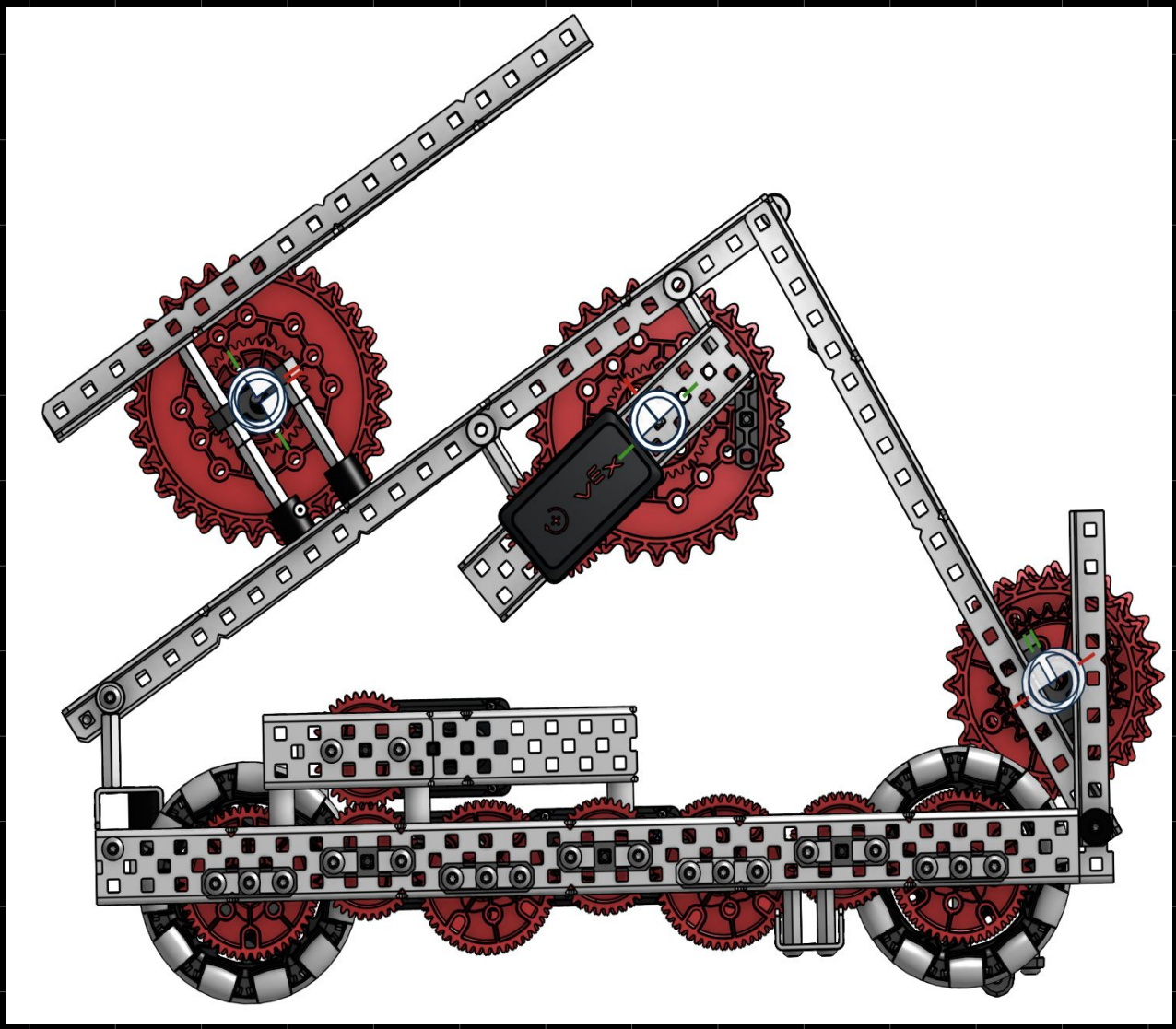
CAD

1. We found out that our first sprocket would extend over size limit if the front channel (light blue) were placed upright, so we tilted it
2. 2 intake sprockets are parallel to the ground (dark blue) laterally
3. back sprocket has equal space from its ground → plastic board(blue) → roof



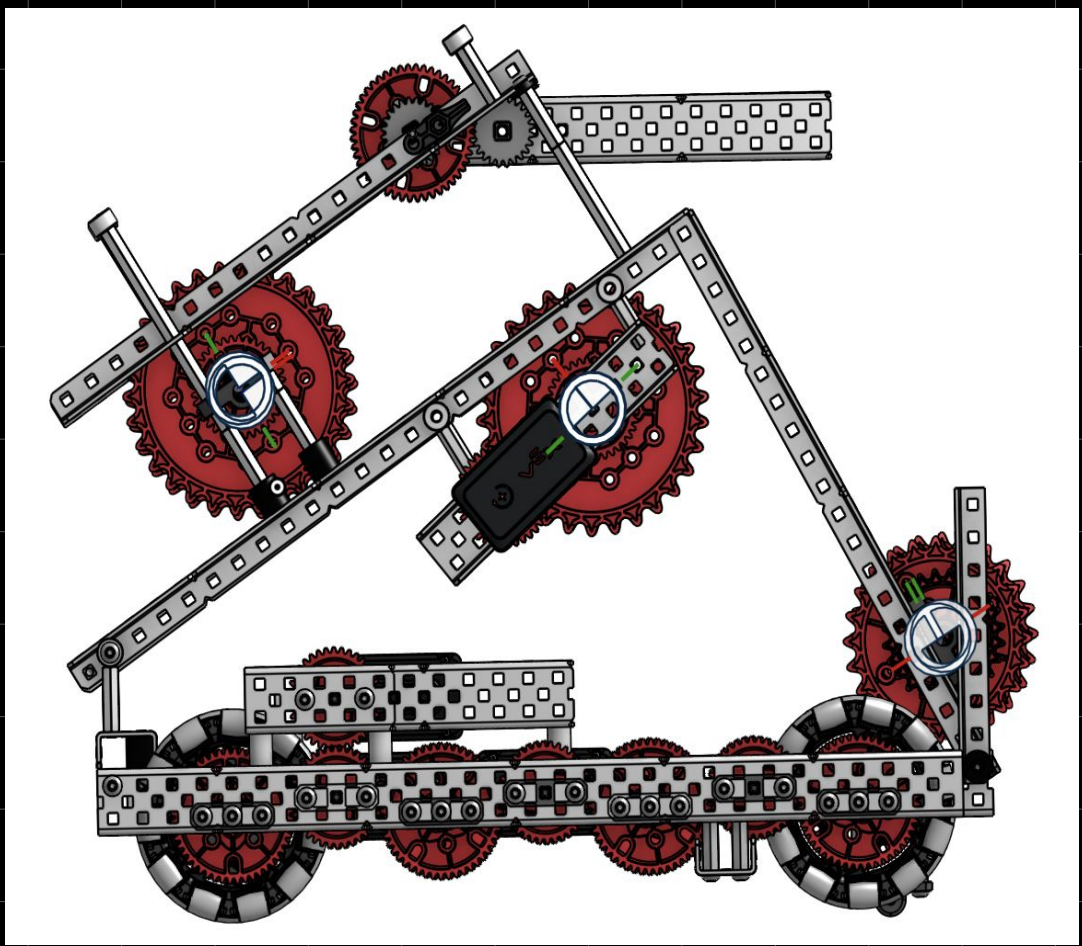
CAD

1. We added the first sprocket and the second sprocket.
2. We decided that we would put a 11W motor on the second sprocket so we had to use another c-channel to extend it since motors can't be on 1*1 angles
3. We decided that we leave the aligner and build it physically so that we can test the height



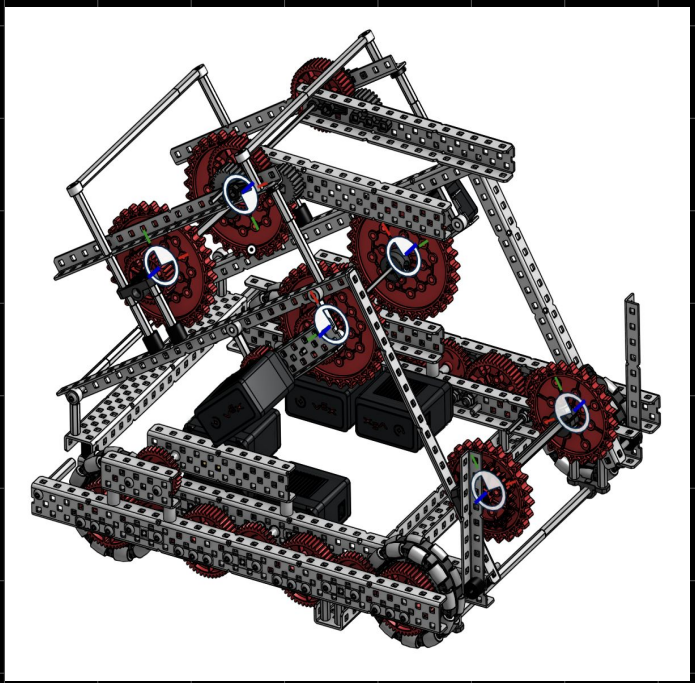
CAD

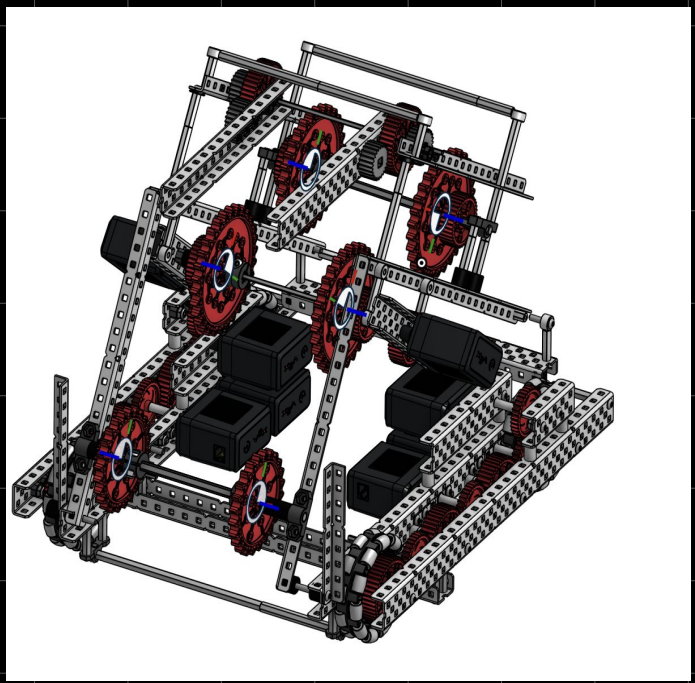
1. We started to design the shooter, which had to make sure the gears wouldn't touch the ball.
2. We changed the gear ratio to 1:2, and the motor would be connected to the rotating axis of the shooter, so that it wouldn't be moving while the piston is extended. But we haven't thought of where to put the motor, it would probably be like the lower one, on an extended c-channel



CAD

These are some more pictures in different angles of our CAD because it's kind of hard to understand it with one viewing angle.

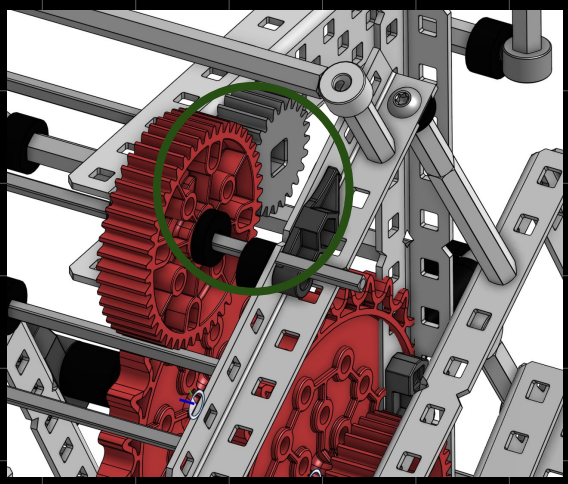


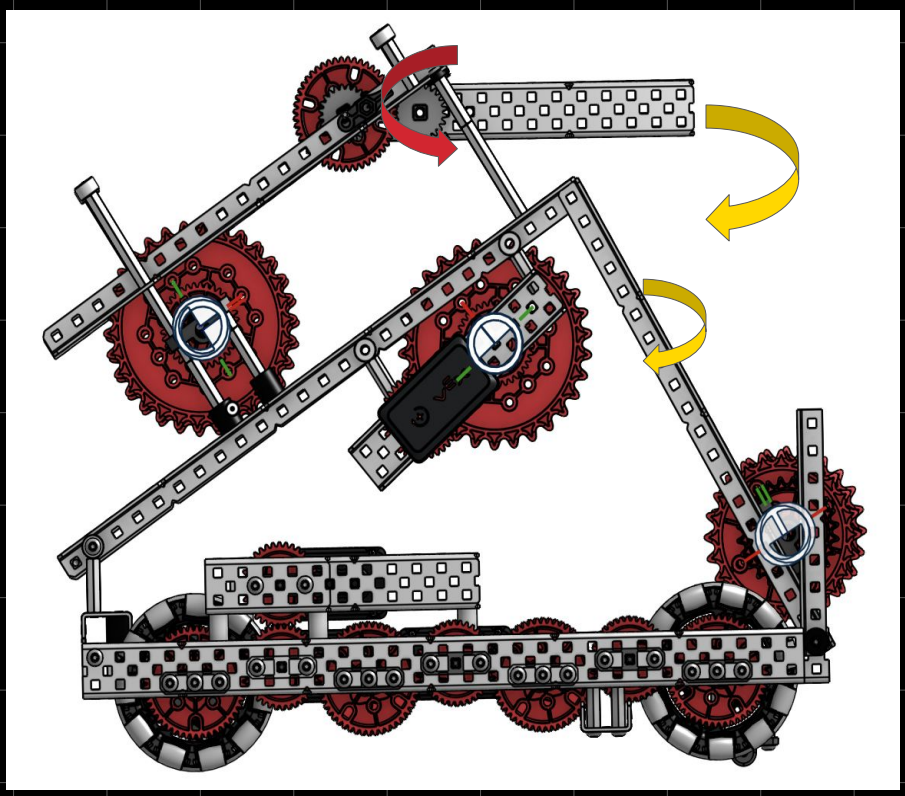


CAD

1. As previously mentioned, the red arrow flex wheel has to be rotating the opposite direction of the 2 yellow arrow flex wheels.

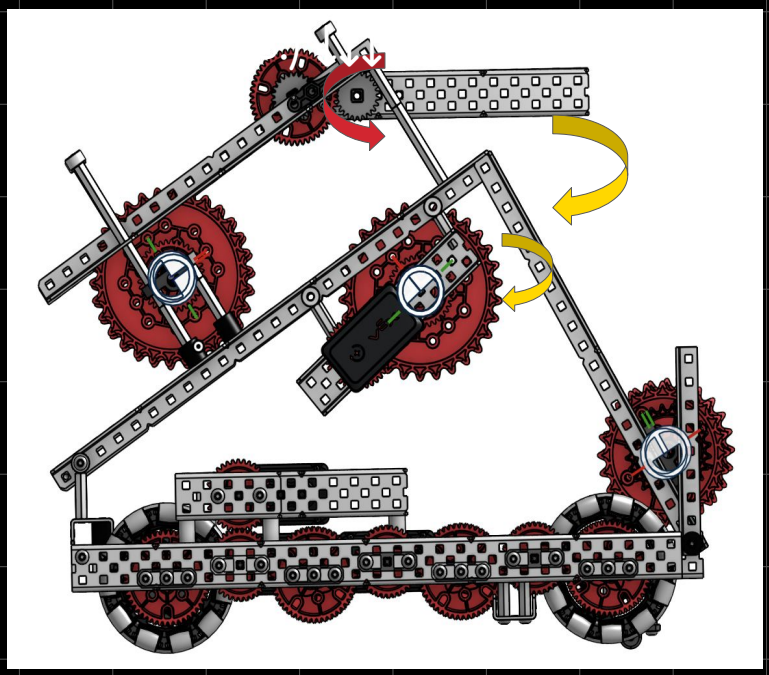
   So we choosed to use gears for the red arrow flex wheels, since it is adjacent and we decided to put the motor connected to the red 36T gear.

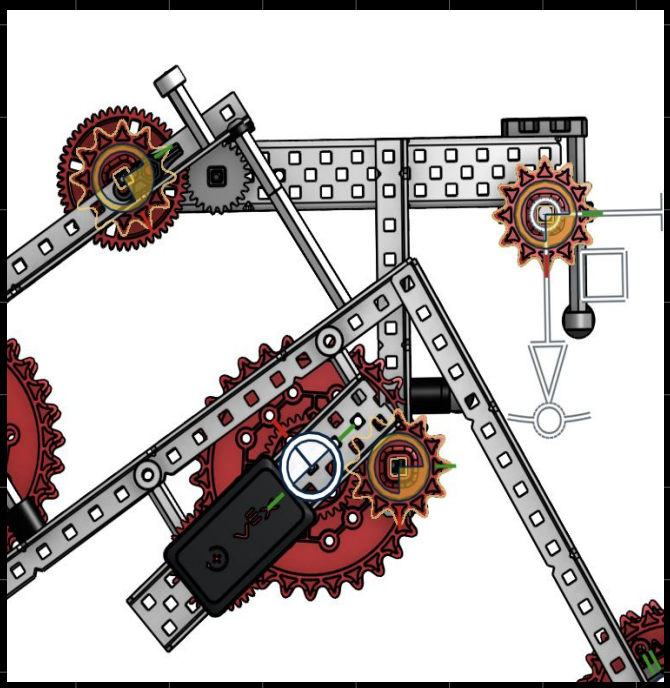




CAD

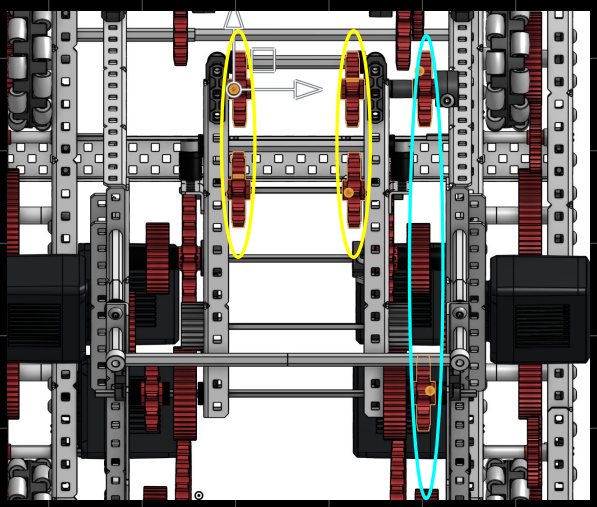
And next we have to chain the motor to the other 2 yellow arrow flex wheels, but we can't just chain together, we have to chain one by one or it would interact with other structure. (The highlighted red

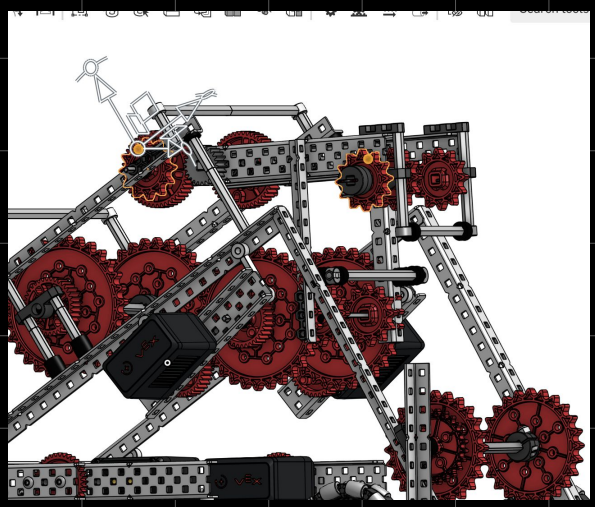


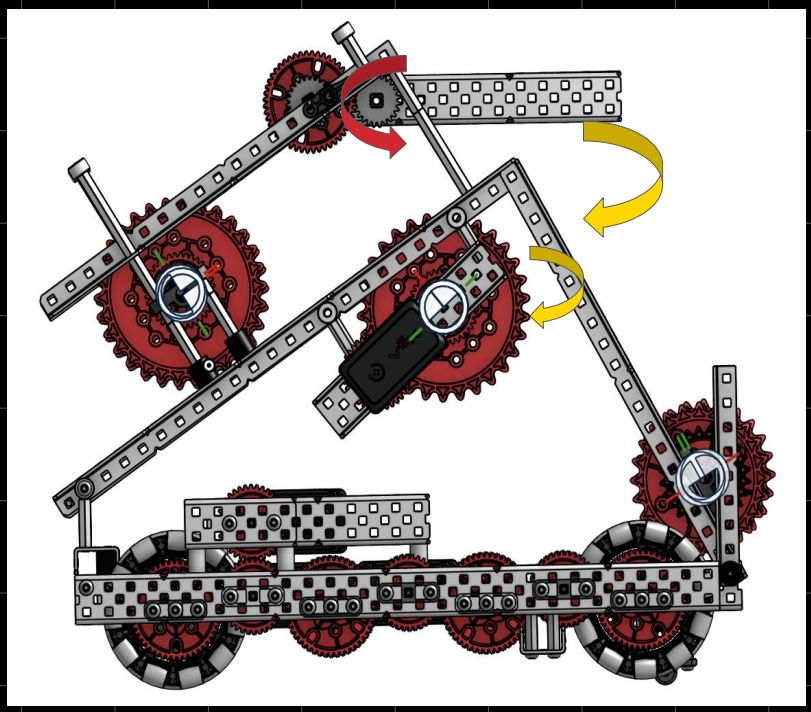


CAD

We chained other flex wheels with sprockets and chains. The circles represent chains, so the upper flex wheel is chain with both the motor connected axis and the lower flex wheel axis.

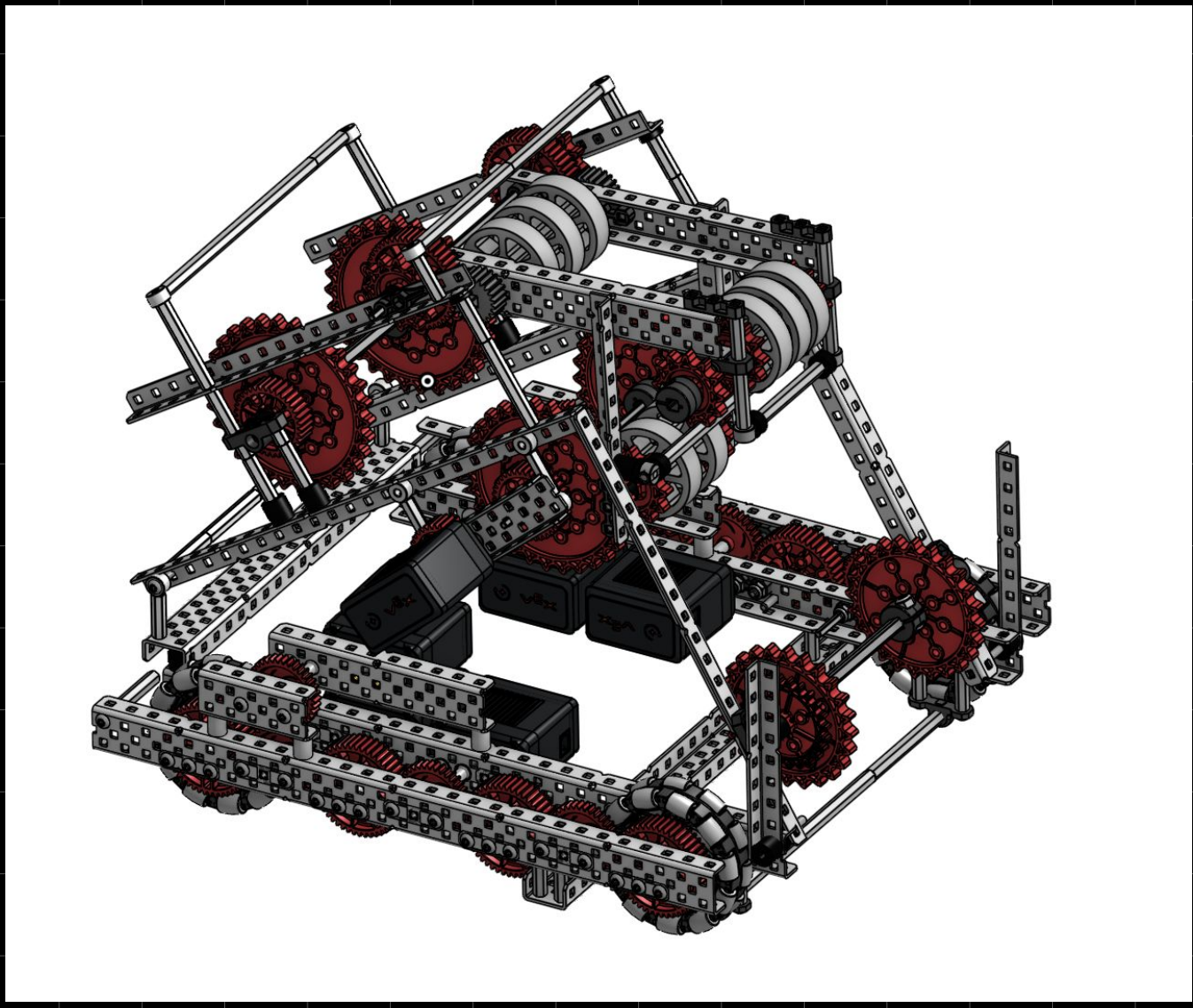






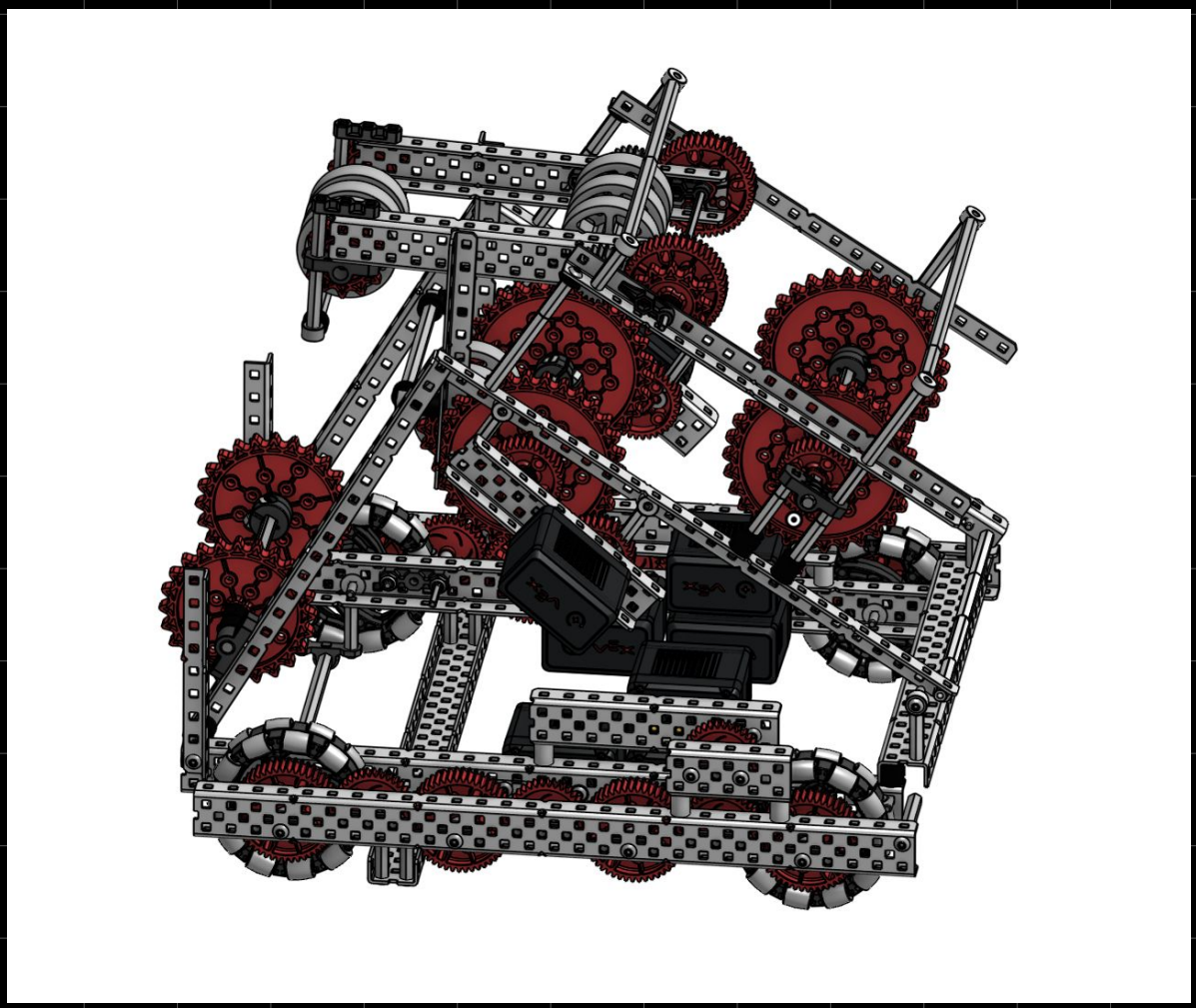
CAD

The CAD now looks really complicated and confusing, since the sprockets look all together. However, we think that we can start building, and obviously, we would have to change some dimensions when testing during building. We added the flex wheels and the robot is mostly finished.



This is the final CAD, we still have to test:

1. height of the piston for both retracted and extended
2. Some spaces between sprockets might need slight adjustments.
3. shooter motor placement
4. aligner
5. tongue
6. plastic boards and mat placement
7. overall intake smoothnest



Meeting

2025/08/30 — 30 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- CAD confirmation - Building Plan

In today's meeting, we

1. catch up with each others' progress and confirmed the current CAD design of our robot
2. assign tasks to each team member

Building Timeline

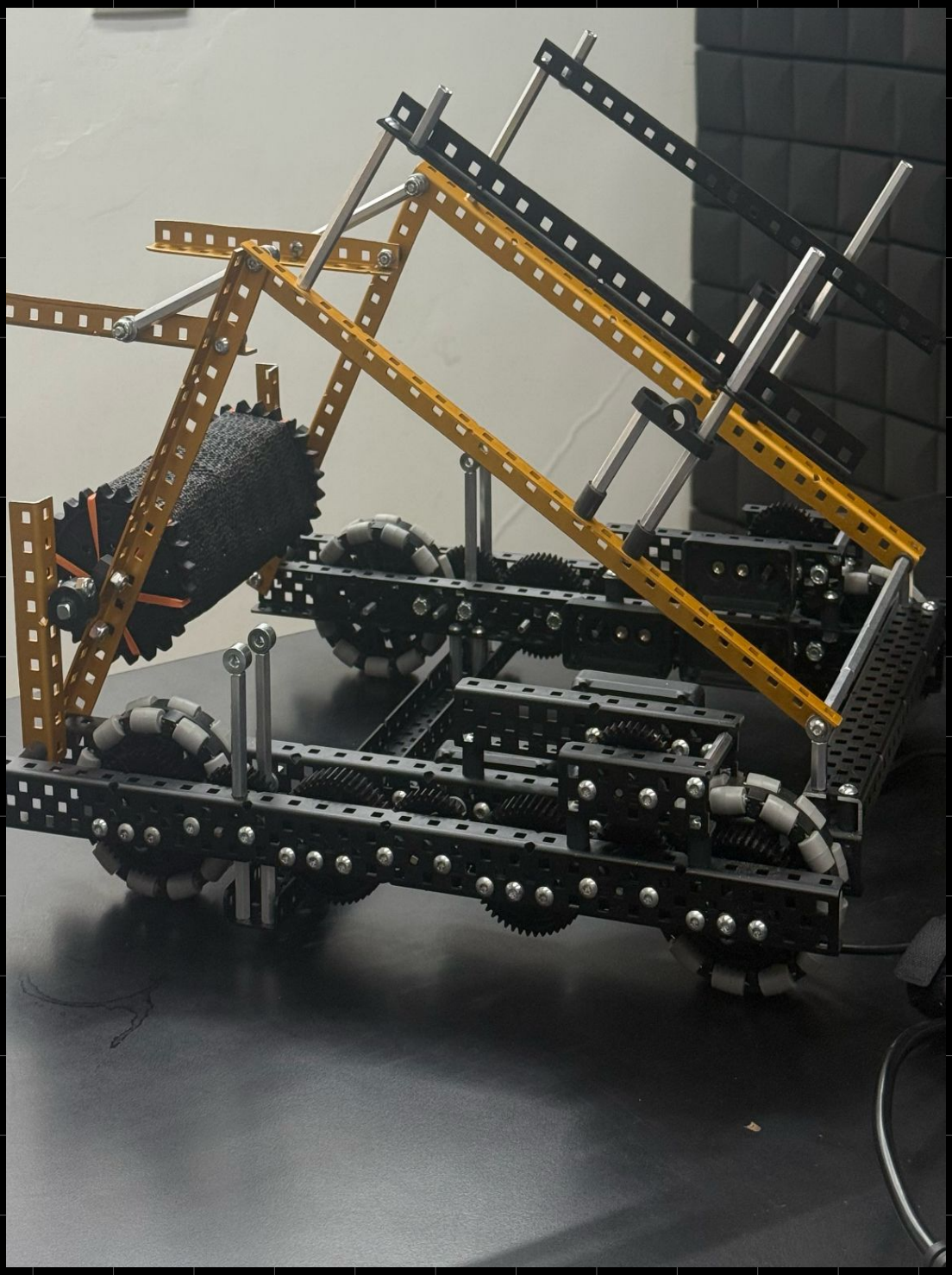
Tasks	8/30-9/6	9/7-9/15	9/16-9/22
Chassis			
Intake			
Aligner			
Tongue			
Color Sorting			
Programming			

Tasks per member:

- Chassis: Pennlaine, Adam
- Intake: Anthony, Jefferson
- Aligner: Pennlaine, Adam
- Tongue: Anthony Jefferson
- Color Sorting: Pennlaine
- Programming: Pennlaine, Adam

Building

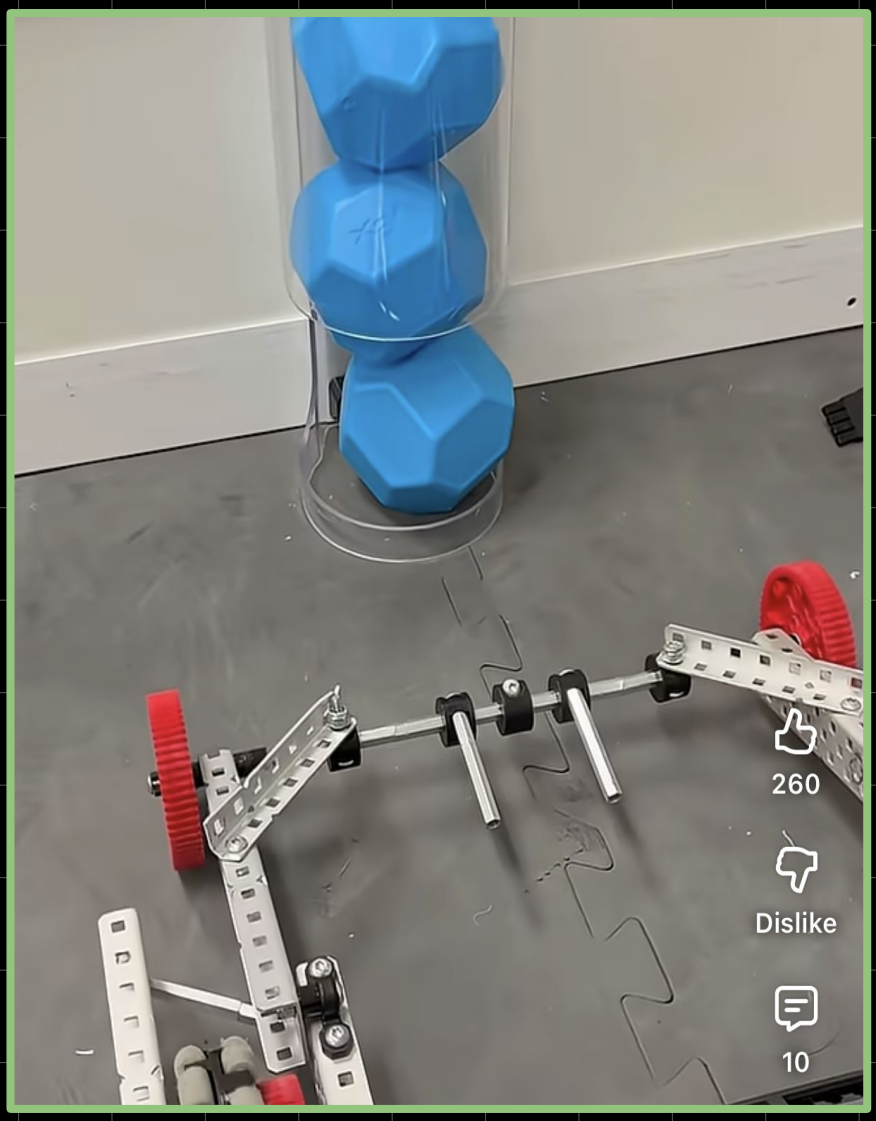
As we have our CAD finished, we started building.



Tongue

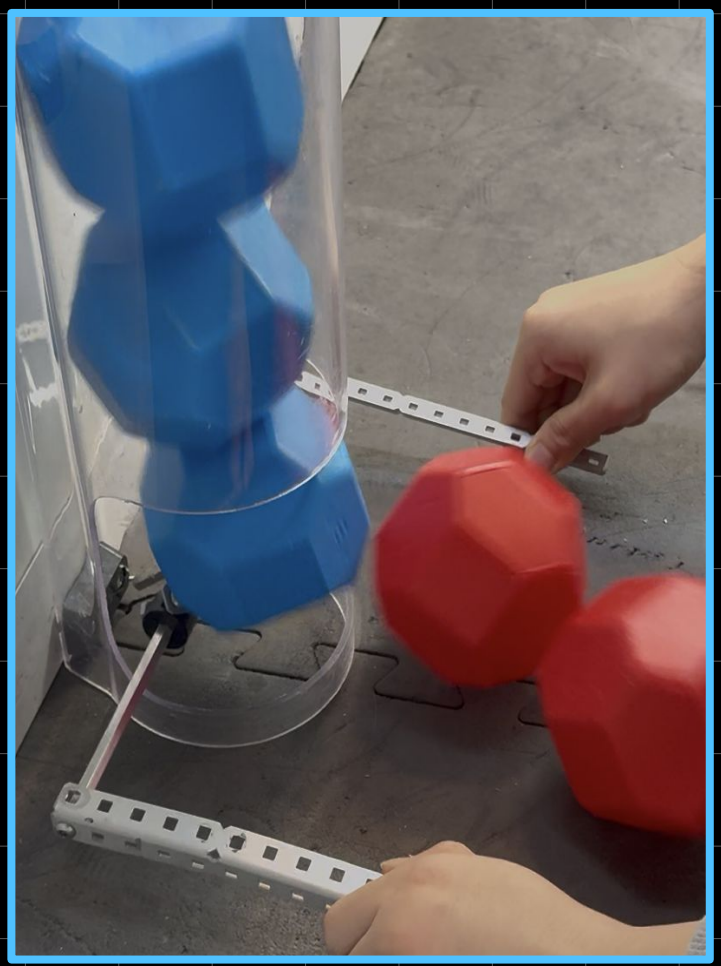
After having built our first intake sprocket, we would build the tongue mech to test if the sprocket could smoothly load with the tongue guiding it.

75088A demonstrates the meta of match loading, which is connected to pistons to extend out and match load.



Tongue

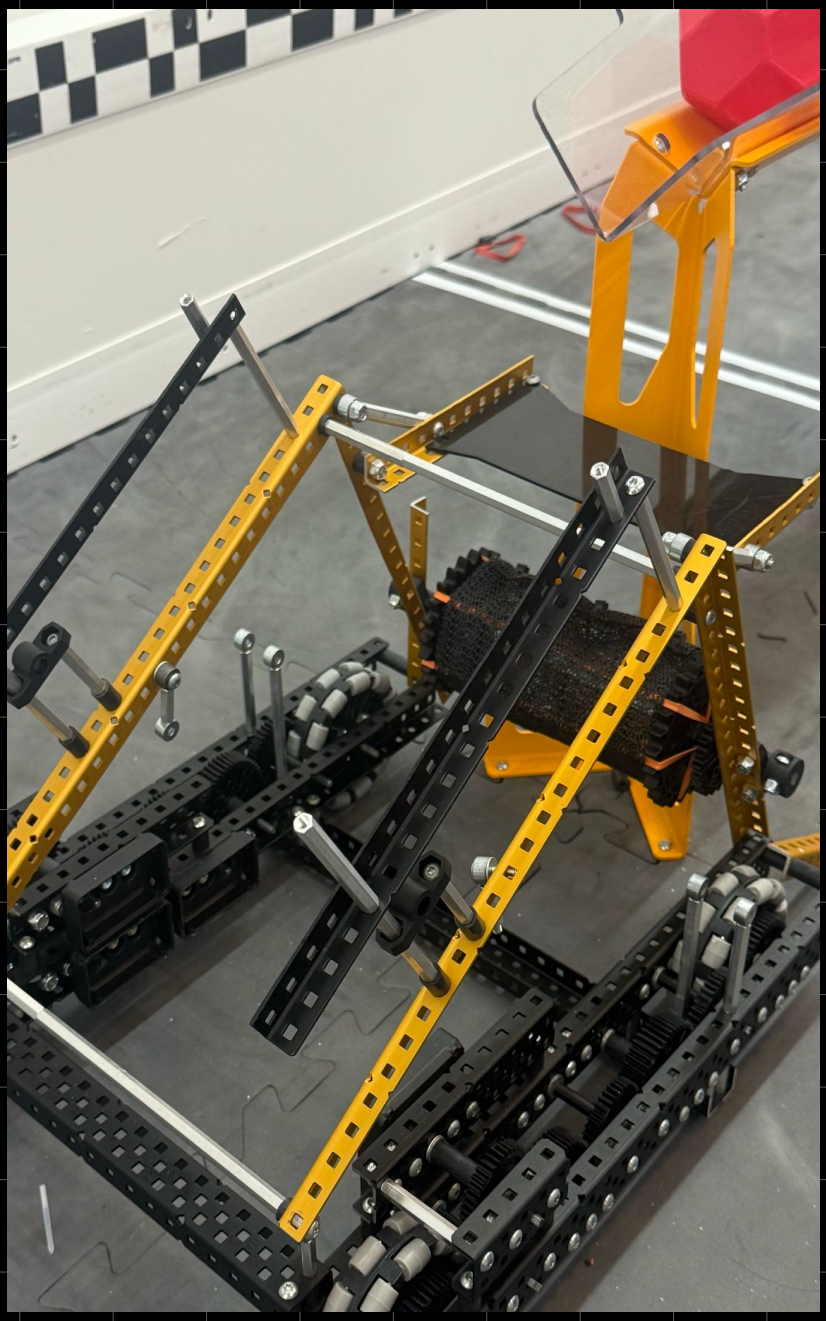
We tested out and adjusted the mech to a suitable and efficient angle. Then we attacheted it to the bot and tested with the intake sprocket.



Aligner

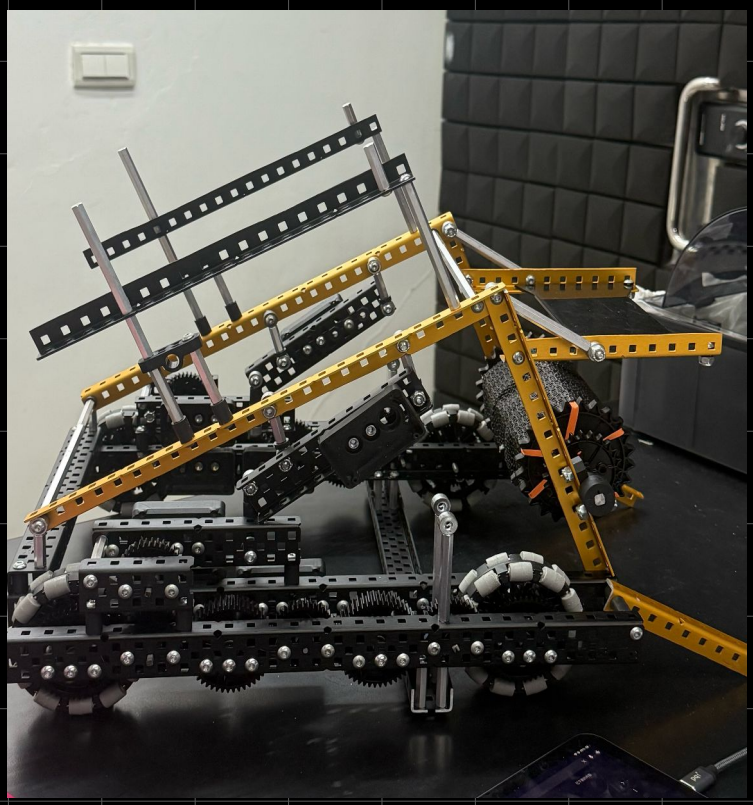
At the same time, we started testing the aligner.

However, as the image shows, the robot is too high, it cannot go under the long goal.

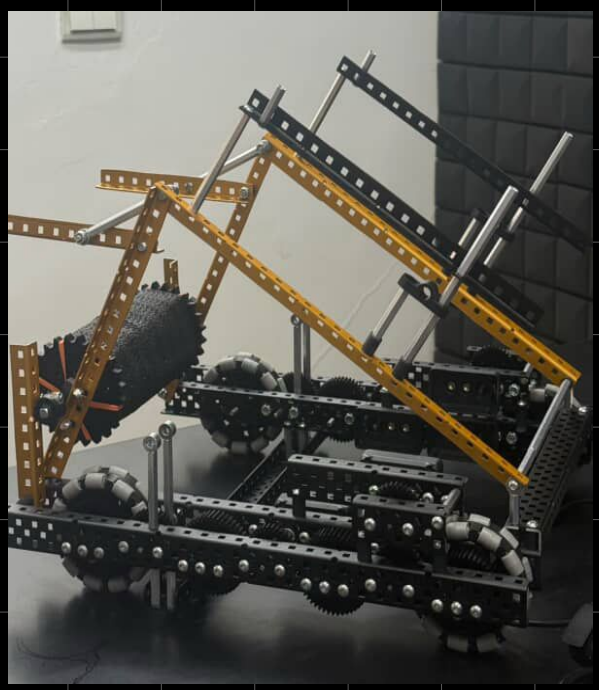


Alterations

Therefore, we altered it's height from 20 holes to 16 holes, so that it is just below the long goal. Now that the dimensions, height, width are accurate and passes inspection, we started add sprockets and the whole process went really fast.



We also add mats for the floor of the lower intake system. The motors act as perfect support for the mat just as we thought.

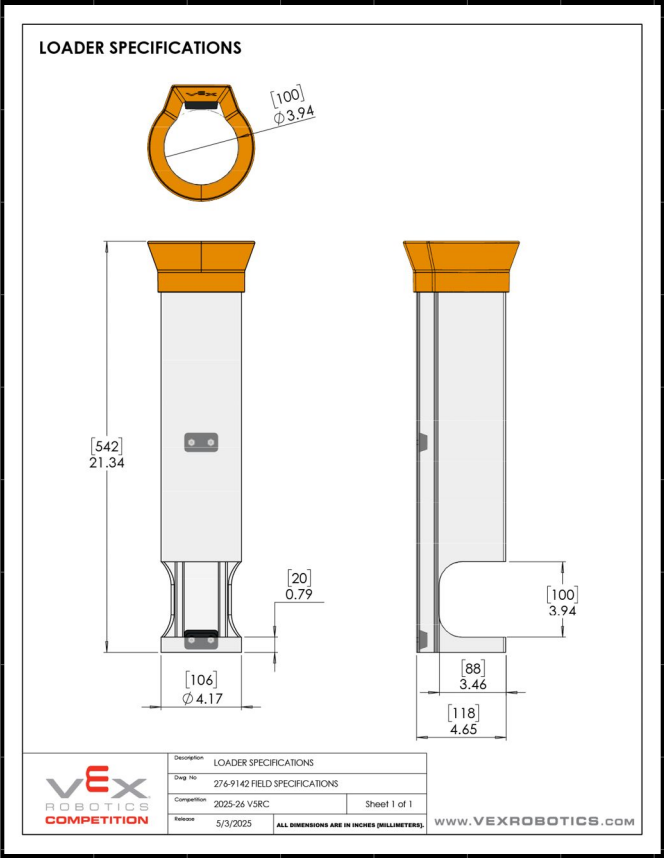


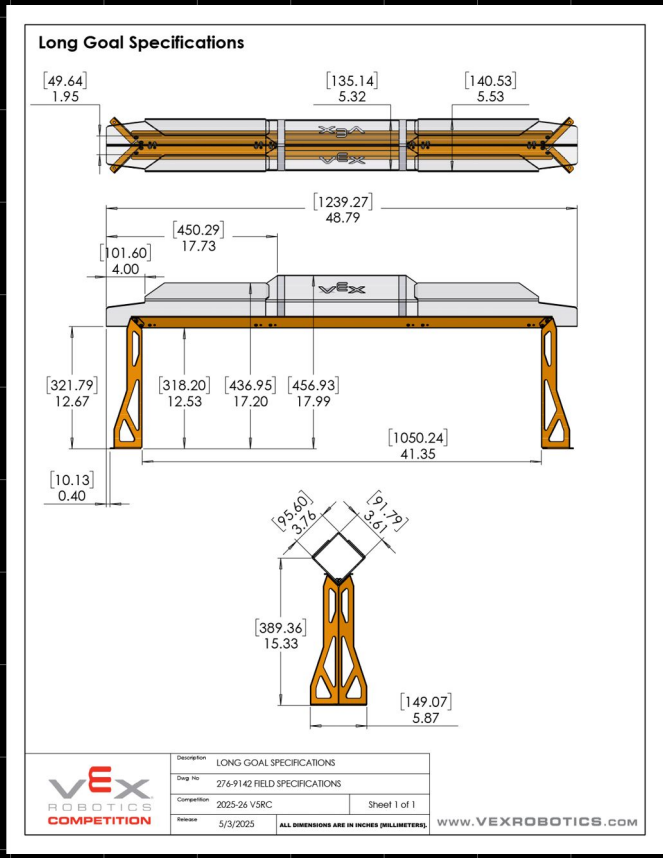
Aligner

Since we didn't CAD our aligner, we are now testing its size physically.

The problem is that the aligner would have to both fit in the loader tube and the triangular long goal column.

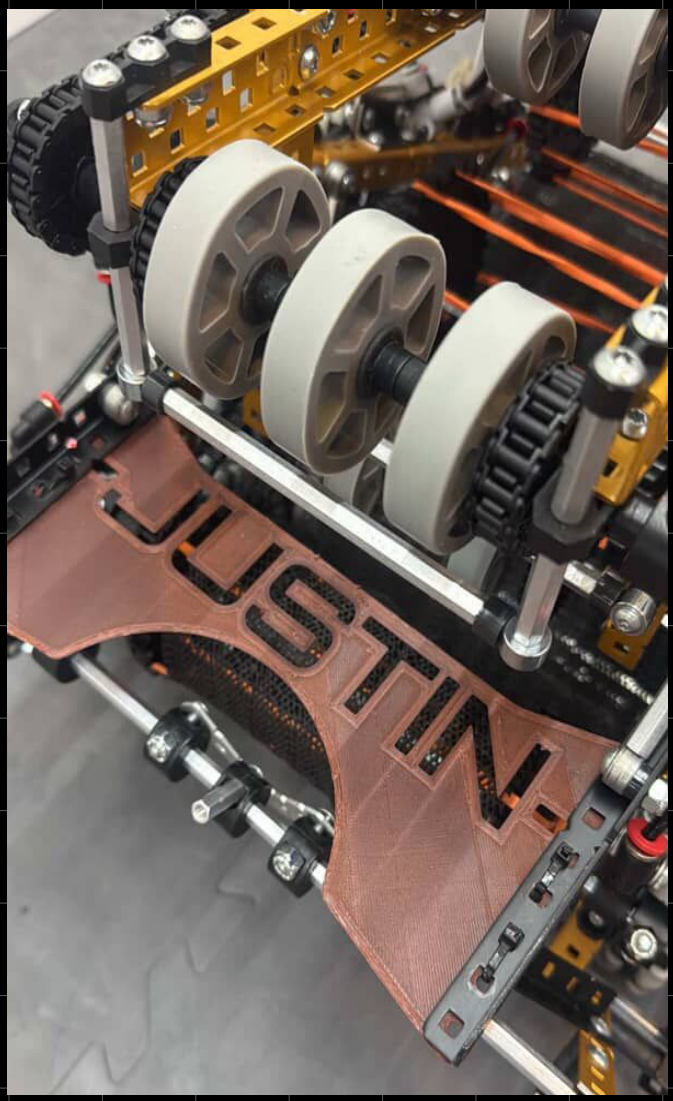
Both the width and the depth has to fit in.





Aligner

Since we don't have a cnc or laser cutter, we have to use 3d printing for testing and prototyping and then ask our friend from a frc team to help cut it with their school's laser cutter.



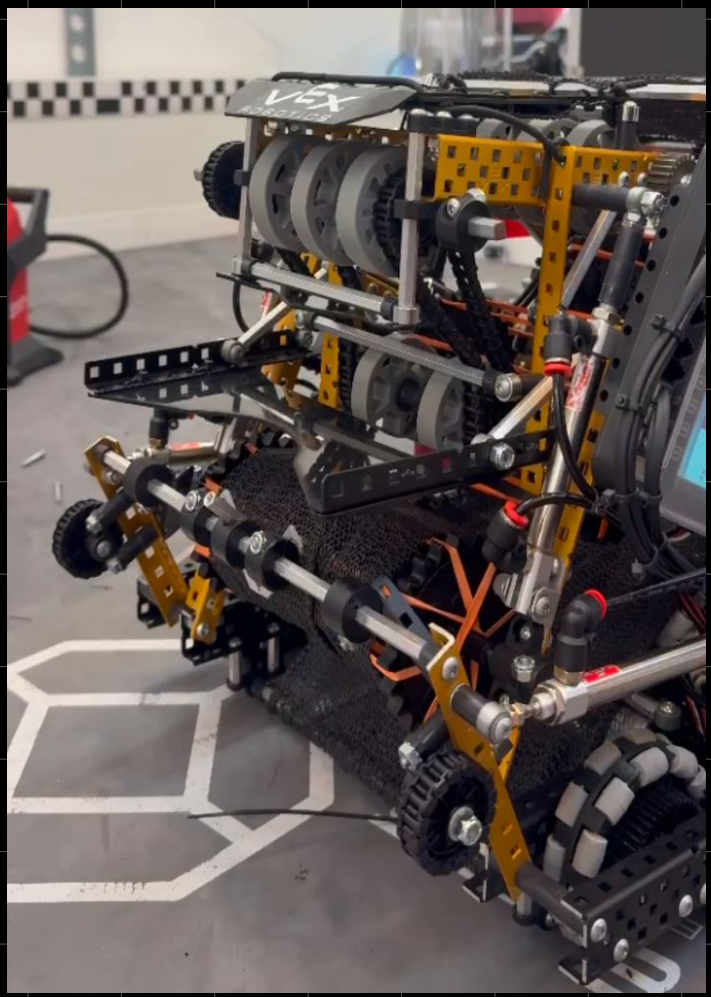
Aligner

Problem 1:

Aligner has to move forward when scoring at long goal or else the whole bot would be too near the long goal.

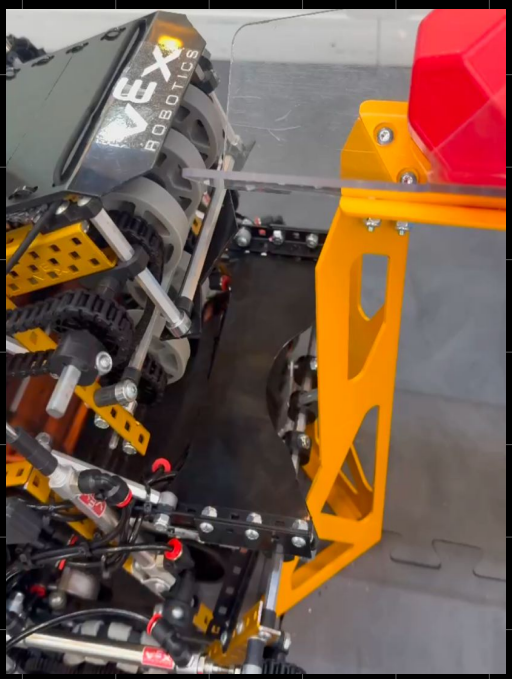
Problem 2:

However, when the robot is loading at the loaders, it needs to be nearer to the loaders



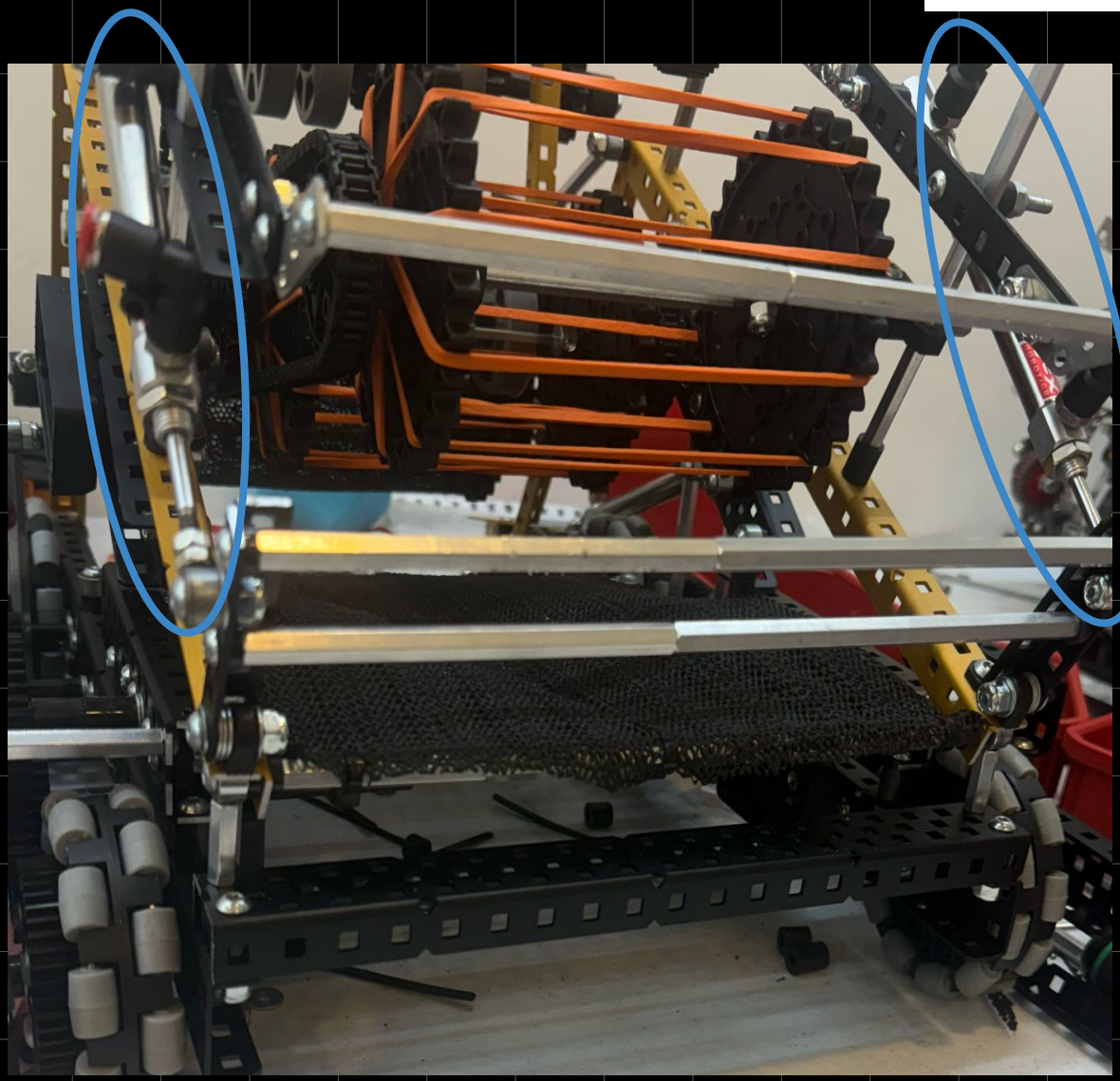
Aligner

We added pistons on the aligner to match the depth.



Robot 1 COLOR SORTING

We could incorporate color sorting at the end of the bot like 7121B BLITZ. So we added pistons at the back of the bot. We would also need to add an optical sensor.



Color Sorting

We are testing if the color sorting mech hardware could work, by cooperating with software.

TEST 1

To verify sensor detection reliability under match conditions.

Test Procedure

1. Place robot in autonomous mode
2. Feed blocks individually
3. Record sorting accuracy
4. Repeat for 4 sets each 5 trials

Testing sets (To simulate real matches incidents)

Set 1: Sort out BLUE - Red, Red, Blue, Red

Set 2: Sort out BLUE - Blue, Blue, Blue, Red

Set 3: Sort out RED - Red, Blue, Red, Blue

Set 4: Sort out RED - Blue, Blue, Blue, Blue

Color Sorting

TEST 1

This is the code snippet that we used to test:

```
while (!Controller1.ButtonB.pressing()) {
   intake.spin(forward, 12, volt);
   intakedown.spin(forward, 12, volt);

   if (Optical.isNearObject()) {
     vex::color detectedColor = Optical.color();
     blockCount++;
     bool shouldSort = false;
     if (selectedTeamColor == vex::color::red &&
detectedColor == vex::color::blue) {
       shouldSort = true;  // Want red, sort blue
     } else if (selectedTeamColor == vex::color::blue &&
detectedColor == vex::color::red) {
       shouldSort = true;  // Want blue, sort red
     }

     if (shouldSort) {
       pushCylinder = true;
       sortCount++;
       wait(0.2, sec);
       pushCylinder = false;
     }
     // Wait for block to pass
     while (Optical.isNearObject() &&
!Controller1.ButtonB.pressing()) {
       wait(20, msec);
     }
   }
```

Color Sorting

TEST 1

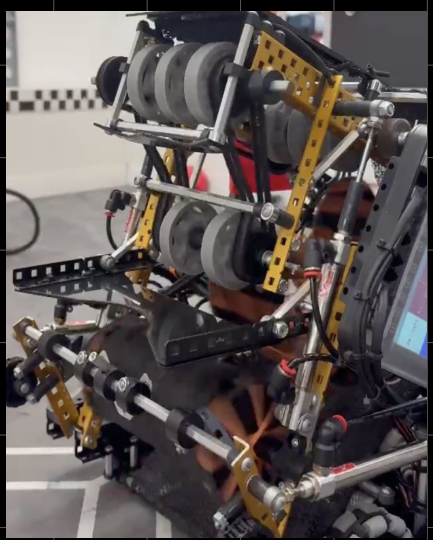
Results

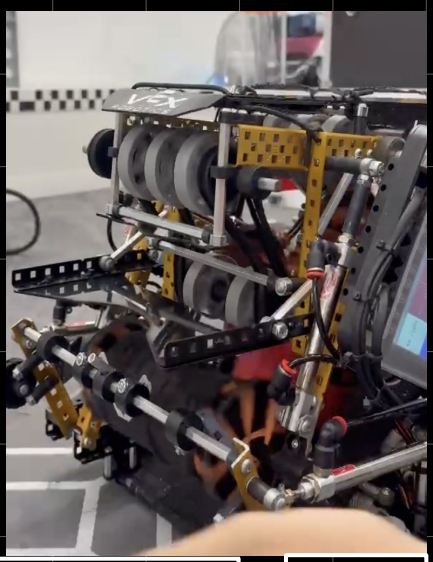
Set	Target	Correct Sorts	Incorrect	Accuracy
1	Blue	4 / 5	Missed	60%
2	Blue	4 / 5	Didnt stop sorting	80%
3	Red	4 / 5	Missed 1st Red	80%
4	Red	5 / 5		100%

Problem

From the testing results, we learned that:

1. the Optical Sensor has lower sensitivity for blue colors
2. It might delay when the wrong block is the first block to sort out.





Color Sorting

After identifying the problems, we:

1. Changed the program logic to NOT RED
2. Moved the Optical Sensor Forward.

TEST 2

Updated Program Snippet:

```
if (selectedTeamColor == vex::color::red && (detectedColor
!= vex::color::red || Optical.hue()<50)) { //Red is 20-40
        shouldSort = true;  // Want red, sort blue
    }
```

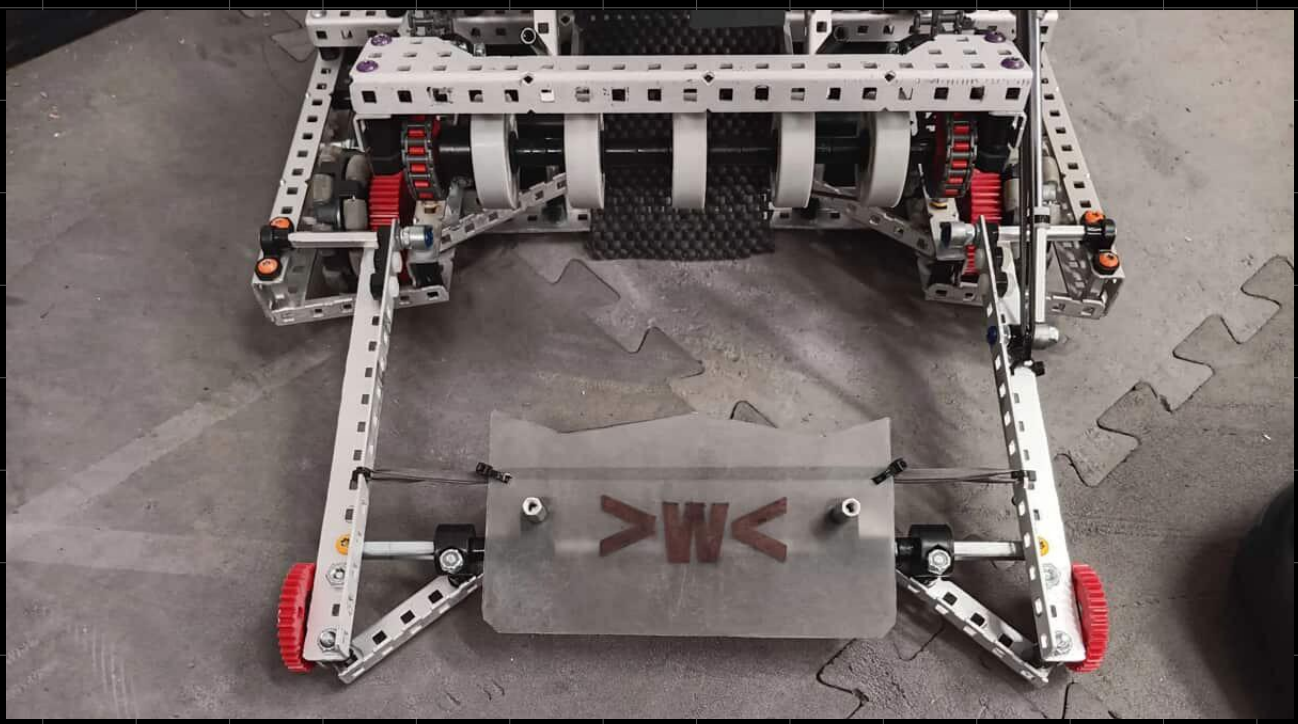
Set	Target	Correct Sorts	Incorrect	Accuracy
1	Blue	5 / 5	Missed	80%
2	Blue	4 / 5		100%
3	Red	5 / 5		100%
4	Red	5 / 5		100%

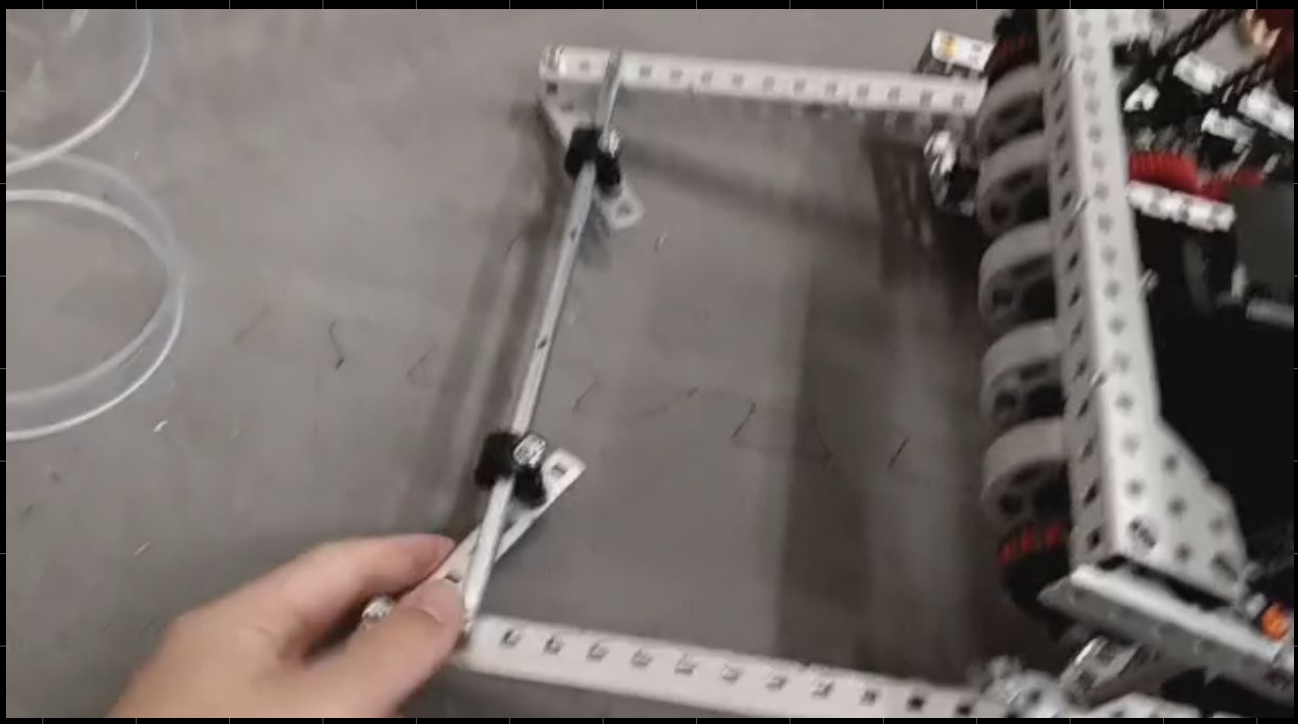
Analysis

1. The accuracy improved significantly, and we are satisfied with our color sort mech.

Tongue

Recently on VEX's Discord server, we were inspired by the new meta of match loading tongue introduced by 338A Iterated.





Tongue

We saw the new meta tongue, so we tested both.

TEST 1

To compare loading efficiency between the standard two-standoff intake interface and a custom plastic board guide under match conditions.

Test Procedure

1. Position robot at loader
2. Align intake with loader opening
3. Start intake and timer simultaneously
4. Allow 6 blocks to load
5. Stop timer when final block is secured

Trial	Standoffs	Plastic
1	5.32	3.91
2	4.95	3.78
3	4.71	3.85
4	4.88	3.73
5	4.90	3.80
6	5.03	3.07
7	4.76	2.97
8	5.62	3.12

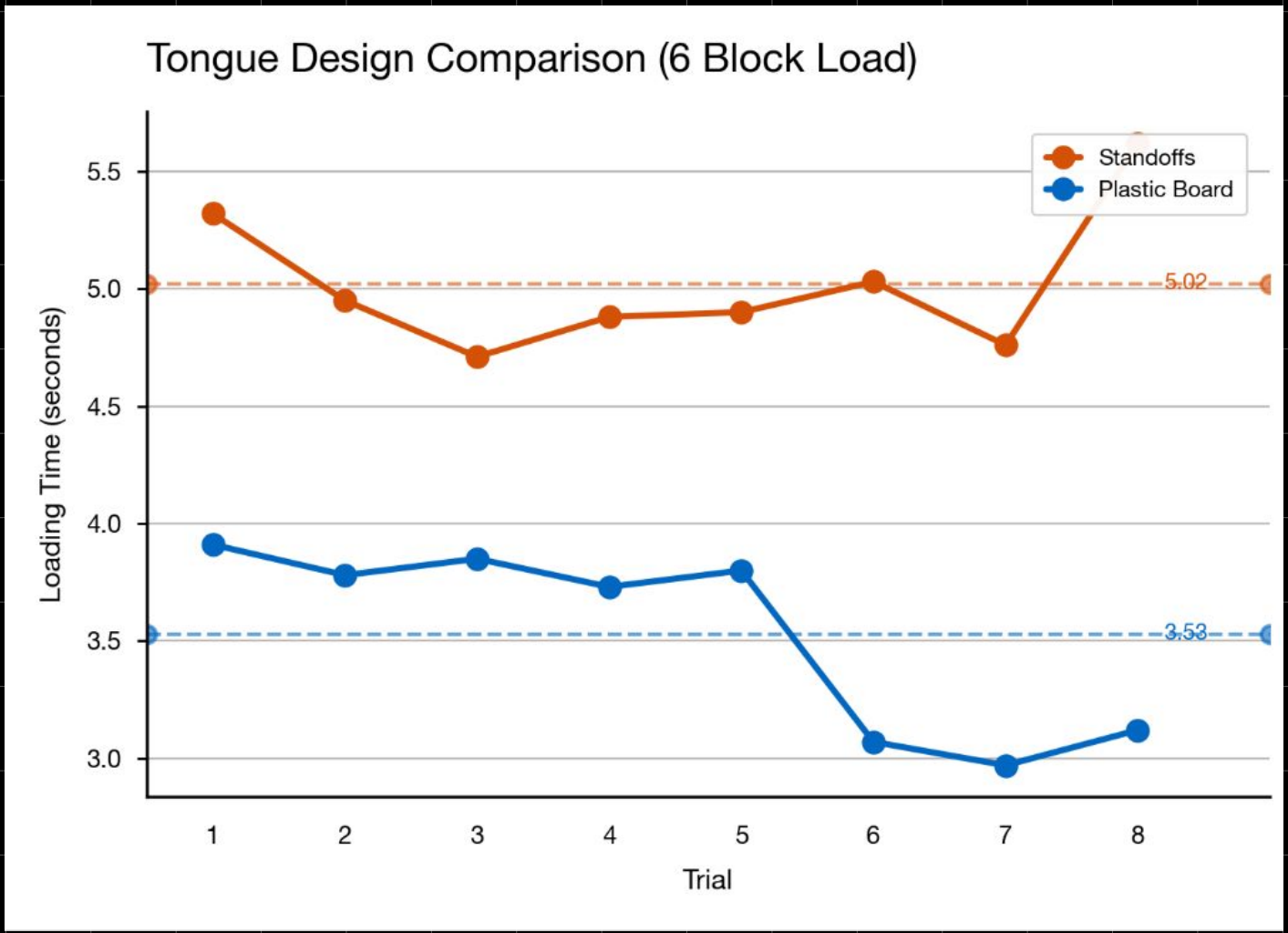
Tongue

TEST 1

Avg Loading Time

Configuration	Avg Time (s)
Standoffs	5.02
Plastic Board	3.53

Below plot shows TEST 1 results:



Tongue

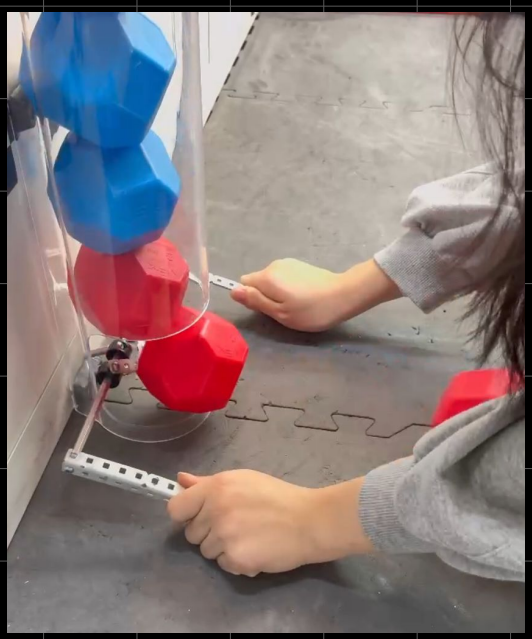
TEST 1

Analysis

The plastic board guide provided continuous surface contact, reducing lateral block movement and minimizing alignment corrections. In contrast, the standoff design allowed blocks to shift between contact points, increasing loading time.

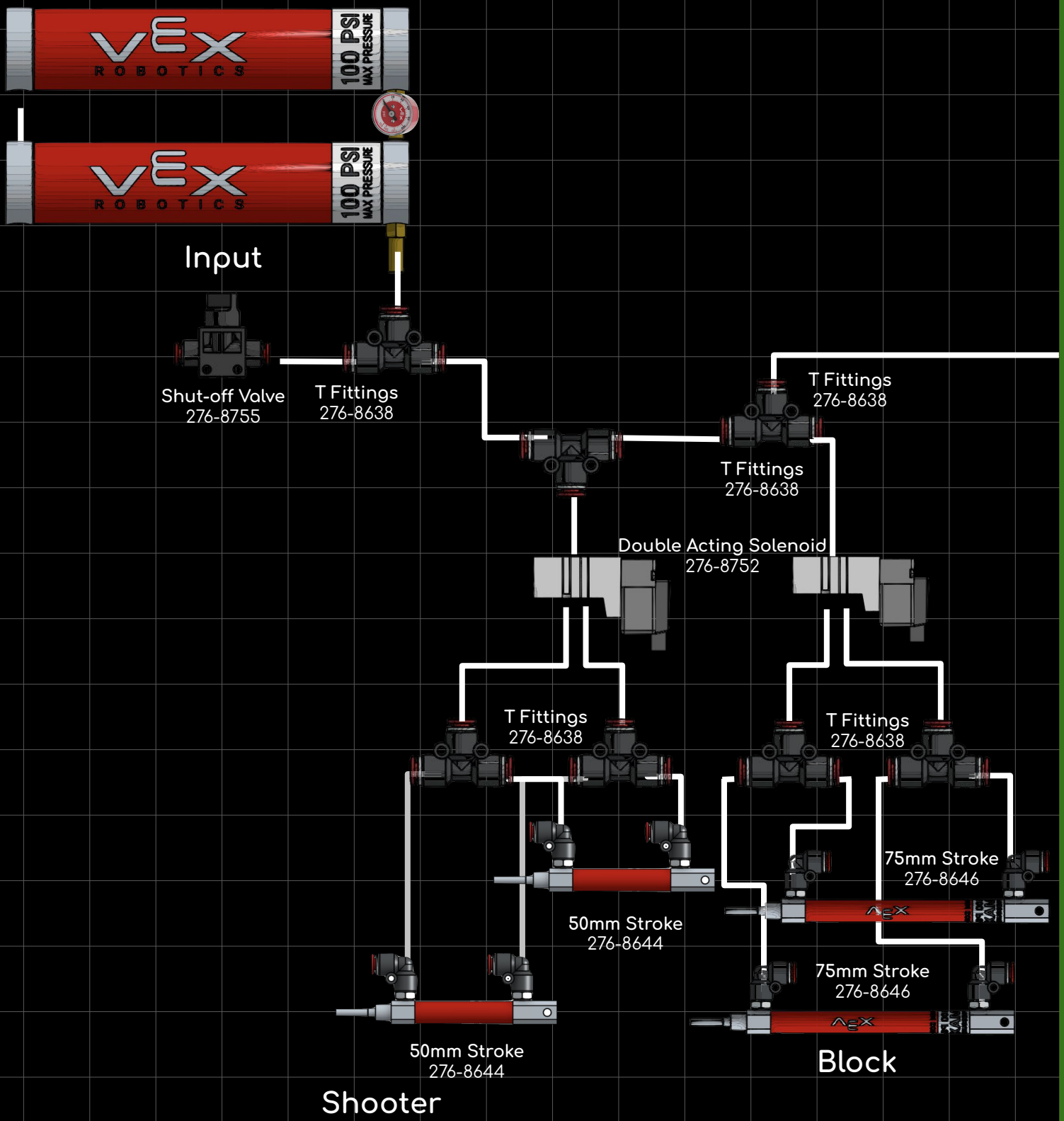
Decision

Due to significantly reduced loading time and improved guiding consistency, the plastic board interface was selected over the standard standoff design.

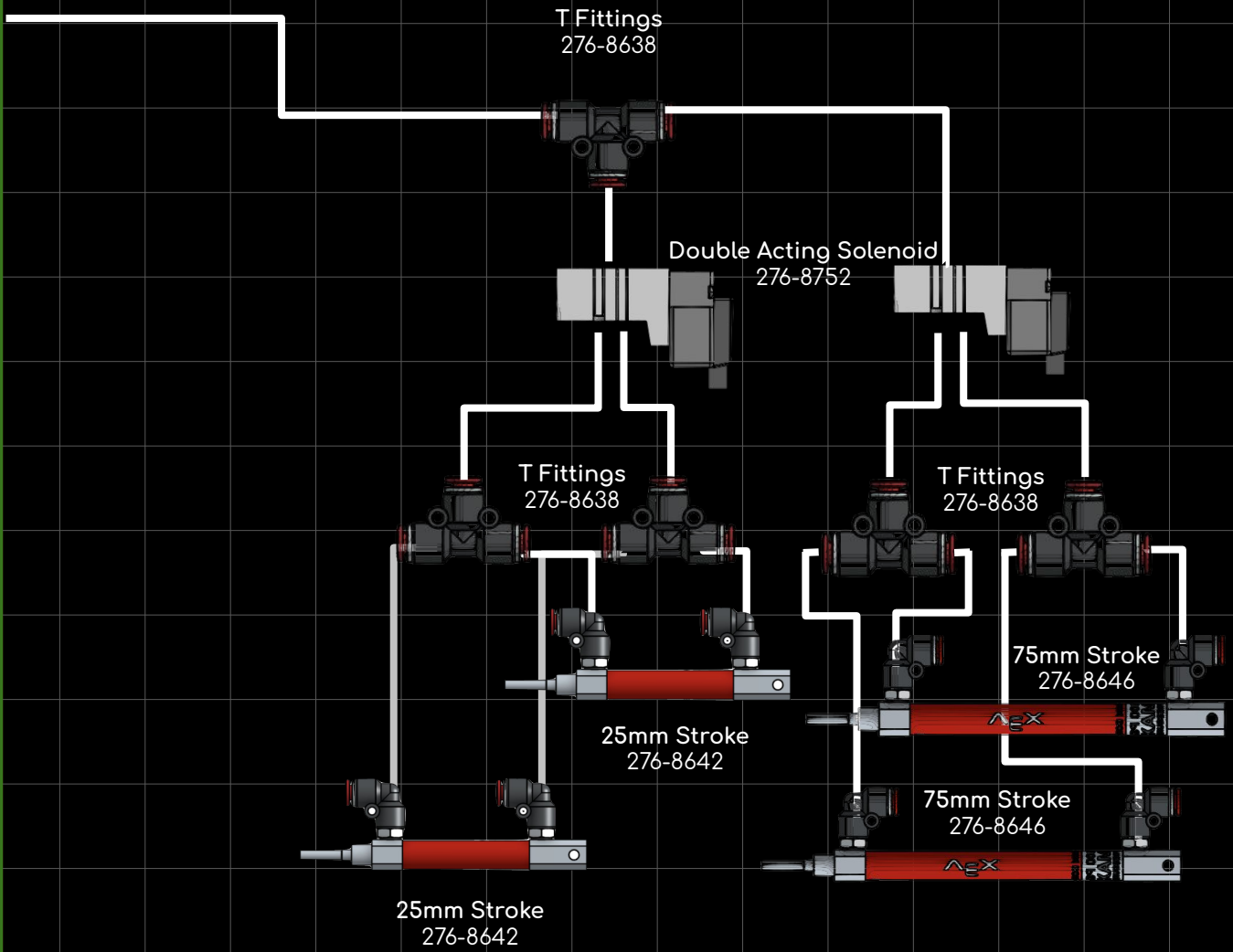


Pneumatic System

Pneumatic System Diagram:



Pneumatic System



Meeting Programming

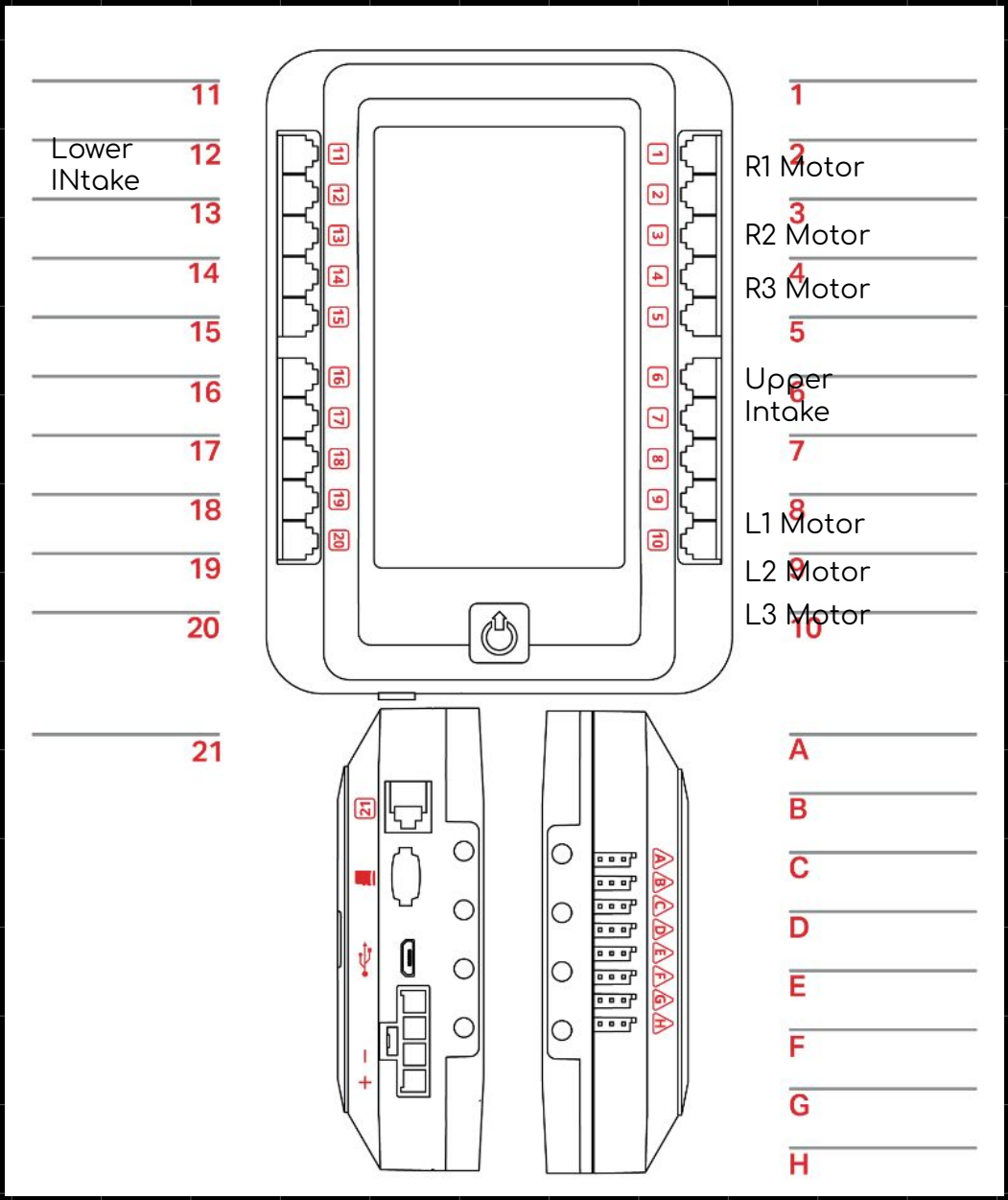
2025/09/15 — 45 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- Programming Plan

The robot is now finished, we have the upcoming AST Scrimmage so we really shall start coding autos. At least we have already tried our new programming system and make sure it works.

Cable Management

The wires were changed ports many times during our building and testing processes for a better / more neater organization.



Controller Functions

- The programmer discussed with driver by brainstorming the required functions:

1. **Motor groups on Drive**
   a. Tank Drive

2. **Lower Intake Belt System**
   a. Forward (pressing)
   b. Reverse (pressing)
   c. Stop (not pressing)

3. **Upper Intake Belt System**
   a. Forward (pressing)
   b. Reverse (pressing)
   c. Stop (not pressing)

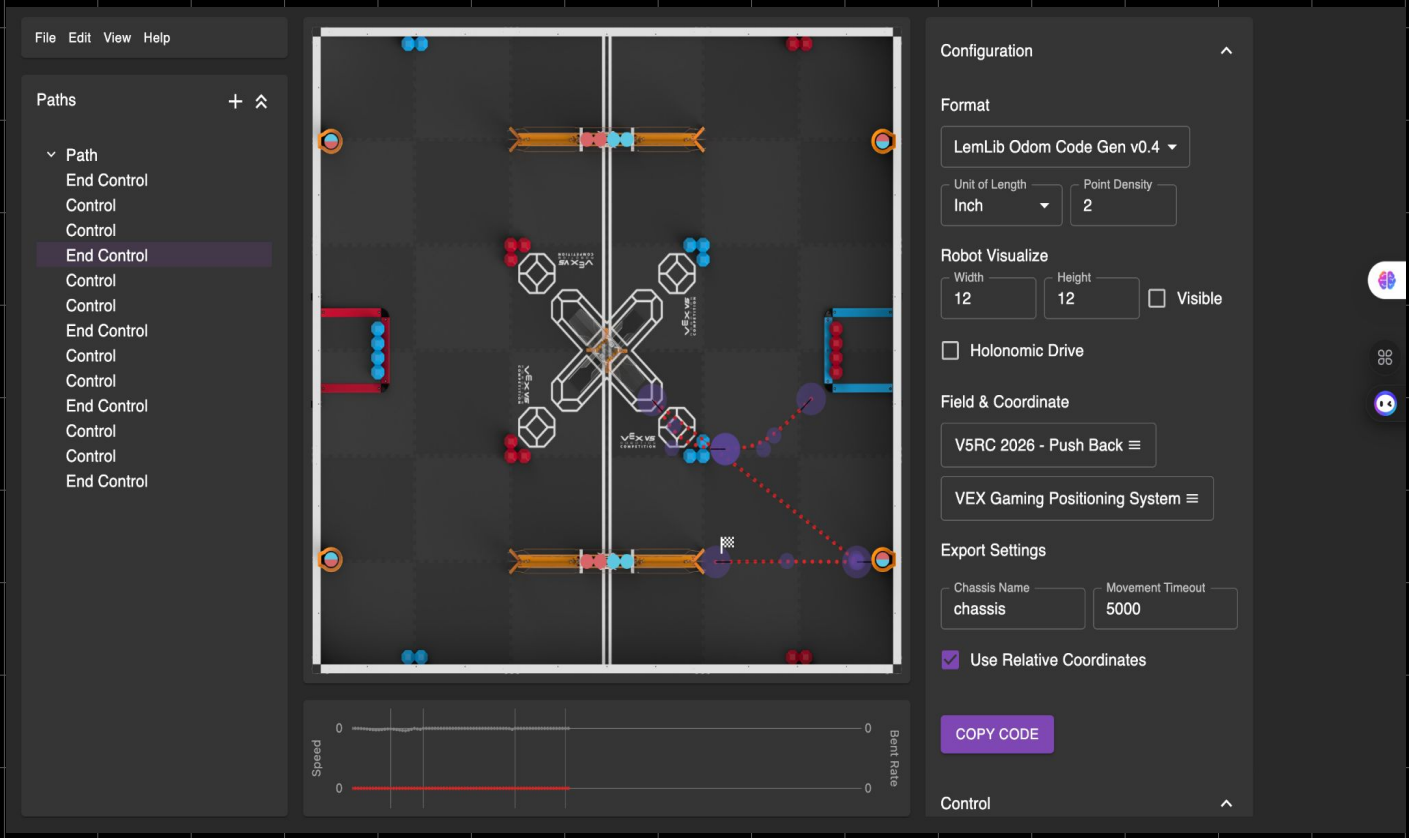
Programming Intro

In the past two seasons, we have always used the VexCode Pro V5 environment for coding, but last season, during the final event, we talked with another team, and they shared that they used a system "LemLib" for autonomous coding and find it relatively easy and efficient.

Programming Intro

LemLib is an open source VEX coding system that helps code for the VEX robotics drivetrain setup and autonomous movements using PROS syntax. It provides the code for the path we drew on the UI.

The image provided is an illustration of a possible solo AWP path.



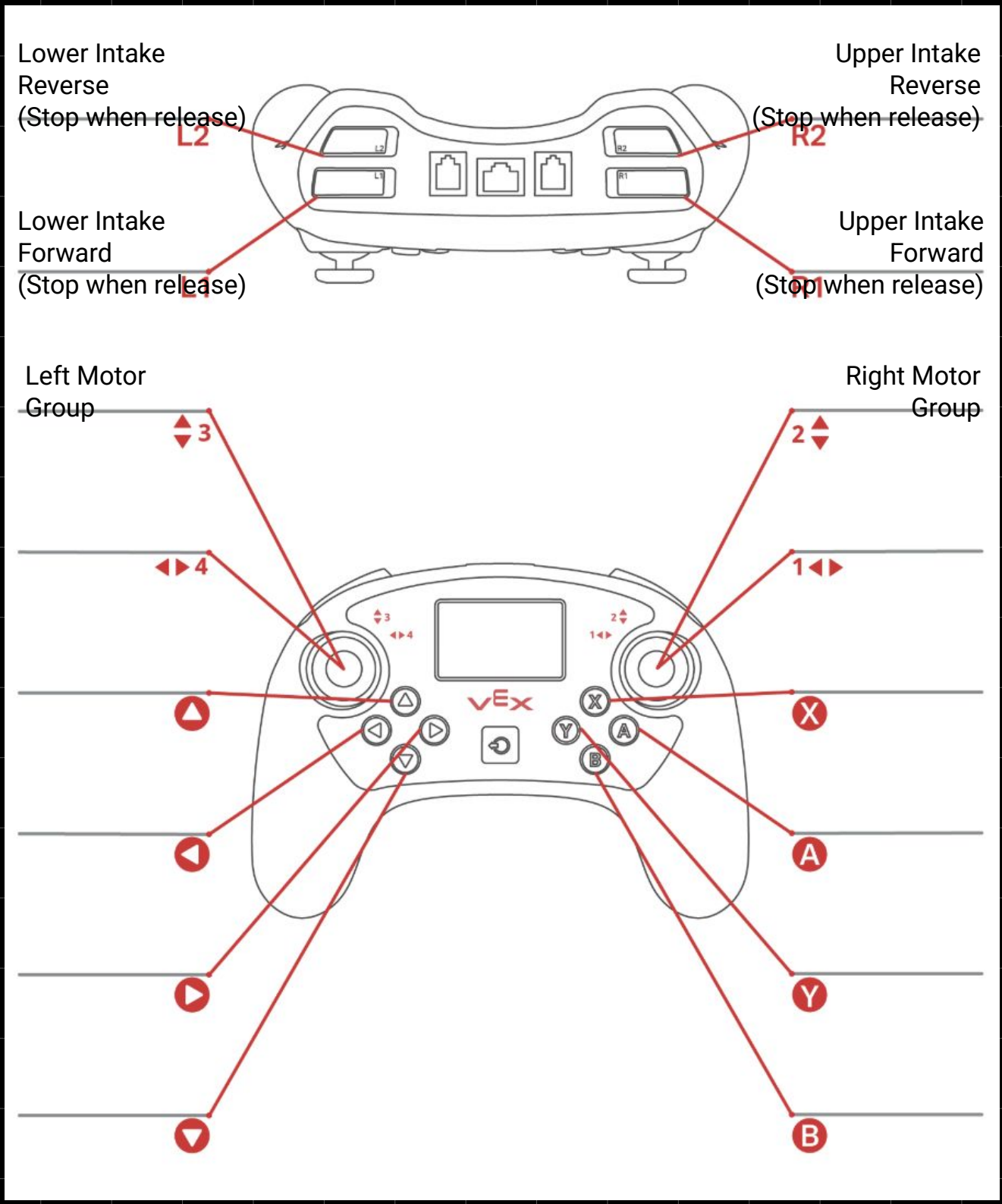
Setting Up

Thus, in order to have the lemlib and pros syntax packages, we cloned the LemLib github repo (https://github.com/LemLib/LemLib.git).

We really wished that we knew this earlier, because working with VSCode is way easier than we thought. Also, this allowed different programmers to share the code through push and pulling from a mutual github repo, unlike that we had to email it back and forward before.

Therefore, we created a github organization of our school and our vex organization team number — "23083 Wego Robotics." Our cloned LemLib files was our first commit to our newly created repo in our organization.

Controller Functions



Programming

Basic setup:

```
// controller
pros::Controller controller(pros::E_CONTROLLER_MASTER);
// Motor Groups
pros::MotorGroup leftMotors({-2, -3, 1},
                            pros::MotorGearset::blue);
pros::MotorGroup rightMotors({9, 8, -7},
pros::MotorGearset::blue);

// Function Motors
pros::Motor Intake1(-11); //lower
pros::Motor Intake2(-5);// upper
```

Drive (within the `opcontrol(void)` function):

```
    while (true) {
        // get joystick positions
        int leftY =
controller.get_analog(pros::E_CONTROLLER_ANALOG_LEFT_Y);
        int rightX =
controller.get_analog(pros::E_CONTROLLER_ANALOG_RIGHT_Y);
        // move the chassis with tank drive
        chassis.tank(leftY, rightY);
        pros::delay(10);
        //--------- functions-------------
        IntakeChain1();
        IntakeChain2();
    }
```

Intake Function

Drive (within the `IntakeChain1()` function):

```
void IntakeChain1() { //pressing button function
   Intake1.set_brake_mode(pros::E_MOTOR_BRAKE_COAST);
   bool lowerIntake1 =
controller.get_digital(pros::E_CONTROLLER_DIGITAL_L1);
   bool lowerIntake2 =
controller.get_digital(pros::E_CONTROLLER_DIGITAL_L2);

   if (lowerIntake1) {
       Intake1.move_velocity(200);    // L1 forward
   }
   else if (lowerIntake2) {
       Intake1.move_velocity(-200);   // L2 reverse
   }
   else {
       Intake1.brake();
   }
}
```

Intake Function

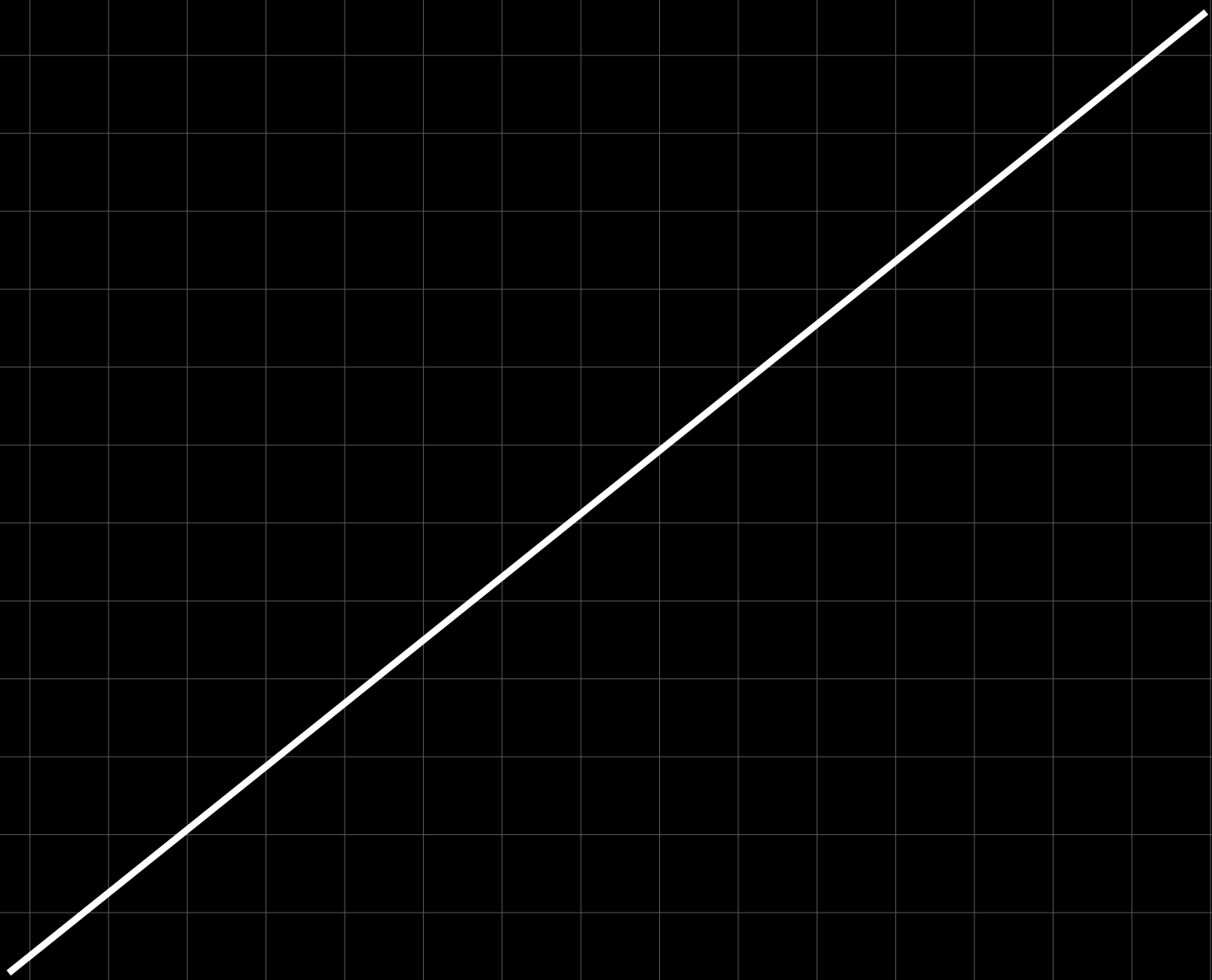
Drive (within the `IntakeChain2( )` function):

```
void IntakeChain2() { //pressing button function
   Intake2.set_brake_mode(pros::E_MOTOR_BRAKE_COAST);
   bool upperIntake1 =
controller.get_digital(pros::E_CONTROLLER_DIGITAL_R1);
   bool upperIntake2 =
controller.get_digital(pros::E_CONTROLLER_DIGITAL_R2);

   if (upperIntake1) {
       Intake2.move_velocity(200);     // R1 forward
   }
   else if (upperIntake2) {
       Intake2.move_velocity(-200);    // R2 reverse
   }
   else {
       Intake2.brake();
   }
}
```

Drivetrain

The driver mentioned that he would like the drivetrain to brake with “coast”.

We found that the “tank()” function was within the lemlib package and we did not know how to change it since it seemed like an installed precompiled binary, not source code.



Drivetrain

The programmer wrote the tank drive manually, setting the individual drivetrain motor's brake mode as **E_MOTOR_BRAKE_COAST**

```
// motors set coast
int deadband(int input, int threshold = 5) {
   return (abs(input) > threshold) ? input : 0;
}

double to_volt(int power) {
   return (power / 127.0) * 12.0;
}
```

Drivetrain

```
void control_tank(float percentage = 100.0f) {
   leftMotors.set_brake_mode(pros::MotorBrake::coast);
   rightMotors.set_brake_mode(pros::MotorBrake::coast);
    int leftInput =
deadband(pros::Controller(pros::E_CONTROLLER_MASTER).get_analog(pros
::E_CONTROLLER_ANALOG_LEFT_Y));
    int rightInput =
deadband(pros::Controller(pros::E_CONTROLLER_MASTER).get_analog(pros
::E_CONTROLLER_ANALOG_RIGHT_Y));
    double leftVoltage = to_volt(leftInput * percentage / 100.0);
    double rightVoltage = to_volt(rightInput * percentage / 100.0);
    if (leftVoltage != 0) leftMotors.move_voltage(leftVoltage *
1000);
    else {
        left1.brake();
        left2.brake();
        left3.brake();
    }
    if (rightVoltage != 0) rightMotors.move_voltage(rightVoltage *
1000);
    else {
        right1.brake();
        right2.brake();
        right3.brake();
    }
}
```

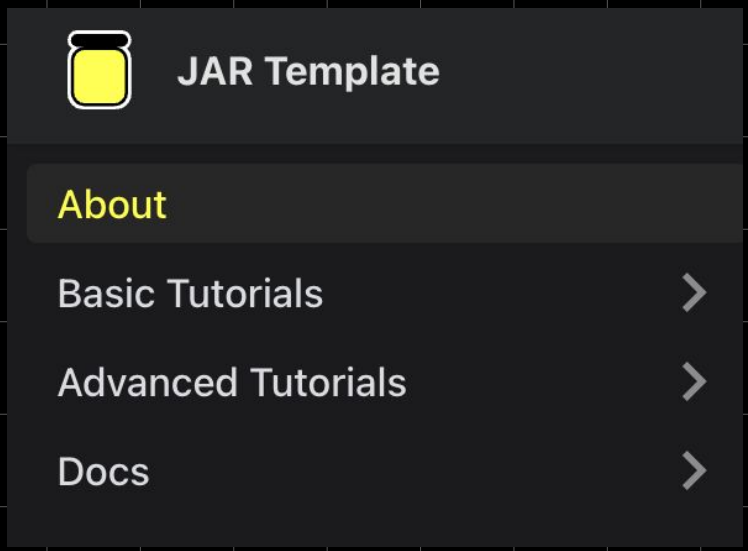
Drivetrain

While we tried to write our autonomous, we found out that Lemlib has several problems.

1. The encoder units for each our wheel changes to different ones for the Chassis function
2. It doesn't really benefit us a lot since we don't have odometry wheels so the copied paths aren't really precise anyways.
3. Combining Lemlib with pros has several library or version problems

Drivetrain

Therefore we changed to JAR-Template which we found more straight forward, since the library is written all within the files not with extensions, and we have more flexibility with changing it and tailoring it.



System Architecture

The robot runs a modular software setup:

- Main Control Loop (main.cpp): Handles competition initialization and match management.
- Drive System (JAR-Template): Tank drive with odometry, PID control, and motion profiling.
- Autonomous Routines (**autons.cpp**): Pre-programmed with several routes to choose before match starts.
- Driver Control: Separate tasks for action and sensors.
- User Interface: Touchscreen selection and real-time dashboard telemetry.

```
VEX/
├── src/
│   ├── main.cpp              # Competition control, UI, custom
movement
│   ├── robot-config.cpp      # Hardware definitions
│   ├── autons.cpp            # 10 autonomous routines
│   ├── arm.cpp               # Hang control task
│   ├── note.cpp              # Color sorting task
│   └── JAR-Template/
│       ├── drive.cpp         # Tank drive implementation
│       ├── odom.cpp          # Odometry calculations
│       ├── PID.cpp           # PID controller logic
│       └── util.cpp          # Helper functions
├── include/
│   └── [corresponding .h files]
└── build/
    └── Jar-Template.bin      # Compiled binary for V5 Brain
```

Drivetrain Control

JAR-Template Drive Framework

Features:

- Tank drive
- PID controllers for drive, turn, heading, and swing
- Autonomous point-to-point navigation with heading correction

Drive Configuration

ZERO_TRACKER_ODOM:

- Uses drive motor encoders + inertial sensor for position tracking
- No dedicated tracking wheels required
- Forward tracker offset (5.2") compensates for center of rotation
- Sideways tracker detects lateral drift during turns

Programming

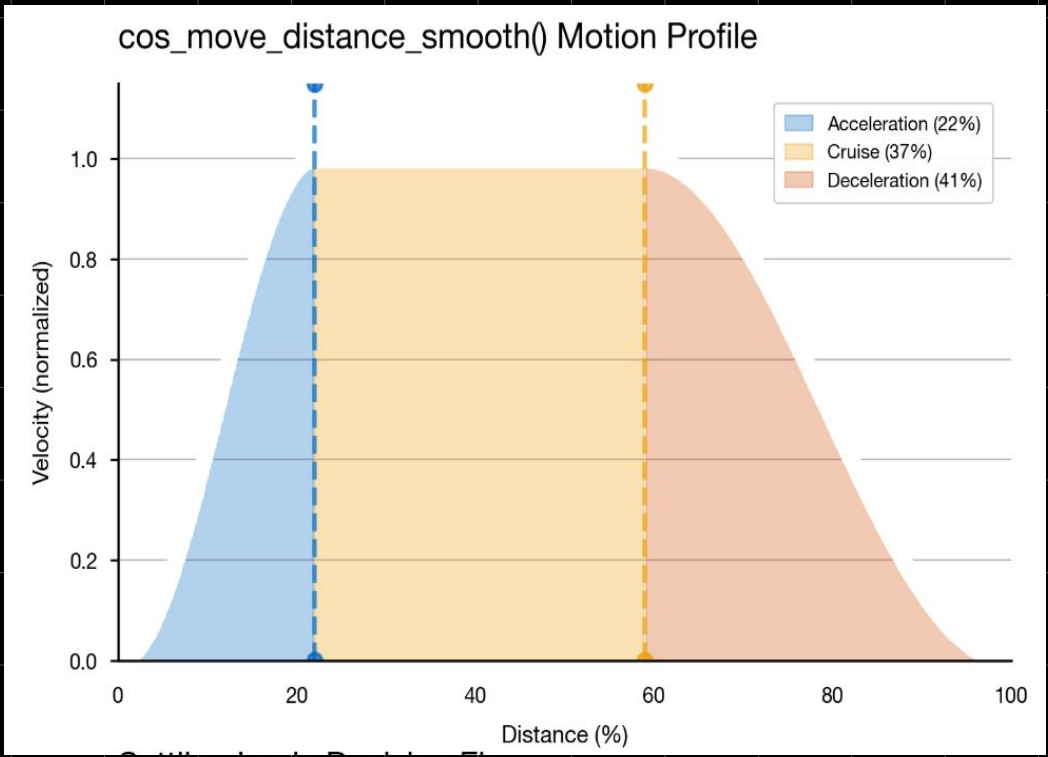
The **`cos_move_distance_smooth()`** function implements a sophisticated motion profile:

- Acceleration Phase (22% of distance): Cosine-based smooth acceleration

  `v = Vmax × 0.5 × (1 - cos(π × progress))`

- Cruise Phase (37% of distance): Constant velocity at Vmax)
- Deceleration Phase (41% of distance): Cosine-based smooth deceleration)

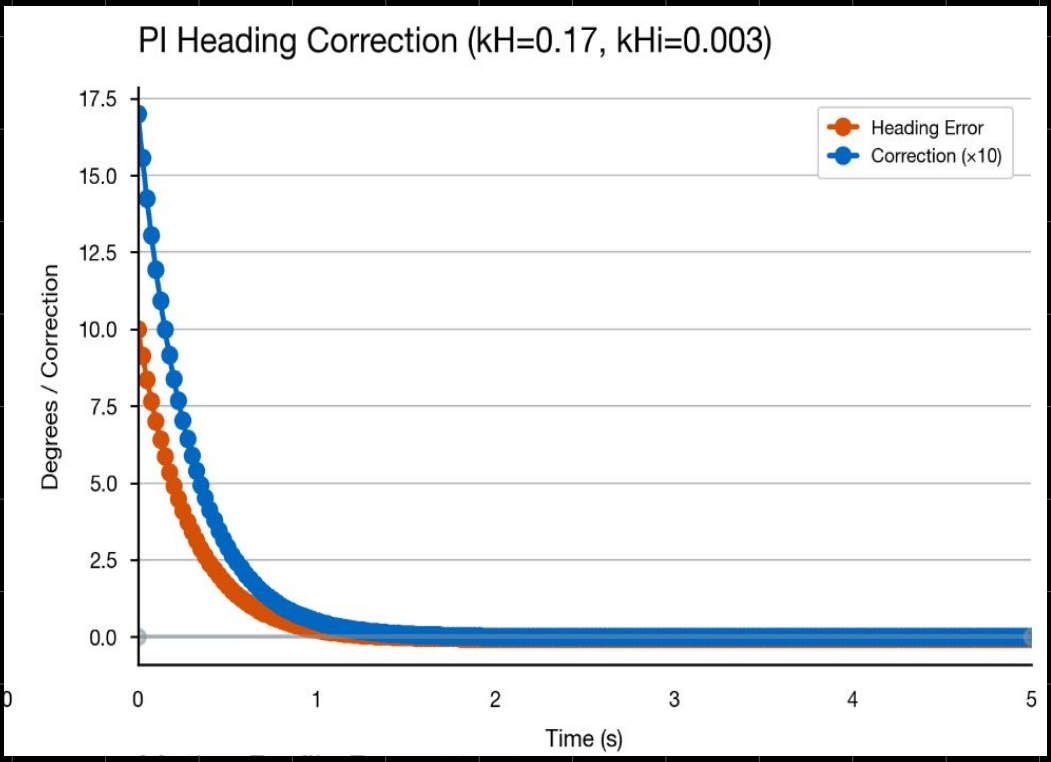
  `v = Vmax × 0.5 × (1 + cos(π × progress))`



Programming

Heading Correction:

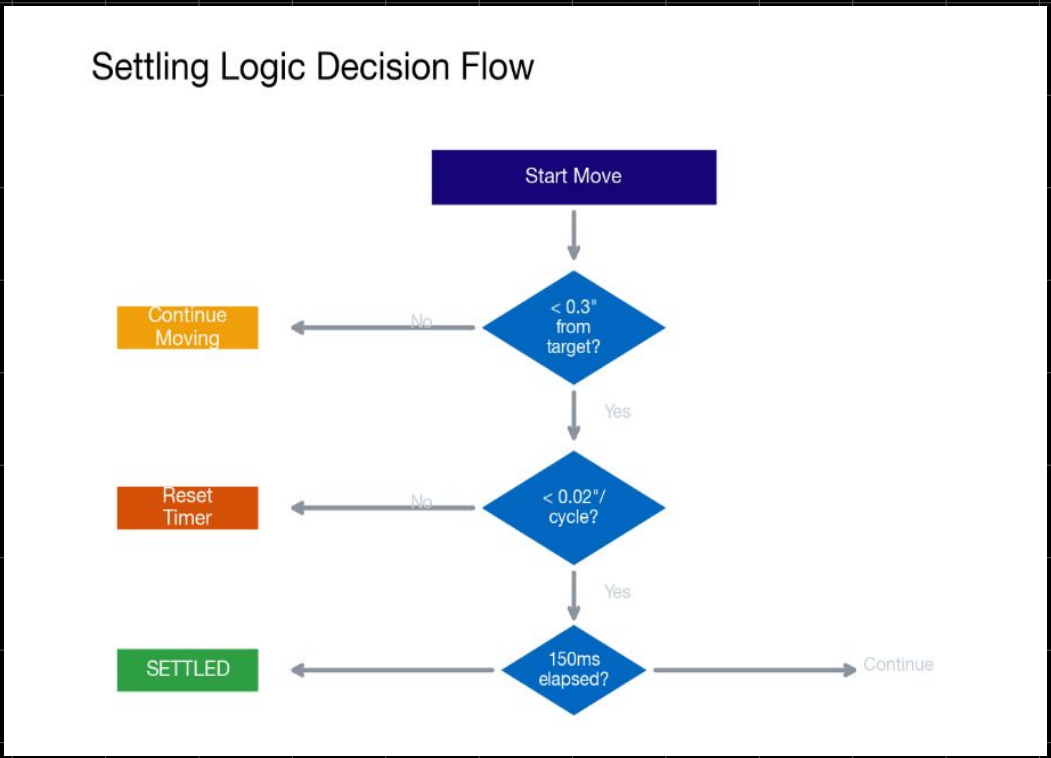
- Instead of using all 3 PID values of correction, since we aren't using odometry, we decided to only use 2 for heading.
- Proportional Gain (kH = 0.17): Corrects heading error
- Integral Gain (kHi = 0.003): Eliminates steady-state error
- Integral Capping (±50% of max): Prevents integral windup



Programming

Settling Logic:

- Detects when within 0.3 inches of target
- Requires < 0.02 inch/cycle movement for 150ms
- Prevents overshoot and oscillation



Programming

AUTONOMOUS PROGRAMMING

Autonomous Selection System

- Touchscreen selects one of 10 routines
- Stored in current_auton_selection (0–9)

```
int previous_selection = -1;

const int cols       = 5;
const int screen_w   = 480;
const int screen_h   = 240;
const int col_width = screen_w / cols;

const int red_height  = screen_h / 2 + 6;
const int blue_height = screen_h - red_height;

const char* labels[10] = {
  "R_right","R_left","R_right_F","R_left_F","R_solo",
  "B_right","B_left","B_right_F","B_left_F","B_solo"
};

Brain.Screen.setFont(vex::fontType::mono20);
```

Programming

When competition for the field is on, the robot automatically starts as the chosen one.

```
while (!auto_started)
 {
   if (current_auton_selection != previous_selection)
   {
     previous_selection = current_auton_selection;
     for (int i = 0; i < 10; i++)
     {
       int col = i % cols;
       bool top = (i < cols);
       int x = col * col_width;
       int y = top ? 0 : red_height;
       int h = top ? red_height : blue_height;

       vex::color fillColor = (current_auton_selection == i) ? white : (top
? red : blue);
       vex::color textColor = (current_auton_selection == i) ? red   :
white;

       Brain.Screen.setFillColor(fillColor);
       Brain.Screen.drawRectangle(x, y, col_width, h);

       Rect r { x, y, col_width, h };
       drawCenteredText(r, labels[i], textColor);
     }
   }
```

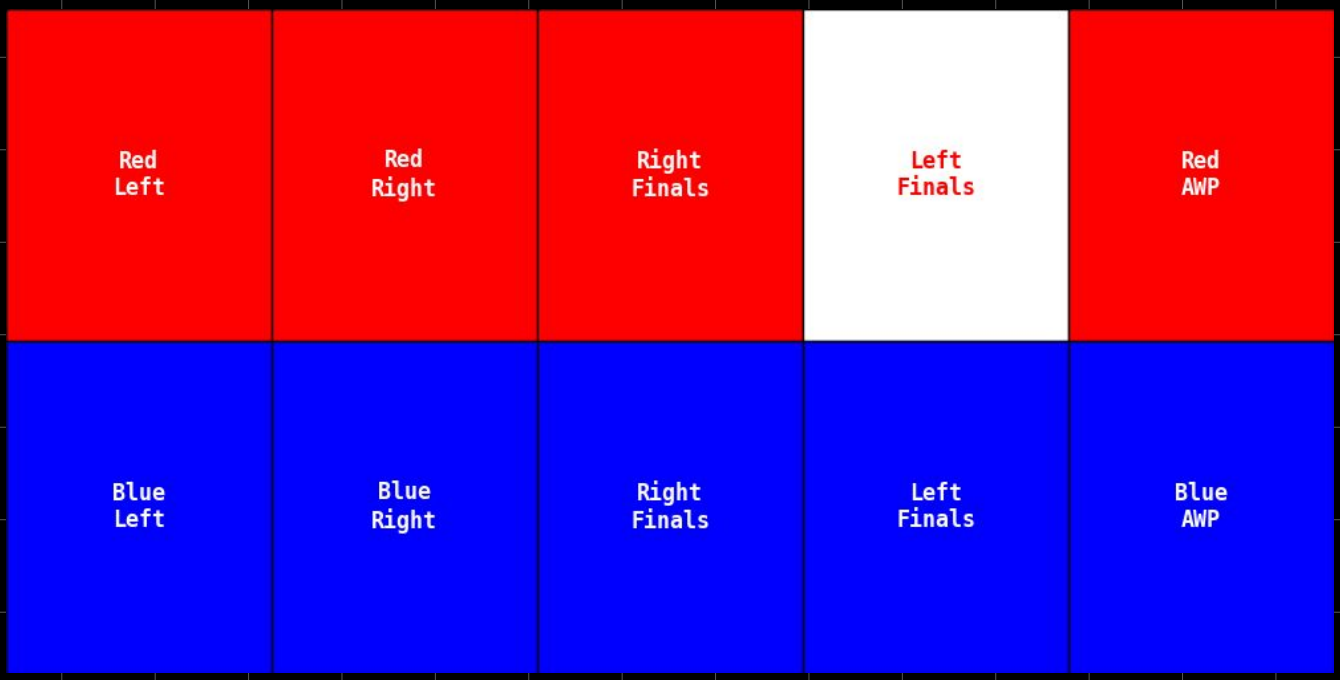
Programming

```
if (Brain.Screen.pressing())
   {
     int touchX = Brain.Screen.xPosition();
     int touchY = Brain.Screen.yPosition();

     int col = touchX / col_width;
     int row = (touchY < red_height) ? 0 : (touchY < red_height +
blue_height ? 1 : -1);

     if (col >= 0 && col < cols && (row == 0 || row == 1)) {
       current_auton_selection = col + row * cols;


       // enter next page (Dashboard)

       show_status_page(current_auton_selection);
     }

     wait(0.3, sec);
   }

   wait(20, msec);
}
```

This is the basic UI of the brain for choosing autons

User Interface

User Interface (on the brain)



1. Touch a box to select autonomous routine
2. Selected box turns **WHITE** with colored text
3. Touch again to enter Dashboard Mode

User Interface

Dashboard Display

Tab 1: INPUTS

Tab 2: MOTORS

Displays motor positions in real-time:

Color-coded elements (blue = selected, green = active)

PID

PID Control Tuning

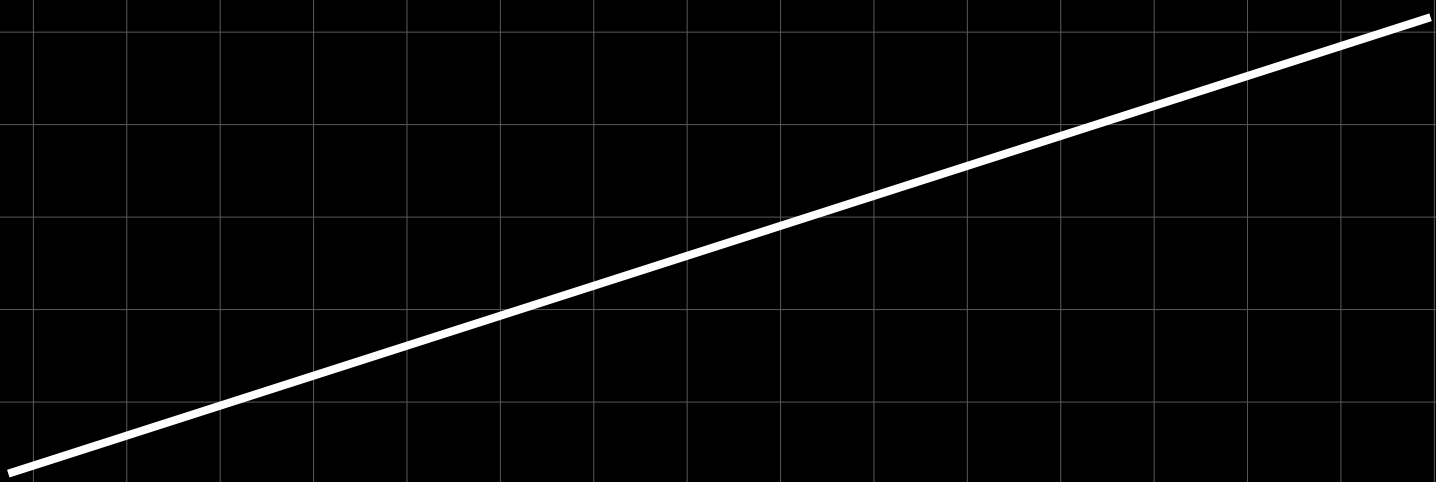
Default Constants

```
void default_constants() {
    // DRIVE PID
    chassis.set_drive_constants(12, 0.5, 0.004, 5, 20);
    //                          ^V  ^kP   ^kI    ^kD  ^startI

    // HEADING PID (gyro correction during drive)
    chassis.set_heading_constants(12, 0.25, 0.007, 5, 40);

    // TURN PID
    chassis.set_turn_constants(12, 0.35, 0.001, 3, 90);

    // SWING PID (one-side pivot)
    chassis.set_swing_constants(12, 0.5, 0.001, 2, 15);

    // EXIT CONDITIONS
    chassis.set_drive_exit_conditions(0.5, 0, 2000);  // ±0.5" error,
2s timeout
    chassis.set_turn_exit_conditions(4, 0, 3000);     // ±4° error,
3s timeout
}
```

PID

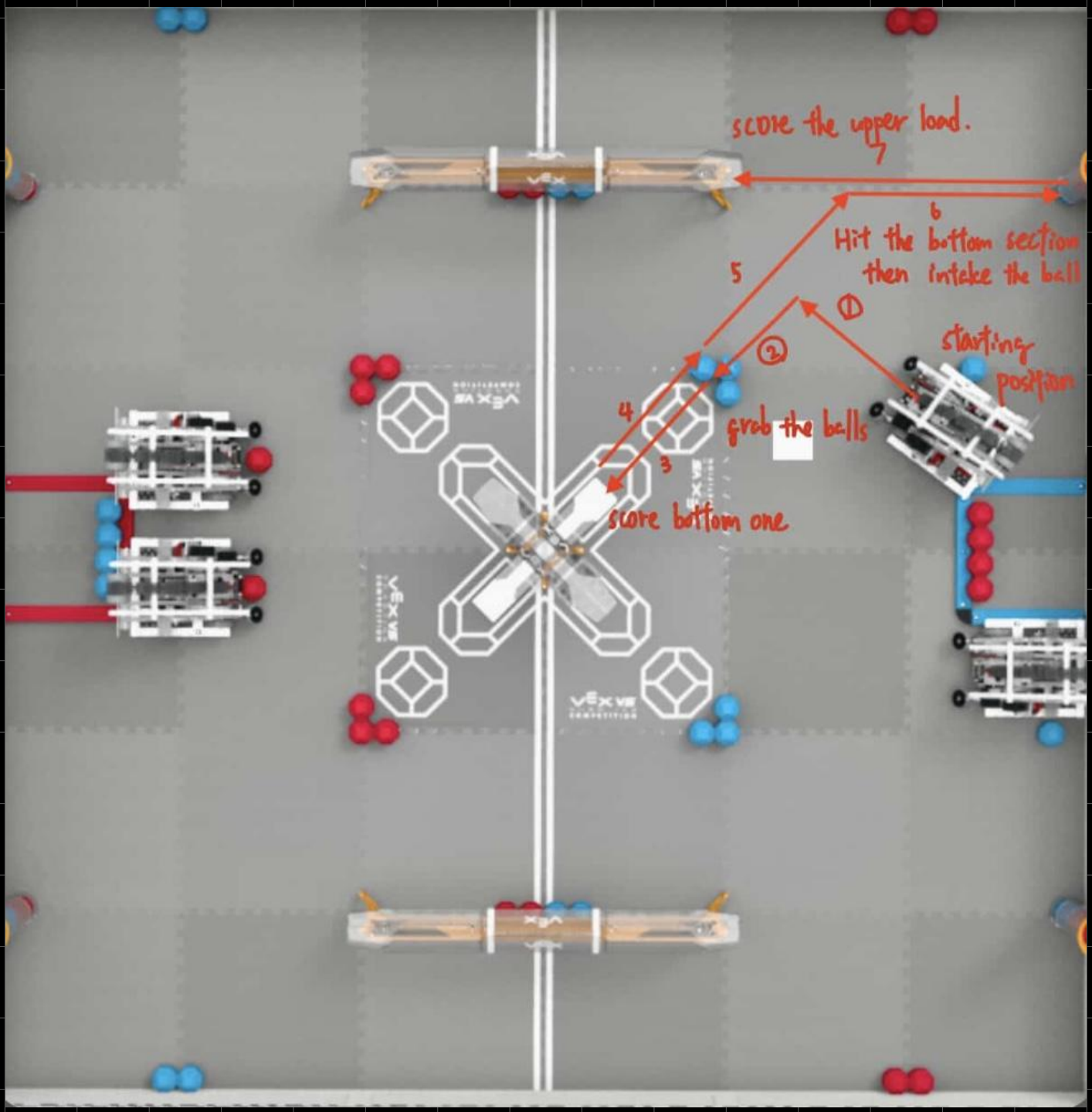
PID Parameter Meanings

Parameter	Value	Effect	Tuning Guide
kP	0.5–2.0	Aggressiveness of correction	Higher = faster response, more overshoot
kI	0.001–0.007	Eliminates steady-state error	Too high = oscillation
kD	2–5	Damping/smoothness	Higher = more stable, slower
startI	15–90	Error threshold to begin integral	Prevents windup at large errors



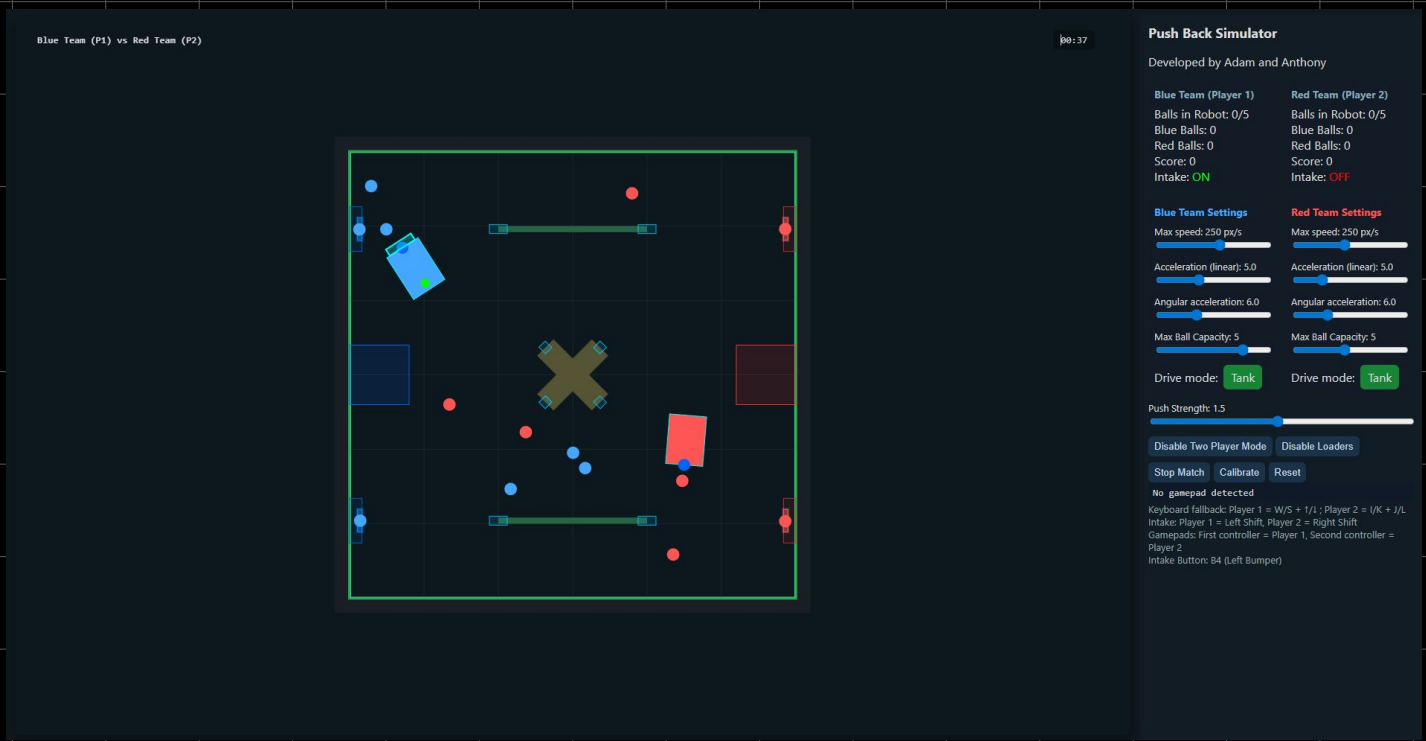
Autonomous Strategy

Autonomous Strategy



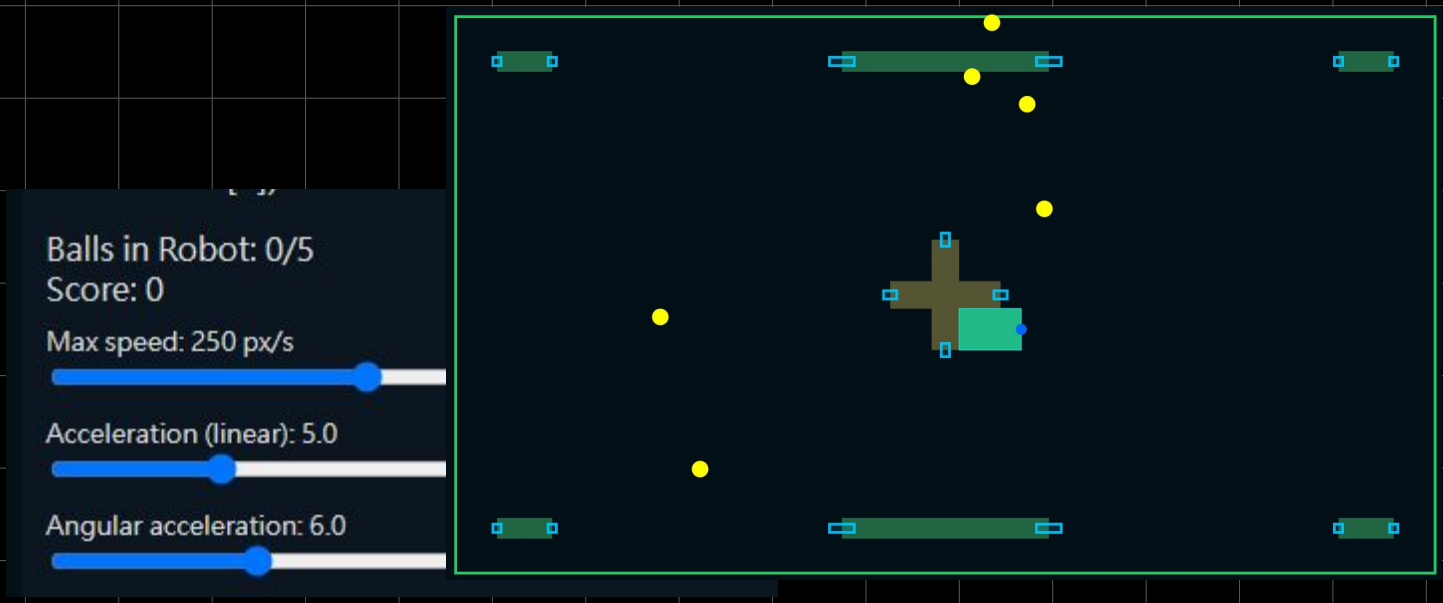
Simulator

Because our school limits our time to practice at our robotics classroom, combined with the fact that after we build our car we are mostly using time for programming, our drivers always say they don't have enough time to practice driving and think of new strategies. To allow drivers to practice effectively, we developed a 2D virtual simulator using HTML.



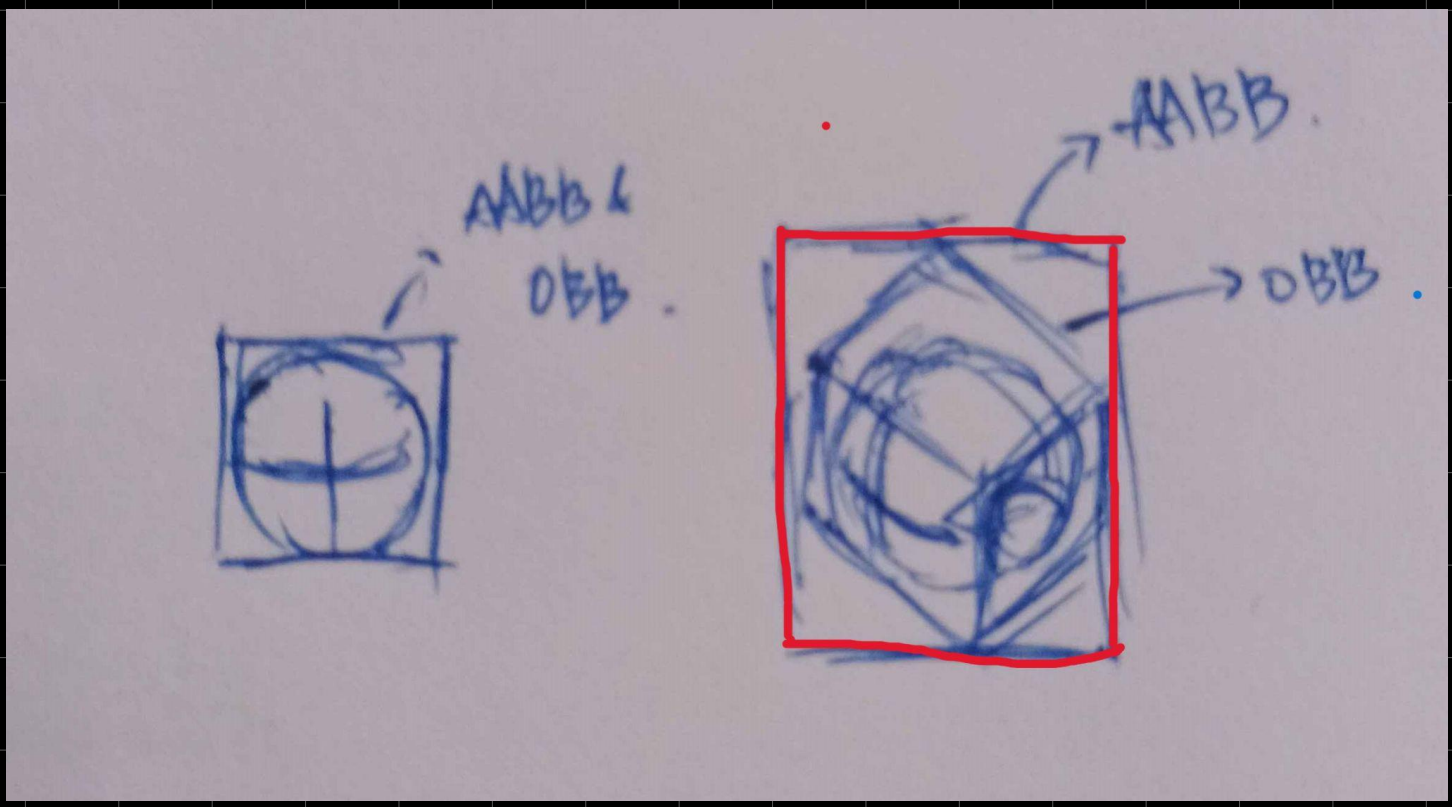
Simulator

The first prototype of our simulation. It looks nothing like the real field as of now as we're still experimenting what works and what doesn't. We added a few goals and expanded the field just to test the mechanics. So far it covers the essentials: customizable robot variables, automatic intakes, and ball spawning. One major issue we faced was the object hitboxes. As we coded on HTML instead of a game engine like Unity, we have to make one from scratch. We started with AABB for it is lightweight and cheap, but it doesn't always yield the desire results.



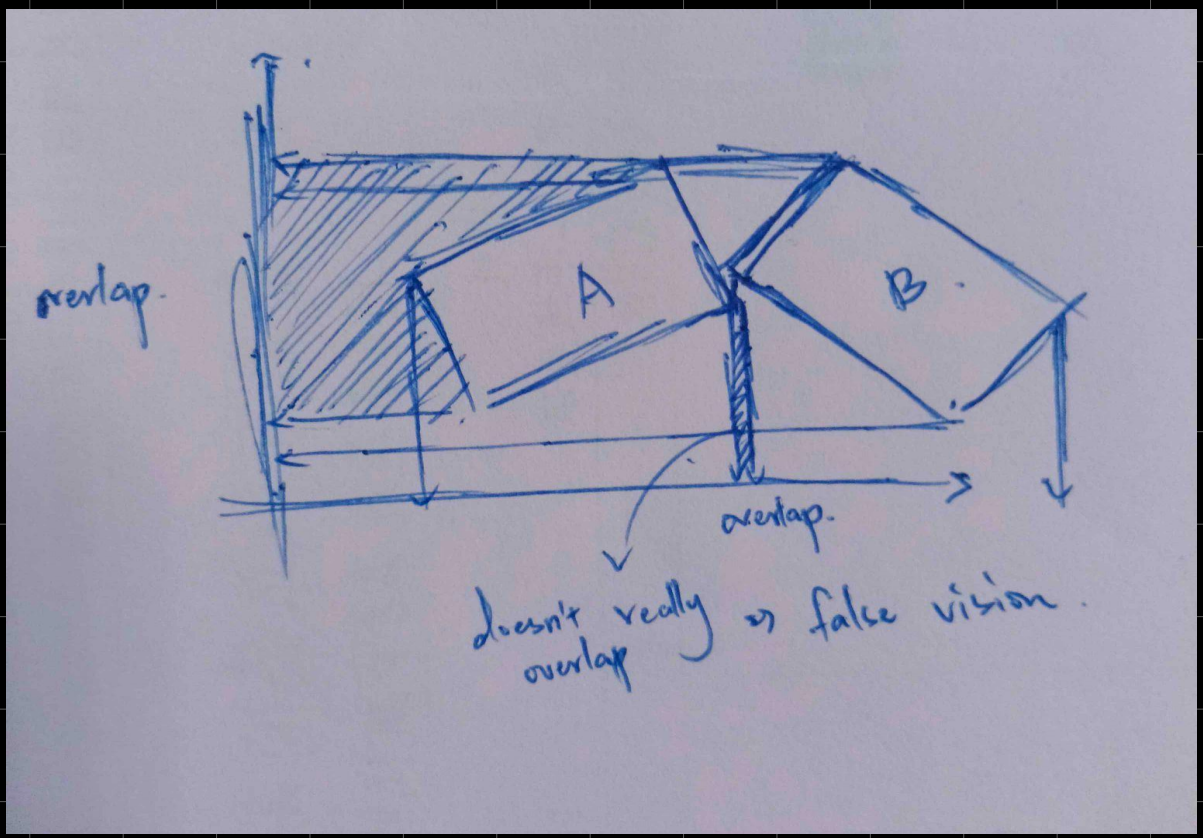
Simulator

The problem with AABB is that it is essentially four directional. Top, bottom, left, and right are all of its directions. But robots rotate. If we had to use AABB, the best way to do it would be to ues multiple boxes, like minecraft. But using multiple boxes kills its purpose: that its cheap and easy to do. Even with using multiple boxes, the hitbox is still going to look something like this(red).



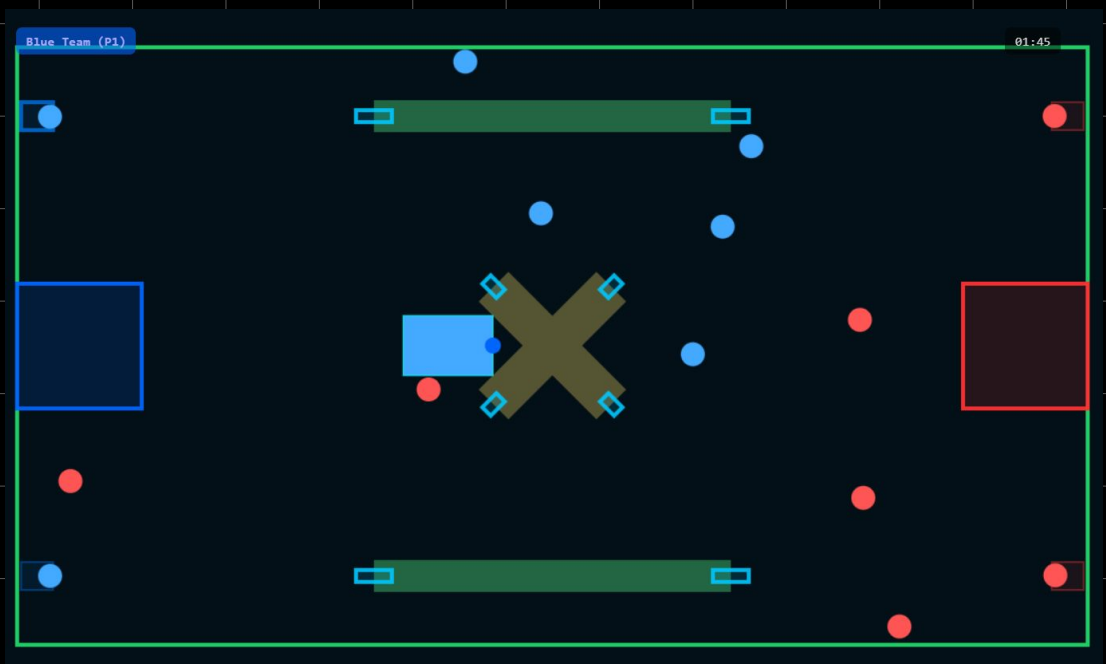
Simulator

This hitbox creates a false vision where the bot realistically should be touching something but isn't. To add salt to the injury, it is horrendously bad with circles. Which will clash with our later ideas of block physics. Where hitboxes should be met with a circular face would be pushed back by a flat one, and throws off the physics engine completely.



Simulator

The second version of the game. This version brought the field much closer to the actual field compared to the last. We decided to keep the field rectangular since a square setup feels too cramped and limits testing space. We also introduced team colors - red and blue - instead of the original yellow to better match real match conditions. We opted for the game to randomly spawn balls rather than fixed positions at the start, allowing continuous practice without having to restart the simulation.



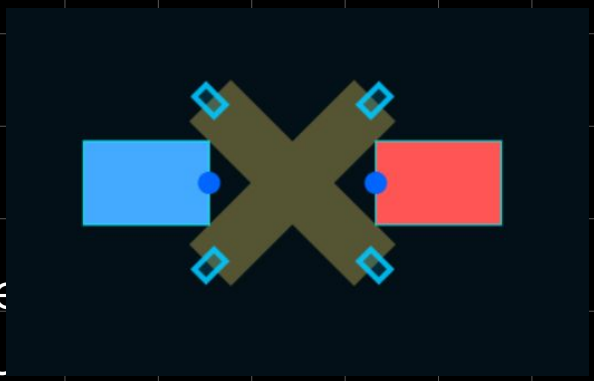
Simulator

We changed from autonomous intake to manual as it more closely represent real match conditions. The sky blue box in the front is the indicator for the area for which balls can be collected, and the green lights indicates that intake is active.

On the interface side, we also enhanced the player's ability to track performance with this new stats panel. It displays real time statistics like the current score, number of balls stored in the robots, and other essential data.

Simulator

We introduced several key elements to better simulate competitive match conditions. Players can now enable a two player mode, which allows players to plug in dual controllers for head-to-head competition. A much more enticing prospect than solo practice.



Additionally, players can now choose between tank drive or arcade drive to match their pre tactics used by other teams. Bu right now.

While testing, we realized camping at loaders takes all the fun away from the game. So we added the option to disable loaders, to prevent camping and infinite scoring tactics.

Simulator

For the hitbox, we figured out how the OBB method works and successfully coded a way to make balls bounce off the robot when not collected. You can also push robots that are scoring off the goal just like real life situations. A bar for customizing object's push strength was added to adapt to different robots.

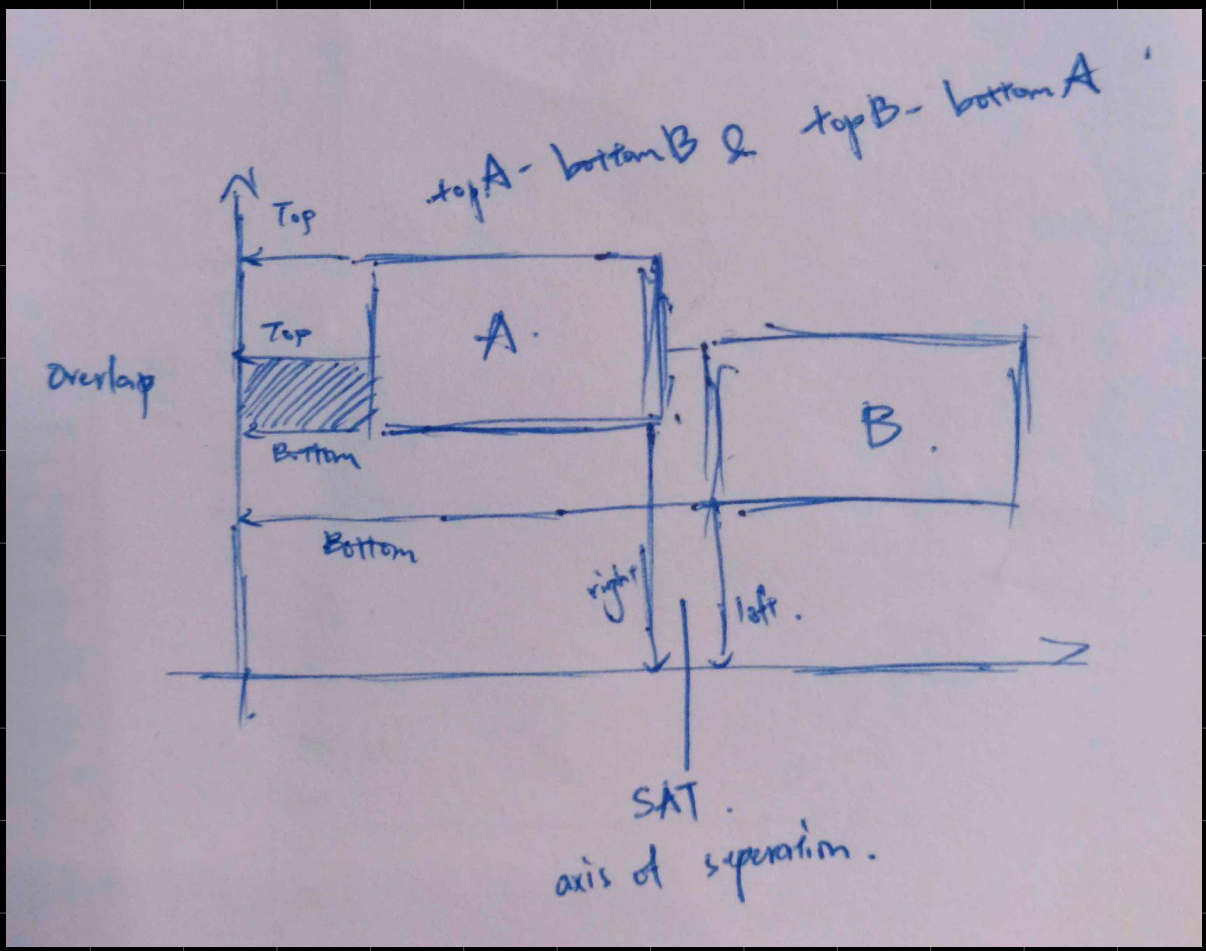
- Converting top-left coordinate system to center-based system with rotation

```
function aabbToObb(aabb){
  return{
    cx: aabb.x + aabb.w/2, cy: aabb.y + aabb.h/2, hx:
aabb.w/2, hy: aabb.h/2, angle:0
  };
}
```

We converted to center-based coordinates becuase the rotations doesn't rotate around corners and it simplifies calculations.

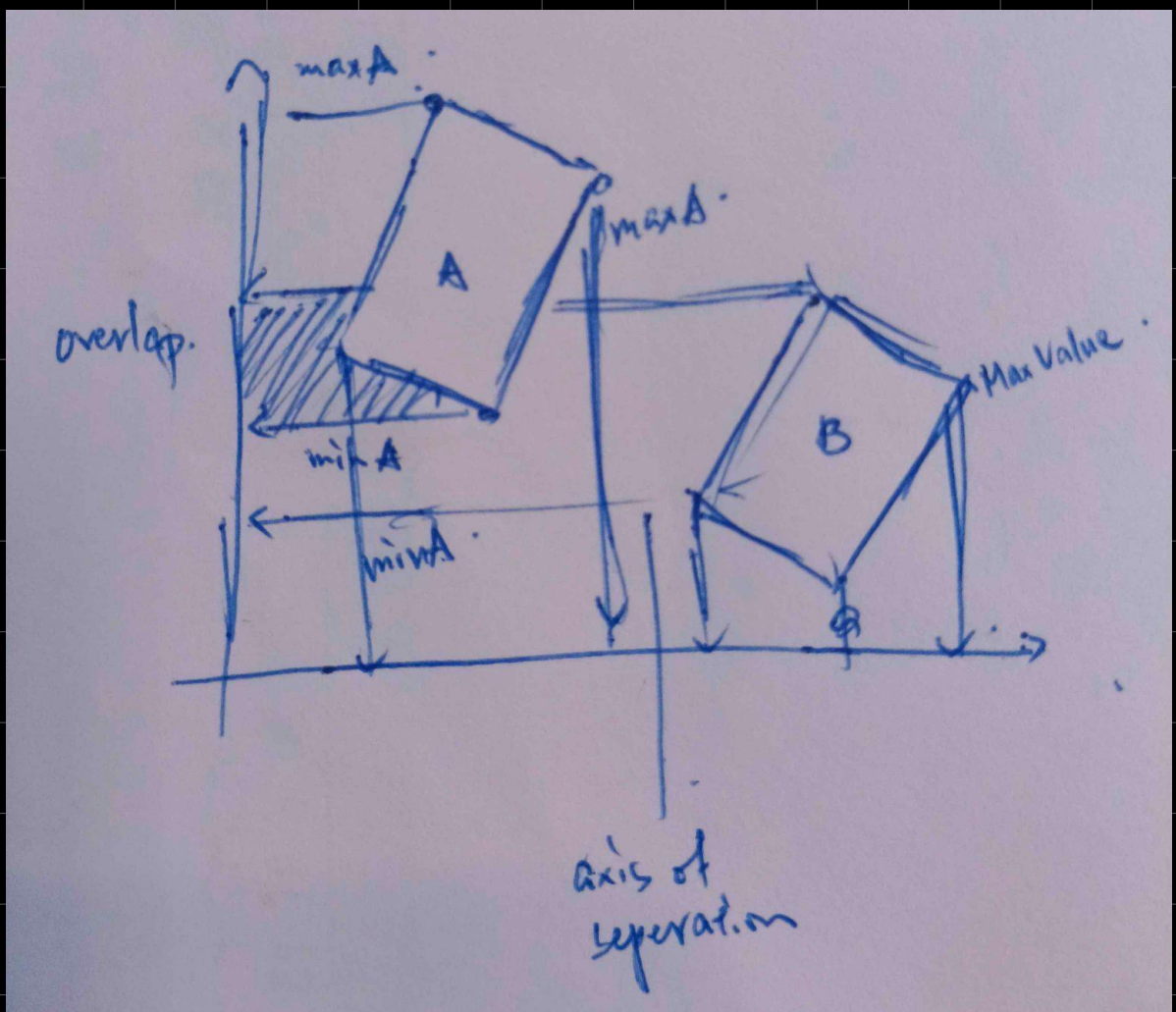
Simulator

To understand OBB, we must first glance at its underlying theory: the Seperation Axis Theory (SAT). Using simple maths we can check if the boxes overlap. By checking if the top of A overlaps the bottom of B and vice versa, we can see there is an overlap between topB and bottomA. But, the theory states if there is a possible way that the objects don't overlap, it doesn't. Now take a look at the right and left. The x axis doesn't overlap in this case. Hence these boxes don't collide.



Simulator

But what if the objects rotated? How do you determine its top and bottom? Great ask. In such situations, we find the maximum and minimum point of the object's x and y axis, and then check for overlapping. If we can draw a line that separates the two boxes, the objects don't collide, as illustrated below.



Simulator

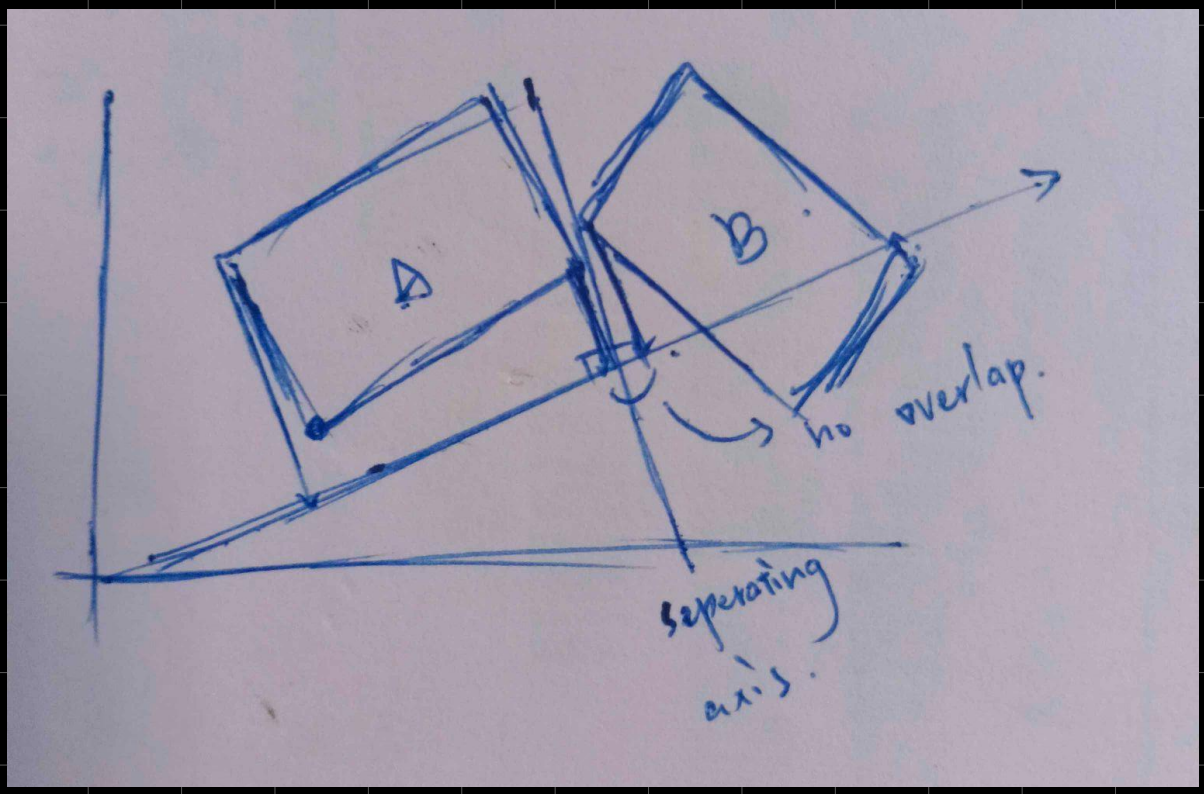
For any convex polygon, the only direction one needs to check for separation are the ones perpendicular to each of its edges. Our OBB (a rectangle) has **four edges**, but the **opposite edges** are parallel and share the same perpendicular direction. Therefore, each OBB only contributes to 2 unique axes. Since we have 2 OBBs (A and B), we need to check 4 axes. These locations are simply the X and Y directions of the robot, which are calculated by using the sin and cosine of the robots rotation angle.

```
function obbObbPenetration(A, B){
  const axes = [];
  const aAx = {x: Math.cos(A.angle), y: Math.sin(A.angle)};
  const aAy = {x: -Math.sin(A.angle), y: Math.cos(A.angle)};
  const bAx = {x: Math.cos(B.angle), y: Math.sin(B.angle)};
  const bAy = {x: -Math.sin(B.angle), y: Math.cos(B.angle)};
  axes.push(aAx,aAy,bAx,bAy);
```

Simulator

Once we have an axis to test, we project both OBBs on it. It's like shining a light directly onto our rectangle, the shadow casts by the boxes represents its projection. If the shadows don't overlap then the robots don't intersect.

Mathematically speaking, these projections are found by using the dot product between each vertex of the robot and the direction of the axis. Normally you'll have to check all the corners and find the largest and smallest points.



Simulator

Code that first calculates if the boxes are overlapping, then it determines which direction the box should push something.

```
let minOverlap = Infinity;
  let smallestAxis = null;
  let axisSign = 1;
  for(const ax of axes){
    const projA = projectObbOnAxis(A, ax.x, ax.y);
    const projB = projectObbOnAxis(B, ax.x, ax.y);
    const overlap = Math.min(projA.max, projB.max) -
Math.max(projA.min, projB.min);
    if(overlap <= 0) return null;
    if(overlap < minOverlap){
      minOverlap = overlap;
      smallestAxis = {x: ax.x, y: ax.y};
      const dirX = A.cx - B.cx, dirY = A.cy - B.cy;
      const dot = dirX*ax.x + dirY*ax.y;
      axisSign = (dot < 0) ? -1 : 1;
      }
    } return {
      px: smallestAxis.x * minOverlap * axisSign, py: smallestAxis.y *
minOverlap * axisSign, penetration: minOverlap
    }
  }
```

Simulator

Continuing from the last page, it is not enough if we only know how far the box should move, we also need to know which direction the box should move.

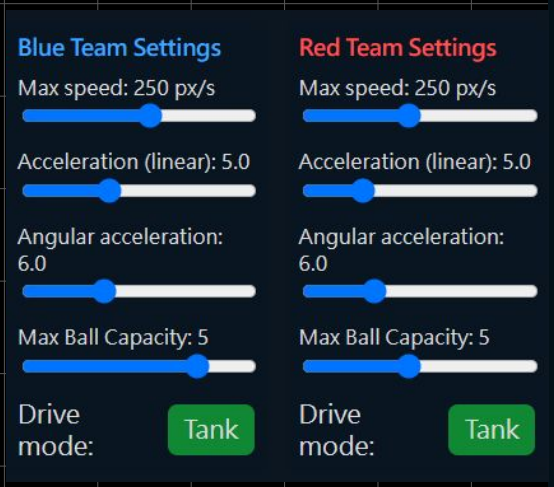
To figure this out, we look at the vector that goes from center of box B to box A, this is basically just a line that points from one box to another. We then compare the same vector with the axis that we're currently using to push the box.

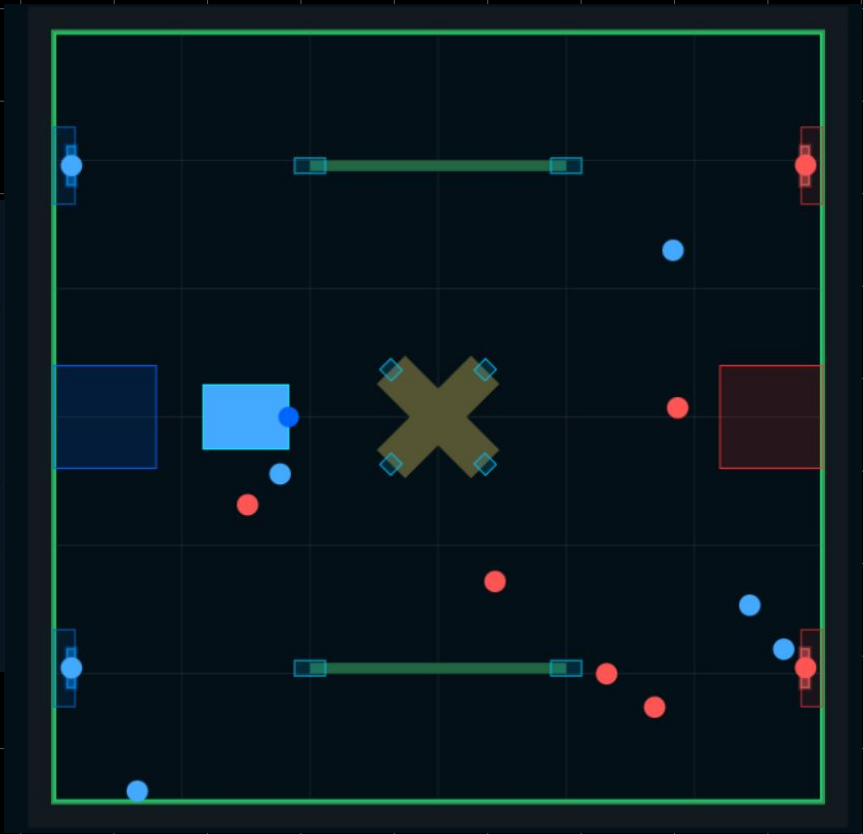
By taking the dot product we can know if the vector is pointing towards the same direction or the opposite. This gives us a +1 or -1 depending on the direction, ensuring that box A is always pushing away from box B.

```
const dirX = A.cx - B.cx, dirY = A.cy - B.cy;
      const dot = dirX*ax.x + dirY*ax.y;
      axisSign = (dot < 0) ? -1 : 1;
```

Simulator

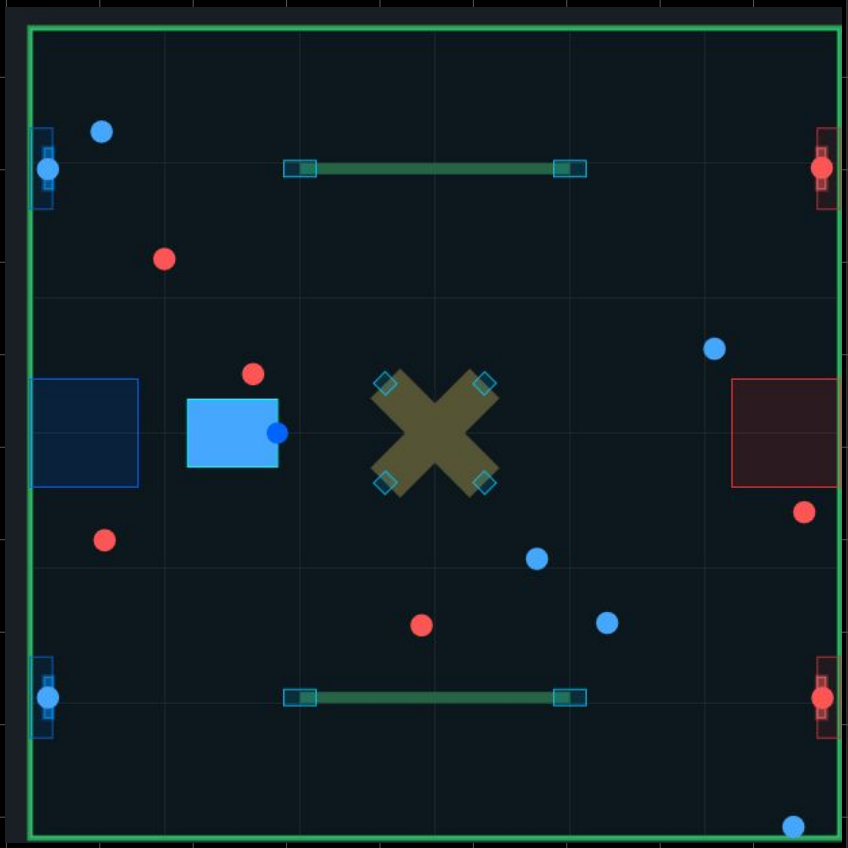
Final version of the simulator. We shrinked everything to fit the square field, which better represents the actual thing. We also fixed the problem where robots can't ues different drive schemes. So now players can choose between tank drive and arcade drive based on their preference. Overall, we polished every one of our functions to make the calculations more accurate and make the game run smoother.





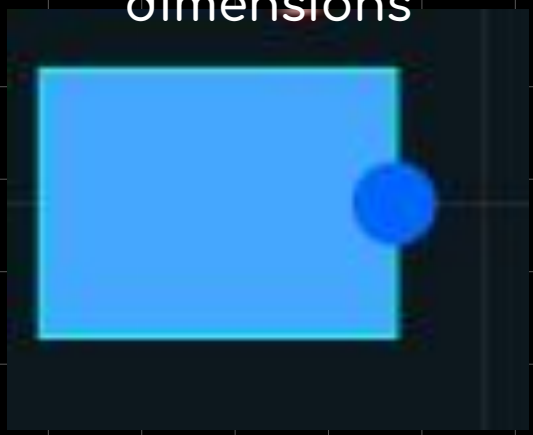
Simulator

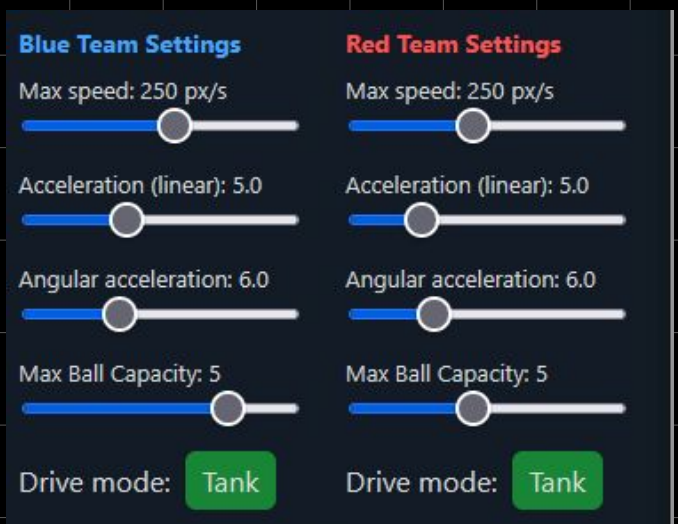
Field Setup



Timed like real matches

Customized robot dimensions





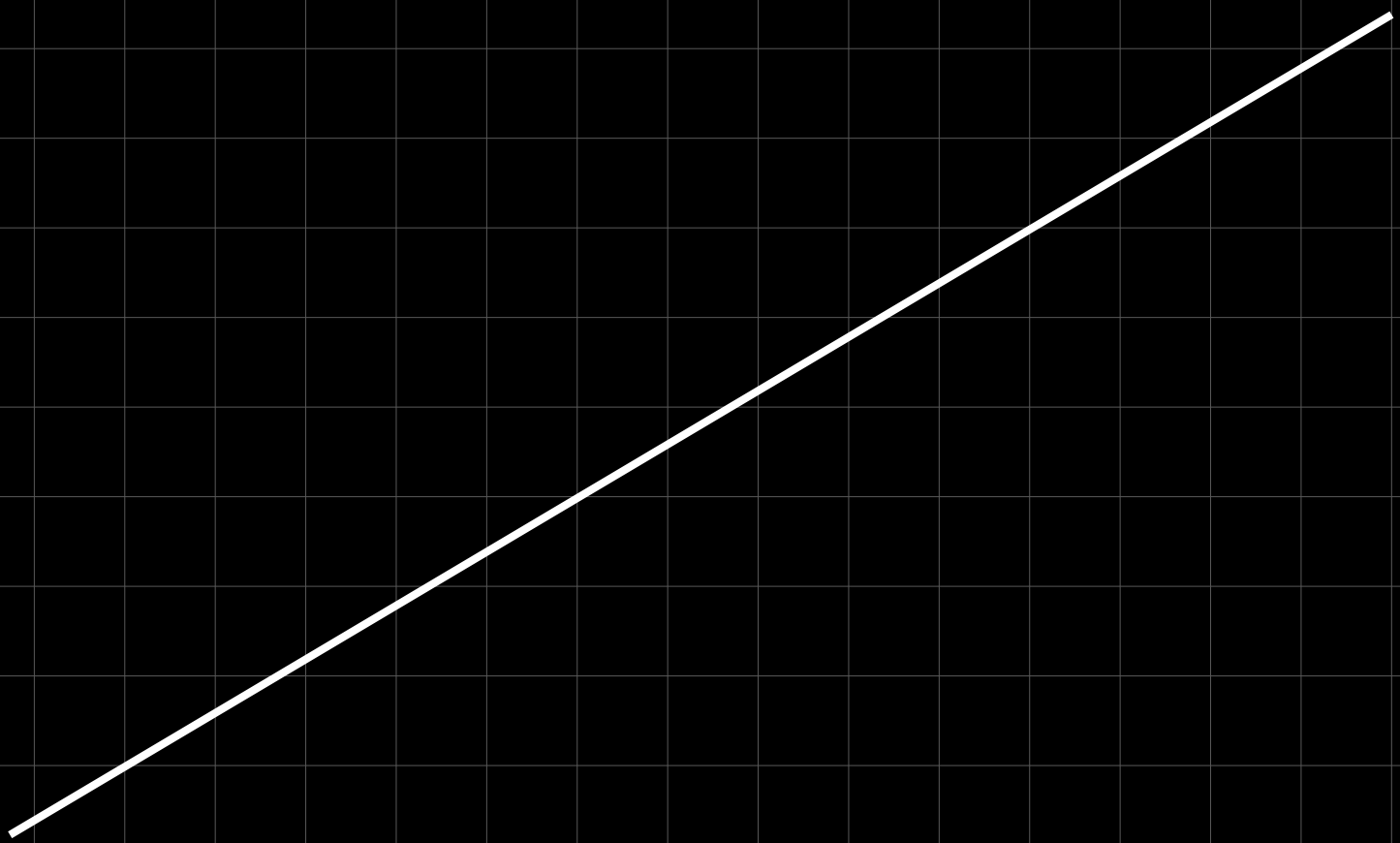
- Drivers can practice at anywhere with our simulator.
- We believe we're the first to make one in this season.

Customized robot drivetrain to simulate our own robot

Simulator

Overall, our simulator features:

- Compatibility with vex controllers
- A 2-player battle mode for practice
- 2D block collision physics for realistic interactions
- Accurate scaling of the field, objects, and our robot
- Scoring mechanisms for both middle and long goals
- Functional loaders for ball manipulation
- Precise driving timers for performance tracking
- Mode switching between tank drive and arcade drive
- Customizable robot features for opposing alliances



We are listing out what we should pack up for every event.

Storage	Item	Qty	✅ Packed
Big Box	Robot	1	☐
	Team license plates	4 per team	☐
	Controllers	1+ spare	☐
Large Red Box	Air pump	2	☐
	Batteries	4 + all spares	☐
	Battery charger	1 + all spares	☐
	Grinder	1	☐
	Grinder batteries	2	☐
	Label maker	1 (in box)	☐
	Safety goggles	9	☐
	Brain + controller	1+ spare set	☐

Storage	Item	Qty	✅ Packed
Bottom Drawer	Screwdrivers	7	☐
	Small green tools	all	☐
	Wrenches	all	☐
	Needle-nose pliers	2	☐
	Diagonal cutters	2	☐
	Pliers	1	☐
	Small & large L hex wrenches	for screws	☐
	Green wire crimping clips	all	☐
	Zip ties	all	☐
	Rubber bands	all	☐
Top Drawer	Rubber wheels	10	☐
Top Drawer	Red sprockets	16 24T	☐

Storage	Item	Qty	✅ Packed
Pneumatic Box	T-connectors	full bag	☐
	Switches	a few	☐
	90° elbow fittings	extra	☐
	Solenoid	4+	☐
	Air tanks	2	☐
	Air tubing	several rolls	☐
	Tubing cutter	1	☐
	Old solenoid cables	2→3, no blue	☐
	New cylinders	all	☐
Other Red Boxes	standoffs	all	☐
	Screws	all	☐
	Shaft collars & spacers	all	☐

TAI VEX PRACTICE

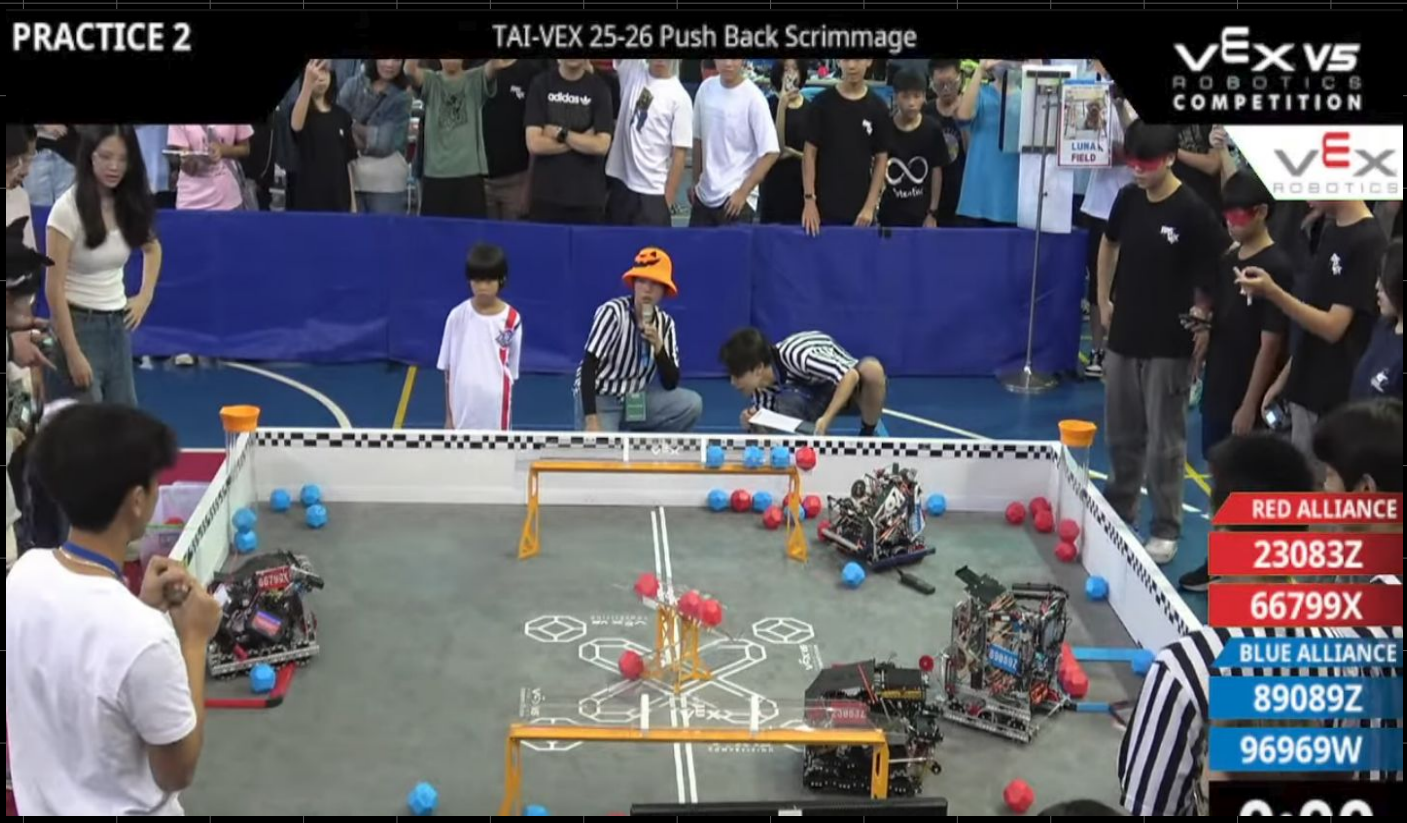
MATCH RESULTS

P2	23083Z 66799X	30	9	89089Z 96969W
Q5	23083Z 66994D	66	11	23083G 66995A
Q11	23083Z 17022J	80	3	3866F 96969Y
Q17	66996X 66995W	14	5	23083Z 66995A
Q22	10004B 10004N	25	50	23083Z 10004X
Q30	23083Z 17022X	30	50	89089Y 17022A
Q35	23083Y 1791V	63	28	66799X 23083Z
R16 6-1	23083Z 96969W	31	43	10004B 66799X

TAI VEX practice

23083Z 66799X	30	9	89089Z 96969W

We had our very first match in P2. From this game, we took notice of how crucial autonomous phase is in this season. Both alliances scored minimally throughout the match, but our victory ultimately came down to the two balls we successfully scored in the middle goal during the autonomous phase.



TAI VEX practice

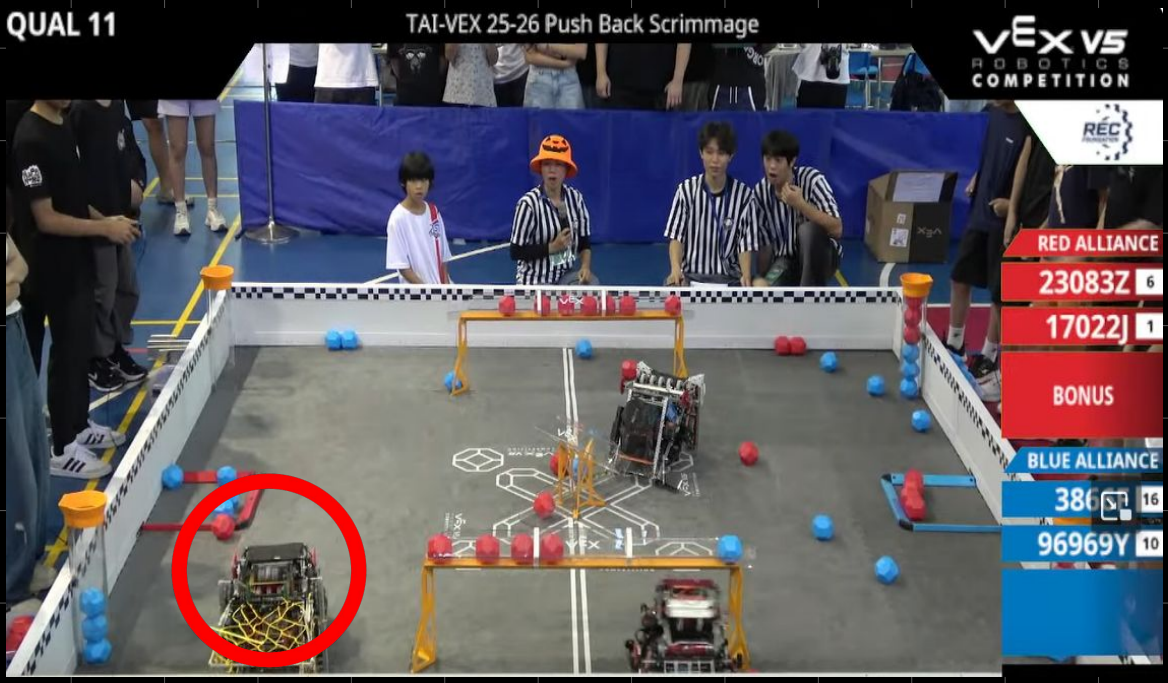
23083Z 66994D	66	11	23083G 66995A

Although Q5 wasn't our most efficient performance, it resulted in a win thanks to solid pre-match coordination. Our alliance planned for 66994D to prioritize the long goal while our robot focused on applying pressure and collecting points from the middle goals. While our execution fell short in scoring output, the strategy still proved effective in maintaining overall control of the match.

TAI VEX practice

23083Z 17022J	80	3	3866F 96969Y

Match Q11 provided a valuable learning experience for our team. We stciked to our established strategy, maintaining a division of roles in which our alliance partner prioritized scoring while our robot focused on defensive maneuvers. Early in the match, one of our opposing robots became immobilized, shifting the competitive dynamic significantly. As a result, our defensive efforts were concentrated on countering team 96969Y's offensive plays. Unfortunately, during one of these defensive interactions, our robot made unintended contact that caused the opposing robot to tip over, leading to our disqualification.



TAI VEX practice

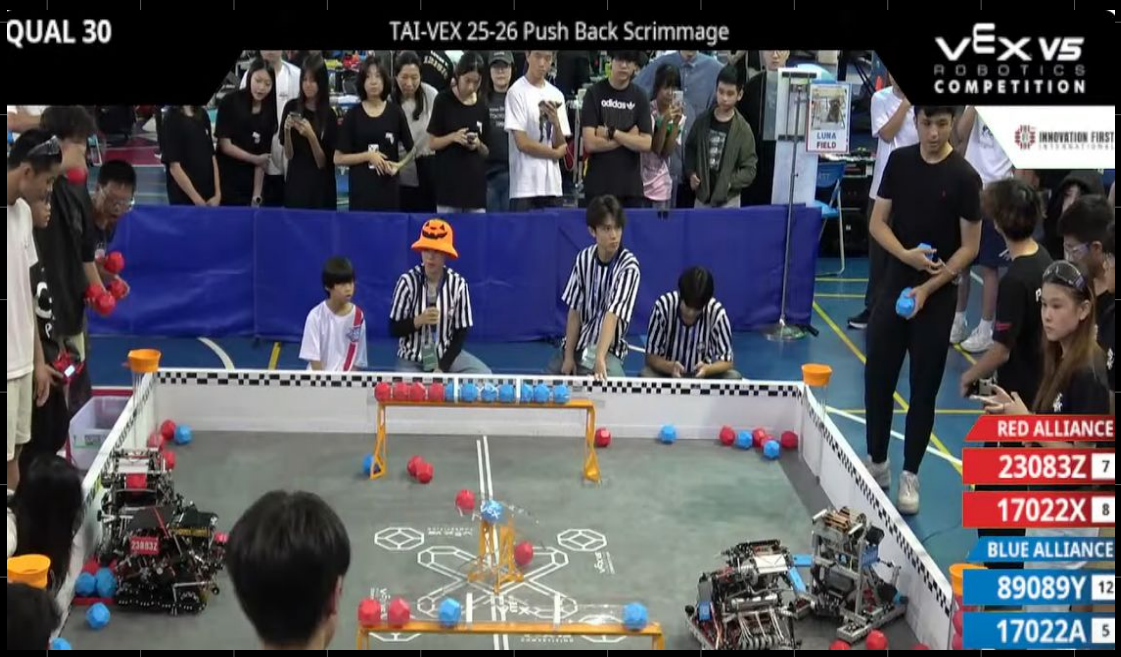
10004B 10004N	25	50	23083Z 10004X

Match Q22 follows the same strategy, securing another win. Although there was a minor conflict regarding entanglement, our team wasn't penalized.

TAI VEX practice

23083Z 17022X	30	50	89089Y 17022A

Match Q30 presented an interesting dynamic, as both our alliance and one of our opponents fielded nearly identical robot designs (17022X and 17022A). Our strategy centered on applying sustained defensive pressure, which proved effective for the most part. We successfully neutralized 17022A, preventing them from scoring throughout most of the match. However, our early loss in the autonomous phase placed us at a scoring disadvantage from the start, which we failed to fully compensate for during driver control. Additionally, our robot wasn't good for double parking. Had we executed a double parking in the endgame, we likely would have secured a narrow victory.



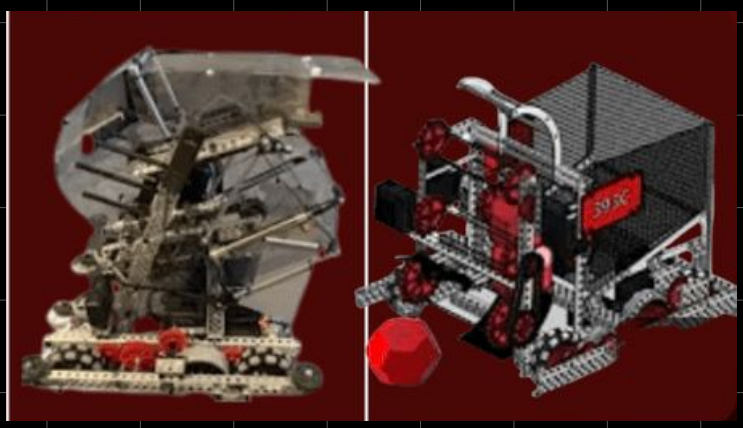
TAI VEX practice

23083Z 96969W	31	43	10004B 66799X

Match R16 #6-1 unfortunately marked the end of our run in this competition. We adjusted our strategy for this round, as our alliance partner also excelled in rapid cycling and defensive play. Consequently, our role shifted toward primary scoring. Our performance, however, fell short of expectations. This is partly due to intermittent intake lag. In hindsight, we realized that we could have capitalized on the cleared-out tube during the final moments of the match, as our alliance's strong defensive coverage provided a strategic opening. Instead, we focused too heavily on reclaiming the middle goal bonus, which ultimately cost us valuable points.

Optimal Alliance

Upon analyzing our performance in TAI-VEX, we came to the conclusion that our bot's archetype is well-fitted in the guerilla-supportive role. Our bot's agiability and fast cycling nature allows us to infiltrate into opponents' side field, sabotaging their game plan while chipping scores. Therefore, it's ideal for us to have an alliance that solely focuses on scoring and occupying the long goals. It's crucial for our alliance to clear out loaders and camp at one of the long goals, since our robot's main limitation lies in its ability to store a large number of blocks. This, we believe, is the primary cause for us to lose the game in the elimination bracket round.



Typical heavy S-Bot & Basket-Bot

Analysis of Others' Robots

Team: 10004X

Tournament: TAI VEX practice

Key Features:

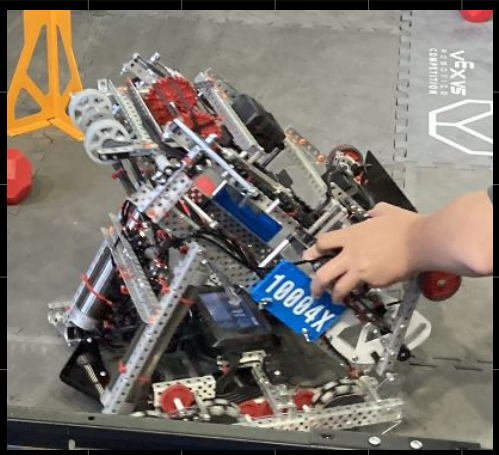
- Innovative design with descoring wings
- Not focused on high speed or large capacity
- Mechanism specialized for descoring and obstruction

Strategy & Performance:

- Used wings for last-second descores
- Obstructed goals and stalled match tempo effectively

Summary:

10004X leverages smart descoring mechanics and tempo control to offset its lower scoring speed, making it a deceptively strategic opponent.



Analysis of Others' Robots

Team: **66994D**

Tournament: TAI VEX practice

Key Features:

- Specialized long-goal control bot
- Well-engineered loader intake system
- Plastic stabilizing plate for consistent block intake
- Reduced risk of blocks exiting the field

Strategy & Performance:

- Focused on long-goal dominance
- Provided stable and reliable block control in matches

Summary:

66994D is a precise long-goal control bot with a highly reliable loader intake that strengthens long-goal scoring consistency.

Analysis of Others' Robots

Team: 1791V

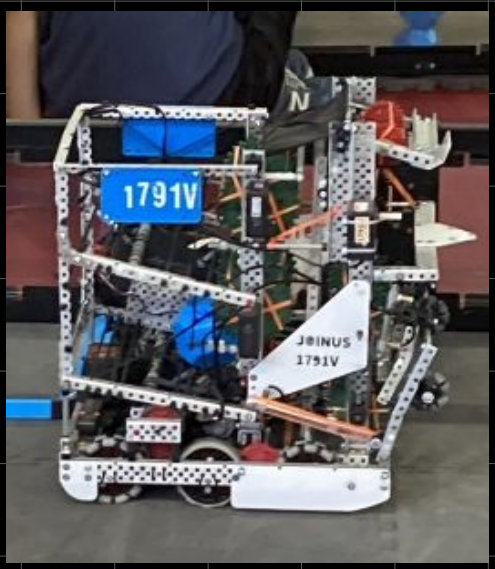
Tournament: TAI VEX practice

Key Features:

- Typical high-throughput basket bot design
- Optimized to take advantage of the large block count (88 blocks on Push Back field)
- Integrated color-sorting system to double scoring efficiency

Summary:

1791V is a high-capacity, quantity-focused basket bot with exceptional scoring potential, making them a consistently dangerous opponent.



Analysis of Others' Robots

Team: 17022X, 17022J, and 17022A

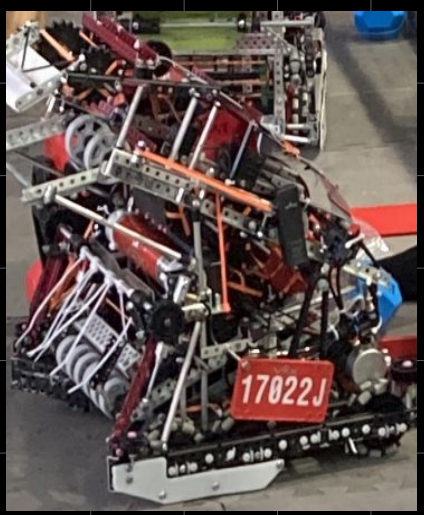
Tournament: TAI VEX practice

Key Features:

- Identical fast C-Bot build
- High-capacity block containment system
- Two-way-out scoring mechanism
- Optimized for rapid long-goal scoring

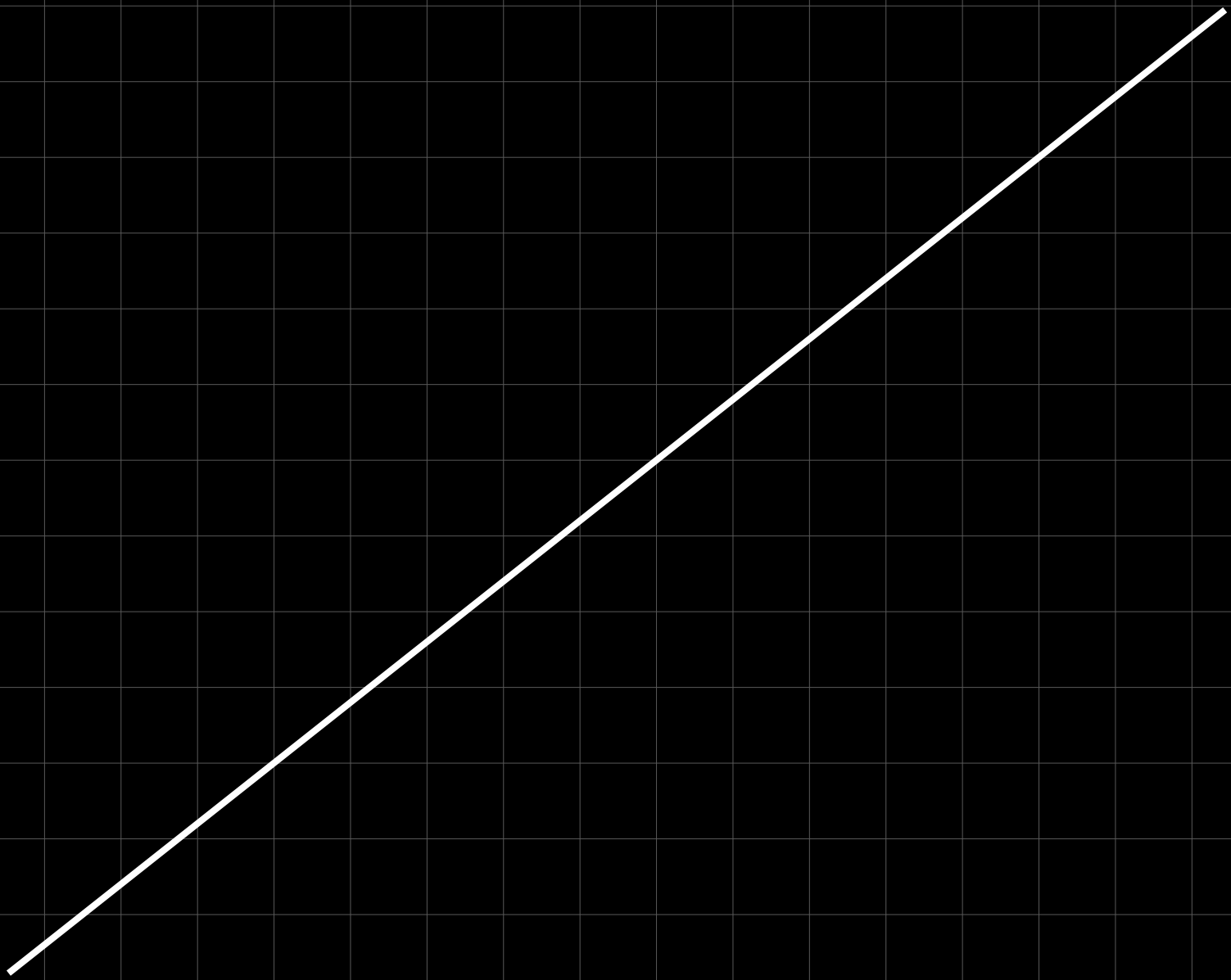
Strategy & Performance:

- Prioritized long goals throughout the match
- Maintained long-goal bonuses almost continuously
- Filled the entire long goal from the start of the match
- Relied on high speed and strong accuracy for dominance



Analysis of Others' Robots

Summary:
These teams used a fast, precise, and aggressive long-goal strategy that allowed them to consistently dominate matches through speed and efficient scoring.



05
ROBOT 2

Meeting Logs

2025/09/23 — 85 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- Reflection - New robot plan

Competition reflection:

After participating in the TAI VEX scrimmage last weekend, overall, our robot ran smoothly, but there were still some major issues. During a few matches, the robot failed to intake consistently after several collisions, which slightly lowered our scoring rate. The new JAR-Template control system worked reliably, and the drive response felt stable.

Since our robot was finished quite late, we didn't have enough time to properly test or tune our autonomous routines after verifying basic programming and functionality. As a result, we went into the event with limited auton reliability. We think this was mainly due to poor planning, but also because the competition was scheduled early in the season while we were still finalizing the design — we always aim to build the best robot possible, which often pushes testing later than planned.

Meeting Logs

Robot performance analysis:

The team agreed that the chassis was not stable and agile enough for both defense and offense and especially autons. The intake compression was sometimes too tight when the ball entered at an angle, causing jams near the second sprocket.

The drivers discussed again about curvature vs tank drive — curvature felt smoother when avoiding obstacles, but the braking felt unpredictable.
Improvements planning:

We thought that our robot should be changed from its chassis since when doing autons it often runs differently. We think that the chassis should be made looser and smoother. So instead of changing parts by parts of the car, we decided to rebuild a similar robot. Robot 2 is the bot mentioned above.

Reflection

After our TAI VEX event we realized that our robot had many defects.

1. Intake

The intake often jammed or slipped. This robot needs a **more forgiving compression system** that can handle off-center entries and still maintain smooth cycling.
This may include adjustable compression spacing, more elastic roller mounting, or wider intake geometry.

2. Structural Durability

Several matches revealed that its intake and chassis were slightly too rigid, leading to vibration and misalignment after impacts.
The new robot should focus on **reinforced yet flexible mounting** — especially for subsystems that experience collisions (intake arms, side plates, etc.).

Reflection

3. Center of Gravity and Stability

During defense, its higher center of gravity made it prone to tilting during collisions or rapid turns.

The next design should lower heavy components (batteries, pneumatics) and optimize weight distribution across the drivetrain for smoother acceleration and balance.

5. Endgame Capability

Robot 1 was limited in its ability to double-park quickly, which cost points in multiple close matches.

The next robot should integrate a more compact and efficient parking mechanism, possibly using a low-profile piston lift or foldable ramp system.

6. Adaptability in Strategy

The next robot should have **modular autonomous options**, allowing it to adapt between offensive and defensive roles depending on alliance needs.

Building Timeline

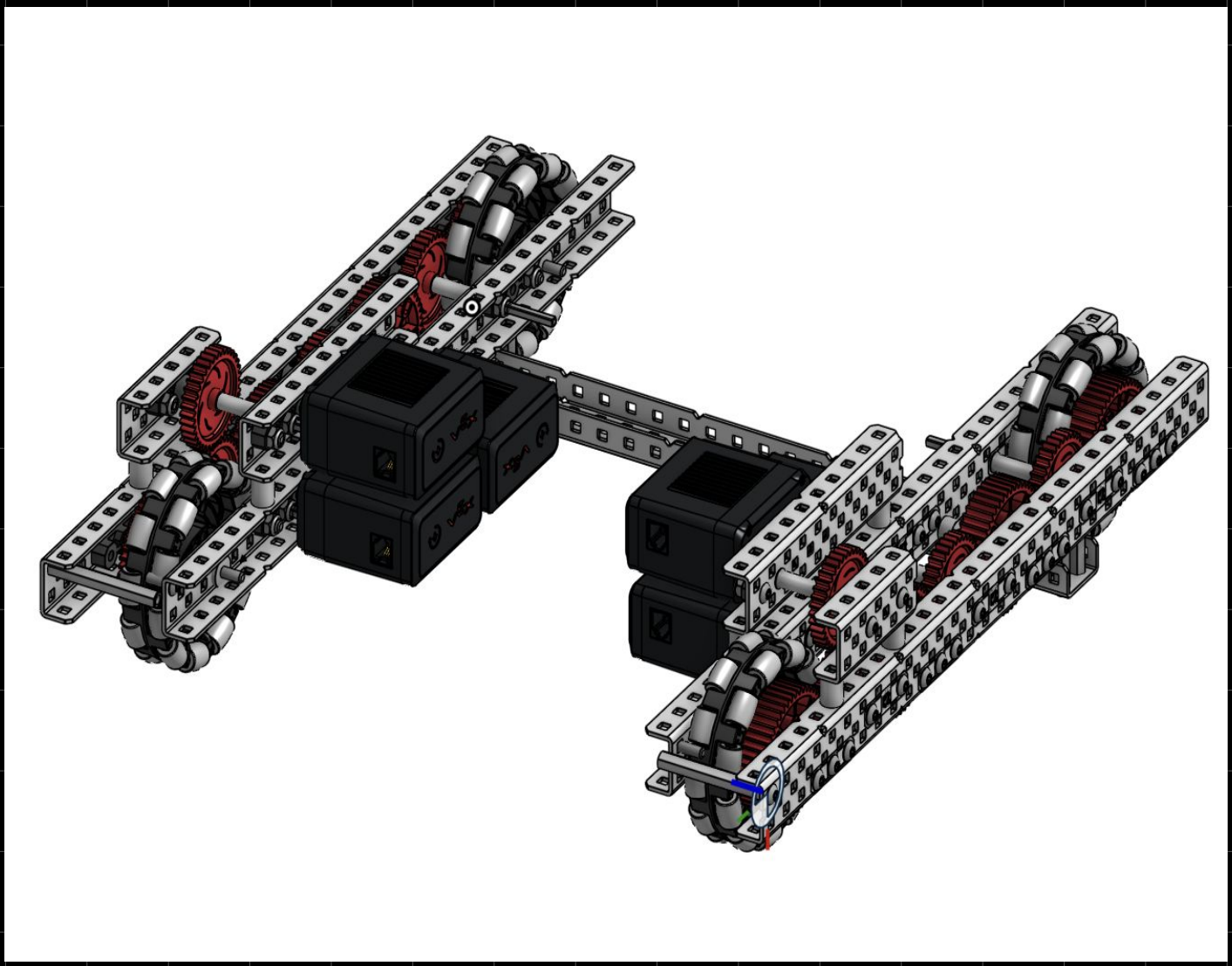
Tasks	9/29-10/6	10/7-10/15	10/16-10/22
Chassis			
Intake Design			
Sprocket			
Roof			
Mat, Plastic Boards, Descorer			
Programming			

<u>Tasks per member:</u>

- Chassis: Pennlaine, Adam
- Intake design: Anthony, Jefferson
- Sprocket: Pennlaine, Adam
- Roof: Anthony
- Mat, Plastic Boards: Pennlaine, Anthony
- Programming: Pennlaine, Adam

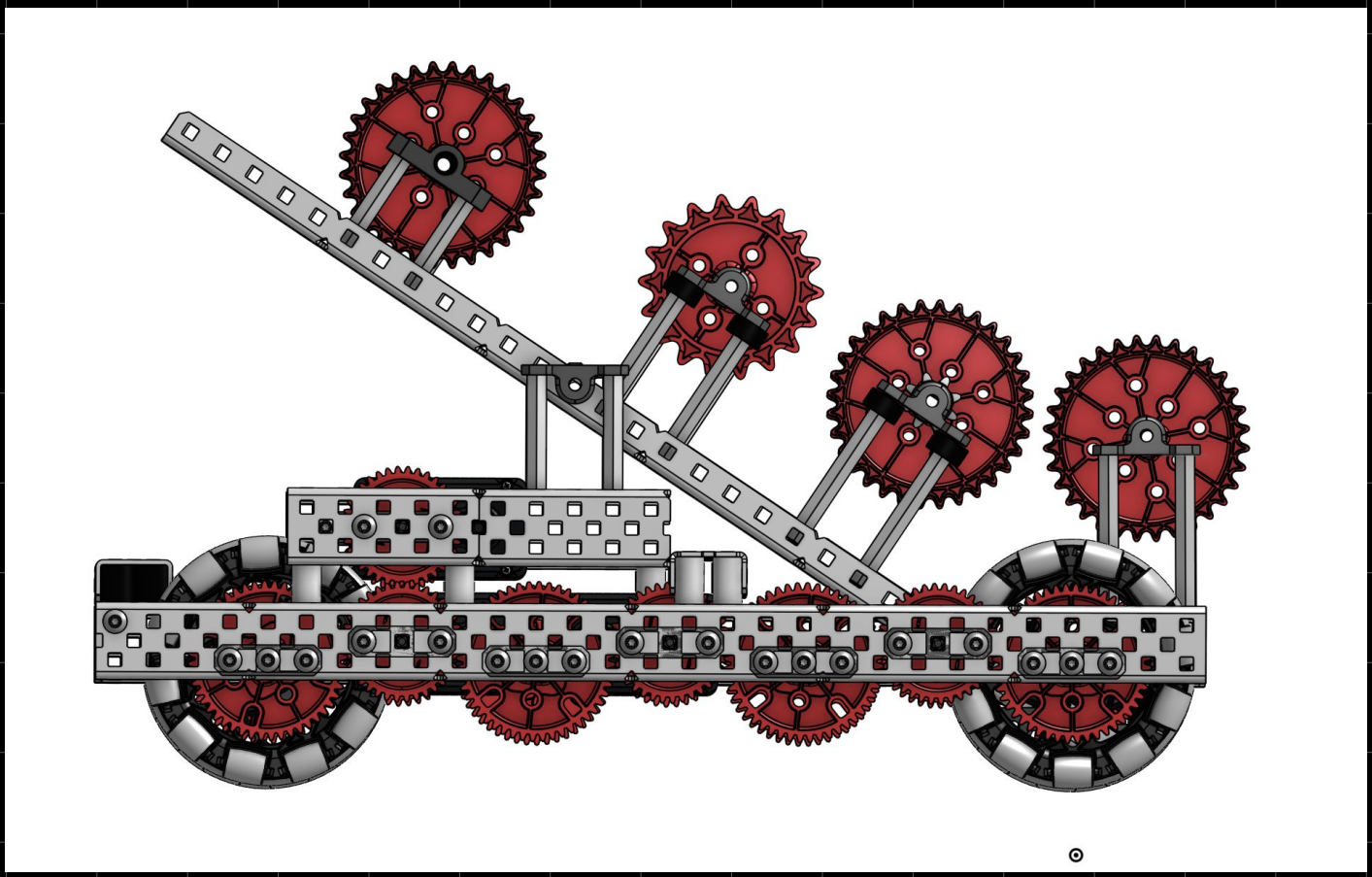
Chassis CAD

We finished the CAD of our chassis with new added details. And we are starting to build the super structure after having brainstormed it.



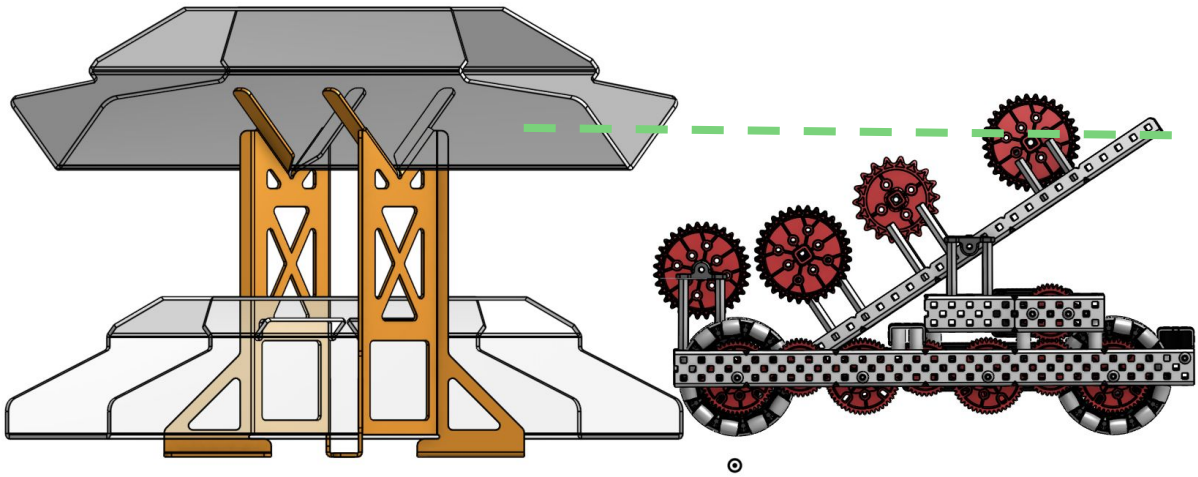
CAD

We started to CAD the super structure, like the first robot, we start from the ramp with sprockets for a series of consistent intake.



CAD

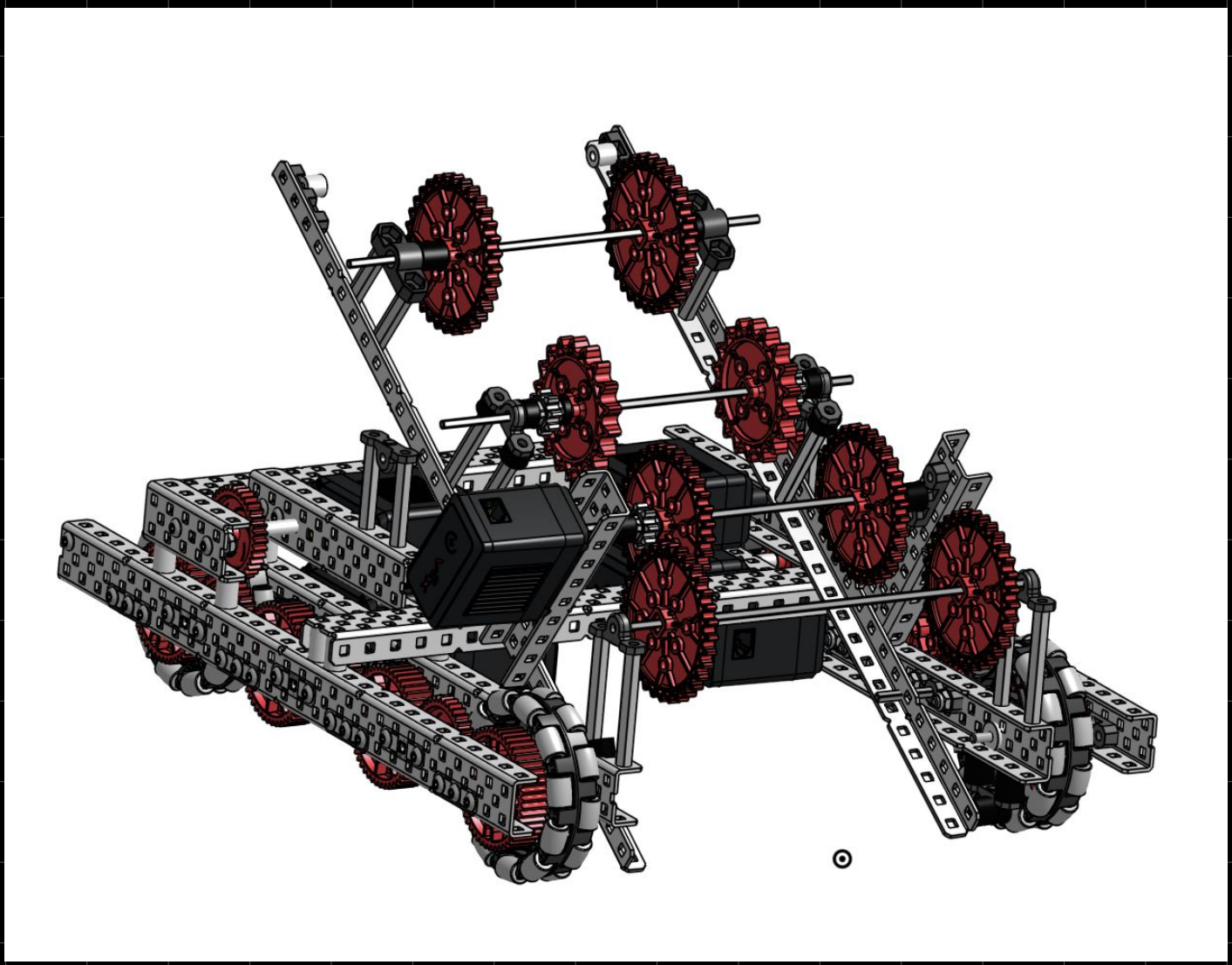
We should adjust the angle of the ramp to that the end of the ramp is about the height of the middle upper goal. Similar to Robot 1, we want the second level, the shooter part to lie horizontally, to minimize the height of the bot, and that exactly score into the the moddle upper goal.



CAD

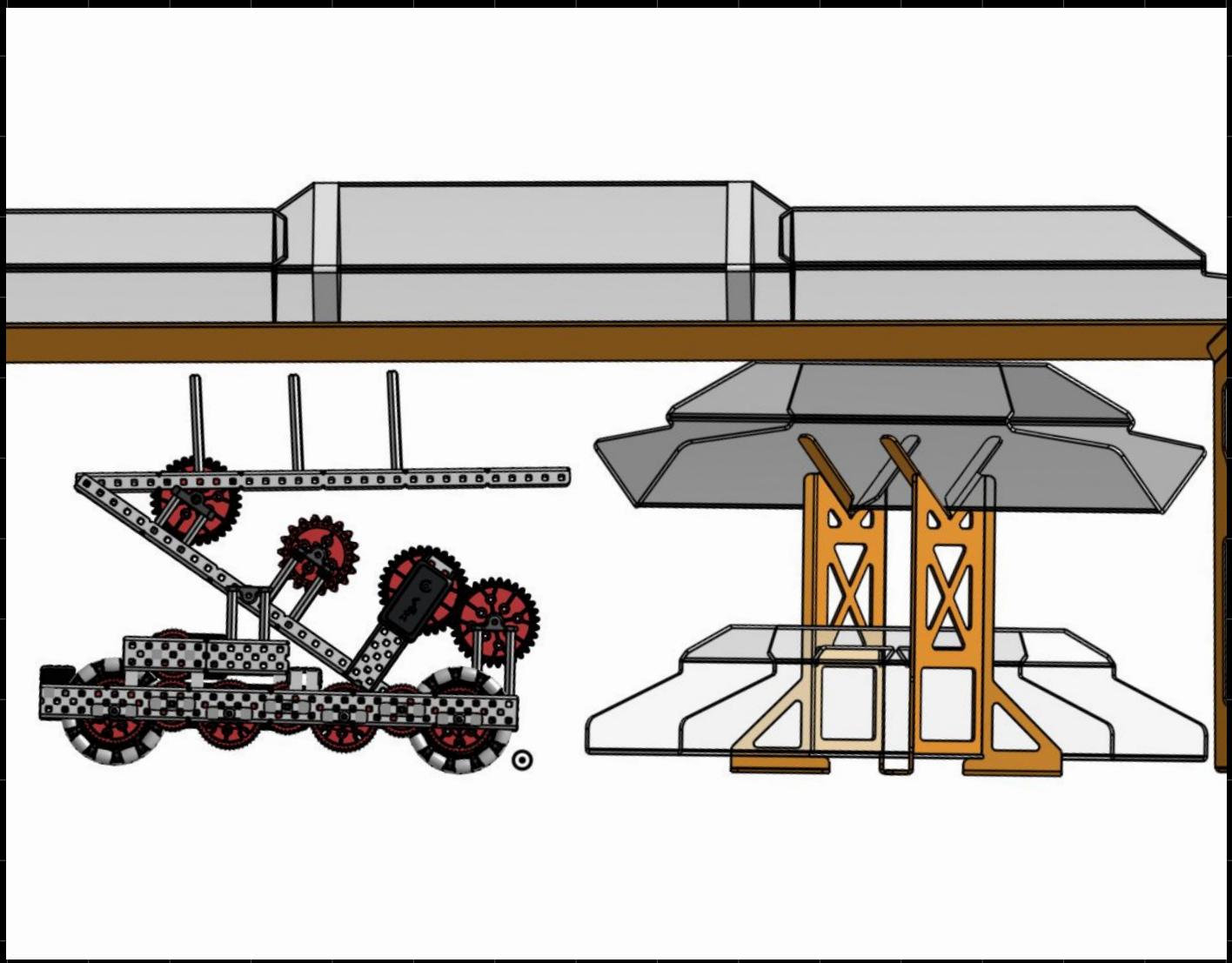
We adjusted the sprockets spacing and added structures to attach motors.

We also adjusted the intake sprockets spacing and widths.



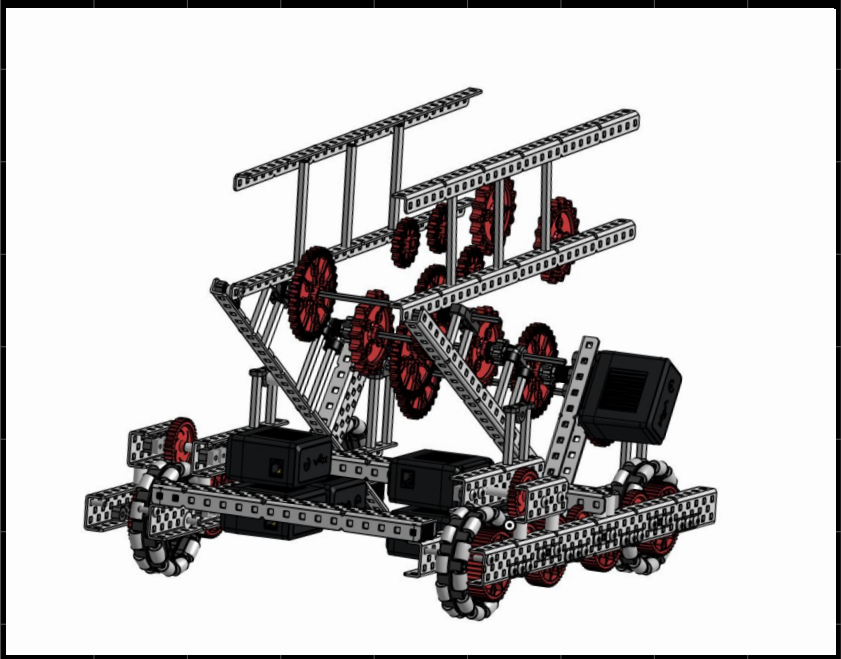
CAD

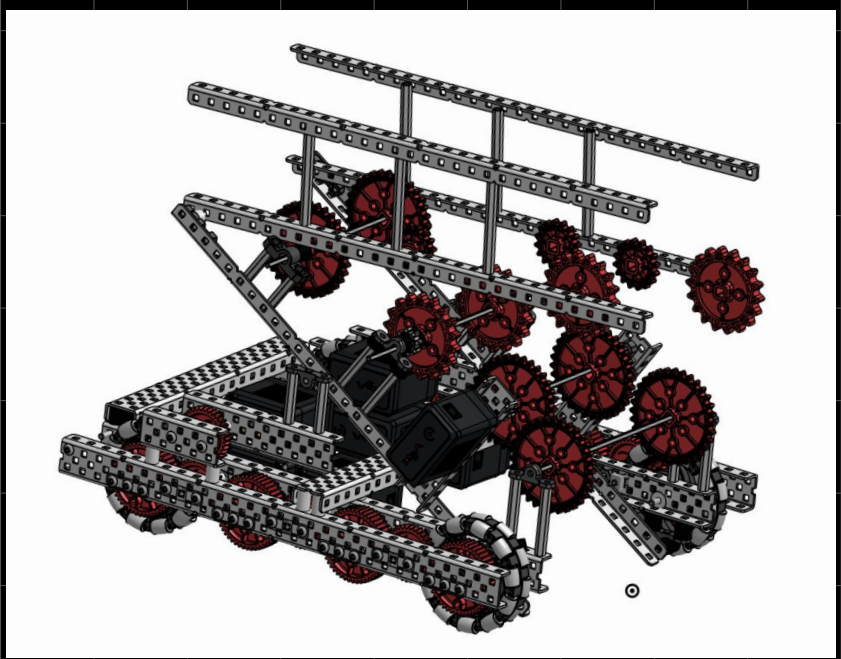
We started to add the second level and align its height. We also maintained that the height was able to go under the long goal.



CAD

We basically finished the main structure of the robot, but we need testing for the sprockets and heights and pistons location. Therefore, we decided to start building along.

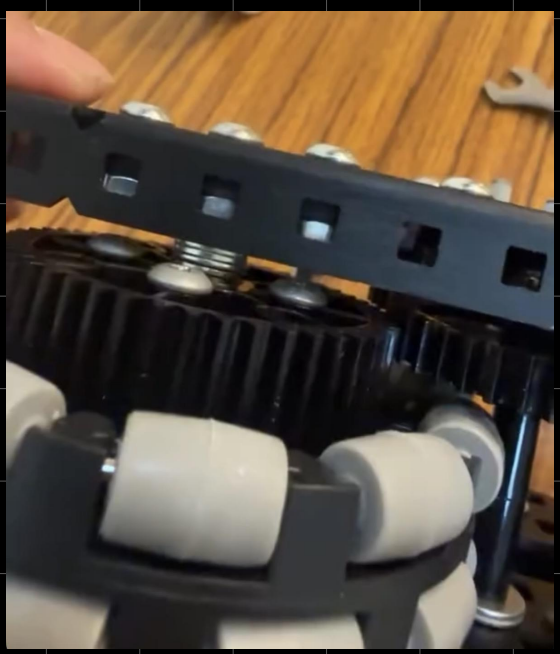




Chassis Spacing

As mentioned, after our TAI VEX event we realized that our robot had many defects. So first of all, we had to change the chassis.

Instead of using symmetric spacing for each gears and wheels and since we found out that each one would be slightly different in its dimensions, we wanted to know if we filling them with washers and leaving each with a washer wide gap would improve its smoothnest.



Chassis Spacing

We did an experiment to compare which would be smoother, we built half of a new chassis and used old spacings first, then switch up to the new spacings.

TEST 1

To measure friction caused by different spacing.

Test Procedure

1. Lift robot so wheels are off ground
2. Run program with drivetrain at 50% power for 3 sec
3. Cut power instantly and start stopwatch
4. Stop when wheels stop spinning

Chassis Spacing

TEST 1

Results

Trial	Old	New
1	7.38	10.21
2	7.12	10.44
3	7.31	10.08
4	7.59	10.63
5	7.33	10.31
6	7.67	10.78
7	7.28	10.17
8	7.05	10.52

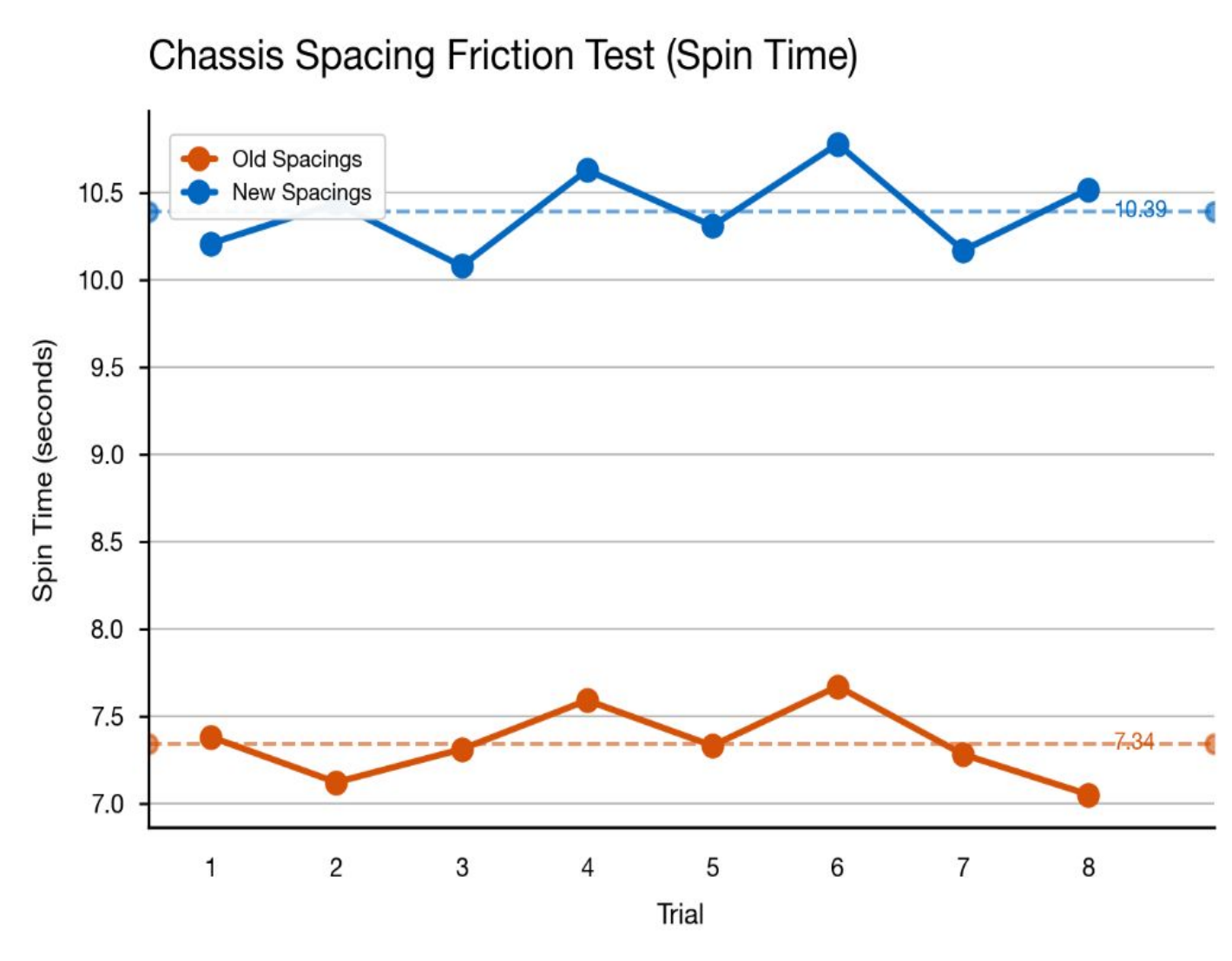
Average Spin Time

Avg	Old	New
	7.34	10.39

Chassis Spacing

TEST 1

Below plot shows TEST 1 results:



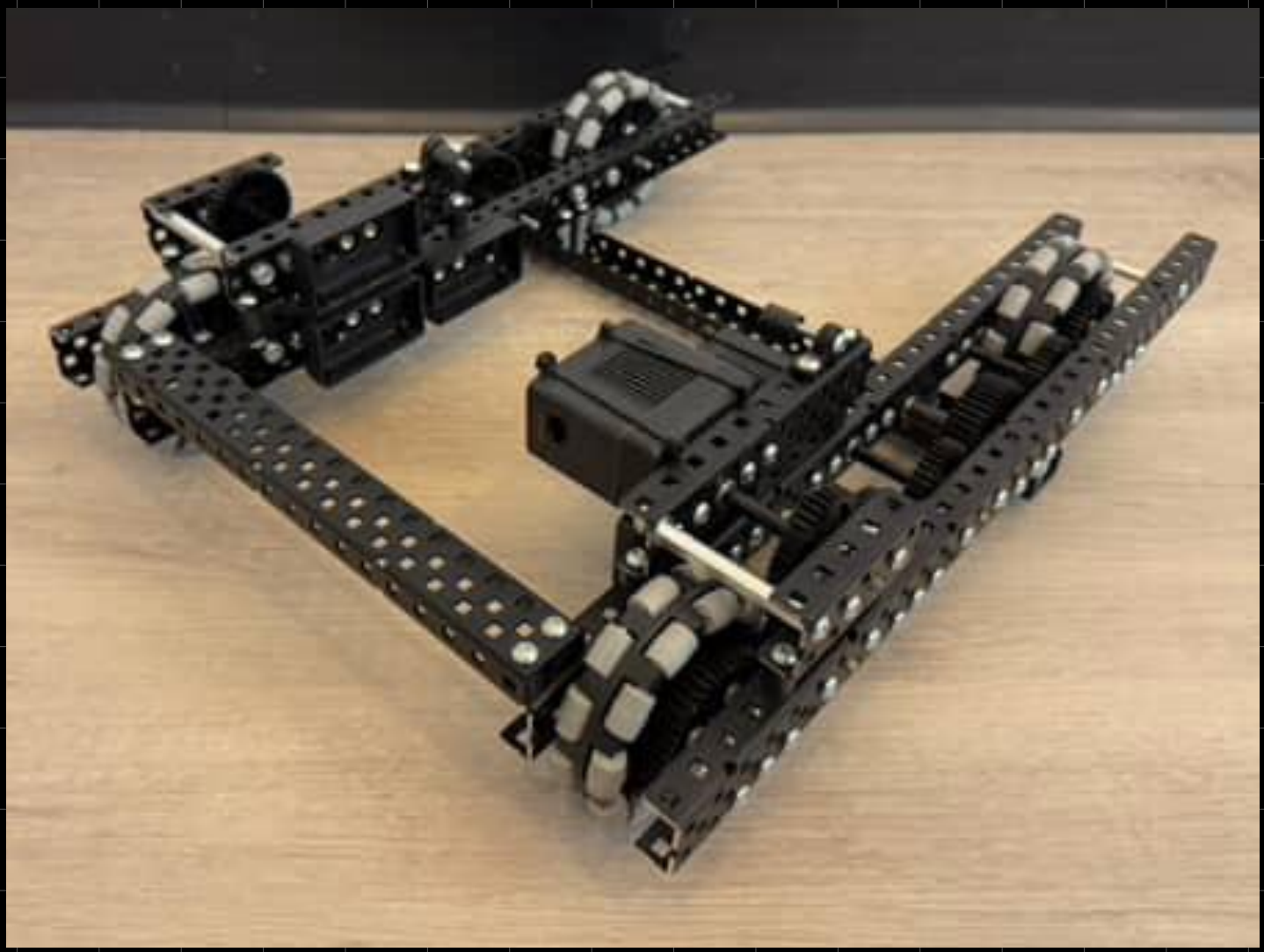
The washer-spaced configuration produced longer free-spin times, indicating less friction. Controlled spacing allowed consistent free-spinning. .

Chassis Spacing

Chassis Spacing:

Therefore, washer spacing was adopted in the final chassis design.

And we are starting to build our new chassis.



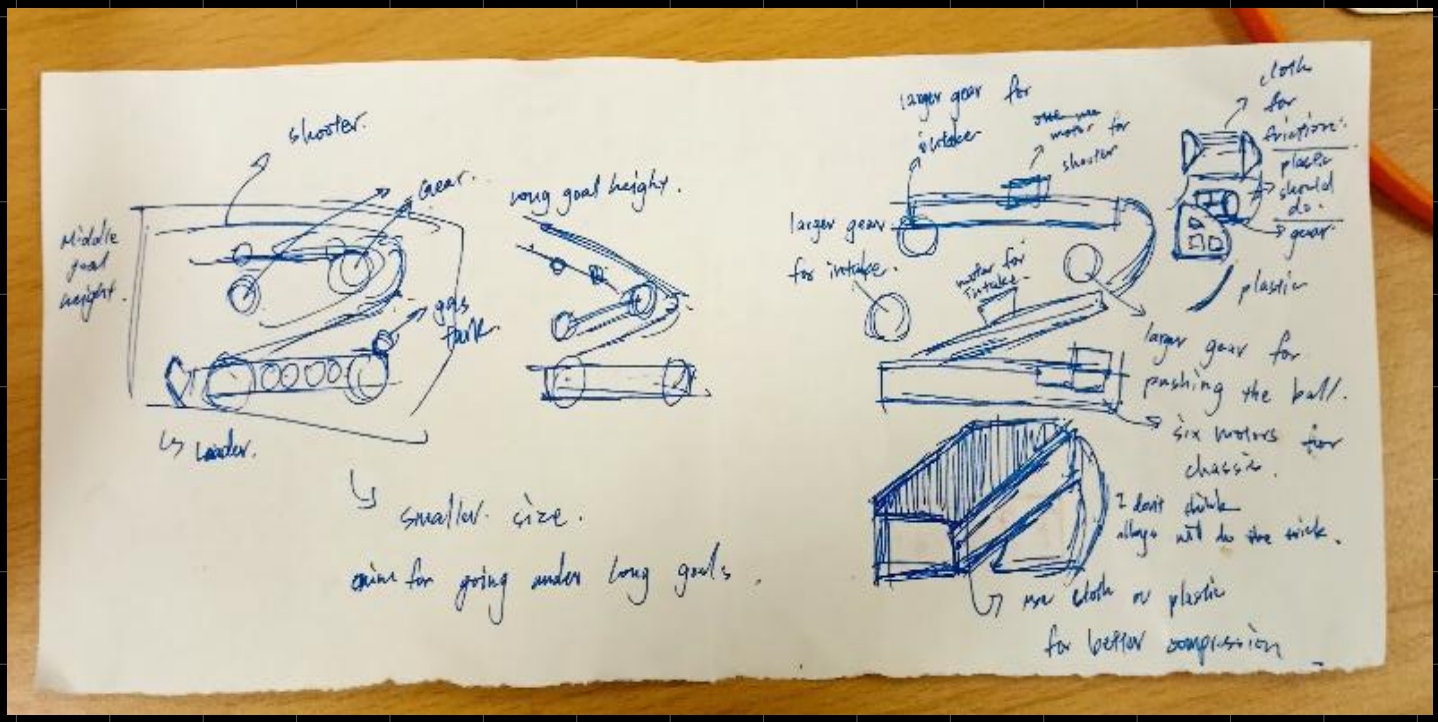
Chassis

After we have finished our new chassis and made sure it works perfectly, we started to think of the intake design. We thought that our last bot central idea was solid, the most fatal problem was that both levels of intake systems share the same level of sprockets.

Intake Design

Therefore, we decided to build two levels, but this meant that we have to precisely control the height to limit it under the long goal.

Robot Sketch:



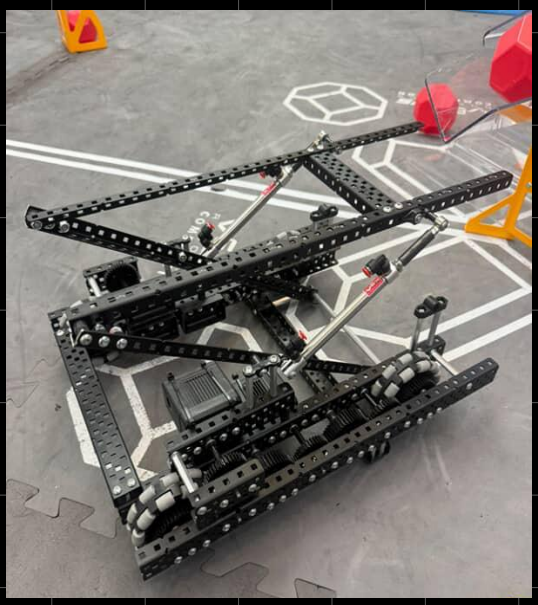
The new design features a dual phase shooter system, utilizing pneumonics for lifting. Aiming, still, for the ability to outmaneuver opponents by going through long goals. The roof could use either plastic boards or anti-slip cloths as alloys probably won't do the trick as cloths provide better compression.

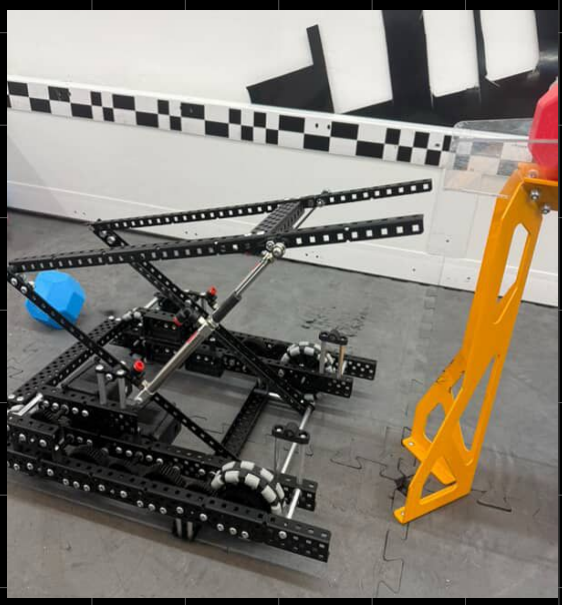
Intake Design

We started to test the height of the second level slope.

Here are some ideas that we came up with:

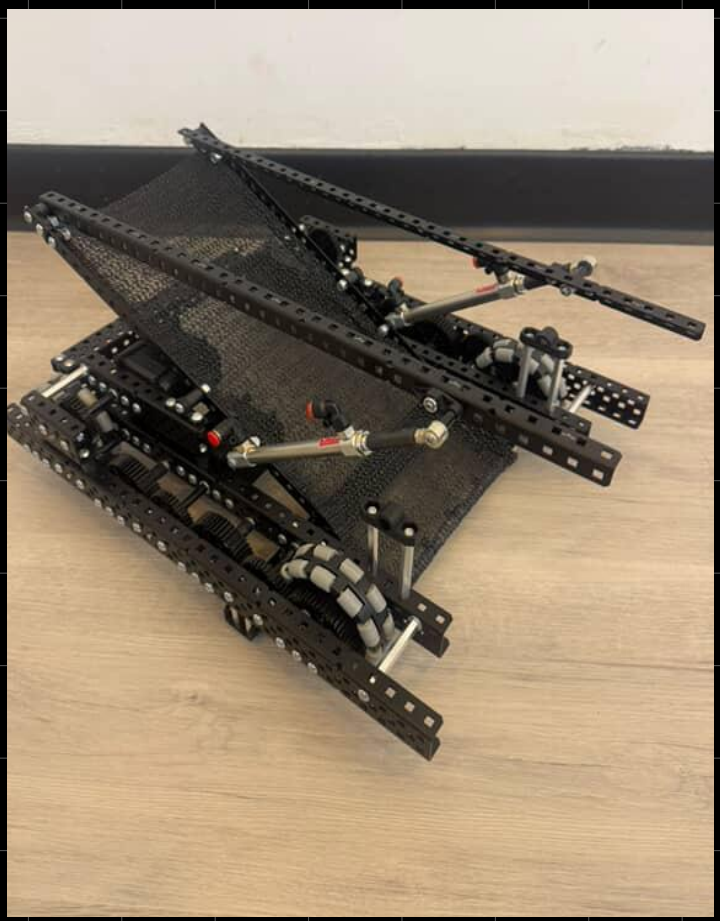
1. minimize the robot size → the second level should be horizontal, so that there isn't a ramped up height.
2. Considering 1., the lower level should be ramped so that its end is at the middle goal scoring level.
3. Pistons should be installed so that when extended, it scores to the long goal level.

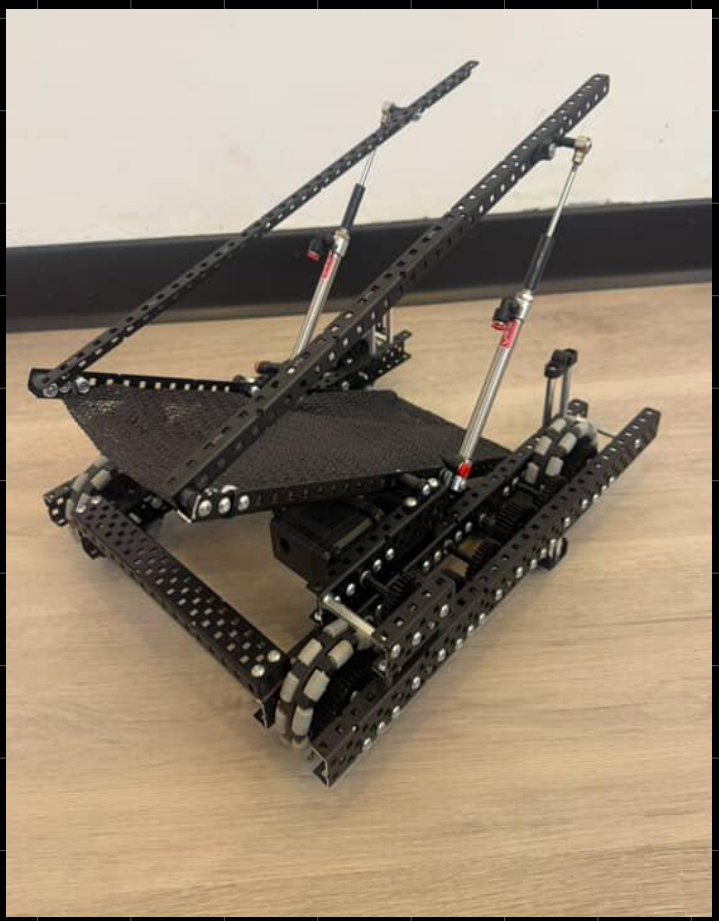




Intake Design

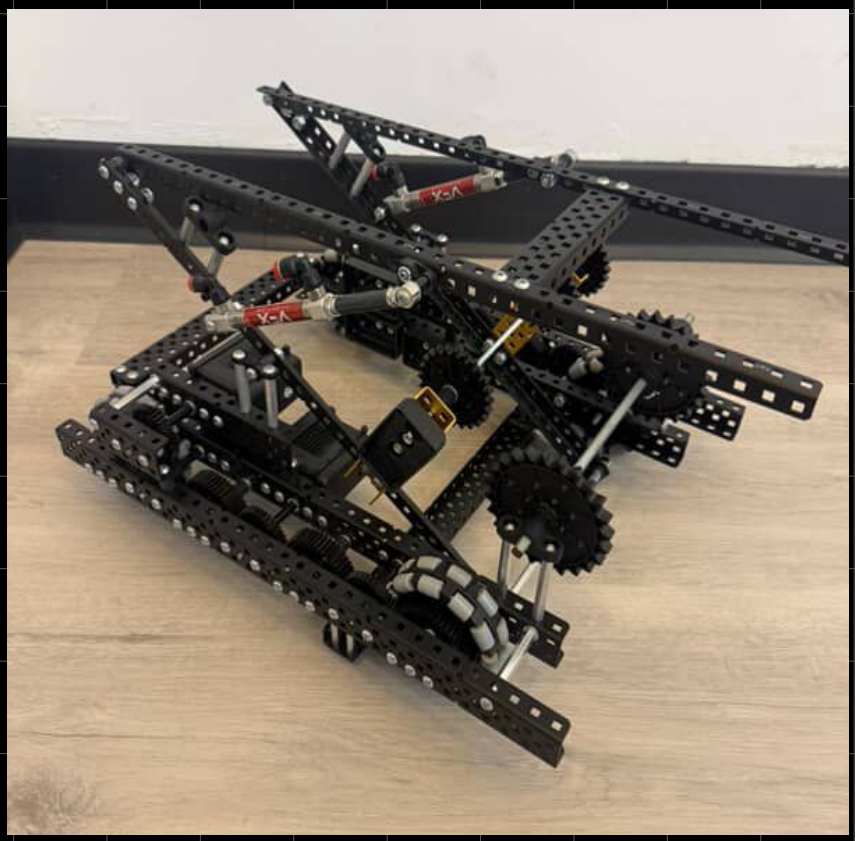
We found out that this height was working but it is interfering with the place we would want to place our motor and sprocket since it was extending over too much space.





Intake Design

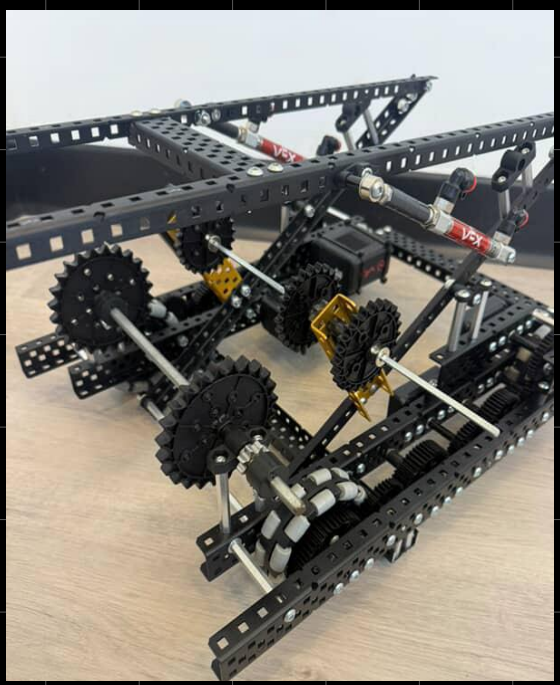
Therefore, we changed our pistons' position and still maintain the same height for both extended and retracted.



Intake Design

We have started to build our lower level sprockets, trying to let the shafts of the sprockets remain parallel to the floor of the first level, which will basically be set along the ramped up angles.

We decided to set the 11W motor on the second sprocket considering that we could accelerate the first/outermost sprocket using gear ratios. We think that a 2:1 ratio which makes the first sprocket intaking 1200 rpm is pretty solid.



Sprocket

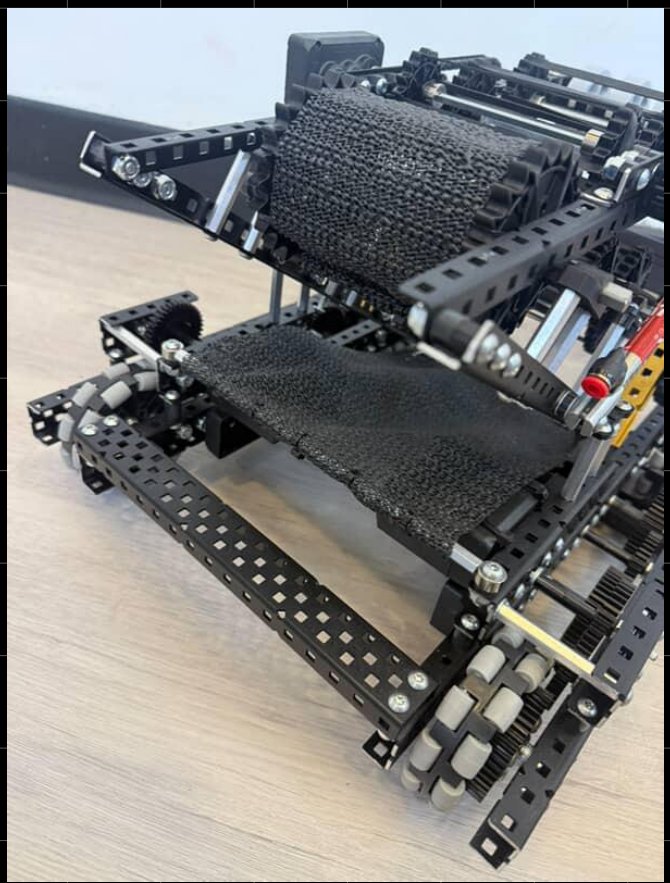
The next part of the robot is the most important sprocket (the last one for the first level of intake simultaneously the first one for the second level of intake)

We found out that if we **only use rubber bands**, when the upper level of intake is already filled up with blocks, which means that the upper level will basically not move anymore, the rubber bands would be stopped, stretched, and snapped by the motionless blocks, leading the lower level of intake to also be stuck.

Sprocket

To solve this problem, we wrapped this sprocket **with the anti-slip mat** after our last robot's experimentation.

Also, the anti-slip map supports a consistent touching of the blocks to the mat, or sprocket surface, ensuring that the ball would be moved up with more friction.

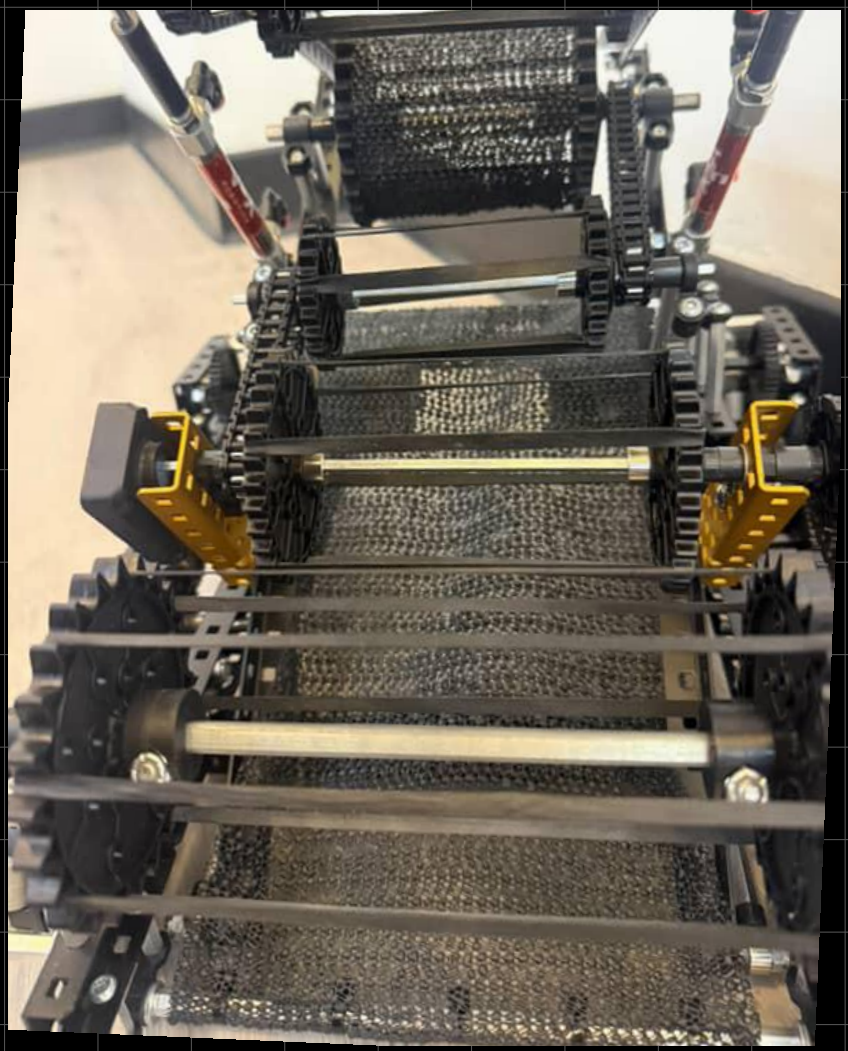


Sprocket

The motion of the important sprocket must be chained to the lower level because:

1. **the sprocket does not move in position.**
2. It is built on the lower level's angle structure.

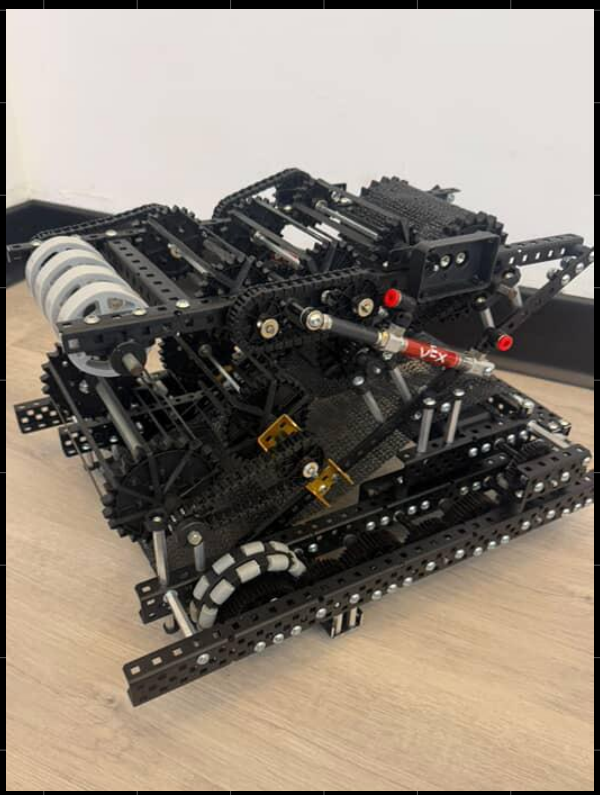
Therefore, if it was chained to the moving-in-height upper level, the chains would be **stretched out** and snap.

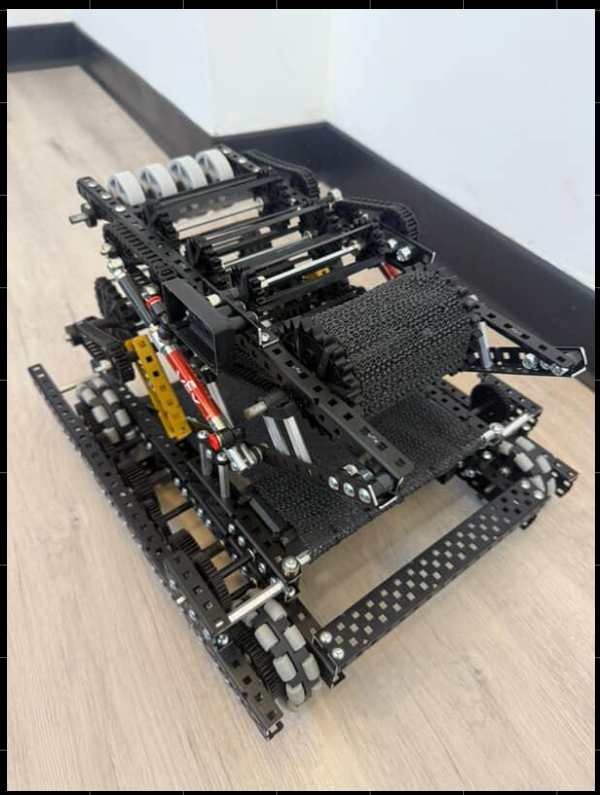


Sprocket

After the sprocket's size was decided, based on its shaft height, like mentioned, maintaining on a same basis of other shafts of the first level and parallel to the mat, we finished up the upper level's ground sprockets. We could only use small sprockets because:

1. We had to minimize the robot's height
2. If the radius were too large it would interfere with the lower level sprockets and intaked blocks

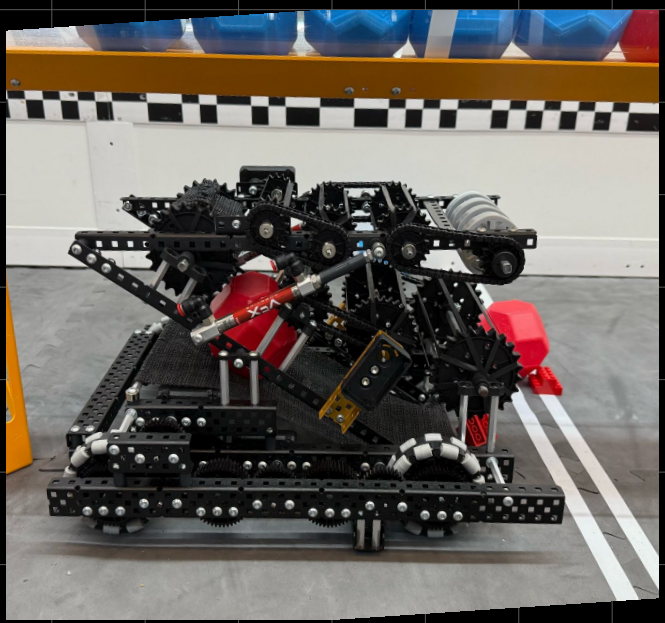


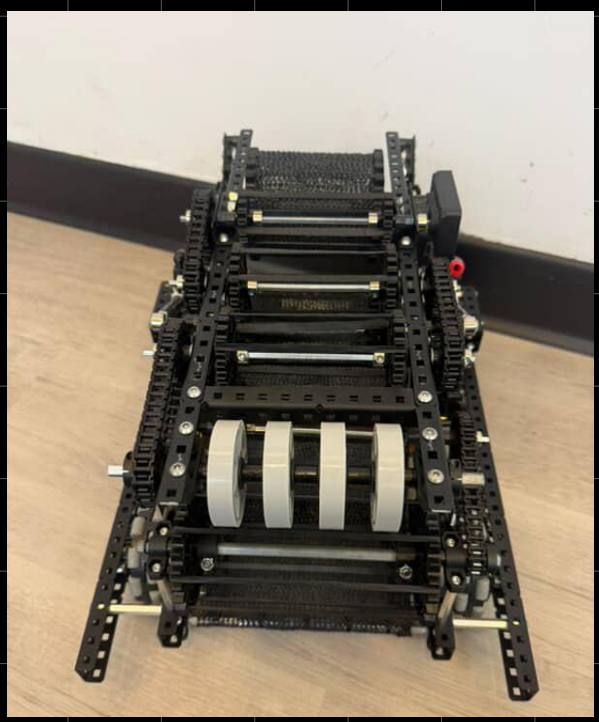


Sprocket

Although we all use rubber bands with sprockets like we reasoned before, we decided to use flex wheels for the outake instead of sprockets because:

1. The blocks are not entirely balls, they consist **edges** that make its size not consistent, therefore we need some amount of compression
2. However, the compression should not be too much because using sprockets with rubber bands would make the outaked blocks come out with **less force**. Little amount of compression would squeeze the blocks out with higher force.



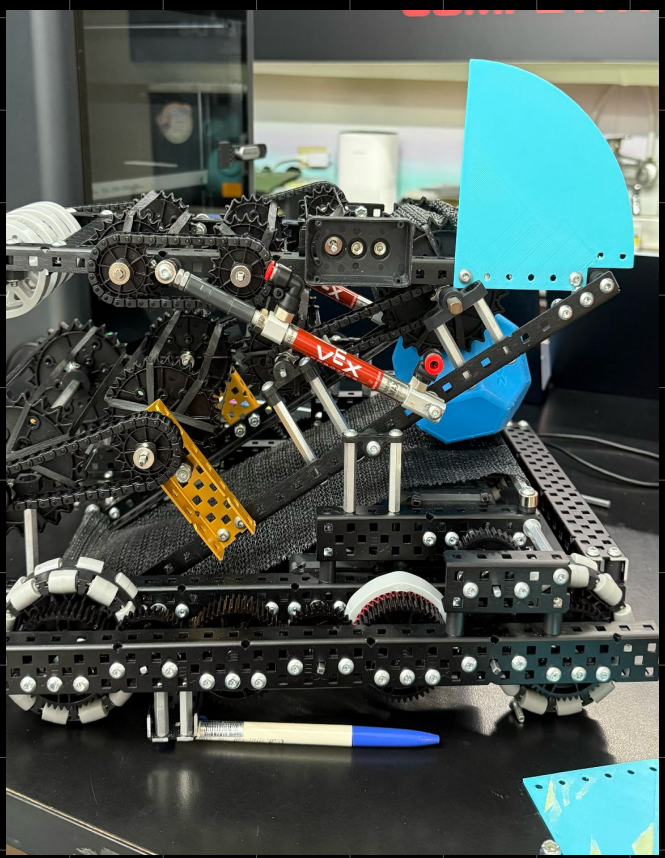


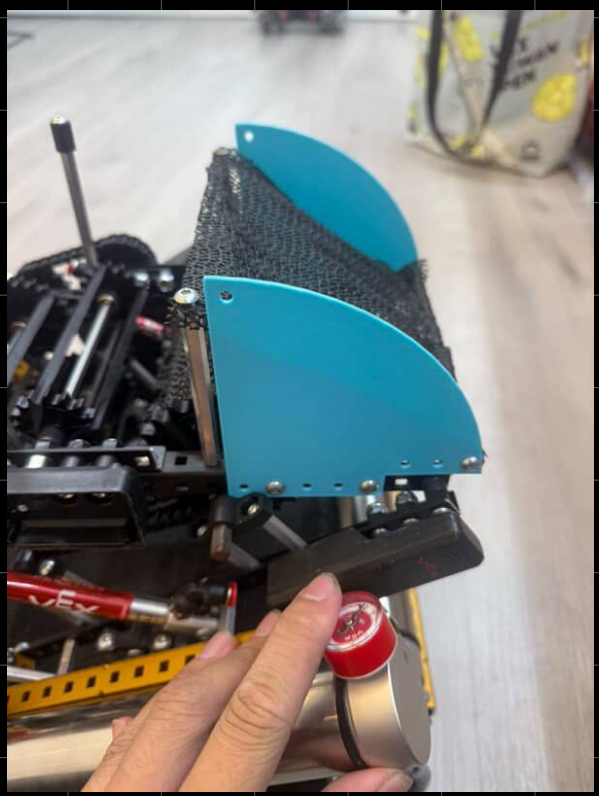
Movement

As mentioned before, our last sprocket will ensure a **consistent touching of the mat and the ball**, creating a curved movement. Therefore, we wanted to add **plastic boards** supporting the mats to be tied on it to create a curved surface.

We used 3d printing to test if our CAD drawing was correct, if so, we would laser cut plastic boards.

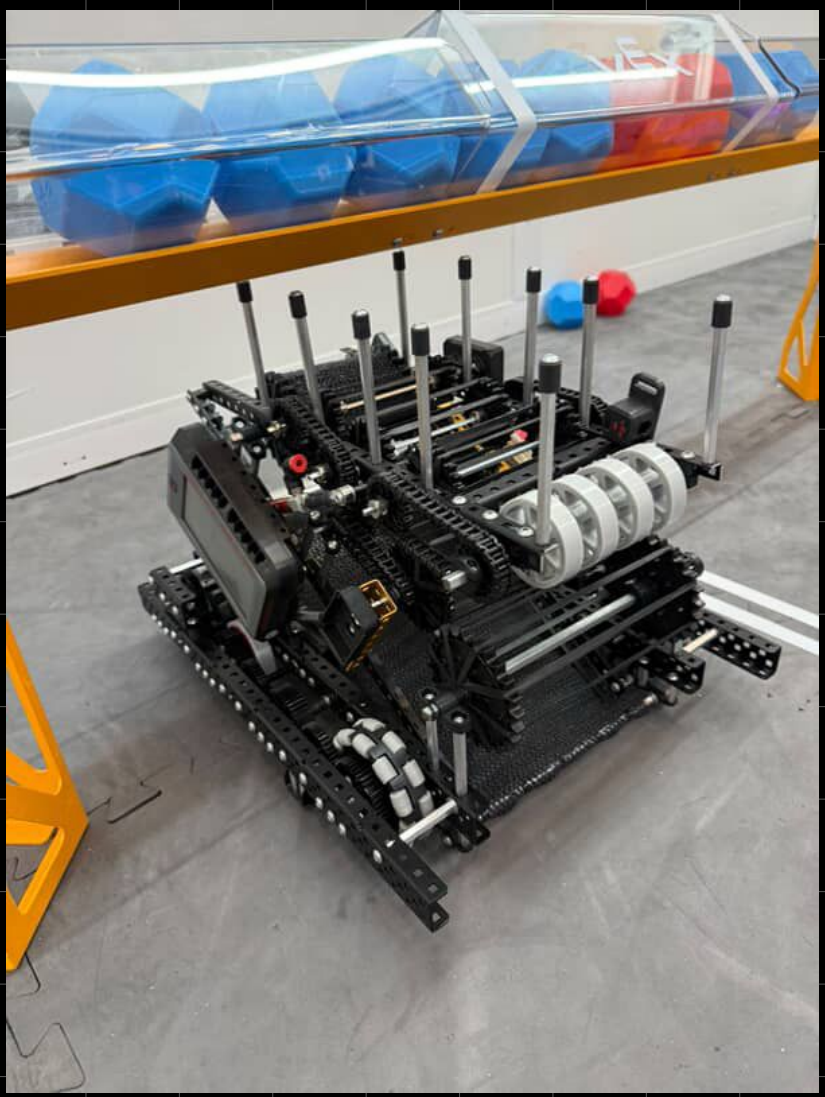
We continued to revise the board shape.





Plastic Boards

While revising the aforementioned plastic boards, we were also starting to design the roof of the robot, which would also be plastic boards, as well as adding the whole pneumatics system and electronics.



Roof

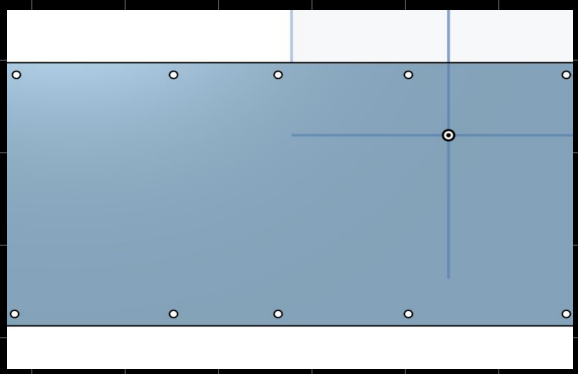
To design the plastic boards:

1. decide where the standoffs would be placed since some places have structures like bearings already.
2. Draw the determined holes
3. The roof's wide is 5.5 inches, over 4 already, so the length could only be 4. Therefore, we decided to cut the roof into two halves, making the width less than 4, specifically 2.75, and the length could be 8 inches. The edges of the boards would have holes to connect them with zip ties.
4. We decided that we would make the roof hollow because this would allow us to :
   - see the colors of the blocks inside
   - increase the amount of compression and extension
5. We then decided to make all our plastic boards hollow with triangular designs.

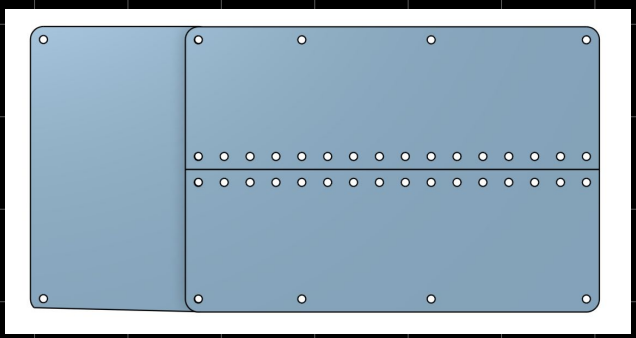
Roof

This is the process of developing our roof

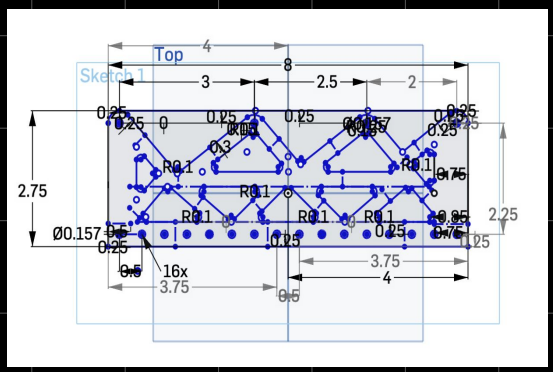
1



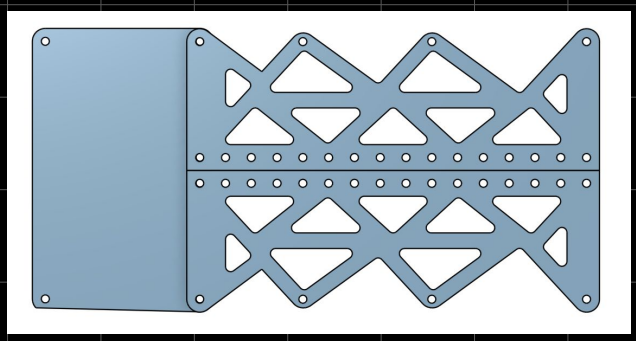
2



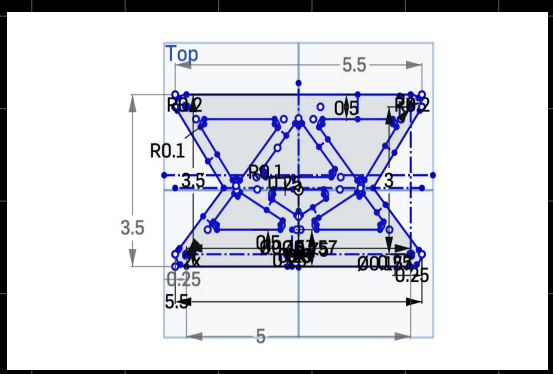
3



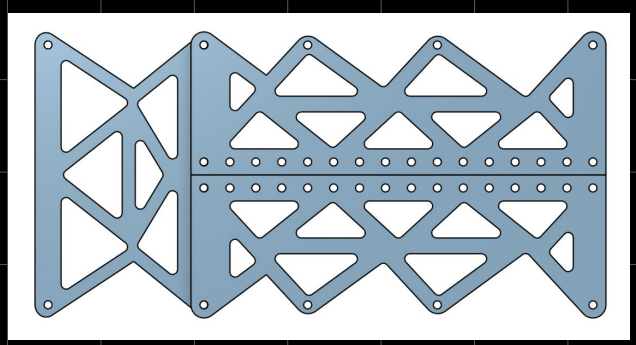
4



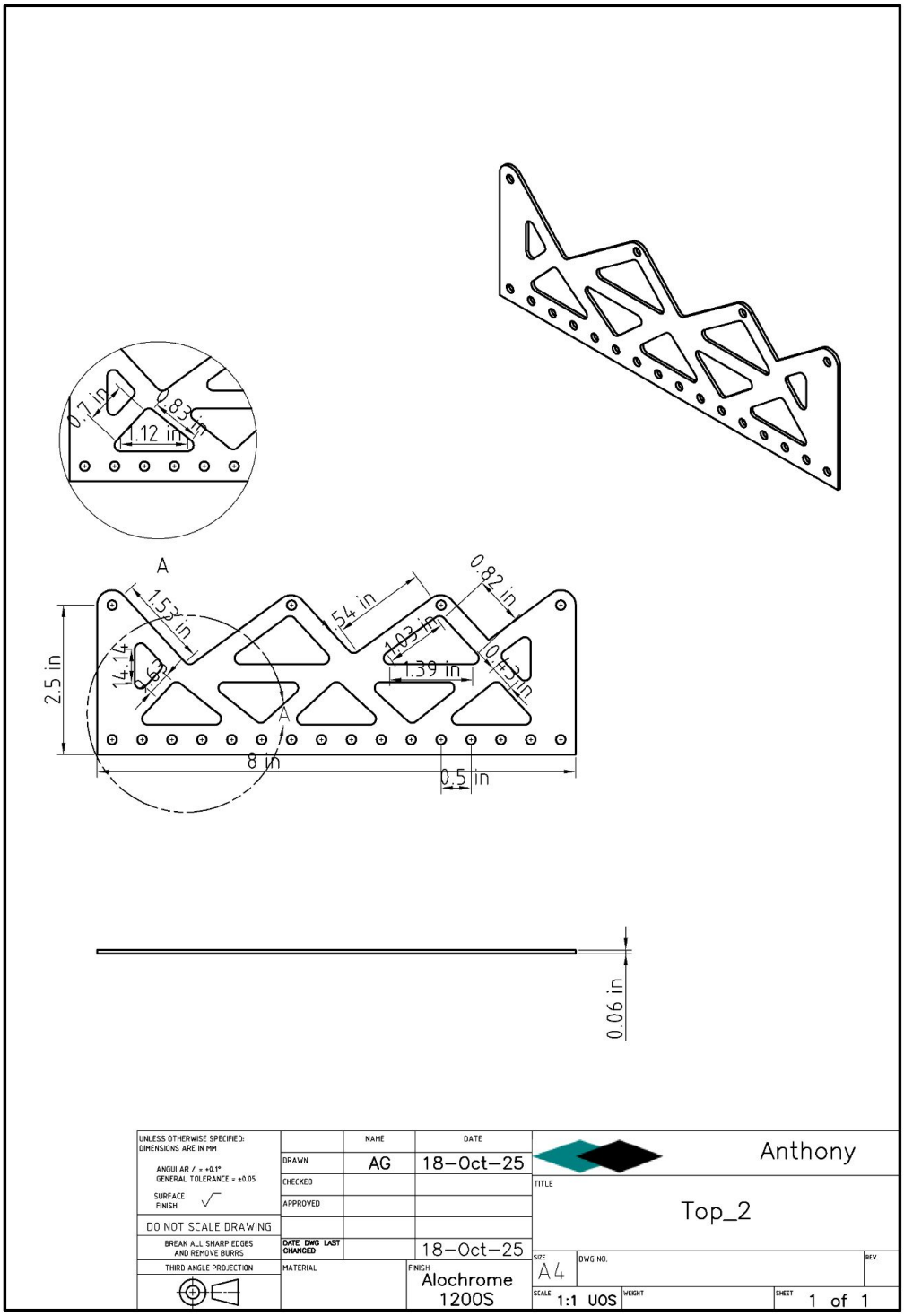
5



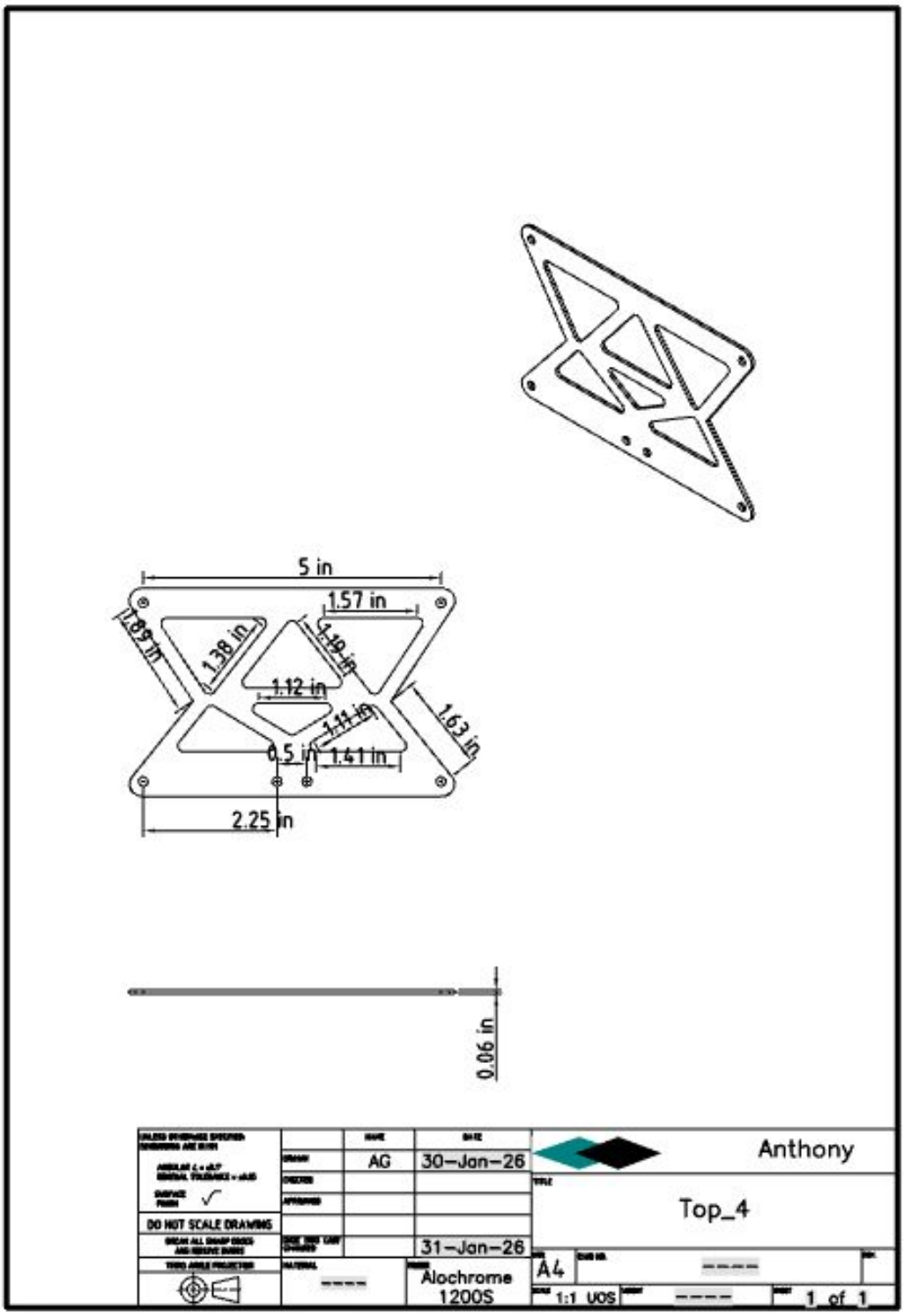
6



Roof


0.7 in
0.83 in
1.12 in
A
1.52 in
.54 in
0.82 in
1.03 in
1.39 in
0.45 in
14.14
0.63
2.5 in
8 in
0.5 in
0.06 in
UNLESS OTHERWISE SPECIFIED:
DIMENSIONS ARE IN MM
ANGULAR ∠ = ±0.1°
GENERAL TOLERANCE = ±0.05
SURFACE FINISH
DO NOT SCALE DRAWING
BREAK ALL SHARP EDGES AND REMOVE BURRS
THIRD ANGLE PROJECTION
NAME
DATE
DRAWN
AG
18–Oct–25
CHECKED
APPROVED
DATE DWG LAST CHANGED
18–Oct–25
MATERIAL
FINISH
Alochrome 1200S
Anthony
TITLE
Top_2
SIZE
A4
DWG NO.
REV.
SCALE
1:1 UOS
WEIGHT
SHEET
1 of 1

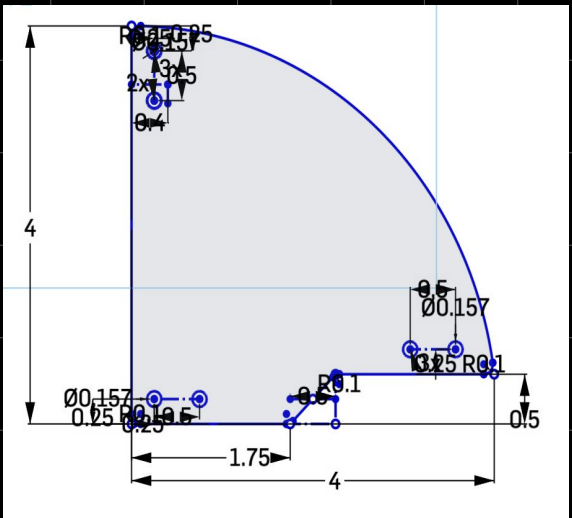
Roof


5 in
1.57 in
1.89 in
1.38 in
1.19 in
1.12 in
1.11 in
1.63 in
0.5 in
1.41 in
2.25 in
0.06 in
AG
30–Jan–26
31–Jan–26
DO NOT SCALE DRAWING
Alochrome
1200S
Anthony
Top_4
A4
1:1 UOS
1 of 1

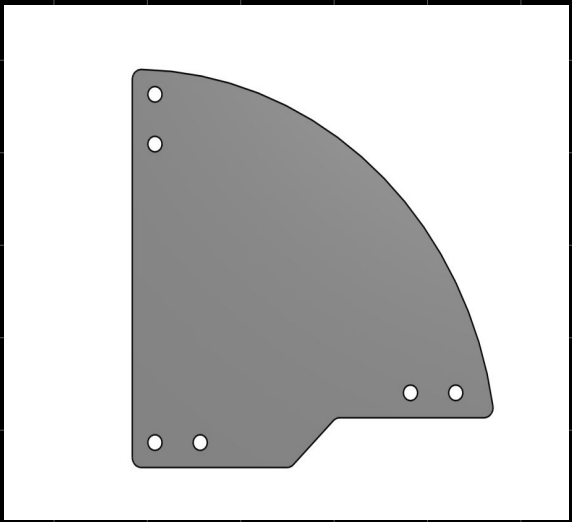
Mat Support

This is the process of designing the side sectors that would support the mats.

1

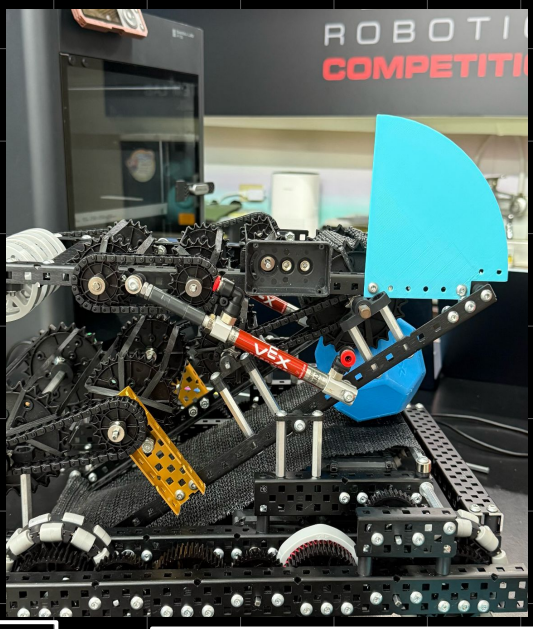


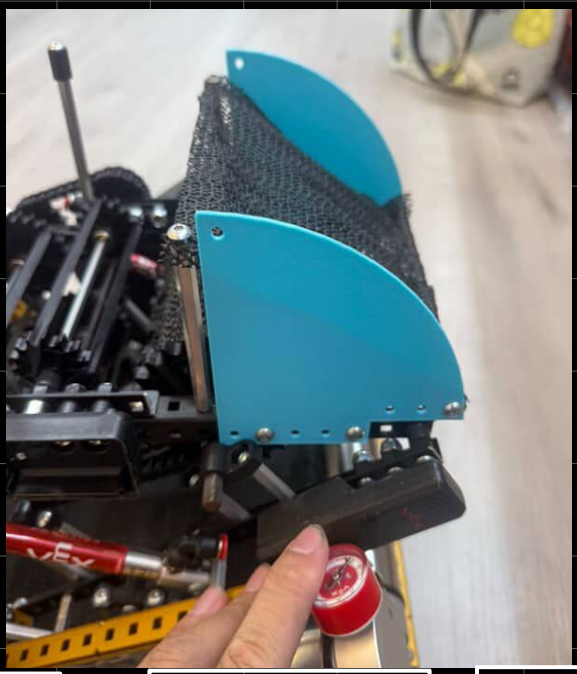
2



Testing Solution

We 3d printed out this prototype to test out its dimensions.

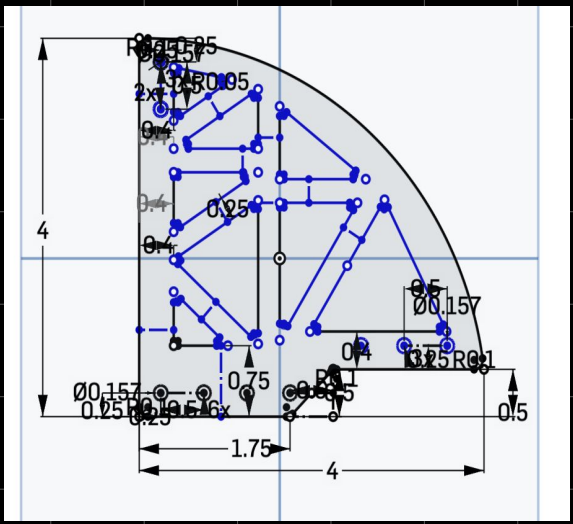




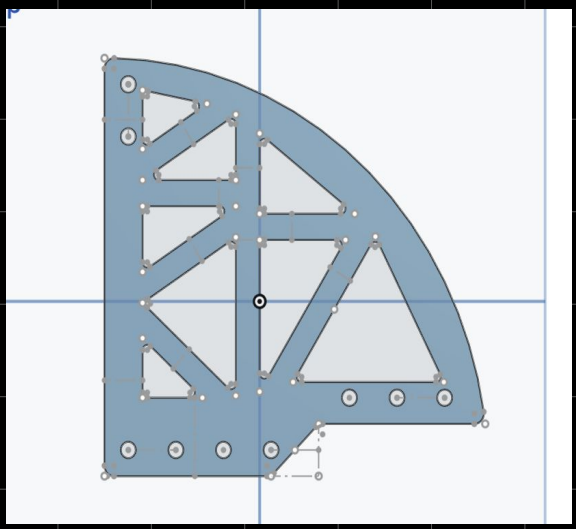
Mat Support

After we 3d printed out this prototype, we hallow designed it for laser cutting.

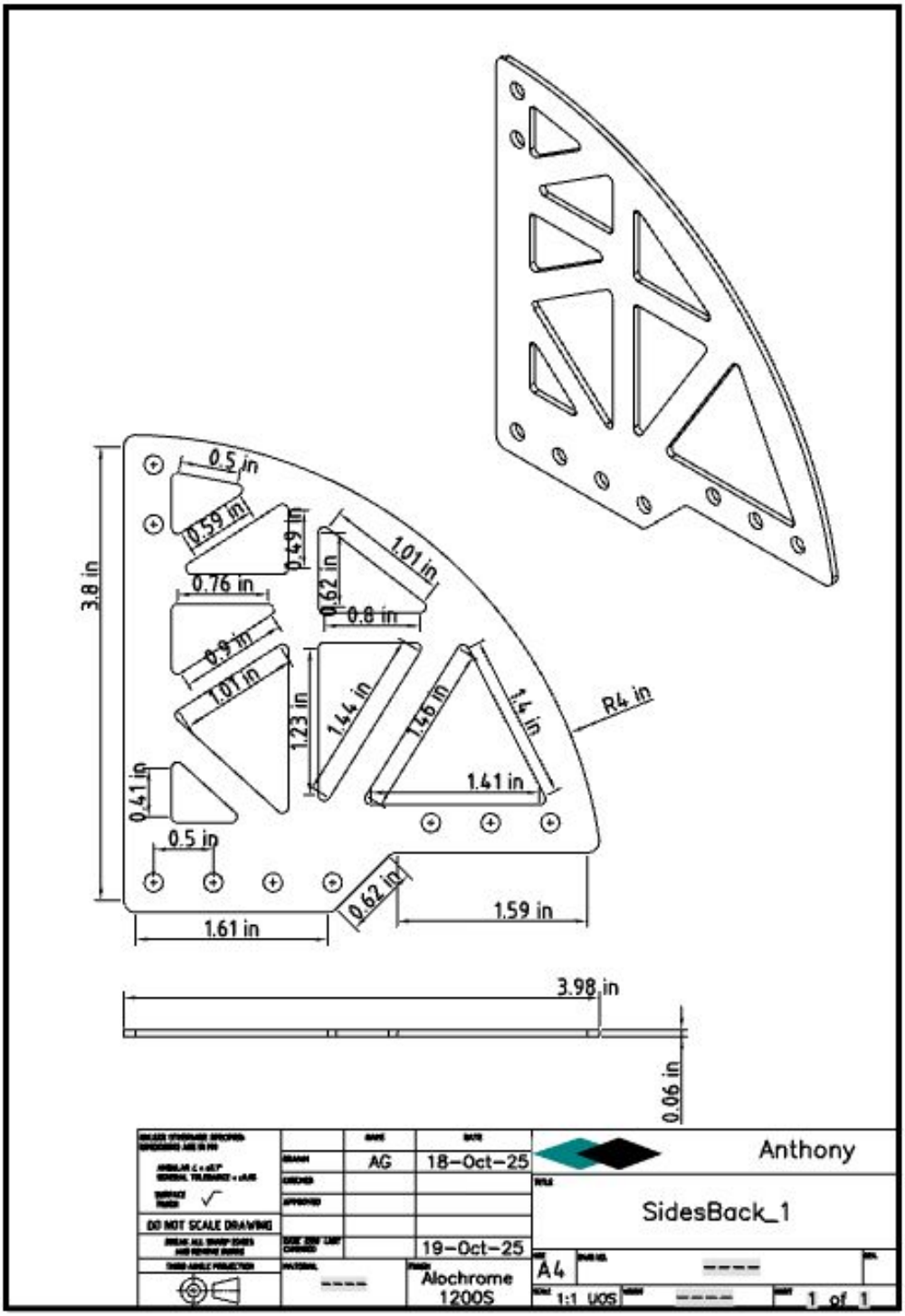
3



4

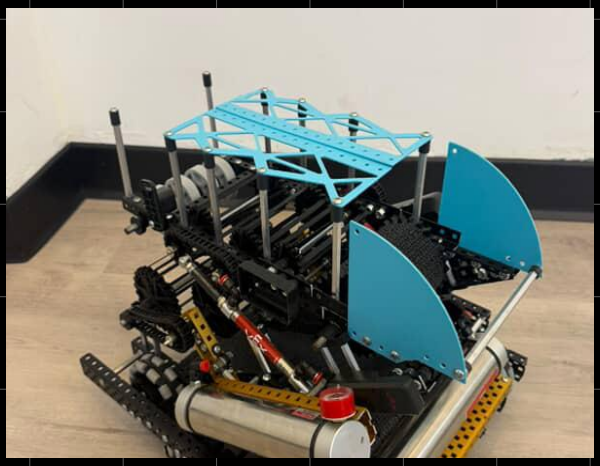


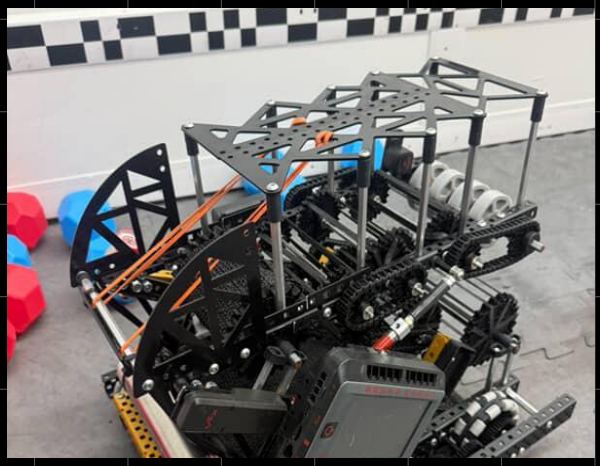
Mat Support


0.5 in
0.59 in
0.49 in
1.01 in
0.62 in
0.76 in
0.8 in
0.9 in
1.01 in
1.23 in
1.44 in
1.46 in
1.4 in
R4 in
3.8 in
0.41 in
0.5 in
1.41 in
0.62 in
1.59 in
1.61 in
3.98 in
0.06 in
AG
18-Oct-25
19-Oct-25
DO NOT SCALE DRAWING
Alochrome 1200S
Anthony
SidesBack_1
A4
1:1 UOS
1 of 1

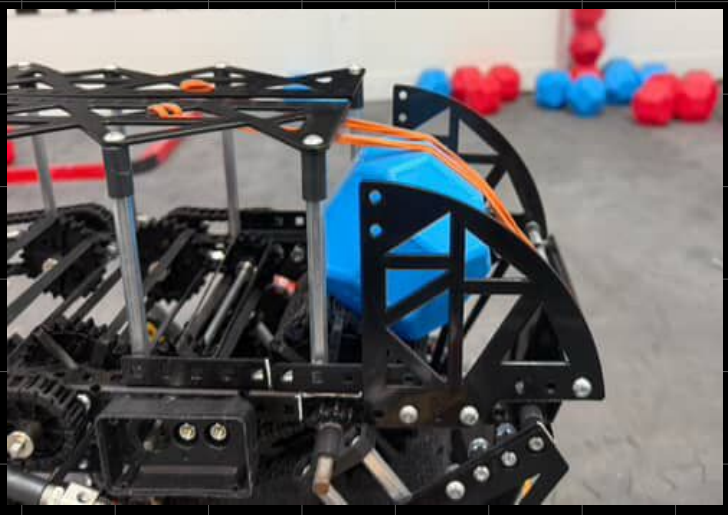
Plastic Boards

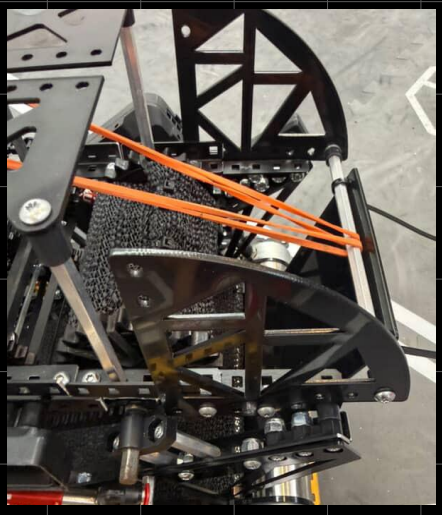
After finishing developing the roof, we also made the aforementioned plastic board for the last sprocket hollow with triangular patterns.





We also found out that the balls would need more amount of compression for going up to the second level, so we added long #117b rubber bands.

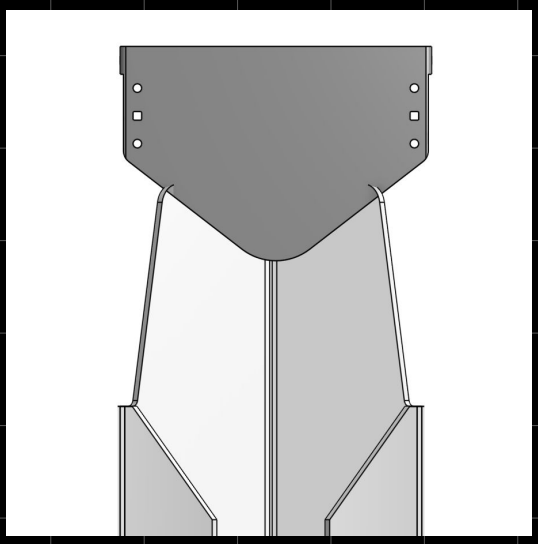


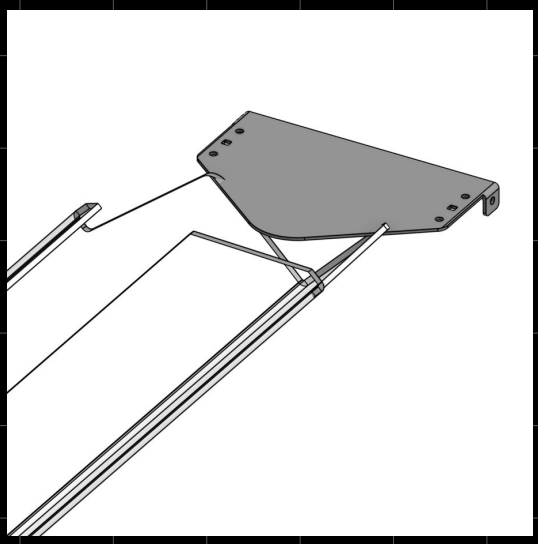


Descorer

We had to use a blocking mech for the blocks to not fall out when the robot is moving or when the intake is on:

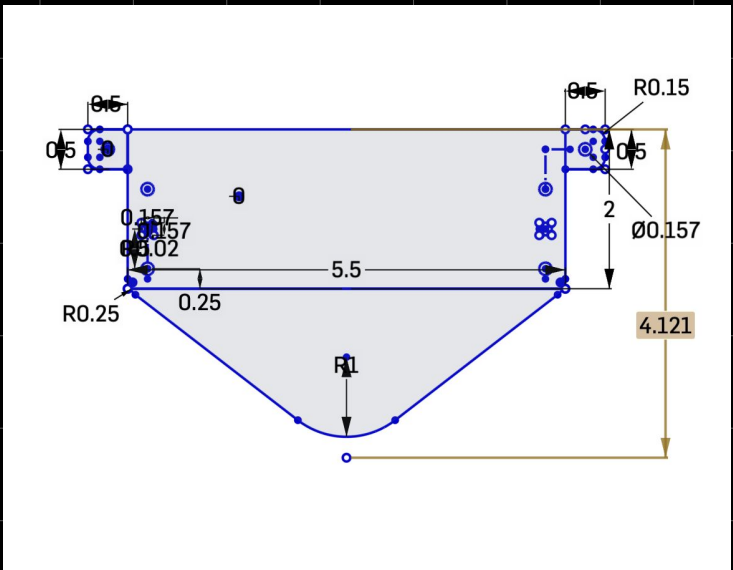
1. We need to design a board driven by pneumatics, which hinders the blocks from falling out when pistons retracted
2. It will also act as descoring when pistons extended. This required it to be a triangular shape to enter the long goal.
3. Ideally, making it triangular could also help align our robot when at goals.
4. Along with this, we had to develop space for the pistons to place.

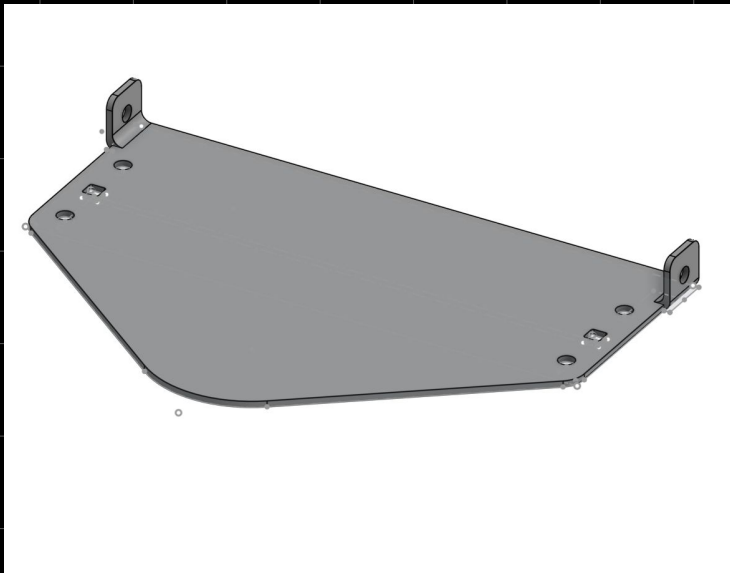




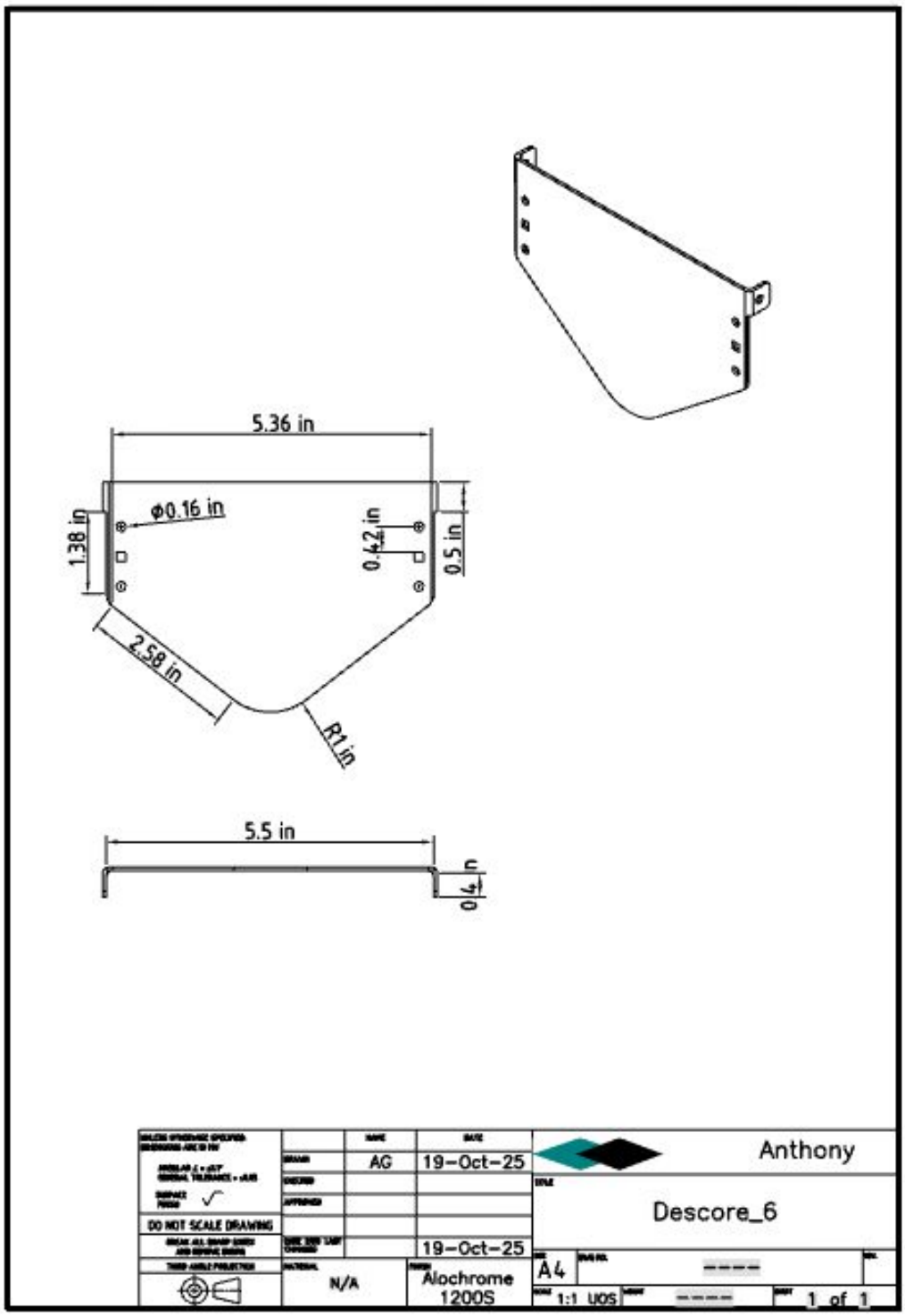
Descorer

Since we had to use 3d printing for prototyping first, we had to CAD a 3d board, since the descorer would be bent.



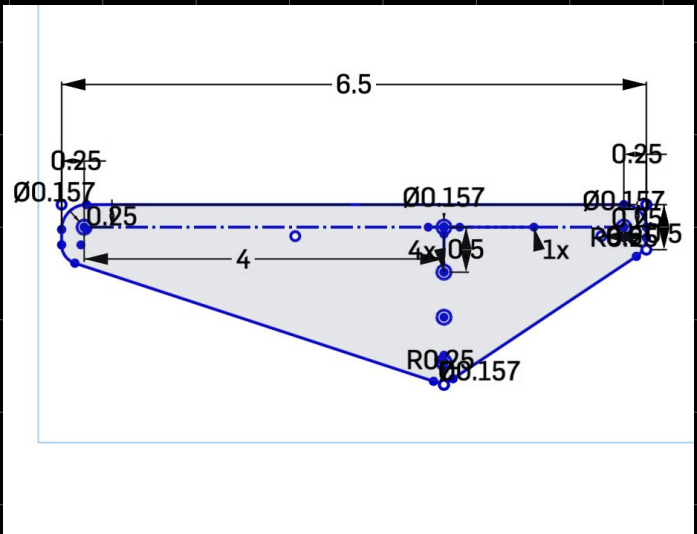


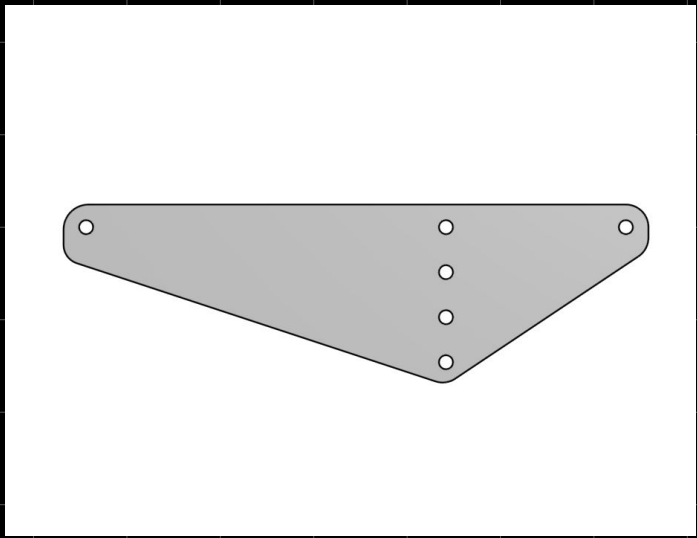
Descorer


5.36 in
Φ0.16 in
1.38 in
0.42 in
0.5 in
2.58 in
R1 in
5.5 in
0.4 in
AG
19-Oct-25
Anthony
Descore_6
DO NOT SCALE DRAWING
19-Oct-25
N/A
Alochrome 1200S
A4
1:1 UOS
1 of 1

Descorer

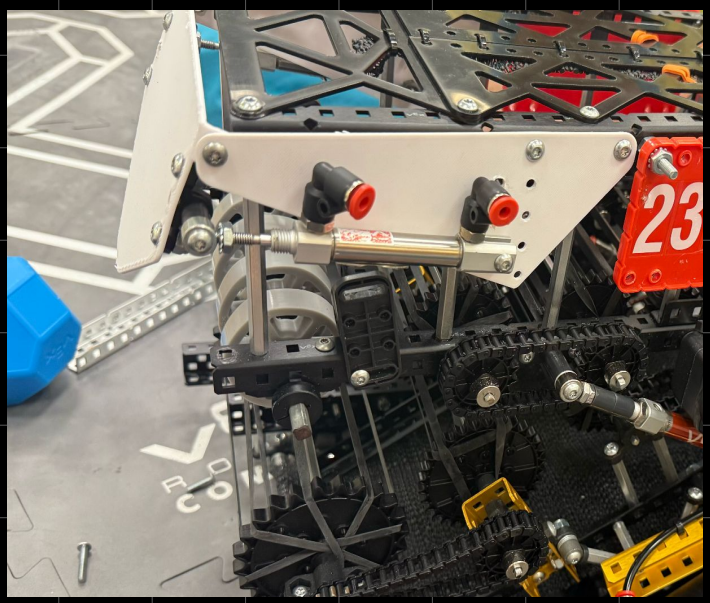
We also had to design the piston holding boards for both sides of the bot and test the doable position for the pistons.

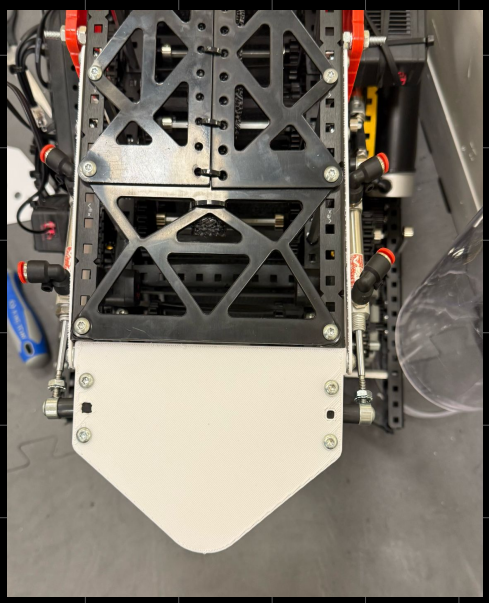




Descorer

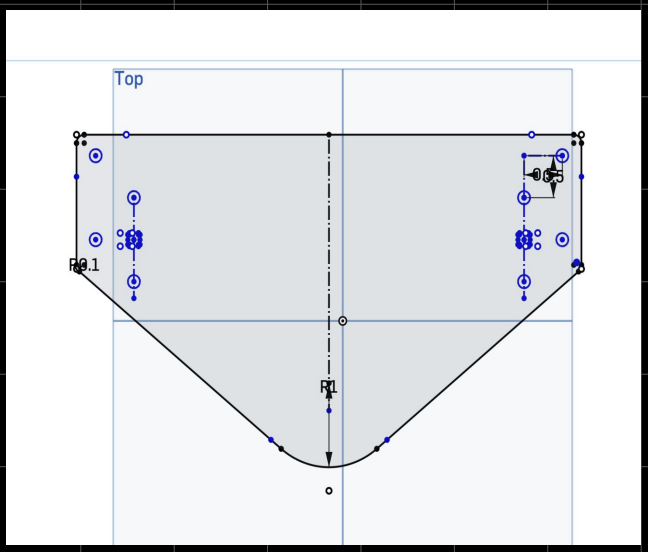
We 3d printed the descorer and the piston holders and tested out the optimal positions.

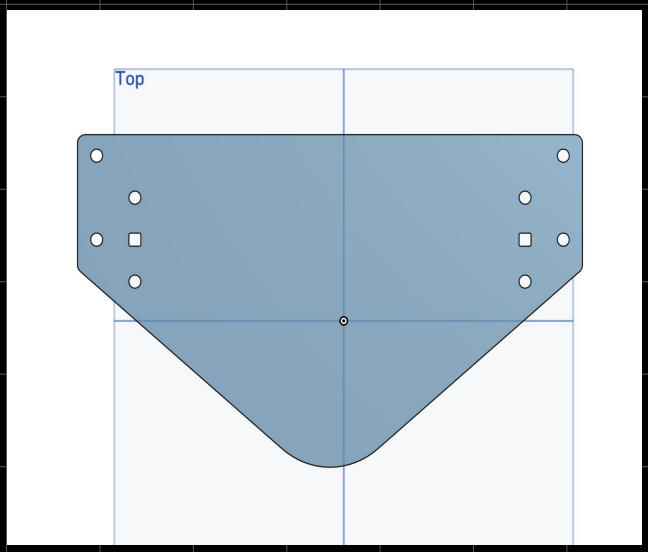


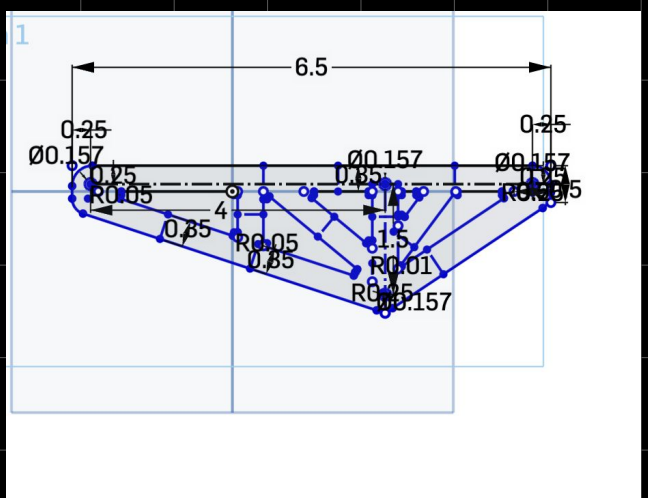


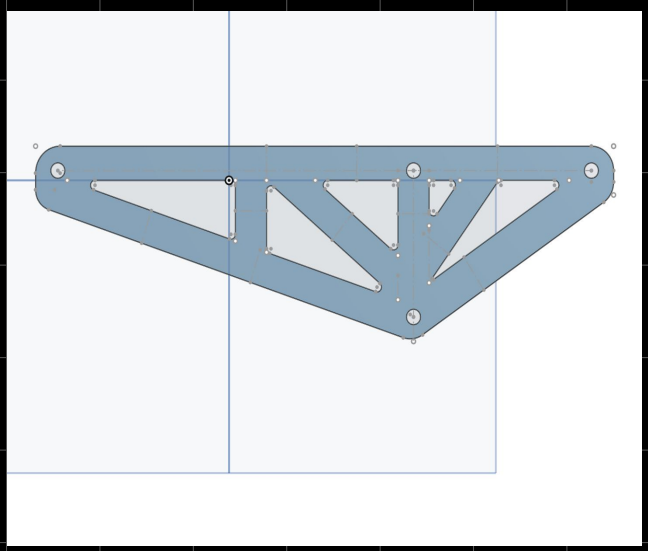
Descorer

After determining and testing the specifications, we could revise and finalize our plastic design our trianglular hollowing.

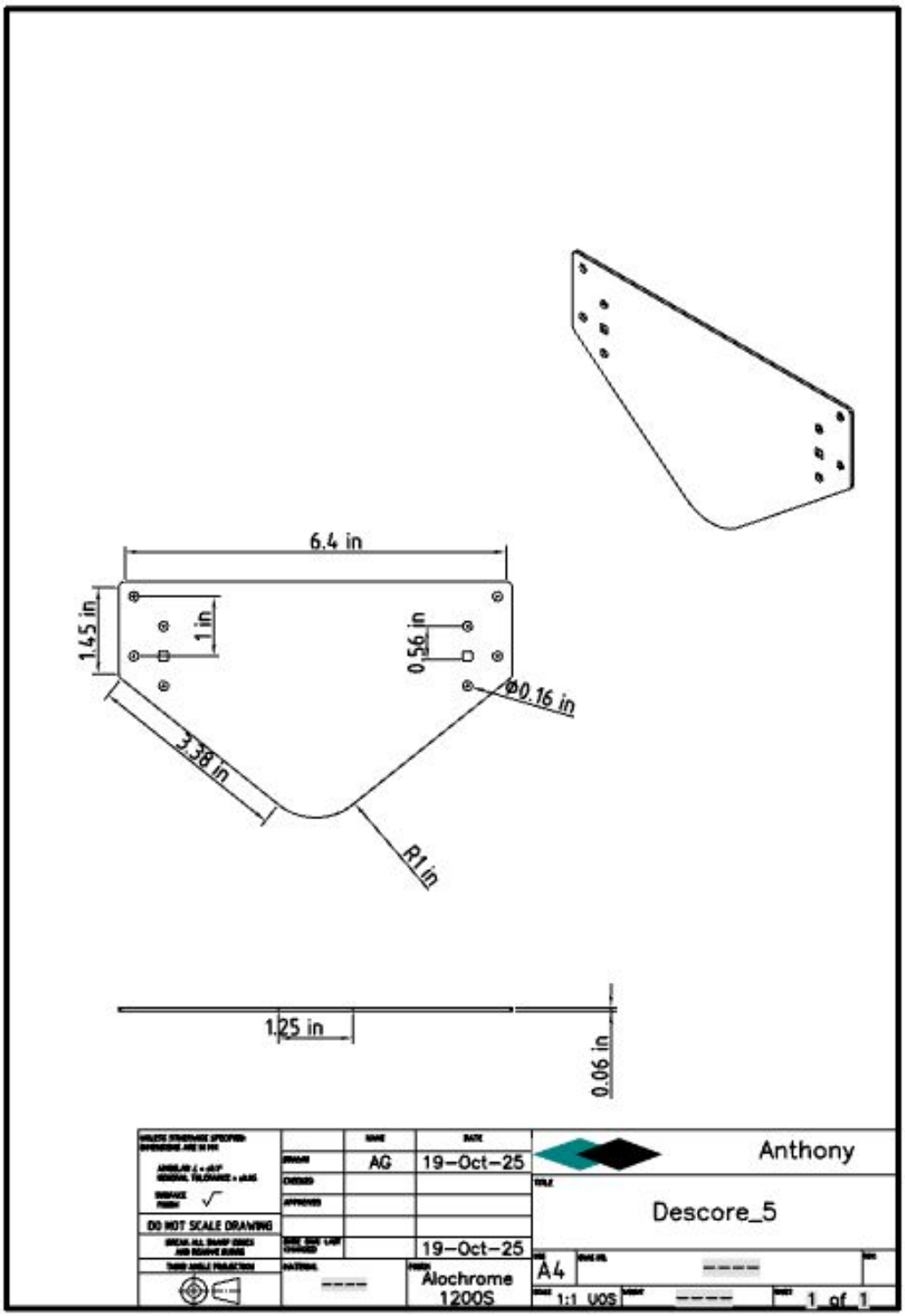




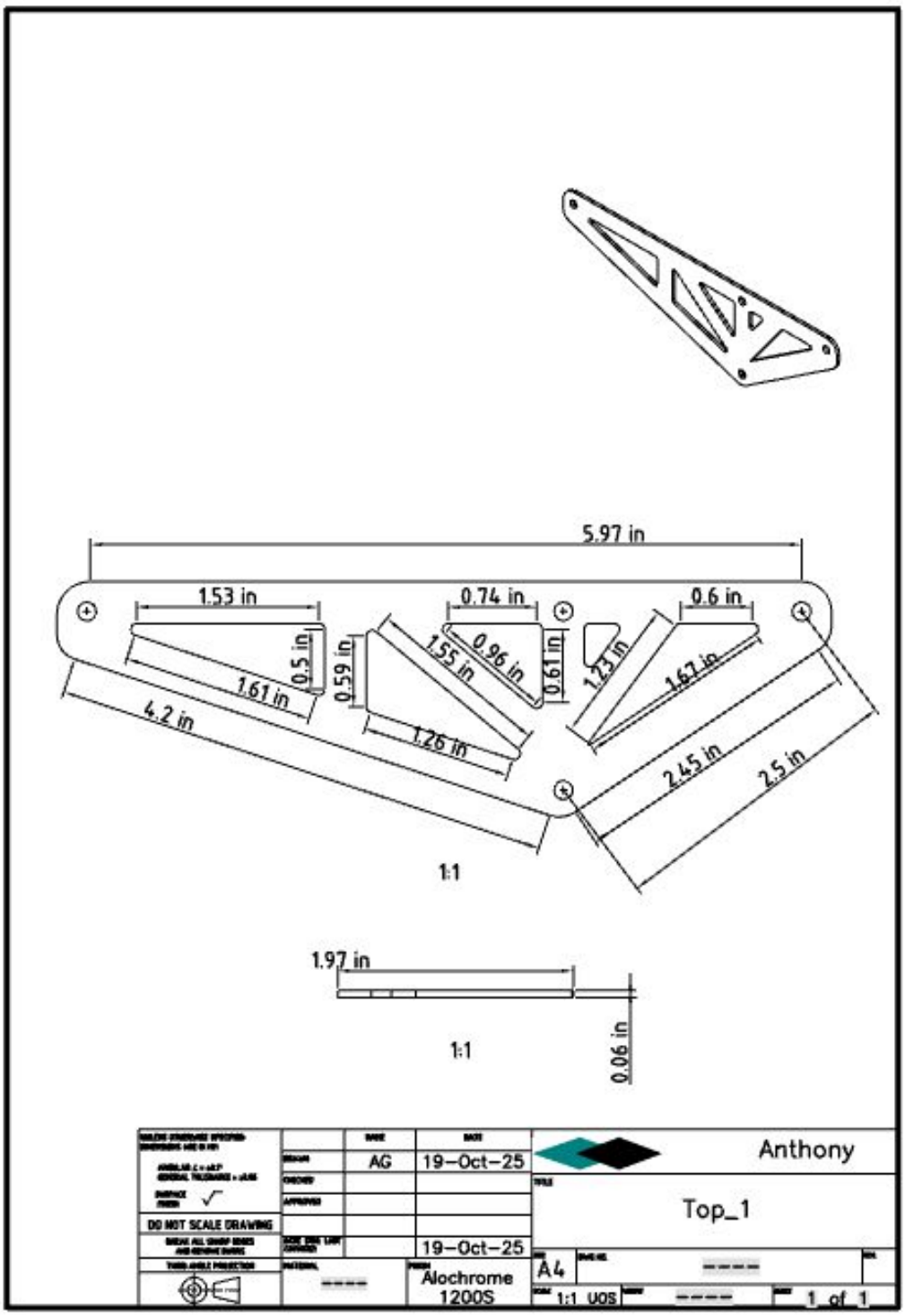




Descorer

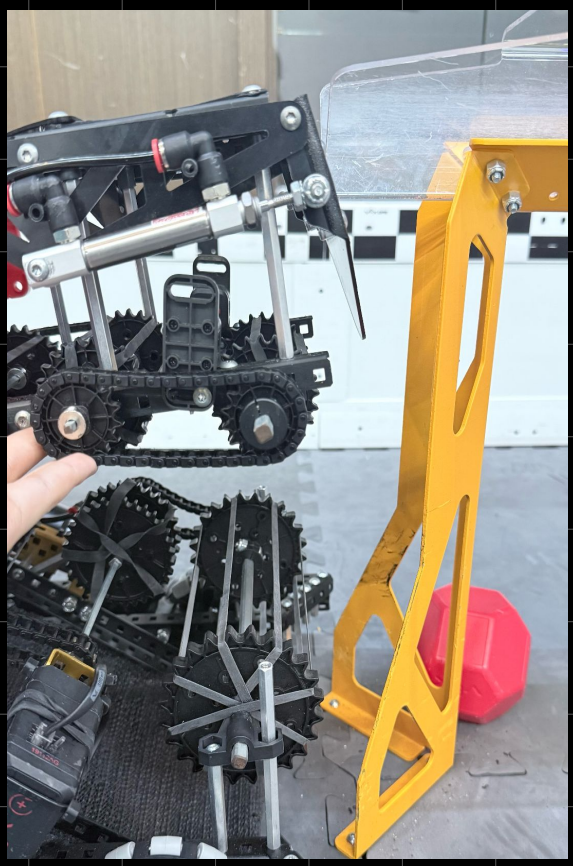

6.4 in
1.45 in
1 in
0.56 in
Ø0.16 in
3.38 in
R1 in
1.25 in
0.06 in
DO NOT SCALE DRAWING
AG
19–Oct–25
19–Oct–25
Alochrome 1200S
Anthony
Descore_5
A4
1:1 UOS
1 of 1

Descorer


5.97 in
1.53 in
0.74 in
0.6 in
0.5 in
0.59 in
1.61 in
0.96 in
0.61 in
1.55 in
1.23 in
1.67 in
4.2 in
1.26 in
2.45 in
2.5 in
1:1
1.97 in
0.06 in
1:1
DO NOT SCALE DRAWING
AG
19–Oct–25
19–Oct–25
Alochrome 1200S
Anthony
Top_1
A4
1:1 UOS
1 of 1

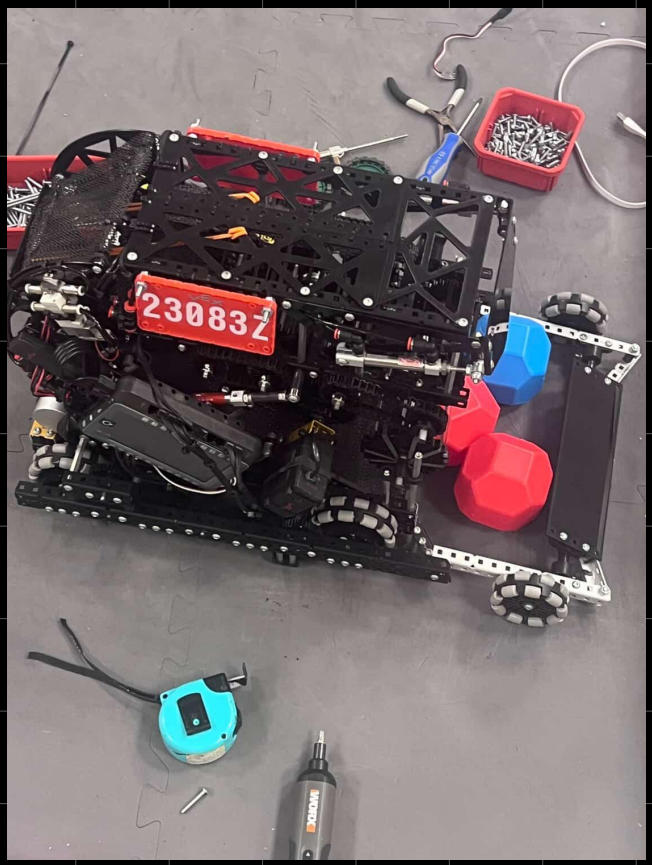
Descorer

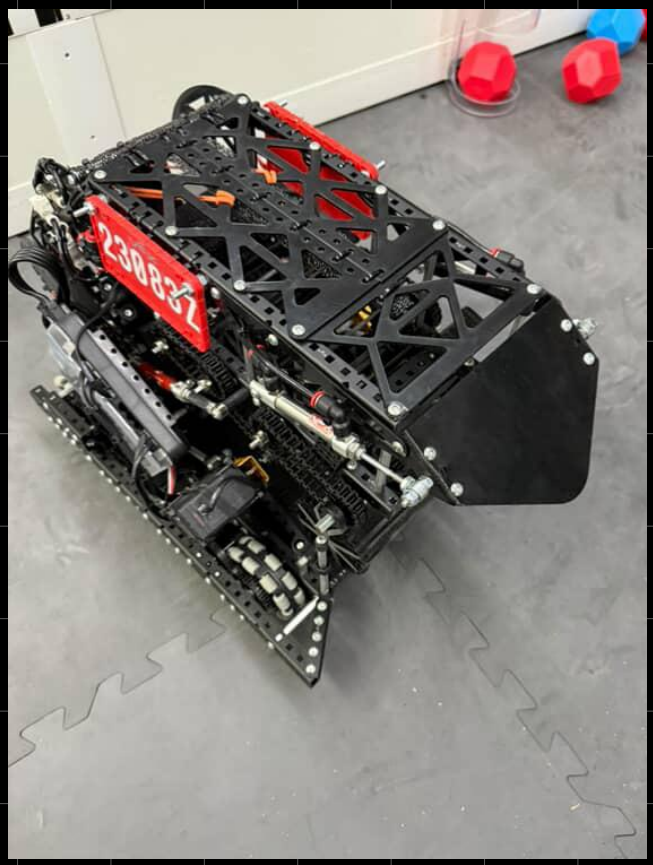
We laser cutted out the boards and tested successfully.



Robot 2

To this point, our robot is mostly finished, what remained was autonomous coding, which will be detailed further ahead. We are now ready for next TAI VEX event.





Programming

Button Mapping

Button	Function	Type	Implementation
Axis 2	Left drive	Analog	Tank drive control
Axis 3	Right drive	Analog	Tank drive control
R1	Intake forward	Hold	Both intake motors
R2	Intake forward	Hold	Both intake motors
L1	Lower intake only	Hold	Fine manipulation
L2	Lower reverse	Hold	Ring rejection
Y	Toggle shooter	Press	Pneumatic toggle
B	Triangle blocker	Press	Pneumatic toggle
Right	Intake extend	Toggle	Pneumatic toggle
Down	Aligner toggle	Press	Pneumatic toggle

Programming

Tank Drive

- Basically the same as previous

```
void usercontrol() {
    task intake(intakeControlTask, 0);
    task hangTask(hangControlTask, 0);
    task momogo(momogoTask, 0);
    task notetask(autonoteTask, 0);

    // Button bindings
    Controller1.ButtonY.pressed(shooterSwitch);
    Controller1.ButtonRight.pressed(intakecylanderon);
    Controller1.ButtonB.pressed(hang);
    Controller1.ButtonDown.pressed(alignerSwitch);
    Controller1.ButtonR1.pressed(shooterOn);
    Controller1.ButtonR1.pressed(alignerON);
    Controller1.ButtonR1.released(shooterOff);
    Controller1.ButtonR1.released(alignerOFF);

    while (1) {
        chassis.control_tank(100);  // 100% speed scaling
    }
}
```

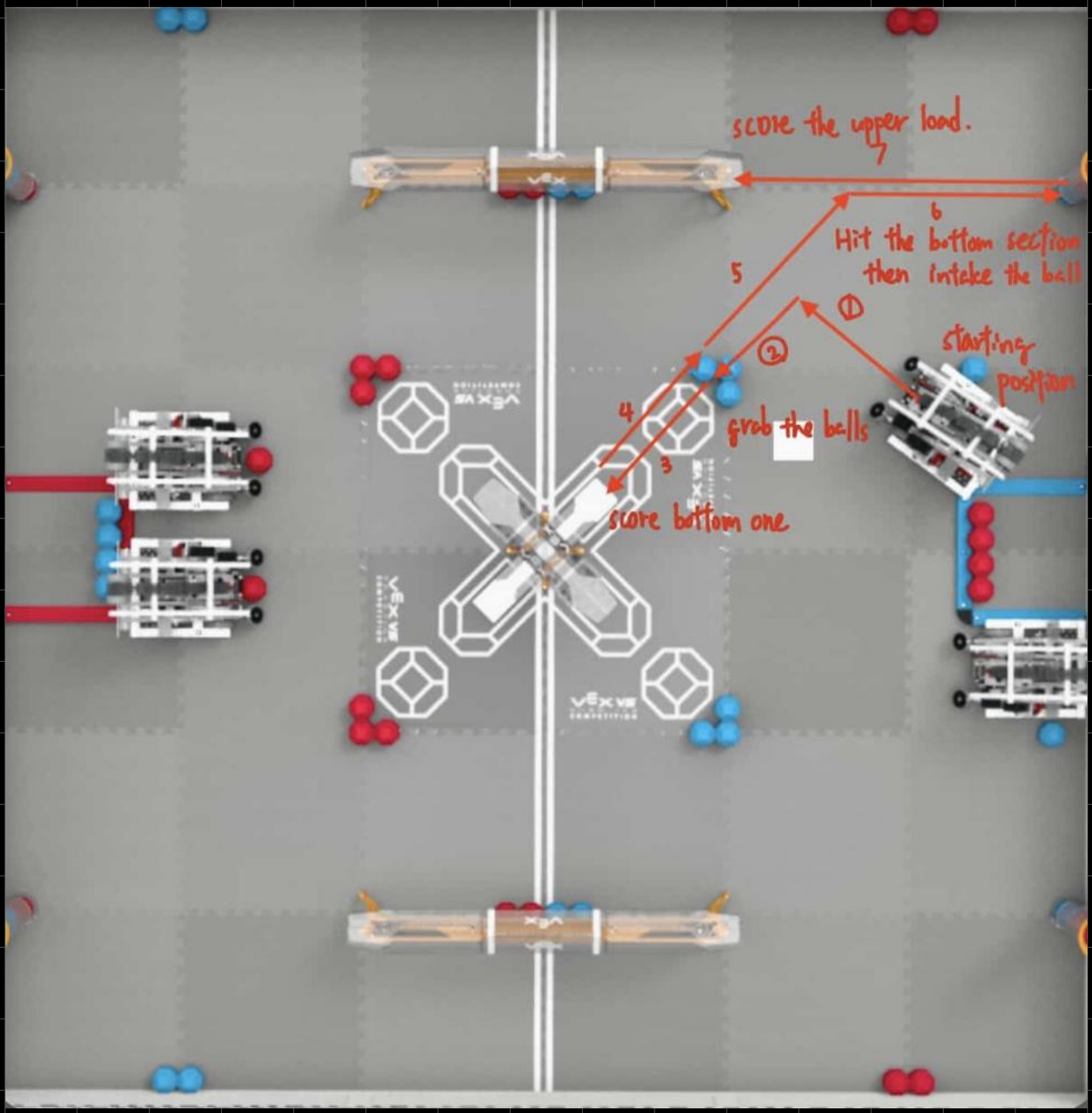
Programming

Intake Functions

Coast mode allows manual back-driving if jammed

```
int intakeControlTask() {
    while (true) {
        // Priority: L1 > L2 > R1 > R2
        if (Controller1.ButtonL1.pressing()) {
            intake.stop(coast);
            intakedown.spin(forward, 12, volt);  // Lower stage only
        }
        else if (Controller1.ButtonL2.pressing()) {
            intake.stop(coast);
            intakedown.spin(reverse, 12, volt);  // Lower stage reverse
        }
        else if (Controller1.ButtonR1.pressing()) {
            intake.spin(forward, 12, volt);      // Both stages forward
            intakedown.spin(forward, 12, volt);
        }
        else if (Controller1.ButtonR2.pressing()) {
            intake.spin(forward, 12, volt);      // Both stages forward
            intakedown.spin(forward, 12, volt);
        }
        else {
            intake.stop(coast);
            intakedown.stop(coast);
        }
        wait(20, msec);
    }
}
```

Autonomous Strategy



Programming

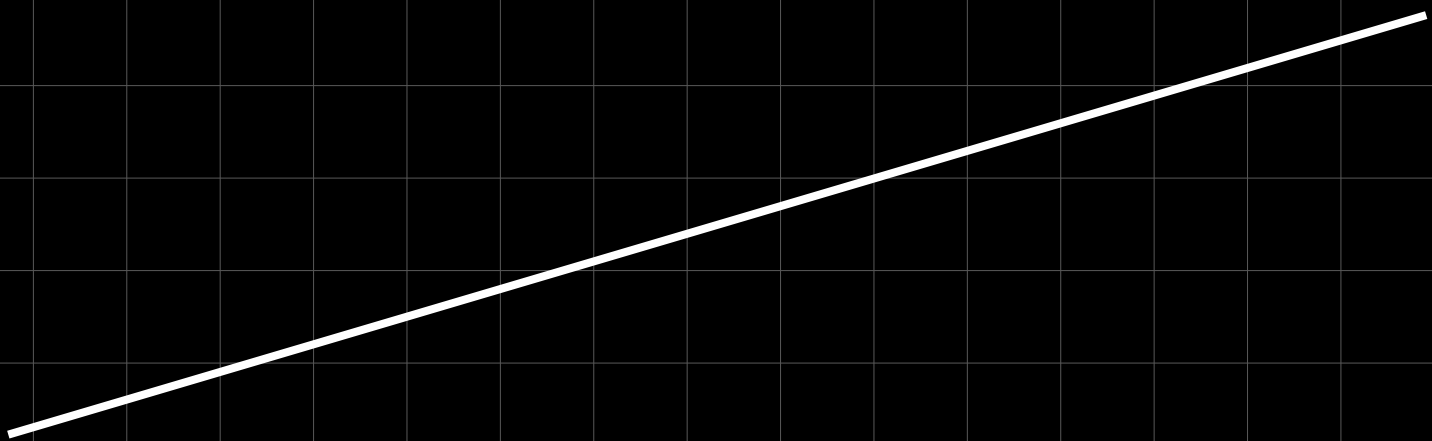
We continued our previous intake strategy and started to code.

As mentioned, we have three main functions to control the robot's path.

1. `void turn_to_angle(float angle)`
2. `chassis.drive_distance(inches, heading, DriveVolt, HeadingVolt)`
3. `void cos_move_distance_smooth(distance_in, angle_deg, turn_maxV, drive_maxV)`

We have to turn to the desired angle before moving, or else the k values would lead it to run unpredictable paths.

The main difference between drive_distance and cos_move_distance_smooth is that the latter accelerates and deccelerates and stops with brake to not let the deceleration continue unpredicatbilly.

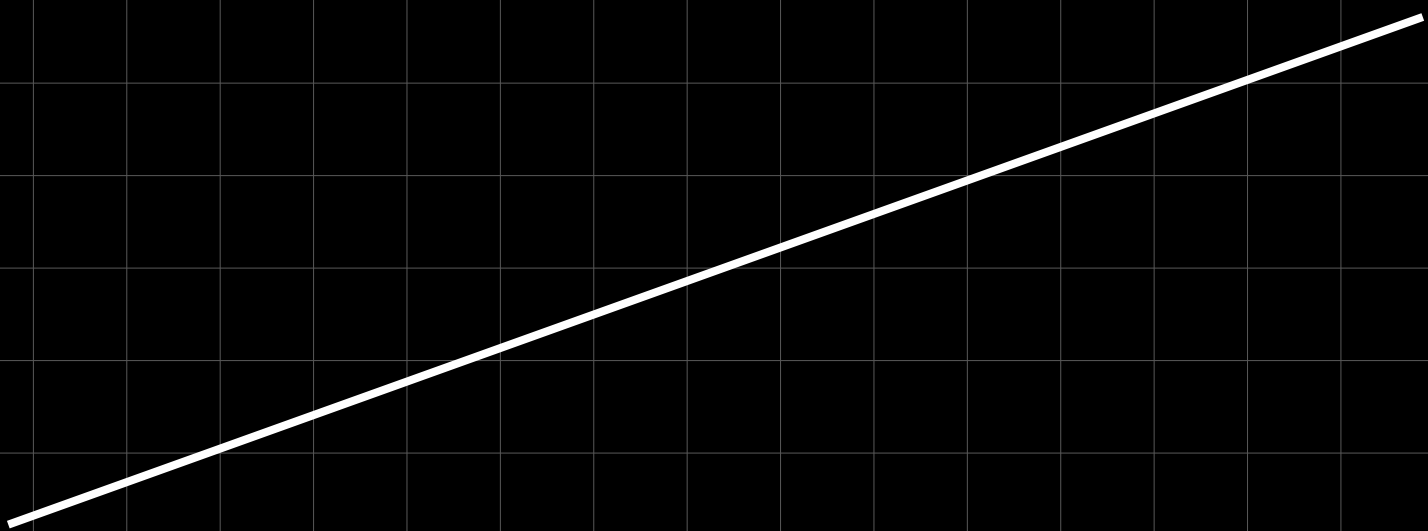


Programming

This is a demonstration of a snippet of our code:

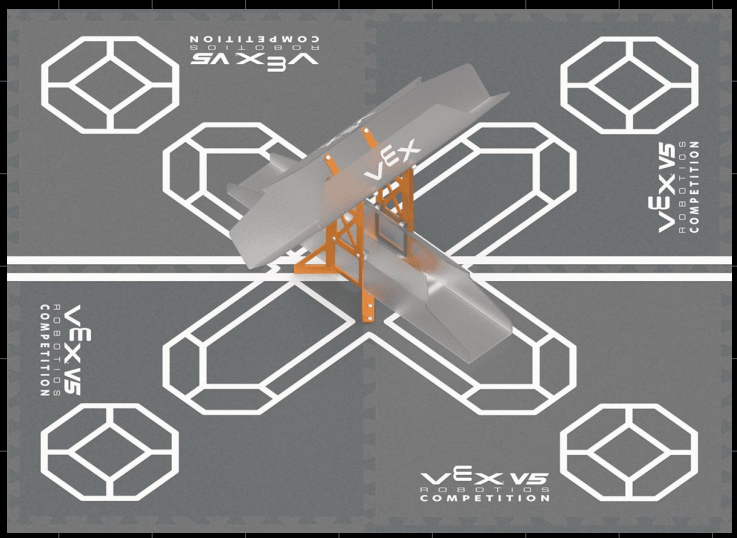
```
chassis.turn_to_angle(5); // to long goal
 wait(0.05,sec);
 chassis.drive_distance(12, 5, 6, 6);
 pushCylinder.set(true);
 wait(0.05,sec);
 shooter.set(true);
 wait(0.05,sec);
 cos_move_distance_smooth(4, 5, 6, 6);
```

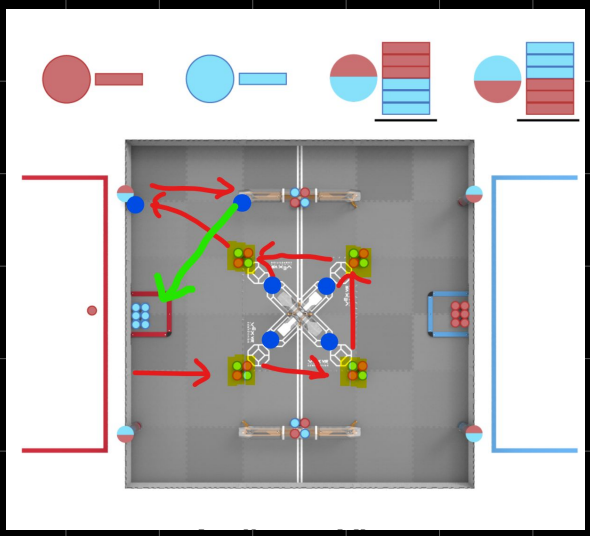
We use **`drive_distance()`** for most because it doesn't overheat the motors since it's using coast for braking. We use cos_move_distance_smooth for smaller distances because it stops with brake precisely and it would be more smooth and slow to prevent small fragments and tilting or fliping the robot.



Skills Strategies Ver.1

We also decided to play in skills in TAI VEX HS.
In consideration of our design of the robot and to align with the rules of driving skills, we will intake in the center and score the center goal first. We decided to go finish scoring the two center goals within the first fourty seconds and use the last twenty seconds for clearing one loader and score the balls into the long goal. Furthermore, we will park if we have sufficient time.





277 | Brainstorming | Anthony | 10/21

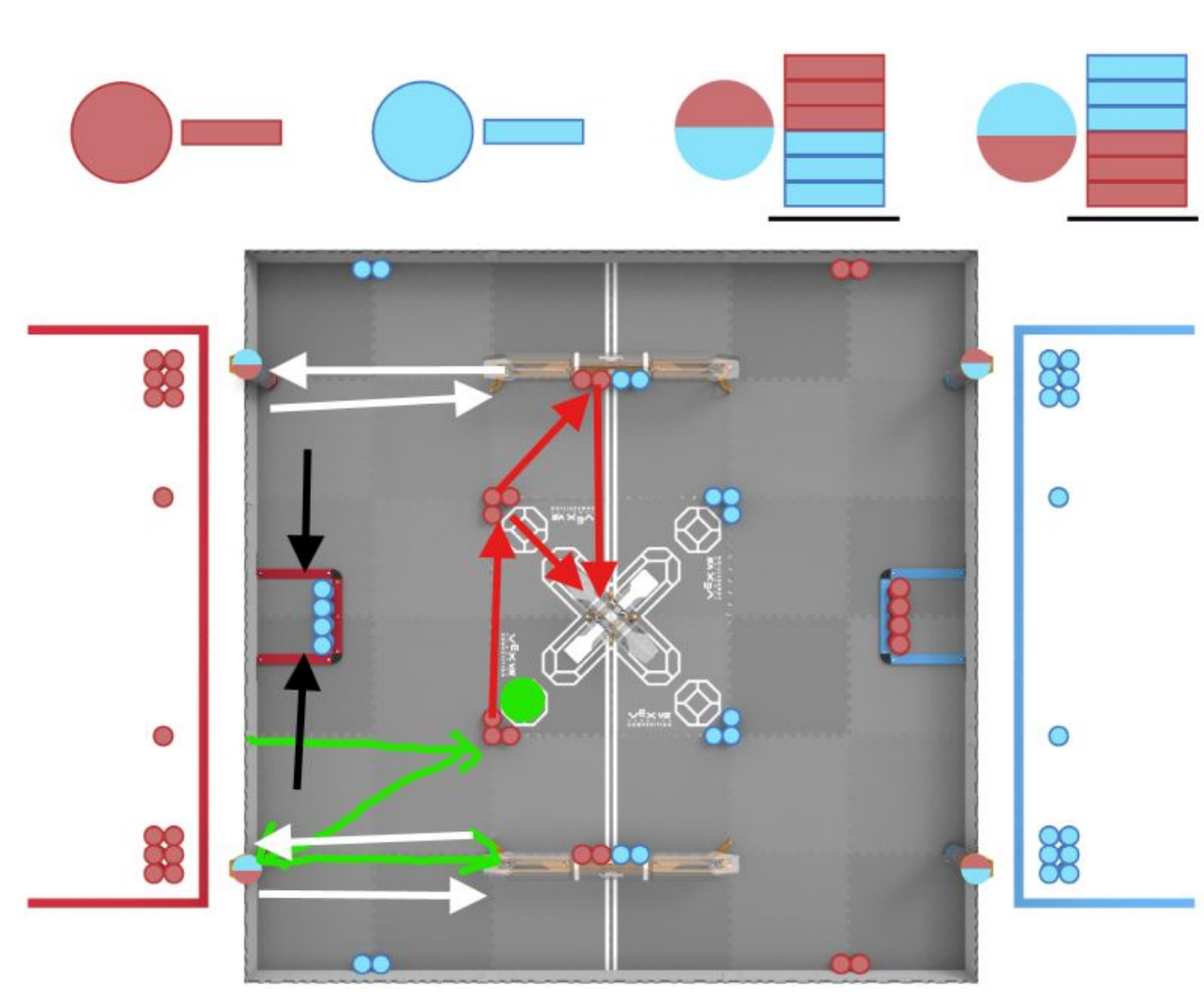
New Driver Strategies

In At the start of the match, the driver immediately drives to the center Triballs and pushes them into the center goal within the first 10–15 seconds. This will give our alliance early control of the scoring area and prevents opponents from taking the center. From 15 to 60 seconds, the driver focuses on collecting remaining Triballs from the defensive side and quickly adding them to the offensive zone. One important thing that we observed through the past matches is that the driver should avoid chasing single balls too far, because wasted time reduces total points.

Green: Auto
Red: initial stage
(10-15 seconds)
White: middle game
(15-60 seconds)
Black:final seconds
(parking)

New Driver Strategies

In the last 15–20 seconds, the driver positions the robot in the parking zone to park for the 8-point bonus.



Meeting Preparation

2025/10/24 — 60 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- Progress before competition - Division of work - Interview practicing

Discussion topics

Progress:

The event is tomorrow and we have listed out things unfinished:

1. Auto skills
   a. did not meet our plan and not tuned
2. Driver practicing
   a. we did not have enough time for the driver to practice
   b. therefore we also did not mock driver skills

However, we're better than last time and ready to face next battle

TAI VEX HS 26

MATCH RESULTS

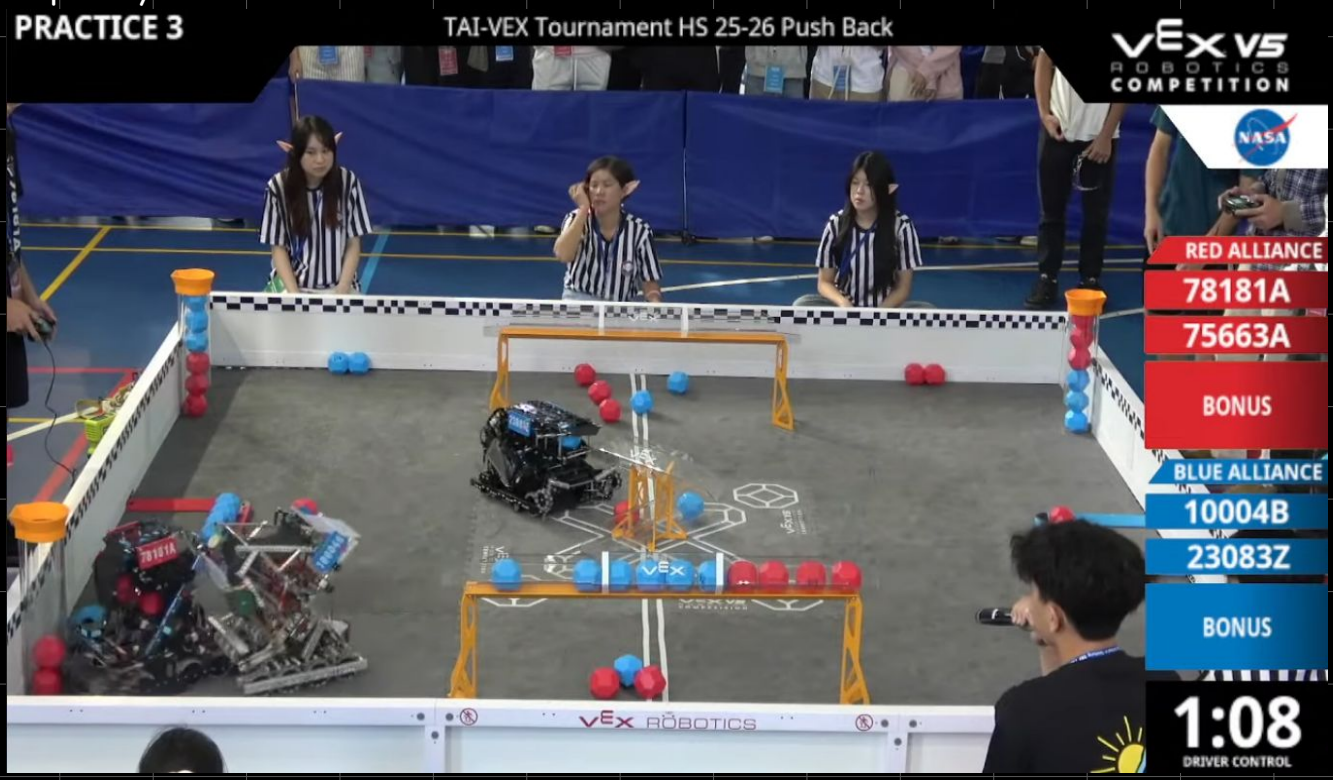
P3	78181A 75663A	42	65	10004B 23083Z
Q1	92641S 17022A	50	37	96969W 23083Z
Q10	23083Z 17022J	54	42	92641A 75663A
Q12	96969Y 11645A	98	18	23083Z 2779A
Q19	23083Z 66994T	87	28	92641C 10004B
Q24	66994B 78181A	155	3	23083Z 75016A
Q29	23083Z 1791V	69	21	75016B 66994D
QF 3-1	78181A 17022A	88	56	23083Z 92641A

TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

78181A 75663A	42	65	10004B 23083Z

Match P3 was a practice round that went fairly well. Our alliance's strategy worked since one of our opponents no-showed. As a result, our alliance took turns applying heavy defense on 78181A. However, 78181A was still able to execute a massive descore near the end of the round. Our alliance won nonetheless, but our coach argued that we should have used the practice round to test whether our bot was functioning properly.

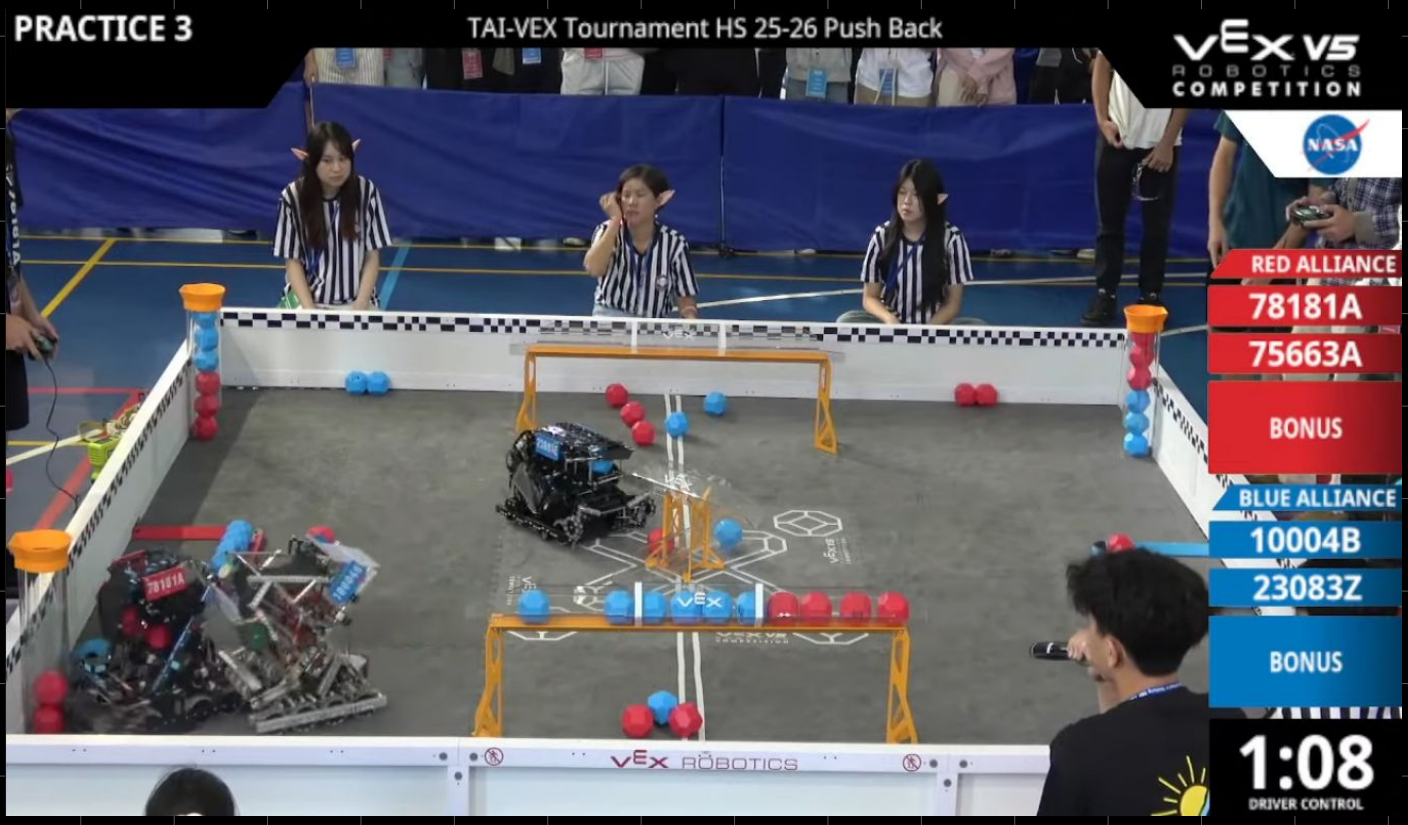


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

92641S 17022A	50	37	96969W 23083Z

Match Q1 was an embarrassing experience. Our intake motor malfunctioned because the rubber band was not in place. As a result, our team was unable to score much, while our opponent, 17022A, a particularly strong team, scored significantly even though their alliance was accidentally tripped at the beginning of the match.

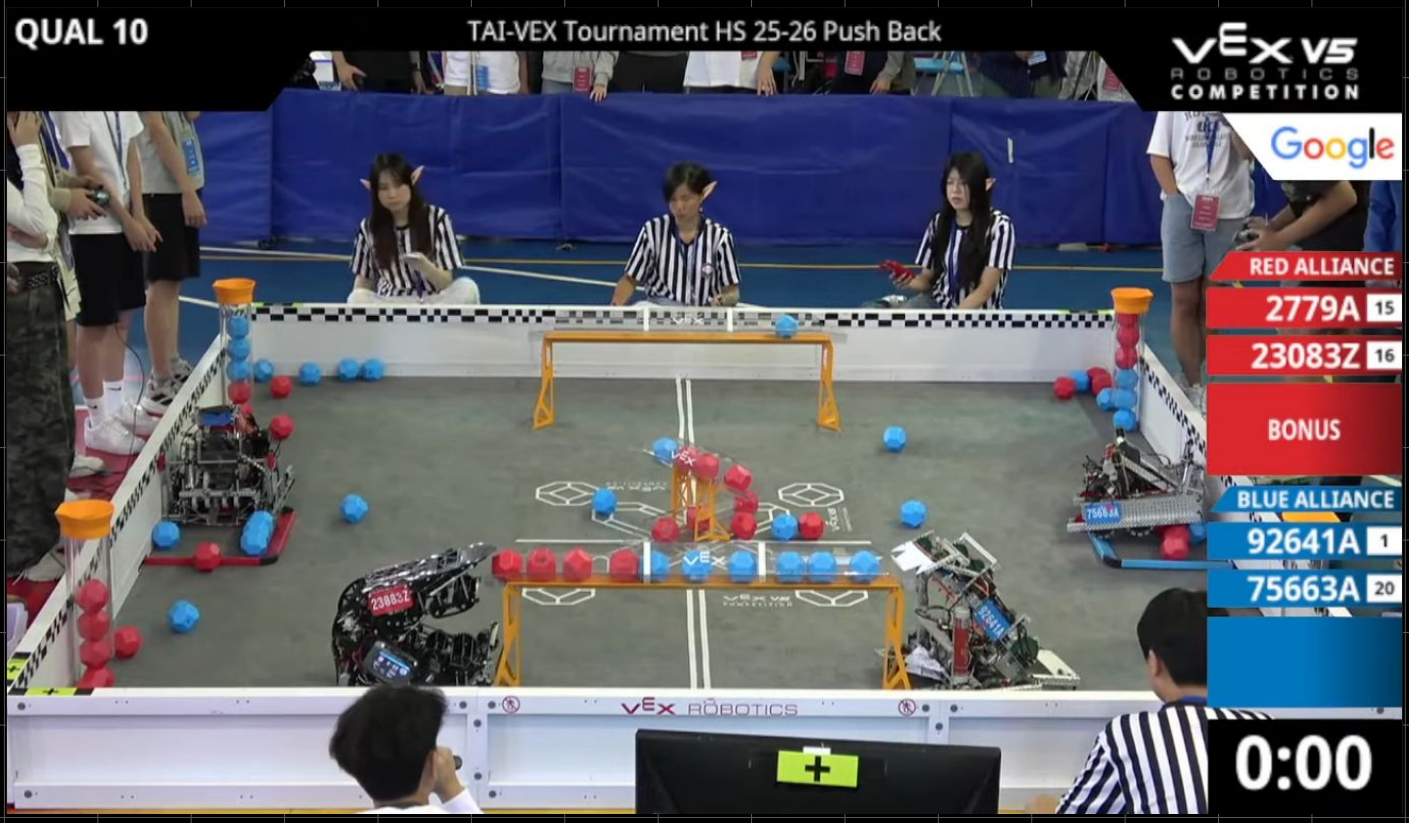


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

23083Z 17022J	54	42	92641A 75663A

Match Q10 went smoothly overall. Theoretically, our team should have dominated since both our alliance and opponents appeared weaker based on our analysis. However, our bot was repeatedly blocked by a pan-shaped bot, and our maneuvering was not very effective. One opponent camped near the long goal, while another effectively gatekept the central goal. We realize we should have a better maneuvering technic.

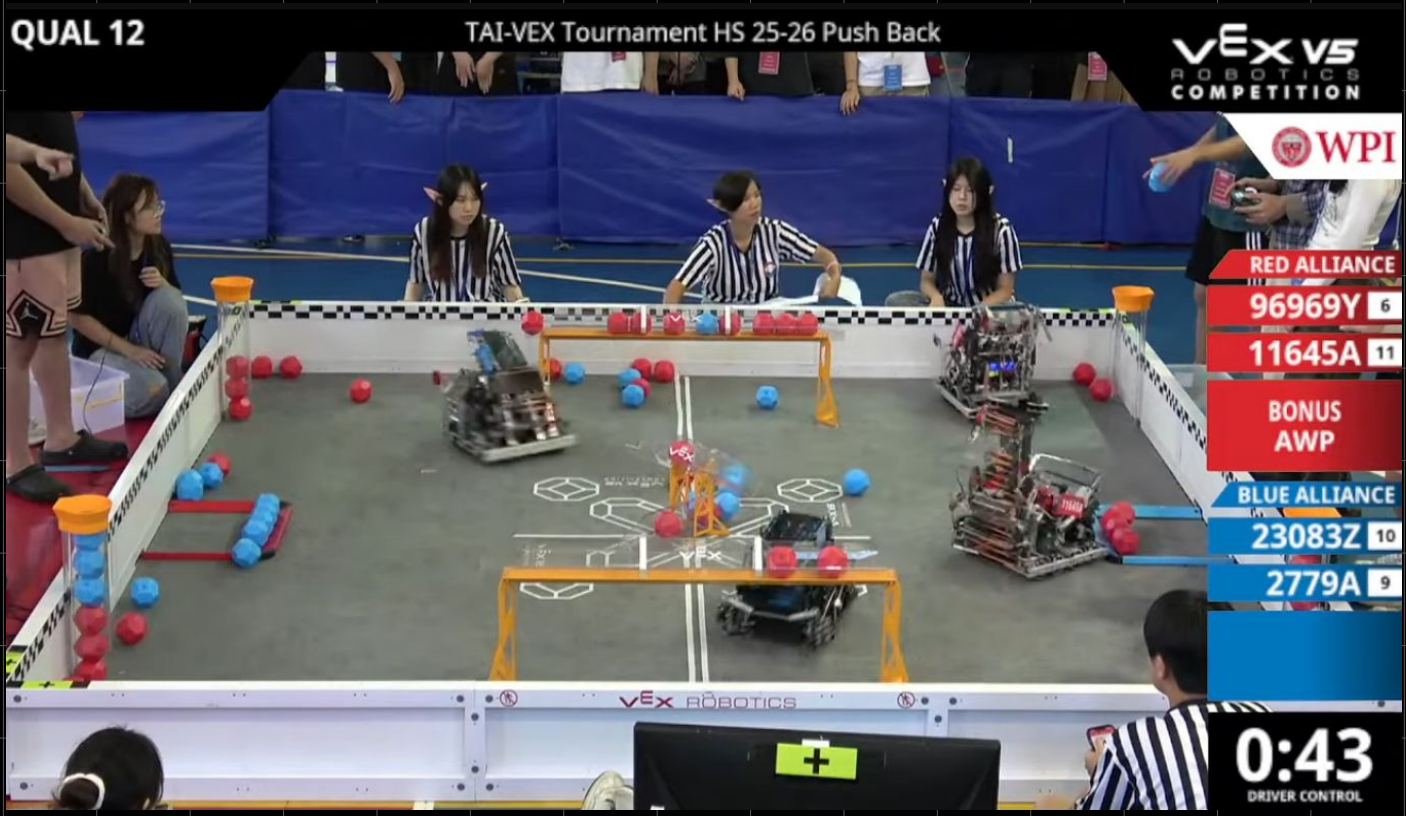


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

96969Y 11645A	98	18	23083Z 2779A

Match Q12 was a disaster. Our opponent applied heavy defense on our bot throughout the match. Our ally was largely ineffective, and we missed many blocks while scoring long goals. We need to improve our scoring speed, as our bot is too easily pushed away by opponents during scoring.

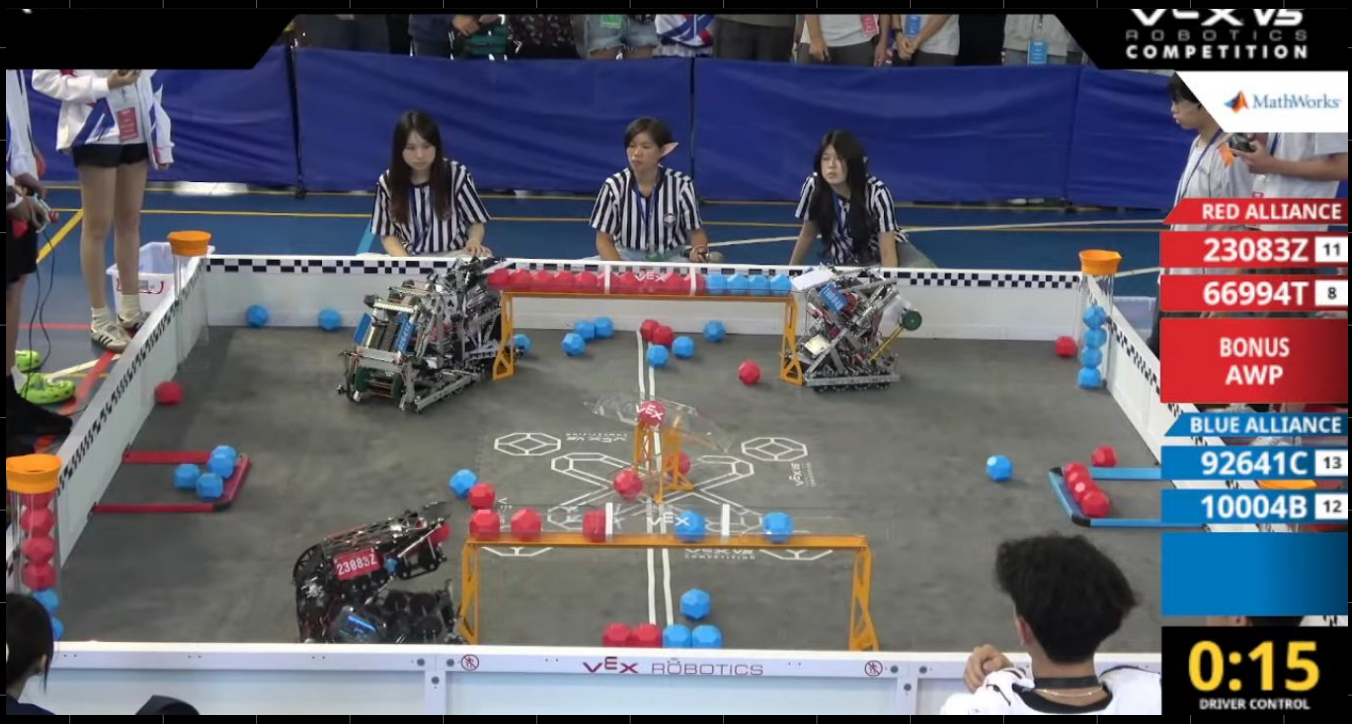


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

23083Z 66994T	87	28	92641C 10004B

Match Q19 went very well. Our alliance was strong, and their bot singlehandedly secured the long-goal control zone after the autonomous phase. Our alliance focused on camping at the long goal to attempt descoring our opponents' remaining blocks, while one opponent did the same. We tried to push them away but were unsuccessful due to our low mass. However, we successfully scored 4 blocks into the other long goal and parked.

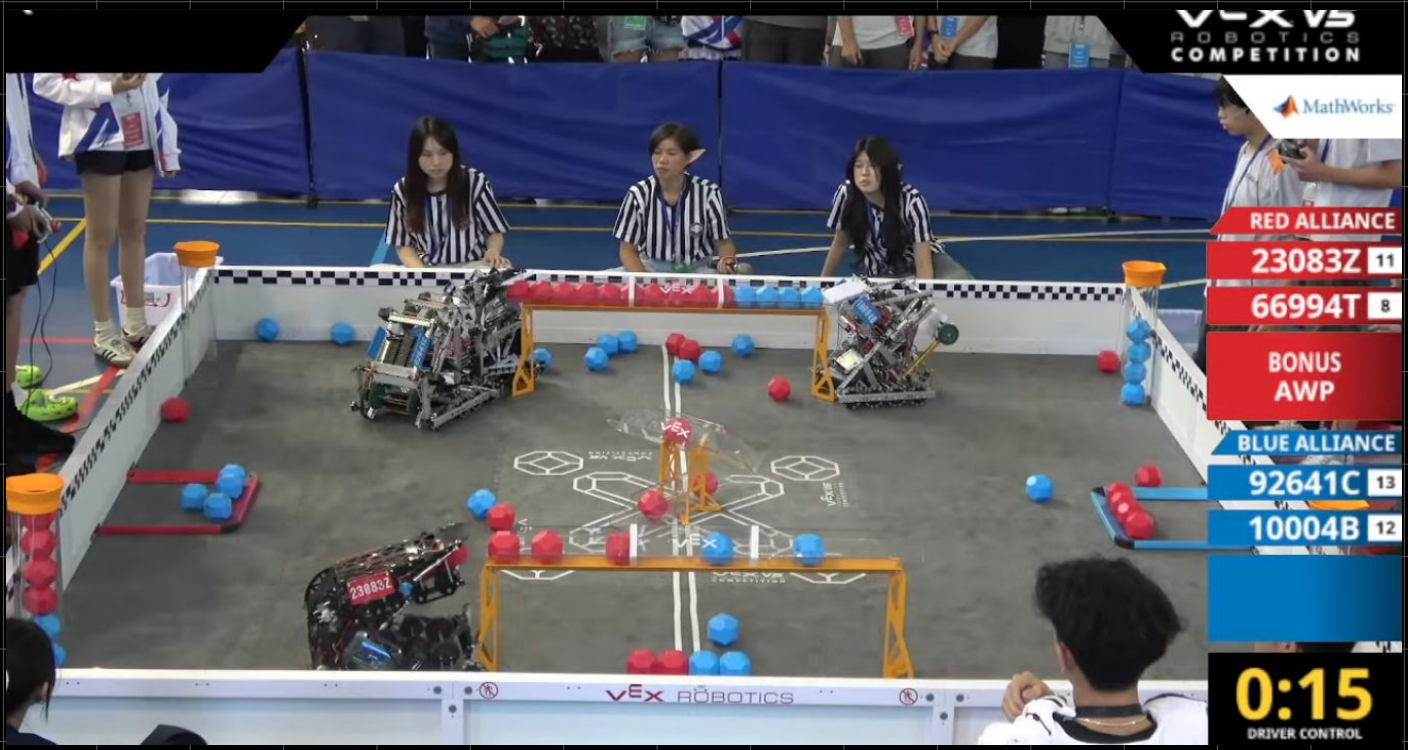


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

66994B 78181A	155	3	23083Z 75016A

Match Q24 is unreviewable. Our team was disconnected at the beginning of the match.

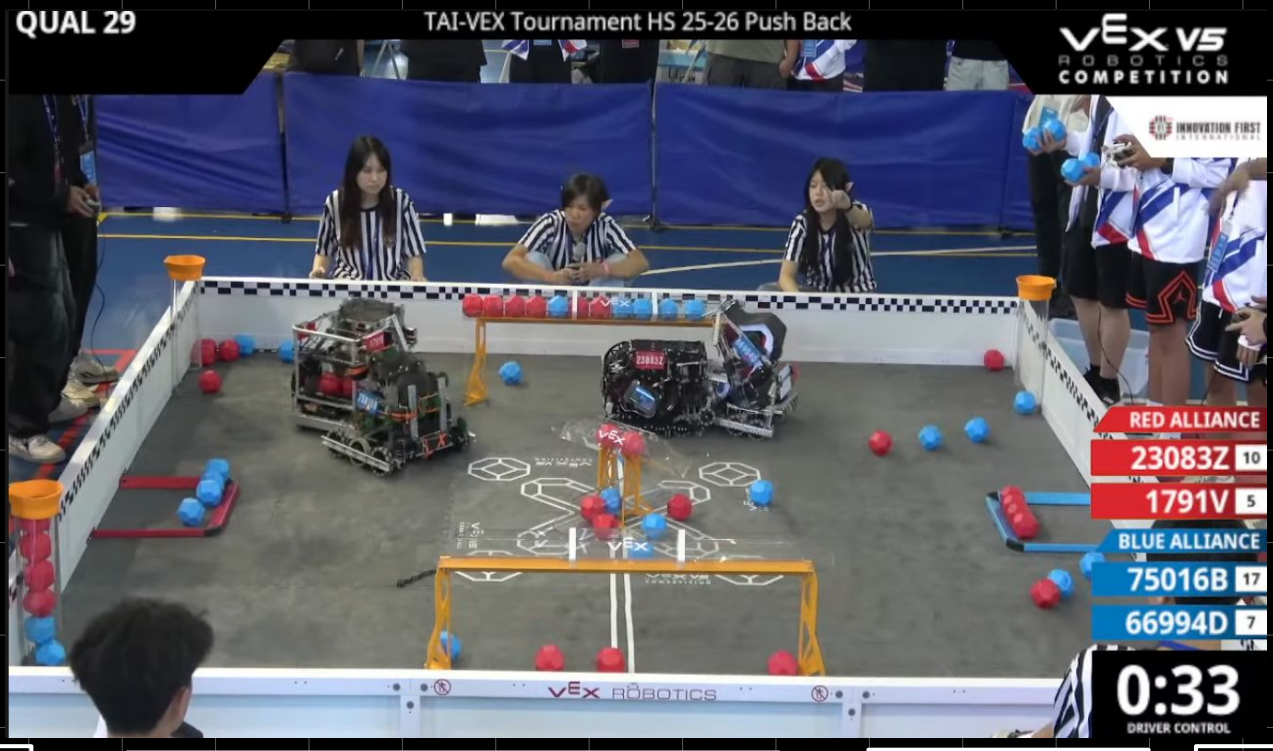


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

23083Z 1791V	69	21	75016B 66994D

Match Q29 was absurd. Our bot played defensively while our alliance focused on offense by targeting the long goal. Our alliance managed to score a considerable number of blocks, and our bot successfully blocked the opponent's wing, allowing our alliance to execute a successful descore. Although we won by score, our team were disqualified because we did not hear the head referee's three-second count while defending against the opponent.

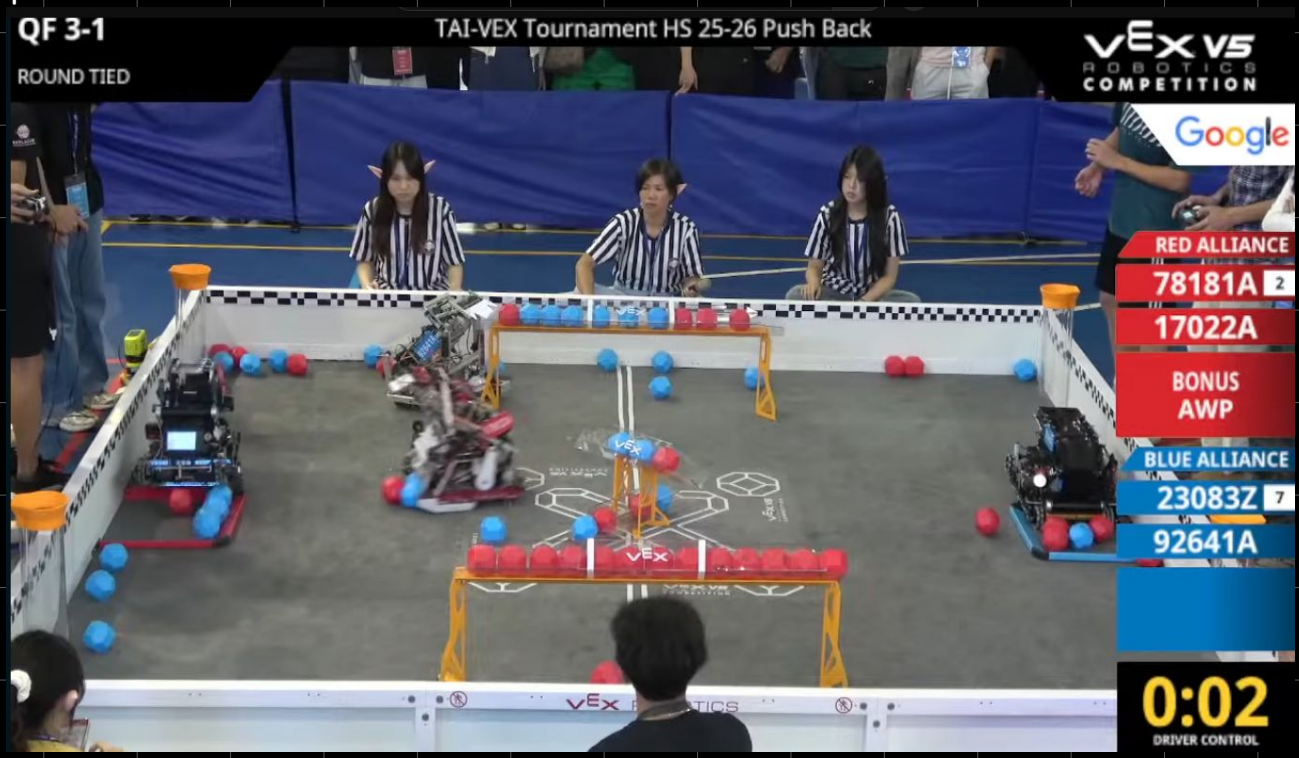


TAI VEX HS 26

REFLECTIONS & IMPROVEMENTS

78181A 17022A	88	56	23083Z 92641A

Match QF #3-1 , we faced a formidable opponent that ultimately placed second. The key takeaway from this match is the need for a strong descoring mechanism, as our opponent, 17022A, scored an entire tube in the first half of the match and we were unable to descore it. This directly led to our decision to add a wing after the tournament. Additionally, our scoring speed is slow, and the final blocks often get stuck. We still have much to improve.



Analysis of Others' Robots

Team: 78181A

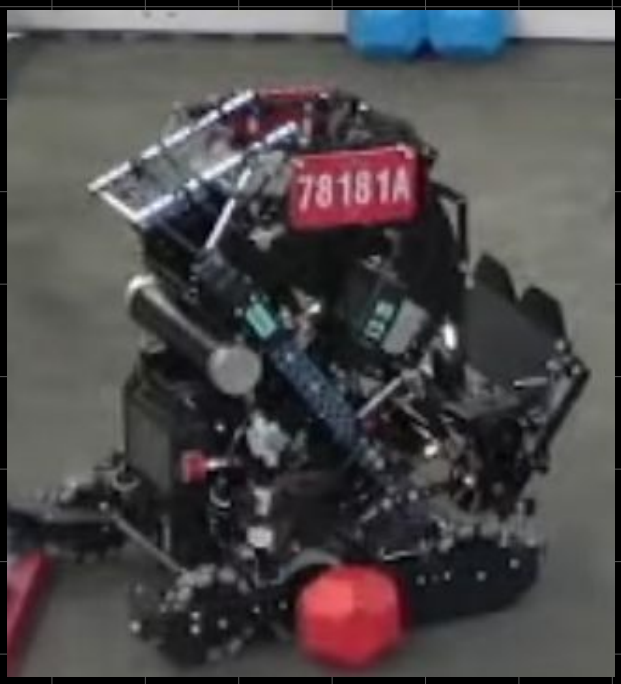
Tournament: TAI VEX HS 26

Key Features:

- High-mobility S-bot design
- Strong descoring mechanism
- Fast cycle time
- Excellent loader-clearing ability

Summary:

78181A is a fast, highly efficient S-bot with exceptional descoring and loader control, an opponent we greatly admire.



Analysis of Others' Robots

Team: 66994B

Tournament: TAI VEX HS 26

Key Features:

- Championship-winning basket bot
- Extremely strong loader intake (~6 blocks per second)
- Dual-sided wings for fast descoring
- High-capacity basket (~10 blocks)

Strategy & Performance:

- Rapid multi-block intake and high-throughput scoring
- Used wings for quick and effective descoring

Summary:
66994B is a high-speed, high-capacity basket bot with powerful intake and descoring, making it the dominant robot of the tournament.

Analysis of Others' Robots

Team: 66994T

Tournament: TAI VEX HS 26

Key Features:

- High-mobility slope bot design
- Ruiguan-style slope variation
- Dual-sided wings for fast descoring
- Strong driver control

Strategy & Performance:

- Executed quick descoring with wings
- Fast and consistent long-goal scoring

Summary:

66994T combines a proven slope-bot design with excellent driver control for fast descoring and efficient long-goal scoring.

06
ROBOT 2.1

Meeting Logs

2025/11/02 — 45 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson - Benjamin	- New member - Improvements - Planning

Meeting information :

We have a new joined team member Benjamin, who will be mainly responsible for programming and aligner.

We started to rebuild thie car and fix the problems that itoccur during the previous competition. Our leader gives each of us a job to do and plan to finish repairing the car after two weeks. We aalso found out some important functional parts of robot that other team has attach and plays a very important role in each battle. We decide to create these parts to maximize our car's ability.

Reflections

After discussion and reflecting our TAI-VEX event, and we focused on major defects we now needed to improve, proved in the event.

1. The height of extending piston for the second level wasn't enough, leading to the robot not able to efficiently and consistently score into the long goal.
2. Our robot also had the problem of not able to intake blocks immediately at the point which it intake touches it.
3. We did not have an effective descore mech that was able to quickly descore the opponent's blocks, and we found out this a major obstacle.
4. We also didn't have an aligner that was able to help us align upon the goals fast enough.
5. We also weren't able to cross the parking zone, which decreased our scoring for skills and the parking score in matches.

Building Timeline

Tasks	11/2-11/8	11/9-11/15	11/16-11/22	11/23-11/30	12/1-12/7
Shooter Design				SAT Prep	Exam Prep
Intake Design					
Aligner					
Descorer (wing)					
Programming					

Tasks per member:

- Shooter: Pennlaine, Adam
- Intake design: Anthony, Jefferson
- Aligner: Pennlaine, Adam
- Wing: Benjamin, Adam
- Programming: Pennlaine
  - Simulator: Anthony

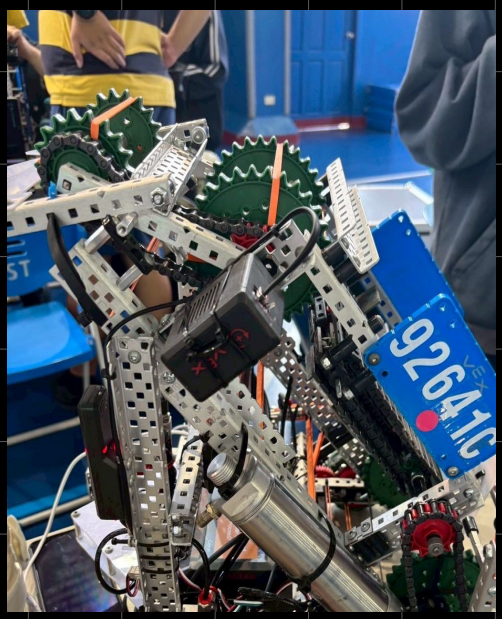
Shooter Height

Aligning to our to-do list, the first one was the easiest, in which we extend the pistons length using standoffs, and because we were using spacers when it retracts, this also allowed it to maintain and not affect the previous height when pistons are retracted

Shooter Design

The second issue is our weak descoring ability. Since it interferes with most of the upper part of the robot, we might as well redesign the intake in the meantime, which mainly comes from the lower part of the structure. While we were brainstorming away our descore mechanism, we ran into a bunch of problems that kind of clash into each other:

1. Our previous descore board was neither long enough nor high enough.
2. However, it provided the function of able to impede blocks from falling out since the intake is always rolling.
3. The most used mech was able to be in use because their last few outake sprockets were individually controlled by another motor. →

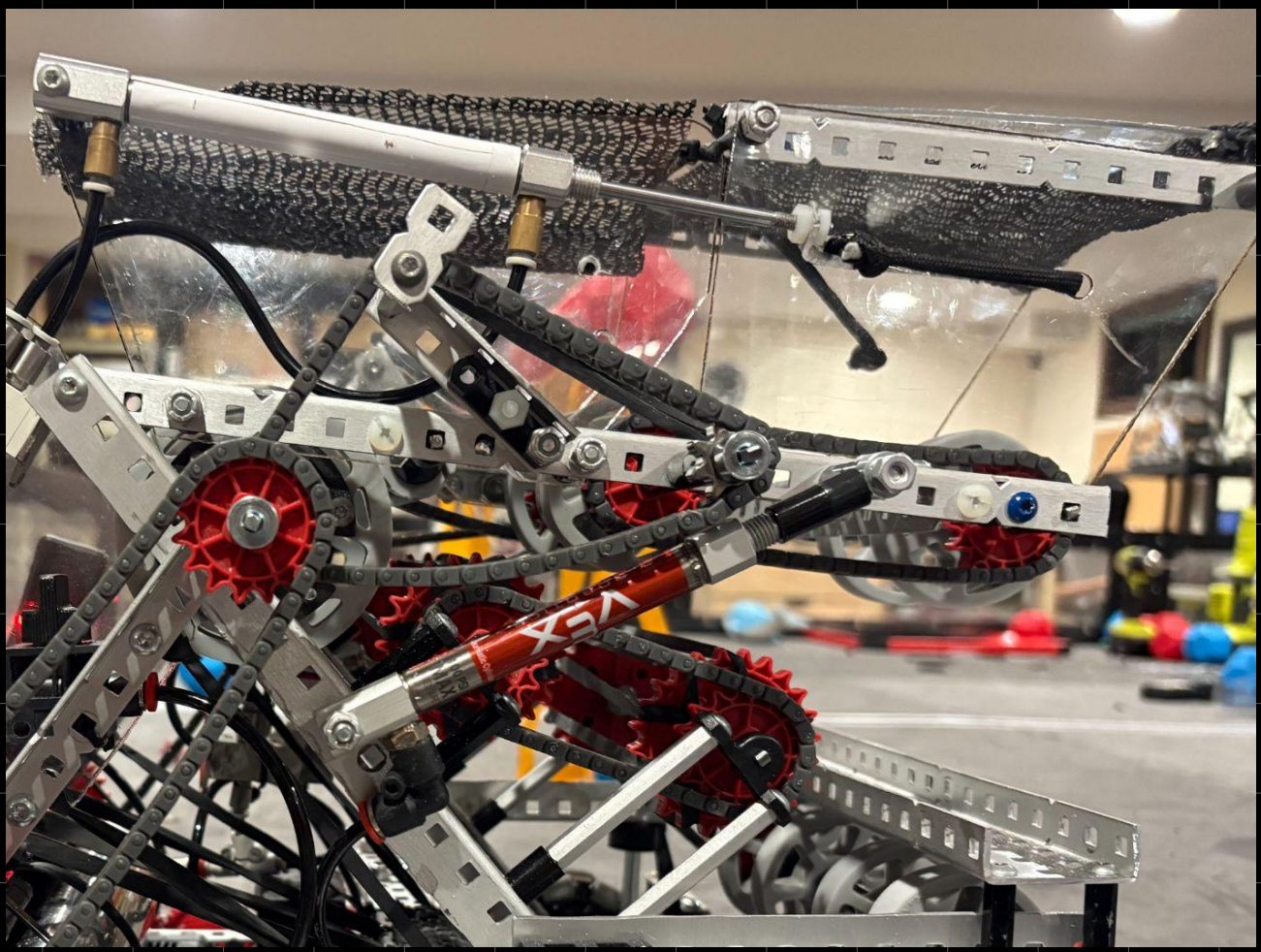


Shooter Design

Therefore, we have to think of a mech that **allows the blocks able to spin freely in the shooter while the intake was on**, but at the same time blocking the blocks to prevent them from falling out.

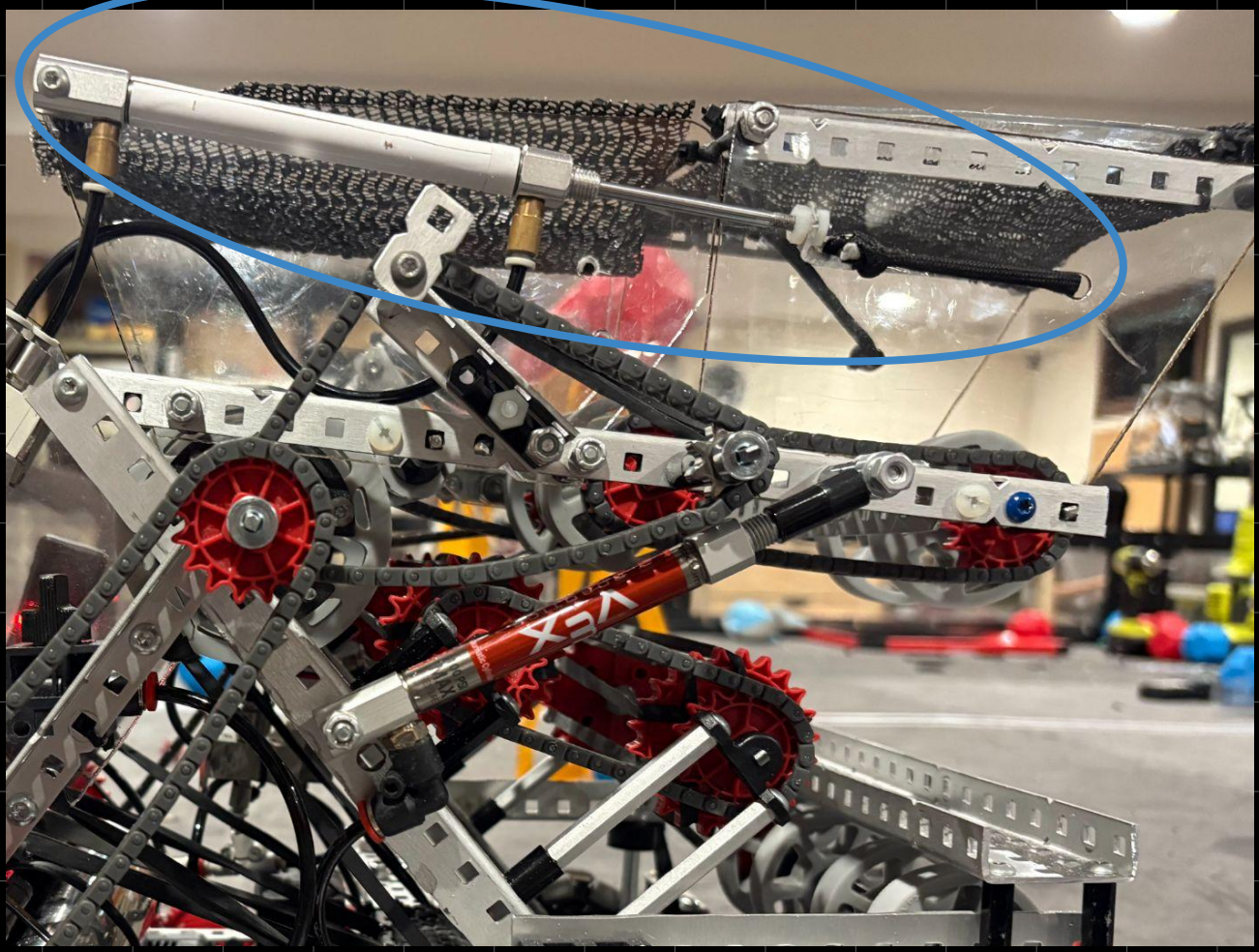
Research:

8889A has a similar design as us, and they exactly meet our problem.



Shooter Design

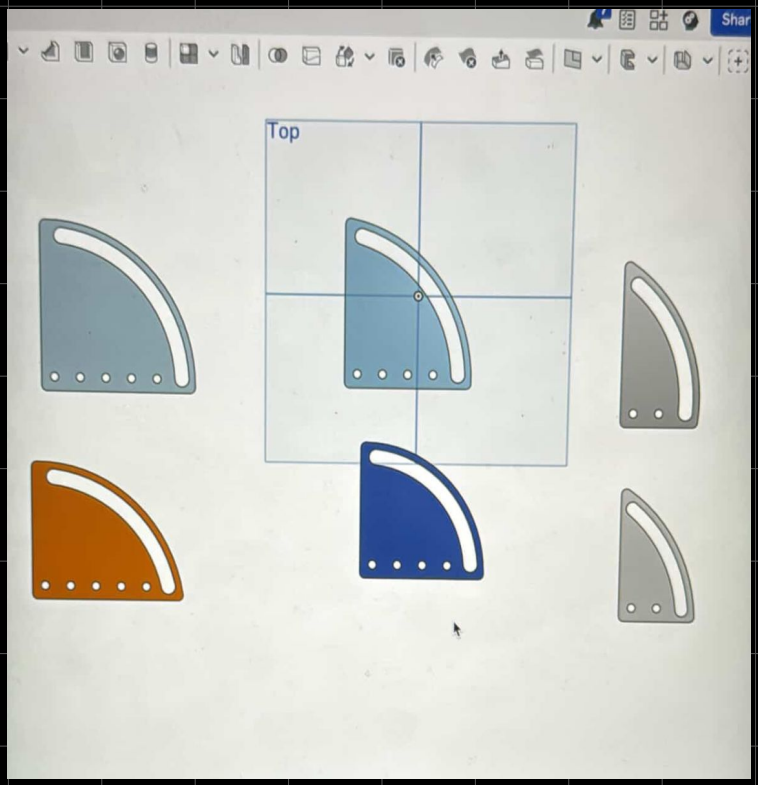
8889A used a unique mech to solve this problem. They used pistons to pull a string, and when the pistons are extended, the string would be loose enough for the blocks to past through and score. When the pistons are retracted, its tension would prevent blocks to pass through and fall out.

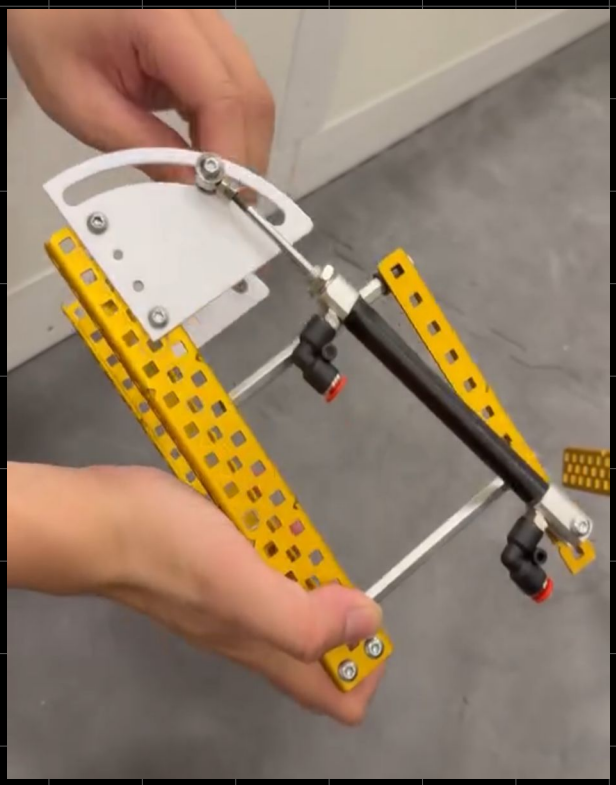


Shooter Design

We built a prototype to test our solution: using pistons as a stopper. When the pistons are retracted, it pulls the standoff up, right in the middle of the shooter pathway, creating a physical barrier that keeps the ball from moving forward.

This way, even with the descorer sitting higher, we can control the ball flow without everything spilling out.

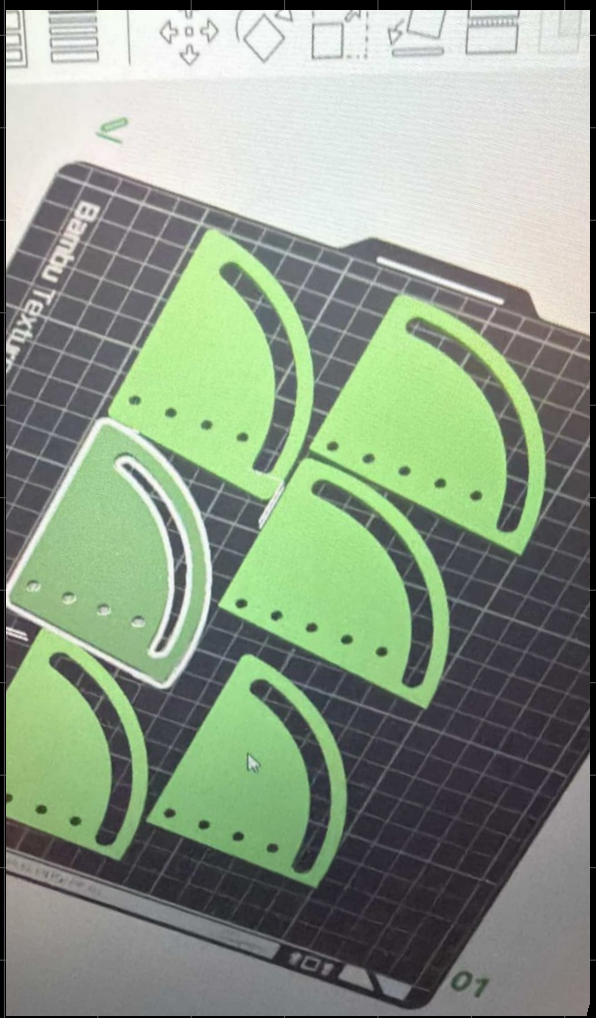


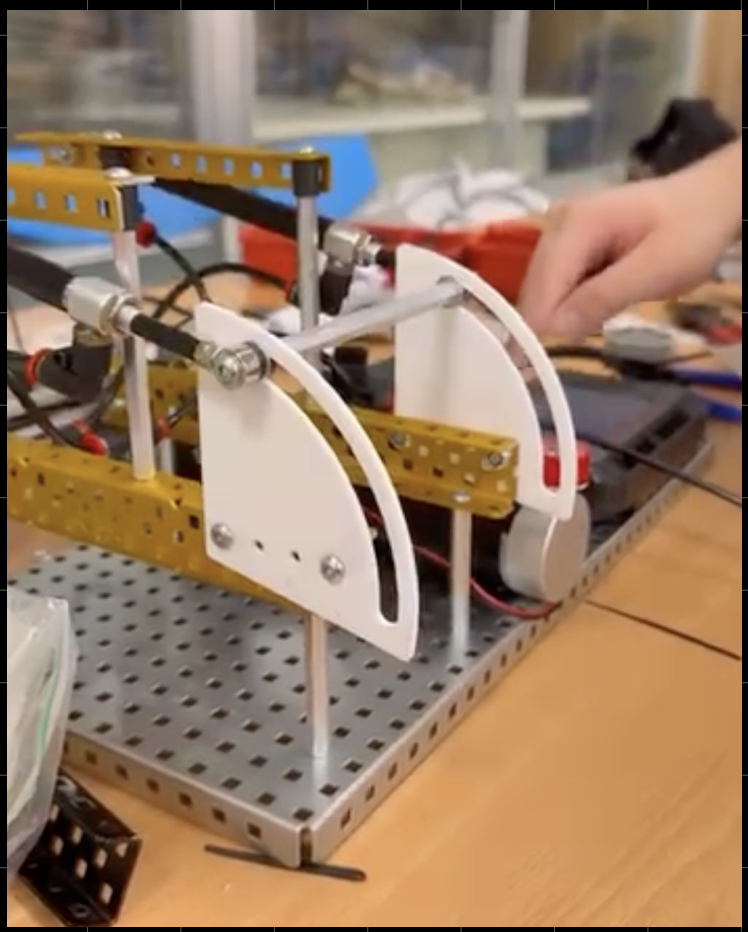


Shooter Design

To design the perfect shape for the height incorporating with the piston extension route.

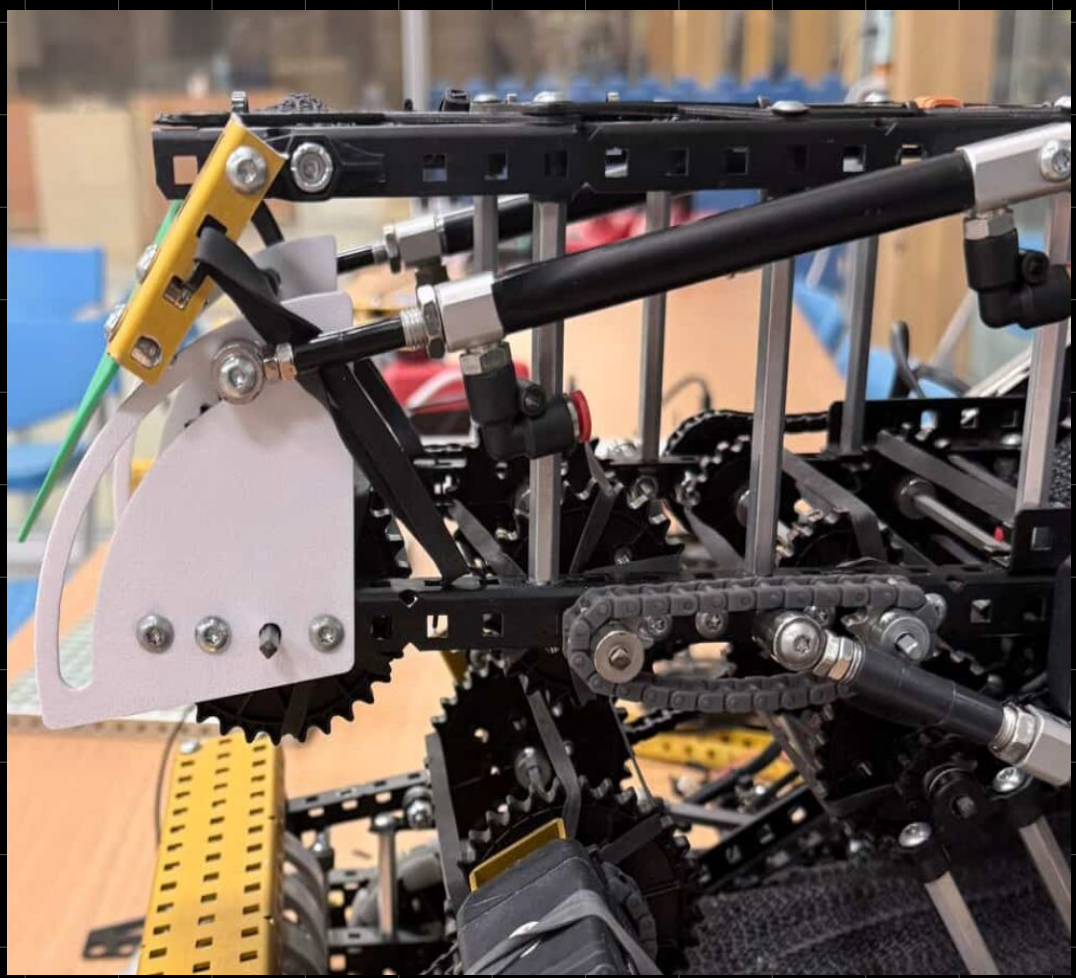
We 3d printed out several prototypes and determine the optimal one.





Shooter Design

We transplanted the prototype to our robot and tested the whole system out, we found several aspects that could improve this mech.

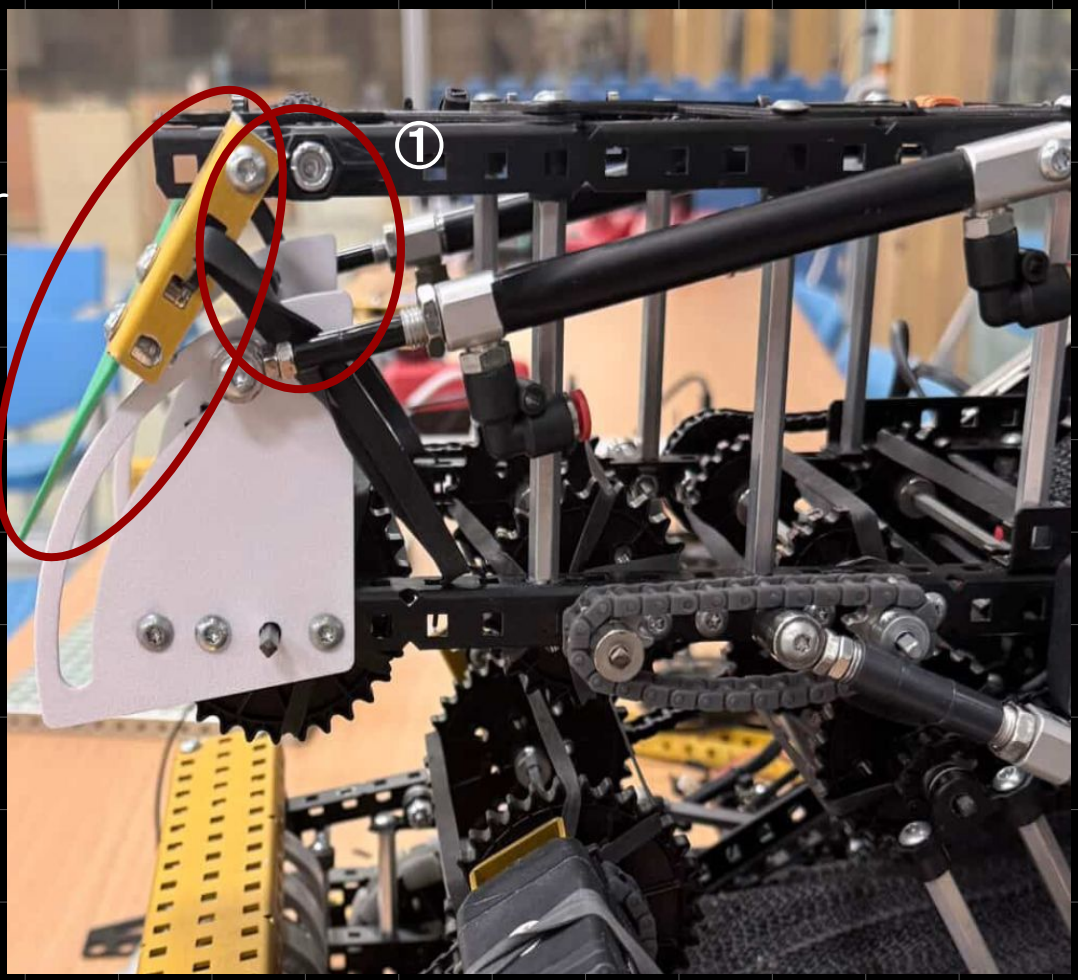


Shooter Design

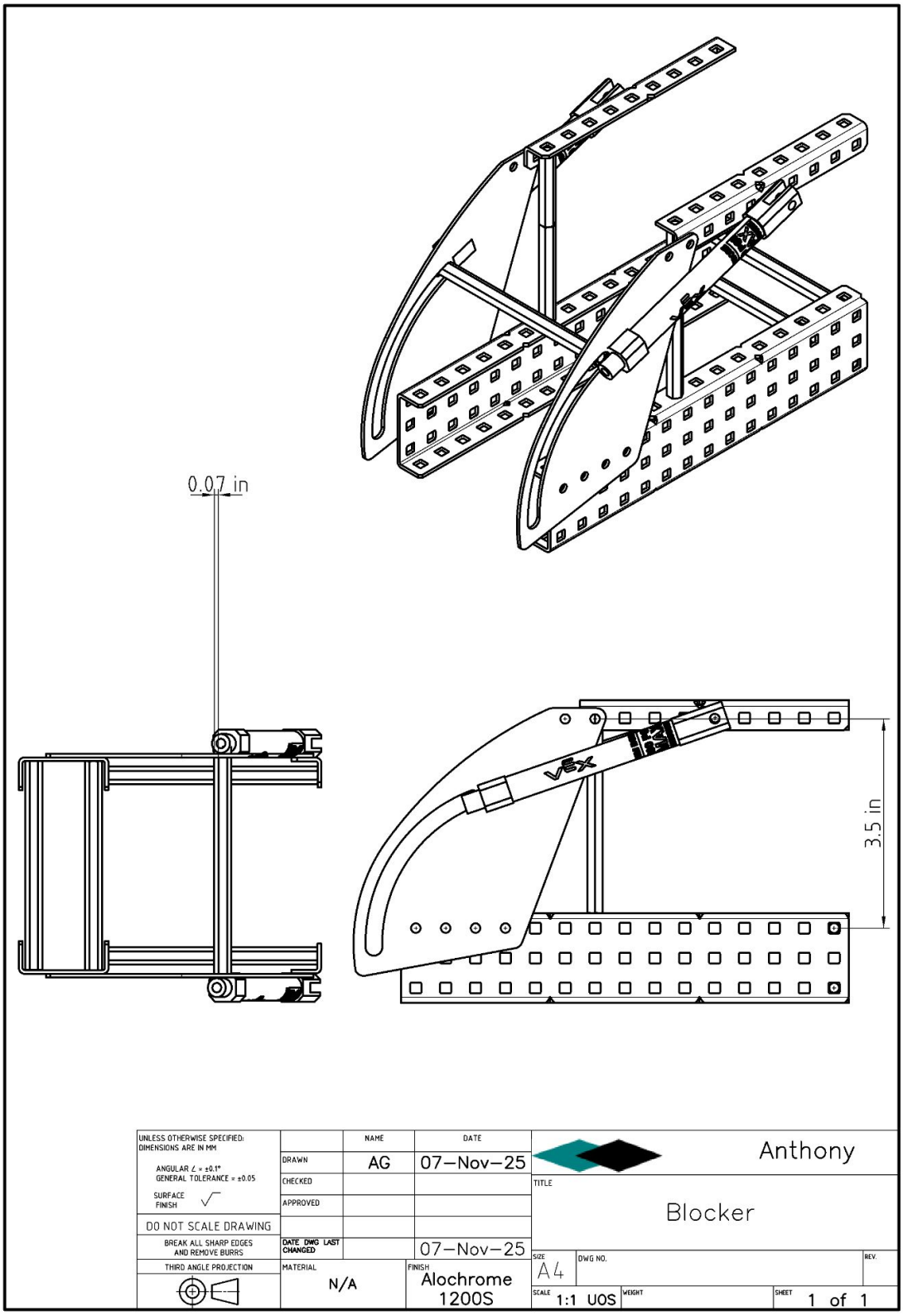
From our 3d printed prototype, we noticed that:

1. We could extend the board to the upper aluminum angle to improve both the plastic boards and the shooter system's stiffness.
2. Because now we can build a flexible descorer board, it needs a limit holding hole to maintain it at the best height for descoring upon the goals.

Flexible Descorer ② →



Shooter Design



		NAME	DATE
UNLESS OTHERWISE SPECIFIED: DIMENSIONS ARE IN MM	DRAWN	AG	07–Nov–25
ANGULAR ∠ = ±0.1° GENERAL TOLERANCE = ±0.05	CHECKED		
SURFACE FINISH √	APPROVED		
DO NOT SCALE DRAWING			
BREAK ALL SHARP EDGES AND REMOVE BURRS	DATE DWG LAST CHANGED		07–Nov–25
THIRD ANGLE PROJECTION	MATERIAL	N/A	FINISH Alochrome 1200S

Anthony

TITLE: Blocker

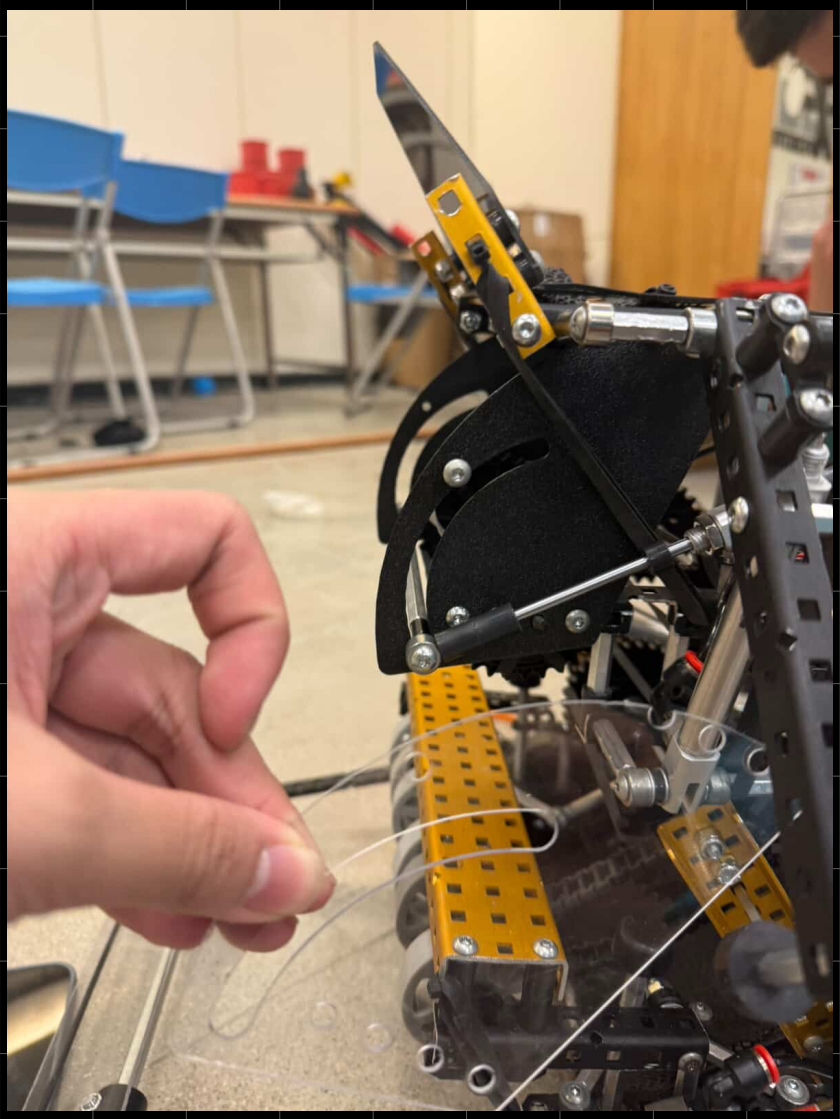
SIZE A4 | DWG NO. | REV.

SCALE 1:1 UOS | WEIGHT | SHEET 1 of 1

Shooter Design

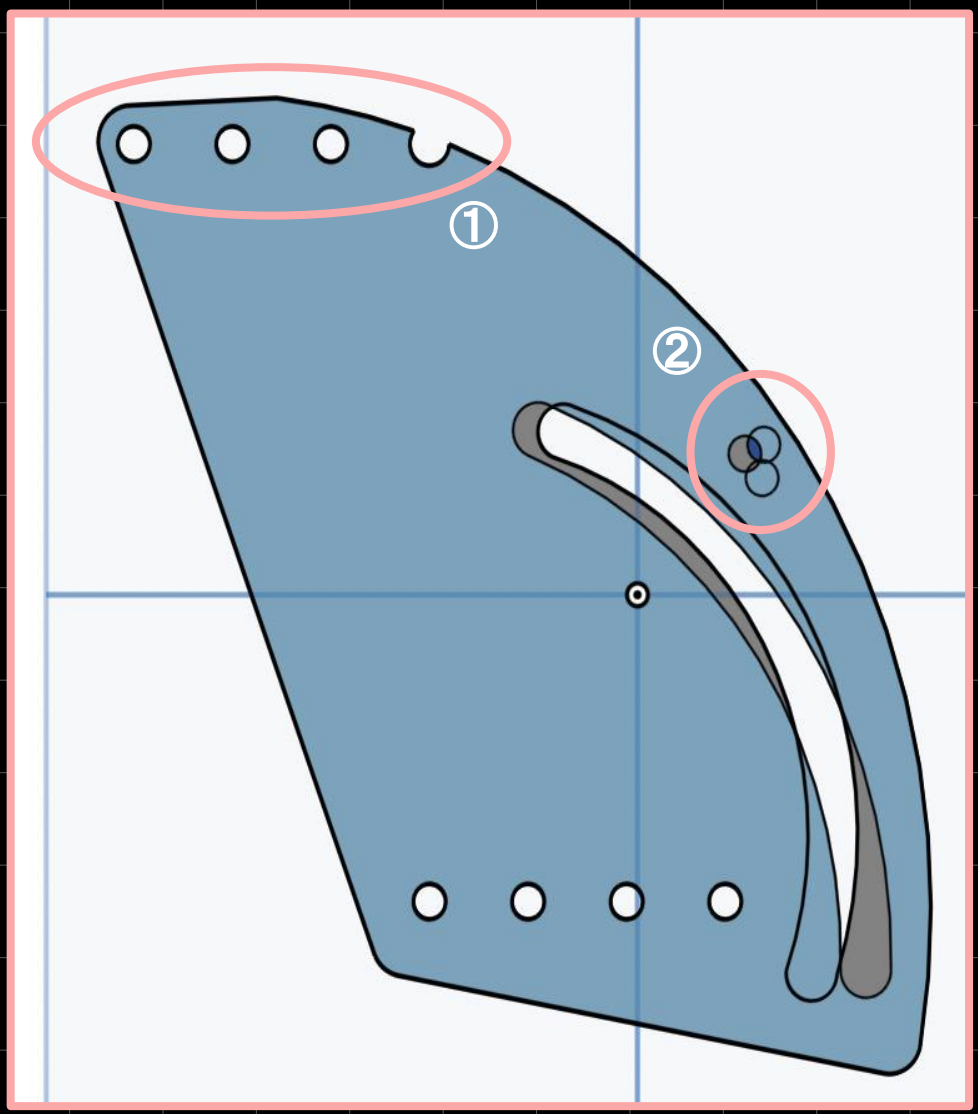
After determining the plastic board shape, we cutted out transparent plastic board and replaced the 3d printed boards.

This is our first CNCed plastic board since we have just received our new CNC a few days ago!



Shooter Design

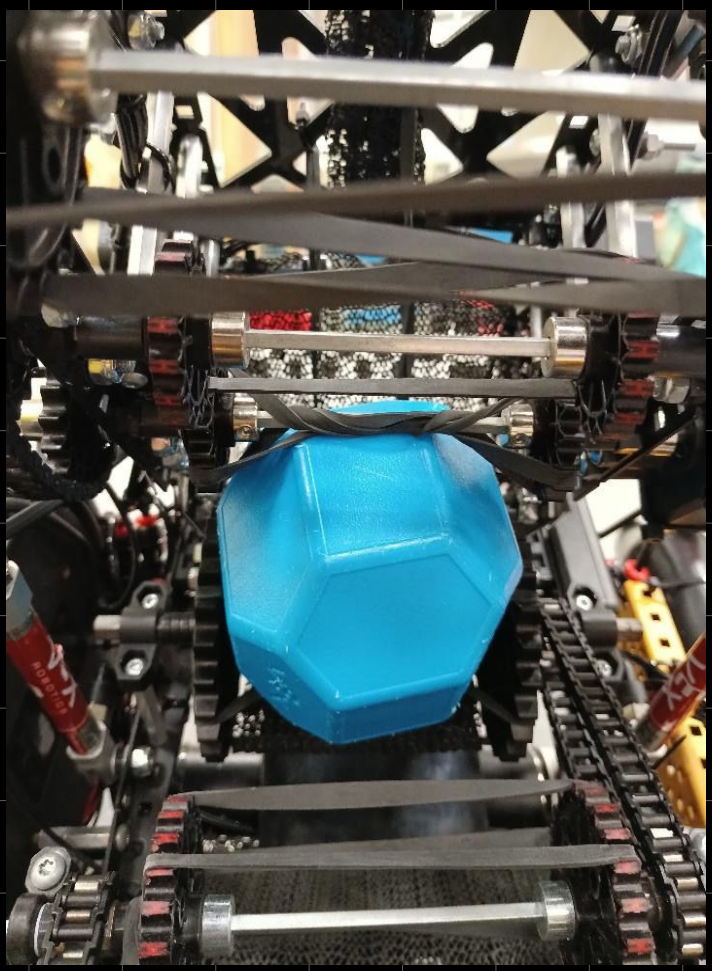
To fix the problems, we re-designed the plastic board shapes to meet the to requirements.



We printed 3 types to figure out which curve and limit height would suit best.

Intake Design

Another issue with the intake is that the last sprocket, which acts both as the ceiling of the intake and the floor of the shooter, tends to get stuck. In theory, our bot can hold 6 or 7 blocks, 8 if you really push it. But after loading around 4 blocks, everything jams. The blocks reach the shooter, the last row stops moving, and that ends up freezing all the sprockets below it since everything's chained together.

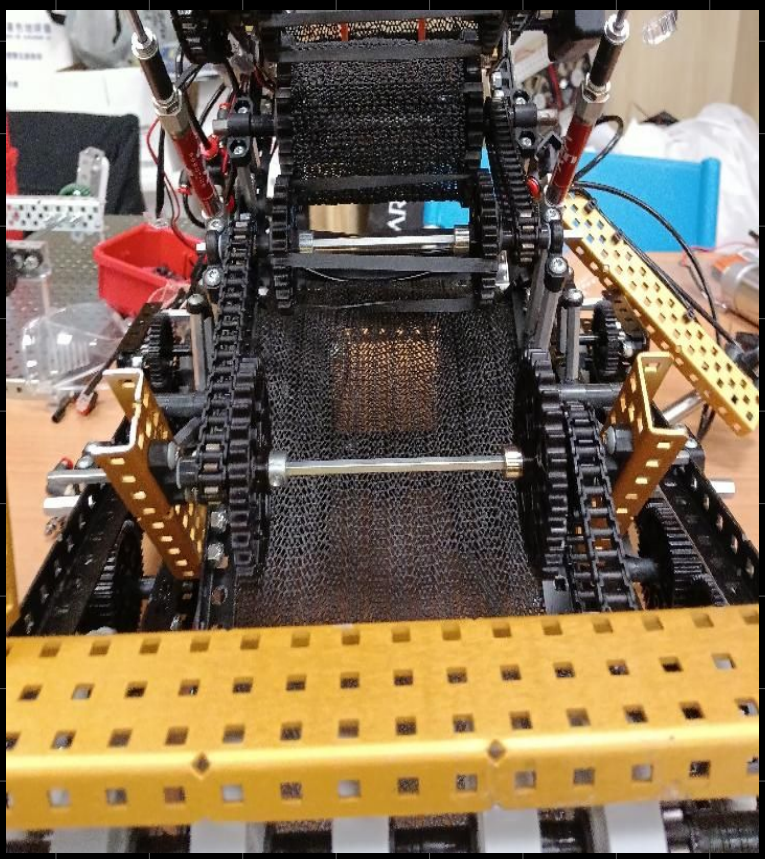


Intake Design

Now, as sprockets are chained together, stopping the last row from moving meant the whole intake has to be reimbursed. Effectively cutting the capacity in half, which is devastating. We can't even clear a loader now.

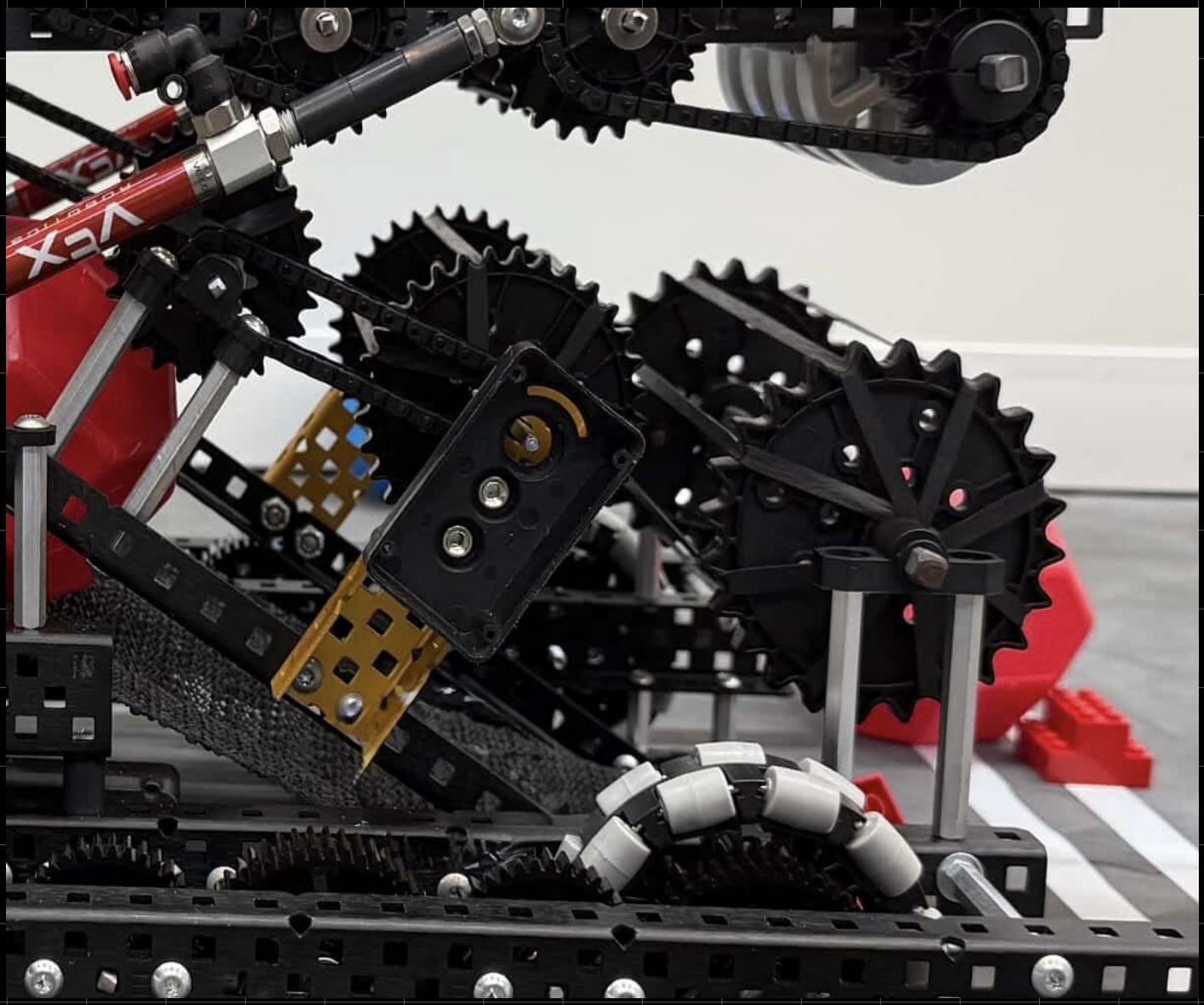
We've tried to fix this issue by:

Issuing a separate 5.5 watt motor for the last row. But, this will leave the shooter will just another 5.5 volt instead of 11. And honestly, that is too risky, because we don't think the shooter will have enough torque to actually fire anything reliably.



Intake Design

We also need to fix the issue where the intake was sitting way too deep inside the bot and couldn't reach blocks near the wall.



Intake Design

To make sure that the flex wheel intake would perform better than our current intake, we did an experiment.

TEST 1

Test the intake efficiency under standardized conditions.

Test Procedure

1. Place the intake prototype upon the loader
2. Along with our tongue prototype, load 6 blocks from the loader
3. Start intake and timer simultaneously
4. Stop timer when final block is fully secured

Results

Trial	Sprocket Intake	Flex Wheel Intake
1	5.82	3.94
2	5.67	3.81
3	5.91	3.76
4	5.74	3.88
5	5.85	3.72

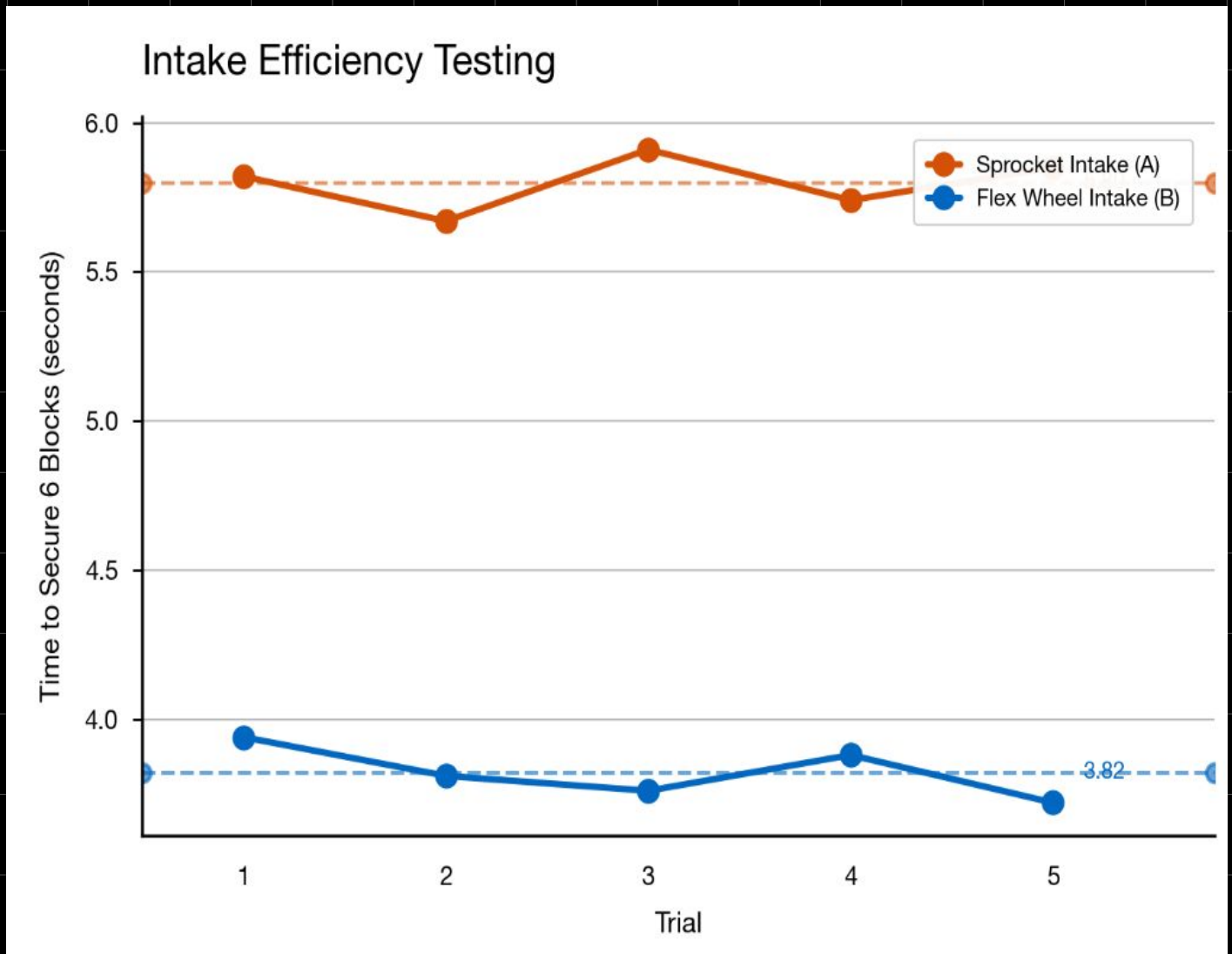
Intake Design

TEST 1

Avg Intake Time

Prototype	Avg Time (s)
Sprocket	5.80
Flex Wheel	3.82

Below plot shows TEST 1 results:



Intake Design

TEST 1

Analysis

The flex wheel intake demonstrated significantly higher efficiency by conforming to block surfaces and maintaining continuous contact. This reduced tangling and allowed faster, more consistent block acquisition.

Intake Design

To make sure that the flex wheel intake would perform consistently on our robot cooperating with sprockets.

TEST 2

Test the intake on the robot.

Test Procedure

1. Place the intake prototype upon the loader
2. Feed 6 blocks into the intake
3. Start intake and timer simultaneously
4. Stop timer when final block is fully secured

Results

Trial	Intake → Storage Time
1	4.21
2	4.58
3	4.15
4	4.42
5	4.30
6	4.48
7	4.25
8	4.12

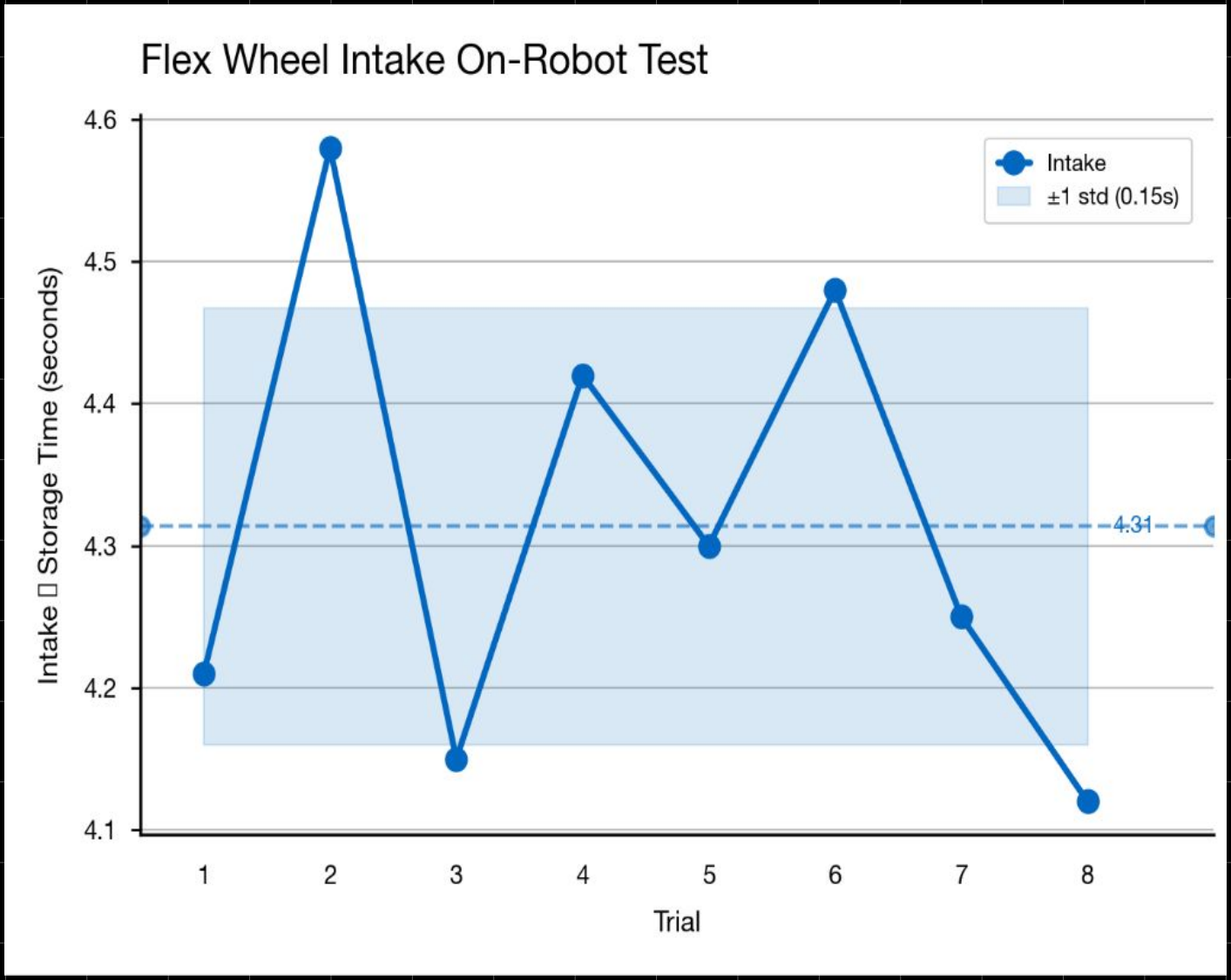
Intake Design

TEST 2

Avg Time Spent intaking

Avg Time (s)	Std Dev (Consistency)
4.34	±0.15 s

Below plot shows TEST 2 results:

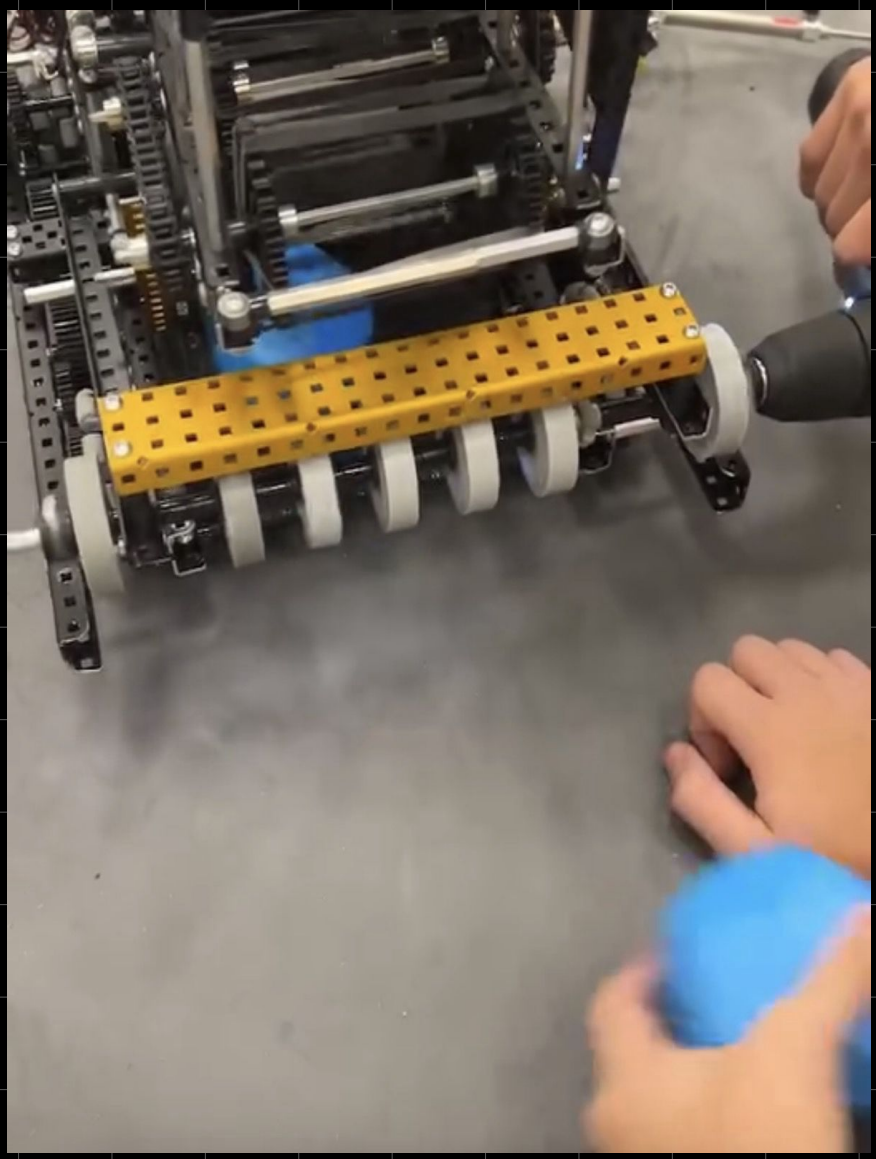


Intake Design

TEST 2

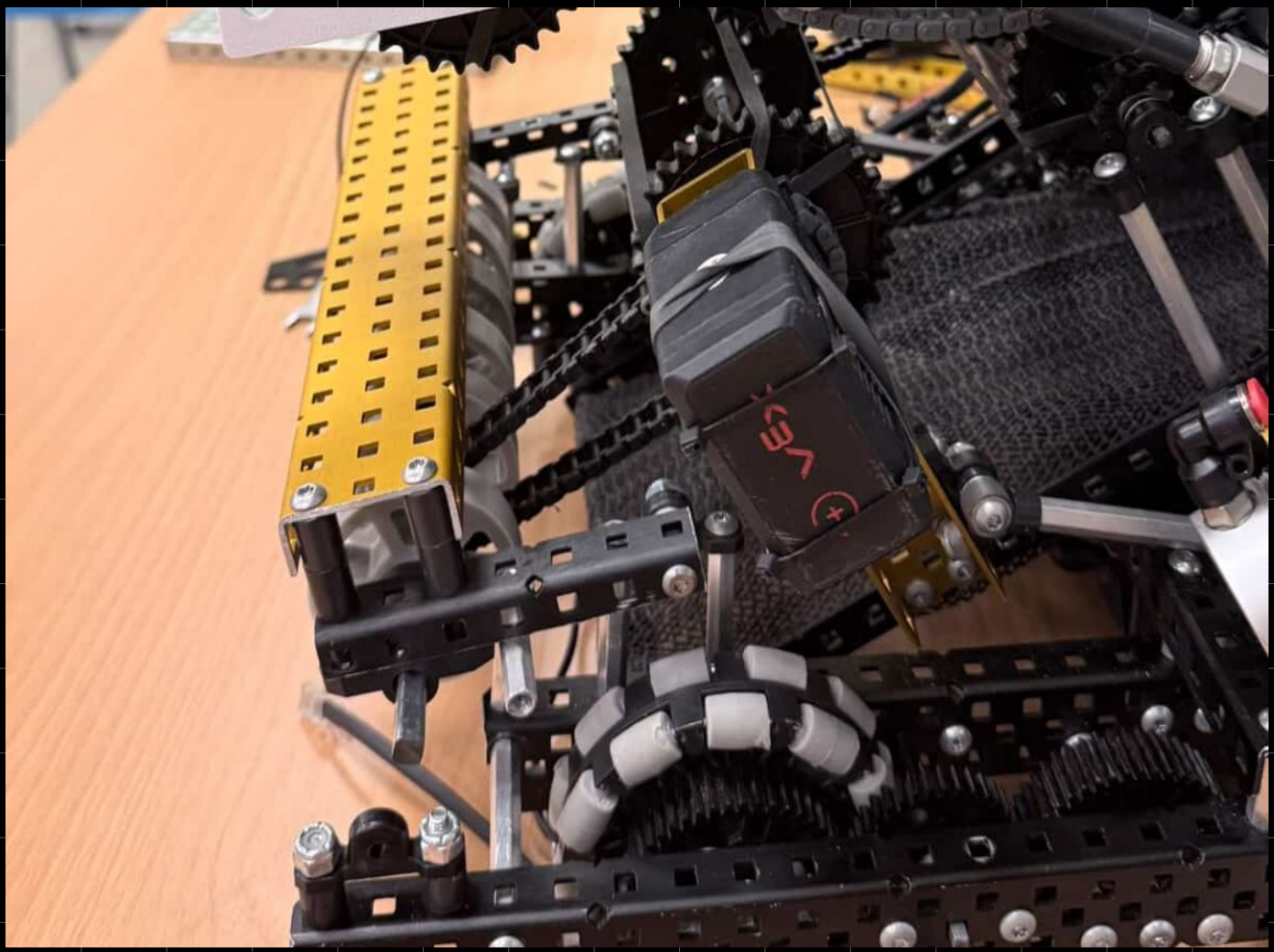
Analysis

The flex wheel intake performed smoothly and incorporated seamlessly with the current chain of sprockets. With a low standard deviation indicating consistent and uniform intaking.



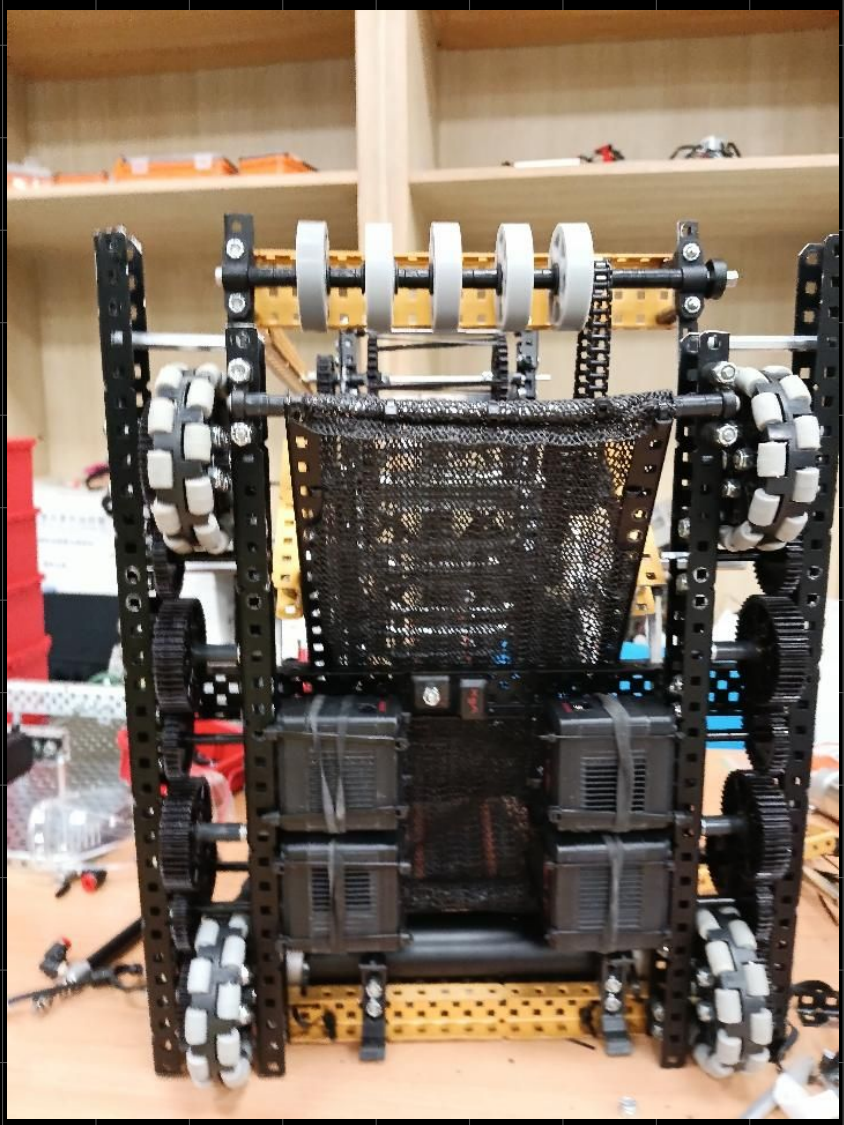
Intake Design

By adding standoffs and a couple extra angle pieces, we 1. swapped the first row from sprockets to flex wheels, and 2. pushed the whole first row of the intake outward.



Intake Design

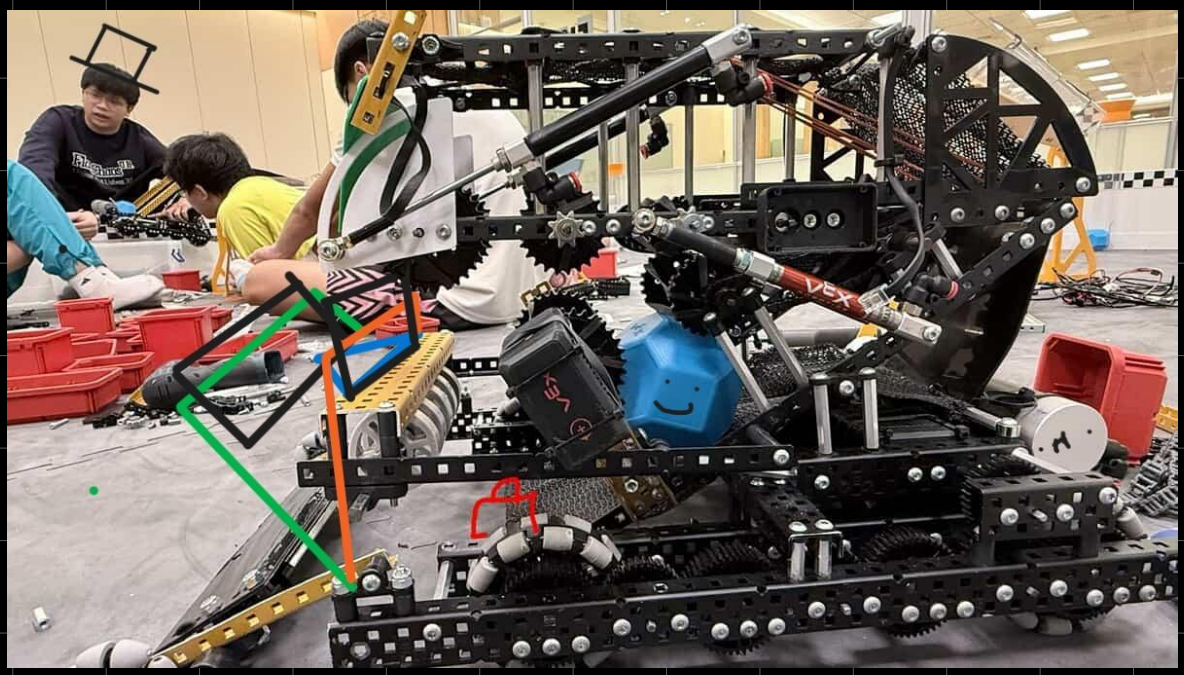
We loosened the extending parts to give the intake some give. But that led to another headache:
now when the robot intakes, the whole first row shoots upward and slams into the shooter level, messing everything up and throwing the alignment completely off.



Intake Design

Initially, the slope of the plastic was not steep enough to let blocks from the loaders slide in properly. It only works if the bot crashes into the loader at enough speed, which isn't exactly ideal. To make things worse, the tongue now interferes with our new intake in a couple of ways:

1. We plan to add an aligner(blue), and the bottom part of the plastic(green) will most likely get stuck there.
2. Even if it somehow doesn't block the aligner, after fully tucking in(orange), it stop the first row of intake from lifting up, which in turn effectively shuts it down.



Tongue

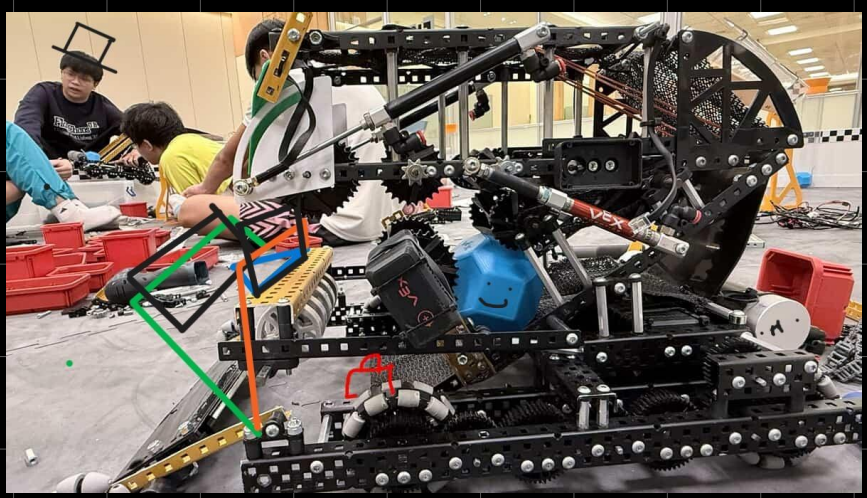
We rebuilt the tongue

1. using longer angle bars, and
2. added a limit so the plastic doesn't wedge itself into the gap between the intake and shooter anymore.

It now stops right before touching the plastic on the side of the shooter, which keeps it from blocking the intake. We also

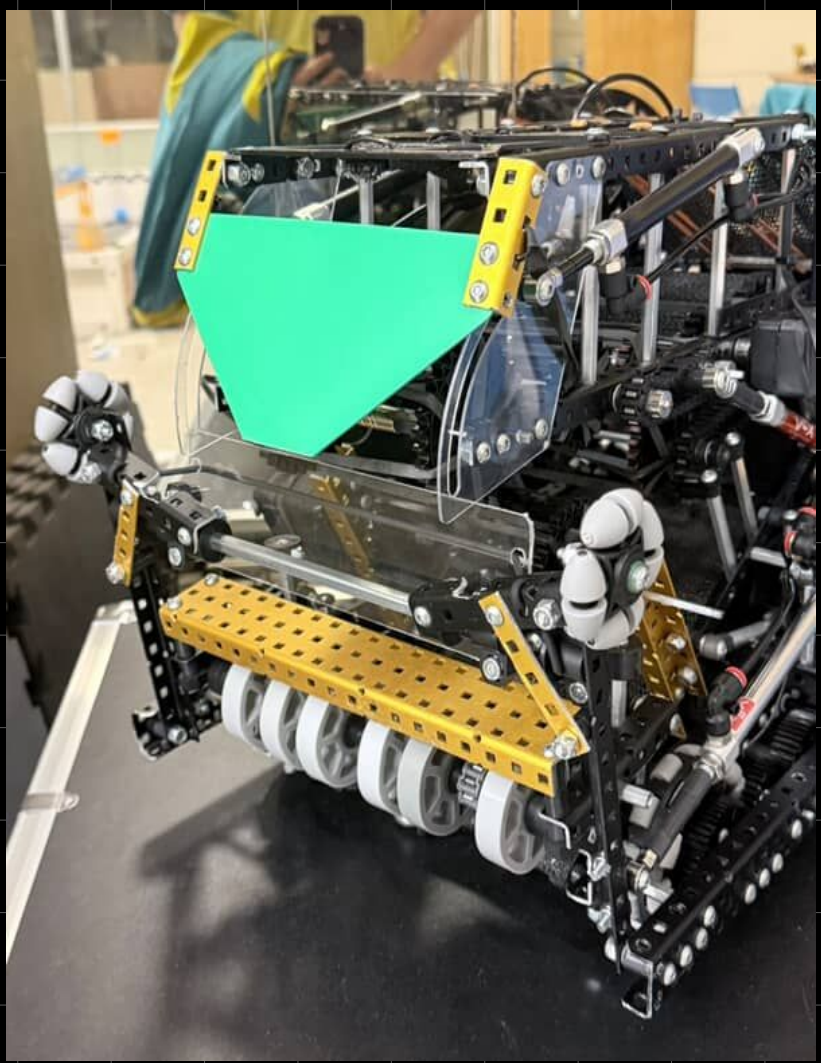
1. shortened the side angle bars, and
2. shifted them forward.

The whole thing became way more rigid, reacted faster, and stopped wobbling around like before. It also put less stress on the joints, so the tongue isn't trying to pry itself apart every time we intake.



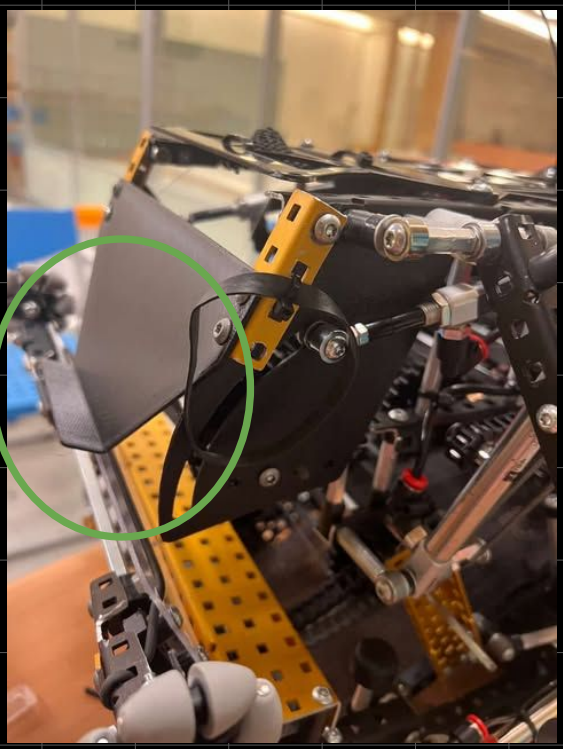
Descorer

For our new descorer, we switched things up and used two angle bars instead of the old setup where we just bent the plastic. The reach is definitely better now, but honestly it still isn't hitting the height we actually need. The limit needs to sit higher so it can push the balls cleanly when the mechanism is at rest. The problem can be solved by extending the plastics on the sides upwards.



Descorer

Since VRC regulations don't allow any 3D-printed parts, we eventually had to swap our original design for the plastic version. The main change we made was bending the front end of the descorer so it could sit closer to the blocks and make more contact during a push. This tweak was meant to help the descorer "catch" the ball more reliably instead of just tapping it with a thin edge. In theory, the wider contact area should make it easier to guide the ball out of the goal without the piece slipping off or glancing past it.



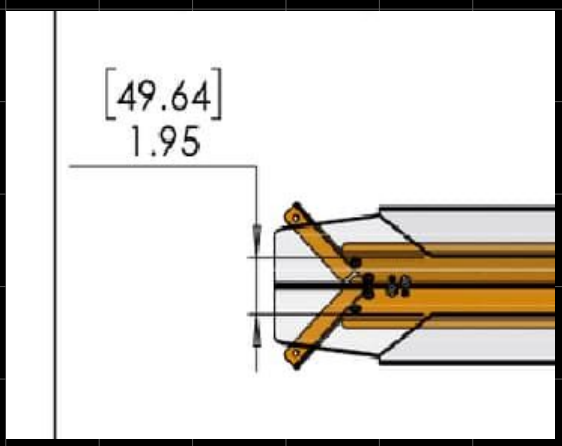
Aligner

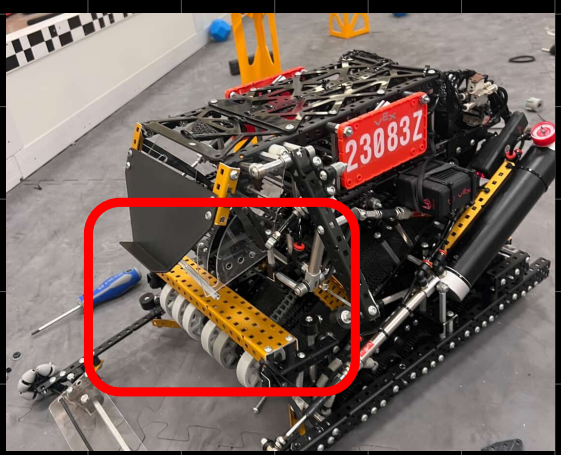
Following the previous TAI-VEX event, we are developing a front-mounted aligner to solve the critical problem of inaccurate long-goal alignment. This will dramatically improve our scoring consistency.

The proposed aligner features:

- Function: A passive guide for precise and repeatable robot positioning.
- Mounting: Front-mounted on the robot, above the intake system.
- Form: Still needs testing to effectively funnel the robot into the correct scoring position.

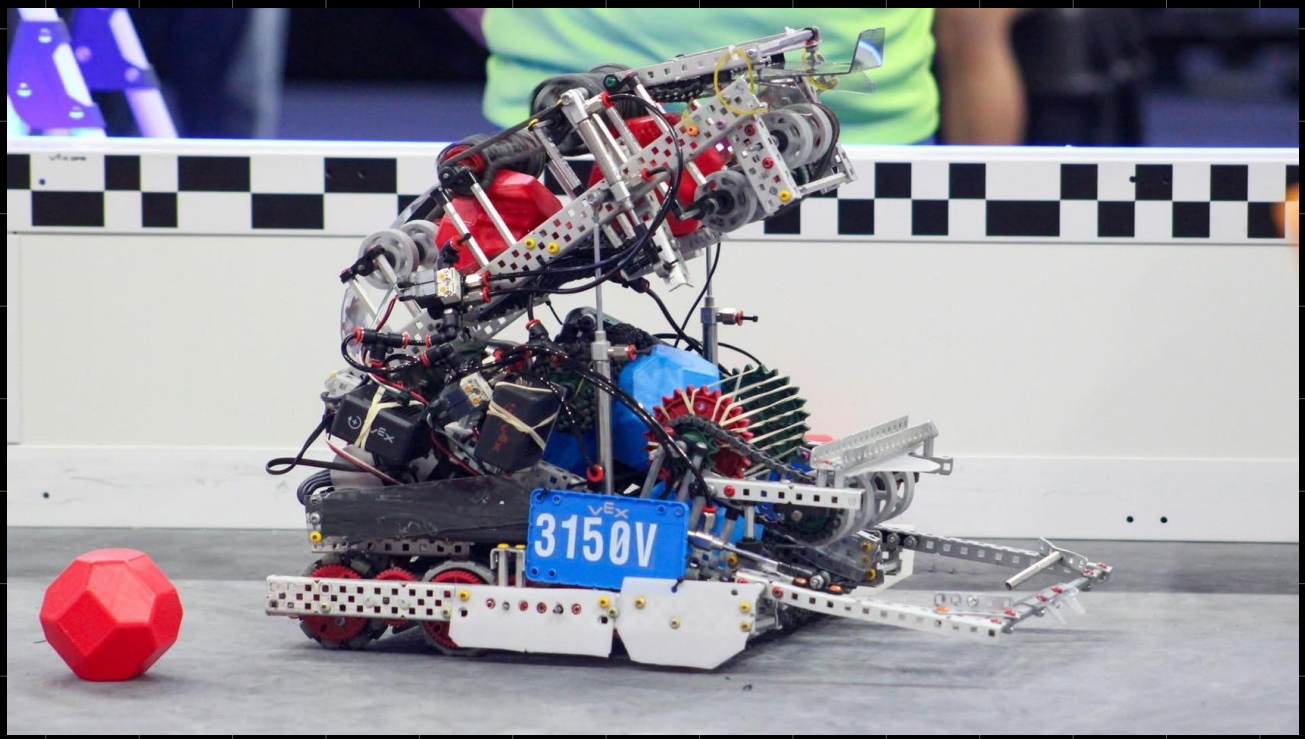
By implementing this solution, we aim to eliminate alignment errors and increase our scoring efficiency.





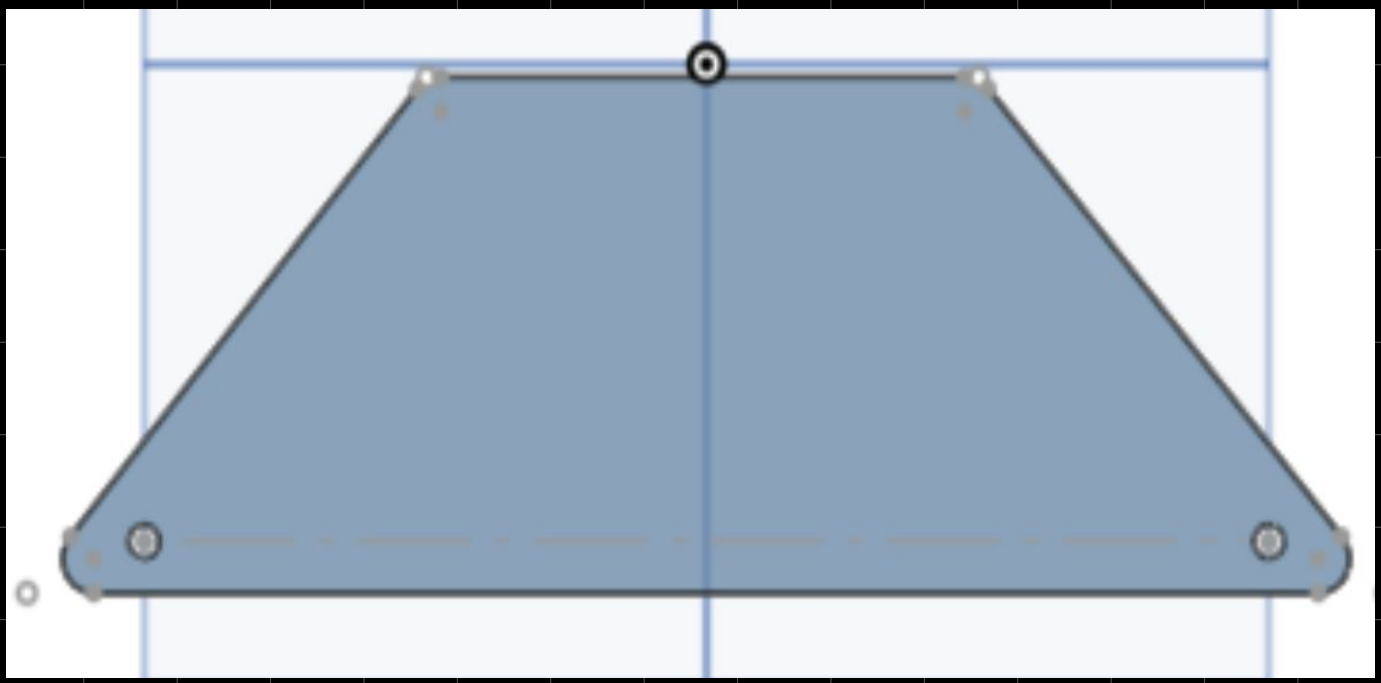
Aligner

We are inspired by 3150V's aligner placement and form, as their robot looks highly identical to ours. Therefore, we are starting to design one with a trapezoid shape like theirs.



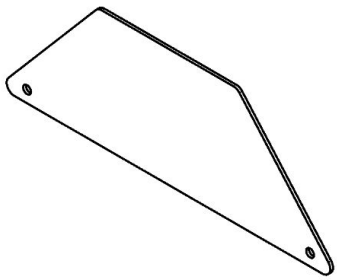
Aligner

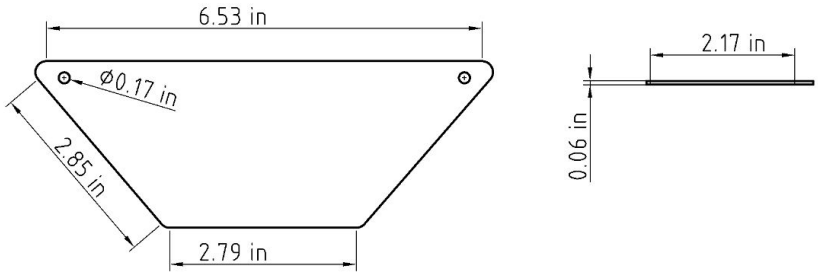
The initial trapezoid-shaped aligner was designed with inspiration from 3150V, but we also thought that we could test different shapes.



Aligner

Aligner_1 Isometric





		NAME	DATE
UNLESS OTHERWISE SPECIFIED: DIMENSIONS ARE IN MM			
ANGULAR ∠ = ±0.1° GENERAL TOLERANCE = ±0.05	DRAWN	AG	11–Nov–25
SURFACE FINISH √	CHECKED		
	APPROVED		
DO NOT SCALE DRAWING			
BREAK ALL SHARP EDGES AND REMOVE BURRS	DATE DWG LAST CHANGED		12–Nov–25
THIRD ANGLE PROJECTION	MATERIAL	FINISH	Alochrome 1200S

Anthony

TITLE: Aligner_1

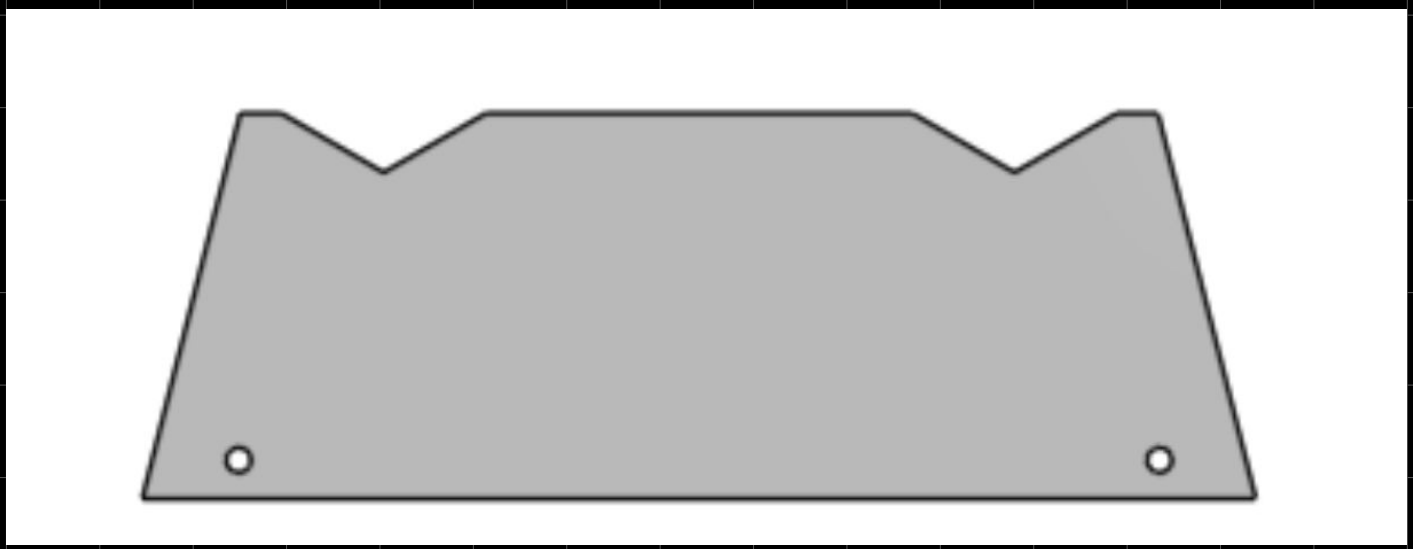
SIZE A4 | DWG NO. | REV.

SCALE 1:1 UOS | WEIGHT | SHEET 1 of 1

Aligner

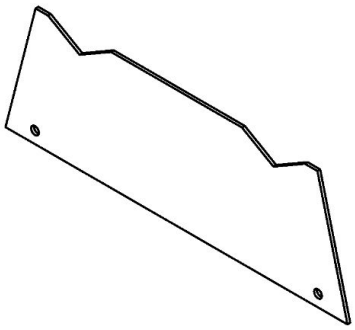
For the second aligner prototype, we implemented a design with matching dents to actively engage the goal's triangular base.

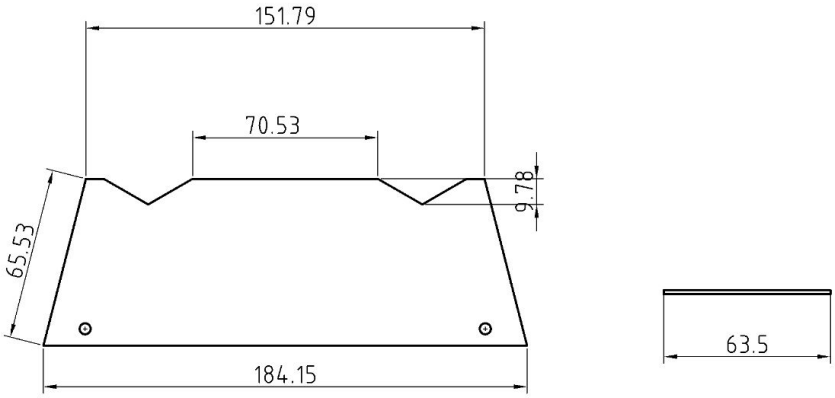
- Intended Function: To slide perfectly into the goal's frame for a secure and accurate lock.
- Active Alignment: A new hole and structural guide that positively "locks" into the goal's bottom triangle for precise alignment.
- Solved Interference: The tongue mechanism was modified (by moving and shrinking the front plastics) to create necessary clearance.



Aligner

Aligner_2 Isometric





UNLESS OTHERWISE SPECIFIED: DIMENSIONS ARE IN MM ANGULAR ∠ = ±0.1° GENERAL TOLERANCE = ±0.05 SURFACE FINISH √		NAME	DATE
	DRAWN	AG	29–Jan–26
	CHECKED		
	APPROVED		
DO NOT SCALE DRAWING			
BREAK ALL SHARP EDGES AND REMOVE BURRS	DATE DWG LAST CHANGED		1–Feb–26
THIRD ANGLE PROJECTION	MATERIAL	FINISH	Alochrome 1200S

Anthony

TITLE: Aligner_2

SIZE A4 | DWG NO. | REV.

SCALE 1:1 UOS | WEIGHT | SHEET 1 of 1

Aligner

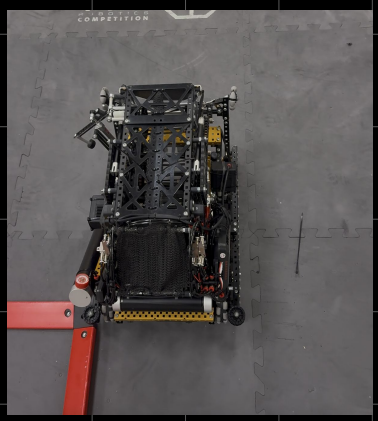
After cutting out both prototypes, we did a small experiment:

TEST 1

To determine which aligner design is more efficient in improving the robot's scoring position accuracy.

Test Procedure

1. Start the robot from the parking zone
2. Start Timer
3. Driver control to long goal
4. Stop timer when robot positioned



Trial	Prototype A	Prototype B
1	4.82	3.91
2	4.95	3.78
3	4.71	3.85
4	4.88	3.73
5	4.90	3.80
6	5.03	3.92
7	4.76	3.74
8	4.92	3.87

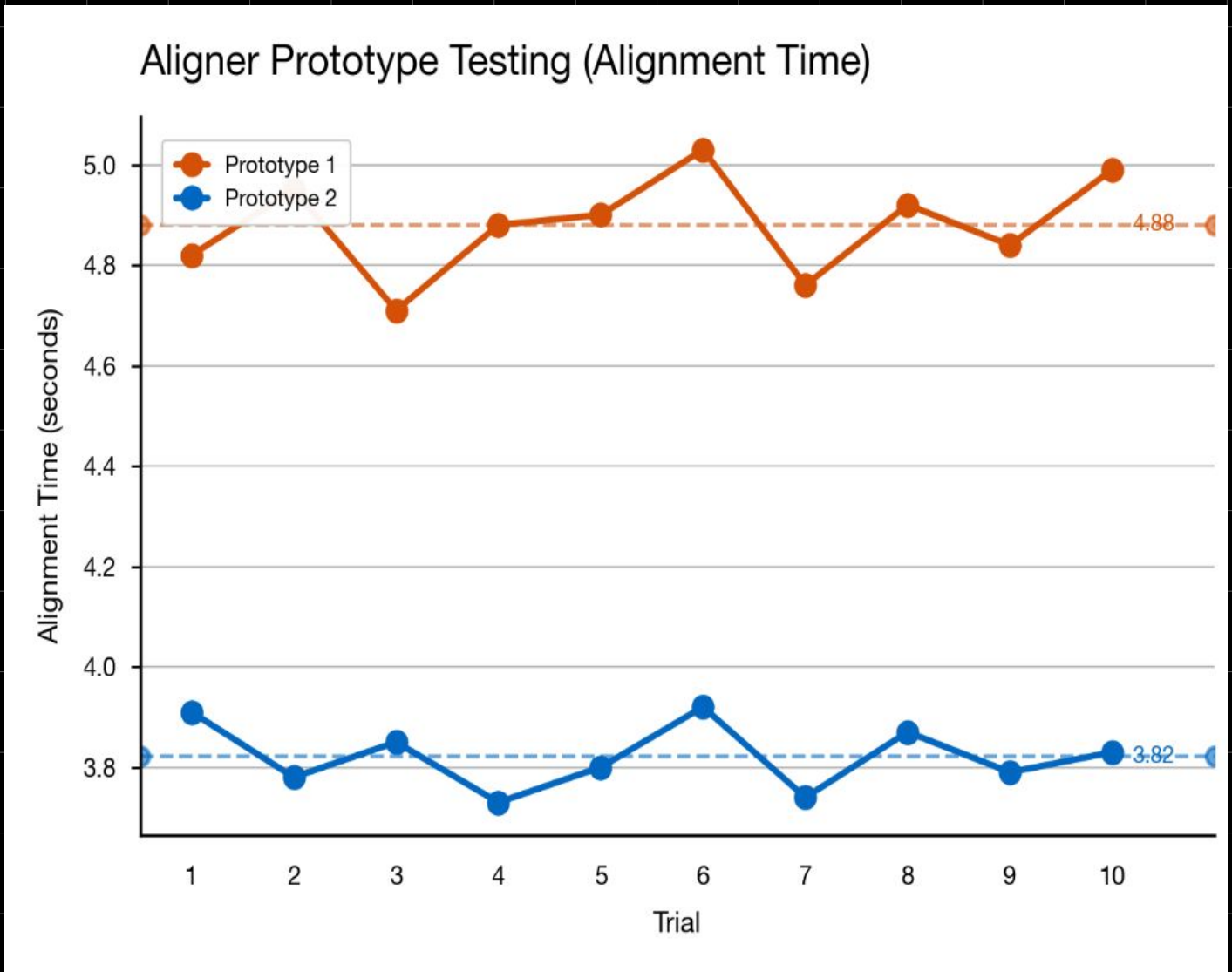
Aligner

TEST 1

Average aligning time:

Prototype	Avg Time (s)
A	4.88
B	3.82

Below plot shows TEST 1 results:



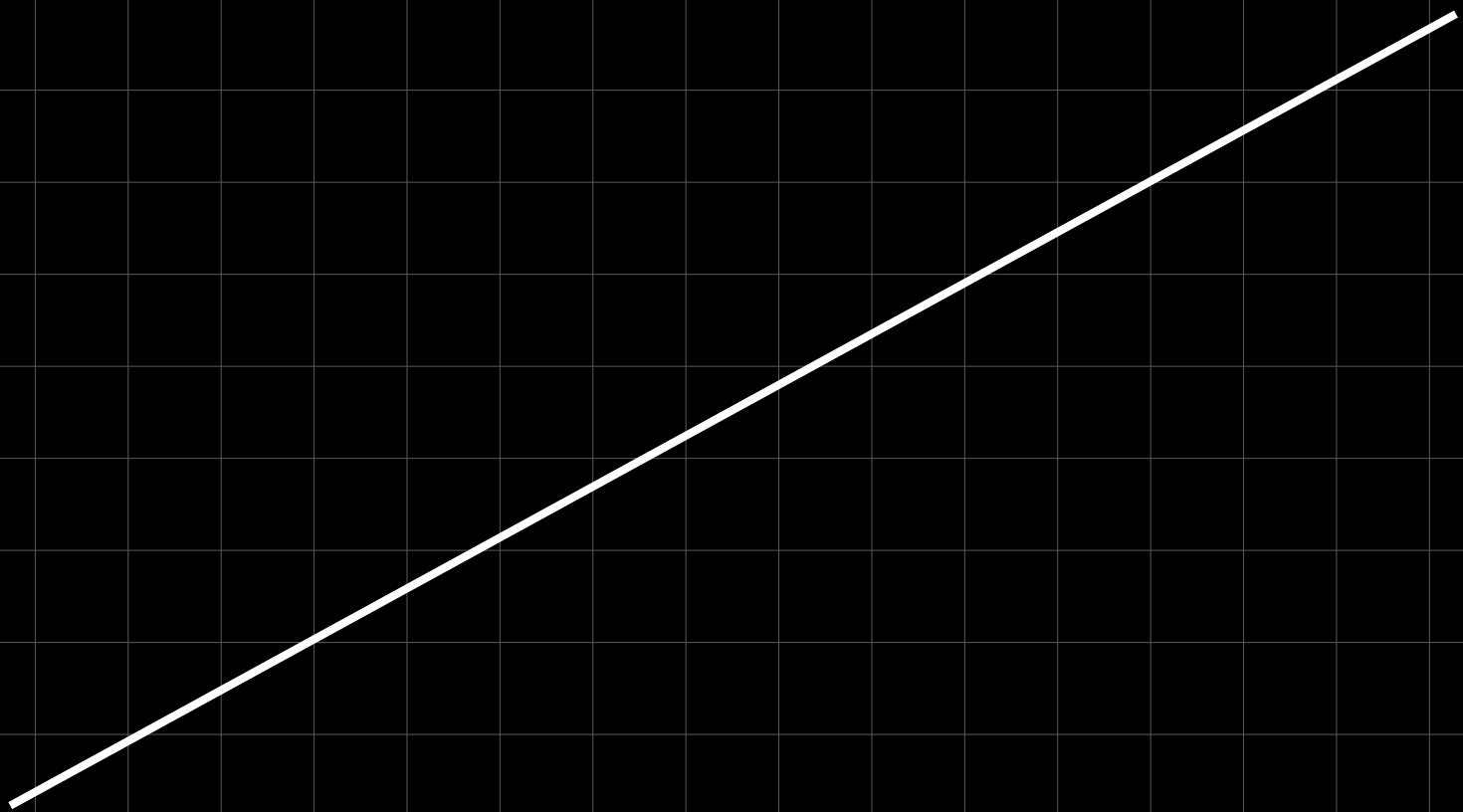
Aligner

TEST 1

Aligner Prototype 1 had several fatal problems:

1. The design had zero tolerance for error. It required a perfectly straight approach, which was impractical during match conditions.
2. Even minor misalignments caused the mechanism to fail, drastically reducing its effectiveness.

Due to this fundamental issue, we have decided to choose Prototype 2.



Aligner

TEST 1

Analysis

Prototype 2 showed significantly faster aligning, about 21% faster. Proving that valleys could help the robot slip into the right position.

Decision

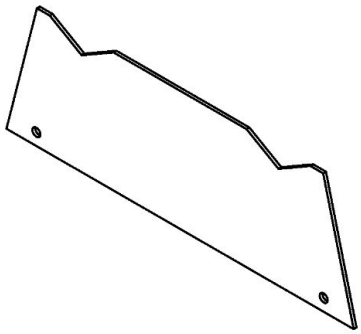
However, we could still improve in the prototype 2's shape and dimension to make it more efficient.

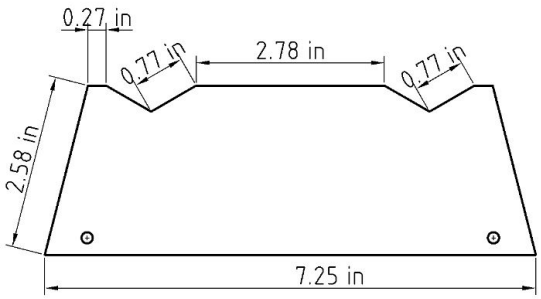
Aligner

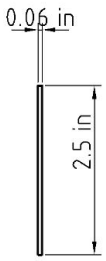
For **the third and final version** of the aligner, we evolved the static dents into an angled, funnel-like guide. This new design features sloped surfaces that automatically correct the robot's position as it approaches the goal. By channeling the robot into the correct alignment, the guide ensures it "locks" precisely into the triangular structure.

Aligner

Aligner_3 Isometric







UNLESS OTHERWISE SPECIFIED: DIMENSIONS ARE IN MM ANGULAR ∠ = ±0.1° GENERAL TOLERANCE = ±0.05 SURFACE FINISH √		NAME	DATE
	DRAWN	AG	15–Nov–25
	CHECKED		
	APPROVED		
DO NOT SCALE DRAWING			
BREAK ALL SHARP EDGES AND REMOVE BURRS	DATE DWG LAST CHANGED		15–Nov–25
THIRD ANGLE PROJECTION	MATERIAL	FINISH Alochrome 1200S	

Anthony

TITLE: Aligner_3

SIZE A4 | DWG NO. | REV.

SCALE 1:1 UOS | WEIGHT | SHEET 1 of 1

Meeting Team Organization

2025/11/13 — 60 minutes

Who is here	Goals for today's meeting
- Pennlaine - Adam - Anthony - Jefferson	- Team organization - Practice Schedule

Meeting information:

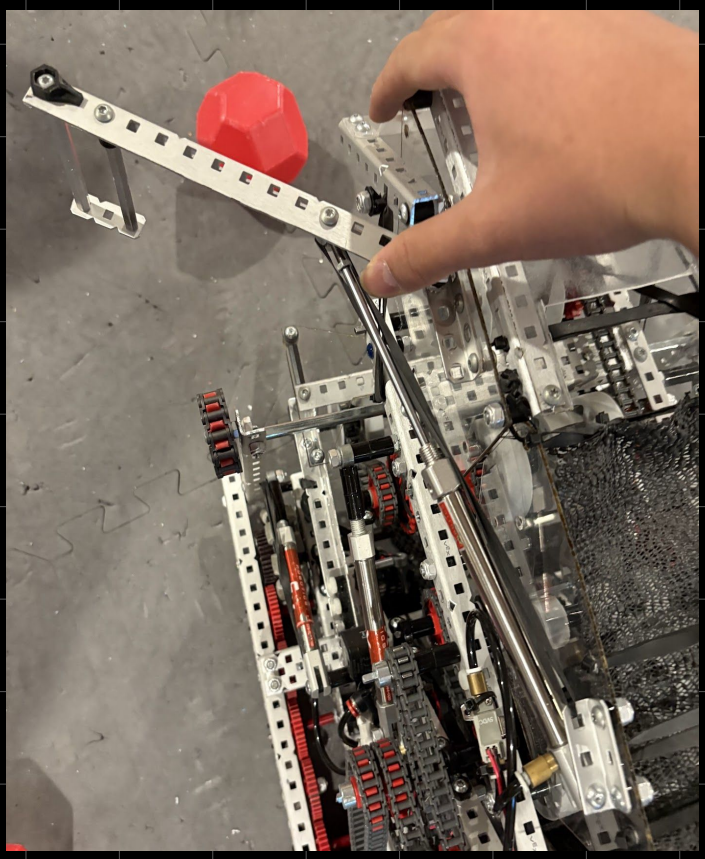
Our first time management isn't well organized. We haven't finish improving our robot. We gave a better time schedule and emphasized each team members jobs.

Due to a little conflict in our team. We think our team is lacking beneficial conversation. We stayed together to discuss about our conflict and solutions.

Our solution was that we need to list out everything that happened by the end of the day, and the team leader would then reply what each member would do the next day based on who will go tomorrow. And then again each would respond what they solved and what new problems happened.

Wing

Our initial descoring strategy was passive; a front-mounted mechanism would displace balls only when the robot forcefully collided with the long goal. However, we recognized this was inconsistent and limited. Inspired by the strategic functionality of team 8889A's wing mechanism, we decided to develop an active descoring system.



WING Demonstration
from team 8889A

Wing

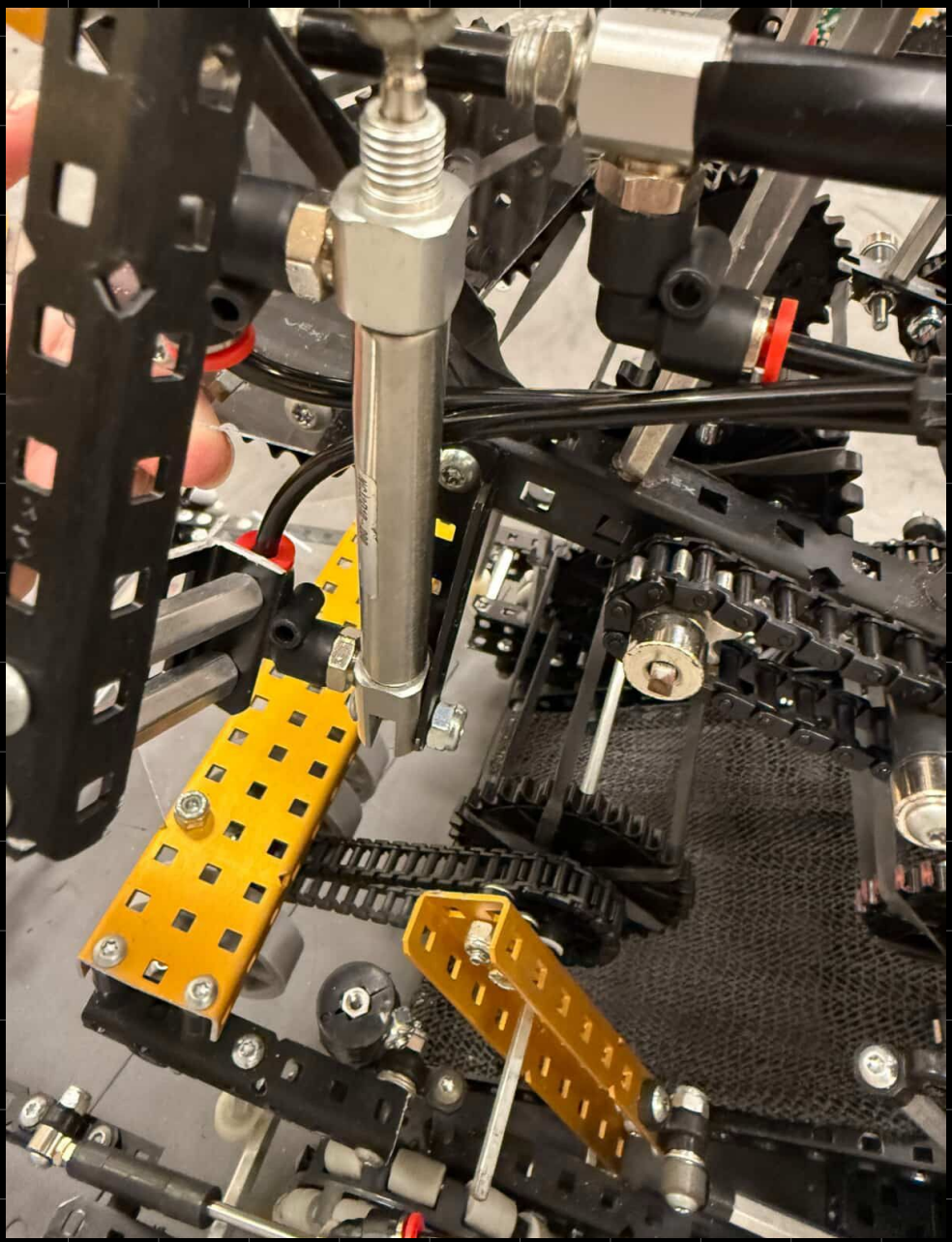
Our new design will feature a dedicated wing that can be driven into the goal's channel on demand, allowing us to:

1. swipe blocks out at a time of our choosing, and
2. place the blocks at the control bonus section,

which both add a significant strategic function to our robot.

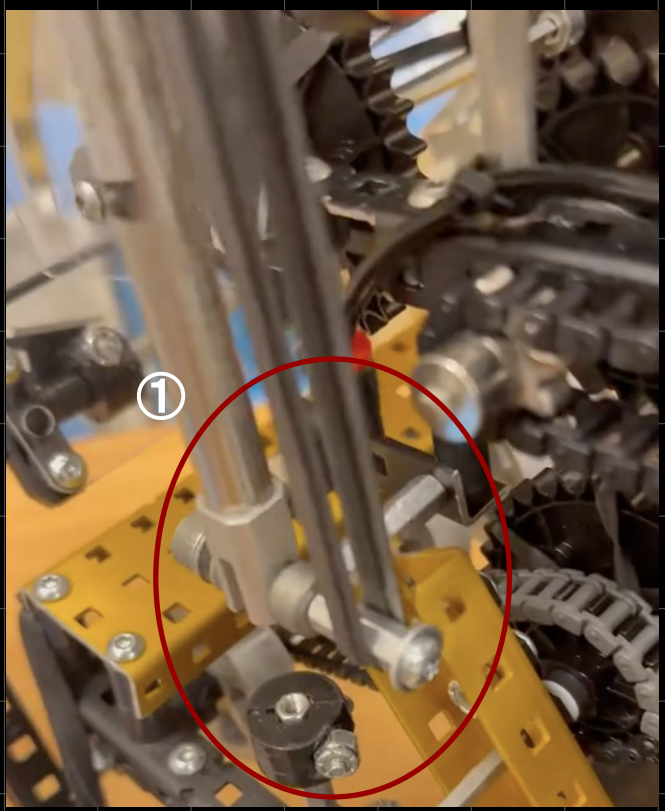
Wing

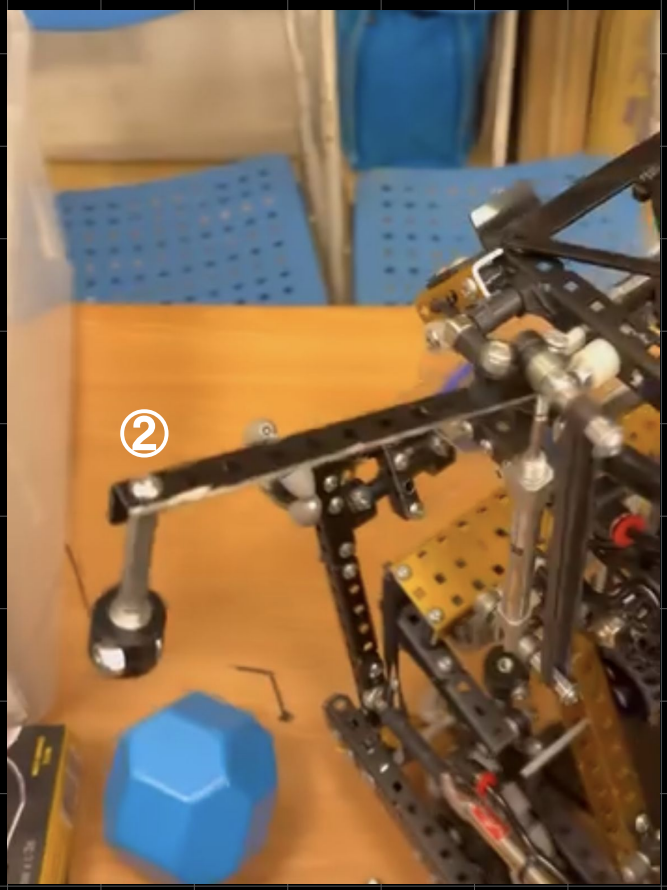
Our initial wing design was a simple assembly: an angled plate attached to a piston, supported by a standoff. However, testing revealed significant flaws.



Wing

1. The structure lacked the necessary stiffness, and
2. the piston's fit created excessive friction against the goal channel, hindering smooth operation.

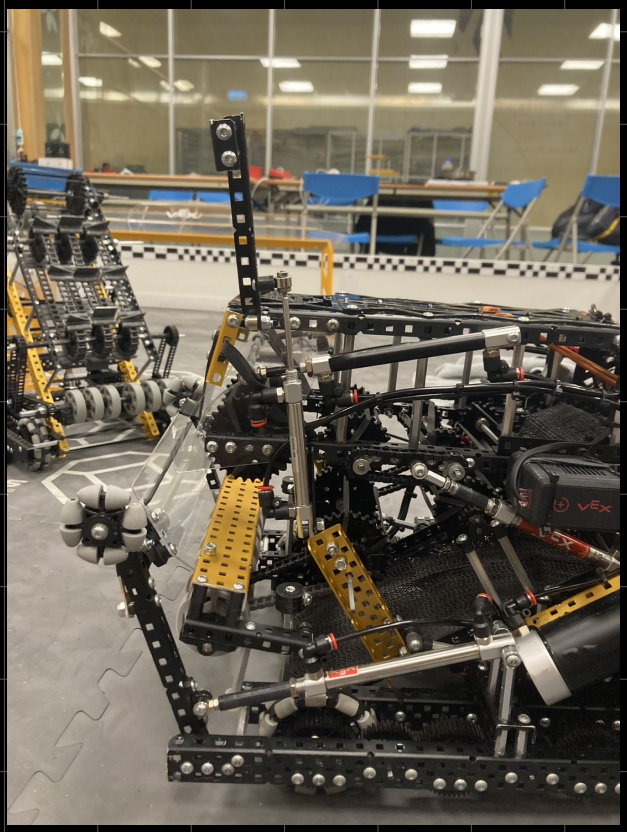




Wing

To resolve these issues, we implemented a comprehensive redesign:

- Revised Mounting: We changed the entire wing-mounting format to improve structural rigidity.
- New Actuation: We switched to a single-acting piston.



- Positive Return: We added a rubber band to provide a reliable return force when the piston is released, ensuring the wing retracts completely.

Wing

Following an iterative process of prototyping and testing, we successfully engineered a final wing structure that perfectly balances **strength, function, and form factor**.

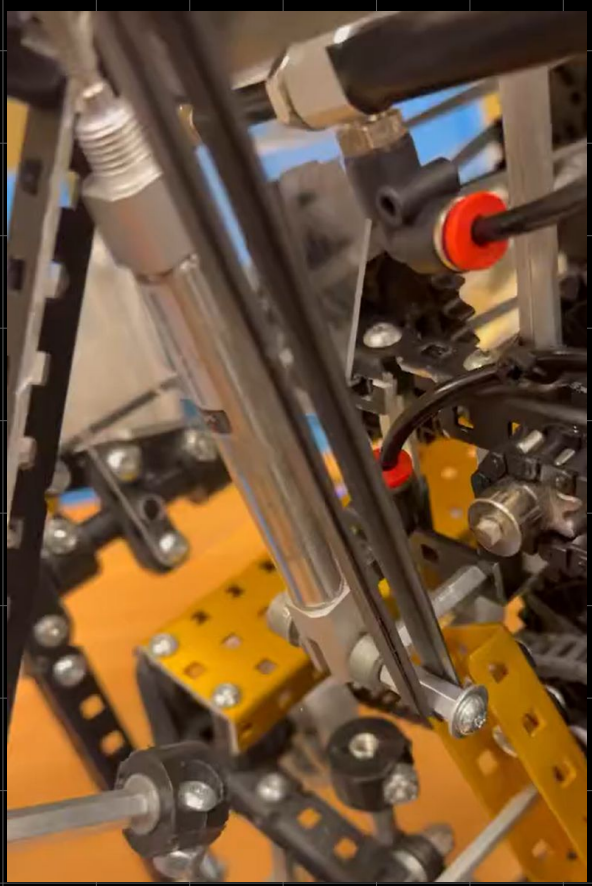
The breakthrough came from building the wing's core framework not from simple plates, but from a composite structure using standoffs (copper pillars) and C-channels. This approach provided the ideal combination of properties:

1. the C-channel's cross-section provides inherent torsional strength, resisting the twisting forces experienced during descoring. When strategically connected and triangulated with solid standoffs, it creates a lightweight yet incredibly rigid frame that does not flex during use.

Wing

2. By carefully sizing the outer dimensions of this C-channel and standoff frame, we created a structure that fits precisely within the goal channel. This "snug but smooth" fit ensures it can slide in reliably without the excessive friction that bound our previous piston-based design.

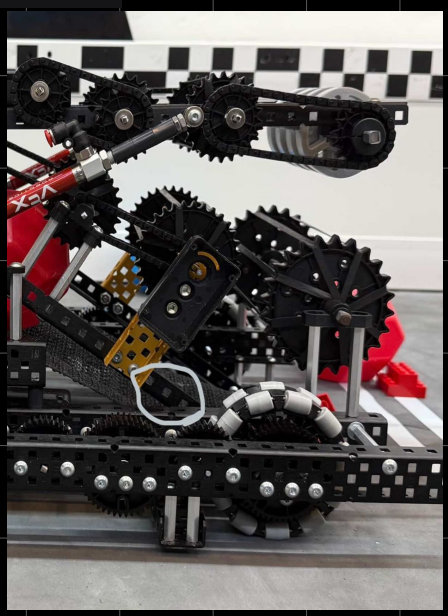
3. The structure naturally incorporates strong, multi-point attachment locations for the single-acting pneumatic cylinder, eliminating the flex inherent in a simple angled-plate mount.

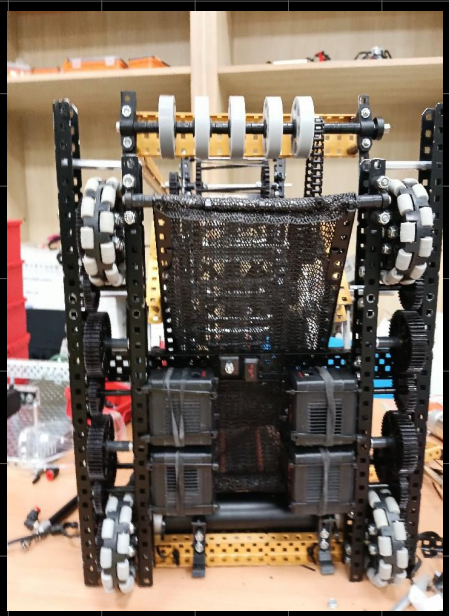


Parking

To solve the problem of unable to either double park or cross the parking zone, we thought of some possible solutions:

1. add side plastic boards to aid
2. change the lateral stiffness structure that connects the chassis underneath
    a. the structure was built to enable double parking, but it is just at the point for half of the parking zone so it actually knocks our opponent off from parking
    b. we decided to change its position to on top of the chassis, but because of the ramp, there is nearly no space.

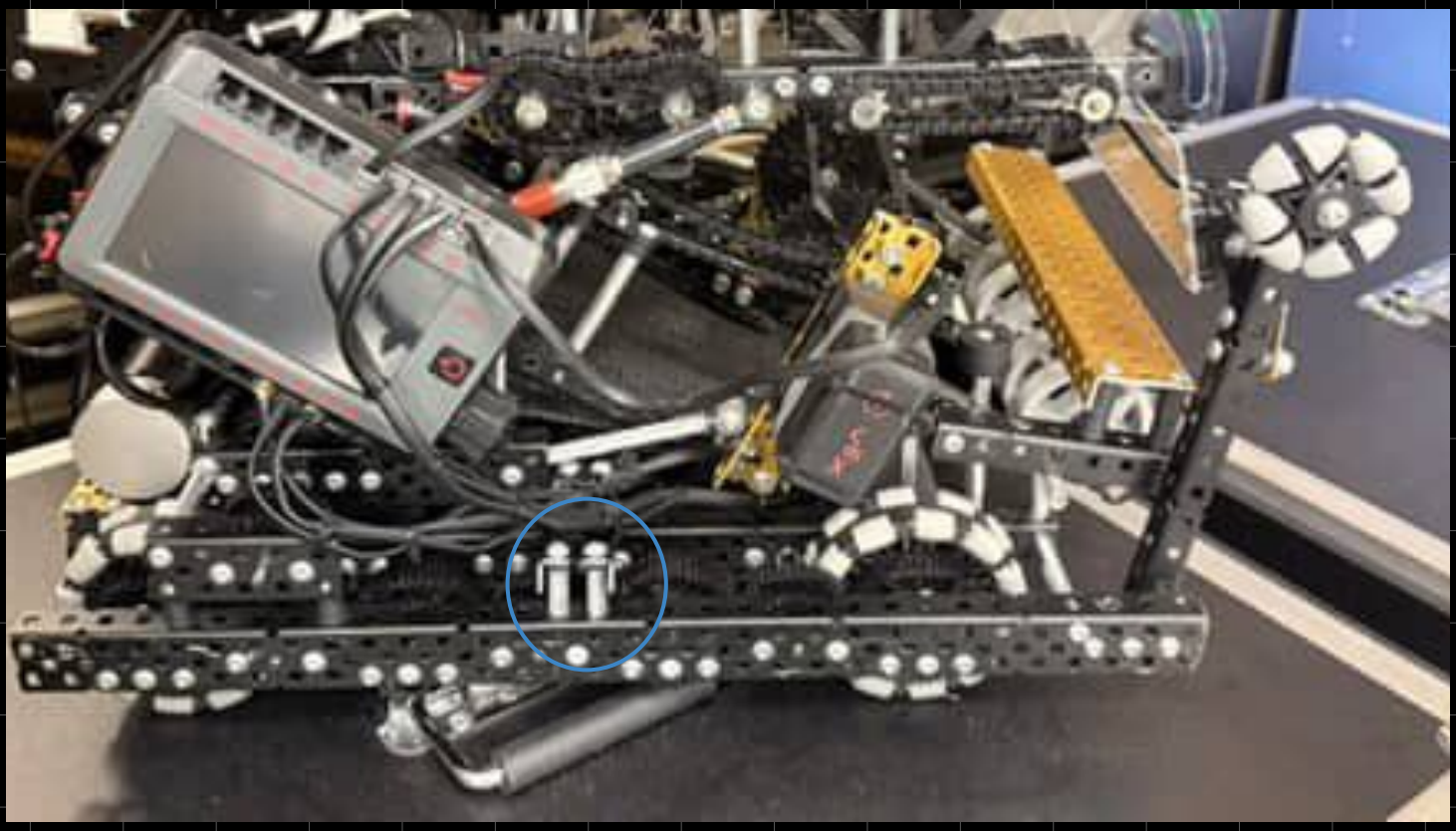




Robot 2

We did eventually find a place to set our lateral stiffness structure, but we had to move our inertia position. Also, this created a risk that the shaft between a sprocket wheel and the structure are too close together.

Since we moved our intake forward out, we had to also reposition the sprockets to connect them smoothly, so we also had to reposition the gold channel for motors.



Building Timeline

Tasks	12/8-12/14	12/15-12/21
Descorer (wing)		
Programming		

Tasks per member:

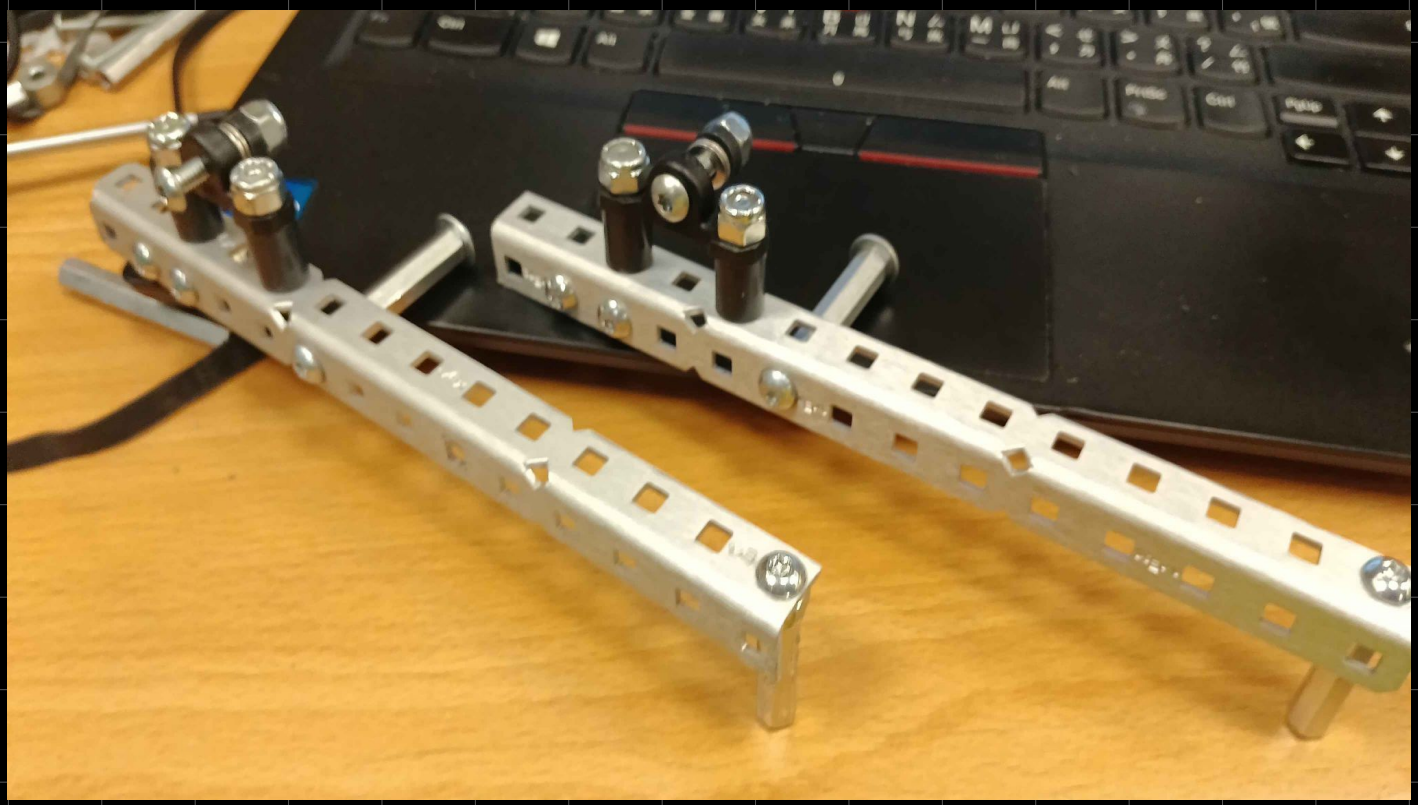
- Wing: Benjamin, Adam
- Programming: Pennlaine
  - Simulator: Anthony
- Driver Practice: Adam

Wing

Although we previously said that the wing is the final product, the wing actually broke off several times during friendly matches due to repeatedly crashing into long goals.

Wing

Therefore, in case it breaks again during competitions, we made some backups that are identical and able to easily and quickly rebuild and fix.



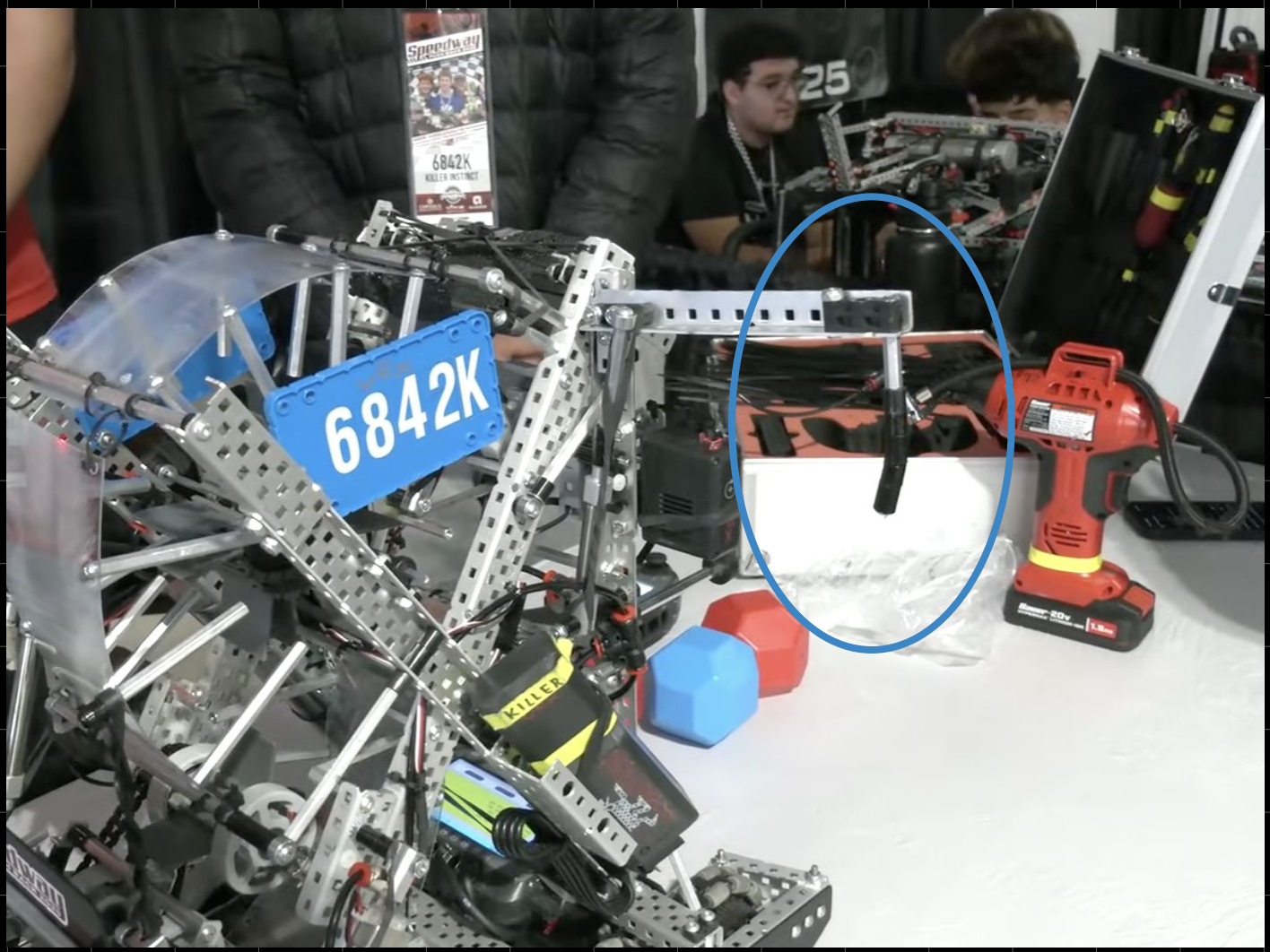
Wing

In the update of V2.1, **<SG10>** is updated to that reaching and anchoring into the goal is prohibited, therefore we need to change our previous design of using a rubber bumper.

PROs	CONs
The rubber bumper had a **larger volume**, giving us **more momentum** while pushing the blocks	It might get us **stuck within the enclosed section** of the long goal if we retract our pistons there, which causes us to **illegally anchor inside the goal**.

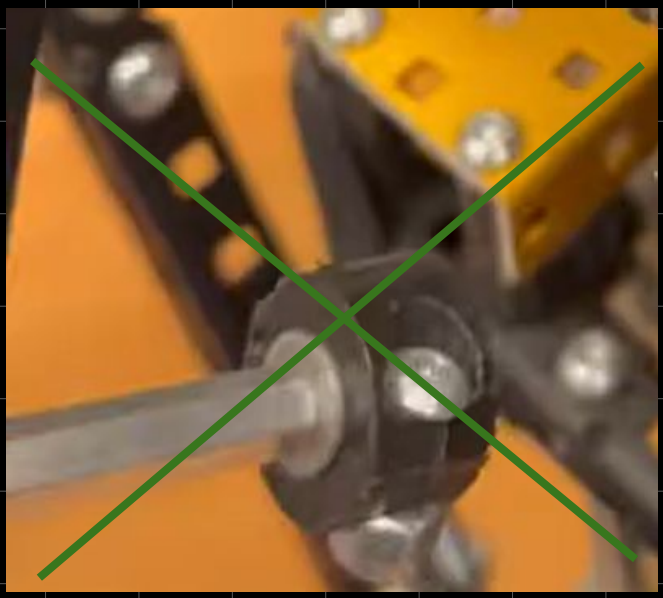
Wing

As opposed to before, instead of using the bumper to push the blocks forward, we opted for a scythe-like tip instead, inspired by Killer Instinct 6842K.



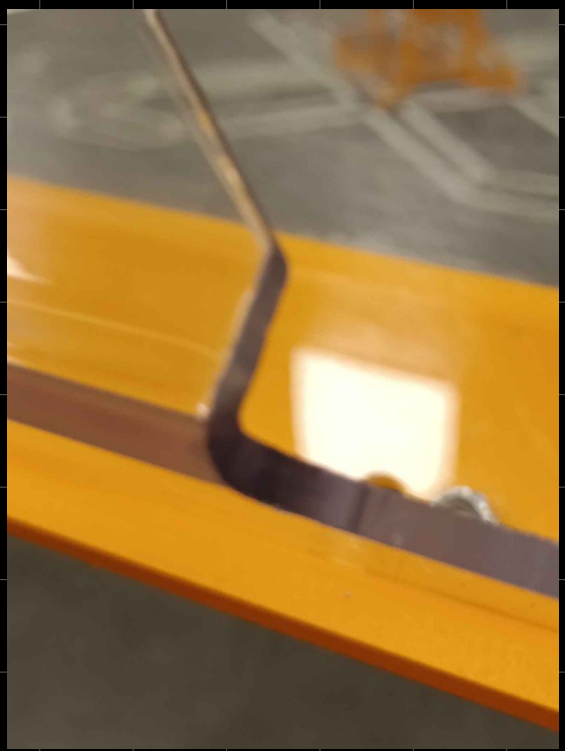
Wing

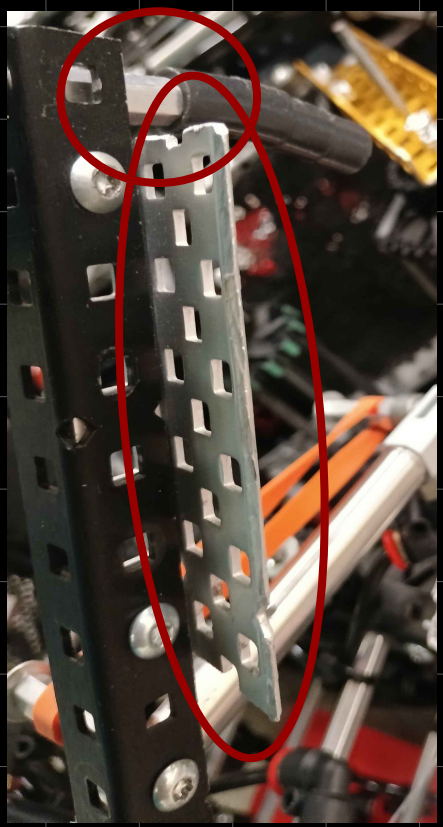
We curved a screw into the state you see here. This change makes it much more easier to dip it in the goal, as its thinner and it follows the trajectory of where it falls.



Wing

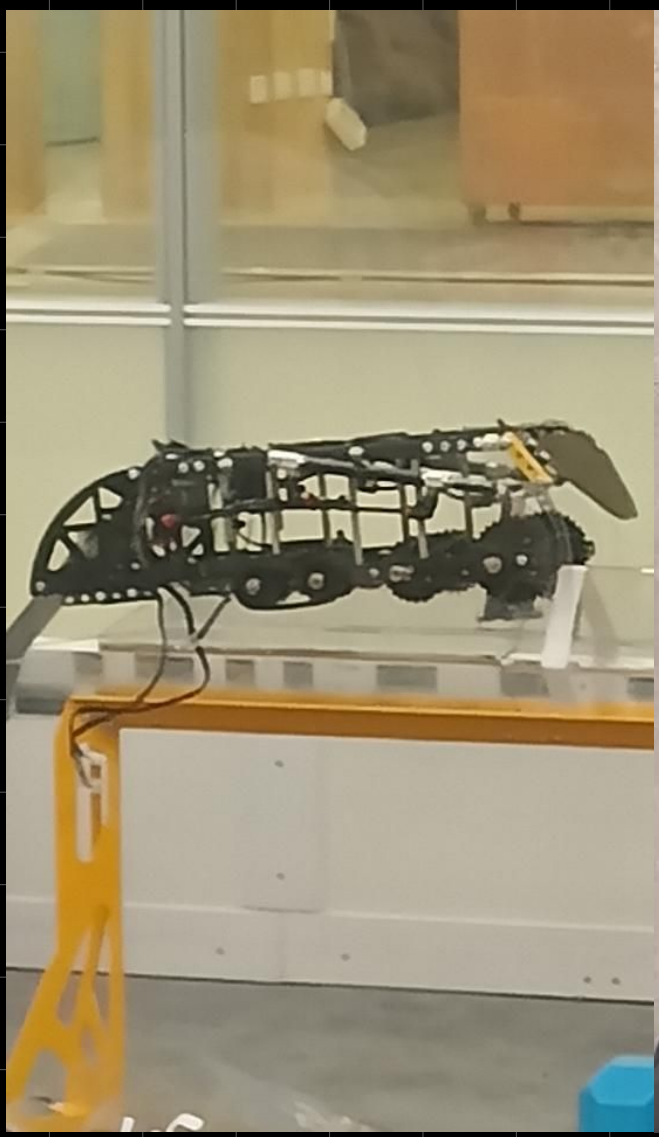
Another problem we encountered was that the suction part of the edge of the long goal effectively shuts down the possibility of our wing advancing into the inner parts of the goal. So we first used a plastic board to stand in the way, bouncing the wing up from the energy of the crash, but upon thinking we realized we have used up all the available plastic parts, we switched to a angle bar instead.

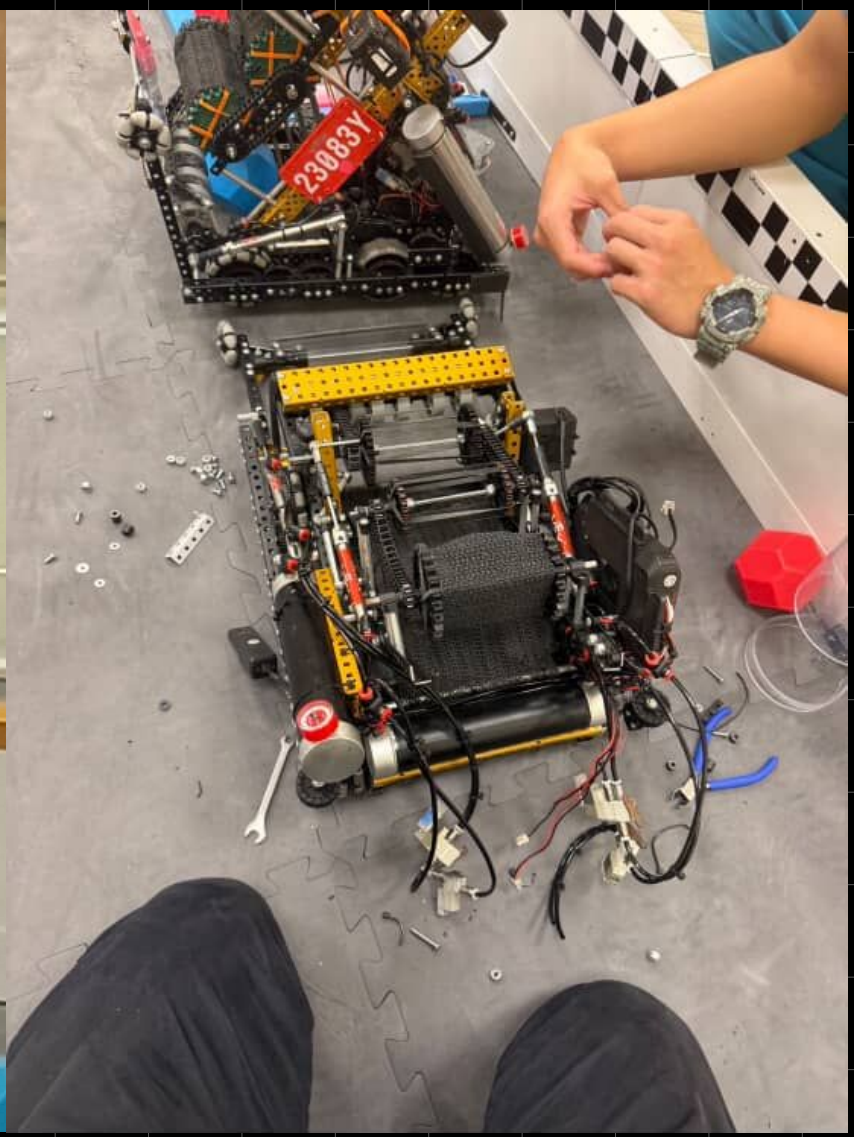




Broke....

Just 5 days before the PAS Tournament...



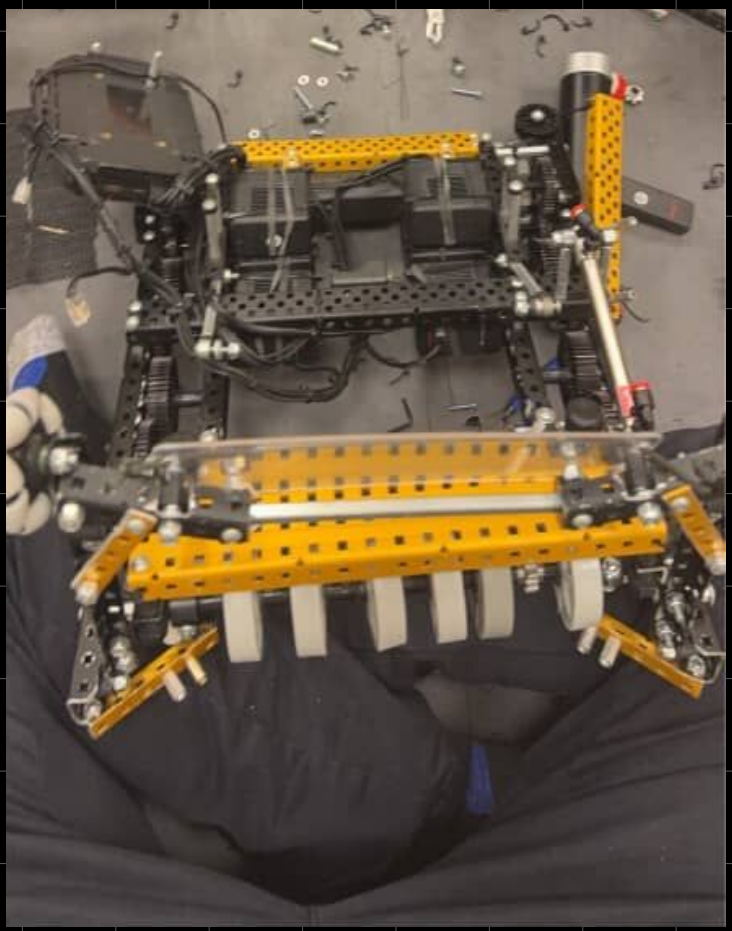


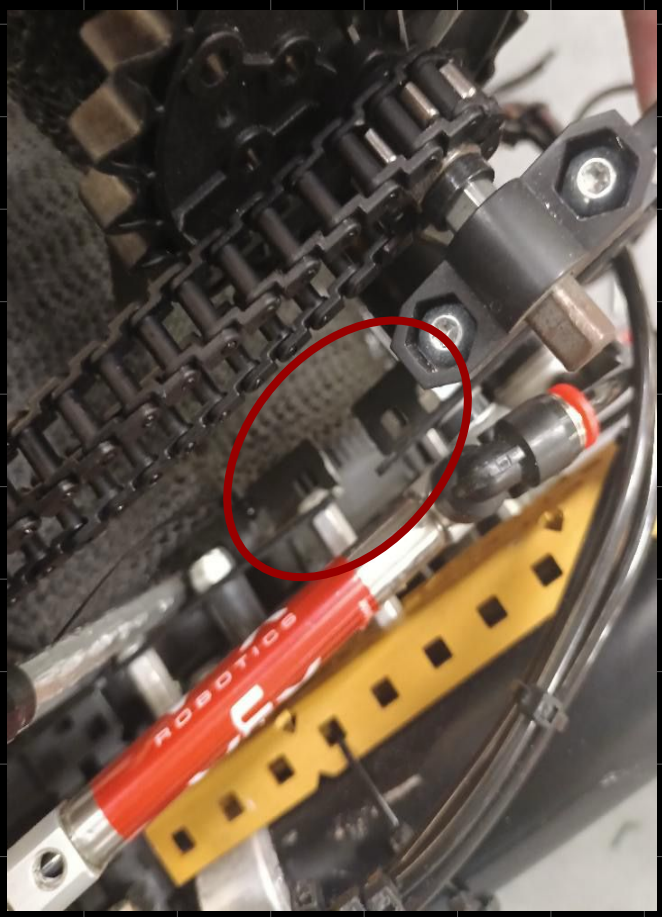
Our robot broke apart...

Broke....

It was an ordinary practice day, progressing as planned, until our driver, Adam, suddenly shouted, “Huh, it broke?!”

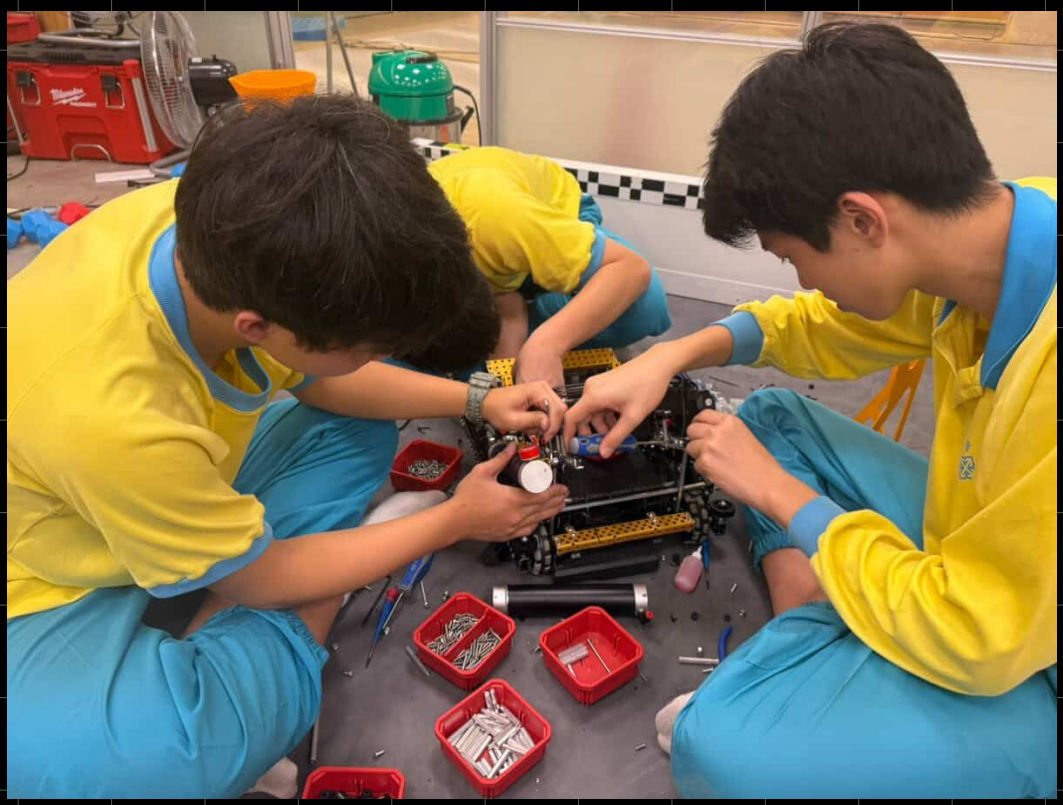
When we inspected the robot more closely, we discovered that a critical structural component—the main supporting ramp structure—had failed. Two angle plates had split in half, compromising the integrity of the robot.





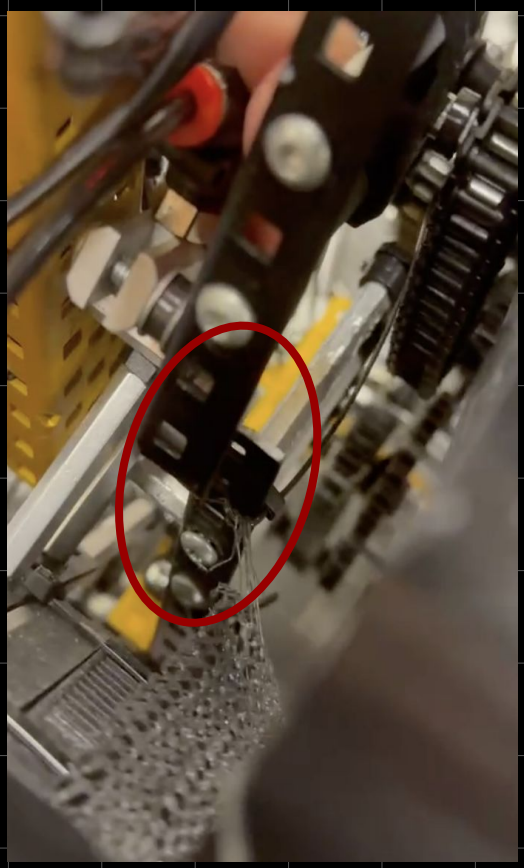
Broke...

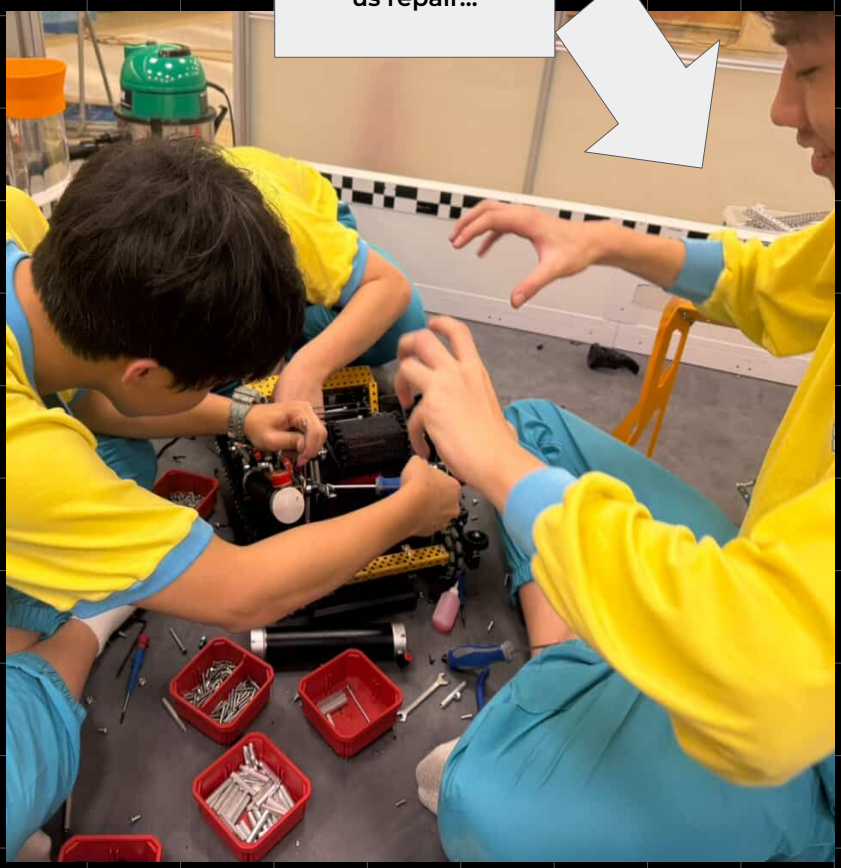
We immediately halted all practice and shifted our focus to repairs. The damaged parts were replaced with stronger components, and additional reinforcement was added to increase overall stiffness. This incident highlighted a weakness in our design and emphasized the importance of structural durability. After completing the repairs, we carefully tested the robot, hoping the improvements would prevent similar failures in the future.



Fixed!

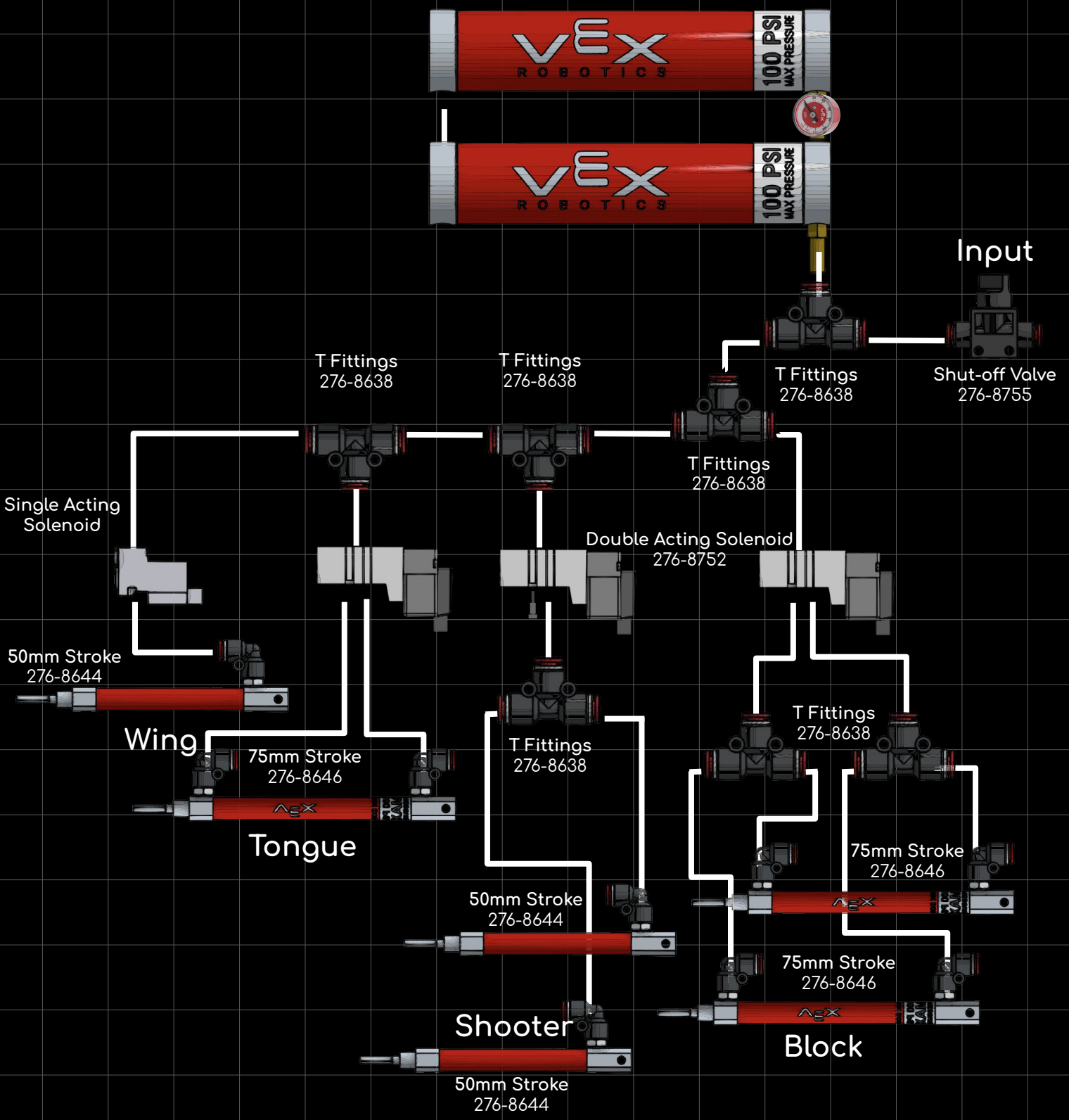
After testing the repaired robot, it felt as though the robot had been completely renewed, with improved strength and performance. Although the failure was unexpected and concerning, it ultimately led to meaningful improvements in our design. The experience was a close call, but it resulted in **positive outcomes and a stronger, more reliable robot.**



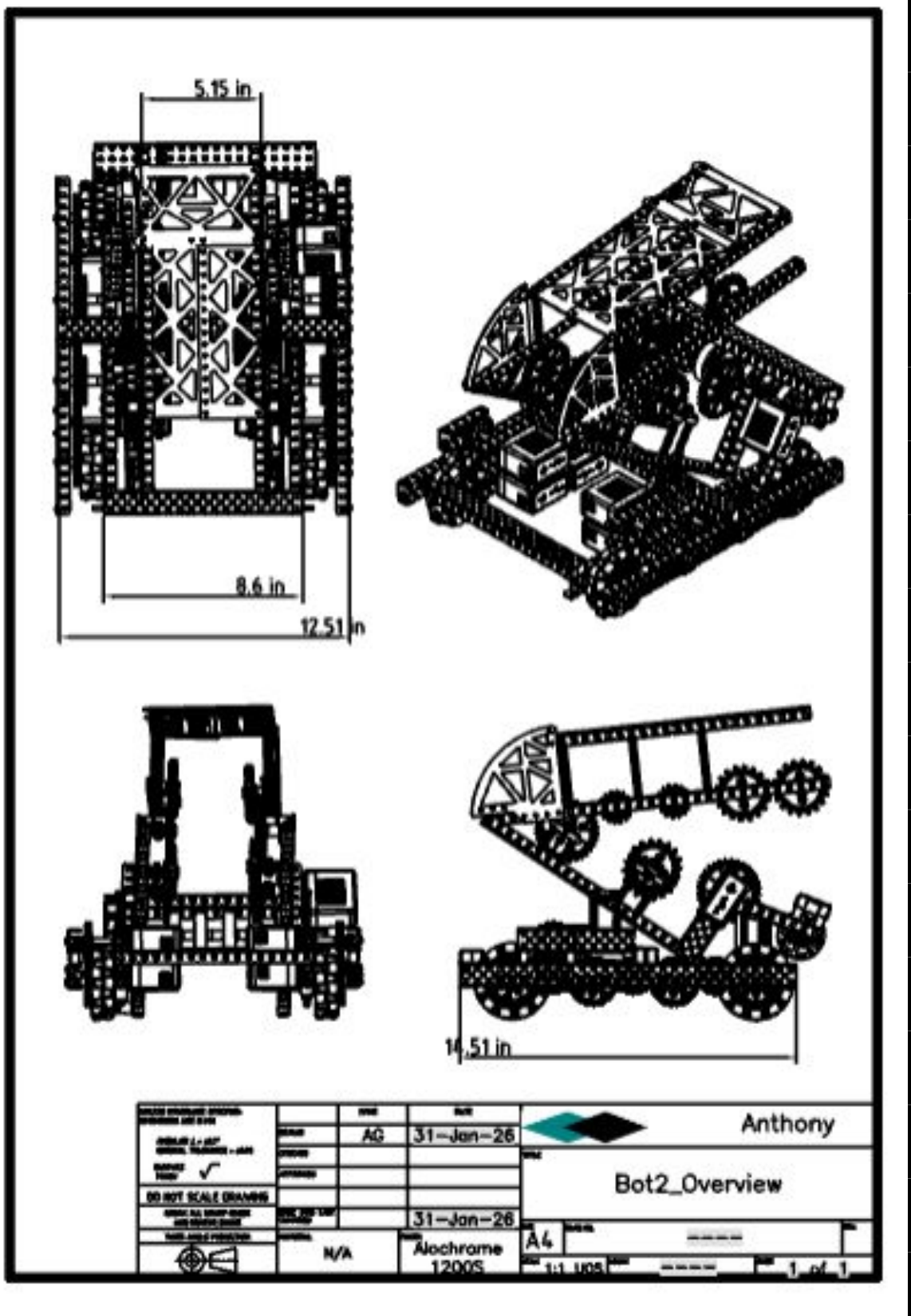


Pneumatic System

Pneumatic System Diagram:

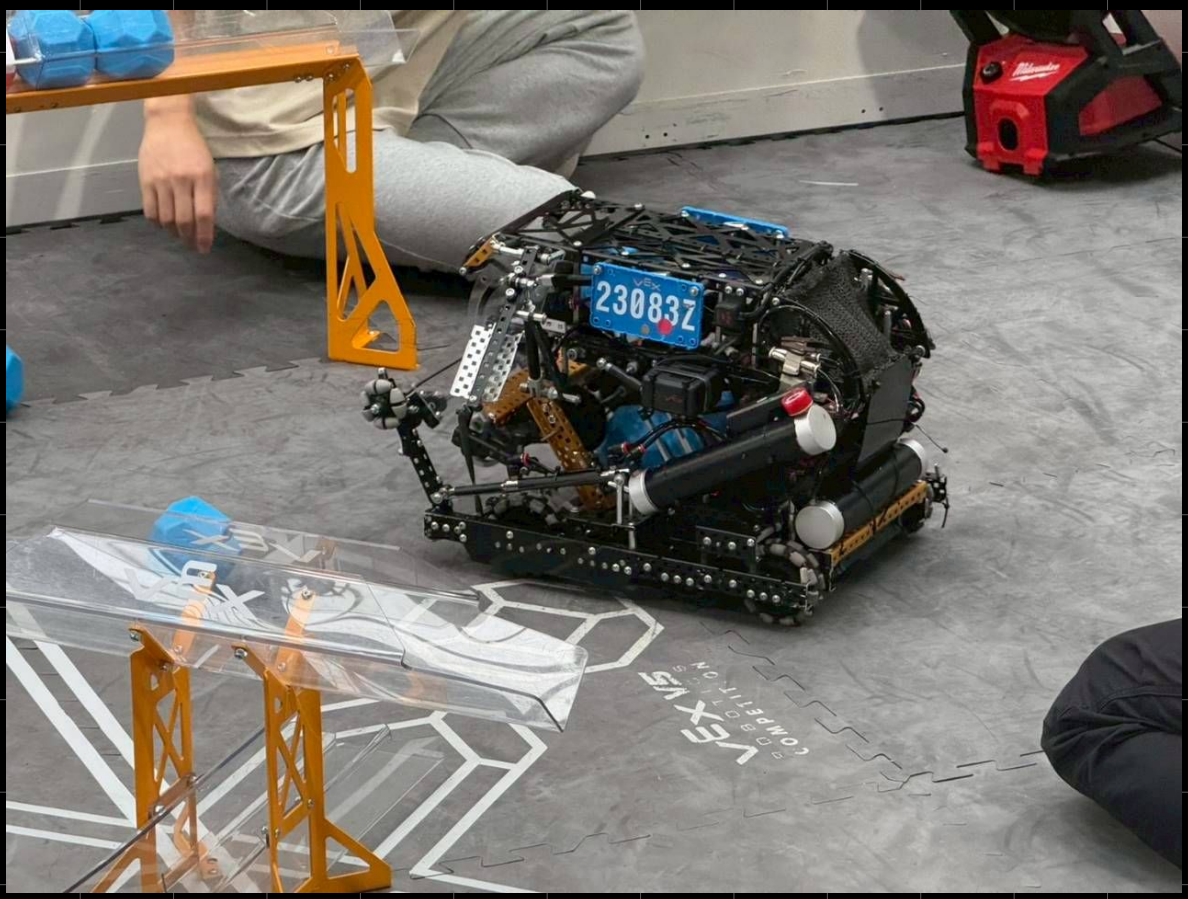


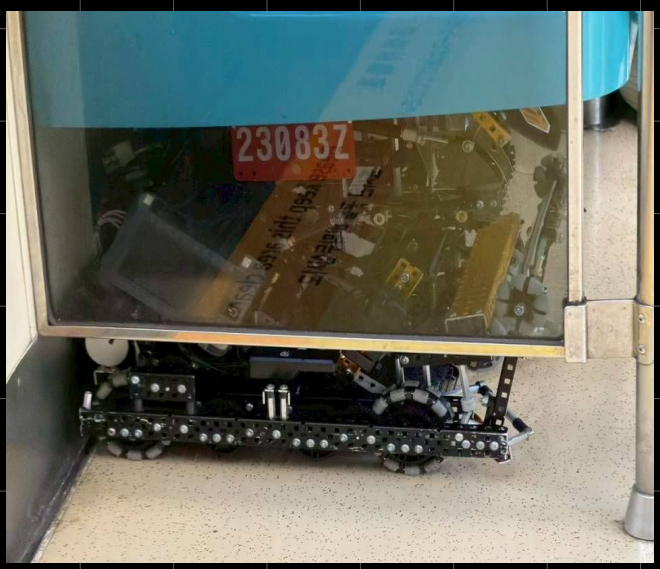
Engineering Drawing for Robot 2



Robot 2

Final 23083Z's Robot 2:





It's even short enough to place under the seats on the Taipei MRT!

Pneumatic System

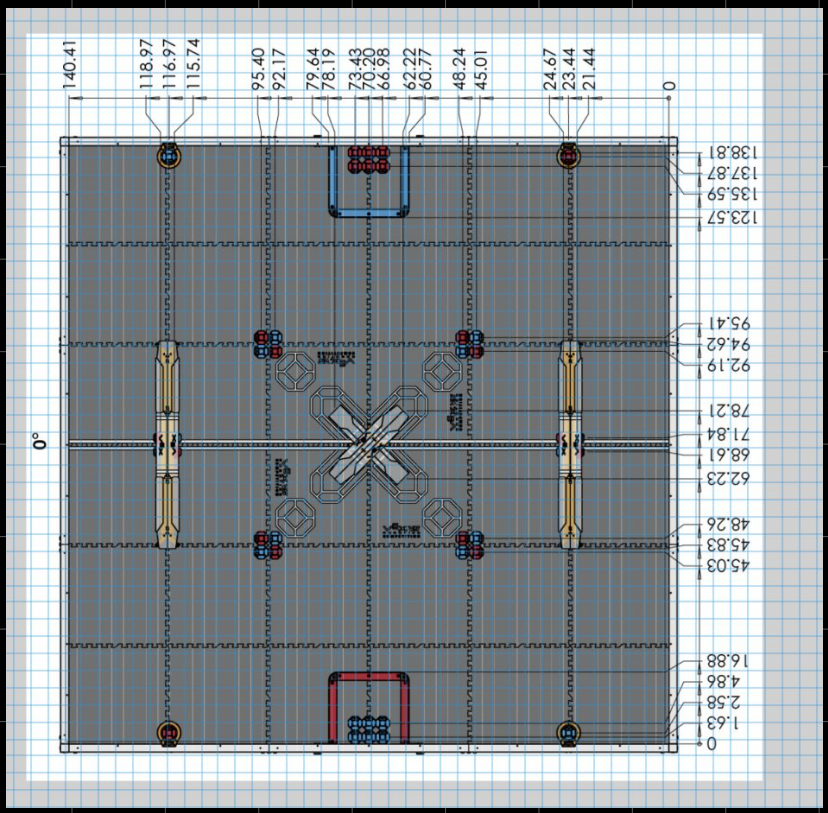
We have 4 pneumatic sets, each representing a function of the robot.

1. Tongue:
   a. double acting
   b. 75mm
   c. 1 piston
2. Shooter:
   a. single acting (double acting solenoid but one end plugged in)
   b. 50mm
   c. 2 pistons
3. Blocker:
   a. double solenoid
   b. 75mm
   c. 2 pistons
4. Wing:
   a. single acting
   b. 50mm
   c. 1 piston

Programming

We decided that our autonomous takes up too much of our time, so that we can't prepare for other things, therefore we either need to improve its accuracy or find a method to write code with less testing/at home.

Because I was designing **clothes** logos using Inkscape, I thought of using x and y positions to improve the accuracy. All I needed was one proportional to real life map that could give me X and Y positions.



Programming

As we wished to still use PID functions of JR-Template, we wanted to maintain **`chassis.turn_to_angle()`** and **`chassis.drive_distance()`** functions.

This was more basic than we thought, we don't need to rebuild, recalculate a recording system of x and y positions. Instead, we just used the distance = $\sqrt{(dx^2 + dy^2)}$ formula to calculate moving distance, while turning to the point first so that the PID won't make it run an unpredictable path.

```
void driveToXY(double targetX, double targetY, double maxV, double turnMaxV) {
    // ========== STEP 1: Calculate target heading (theta) ==========
    double dx = targetX - robot_pose.x;
    double dy = targetY - robot_pose.y;
    double targetHeading = std::atan2(dx, -dy) * 180.0 / PI;

    // Normalize to 0-360
    while(targetHeading < 0.0) targetHeading += 360.0;
    while(targetHeading >= 360.0) targetHeading -= 360.0;

    // ========== STEP 2: Turn to face target heading ==========
    chassis.turn_to_angle(targetHeading);
    vex::wait(50, vex::msec);  // Small delay after turning

    // ========== STEP 3: Drive straight to target ==========
    // Calculate distance to target
    double distance = std::sqrt(dx * dx + dy * dy);

    // Use chassis.drive_distance to drive straight while maintaining heading
    // This will maintain the targetHeading we just turned to
    chassis.drive_distance(distance, targetHeading, maxV, turnMaxV);
}
```

Programming

At times we wanted the robot to move in a desired heading, and we only want to restrict its x or y position, the other position would be based on the heading, we used **`driveToXAtHeading()`**. It moves the robot to a specific X coordinate while maintaining a desired heading angle. This is useful when the robot needs to approach a goal or obstacle at a specific angle.

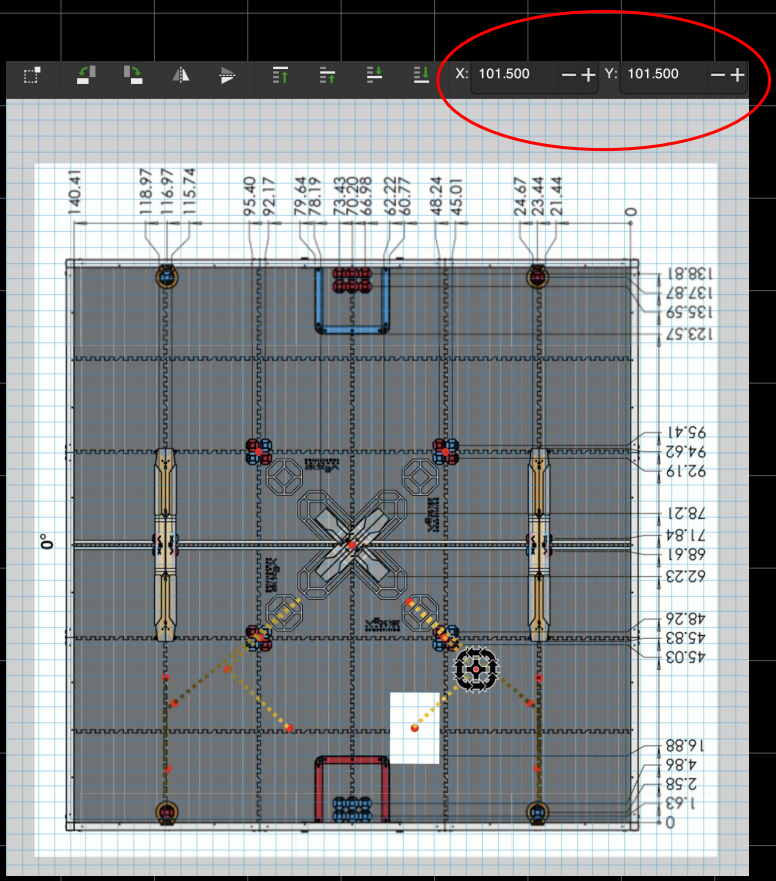
Therefore we needed to calculate arctan(heading) and get the other unknown position.

```
void driveToXAtHeading(double targetX, double targetHeading, double maxV, double turnMaxV) {
    double dx = targetX - robot_pose.x;
    double heading_rad = targetHeading * PI / 180.0;

    // Handle vertical headings (90° or 270°) where tan is undefined
    if(std::fabs(std::cos(heading_rad)) < 0.001) {
        // Heading is approximately 90° or 270° (moving purely in Y direction)
        // In this case, just drive straight at the heading
        double distance = std::fabs(dx);  // Use a reasonable distance
        chassis.turn_to_angle(targetHeading);
        vex::wait(50, vex::msec);
        chassis.drive_distance(distance, targetHeading, maxV, turnMaxV);
        return;
    }

    // Calculate target Y coordinate
    double dy = -dx / std::tan(heading_rad);
    double targetY = robot_pose.y + dy;

    // Now drive to the calculated X,Y coordinate
    driveToXY(targetX, targetY, maxV, turnMaxV);
}
```

Programming

For the actual implementation, we drew our car with our actual width and length, and at every point we needed to make a turn or stop, there is an dot, the dot would have the x and y positions, which should be in inches, similar to real life.

I used several points to adjust and calibrate so that the map is closest to real life.



Programming

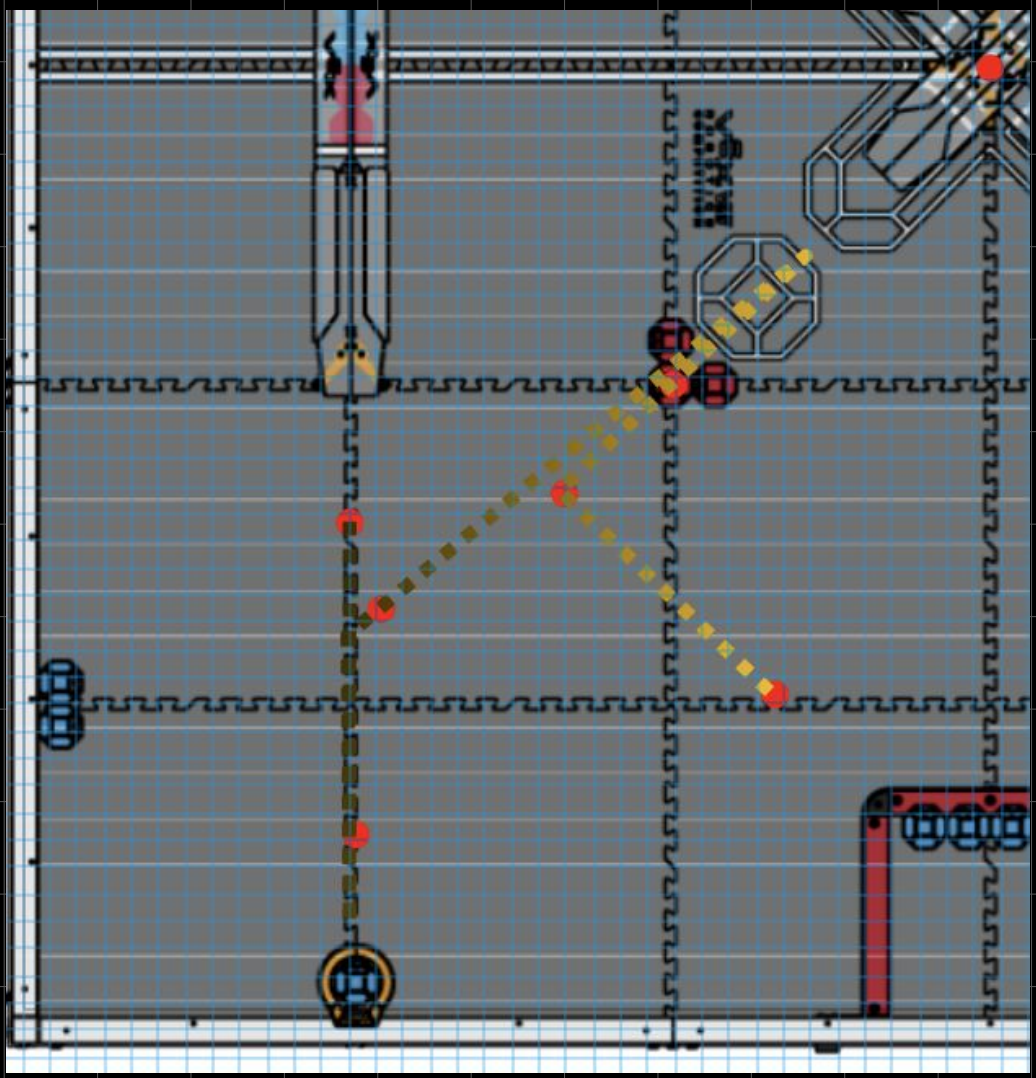
This allowed us to write code at home, and plan out our paths, so that the testing time would be minimized. We still used drive distance for unsure movements like moving forward, because it would be complicated if we had to set the coordinates. But no matter how much we (drive_distance) at, we can still go to the next accurate position not needing to change any later codes. This is the implementation example:

```
// Set initial pose: X, Y, and heading (0° = up/forward, 90° = right,
180° = down, 270° = left)
 set_robot_pose(86, 116.3, 0.0); // Robot starts facing up/forward (0°)
 intakedown.spin(forward, 10, volt);
 // ========== PHASE 1: Drive to intake position ==========
 // driveToXY automatically calculates and turns to face the target
 wait(0.05, sec);
 driveToXY(103, 105, 8.0, 6.0);
 wait(0.05, sec);
 intake.spin(forward, 10, volt);

 // ========== PHASE 2: Score 3 blocks ==========
 chassis.turn_to_angle(315);
 wait(0.05, sec);
 driveToXY(93.5, 93.5, 8.0, 6.0); // Approach slowly
 wait(0.2, sec);
 chassis.drive_distance(15, 315, 8.0, 6.0); // Drive to low goal
```

Programming

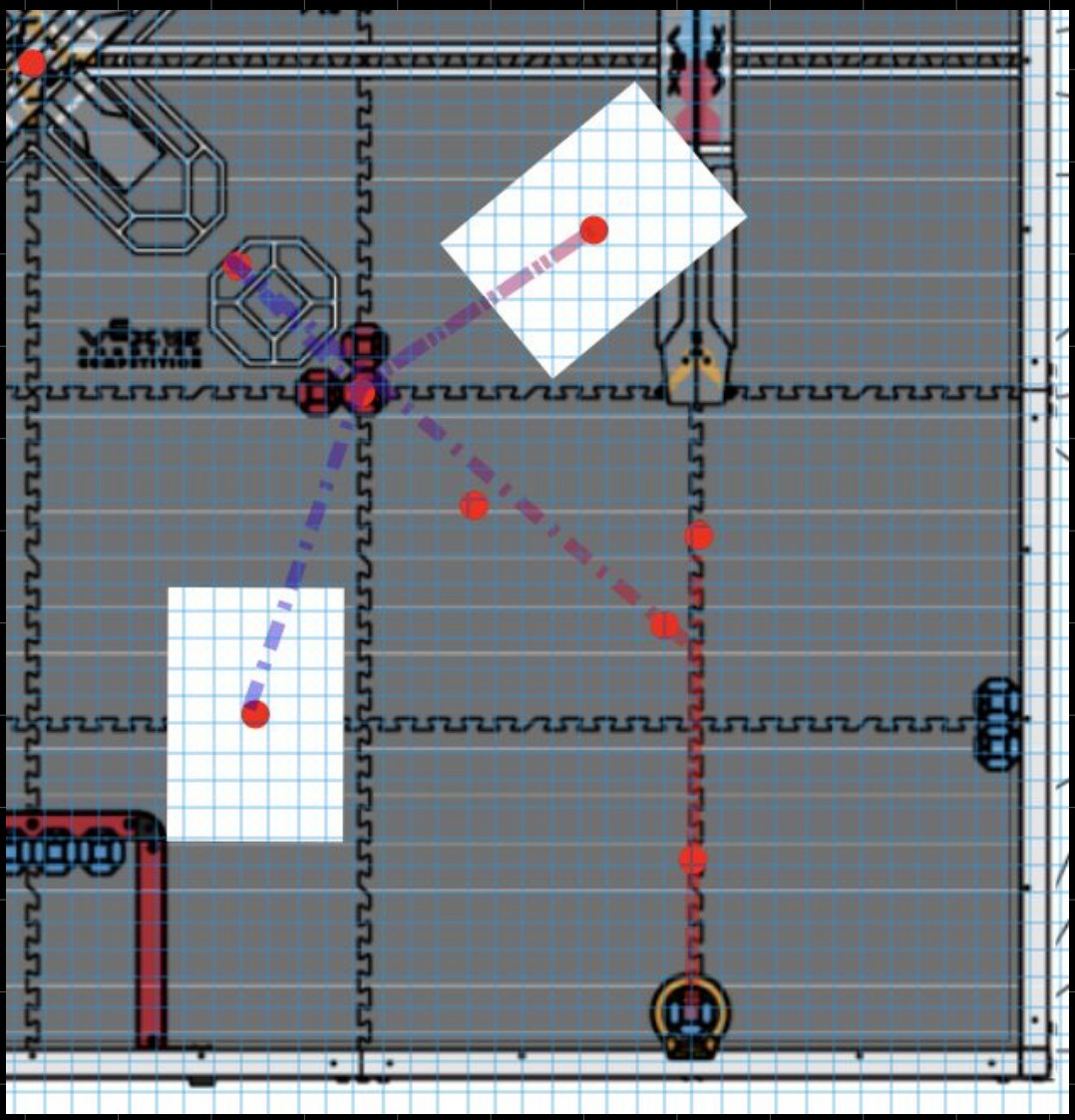
Our first autonomous path is scoring into 2 goals, ideally 4 into the middle goal, and 3 from the loader into the long goal.

This would be more strategic if our alliance can only score into one goal, so that we can get AWP for scoring into 3 goals.



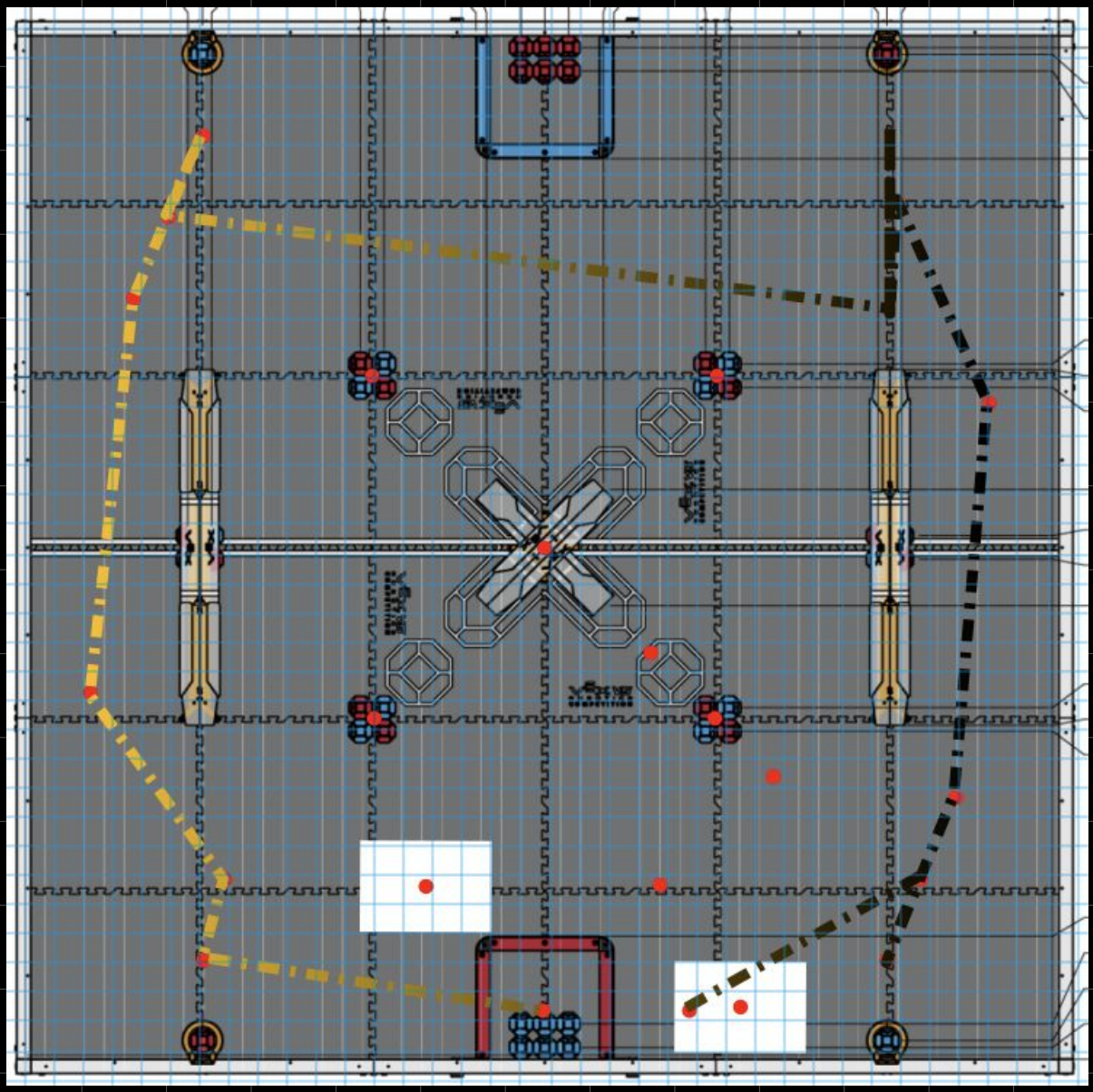
Programming

Our other autonomous path is developed for the long goal **control zone**, in which we will ideally score 9 into one long goal. This can be better if our alliance can score 2 goals, in order for us to as well get AWP.



Programming

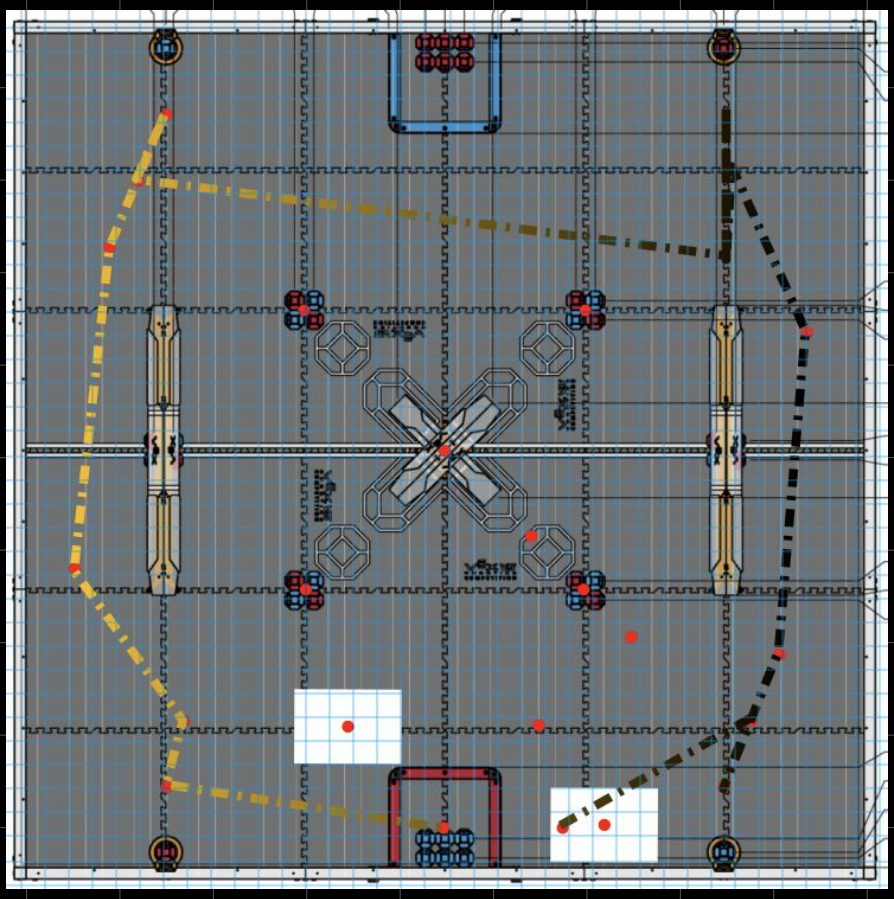
Our autonomous skills path is slightly different from our driver skills, and is expected to have a lower score due to the difficulty of parking and clearing parking zones.



Programming

To summarize our strategy, we load from all 4 loaders and score into the long goals, aiming to get both long goal control zone bonus. Because of the field set up, in order to guarantee control zone bonus, we shall load and score into a long goal from the same open end, therefore both long goals should be red bonus.

And finally the robot moves to the parking zone and clears the parking zone to gain 20 points.



Simulator

To further improve our driver's skills, we've updated our simulator after an Signature event on 12/26. We basically re-did the whole thing.

First off, we re-designed how the size of everything is calculated.

```
function fit() {
      const reserved = 420, minW = 800, minH = 600;
      canvas.width = Math.max(minW, window.innerWidth -
reserved);
      canvas.height = Math.max(minH, window.innerHeight -
24);
      canvas.style.width = canvas.width + 'px';
      canvas.style.height = canvas.height + 'px';
```

const reserved = 420 is to reserve space on the right side for the control panel. minW and minH ensures game elements remain above a certain size and remains constant.

Simulator

So the code “resizes” multiple times every second during a window drag. The loop goes as follows:
It calculates the new dimensions.

```
canvas.width = Math.max(minW, window.innerWidth - reserved);
```

It then recenters the playing field.

```
FIELD_MARGIN_H = (canvas.width - fieldWidth) / 2;
```

It at last rebuilds everything from every single thing.

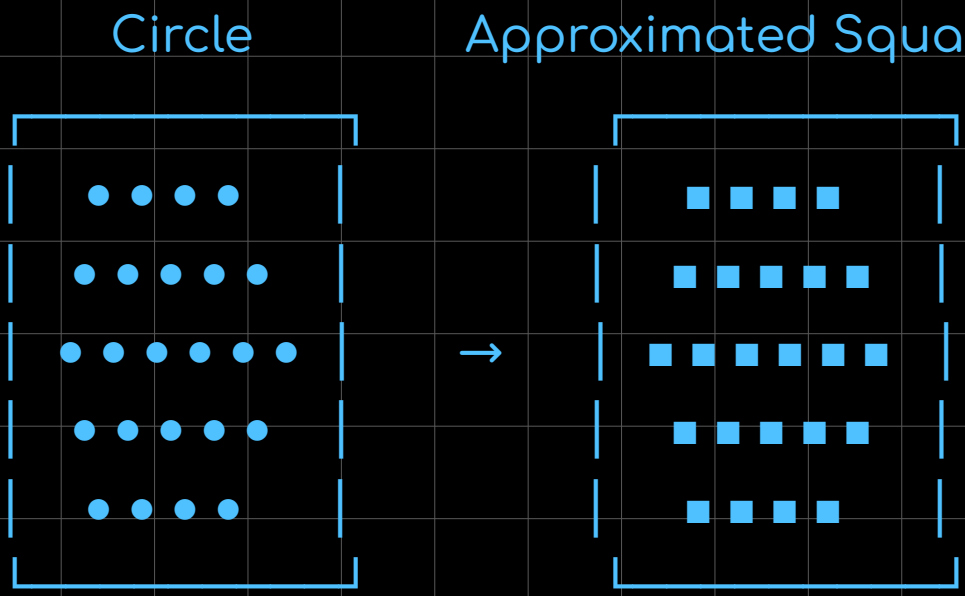
```
rebuildFieldObjects();
```

A sneak peak of the function:

```
function rebuildFieldObjects() {
      const fieldWidth = TILE_SIZE * FIELD_WIDTH_TILES;
      const fieldHeight = TILE_SIZE * FIELD_HEIGHT_TILES;
      BALL_R = TILE_SIZE / 12;
      BALL_DIAM = BALL_R * 2;
      robot1.length = TILE_SIZE * (2 / 3);
    //.....etc
```

Simulator

For the ball physics, we have to do something really creative, as there isn't a box for circles. If you get what I mean. We fetched the approximation of the circle and use little boxes to build a circle. Like how calculus calculates an objects volume, but reverse.



It turns a weird square-circle calculation into a much simpler square-square check. While not mathematically perfect, the approximation saves me a lot of time and also provides enough accuracy for the engine to make it work.

Simulator

Practically speaking, it enhances computational efficiency as I don't need to code a specific square-to-circle method to handle such situations when its just square-square. Furthermore, the blocks are relatively small, paired up with the fast speed of bots, the errors are usually minimal, or go unnoticed.

```
function circleObbPenetration(circle, obb) {
      // Convert circle to OBB for collision
detection
      const circleObb = { cx: circle.x, cy:
circle.y, hx: circle.r, hy: circle.r, angle: 0
};
      return obbObbPenetration(circleObb, obb);
    }
```

We also added an indicator to let players better visualize what balls they picked up.

Simulator

We also updated our driving physics:

```
function updateRobot(robot, dt) {
      const targetVl = robot.leftPower *
robot.maxSpeed;
      const targetVr = robot.rightPower *
robot.maxSpeed;
      const v = (targetVl + targetVr) / 2;
      const omega = (targetVr - targetVl) /
(robot.width);
      const vxTarget = v *
Math.cos(robot.angle);
      const vyTarget = v *
Math.sin(robot.angle);
      robot.vx += (vxTarget - robot.vx) *
robot.linearAccel * dt;
      robot.vy += (vyTarget - robot.vy) *
robot.linearAccel * dt;
      robot.angularVel += (omega -
robot.angularVel) * robot.angAccel * dt;
      robot.x += robot.vx * dt;
      robot.y += robot.vy * dt;
      robot.angle += robot.angularVel * dt;
}
```

Simulator

Because our robot is a C-bot, it has to lift its upper structure to score long goals, so that functionality was added. We also fixed an issue where lifted C-bots could incorrectly pass through long goals.

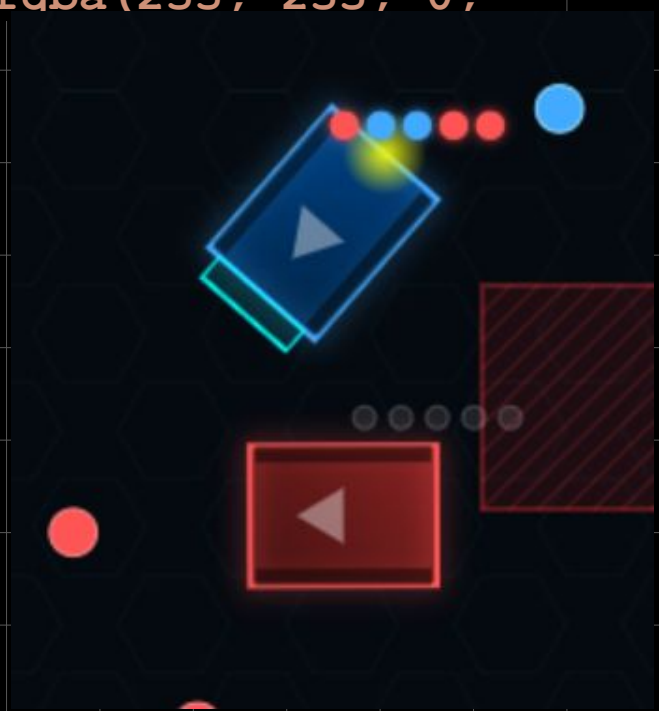
```
if (goalSet === longGoals) {
        if (!robot.isLifted) retu
        const padding = 15;
        for (const g of goalSet)
          const goalObb = aabbToObb(g);
          const triggerObb = { cx: goalObb.cx,
cy: goalObb.cy, hx: goalObb.hx + padding, hy:
goalObb.hy + padding, angle: 0 };
          }
        return;
      }
```

Simulator

VISUAL OVERHAULS!!!

If you can't tell by now, I did some visual overhauls to make the game look prettier. It took surprisingly more time than I had anticipated. Some of the hardest ones to do would be the gradient and the glowing edges.

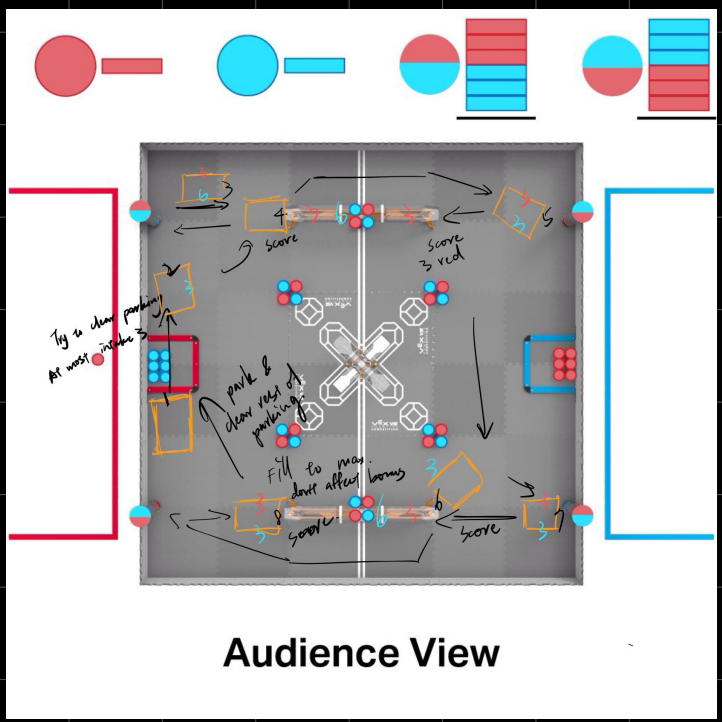
```
const pulse = Math.sin(Date.now() / 200) * 0.2
+ 0.8;
      ctx.beginPath();
      ctx.arc(0, 0, robot.length * 0.7, 0,
Math.PI * 2);
      ctx.strokeStyle = `rgba(255, 255, 0,
${0.4 * pulse})`;
      ctx.lineWidth = 2;
      ctx.stroke();
```



This segment of the code processes the rim pulses. It allows the edge of the bot to glow and pulse, give it a more lively feel. Is it useful? No. Does it serve a function? No. But is it cool? Hell yes.

Skills Strategies Ver.2

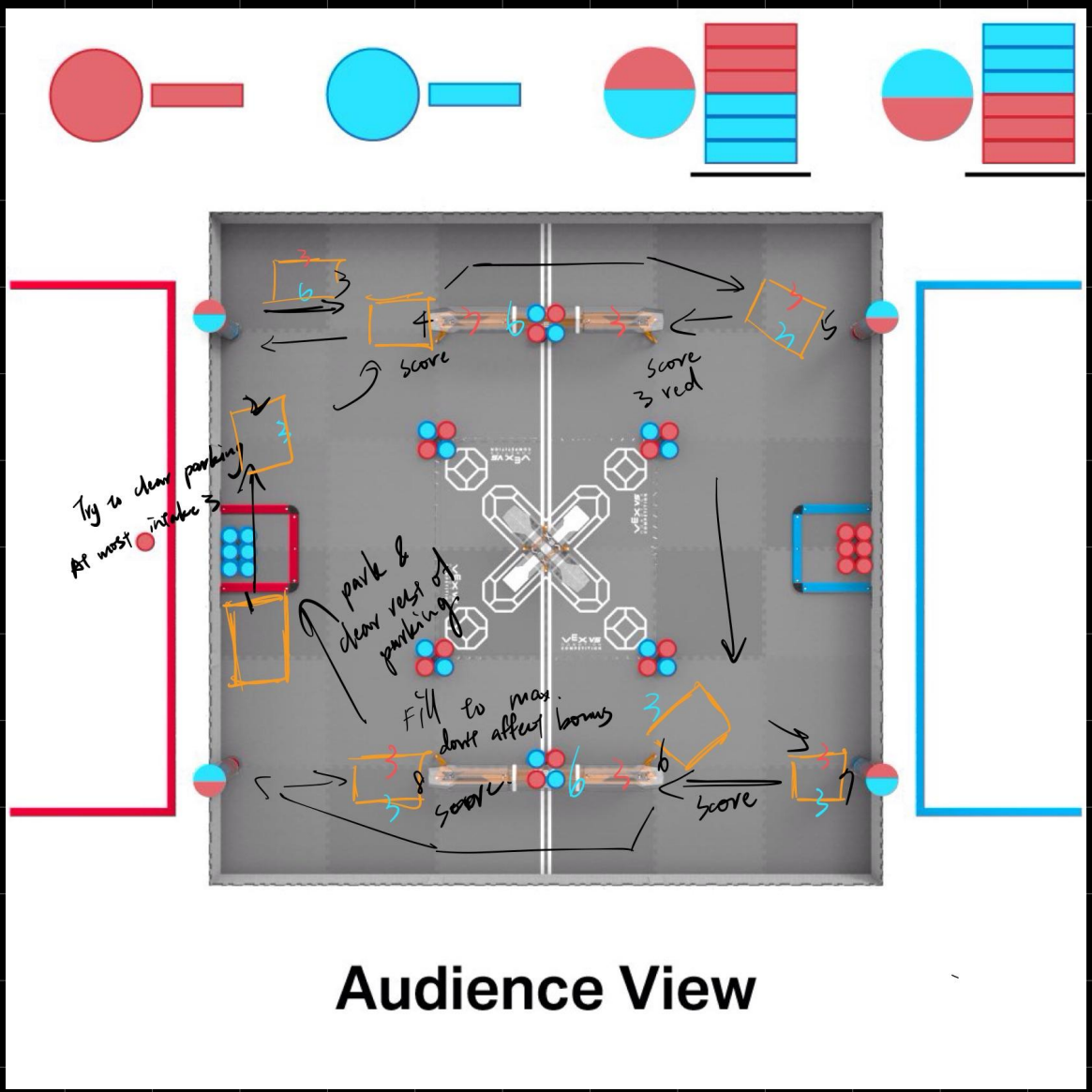
Considering that it is highly difficult to get the middle goals bonus, we decided to give up that score.
We emphasize the bonus points for long goals. We should make sure that we park and clear the red parking zone because that would make a 20 points difference, but our robot can't efficiently clear the 6 blocks at once, it would take too much time. Therefore we start driver skills as intaking about 3 blue blocks and then connecting to the first loader.



Audience View

Skills Strategies Ver.2

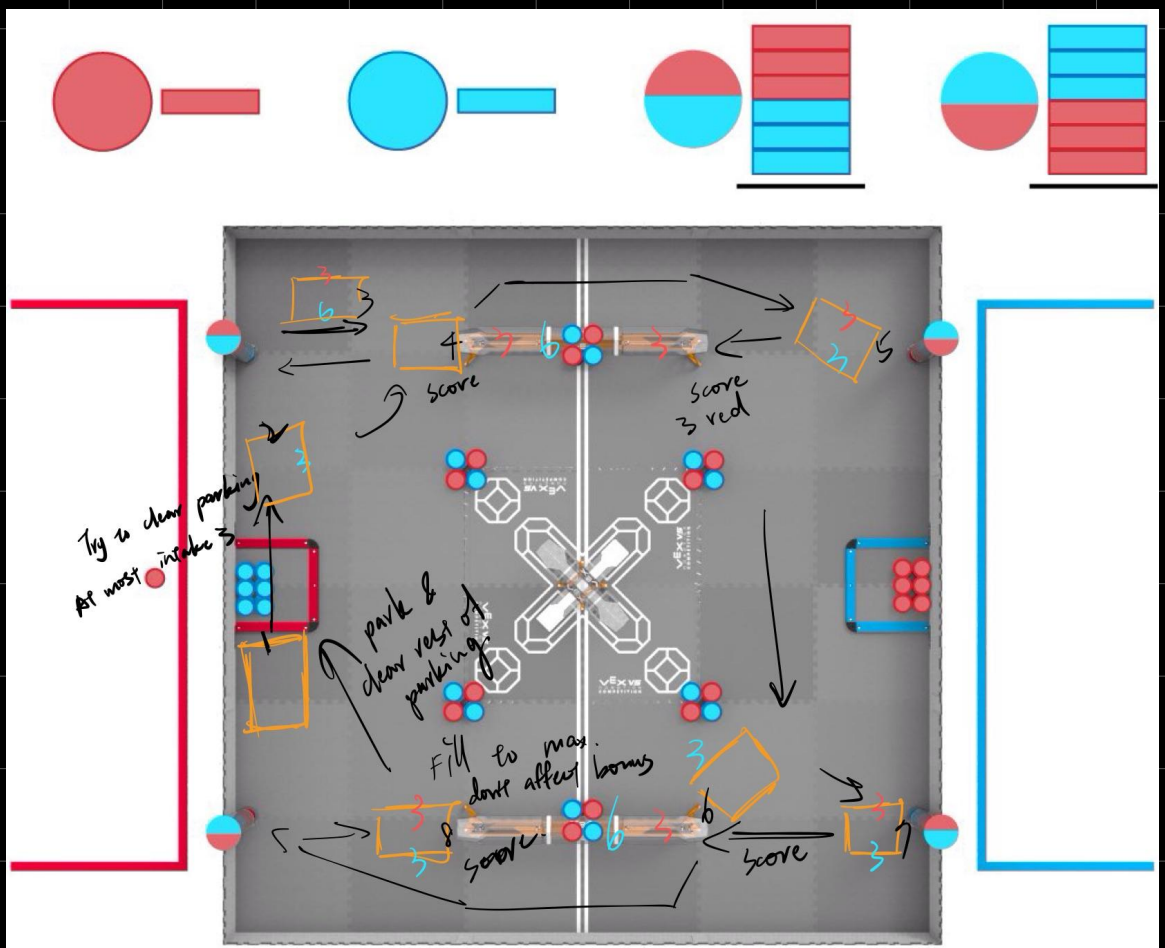
As shown in the figure below, we start from the left side of the red parking zone, try to clear half the blocks and pick them up in order to utilize them later. In an ideal situation, we should clear half the zone and pick them up.



Audience View

Skills Strategies Ver.2

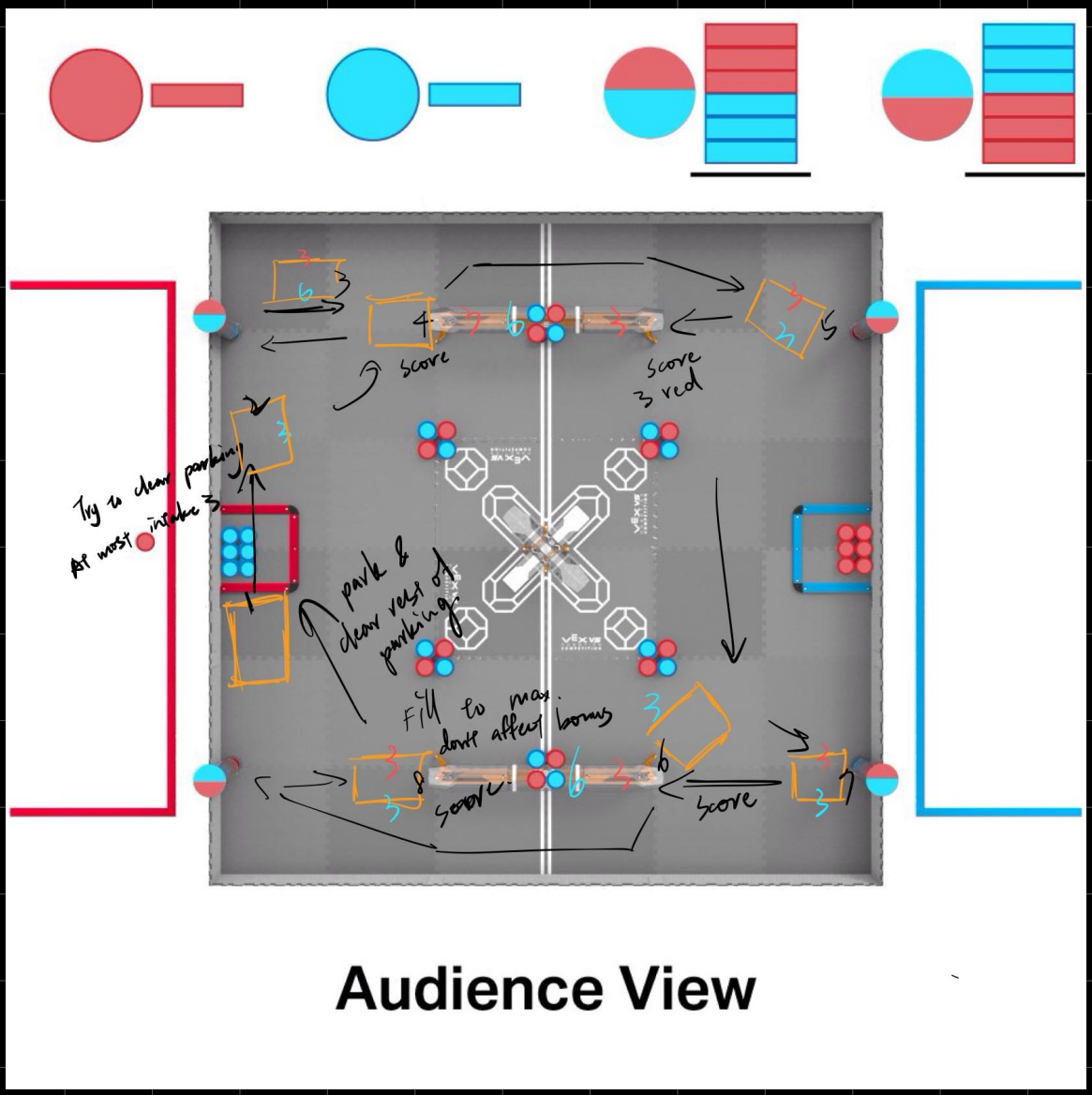
3 blue pieces already in the bot. After loading from the loader, the bot should hold 9 pieces total, 6 blues followed by 3 reds. Score them in that order into the long goal to secure the blue control bonus.



Audience View

Skills Strategies Ver.2

Crossing the field horizontally, we approach the second long goal. After loading all six blocks, we score only the three red blocks, saving the three blue blocks for the next control zone.

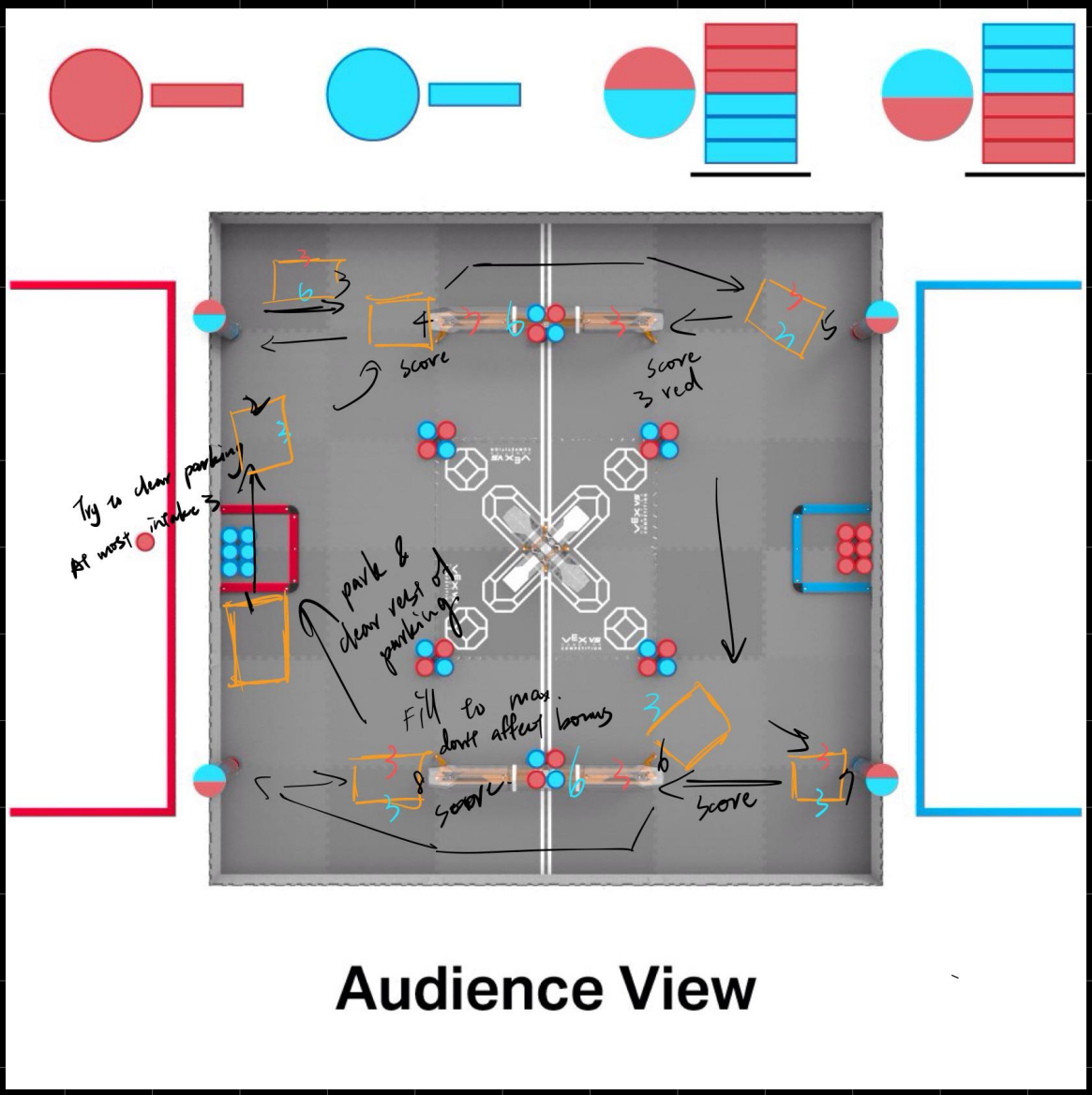


Audience View

379 | Brainstorming | Anthony | 12/16

Skills Strategies Ver.2

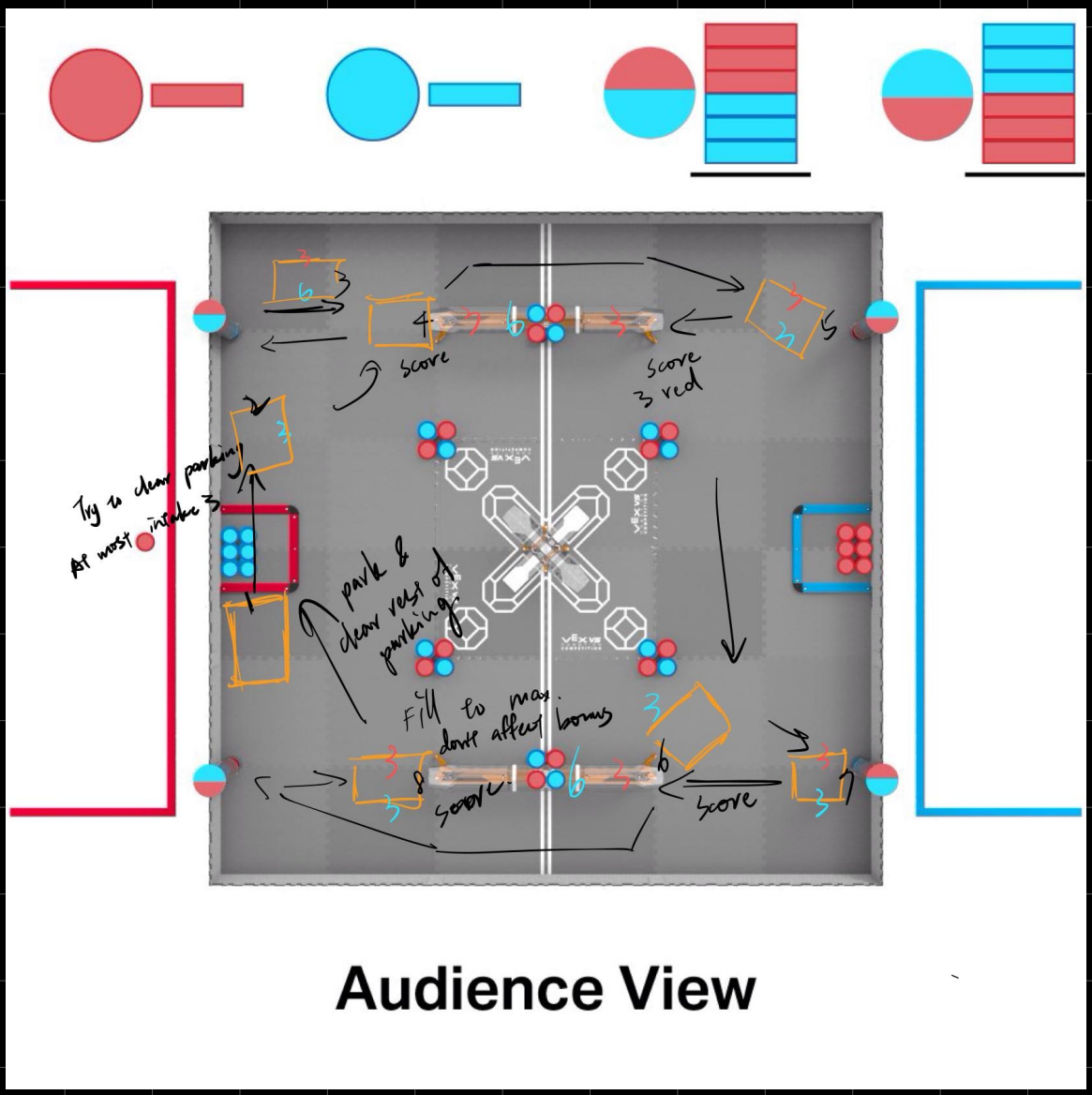
After crossing, we don't load the 3 blues. Theres a mistake in the picture below. Instead, we try to fully load the loader with the 3 blues already in there, adding up to 9 in total, and score. Blues should reach the control zone with no problem in theory.



Audience View

Skills Strategies Ver.2

After scoring the last goal, we turn back to parking. There should be at least 3 or more blocks still in there, so after clearing that out we should be able to park.

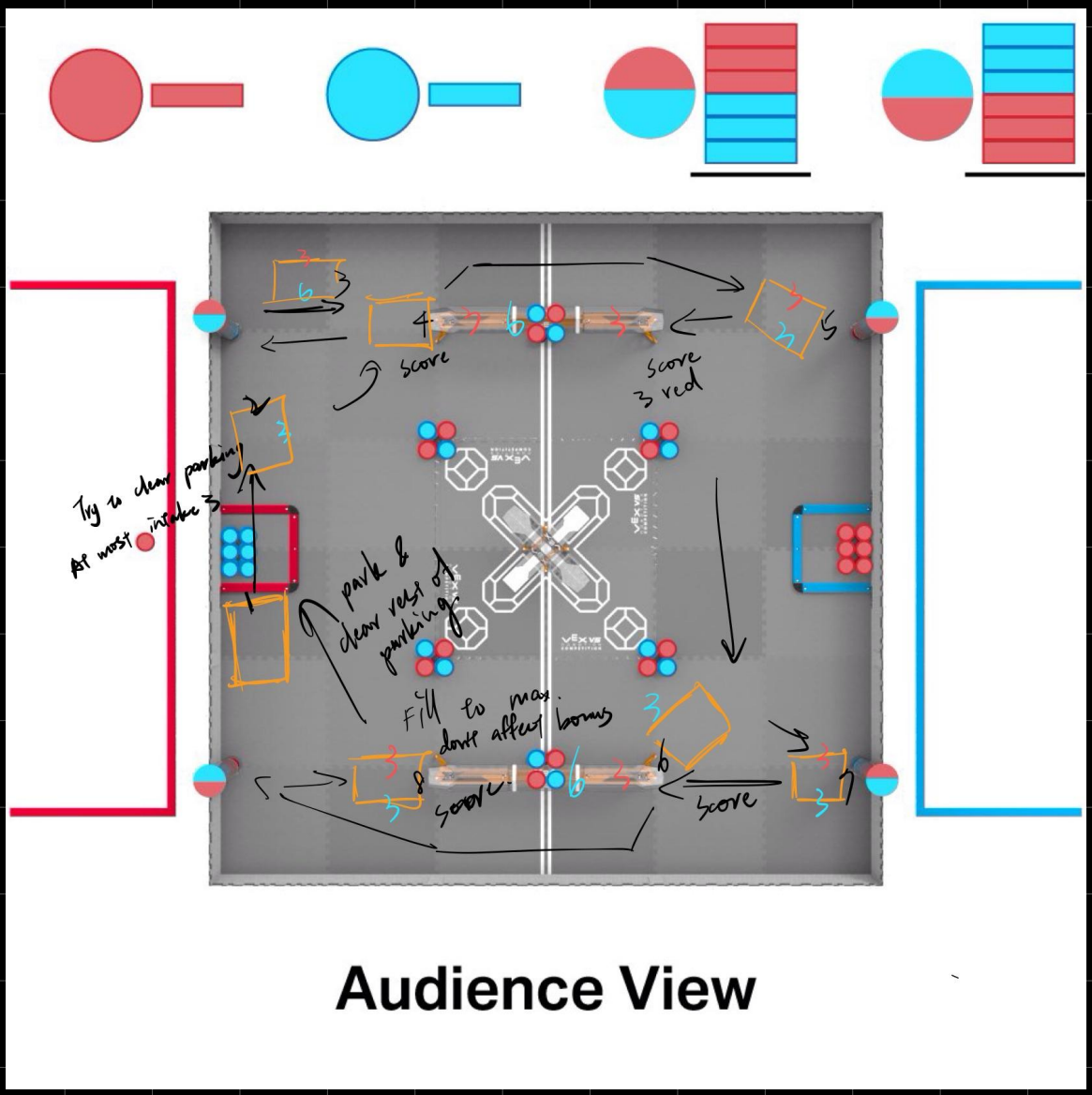


Audience View

381 | Brainstorming | Anthony | 12/16

Skills Strategies Ver.2

Ideally, the score would be: 4 cleared loaders (4*5=20), 2 filled control zones (2*5=10), 24 blocks scored (24*1=24), 1 cleared parked zone (1*5=5), and a parked bot (1*15=15). So the scores would tally up to be 20+10+24+5+15=74.



Audience View

Create Award

As for VEX competition itself, we luckily struck gold during the PAS Signature Event and ended up taking home the Create Award. While unexpected, the recognition validated our creative and deliberate approach to this season.

On the topic of this season, it is a grind. Push Back is simple yet demands immaculate precision and consistency. Making us feel like we can't even manage simple tasks at times. Some people on our team even left mid-season. It was that gruelling.

When progress is slow it is easy to focus only on the difficulty of things. Which, inevitably, leads to discourse among ourselves. Despite it all, receiving the Create Award reminded us why we began this season, to pursue creative engineering solutions, and encouraged us to continue improving as a team.

PAS Signature

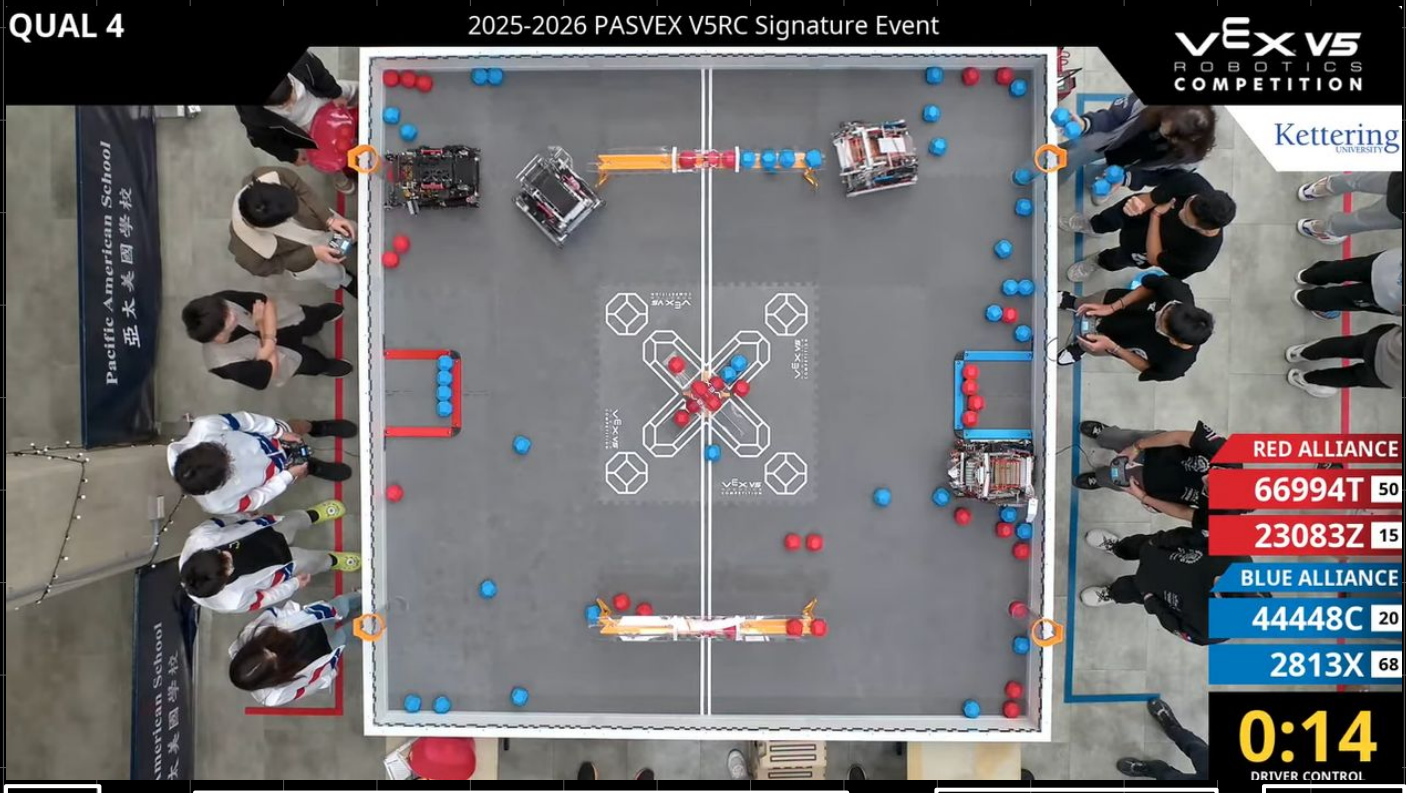
MATCH RESULTS

Q4	66994T 23083Z	106	11	44448C 2813X
Q21	67000A 85301Z	14	88	85301V 23083Z
Q40	50922S 4139B	19	50	23083Z 15383A
Q55	4139C 23083Z	31	105	96969W66994B
Q79	85301W 936D	3	109	23083Z 17022A
Q97	66799X 23083Z	31	39	4253S 1002X
Q114	23083Z 2779A	45	28	23083Y 2813Z
Q140	2769B 23083Z	43	64	4253W 92641C
R16 #8-1	4253Z 2053V	82	15	13176A 23083Z

PAS Signature

23083Z 66994T	106	11	44448C 2813X

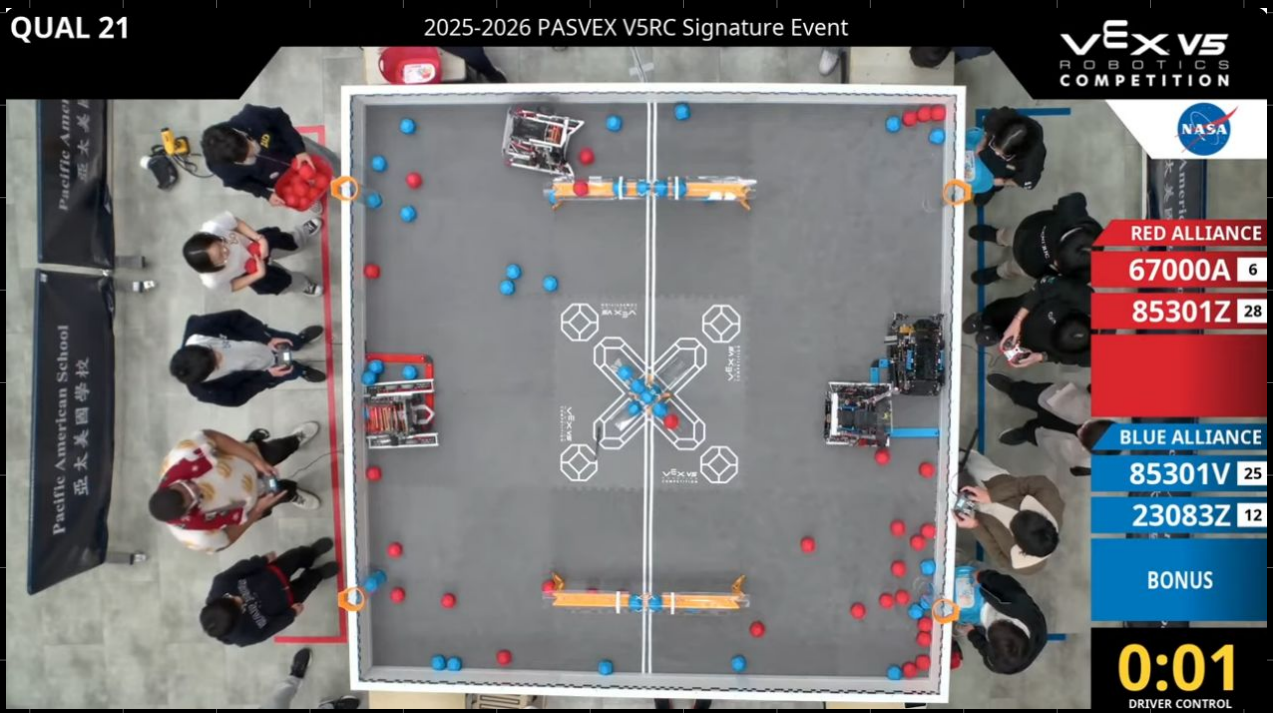
One of our opponents was unable to move from the start of the match, which gave our alliance a significant advantage. This allowed us to secure the win more easily. Our alliance partner, 66994T, demonstrated strong wing play by defending the left-side long goal in the final moments of the match, teaching us the importance of correctly timing match loads.



PAS Signature

67000A 85301Z	14	88	85301V 23083Z

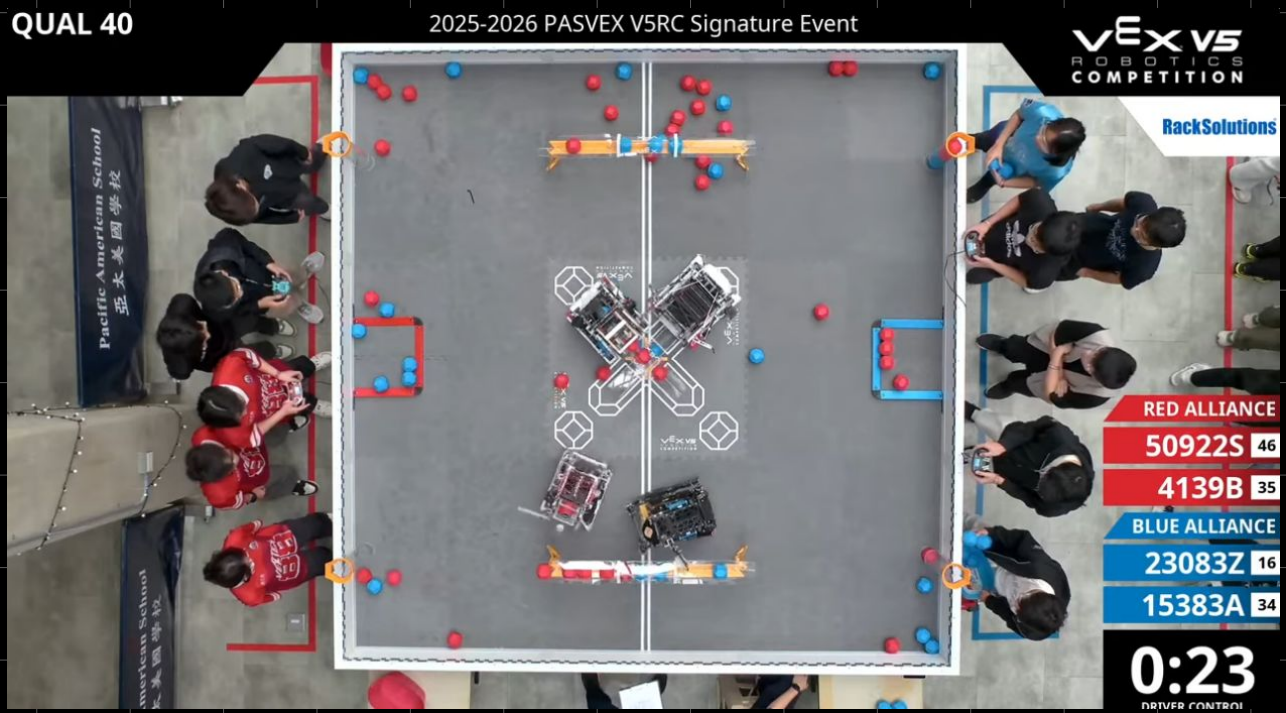
We had a strong alliance partner who controlled one side of the long goal for most of the match. On our side, we initially struggled to regain control of the long goal control zone, but we were eventually able to retake it by approaching from the opponents' side of the field. The main drawback of our performance was that we were unable to successfully complete a double park with our partner's double park mechanism.



PAS Signature

50922S 4139B	19	50	15383A 23083Z

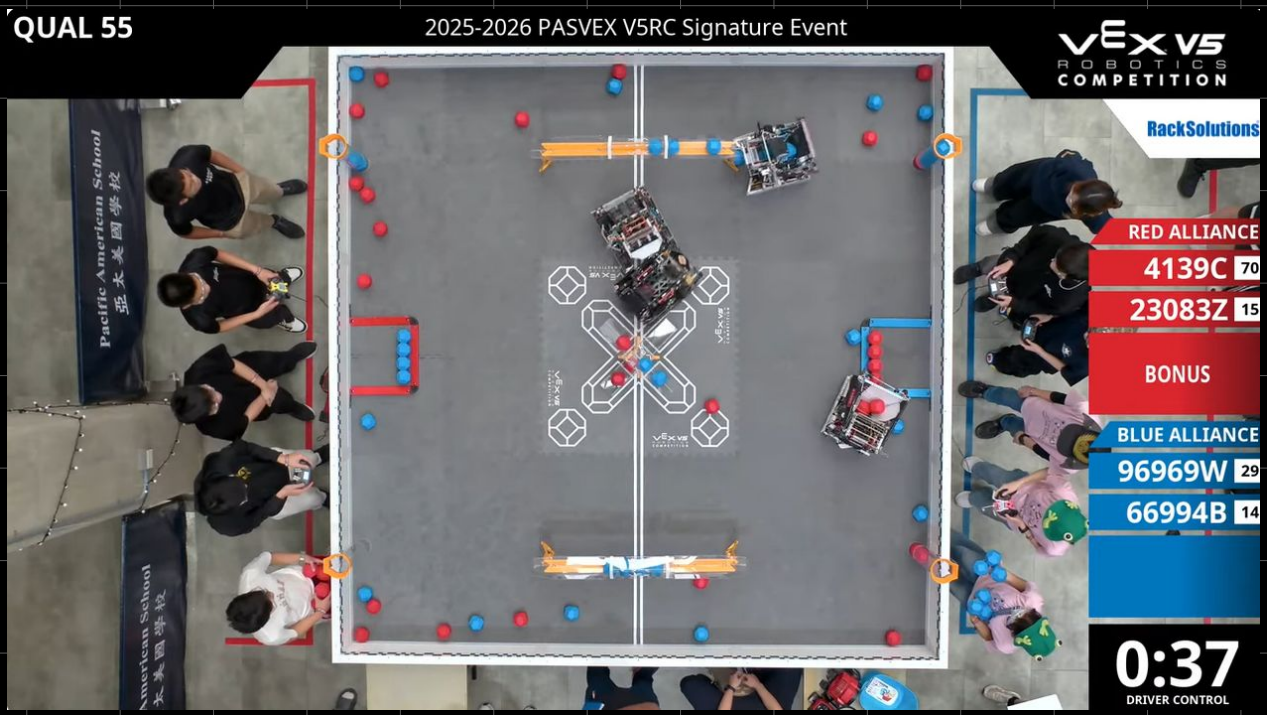
Our opponents controlled the long goal for most of the match. Although our team briefly lost momentum, we executed a strong recovery near the end by scoring three balls, regaining the control zone, and removing our opponents' blocks through a solid wing play.



PAS Signature

23083Z 4139C	31	105	96969W 66994B

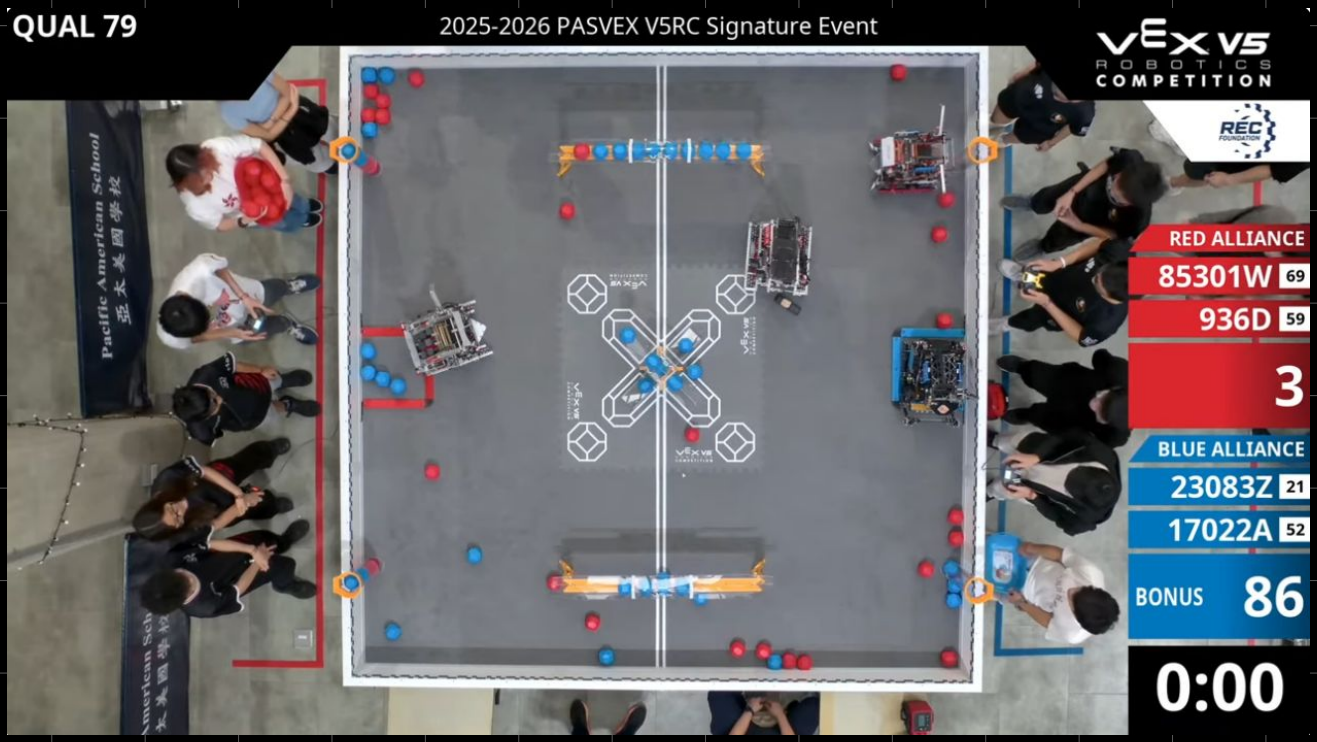
We entered this match knowing we were facing the eventual tournament champion, 66994B, along with another strong opponent. Although our alliance partner lost connection partway through the match, we were able to secure the autonomous bonus by almost completing a solo AWP at the start. Overall, despite the loss, this was a match we felt positive about due to our strong autonomous performance.



PAS Signature

85301W 936D	3	109	23083Z 17022A

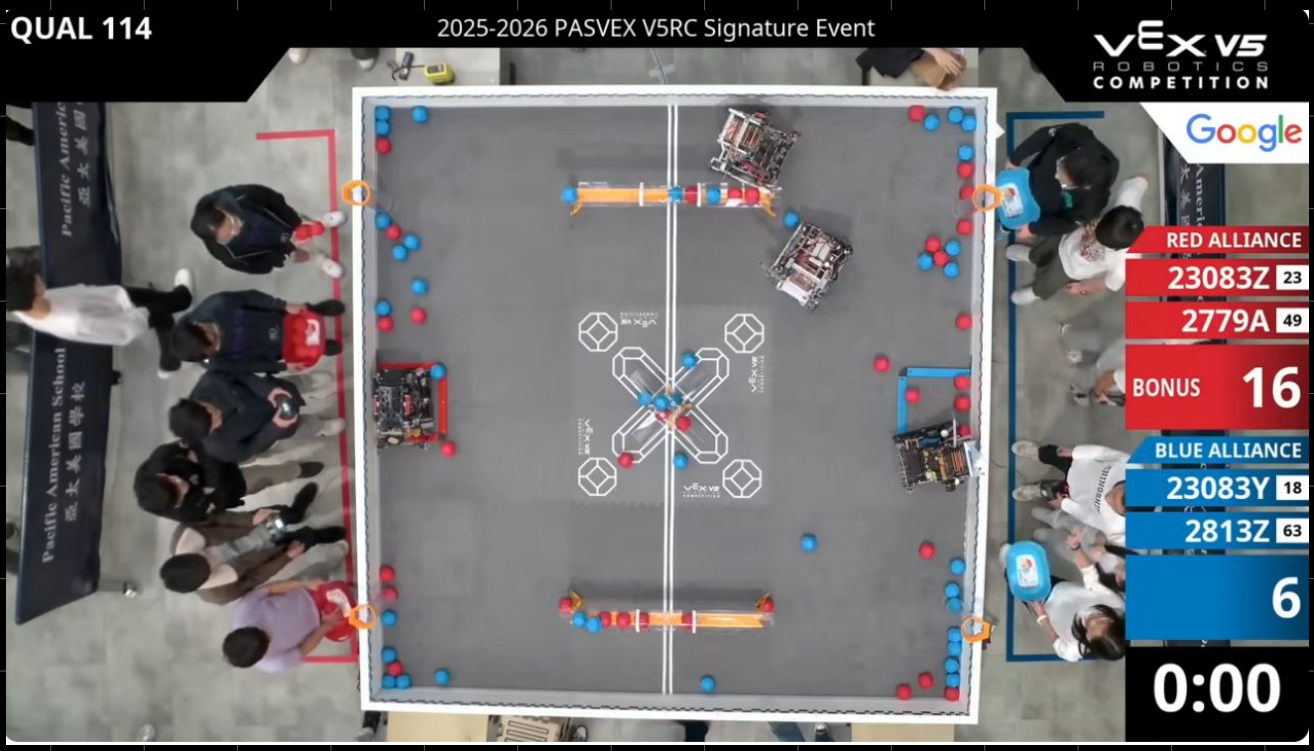
Qualification Match 79 was a textbook example of effective alliance coordination. Our alliance maintained control for the entire match. We began by focusing on the center goals while our alliance partner secured their side of the long goal. We then switched sides and successfully defended the long goal, allowing our alliance to establish control over all control zones and secure another victory.



PAS Signature

23083Z 2779A	45	28	23083Y 2813Z

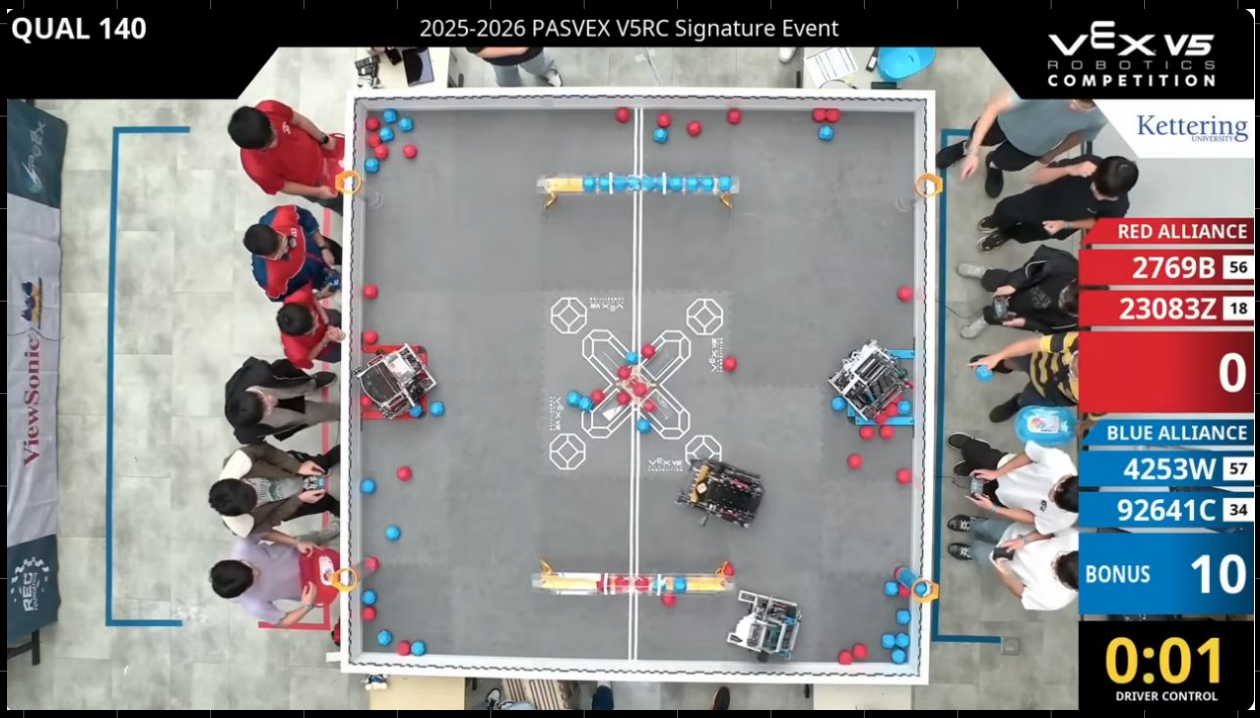
In Qualification Match 114, we executed a starvation strategy. From the start of the match, after securing our own control zone, we focused on defending our opponents for most of the match. This prevented them from scoring into the long goal, while we occasionally added blocks to the center goals to increase our advantage. We concluded the match by parking our robot, securingthe the win.



PAS Signature

23083Z 2769B	43	64	4253W 92641C

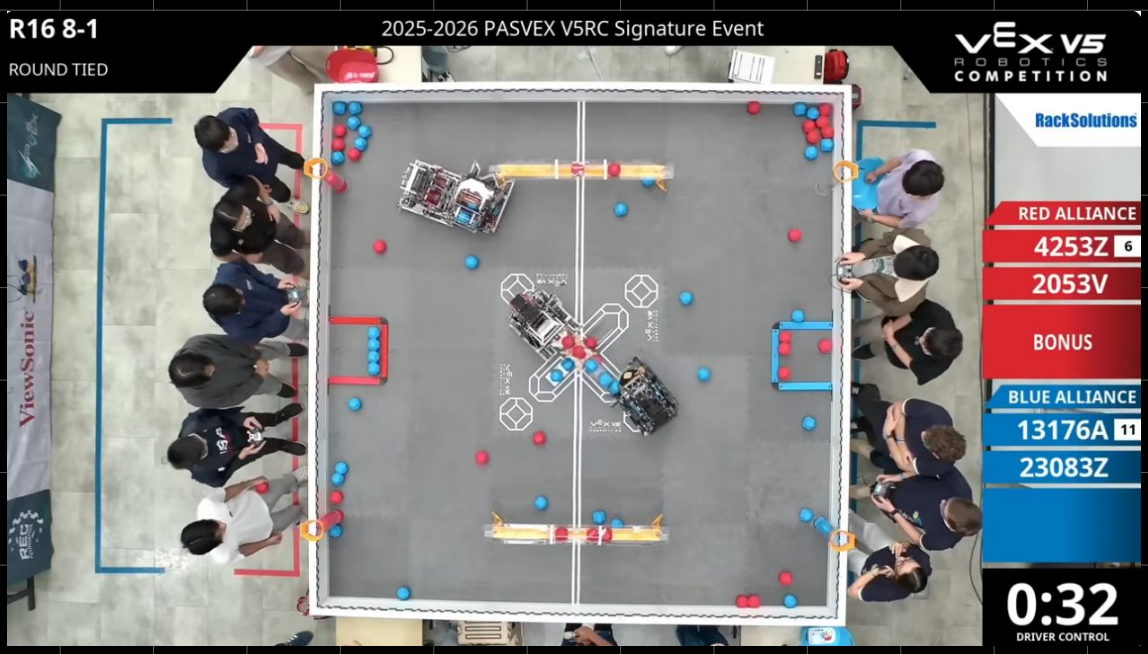
In our final qualification match, we faced a formidable opponent. We secured the center goal control bonus and made strong wing plays throughout the match. However, in the very last second, our opponents took back control of the long-goal control zone by a strong wing play. At the same time, our alliance partner's side of the long goal was filled with opposing blocks.



PAS Signature

4253Z 2053V	82	15	13176A 23083Z

In this match, we were unable to regain control of the center goals, which are a key strength of our robot. Our opponents used strong center-goal descoring and, after gaining control of the long-goal control zone, applied heavy defense against us. In addition, our chassis experienced overheating and we ran out of blocks to score early in the match. Despite the loss, it was a valuable match that provided important lessons for future improvement.



07
Robot 2.2

Wing

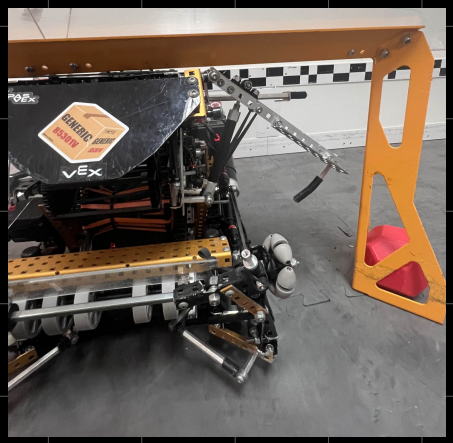
Because our robot is low enough to pass under the long goals, installing a wing is more difficult:

1. A typical wing would be easily damaged by opponents at that height.
2. Due to the narrower design of the upper shooter, our wing must be significantly longer than others (15-hole angles) in order to swing efficiently into the long goals.

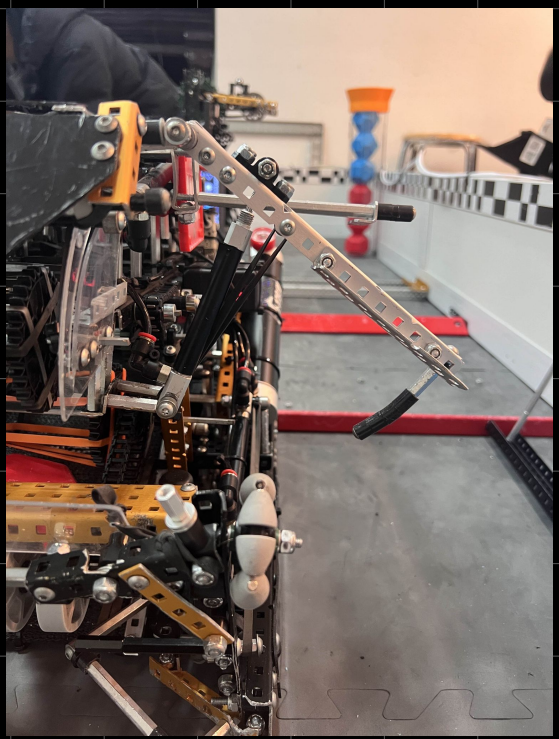
Standard wing (98601Y Atlantis)

Wing

Ultimately, we decided to use a single acting 50mm stroke pneumatic cylinder with a rubber band, directing its rod and hook to the lowest possible point. This allows us to pass under the long goals while maintaining strong wing performance.



Going under long goals

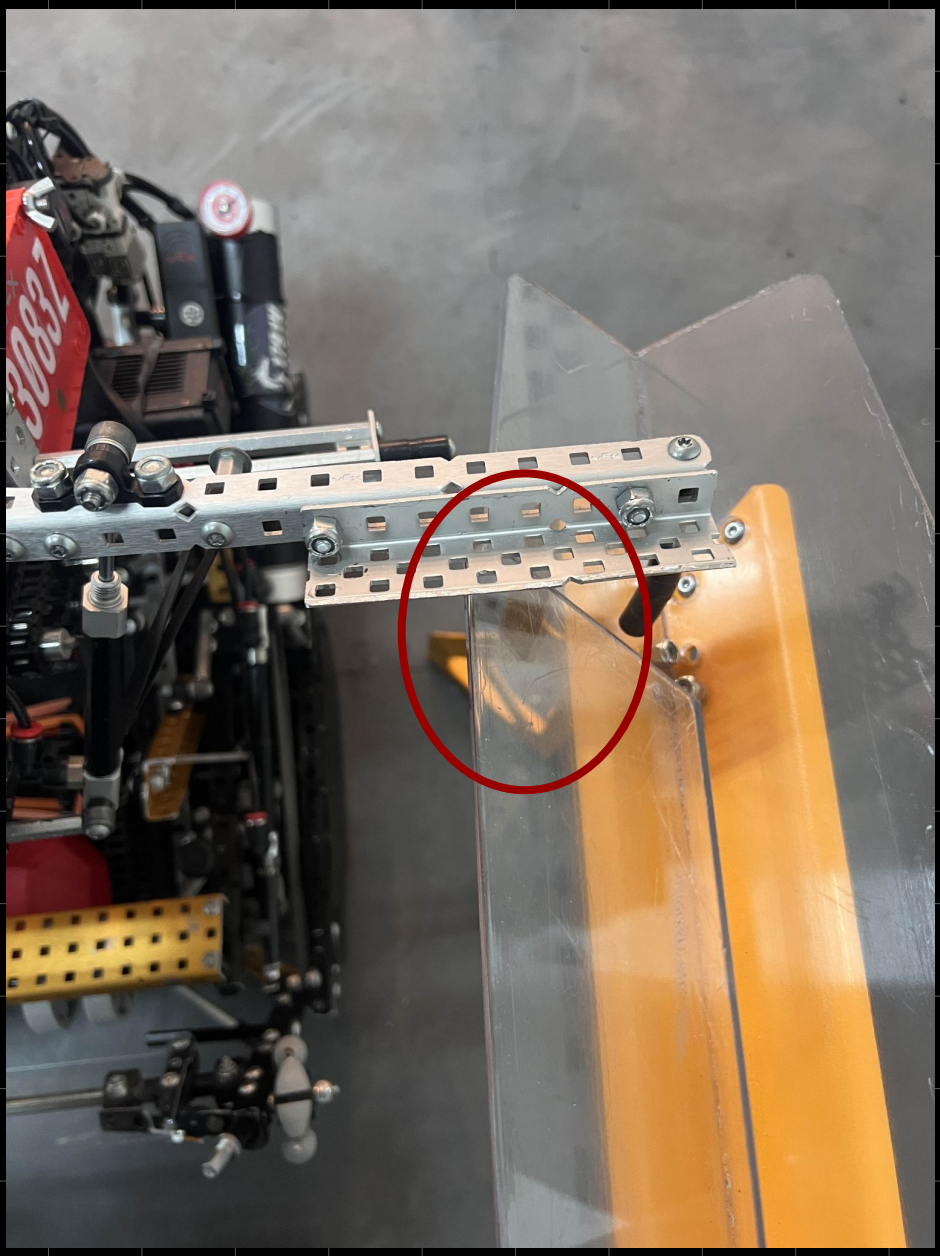


Unpressurized

Maximum reach

Wing

Because the wing's height depends on the surface it contacts, we identified a risk that the wing could become caught on the side ramps of the long goals.

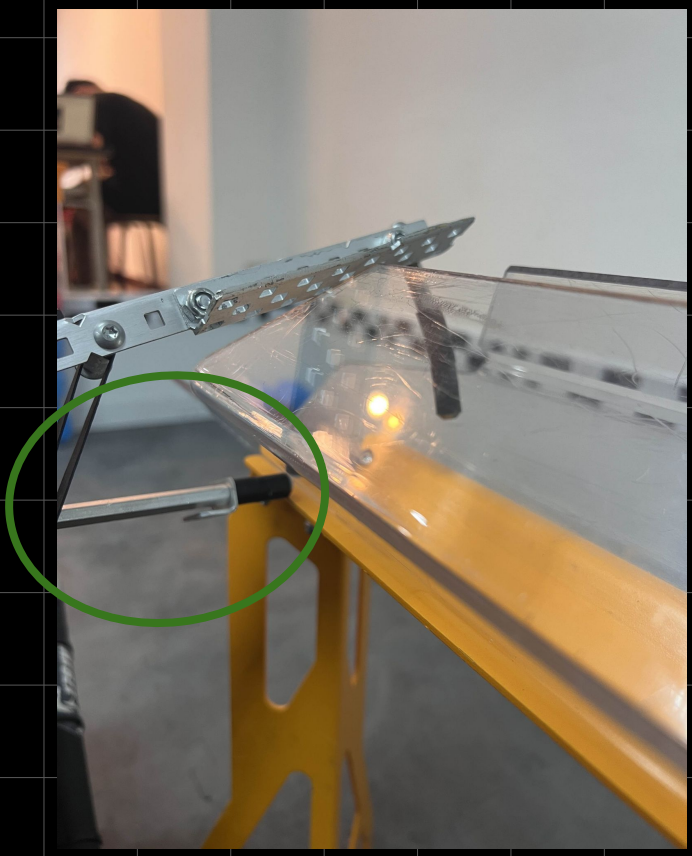


sliding surface that allows the wing to pass through the ramp

Wing

To mitigate this issue:

1. we added a low-friction sliding surface that allows the wing to pass smoothly through the ramp opening.
2. we incorporated wing aligners that mechanically engage with the metal frame and acrylic panels of the long goal. These aligners help guide the wing into position, improving consistency and reducing the driver's alignment effort during matches.

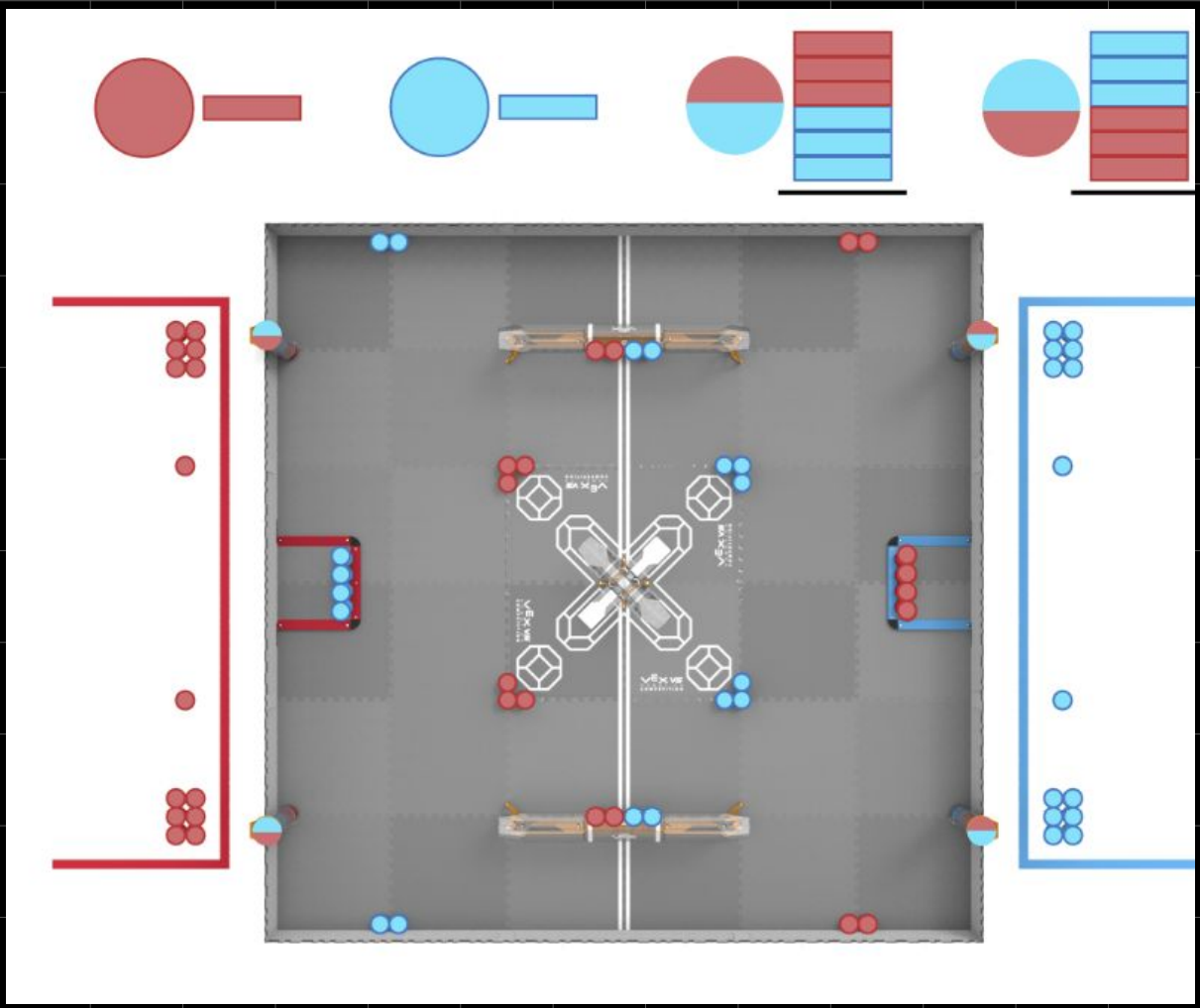


Wing aligner

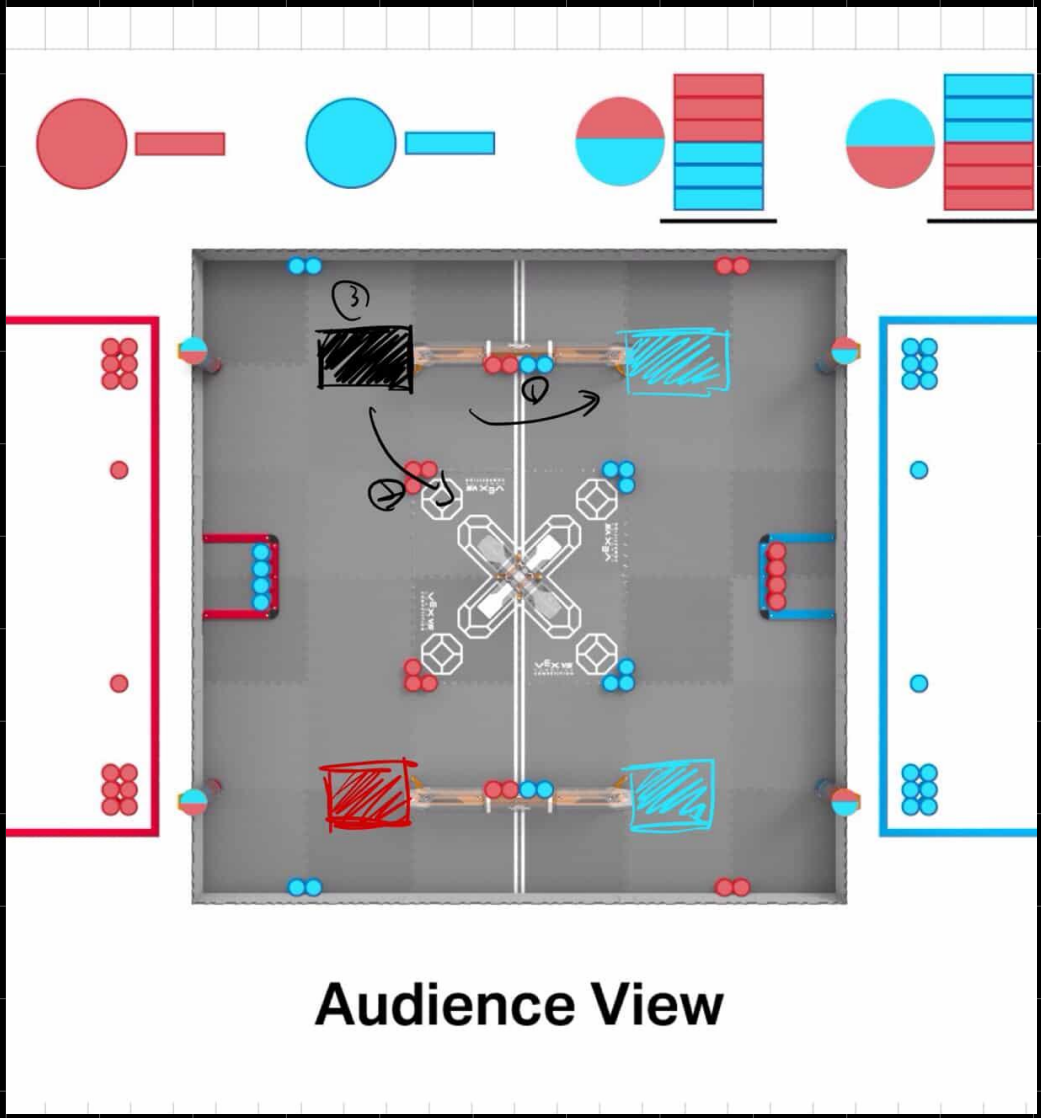
Driver

The driver's main goal is to control the field by pushing balls into the center goal and long goal while defending their scored balls.

Unlike Driver Skills, each scored ball would earn 3 points, controlling the long goal would earn 10 points, top center goal for 8 points, and bottom center goal for 6 points.



Game Strategies

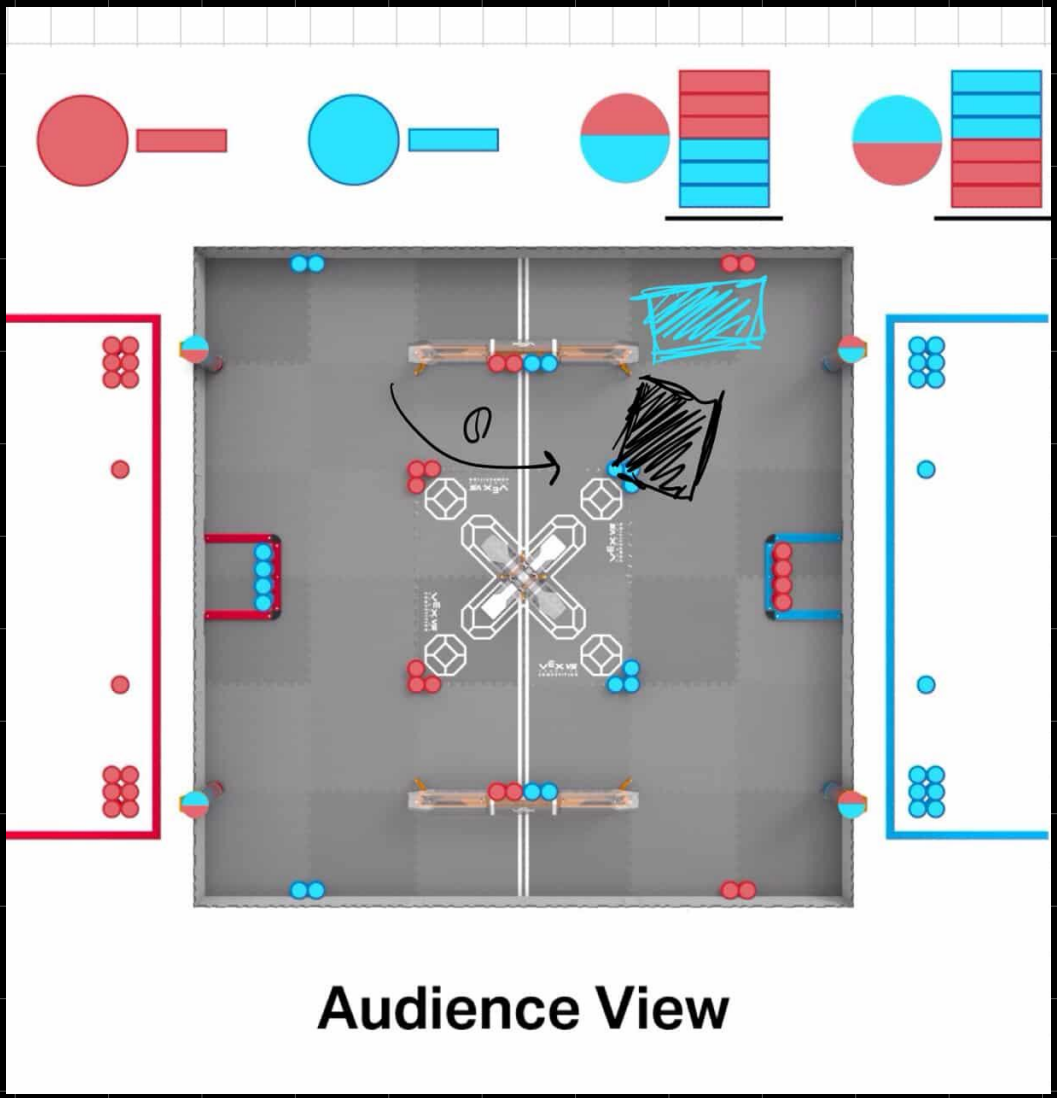


Audience View

We have 3 ways to approach the match.

1. We rush the mirror opposition as people tend to want to score bonus.
2. We play offense score-wise and try to score and defend the middle goals.
3. We play fully defensive and preserve the balls we scored in auto.

Game Strategies



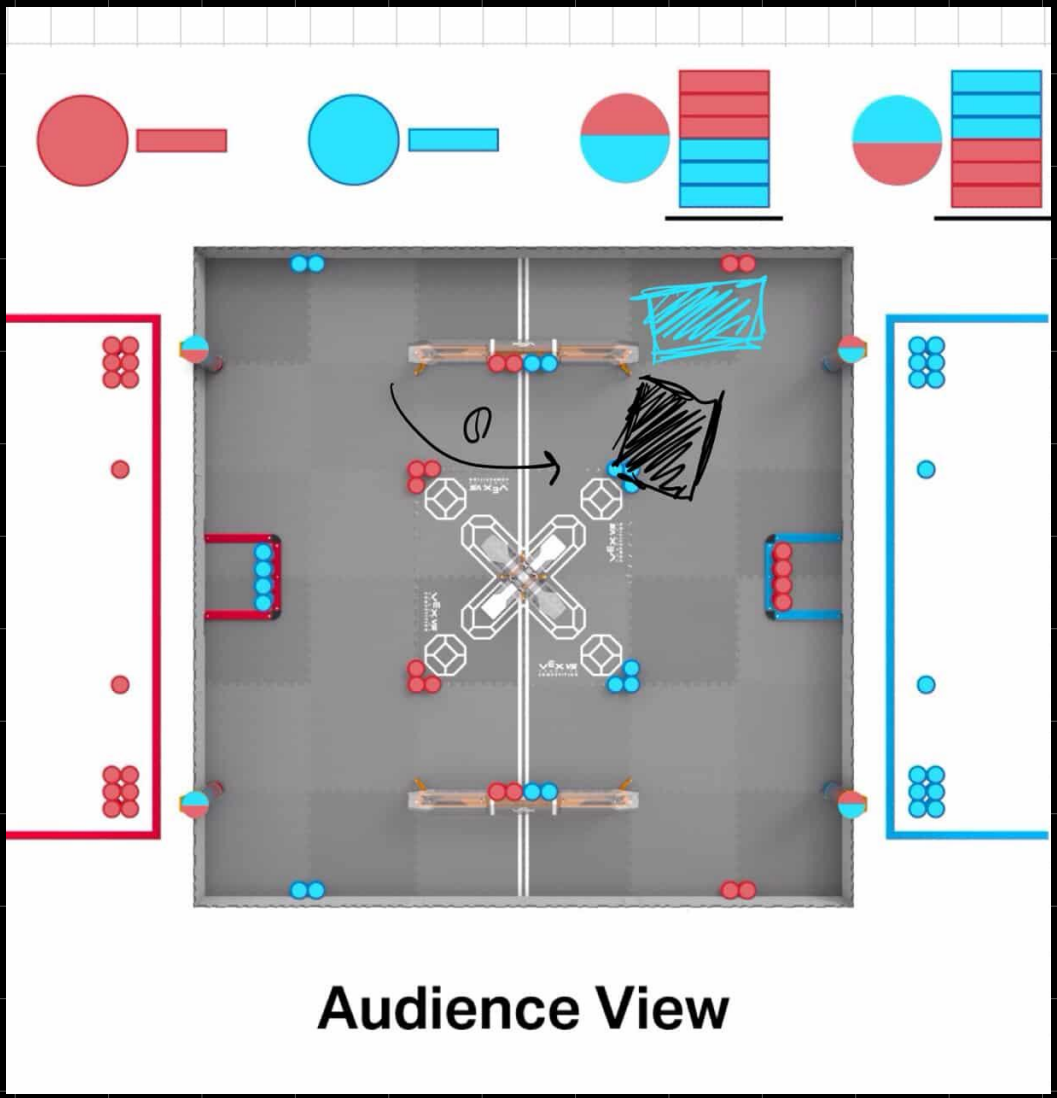
Audience View

Early Game Route 1:

1. This works best against lightweight bots or bots that have a slower scoring mechanic.

2. Utilizing good mobility for these attacks is often unexpected and invokes a direct response from the opposition, breaking protocol. Essentially a cc.

Game Strategies



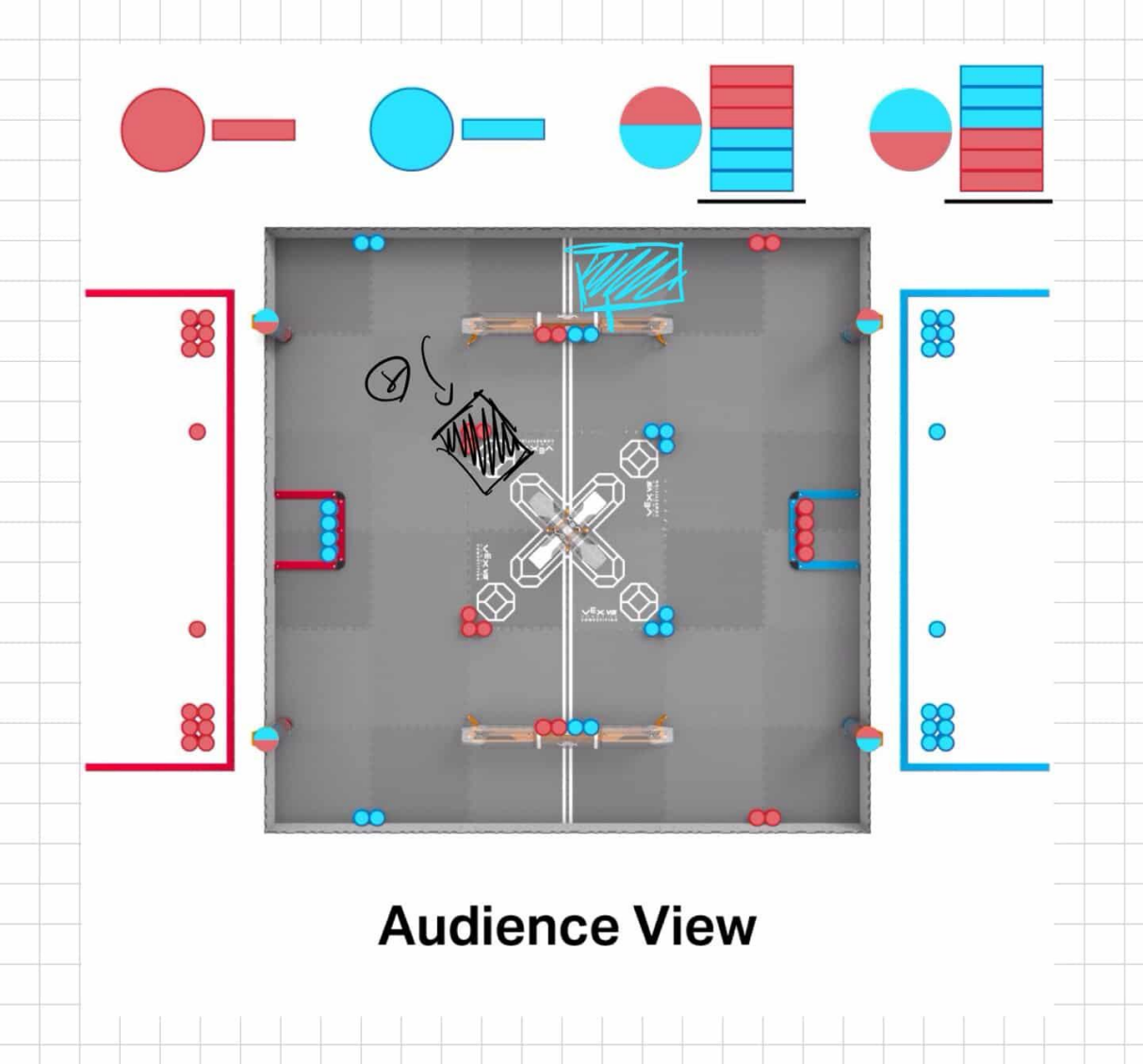
Audience View

Early Game Route 1:

3. Does not work well against heavy bots and fast scorers.

4. Leaving your post means the opposition can descore your blocks altogether.

Game Strategies



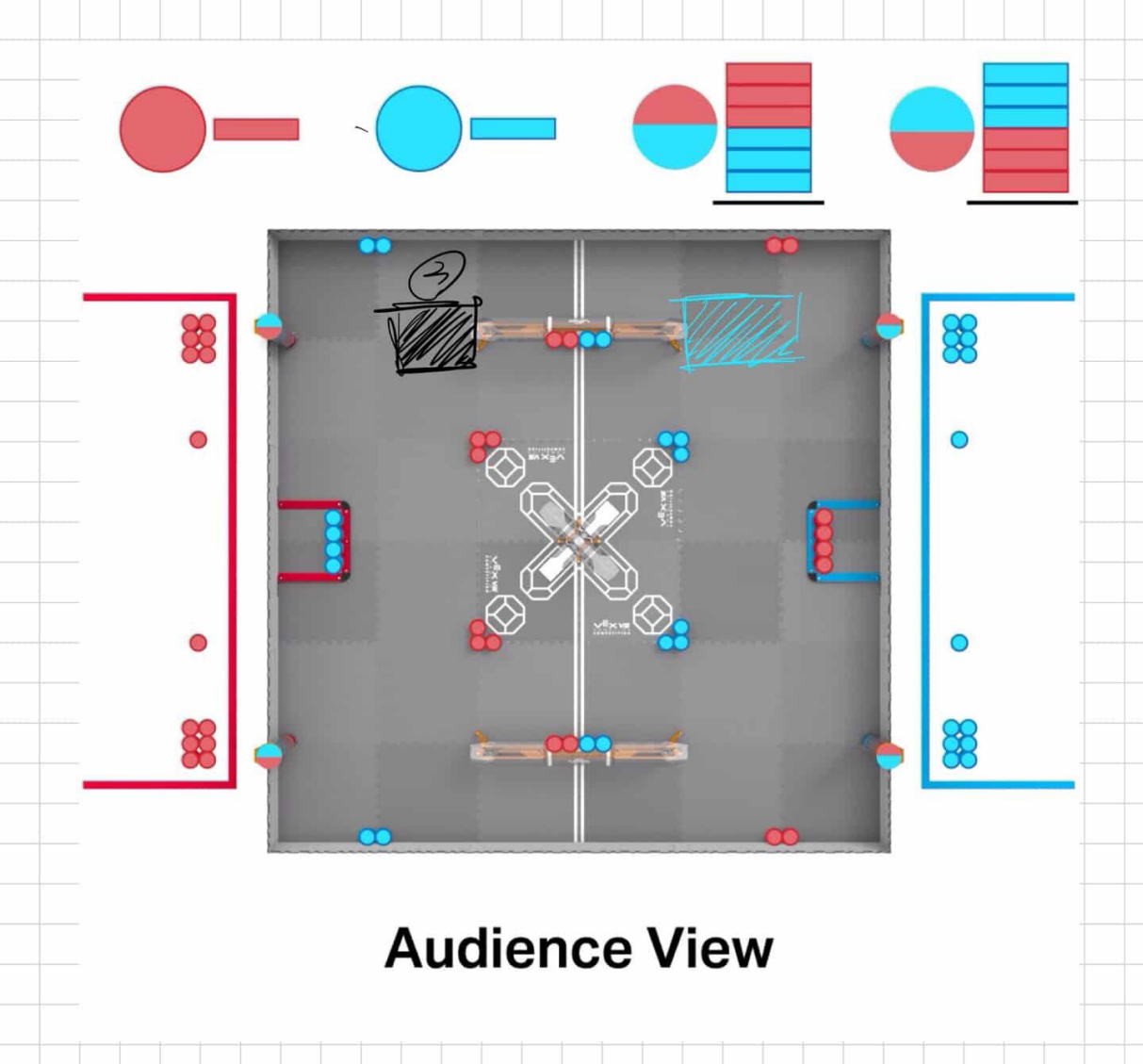
Audience View

Early Game Route 2:

1. Prioritizes the middle goals, which offer a combined potential of 35 points.

2. If opponents push balls toward your side during auto, we currently have zero counterplay options.

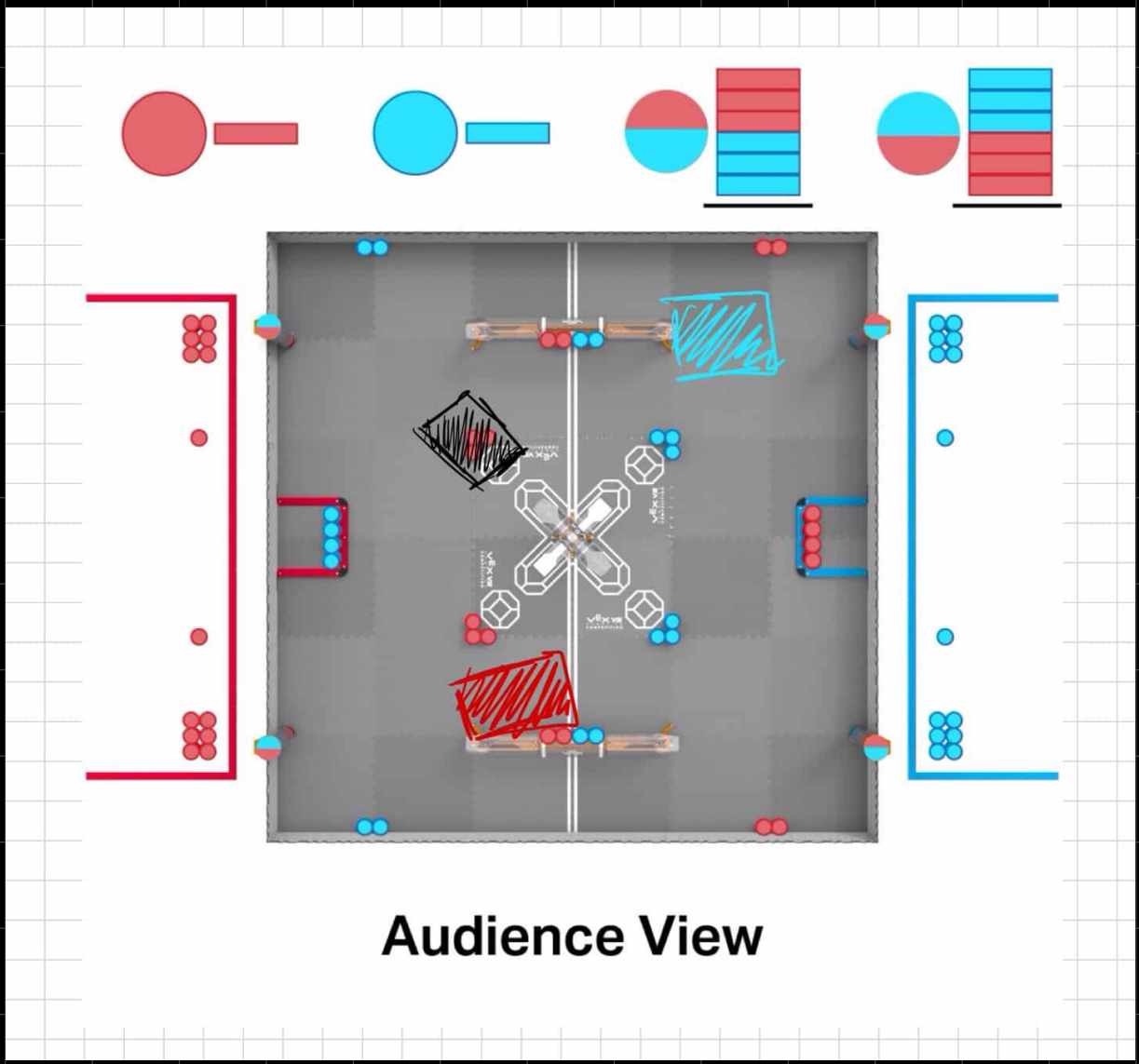
Game Strategies



Route 3:

1. To preserve the balls already scored during the autonomous period.

Game Strategies

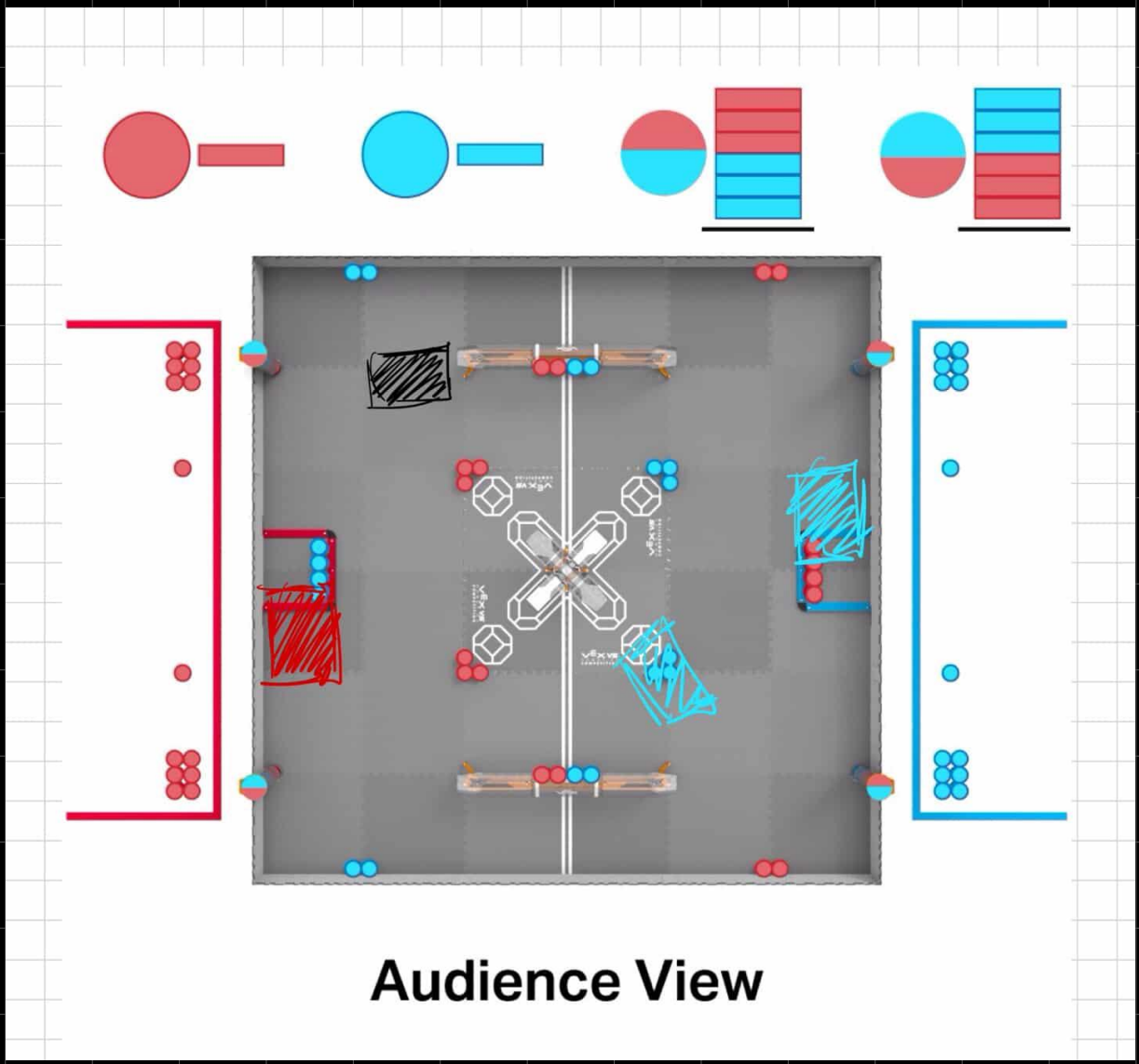


Audience View

Mid Game:

1. Generally opt for heavy middle defense
2. If an ally is collapsed on by two enemies, we can use our agility to help them or counter-attack by scoring on the opposite side of the map.

Game Strategies

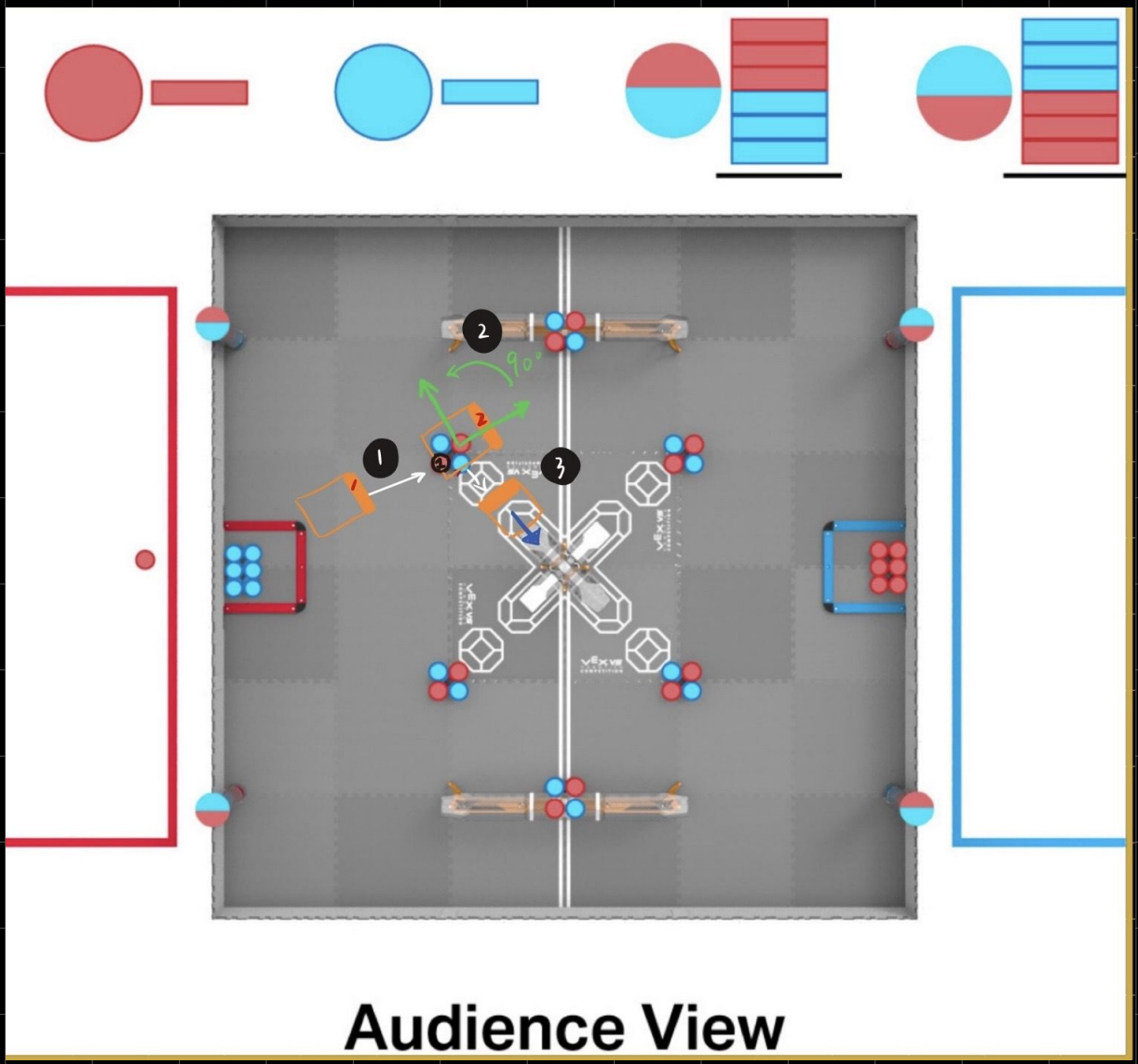


Late Game:

1. Maintaining a hold on the center of the map is important as teams would want to score 1 for bonus.

2. One park is worth less than three scored blocks, plus the fact that we can't double park, we don't park.

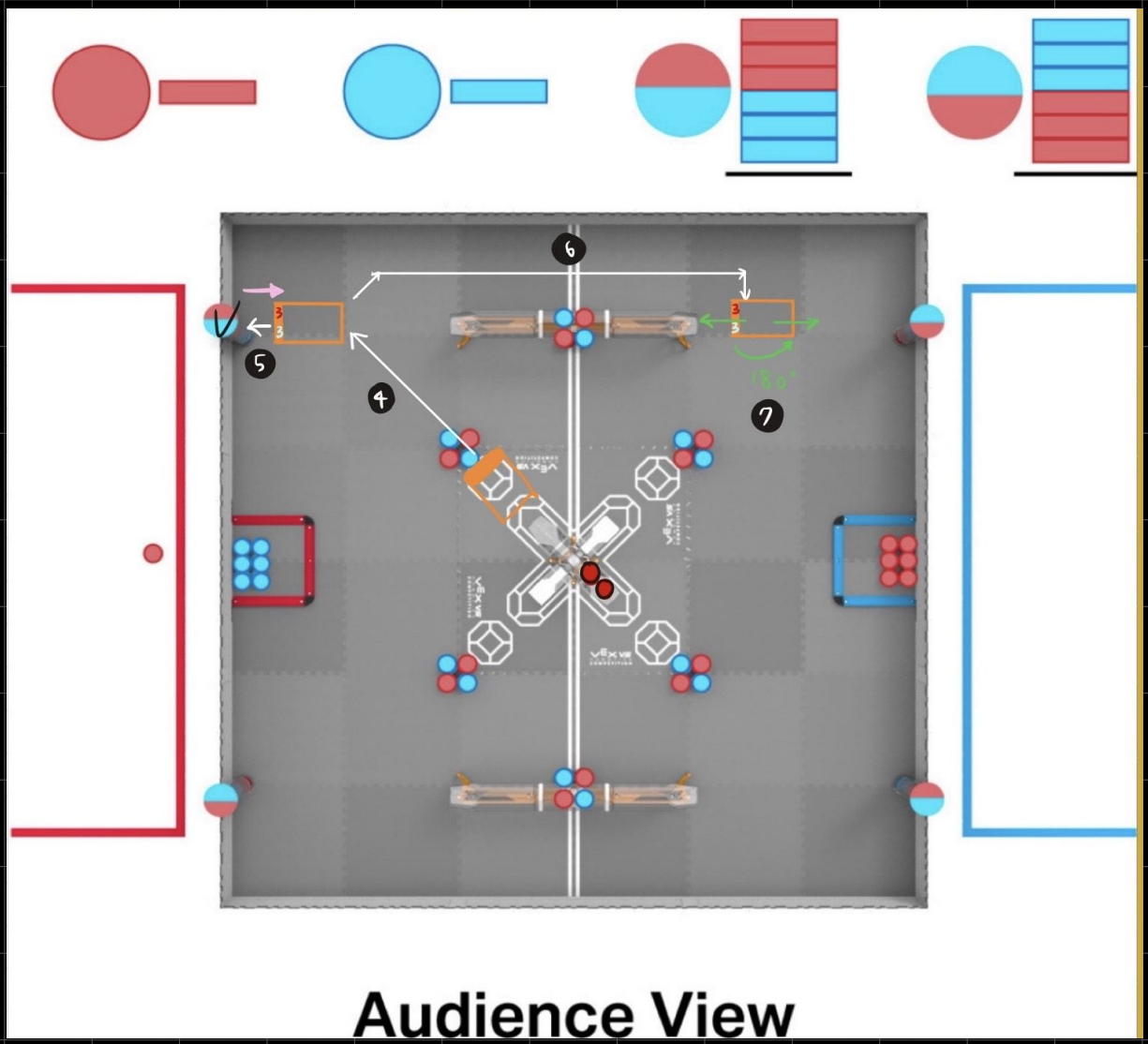
Skills Strategies Ver.3



1. Start from the parking zone. Drive straight and intake ① (the red block at the upper left corner)
2. Turn the car 90° counterclockwise
3. Drive to farther center goal, score 2 red blocks

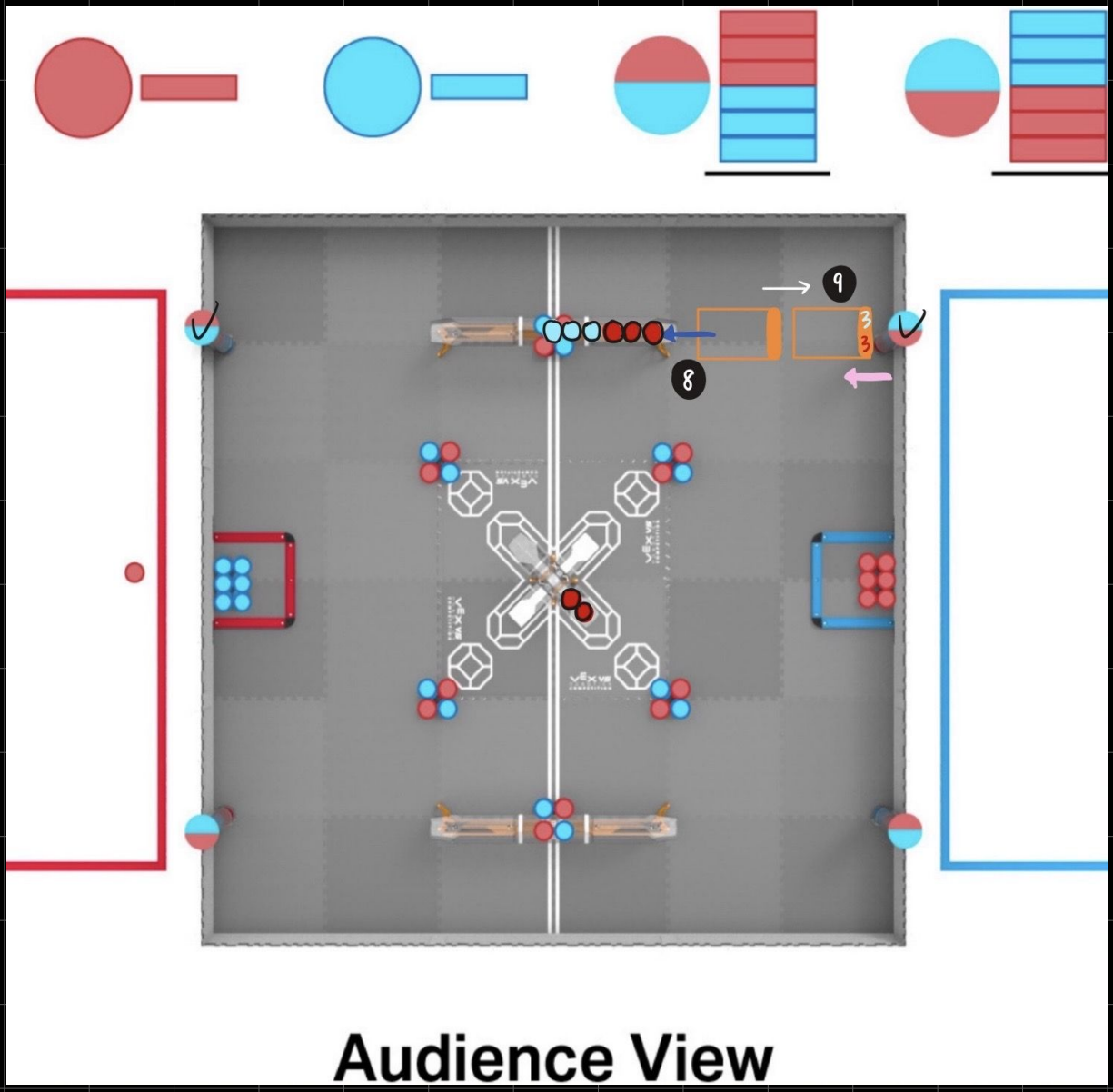
+2

Skills Strategies Ver.3



4. drive to the loader at the left upper corner, turn while moving
5. drive forward and intake 3 blue + 3 red blocks +5
6. drive around the farther long goal (don't change direction)
7. turn 180°

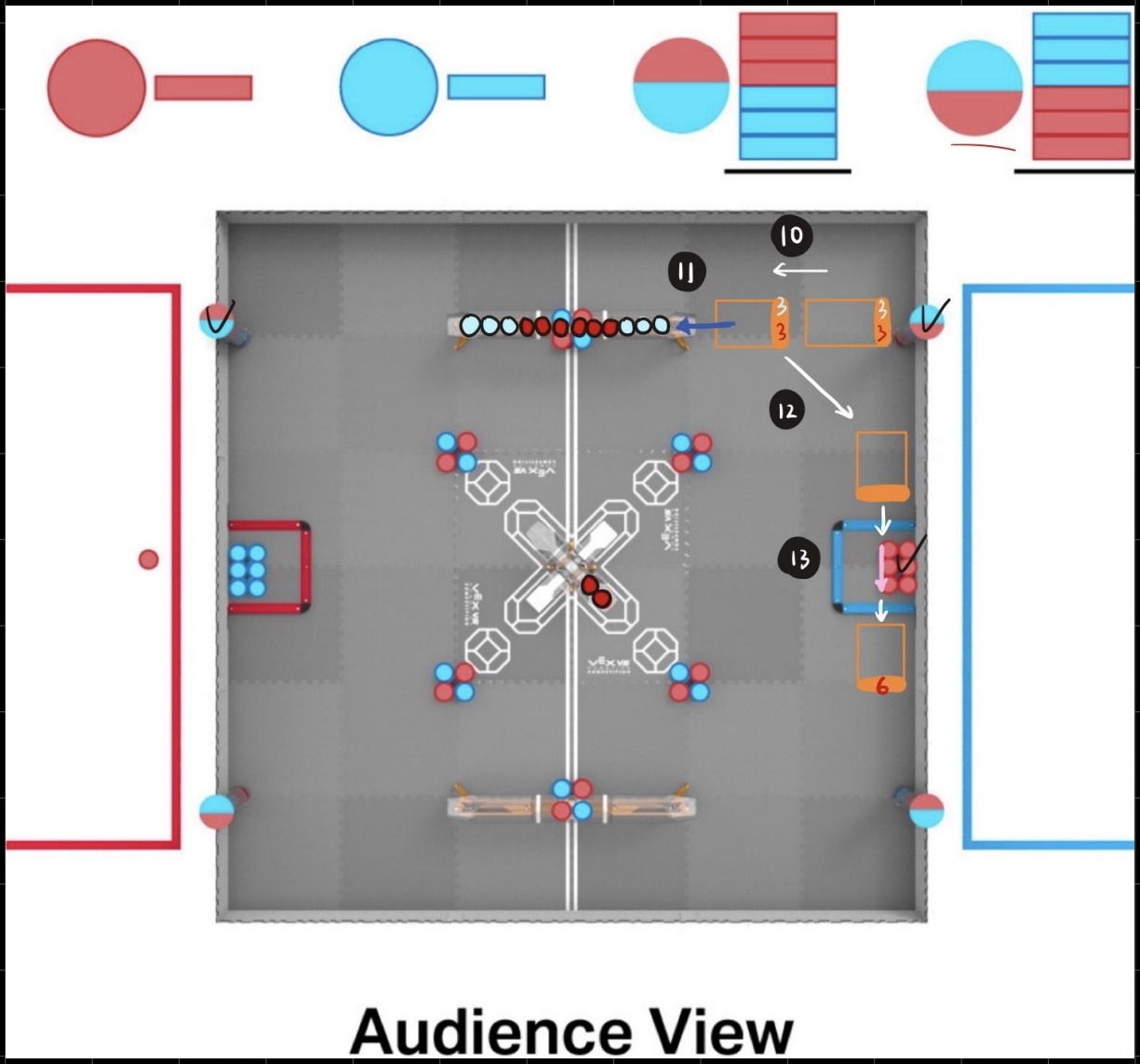
Skills Strategies Ver.3



8. score 3 blue + 3 red +2

9. drive forward to loader at the right farther corner and intake 3 red + 3 blue +5

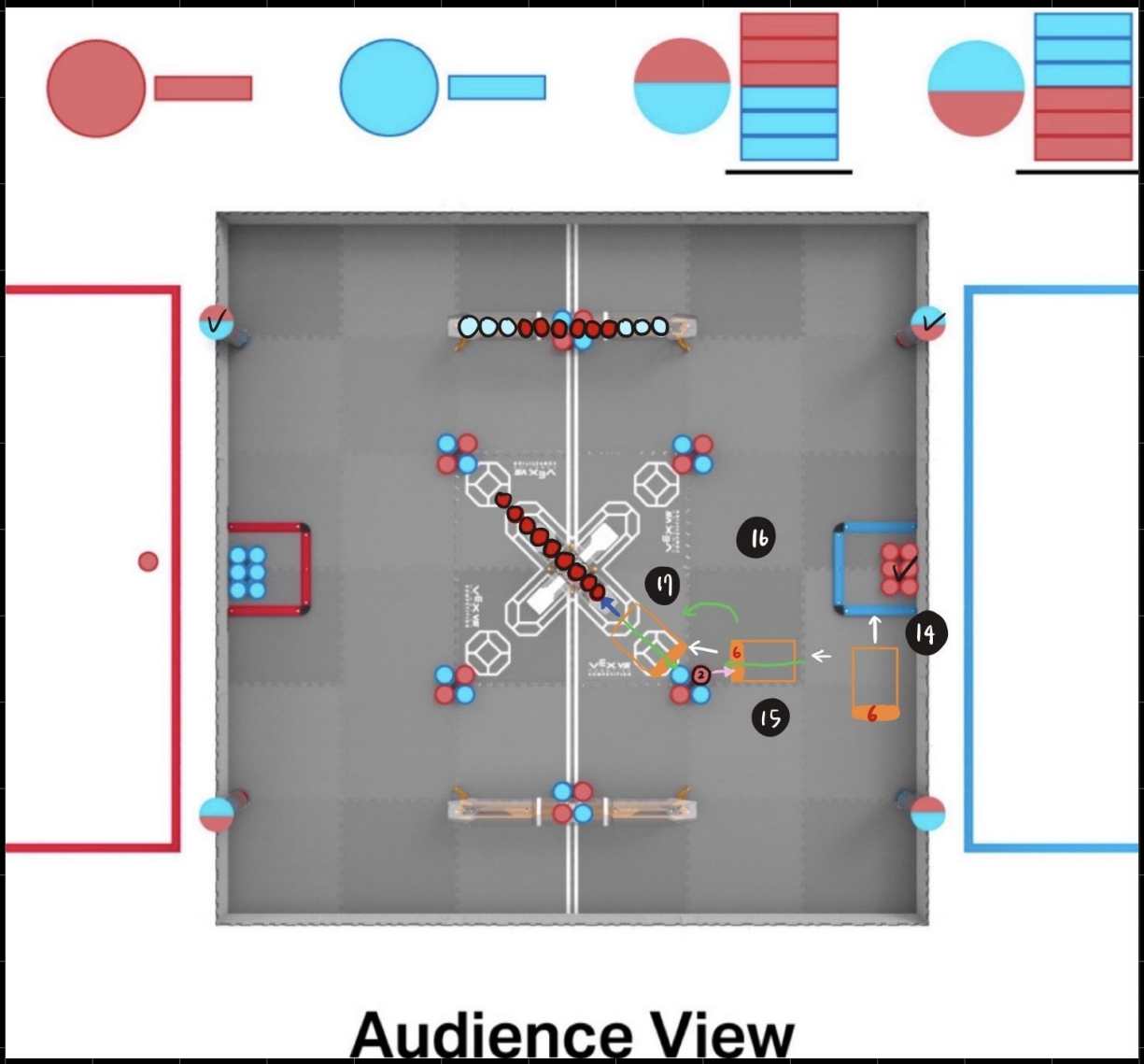
Skills Strategies Ver.3



Audience View

10. drive backward to farther long goal
11. score 3 red + 3 blue +6 +5
12. drive to the blue parking place
13. drive through while intaking 6 red (at least 4) +5

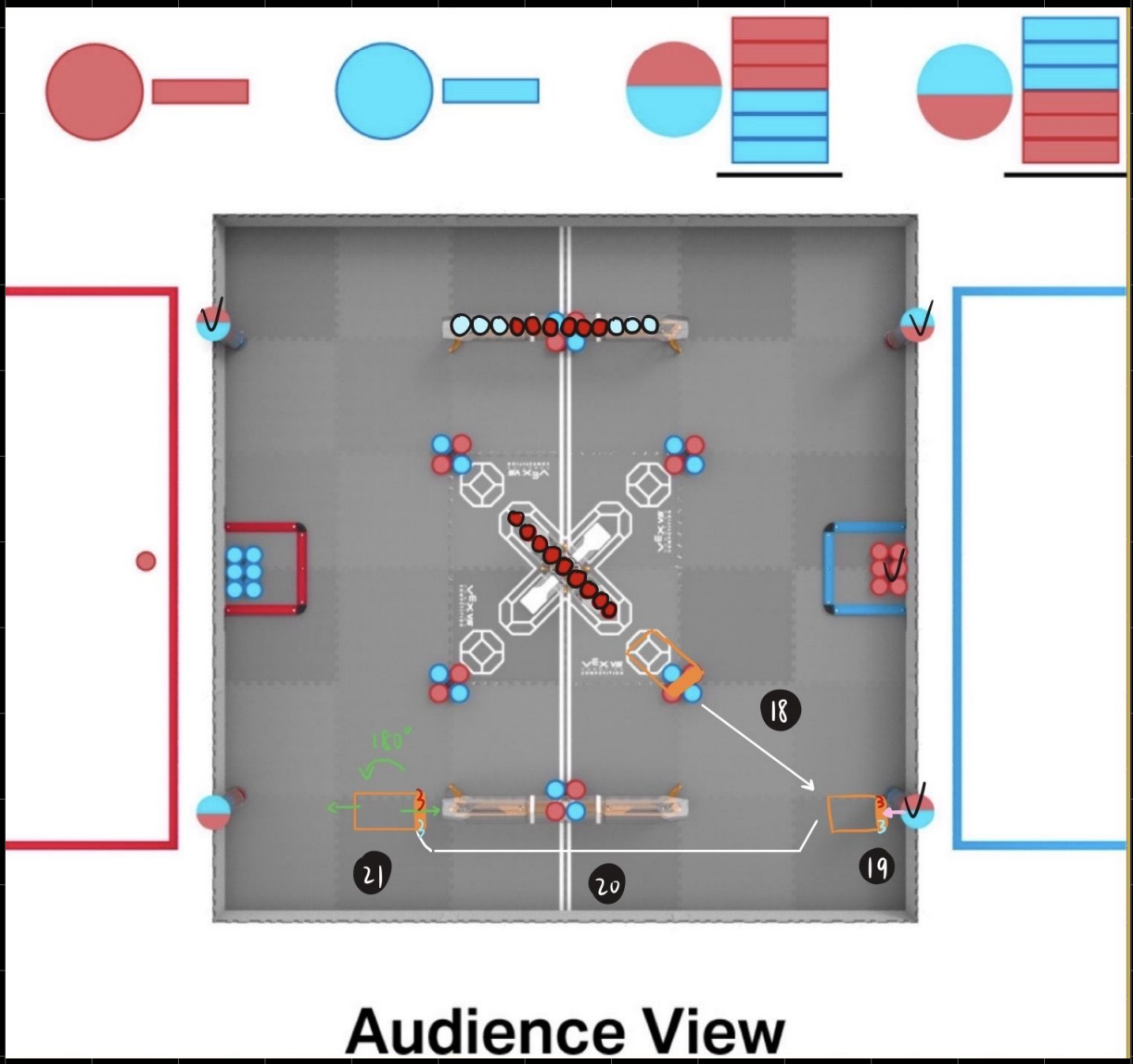
Skills Strategies Ver.3



14. drive backward until the car touches the parking zone in order to calibrate its location
15. Intake ② (red block)
16. Turn to the right direction
17. drive to the center upper goal. Score 7 red blocks

+5
+10

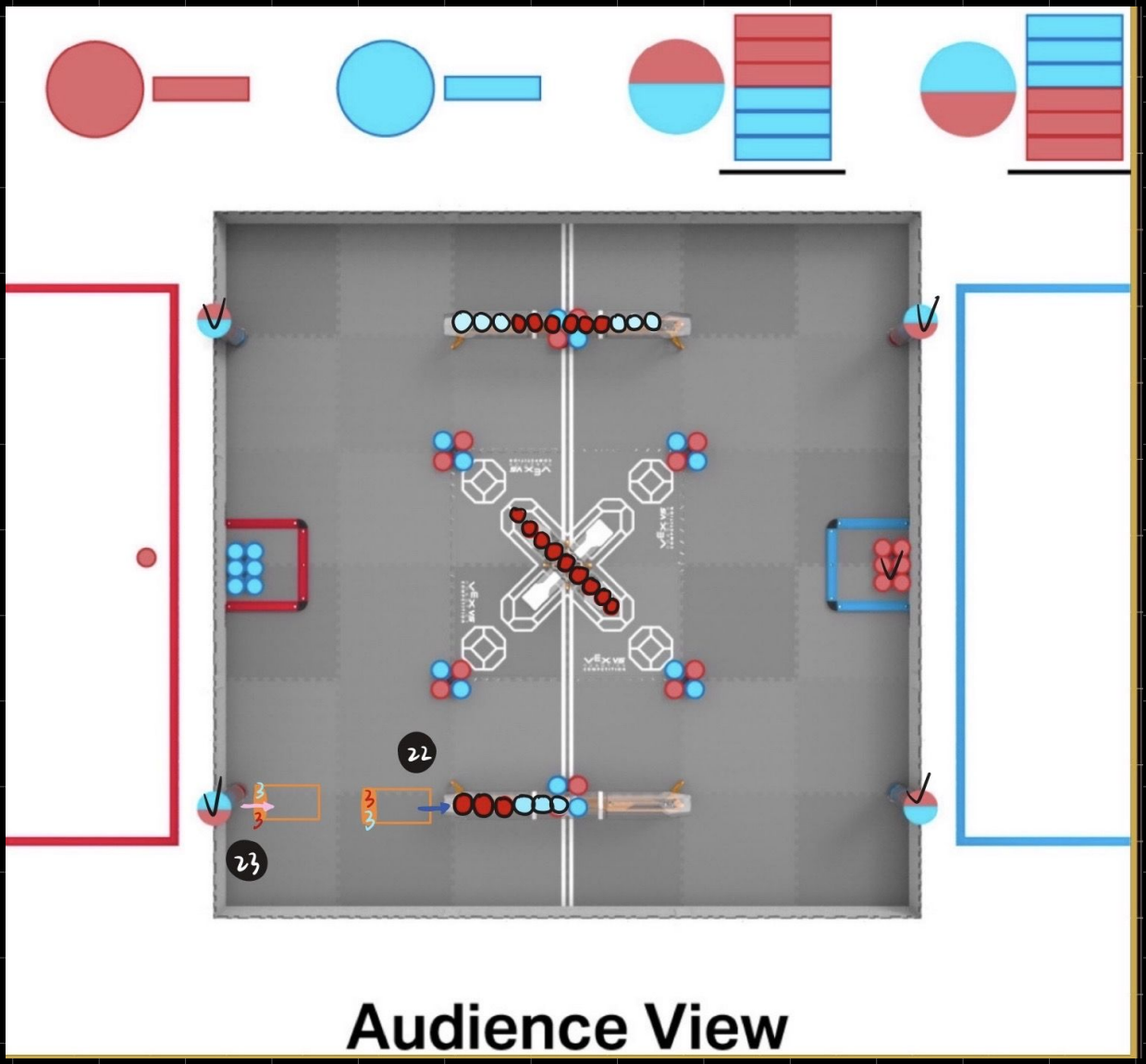
Skills Strategies Ver.3



Audience View

18. Drive to the loader at the closer right corner. Turn to the right direction while driving.
19. Intake 3 blue + 3 red +5
20. Drive around the closer long goal
21. turn 180° so the back faces the goal

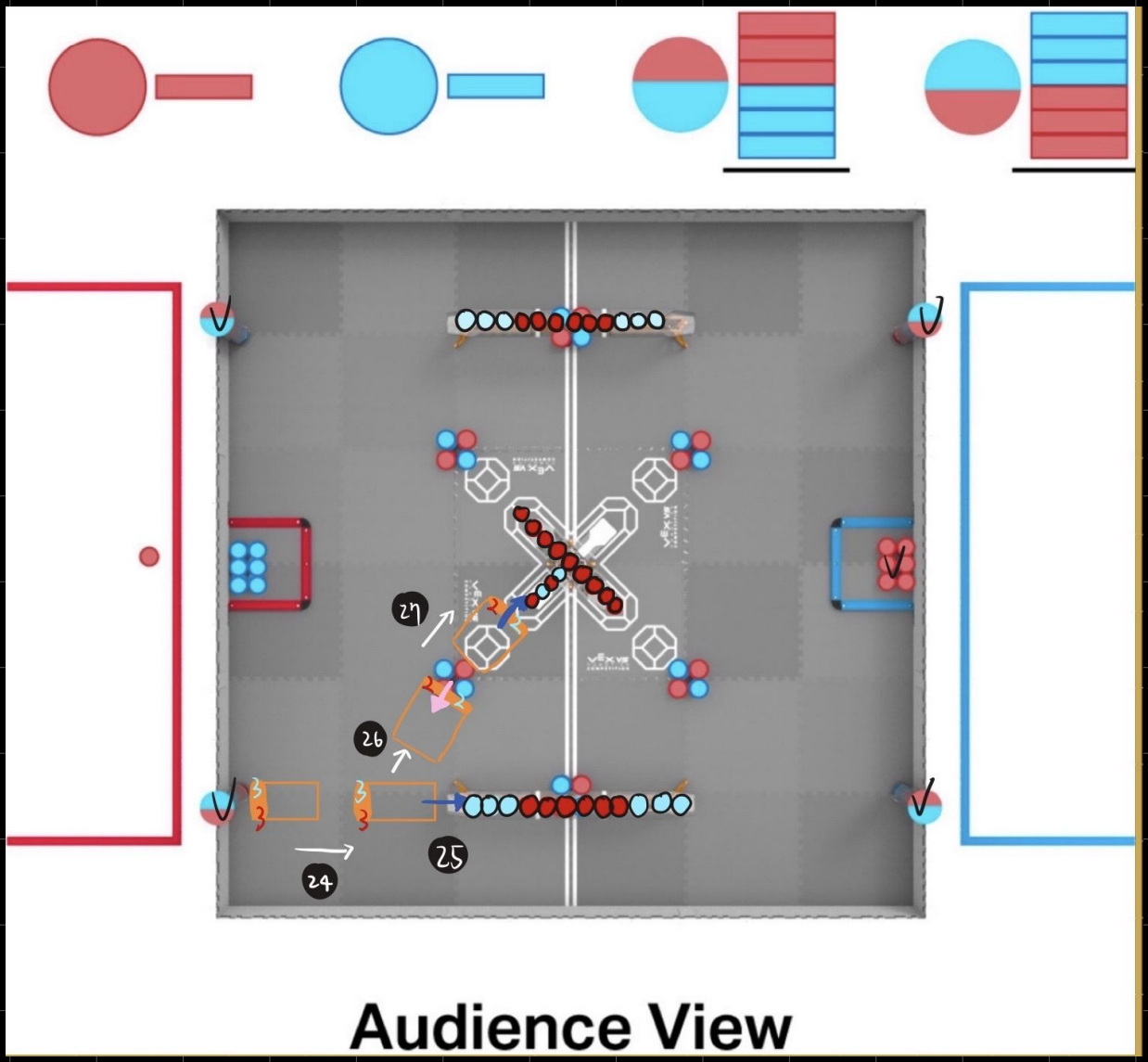
Skills Strategies Ver.3



22. score 3 blue + 3 red +6

23. drive backwards to loader at the closer left corner and intake 3 red + 3 blue +5

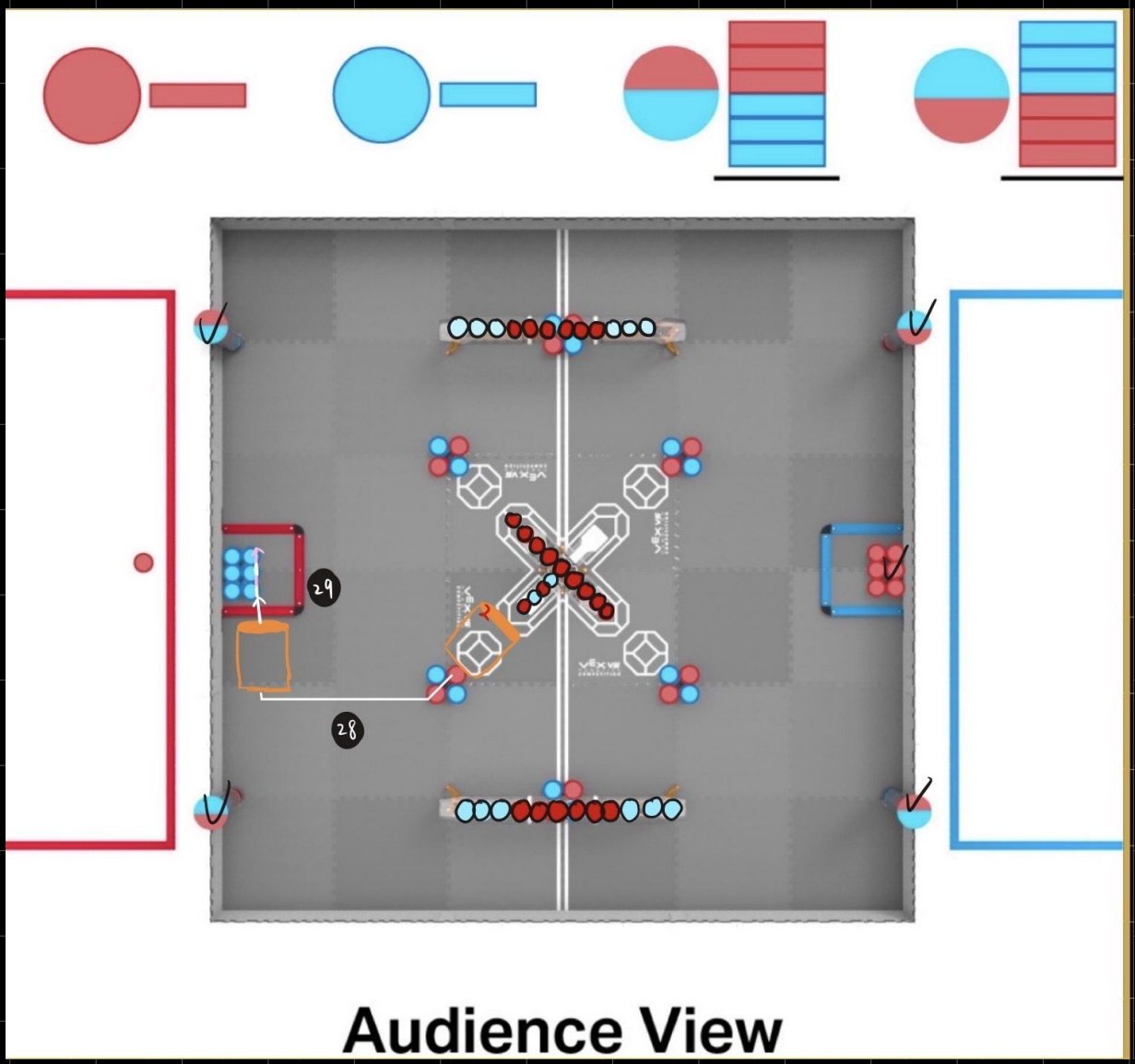
Skills Strategies Ver.3



Audience View

24. Drive backwards to return to the long goal
25. Score 3 red + 3 blue into the closer long goo +6 +5
26. Turn and drive to intake 2 red + 2 blue for the closer left
27. Score them into the lower center goal +4

Skills Strategies Ver.3



28. Drive toward the parking zone

29. Park. Clear the parking zone by intaking/pushing out blocks in the park zone +5 +15

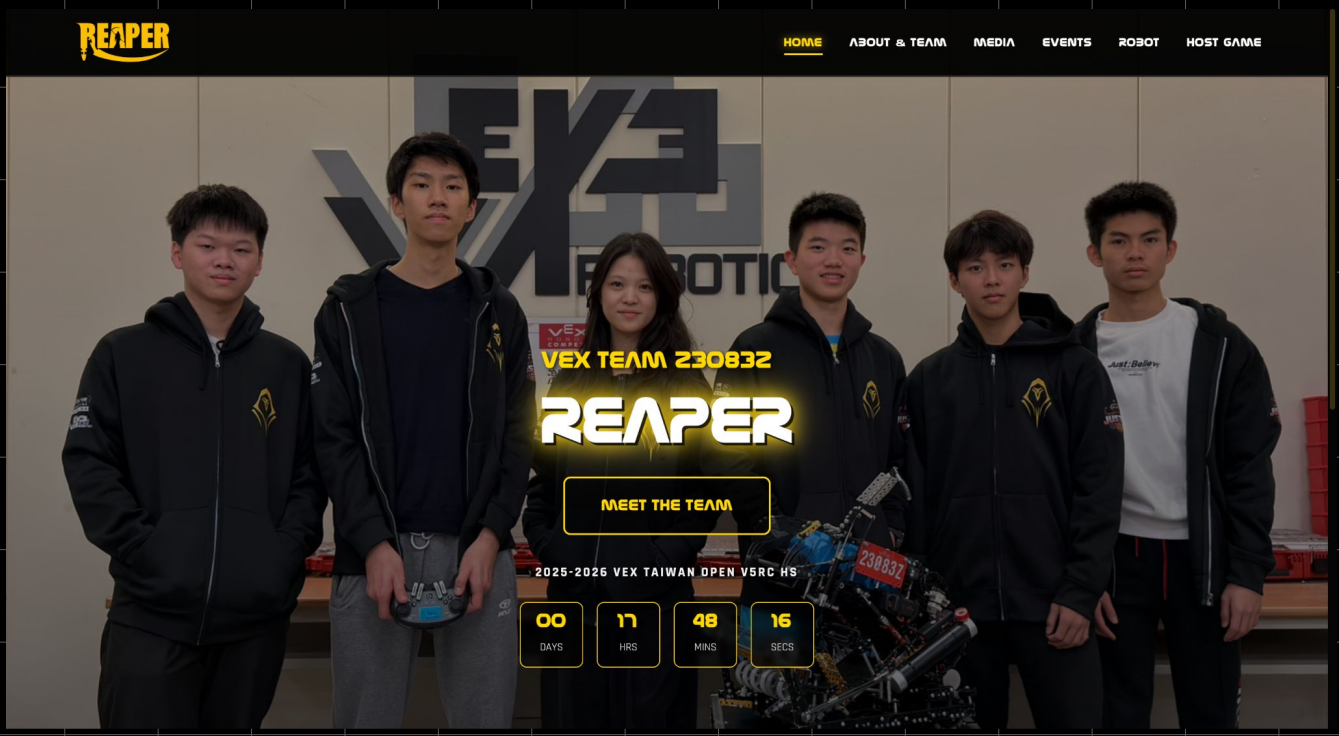
08
Team Mangement

Club Website

To make a record of our club and team on the open source web, we built our own website. It acts as a site to:

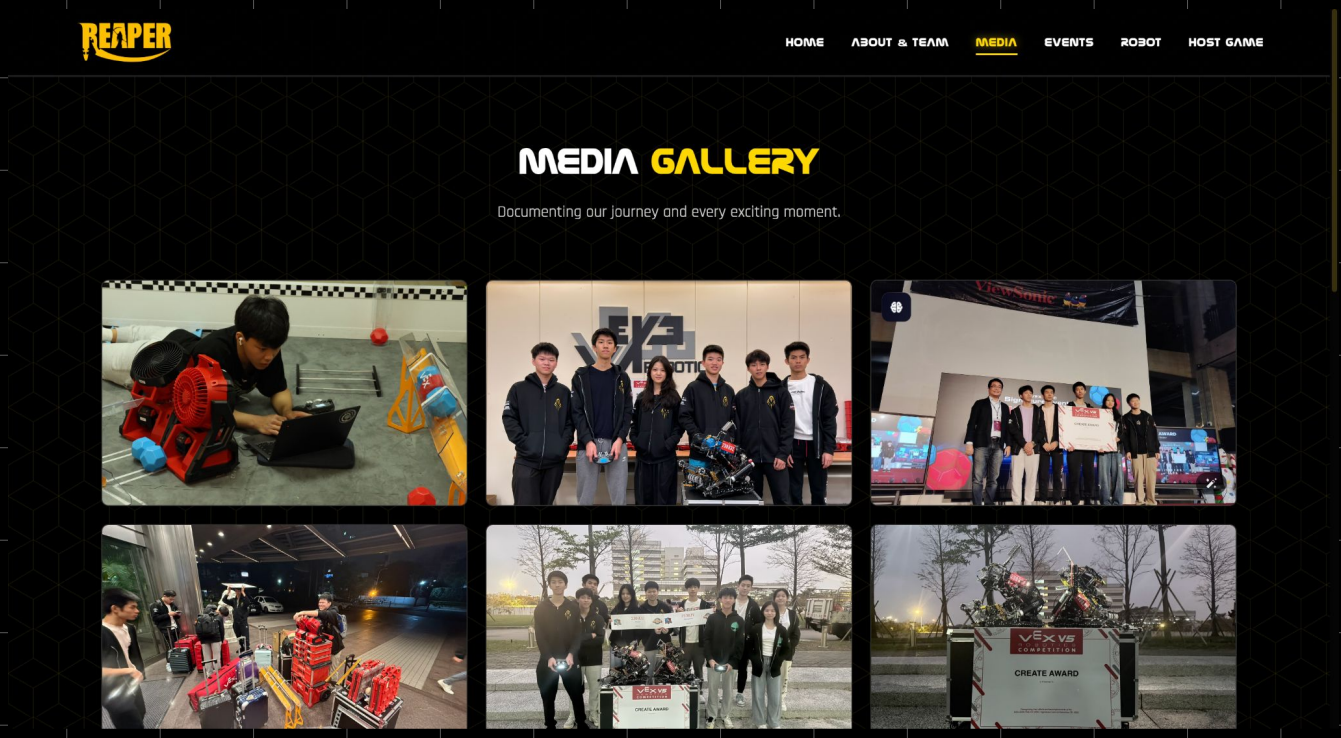
1. Record and demonstrate our team photos
2. Record and demonstrate our robots and projects
3. Help us host mock tournaments for our teams

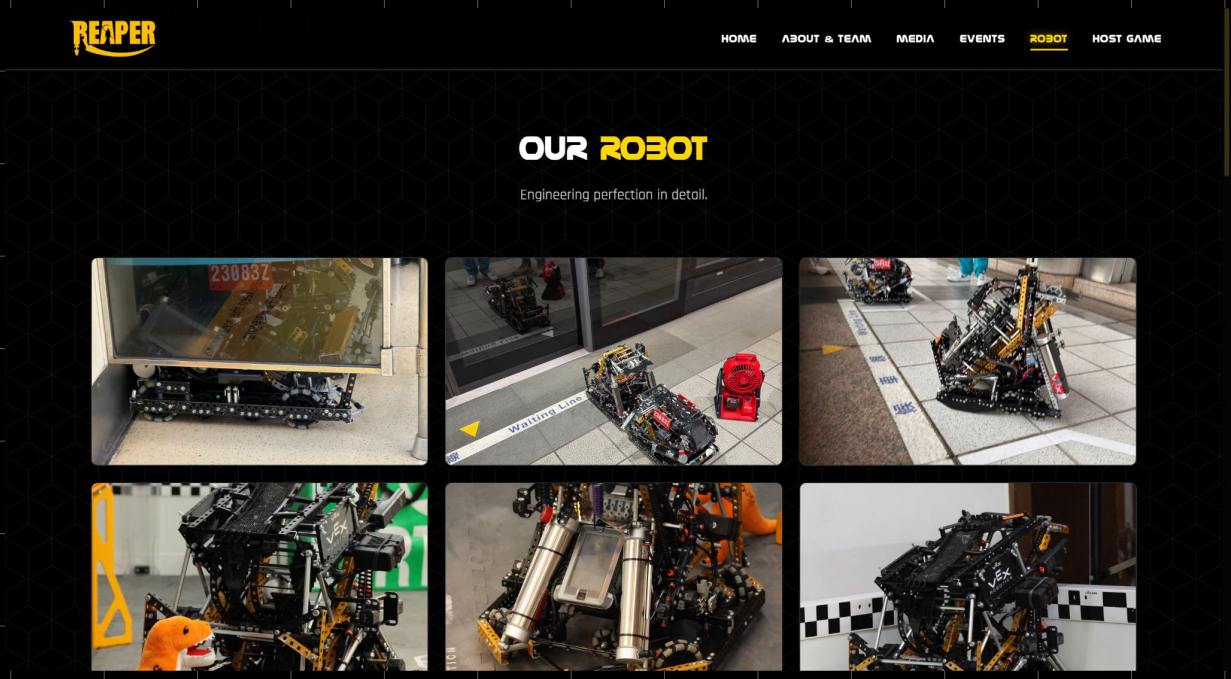
https://23083z.com/



Club Website

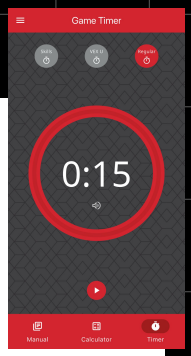
We put the photos we love the most and the ones that we hope to review in our later years on the website.





Club Website

We previously use VEX's official site for practice matches internally, but it was hard and inconvenient to use: https://education.vex.com/leaderboard/v5



VEX V5

01:45:00

Start Stop Reset

Timer Mode: Count Down

Count Down from (sec): 105

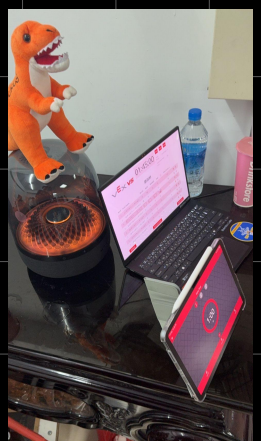
23083 Scrimmage

Rank	Team Name	BLUE1	RED2	WINNER	awp	Total Score
1	Q1 B1 A B2 G---------R1 Y R2 Z	37	82	2	0	121
2	Q3 B1 G B2 Y---------R1 A R2 Z	12	89	2	0	103
3	Q4 B1 A B2 Y---------R1 G R2 Z	16	73	2	0	91
4	Q8 B1 A B2 Z---------R1 G R2 Y	51	38	1	0	90
5	Q5 B1 G B2 Z---------R1 A R2 Y	34	50	2	0	86
6	Q2 B1 A B2 Y---------R1 G R2 Z	48	31	1	2	82
7	Q6 B1 A B2 G---------R1 Y R2 Z	41	33	1	1	76
8	Q7 B1 G B2 Y---------R1 A R2 Z	19	36	2	0	57

Add Team Print Leaderboard Restart Game

Problems of this site:

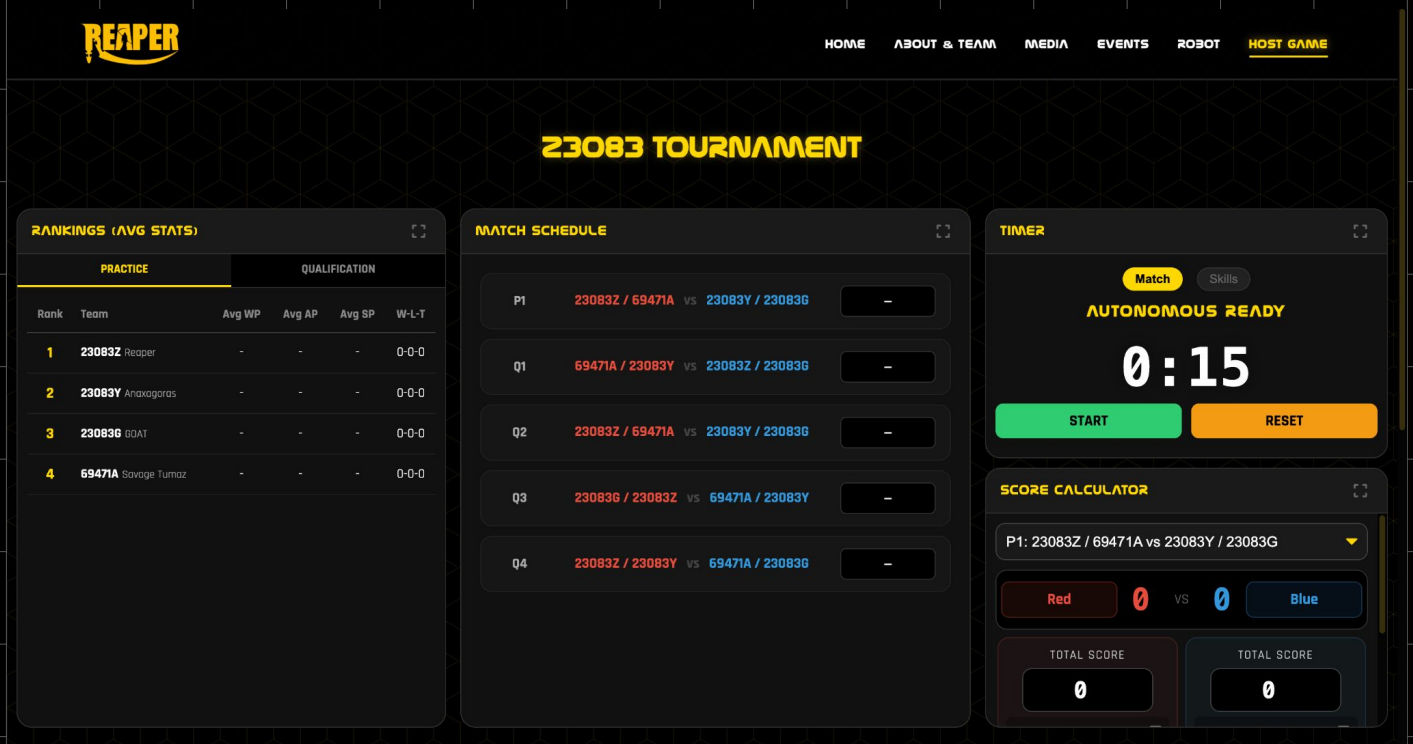
1. It can't record a series of matches, only individual ones
2. It doesn't have a count down timer for matches, only a stopwatch, so we have to corporate with the V5RC Hub app
3. It also doesn't have the alarms for matches



Club Website

Therefore, we developed our own API of hosting mock tournaments. The site features:

1. Computer calculated match schedules randomly assigning teams with alliances.
2. Countdown Timer for both Autonomous and Driver.
3. Team rankings simulating official calculation methods with WPs, SPs, and APs.



Club Budget

We have very much limit with our budget plan every year, since our school gives us minimal support:

1. We collected $640,000 NTD this season
2. The registration plan is about $50,000 NTD each team, which makes the registration fee 24% of our budget.
3. **Materials and Tools (Top Priority):** We already have most of the tools in our lab. We are planning for $300,000 NTD for abundant materials for all 3 teams over the season, making up 46% budget.
4. **Team Image and Promotion:** Uniforms, Pit decorations, and giveaways should be made with leisure time and extra money.
5. **Event-Related Fees:** Since not every participant would participate in every event, the fee might vary.

Practice

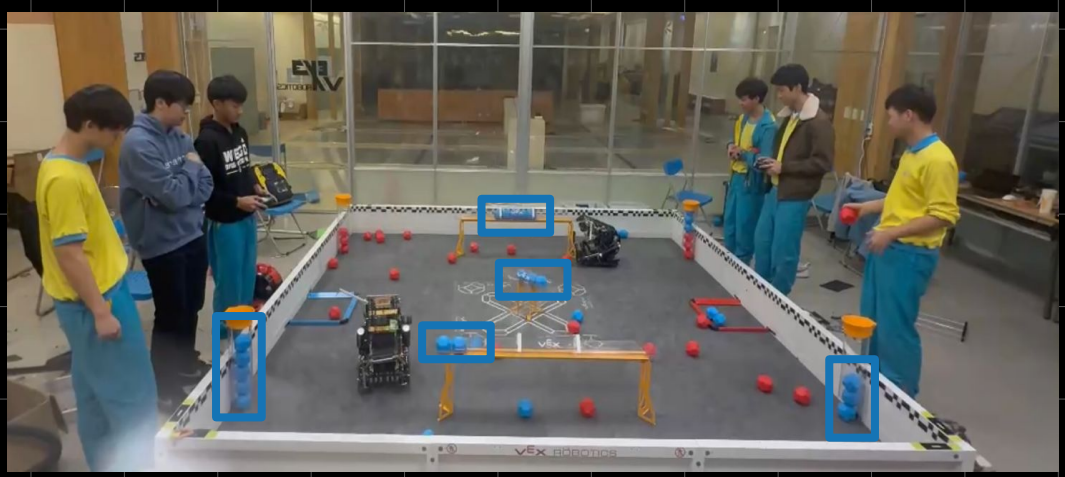
Wego Robotics Practice Sessions (Team 23083Z and Team 23083Y):

Before each tournament or competition, our team will participate in multiple practice sessions with the other Wego Robotics team (23083Y). During each practice, we will analyze our strategies, identify errors, and make adjustments to improve overall performance. These sessions allow us to evaluate both teams' strengths and weaknesses and share feedback to support one another's development. By practicing together, we aim to strengthen our driver skills, improve teamwork, and refine our competitive strategies.

Practice

Wego Robotics Practice Sessions (Team 23083Z and Team 23083Y):

In times when we only have another team that can practice with us, in order to mock real time matches, we use a special setup for practicing. Our problem is that with only two bots on the field, either both robots are focusing on scoring or both are defending each other. Therefore, we set up the field with what we call "a hellish beginning." For the driver who is mainly practicing now, we will setup the field as if the opposing alliance has a big win. For example, this pictures shows a setup for the red alliance to practice and try to still win, and the blue alliance driver should act as a defender who already has advantage.



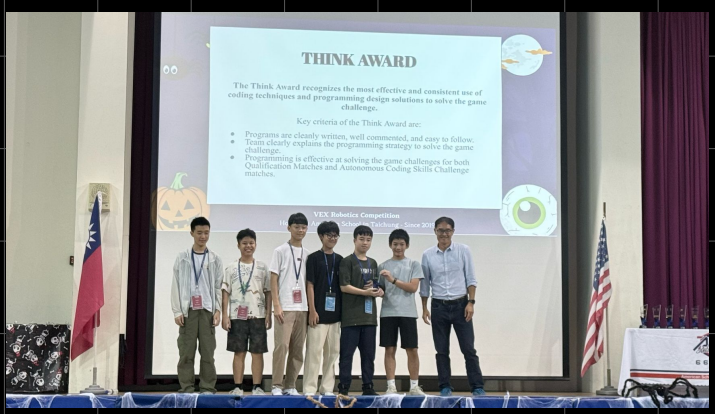
Team Impact

Our club technically does not have a coach, we self learn all by ourselves and receive knowledge from our seniors, our experiences, and researching online.

At the same time, since most of our clubmates are newbies to VEX, our team plays an important role (also spends a lot of time in) teaching our clubmates. We also try to make sure that the whole process is finished by students themselves, and we only help with **(1)** solving their questions, **(2)** providing them essential information, and **(3)** keep an eye on them to make sure their on task/on schedule.

We are also newbies in teaching students, and we also learn a lot while working and studying different robot structures. While having our own time pressure, we do our best to help our other teams whenever they need help, and we feel proud when they win awards.

23083G



23083Y

Team Impact

We also helped a newly formed rookie team 69471A which contains students from our friend FRC team we mentioned before. They started a middle school VEX team this year and we also helped them.

Club Organization

Our school is a highly academic school, which prioritizes classes and grades first. We only have 2 hours of club time a week. We have to make up practice time ourselves, while also keeping up our grades, or else we would be banned from practicing VEX.

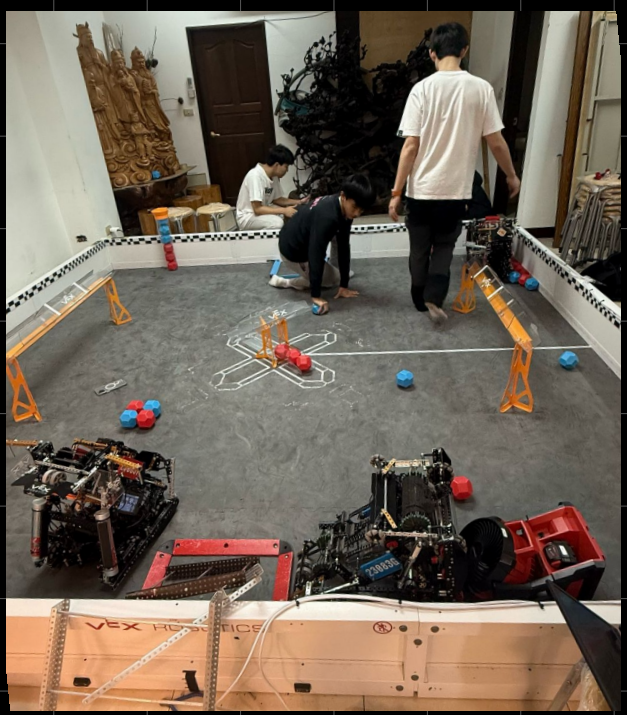
Club Organization

One of our biggest obstacle is the school. Although we have our own robotics lab, we have to apply at least the day before we are going to use it. And if we sometimes forget to apply, we can't use it the next day. Our supervisor keeps our keys, so anytime we need to enter the lab she has to permit. We aren't allowed to go to the lab during class times, short break times, or when we did not apply. Also, the school opens at 8:00 A.M. to 4:00 P.M. on the weekends. All these factors cause a lot of inconvenience, it compresses our practicing time. so time management becomes essential.

We even built the lab with little of the school's help!

Club Organization

Because of this, we often move our field around, trying to find a shelter to practice:

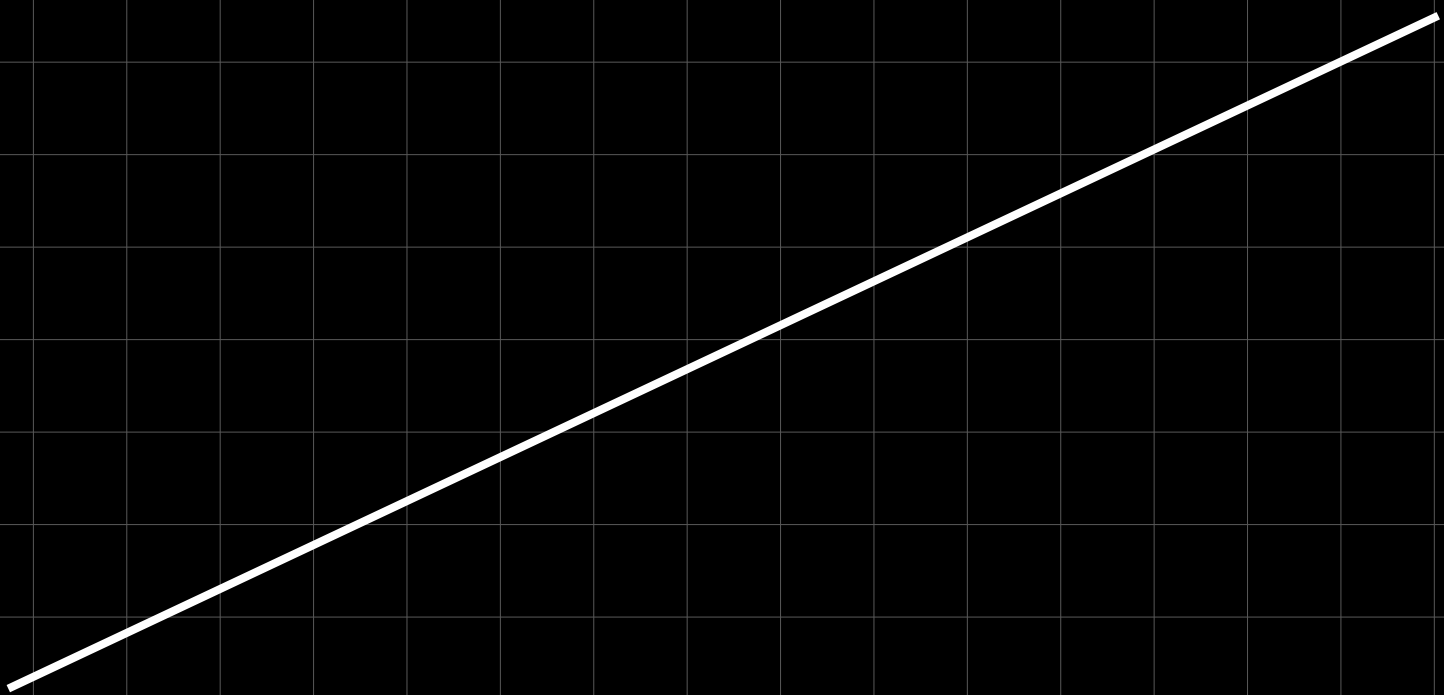


Club Organization

We have a very limited budget. Our money comes purely from the students. The VEX participant fee is 40,000NTD each season, which start in May each year.

Our main cost comes from:

- VEX materials
- Tools and instruments
- Team and Events registration fee
- Transportation, Accommodation, Food Fee during events
- Team uniforms
- Pitlane Decorations
- Team giveaways



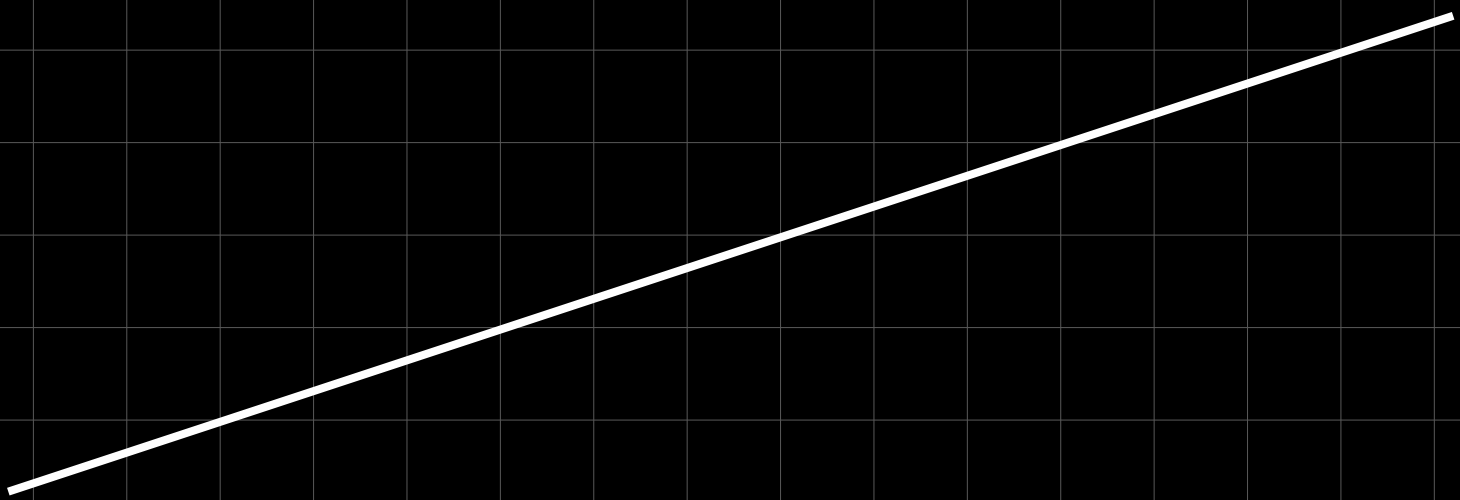
Club Budget

We have very much limit with our budget plan every year, since our school gives us minimal support:

1. We collected $640,000 NTD this season
2. The registration plan is about $50,000 NTD each team, which makes the registration fee 24% of our budget.
3. **Materials and Tools (Top Priority):** We already have most of the tools in our lab. We are planning for $300,000 NTD for abundant materials for all 3 teams over the season, making up 46% budget.
4. **Team Image and Promotion:** Uniforms, Pit decorations, and giveaways should be made with leisure time and extra money.
5. **Event-Related Fees:** Since not every participant would participate in every event, the fee might vary.

Club Budget

Category	%	Amount (NTD)	Expalnation
VEX Materials	50%	320,000	Priority
Tools & Instruments	20%	128,000	Only essentials
Travel / Food / Hotel	— —	— —	Paid before each event
Uniforms	5%	32,000	Bargain!
Pit Decorations	1.5%	9,600	Bargain!
Giveaways	0.5%	3,200	Stickers/Souveniers
Emergency Fund	23%	147,200	Bigger safety net



VEX Materials

We actually have a diversed source of VEX official materials:

1. We have a robotics teacher who has teached us EV3 for years, therefore he initially helps us with buying VEX materials from Taiwan's importer, which he has known for years.
   a. We list out what we need in a table and he helps us order with the importer
   b. We can't really know the price until the moment we pay for it, after we get the products
   c. We normally just multiply the price from the official site by 140% to get an idea of the total

Product	Code(sku)	Amount
Inertial Sensor	276-4855	1
3.25" (260mm Travel) Omni-Directional Anti-Static Wheel (2-Pack)	276-8026	2
Rotation Sensor	276-6050	3
45 Degree Gusset Pack	276-7759	8
1x2x1x35 Aluminum C-Channel (6-pack)	276-2289	4
1x1x35 Aluminum Angle (6-pack)	276-6484	2
0.375" OD Nylon Spacer Variety Pack	276-6340	2

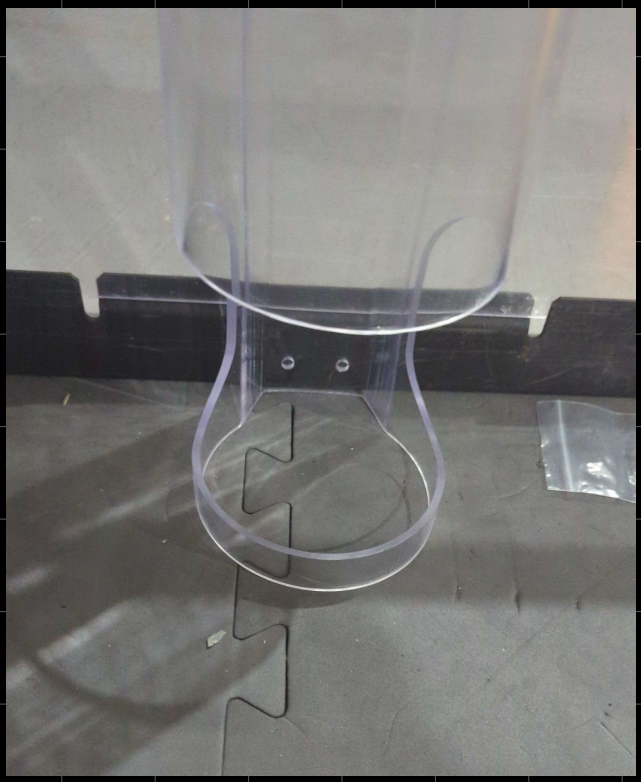
VEX Materials

However, we noticed that this method has several problems:

1. It's expensive and we don't know the price!
2. The shipping time is unpredictable

We learned from team 78181A Genesis that they directly buy from VEX's official site, which is surprisingly not taking a lot of time.

The first thing we bought was this year's field mats, since the old mats' dimensions were slightly smaller, making the field not accurate!



VEX Materials

2. Therefore, we also started to order parts from the VEX official website, and it only takes about 1 week shipping time! We renewed our field with the accurate mats., finally.

Order # 7105408 COMPLETE

December 21, 2025

Reorder Print Order

Order Information | Invoices | Order Shipments

Product Name	SKU	Price	Qty	Subtotal
Inertial Sensor	276-4855	Excl. Tax: HK$449.00	Ordered: 1 Shipped: 1	Excl. Tax: HK$449.00
VEX V5 Competition Anti-Static Full Field Tile Kit (36-pack)	276-7175	Excl. Tax: HK$2,209.00	Ordered: 1 Shipped: 1	Excl. Tax: HK$2,209.00
			Subtotal	HK$2,658.00
			Shipping & Handling	HK$2,249.24
			Grand Total	**HK$4,907.24**

Assembling our new field and new mats!

VEX Materials

3. We also bought parts from RoboSource, since we like the anodized standoffs, and lightweight plastic screws and nuts the site provides.

ROBOSOURCE® .NET
Robotics Parts, Tools & More at Low Prices

INVOICE
12/23/2025
#IN025978

Tax Detail	Tax Rate	Base price	Total Tax
No taxes			

Total Products	$306.22
Shipping Cost	$64.95
Total (Tax excl.)	**$371.17**
Total	**$371.17**

Payment Method	Credit Card (Authorize.net)	$371.17
Carrier	International Express/Air Mail	

If paying by check, make check payable and remit to:
Robosource, LLC
PO Box 2370
Worcester, MA, 01613

3D Printing

Bambu Lab P1S

Introduction of Bambu Lab P1S

- The Bambu Lab P1S is a CoreXY high-speed 3D printer designed for speed, reliability, and multi-material printing.
- It has an enclosed chamber for better temperature control and print quality for high-heat materials.

Usage

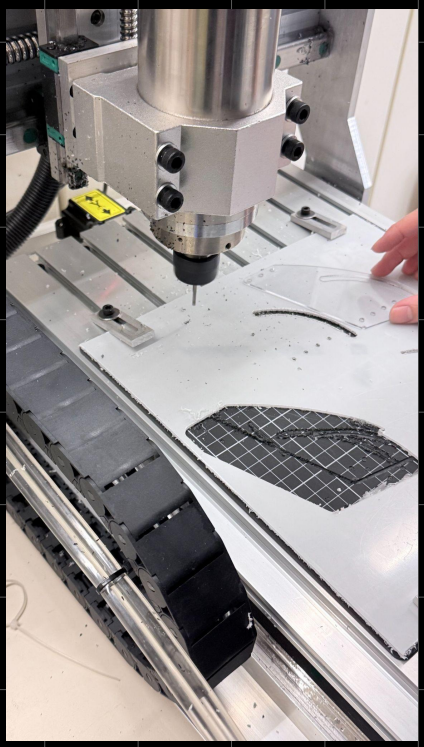
- Although 3dp materials can't be used on the robot for any purpose, we use it for controller extension parts.
- We don't have CNC through most of the season, so we use 3dp for plastic boards testing, then ask one of our FRC friend team to laser cut it for us.

https://makerworld.com/en/models/1616053-vex-v5-scuff-tntn#profileId-1705341

Computer Numerical Control (CNC)

3 Axis CNC 3040T Router Engraver

With limited budget, we asked our school if they would like to sponsor us for a CNC. After long discussion and bargaining, our school agreed to install the **3 Axis CNC 3040T Router Engraver** for us with a price of $120,000 NTD.



3D Printing

Bambu Lab P1S

We decided to use 3D Printing for giveaways, because:

1. relatively cheap
2. high versatility and easy to change

We used Blender to create **giveaways** that are designed as license plate and push back blocks. We will make them into keychains.

Acrylic Bending Machine

Manual Acrylic Bending Machine

For customizing plastic boards, we bought a **Acrylic Bending Machine** so that we can bend, since a lot of teams use upward bent plastic boards as descorer mech.

Thank you for reading